

Python

K19G

2026-10-07

Table of contents

Python	34
Start here	34
Book Roadmap & Master Architecture	34
Architectural Conventions	36
Formats	37
 Python Foundations	 38
 Intro	 39
 What Is Python	 40
 What Is Python	 41
Mental model	41
Minimal example	42
Worked examples	43
1. Peeking inside the CPython compiler with <code>dis</code>	43
2. Inspecting runtime execution frames	44
3. Checking Python 3.14 free-threading (GIL status)	45
Pitfalls	46
1. Thinking Python is “just interpreted”	46
2. Modifying objects during iteration	46
3. Mixing tabs and spaces	46
Exercises	47
Further reading	47
 Modern Toolchain: uv	 48
 The Modern Toolchain with uv	 49
Mental model	49
Minimal example	50
Worked examples	50
1. Single-file scripts with inline dependencies (PEP 723)	50
2. Managing Python runtimes with <code>uv python</code>	52
3. Creating and locking a modern project	52
Pitfalls	53
1. Manually activating <code>virtualenvs</code> in every terminal tab	53
2. Hand-editing <code>uv.lock</code>	54
3. Polluting the system Python with <code>sudo pip</code>	54

Exercises	54
Further reading	54
Code Hygiene: Ruff	55
Developer Environment & Code Hygiene with Ruff	56
Mental model	56
Minimal example	57
Worked examples	58
1. Configuring <code>ruff</code> in <code>pyproject.toml</code>	58
2. How linters work: building an AST inspector	59
3. Automated Git pre-commit hooks	60
Pitfalls	61
1. Running multiple competing formatters	61
2. Blindly running <code>ruff check --fix</code> without review	61
3. Committing code without running formatting in CI	61
Exercises	62
Further reading	62
Editor Setup: VS Code	63
Editor Setup: VS Code with Modern Tooling	64
Mental model	64
Minimal example	65
Worked examples	66
1. Workspace extension recommendations (<code>.vscode/extensions.json</code>)	66
2. Interactive debugging with <code>.vscode/launch.json</code>	66
3. Testing format-on-save and auto-import sorting	67
Pitfalls	68
1. Conflicting legacy extensions	68
2. VS Code selecting the wrong Python interpreter	68
3. Expecting editor tools to run in external shells	68
Exercises	69
Further reading	69
Variables	70
Names, Binding, and Object References	71
Names, Binding, and Object References	72
Mental model	72
Minimal example	72
Worked examples	73
Case 1: Inspecting the namespace dictionary	73
Case 2: Binding vs in-place mutation	74
Pitfalls	75
Pitfall 1: Expecting assignment to copy compound objects	75
Pitfall 2: Attempting to modify immutable objects	75

Exercises	76
Further reading	76
Identity, Equality, and Mutability	77
Identity, Equality, and Mutability	78
Mental model	78
Minimal example	78
Worked examples	79
Case 1: The small integer caching mechanism	79
Case 2: Shallow copy vs Deep copy	80
Pitfalls	81
Pitfall 1: Comparing values using <code>is</code>	81
Pitfall 2: Mutating objects inside tuples	81
Exercises	81
Further reading	81
Naming Conventions, Constants, and Unpacking	82
Naming Conventions, Constants, and Unpacking	83
Mental model	83
Minimal example	83
Worked examples	84
Case 1: Star unpacking in loops and data processing	84
Case 2: Marking constant intent with <code>typing.Final</code>	85
Pitfalls	85
Pitfall 1: Unbalanced unpacking assignments	85
Pitfall 2: Overriding built-in functions with variable names	85
Exercises	86
Further reading	86
Variable Scopes, LEGB, and Object Lifetimes	87
Variable Scopes, LEGB, and Object Lifetimes	88
Mental model	88
Minimal example	88
Worked examples	89
Case 1: Mutating enclosing state with <code>nonlocal</code>	89
Case 2: Object lifecycle and reference counting	90
Pitfalls	91
Pitfall 1: <code>UnboundLocalError</code> caused by shadowing assignment	91
Exercises	92
Further reading	92
Data Types	93
Integers and Arbitrary Precision	94

Integers and Arbitrary Precision	95
Mental model	95
Minimal example	95
Worked examples	96
Case 1: Bit manipulation and permission masks	96
Case 2: Integer division vs float division	97
Pitfalls	98
Pitfall 1: Converting colossal integers to strings	98
Pitfall 2: Negative modulo differences from C	98
Exercises	98
Further reading	98
Floating-Point Numbers and Decimal Precision	99
Floating-Point Numbers and Decimal Precision	100
Mental model	100
Minimal example	100
Worked examples	101
Case 1: Financial calculations with <code>decimal.Decimal</code>	101
Case 2: Exact rational arithmetic with <code>fractions.Fraction</code>	102
Pitfalls	103
Pitfall 1: Initializing <code>Decimal</code> from a <code>float</code> literal	103
Pitfall 2: Comparing floats with <code>==</code>	103
Exercises	103
Further reading	103
Booleans, Truthiness, and Short-Circuit Logic	104
Booleans, Truthiness, and Short-Circuit Logic	105
Mental model	105
Minimal example	105
Worked examples	106
Case 1: Short-circuit evaluation as a safety guard	106
Case 2: The conditional ternary expression	107
Pitfalls	108
Pitfall 1: Fallback idiom swallowing valid falsy values	108
Pitfall 2: Explicit comparison with <code>True</code> or <code>False</code>	108
Exercises	108
Further reading	108
Strings, UTF-8 Encoding, and Slicing	109
Strings, UTF-8 Encoding, and Slicing	110
Mental model	110
Minimal example	110
Worked examples	111
Case 1: String building performance (<code>join</code> vs <code>+=</code>)	111
Case 2: Encoding strings to bytes and decoding back	112

Pitfalls	112
Pitfall 1: Confusing <code>bytes</code> and <code>str</code>	112
Pitfall 2: Slice out-of-range behavior	112
Exercises	113
Further reading	113
Type Casting, Type Inspection, and Annotations	114
Type Casting, Type Inspection, and Annotations	115
Mental model	115
Minimal example	115
Worked examples	116
Case 1: Checking types: <code>isinstance</code> vs <code>type()</code>	116
Case 2: Modern Python 3.14 type annotations	117
Pitfalls	117
Pitfall 1: Using <code>type(x) == T</code> instead of <code>isinstance()</code>	117
Pitfall 2: Assuming type annotations enforce runtime checks	118
Exercises	118
Further reading	118
Template Strings (t-strings)	119
Template Strings (t-strings)	120
Mental model	120
Worked examples	120
Case 1: Same braces, different type	120
Case 2: Walk the parts at the desk	121
Case 3: Render like an f-string (on purpose)	122
Case 4: Escape guest text for HTML	123
Case 5: SQL placeholders, not string glue	124
The trap	125
The boring rule	126
Try this	126
Functions	127
Defining Functions, Signatures, and Return Values	128
Defining Functions, Signatures, and Return Values	129
Mental model	129
Minimal example	129
Worked examples	130
Case 1: Early returns (The “Bouncer Pattern”)	130
Case 2: Inspecting function introspection attributes	131
Pitfalls	131
Pitfall 1: Forgetting explicit returns in branches	131
Pitfall 2: Mutating input arguments unexpectedly	132
Exercises	132

Further reading	132
Parameters, Keyword Arguments, and Defaults	133
Parameters, Keyword Arguments, and Defaults	134
Mental model	134
Minimal example	134
Worked examples	135
Case 1: Preventing the “Boolean Parameter Trap”	135
Case 2: The Mutable Default Argument Trap and the Fix	136
Pitfalls	137
Pitfall 1: Evaluating dynamic defaults like <code>datetime.now()</code> in the signature	137
Exercises	137
Further reading	137
Variadic Arguments (*args and **kwargs)	138
Variadic Arguments (*args and **kwargs)	139
Mental model	139
Minimal example	139
Worked examples	140
Case 1: Argument forwarding in decorator wrappers	140
Case 2: Unpacking dictionaries and lists at call sites	141
Pitfalls	141
Pitfall 1: Signature ordering violations	141
Pitfall 2: Overusing <code>**kwargs</code> and destroying API readability	141
Exercises	142
Further reading	142
First-Class Functions, Callables, and Lambdas	143
First-Class Functions, Callables, and Lambdas	144
Mental model	144
Minimal example	144
Worked examples	145
Case 1: Command Dispatch Tables (Replacing <code>if/elif</code> ladders)	145
Case 2: Custom sorting keys with lambdas and operator	146
Pitfalls	147
Pitfall 1: Over-complicating lambdas	147
Pitfall 2: Assigning a lambda to a variable instead of <code>def</code>	147
Exercises	147
Further reading	148
Closures, Lexical Scopes, and Factory Functions	149
Closures, Lexical Scopes, and Factory Functions	150
Mental model	150
Minimal example	150

Worked examples	151
Case 1: Stateful API Rate Limiter using <code>nonlocal</code>	151
Case 2: The Notorious Late-Binding Loop Trap	152
Pitfalls	153
Pitfall 1: Retaining large memory references in closures	153
Exercises	153
Further reading	153
Generator Functions and Subgenerator Delegation	154
Generator Functions and Subgenerator Delegation	155
Mental model	155
The <code>yield from</code> Bi-Directional Pipe (PEP 380)	155
Minimal example	156
Worked examples	157
Case 1: Two-Way Coroutines for Rolling Averages with <code>.send()</code>	157
Case 2: Deep Tree Traversal and Recursive Flattening with <code>yield from</code>	158
Case 3: Capturing Return Values from Subgenerators via <code>yield from</code>	159
Pitfalls	160
Pitfall 1: Sending Data to an Unprimed Generator	160
Pitfall 2: Re-iterating an Exhausted Generator	161
Exercises	161
Further reading	162
Control Flow	163
The <code>if</code>, <code>elif</code>, and <code>else</code> Statements	164
The <code>if</code>, <code>elif</code>, and <code>else</code> Statements	165
Mental model	165
Short-Circuit Evaluation Guarding	165
Minimal example	166
Worked examples	167
Case 1: The Bouncer Pattern (Guard Clauses vs Arrow Anti-Pattern)	167
Case 2: Truthiness and The Numeric 0 Trap	168
Pitfalls	169
Pitfall 1: Comparing Explicitly to <code>True</code> or <code>False</code>	169
Pitfall 2: Combining <code>and</code> with <code>or</code> Without Parentheses	169
Exercises	170
Further reading	170
Conditional Expressions and the Walrus Operator	171
Conditional Expressions and the Walrus Operator	172
Mental model	172
The Walrus Operator Pipeline (<code>:=</code>)	172
Minimal example	173

Worked examples	173
Case 1: Streamlining Configuration Fallbacks	173
Case 2: Avoiding Duplicate Computations with the Walrus Operator	174
Pitfalls	175
Pitfall 1: The Nested Ternary Spaghetti Trap	175
Pitfall 2: Precedence Involving <code>or</code> and Ternaries	175
Exercises	175
Further reading	176
The while Loop and State-Driven Execution	177
The while Loop and State-Driven Execution	178
Mental model	178
Minimal example	178
Worked examples	179
Case 1: Exponential Backoff with Jitter for Network Retries	179
Case 2: State-Machine Dispatcher Loop	180
Pitfalls	182
Pitfall 1: The Accidental Runaway Infinite Loop	182
Pitfall 2: Busy-Waiting Without Sleeping	182
Exercises	182
Further reading	182
The for Loop and Sequence Iteration	183
The for Loop and Sequence Iteration	184
Mental model	184
Minimal example	184
Worked examples	186
Case 1: Deconstructing the Iterator Protocol Manually	186
Case 2: File Iteration Line by Line (Constant Memory)	186
Pitfalls	187
Pitfall 1: Modifying a Collection While Iterating Over It	187
Pitfall 2: The <code>range(len(sequence))</code> Anti-Pattern	188
Exercises	188
Further reading	188
Loop Control: <code>break</code>, <code>continue</code>, and <code>Loop else</code>	189
Loop Control: <code>break</code>, <code>continue</code>, and <code>Loop else</code>	190
Mental model	190
Minimal example	190
Worked examples	192
Case 1: Search and Validate Without Boolean Flags (<code>for ... else</code>)	192
Case 2: Breaking Out of Nested Loops	192
Pitfalls	193
Pitfall 1: Assuming <code>break</code> Exits All Nested Loops	193
Pitfall 2: Confusing <code>Loop else</code> with <code>if else</code>	194
Exercises	194

Further reading	194
Iteration Utilities: enumerate and zip	195
Iteration Utilities: enumerate and zip	196
Mental model	196
Minimal example	196
Worked examples	197
Case 1: Catching Silent Data Truncation with <code>strict=True</code>	197
Case 2: Unequal Streams with Padding Using <code>itertools.zip_longest</code>	198
Pitfalls	199
Pitfall 1: Relying on Default <code>zip()</code> Without <code>strict=True</code>	199
Pitfall 2: Re-Iterating a <code>zip</code> or <code>enumerate</code> Object	199
Exercises	199
Further reading	200
Structural Pattern Matching with match and case	201
Structural Pattern Matching with match and case	202
Mental model	202
Minimal example	202
Worked examples	203
Case 1: Mapping Patterns for Telemetry Event Routing	203
Case 2: Class Patterns with Dataclasses	204
Pitfalls	206
Pitfall 1: The Variable Capture Trap (Constants vs Variables)	206
Exercises	206
Further reading	207
List, Dict, and Set Comprehensions	208
List, Dict, and Set Comprehensions	209
Mental model	209
Minimal example	210
Worked examples	211
Case 1: Ingesting and Sanitizing Raw Metric Records (List Comprehensions)	211
Case 2: Inverting and Indexing Service Registries (Dict Comprehensions)	212
Case 3: Matrix Flattening and Cartesian Grids (Nested Loops)	213
Case 4: Avoiding Duplicate Computation with the Walrus Operator (<code>:=</code>)	214
Pitfalls	216
Pitfall 1: Misusing Comprehensions for Side Effects	216
Pitfall 2: Over-Nesting (Write-Only Code)	216
Pitfall 3: Scope Isolation (Python 3 Comprehension Scope)	217
Exercises	217
Further reading	218
Generator Expressions	219

Generator Expressions	220
Mental model	220
Minimal example	221
Worked examples	222
Case 1: Streaming Gigabyte Log Parsing with Constant O(1) Memory	222
Case 2: Short-Circuiting Aggregations with <code>any()</code> and <code>all()</code>	223
Case 3: Composing Multi-Stage Data Pipelines	224
Pitfalls	225
Pitfall 1: Generator Exhaustion (The “Read-Once” Trap)	225
Pitfall 2: Late-Binding Closures in Loop Expressions	226
Pitfall 3: Indexing and Length Queries (<code>gen[0]</code> or <code>len(gen)</code>)	226
Exercises	227
Further reading	227
 Data Structures	 228
 Lists and Dynamic Array Mechanics	 229
 Lists and Dynamic Array Mechanics	 230
Mental model	230
Minimal example	231
Worked examples	232
Case 1: In-Place Mutations vs Full Copy Allocations	232
Case 2: Custom In-Place Sorting with Timsort	233
Case 3: Shallow Copies vs Deep Copies	234
Pitfalls	235
Pitfall 1: Modifying a List While Iterating Over It	235
Pitfall 2: The Repeated Multiplication Reference Duplication Bug	235
Pitfall 3: Assuming <code>list.sort()</code> Returns the Sorted List	236
Exercises	237
Further reading	237
 Tuples and Immutable Sequence Records	 238
 Tuples and Immutable Sequence Records	 239
Mental model	239
The Hashability Rule for Tuples	239
Minimal example	240
Worked examples	241
Case 1: Composite Coordinates and Multi-Dimensional Indexing	241
Case 2: Defensive API Design (Immutable Return Values)	242
Case 3: Tuple Struct Packing and Structural Records	243
Pitfalls	244
Pitfall 1: The Missing Comma in Singleton Tuples	244
Pitfall 2: The Mutable-Element-in-Tuple Trap	244
Pitfall 3: The Augmented Assignment Trap on Tuples	245
Exercises	246
Further reading	246

Dictionaries and Hash Table Mechanics	247
Dictionaries and Hash Table Mechanics	248
Mental model	248
Key Advantages	248
Minimal example	249
Worked examples	250
Case 1: Grouping with <code>.setdefault()</code> vs Explicit Checking	250
Case 2: Multi-Layer Configuration Cascades with Dict Merges	251
Case 3: Dynamic Dictionary Views vs Static Snapshots	252
Pitfalls	253
Pitfall 1: Modifying a Dictionary During Iteration	253
Pitfall 2: Direct Subscripting on Optional Keys (<code>KeyError</code>)	253
Pitfall 3: Mutable Objects as Dictionary Keys	254
Exercises	254
Further reading	254
Sets and Set Theory Operations	255
Sets and Set Theory Operations	256
Mental model	256
Set Theory Operations	256
Minimal example	257
Worked examples	258
Case 1: Declarative Infrastructure State Reconciliation	258
Case 2: Role-Based Access Control (RBAC) with Subset Checks	259
Case 3: Immutable Sets with <code>frozenset</code> as Dictionary Keys	260
Pitfalls	260
Pitfall 1: The Empty Set Syntax Trap (<code>{}</code> vs <code>set()</code>)	260
Pitfall 2: Adding Mutable Objects to a Set	261
Pitfall 3: Mutating a Set While Iterating	261
Exercises	262
Further reading	262
Advanced Slicing and Pattern Unpacking	263
Advanced Slicing and Pattern Unpacking	264
Mental model	264
The Stride Equation	264
Extended Starred Unpacking	264
Minimal example	265
Worked examples	266
Case 1: In-Place Slice Replacement and Resizing	266
Case 2: Parsing Protocol Frames with Named Slices	266
Case 3: Starred Head-Tail Splitting for Recursive Processing	267
Pitfalls	268
Pitfall 1: Assigning a Non-Iterable to a Slice	268
Pitfall 2: Multiple Starred Expressions in a Single Target	269
Exercises	269

Further reading	269
Binary Search with Bisect and Priority Queues with Heapq	270
Binary Search with Bisect and Priority Queues with Heapq	271
Mental model	271
1. <code>bisect</code> : Bisection and Binary Search	271
2. <code>heapq</code> : Min-Heap Priority Queues	271
Minimal example	272
Worked examples	273
Case 1: Numeric Score / Severity Bucketing with <code>bisect</code>	273
Case 2: Multi-Criteria Priority Scheduler with Tie-Breaking Counter	274
Case 3: Finding Top <i>K</i> Extremes with <code>nlargest</code> and <code>nsmallest</code>	275
Pitfalls	276
Pitfall 1: Calling <code>bisect</code> on an Unsorted List	276
Pitfall 2: Modifying Elements Inside a Heap Directly	277
Exercises	277
Further reading	277
Zero-Copy Binary Memory with <code>memoryview</code> and <code>bytearray</code>	278
Zero-Copy Binary Memory with <code>memoryview</code> and <code>bytearray</code>	279
Mental model	279
Binary Types Compared	280
Minimal example	280
Worked examples	281
Case 1: Zero-Copy Network Packet Header Dissection	281
Case 2: In-Place Mutation with <code>bytearray</code> and <code>memoryview</code>	282
Case 3: Casting Memory Layouts with <code>.cast()</code>	283
Pitfalls	284
Pitfall 1: Attempting In-Place Writes on an Immutable <code>bytes</code> View	284
Pitfall 2: The Buffer Resizing Lock Trap	285
Exercises	285
Further reading	286
Error Handling	287
Exceptions, Tracebacks, and Propagation	288
Exceptions, Tracebacks, and Propagation	289
Mental model	289
The Four-Part Exception Lifecycle	289
Minimal example	290
Worked examples	291
Case 1: Attaching Diagnostics with Exception Notes (<code>add_note()</code>)	291
Case 2: Concurrent Failure Bundling with <code>ExceptionGroup</code>	292
Pitfalls	293
Pitfall 1: The Bare <code>except:</code> or <code>except Exception:</code> Anti-Pattern	293

Pitfall 2: Forgetting to Re-Raise After Logging	293
Exercises	294
Further reading	294
Custom Exceptions and Error Hierarchies	295
Custom Exceptions and Error Hierarchies	296
Mental model	296
Exception Chaining (<code>__cause__</code> vs <code>__context__</code>)	296
Minimal example	297
Worked examples	297
Case 1: Exception Chaining with <code>raise ... from err</code>	297
Case 2: Suppressing Internal Noise with <code>from None</code>	298
Pitfalls	299
Pitfall 1: Inheriting from <code>BaseException</code> instead of <code>Exception</code>	299
Pitfall 2: Overly Flat or Monolithic Exceptions	300
Exercises	300
Further reading	300
Context Managers and Resource Management	301
Context Managers and Resource Management	302
Mental model	302
Minimal example	302
Worked examples	304
Case 1: Benchmark Timer with <code>@contextlib.contextmanager</code>	304
Case 2: Clean Error Suppression with <code>contextlib.suppress</code>	305
Case 3: Dynamic Multi-Resource Coordination with <code>ExitStack</code>	305
Pitfalls	306
Pitfall 1: Accidentally Suppressing All Exceptions in <code>__exit__</code>	306
Pitfall 2: Omitting <code>try/finally</code> in <code>@contextmanager</code> Generators	307
Exercises	307
Further reading	308
Exception Groups and the <code>except*</code> Operator	309
Exception Groups and the <code>except*</code> Operator	310
Mental model	310
Traditional <code>except</code> vs Modern <code>except*</code>	311
Minimal example	311
Worked examples	312
Case 1: Concurrent Node Health Probing with Diagnostic Subgroups	312
Case 2: Programmatic Group Splitting and Filtering (<code>.split()</code> and <code>.subgroup()</code>)	313
Case 3: Enriching Individual Errors in Groups with <code>add_note()</code>	314
Pitfalls	315
Pitfall 1: Mixing <code>except</code> and <code>except*</code> in the Same <code>try</code> Block	315
Pitfall 2: Partial Matching Leaves Unhandled Errors to Propagate	316
Pitfall 3: Catching <code>ExceptionGroup</code> Directly with Traditional <code>except</code>	316
Exercises	317

Further reading	317
Objects and Classes	318
Classes, Instances, and Encapsulation	319
Classes, Instances, and Encapsulation	320
Mental model	320
Minimal example	320
Worked examples	322
Case 1: Computed Properties & Caching	322
Case 2: Demystifying Name Mangling (<code>__var</code>)	323
Pitfalls	324
Pitfall 1: Mutating Class Attributes via an Instance Reference	324
Exercises	324
Further reading	325
The Python Data Model and Dunder Protocols	326
The Python Data Model and Dunder Protocols	327
Mental model	327
Minimal example	327
Worked examples	329
Case 1: The Golden Rule of <code>__repr__</code> vs <code>__str__</code>	329
Case 2: Equality (<code>__eq__</code>) and Hashing (<code>__hash__</code>)	330
Case 3: Automatic Ordering with <code>functools.total_ordering</code>	331
Case 4: Callable Objects with <code>__call__</code>	332
Pitfalls	333
Pitfall 1: Defining <code>__eq__</code> Without <code>__hash__</code>	333
Exercises	333
Further reading	334
Inheritance, MRO, and Composition	335
Inheritance, MRO, and Composition	336
Mental model	336
Cooperative <code>super()</code> Dispatch	336
Minimal example	337
Worked examples	338
Case 1: Formal Interfaces with Abstract Base Classes (<code>abc.ABC</code>)	338
Case 2: Composition Over Inheritance	339
Pitfalls	340
Pitfall 1: Calling Direct Parent Names Instead of <code>super()</code>	340
Pitfall 2: Overusing Multiple Inheritance	341
Exercises	341
Further reading	341
Dataclasses and Declarative Models	342

Dataclasses and Declarative Models	343
Mental model	343
Memory Architecture: <code>__dict__</code> vs <code>__slots__</code>	343
Minimal example	344
Worked examples	345
Case 1: Mutable Defaults via <code>field(default_factory=...)</code>	345
Case 2: Memory Benchmark: <code>slots=True</code> vs Standard Class	345
Case 3: Dataclasses vs Pydantic v2: Architectural Boundaries	346
Pitfalls	347
Pitfall 1: Assuming <code>@dataclass</code> Enforces Types at Runtime	347
Exercises	347
Further reading	348
Decorators and Metaprogramming	349
Decorators and Metaprogramming	350
Mental model	350
The Wrapper Wrapping Pipeline	350
Minimal example	351
Worked examples	352
Case 1: Parameterized Decorator Factory (<code>@retry</code>)	352
Case 2: Method Decorators: <code>@classmethod</code> vs <code>@staticmethod</code>	353
Case 3: Class Decorators for Automated Registration	354
Pitfalls	355
Pitfall 1: Omitting <code>@functools.wraps</code>	355
Pitfall 2: Decorator Definition-Time vs Execution-Time Confusion	355
Exercises	355
Further reading	355
Descriptors and the Attribute Access Protocol	356
Descriptors and the Attribute Access Protocol	357
Mental model	357
The Descriptor Protocol Methods	358
Minimal example	358
Worked examples	359
Case 1: How <code>@property</code> Works Under the Hood	359
Case 2: Lazy Cached Properties with Non-Data Descriptors	361
Pitfalls	362
Pitfall 1: Storing Value on the Descriptor Instance	362
Pitfall 2: Forgetting if <code>instance</code> is <code>None</code> : in <code>__get__</code>	363
Exercises	364
Further reading	364
Metaclasses and Class Construction Mechanics	365
Metaclasses and Class Construction Mechanics	366
Mental model	366
The CPython Class Construction Pipeline	366

Minimal example	367
Worked examples	368
Case 1: The Modern Alternative: Zero-Boilerplate Plugins with <code>__init_subclass__</code>	368
Case 2: Enforcing Architectural Invariants with a Custom Metaclass	369
Case 3: Capturing Attribute Declaration Order with <code>__prepare__</code>	371
Pitfalls	372
Pitfall 1: The Metaclass Conflict Error	372
Pitfall 2: Overusing Metaclasses when Simpler Tools Suffice	372
Exercises	373
Further reading	373
Abstract Base Classes and Structural Protocols	374
Abstract Base Classes and Structural Protocols	375
Mental model	375
Comparison Matrix	375
Minimal example	376
Worked examples	377
Case 1: Enforcing Framework Contracts with Abstract Base Classes	377
Case 2: Structural Decoupling with Protocols Across Unrelated Packages	378
Case 3: Virtual Subclasses via <code>ABC.register()</code>	379
Pitfalls	381
Pitfall 1: Attempting to Instantiate an Incomplete ABC	381
Pitfall 2: Inverting Decorator Order with <code>@property</code> and <code>@abstractmethod</code>	381
Pitfall 3: Assuming <code>@runtime_checkable</code> Validates Types and Signatures	382
Exercises	382
Further reading	383
Modules and Packages	384
Modules and the Import System	385
Modules and the Import System	386
Mental model	386
Minimal example	386
Worked examples	388
Case 1: Dynamic Driver Registry with <code>importlib</code>	388
Case 2: Diagnosing and Breaking Circular Imports	388
Case 3: The <code>if __name__ == "__main__":</code> Boundary	390
Pitfalls	390
Pitfall 1: Standard Library Shadowing	390
Exercises	391
Further reading	391
Packages, Namespaces, and Public APIs	392
Packages, Namespaces, and Public APIs	393
Mental model	393

Minimal example	393
Worked examples	395
Case 1: Relative vs Absolute Imports	395
Case 2: The <code>__all__</code> Contract and <code>from module import *</code>	395
Case 3: PEP 420 Namespace Packages	396
Pitfalls	397
Pitfall 1: Executing Submodules Directly Instead of via <code>-m</code>	397
Pitfall 2: Heavy Work Inside <code>__init__.py</code>	397
Exercises	397
Further reading	397
Workspaces, Virtual Environments, and Dependency Isolation	398
Workspaces, Virtual Environments, and Dependency Isolation	399
Mental model	399
Minimal example	399
Worked examples	400
Case 1: Inline Script Dependencies (PEP 723)	400
Case 2: Multi-Package Workspaces with <code>uv</code>	401
Pitfalls	402
Pitfall 1: Breaking OS Packages with <code>sudo pip install</code> (PEP 668)	402
Pitfall 2: Committing <code>.venv/</code> into Version Control	402
Exercises	403
Further reading	403
Import Hooks, Finders, and ModuleSpec Mechanics	404
Import Hooks, Finders, and ModuleSpec Mechanics	405
Mental model	405
Minimal example	406
Worked examples	407
Case 1: Loading Modules Directly from JSON Configurations	407
Case 2: Auditing Dependency Load Latency via a <code>MetaPath</code> Hook	409
Pitfalls	410
Pitfall 1: Circular Import Deadlocks	410
Pitfall 2: Forgetting to Clean Up <code>sys.meta_path</code>	411
Exercises	411
Further reading	412
Lazy Imports (PEP 810)	413
Lazy Imports (PEP 810)	414
Thesis	414
Mental model	414
Worked examples	415
Case 1: A multi-command CLI that only pays for the command you run	415
Case 2: The error moves — and so must your smoke test	417
Case 3: One source file for 3.13, 3.14, and 3.15 with <code>__lazy_modules__</code>	419
Case 4: Type-only imports without <code>if TYPE_CHECKING:</code>	420

Case 5: Application-wide laziness, scoped by a filter	421
The trap	423
The boring rule	424
Try this	425
Sources	425
Standard Library	426
Specialized Collections, Enums, and Heaps	427
Specialized Collections, Enums, and Heaps	428
Mental model	428
Minimal example	428
Worked examples	430
Case 1: High-Speed Counter Arithmetic for Anomaly Detection	430
Case 2: Logarithmic Table Lookup with <code>bisect</code>	430
Pitfalls	431
Pitfall 1: Modifying Elements in a <code>heapq</code> Directly	431
Exercises	432
Further reading	432
High-Performance Iteration and Functional Tools	433
High-Performance Iteration and Functional Tools	434
Mental model	434
Minimal example	434
Worked examples	436
Case 1: Telemetry Grouping with <code>itertools.groupby()</code>	436
Case 2: Clean Polymorphism with <code>functools singledispatch</code>	436
Pitfalls	438
Pitfall 1: Unsorted Input to <code>itertools.groupby</code>	438
Pitfall 2: Decorating Functions with Mutable Arguments in <code>@lru_cache</code>	438
Exercises	438
Further reading	438
Text Processing, Unicode, and Regular Expressions	439
Text Processing, Unicode, and Regular Expressions	440
Mental model	440
Minimal example	440
Worked examples	442
Case 1: Dynamic Data Masking with <code>re.sub()</code> Callables	442
Case 2: Unicode Normalization for Security Checks	442
Pitfalls	443
Pitfall 1: Catastrophic Backtracking (ReDoS)	443
Pitfall 2: Omitting Raw String Prefix (<code>r""</code>)	444
Exercises	444
Further reading	444

Filesystem and Modern Path Manipulation	445
Filesystem and Modern Path Manipulation	446
Mental model	446
Minimal example	446
Worked examples	448
Case 1: Buffered Chunk Streaming for Gigabyte-Scale Files	448
Case 2: Atomic File Writes via Temporary Renaming	448
Pitfalls	449
Pitfall 1: Mixing <code>os.path</code> String Manipulation with <code>Path</code> Objects	449
Pitfall 2: Forgetting <code>parents=True</code> on <code>mkdir()</code>	449
Exercises	450
Further reading	450
Structured Data Serialization: JSON, CSV, and TOML	451
Structured Data Serialization: JSON, CSV, and TOML	452
Mental model	452
Minimal example	452
Worked examples	454
Case 1: Custom JSON Serialization for <code>datetime</code> , <code>UUID</code> , and <code>set</code>	454
Case 2: Reading CSV with <code>DictReader</code> and Type Conversion	455
Pitfalls	456
Pitfall 1: Writing CSVs on Windows Without <code>newline=""</code>	456
Pitfall 2: <code>tomllib</code> is Read-Only	456
Exercises	457
Further reading	457
Embedded Relational Storage with SQLite3	458
Embedded Relational Storage with SQLite3	459
Mental model	459
Minimal example	459
Worked examples	460
Case 1: High-Throughput Batch Insertion with <code>executemany()</code>	460
Case 2: Transaction Atomicity and Automatic Rollback	462
Pitfalls	463
Pitfall 1: SQL Injection via String Interpolation	463
Pitfall 2: Trailing Comma in Single Parameter Tuples	463
Exercises	463
Further reading	464
Dates, Times, and Timezones	465
Dates, Times, and Timezones	466
Mental model	466
Minimal example	466
Worked examples	467
Case 1: The Daylight Saving Time (DST) Arithmetic Trap	467

Case 2: TTL Expiration and Token Age Validation	468
Pitfalls	469
Pitfall 1: Calling <code>datetime.now()</code> Without Arguments	469
Pitfall 2: Using Deprecated <code>datetime.utcnow()</code>	469
Exercises	469
Further reading	470
Math, Randomness, and Cryptography	471
Math, Randomness, and Cryptography	472
Mental model	472
Minimal example	472
Worked examples	474
Case 1: Timing Attack Defense with <code>hmac.compare_digest</code>	474
Case 2: Exact Rational Fractions with <code>fractions.Fraction</code>	474
Pitfalls	475
Pitfall 1: Using <code>random</code> for Security Tokens	475
Pitfall 2: Initializing <code>Decimal</code> from a <code>float</code>	475
Exercises	476
Further reading	476
OS, System Environment, and Subprocess Management	477
OS, System Environment, and Subprocess Management	478
Mental model	478
Minimal example	478
Worked examples	480
Case 1: Piping Streams Between Child Processes (<code>p1 p2</code>)	480
Case 2: Sandboxing Environment Variables	481
Pitfalls	481
Pitfall 1: The <code>shell=True</code> Security Vulnerability	481
Pitfall 2: Forgetting <code>timeout=</code>	482
Exercises	482
Further reading	482
CLI Parsing and Structured Application Logging	483
CLI Parsing and Structured Application Logging	484
Mental model	484
Minimal example	484
Worked examples	486
Case 1: Structured JSON Logging for Observability Systems	486
Case 2: Log File Rotation with <code>RotatingFileHandler</code>	487
Pitfalls	488
Pitfall 1: Adding Duplicate Handlers	488
Pitfall 2: Using <code>print()</code> Instead of <code>logging</code> in Libraries	488
Exercises	489
Further reading	489

Compression, Tarballs, and Archive Management	490
Compression, Tarballs, and Archive Management	491
Mental model	491
Minimal example	491
Worked examples	492
Case 1: In-Memory Gzip Payload Compression for Network APIs	492
Case 2: Defending Against Path Traversal (Tar Slip / Zip Slip)	493
Pitfalls	494
Pitfall 1: Creating Uncompressed ZIP Archives	494
Pitfall 2: Extracting Untrusted Archives Without Filters	494
Exercises	495
Further reading	495
Networking, Low-Level Sockets, and IP Address Calculations	496
Networking, Low-Level Sockets, and IP Address Calculations	497
Mental model	497
Minimal example	497
Worked examples	499
Case 1: Low-Level TCP Loopback Echo Service	499
Case 2: Subnet Splitting and Overlap Detection	500
Pitfalls	501
Pitfall 1: Host Bits Set in <code>IPv4Network</code>	501
Pitfall 2: Assuming <code>recv()</code> Returns a Full Packet	501
Exercises	502
Further reading	502
Signal Handling and Process Lifecycle Management	503
Signal Handling and Process Lifecycle Management	504
Mental model	504
Minimal example	505
Worked examples	506
Case 1: Container Termination Controller with In-Flight Drain	506
Case 2: Zero-Downtime Configuration Reload via <code>SIGHUP</code>	507
Case 3: Watchdog Execution Timers with <code>SIGALRM</code>	508
Pitfalls	510
Pitfall 1: Registering Signals Outside the Main Thread	510
Pitfall 2: Deadlocks from Non-Reentrant Code in Signal Handlers	510
Pitfall 3: PID 1 in Docker Containers Ignoring Signals	511
Exercises	511
Further reading	511
Built-In HTTP Networking: Servers, Clients, and Request Handlers	512
Built-In HTTP Networking: Servers, Clients, and Request Handlers	513
Mental model	513
Minimal example	514

Worked examples	515
Case 1: Building a REST Metrics Ingestion Webhook Handler	515
Case 2: Low-Level Streaming Requests with <code>http.client.HTTPConnection</code>	517
Pitfalls	518
Pitfall 1: Using <code>HTTPServer</code> Instead of <code>ThreadingHTTPServer</code>	518
Pitfall 2: Omitting the <code>timeout</code> Parameter in <code>urllib.request.urlopen</code>	519
Pitfall 3: Not Reading <code>Content-Length</code> in <code>do_POST</code>	519
Exercises	519
Further reading	520
High-Performance File I/O with Memory-Mapped Files (<code>mmap</code>)	521
High-Performance File I/O with Memory-Mapped Files (<code>mmap</code>)	522
Mental model	522
Minimal example	523
Worked examples	524
Case 1: Regex Searching Gigabyte Logs with Zero RAM Allocation	524
Case 2: In-Place Binary Patching and Header Updates	525
Case 3: Anonymous Shared Memory IPC Between Processes	526
Pitfalls	527
Pitfall 1: Attempting to Map an Empty (0-Byte) File	527
Pitfall 2: Modifying File Size Outside <code>mmap</code>	528
Exercises	528
Further reading	528
Runtime Introspection and Reflection with <code>inspect</code>	529
Runtime Introspection and Reflection with <code>inspect</code>	530
Mental model	530
Minimal example	531
Worked examples	532
Case 1: Runtime Type-Enforcing Decorator with <code>Signature.bind()</code>	532
Case 2: Zero-Magic Dependency Injection Container	534
Case 3: Caller-Context Telemetry and Frame Navigation	536
Pitfalls	537
Pitfall 1: Retaining Frame References and Creating GC Leaks	537
Pitfall 2: <code>inspect.getsource()</code> on Dynamically Evaluated or C Code	538
Exercises	538
Further reading	539
Weak References and Garbage Collection (<code>weakref</code>, <code>gc</code>)	540
Weak References and Garbage Collection (<code>weakref</code>, <code>gc</code>)	541
Mental model	541
Minimal example	542
Worked examples	543
Case 1: Self-Pruning In-Memory Entity Cache with <code>WeakValueDictionary</code>	543
Case 2: Deterministic OS Resource Cleanup with <code>weakref.finalize</code>	545
Case 3: Diagnosing and Breaking Cyclic Reference Leaks with <code>gc</code>	546

Pitfalls	548
Pitfall 1: Attempting to Weakly Reference Built-in Types Directly	548
Pitfall 2: Capturing <code>self</code> in a Finalizer Callback	549
Exercises	549
Further reading	550
Mocking and Test Doubles (<code>unittest.mock</code>)	551
Mocking and Test Doubles (<code>unittest.mock</code>)	552
Mental model	552
The Golden Rule of Patching: “Where to Patch”	552
Minimal example	553
Worked examples	554
Case 1: Simulating Flaky Networks with <code>side_effect</code> Retries	554
Case 2: Preventing Mock Drift with <code>autospec=True</code>	555
Case 3: Testing Asynchronous Event Pipelines with <code>AsyncMock</code>	556
Pitfalls	558
Pitfall 1: Patching the Definition Site Instead of the Lookup Site	558
Pitfall 2: Misspelling Assertion Methods (<code>assert_called_once_with</code> typo)	558
Exercises	559
Further reading	559
Binary Data and Network Protocols (<code>struct</code>)	560
Binary Data and Network Protocols (<code>struct</code>)	561
Mental model	561
Endianness & Byte Order Modifiers	561
Minimal example	562
Worked examples	563
Case 1: Custom Binary RPC Wire Framing with Checksum Validation	563
Case 2: High-Throughput Stream Parsing with <code>struct.iter_unpack</code>	565
Case 3: Zero-Copy In-Place Buffer Updates with <code>pack_into</code> and <code>unpack_from</code>	566
Pitfalls	567
Pitfall 1: Relying on Native <code>@</code> Alignment for Network or Disk Formats	567
Pitfall 2: String Length Mismatches in <code>s</code> Format Strings	568
Exercises	568
Further reading	569
Object Serialization and Persistent Storage (<code>pickle</code>, <code>shelve</code>)	570
Object Serialization and Persistent Storage (<code>pickle</code>, <code>shelve</code>)	571
Mental model	571
Security Warning: Never Unpickle Untrusted Data	572
Minimal example	572
Worked examples	573
Case 1: Sandboxed Deserialization with <code>SafeUnpickler</code>	573
Case 2: Persistent Disk State with <code>shelve</code>	574
Case 3: Excluding Transient Resources with <code>__getstate__</code> and <code>__setstate__</code>	576

Pitfalls	577
Pitfall 1: Mutating Mutable Objects in <code>shelve</code> Without <code>writeback=True</code>	577
Pitfall 2: Protocol Incompatibility Across Python Versions	578
Exercises	578
Further reading	578
Typing and Testing	579
Modern Type Hints and Static Analysis	580
Modern Type Hints and Static Analysis	581
Mental model	581
Nominal Subtyping vs Structural Subtyping (<code>Protocol</code>)	581
Minimal example	581
Worked examples	582
Case 1: Generic Data Containers (<code>typing.Generic</code>)	582
Case 2: Fluent Method Chaining with <code>typing.Self</code>	583
Pitfalls	584
Pitfall 1: Confusing <code>Type[T]</code> with <code>T</code>	584
Pitfall 2: Using <code>Union</code> Instead of the Modern <code> </code> Operator	584
Exercises	585
Further reading	585
Unit and Integration Testing with Pytest	586
Unit and Integration Testing with Pytest	587
Mental model	587
Minimal example	587
Worked examples	589
Case 1: Fixtures with Setup and Teardown Lifecycles	589
Case 2: Isolating External Dependencies with <code>unittest.mock</code>	589
Pitfalls	590
Pitfall 1: Mutating Shared Session Fixtures	590
Pitfall 2: Testing Implementation Details Instead of Contracts	590
Exercises	591
Further reading	591
Packaging, Wheel Distribution, and Publishing	592
Packaging, Wheel Distribution, and Publishing	593
Mental model	593
Minimal example	593
Worked examples	595
Case 1: Defining CLI Console Scripts via <code>[project.scripts]</code>	595
Case 2: Building Wheels with <code>uv build</code>	595
Case 3: Inspecting Wheel Metadata (<code>.dist-info</code>)	595
Pitfalls	596
Pitfall 1: Relying on Legacy <code>setup.py</code>	596

Pitfall 2: Missing Package Data Files	596
Exercises	597
Further reading	597
Advanced Type System: PEP 695, ParamSpec, and Type Narrowing	598
Advanced Type System: PEP 695, ParamSpec, and Type Narrowing	599
Mental model	599
Type Narrowing: TypeGuard vs TypeIs	599
Minimal example	600
Worked examples	601
Case 1: Signature-Preserving Decorators with ParamSpec and Concatenate	601
Case 2: Bidirectional Narrowing with TypeIs (PEP 742)	602
Case 3: Compile-Time Injection Immunity with LiteralString (PEP 675)	603
Pitfalls	604
Pitfall 1: Assuming TypeGuard Narrows the else Branch	604
Pitfall 2: Forgetting assert_never in Exhaustive Pattern Matching	605
Exercises	605
Further reading	605
Deferred Annotations	607
Deferred Annotations	608
Mental model	608
Worked examples	608
Case 1: Forward ref without quotes	608
Case 2: All three formats on a resolved function	609
Case 3: Class annotation before the name exists	610
Case 4: Evaluate when you need types at runtime	611
Case 5: The __annotations__ / get_type_hints habit	612
The trap	613
The boring rule	614
Try this	614
Concurrency and Internals	616
CPython VM, Bytecode, and Code Execution	617
CPython VM, Bytecode, and Code Execution	618
Mental model	618
Minimal example	618
Worked examples	620
Case 1: The Adaptive Specializing Interpreter (PEP 659)	620
Case 2: Inspecting Call Stack Frames with sys._getframe()	620
Pitfalls	621
Pitfall 1: Modifying Code Objects at Runtime	621
Exercises	622
Further reading	622

Memory Allocator, PyObject, and Garbage Collection	623
Memory Allocator, PyObject, and Garbage Collection	624
Mental model	624
Minimal example	625
Worked examples	626
Case 1: Reference Counting Mechanics with <code>sys.getrefcount()</code>	626
Case 2: Leak-Free Caching with <code>weakref.WeakValueDictionary</code>	626
Pitfalls	627
Pitfall 1: Believing <code>del</code> Immediately Frees OS RAM	627
Pitfall 2: Disabling the Garbage Collector in Long-Running Daemons	628
Exercises	628
Further reading	628
The Global Interpreter Lock and Python 3.14 Free-Threading	629
The Global Interpreter Lock and Python 3.14 Free-Threading	630
Mental model	630
Why Did the GIL Exist?	630
How PEP 703 Removes the GIL	631
Minimal example	631
Worked examples	632
Case 1: CPU-Bound Parallel Scaling Benchmark	632
Case 2: Why Thread Locks Are Still Mandatory in Free-Threaded Python	633
Pitfalls	634
Pitfall 1: Assuming “No GIL” Means No Thread Synchronization	634
Pitfall 2: Legacy C-Extensions Without Free-Threading Support	634
Exercises	634
Further reading	635
Multithreading, Multiprocessing, and Process Pools	636
Multithreading, Multiprocessing, and Process Pools	637
Mental model	637
Minimal example	637
Worked examples	639
Case 1: Thread-Safe Producer-Consumer Pipeline with <code>queue.Queue</code>	639
Case 2: Zero-Copy Shared Memory Across Processes	640
Pitfalls	641
Pitfall 1: Forgetting <code>if __name__ == "__main__":</code> in Multiprocessing	641
Pitfall 2: High IPC Pickling Overhead	641
Exercises	641
Further reading	641
Subinterpreters	642
Subinterpreters	643
Mental model	643

Worked examples	644
Case 1: Create, call, close	644
Case 2: Isolation with a queue	644
Case 3: <code>InterpreterPoolExecutor</code>	645
Case 4: Thread + queue for a result	646
The trap	647
The boring rule	648
Try this	648
InterpreterPoolExecutor	649
InterpreterPoolExecutor	650
Mental model	650
Worked examples	651
Case 1: Map CPU chunks across interpreters	651
Case 2: <code>submit</code> + <code>as_completed</code>	651
Case 3: When vs Thread vs Process	652
Case 4: Initializer runs in each worker interpreter	653
The trap	654
The boring rule	655
Try this	655
Asynchronous Programming and TaskGroups	656
Asynchronous Programming and TaskGroups	657
Mental model	657
Structured Concurrency with <code>asyncio.TaskGroup</code>	657
Minimal example	658
Worked examples	659
Case 1: Deterministic Deadlines with <code>asyncio.timeout()</code>	659
Case 2: Throttling Concurrent Requests with <code>asyncio.Semaphore</code>	660
Pitfalls	661
Pitfall 1: Calling Synchronous Blocking Code in Coroutines	661
Pitfall 2: Using Legacy <code>asyncio.gather()</code> Without Shielding	661
Exercises	661
Further reading	661
Profiling, Optimization, and Native C/Rust Interoperability	662
Profiling, Optimization, and Native C/Rust Interoperability	663
Mental model	663
Minimal example	663
Worked examples	665
Case 1: Locating Memory Leaks with <code>tracemalloc</code>	665
Case 2: Calling Native C Functions with <code>ctypes.Structure</code>	666
Case 3: Modern High-Performance Extensions: PyO3 and Rust	666
Pitfalls	667
Pitfall 1: Premature Micro-Optimization	667
Pitfall 2: Memory Hazards in <code>ctypes</code>	667

Exercises	667
Further reading	667
Attaching to a Live Process	668
Attaching to a Live Process	669
Mental model	669
Worked examples	670
Case 1: Run a probe inside a running worker	670
Case 2: Dump every thread's stack and bring it home	672
Case 3: <code>pdb -p</code> — a debugger on a process you did not start under one	673
Case 4: See every asyncio task with <code>ps</code> and <code>pstree</code>	674
The trap	676
The boring rule	678
Try this	678
Capstone Lab: High-Throughput Asynchronous Log Processing Engine	679
Capstone Lab: High-Throughput Asynchronous Log Processing Engine	680
Architectural blueprint	680
Complete runnable engine	680
Running the capstone	684
Architectural takeaways	685
Data Structures and Algorithms	686
Algorithmic Complexity and Big-O	687
Algorithmic Complexity and Big-O	688
Mental model	688
Worked examples	689
Case 1: Dict lookup $O(1)$ vs List linear search $O(n)$	689
Case 2: Detecting duplicates (Quadratic nested loops vs Linear Set)	690
The trap	691
The boring rule	692
Try this	693
Stacks, Queues, and Deques	694
Stacks, Queues, and Deques	695
Mental model	695
Worked examples	696
Case 1: LIFO Stack for Expression and Delimiter Validation	696
Case 2: High-Performance FIFO Queue with <code>collections.deque</code>	697
Case 3: Fixed-size Ring Buffer (Circular Buffer)	698
The trap	700
The boring rule	700
Try this	700

Linked Lists	701
Linked Lists	702
Mental model	702
Python List vs Linked List: The Cache Reality	703
Worked examples	703
Case 1: Singly Linked List with in-place reversal	703
Case 2: Doubly Linked List with O(1) Node Removal	705
The trap	707
The boring rule	707
Try this	707
Binary Search Trees	708
Binary Search Trees	709
Mental model	709
Worked examples	710
Case 1: Generic BST with Insert, Search, and In-Order Generator	710
Case 2: Deleting Nodes (Handling 0, 1, and 2 Children)	712
The trap	714
The boring rule	714
Try this	715
Heaps and Priority Queues	716
Heaps and Priority Queues	717
Mental model	717
Worked examples	718
Case 1: Min-Heap from scratch using index math	718
Case 2: Priority Queue with standard library heapq	720
The trap	721
The boring rule	721
Try this	722
Tries and Prefix Trees	723
Tries and Prefix Trees	724
Mental model	724
Worked examples	724
Case 1: Catalog SKU and Keyword Autocomplete	724
The trap	727
The boring rule	727
Try this	727
Sorting Algorithms	728
Sorting Algorithms	729
Mental model	729
Stable Sorting	730

Worked examples	731
Case 1: Adaptive Insertion Sort for nearly-sorted batches	731
Case 2: Stable Divide-and-Conquer Merge Sort	731
Case 3: In-Place Quicksort with Lomuto Partition	733
The trap	734
The boring rule	735
Try this	735
Binary Search and the bisect Module	736
Binary Search and the bisect Module	737
Mental model	737
Binary Search Halving	737
The <code>bisect</code> Invariant	737
Worked examples	737
Case 1: Binary Search from scratch	737
Case 2: Production Range and Tier Queries with standard library <code>bisect</code>	739
Case 3: Quickselect for Median Response Time	740
The trap	741
The boring rule	742
Try this	742
Graphs and Traversals (BFS and DFS)	743
Graphs and Traversals (BFS and DFS)	744
Mental model	744
Adjacency List Representation	744
BFS vs DFS Traversals	745
Worked examples	745
Case 1: BFS for Shortest Hop Delivery Routes	745
Case 2: DFS for Workflow Cycle and Deadlock Detection	747
The trap	748
The boring rule	749
Try this	749
Shortest Paths (Dijkstra’s Algorithm)	750
Shortest Paths (Dijkstra’s Algorithm)	751
Mental model	751
Worked examples	752
Case 1: Delivery Network Optimization with <code>heapq</code>	752
The trap	754
The boring rule	754
Try this	754
String Search and Bloom Filters	755

String Search and Bloom Filters	756
Mental model	756
Rabin-Karp Rolling Hash	756
Bloom Filter: Probabilistic Membership	756
Worked examples	757
Case 1: Rabin-Karp Rolling Hash Substring Search	757
Case 2: In-Memory Bloom Filter for Order Existence	758
The trap	760
The boring rule	760
Try this	760
 Long Term Engineering	 761
inspect and Reflection	762
inspect and Reflection	763
Mental model	763
Worked examples	763
Case 1: <code>inspect.signature</code>	763
Case 2: <code>getattr</code> with a default	764
Case 3: <code>inspect.getdoc</code>	764
The trap	765
The boring rule	767
Try this	767
 ctypes and FFI	 768
ctypes and FFI	769
Mental model	769
Worked examples	769
Case 1: <code>c_int</code> and <code>sizeof</code> (no extra C files)	769
Case 2: A tiny C-like struct	770
Case 3: The Python you wanted instead	771
The trap	771
The boring rule	772
Try this	772
 Performance Tuning	 773
Performance Tuning	774
Mental model	774
Worked examples	774
Case 1: Get the answer right, then time it	774
Case 2: <code>cProfile</code> names the function	775
Case 3: Two implementations, same answer	776
The trap	777
The boring rule	777
Try this	778

Designing for the Long Term	779
Designing for the Long Term	780
Mental model	780
Worked examples	780
Case 1: Two modules, one public function	780
Case 2: <code>pyproject.toml</code> is the project boundary	781
Case 3: One process that still looks like the first chapter	782
The trap	782
The boring rule	783
Try this	783
 Appendices	 784
Glossary	785
Glossary	786
Command Cheat Sheet	788
Command Cheat Sheet	789
Python	789
uv	789
ruff	789
pytest	790
Tiny reminders	790

Python

A comprehensive, self-contained engineering volume covering Python from scratch across the major domains of modern computing: **DevOps, Network Automation, Data Science, AI/GenAI, Robotics, Cybersecurity, and Distributed Web Systems.**

Part 1 (Python Foundations) begins with modern toolchain setup using **uv** and **ruff**, moves through core syntax and OOP, and provides an **exhaustive deep dive into the Python Standard Library (stdlib)** before covering modern typing, testing, packaging, concurrency, and CPython internals in a direct topic progression. Dedicated parts follow for each primary engineering domain, concluding with cross-domain **Enterprise Production Capstones.**

Baseline: Python **3.14** (free-threading enabled) · toolchain **uv** · linter/formatter **ruff**. Completely self-contained. No other title in this library is required.

Start here

Open **Basics** in the sidebar. You need a computer, a terminal, and any text editor. Zero prior Python experience required.

All examples are complete, runnable files:

```
uv run python hello.py
```

Book Roadmap & Master Architecture

Part	Slug	Domain / Focus	Key Concepts & Technologies
01	01-python-foundations	Python Foundations	Modern setup (uv, ruff), syntax, memory, data types, loops, functions, OOP, dataclasses, comprehensive Standard Library (collections, itertools, re, pathlib, json, sqlite3, datetime, subprocess, logging, socket), typing (mypy), testing (pytest), packaging, concurrency, CPython bytecode & free-threading
02	02-data-structures-and-algorithms	Data Structures & Algorithms	Algorithmic complexity, stacks, queues, linked lists, binary search trees, heaps, tries, sorting algorithms, graph traversals, Dijkstra, string search & Bloom filters
03	03-long-term-engineering	Long-Term Engineering	Runtime inspection & reflection, CTypes & native FFI, performance profiling, and designing for maintainability
04	02-devops-and-cloud-engineering	DevOps, SRE & Cloud (In Development)	Terminal UIs (rich/typer), docker-py, Kubernetes client, Pulumi IaC, OpenTelemetry, Boto3
05	03-network-automation	NetDevOps & Telemetry (In Development)	netmiko, scrapli, napalm, nornir, NETCONF (ncclient), RESTCONF, gNMI, NetBox, pyATS

Part	Slug	Domain / Focus	Key Concepts & Technologies
06	04-data-science-and-analytics	Python Engines & Analytics (In Development)	NumPy, Pandas 2, Polars (LazyFrame), DuckDB OLAP, Apache Arrow, SciPy, Plotly
07	05-artificial-intelligence	ML/Deep Learning & GenAI (In Development)	Scikit-Learn, PyTorch 2.x, Transformers, RAG (Qdrant), Instructor, LangGraph agents
08	06-robotics-and-simulation	Robotics & Physical Systems (In Development)	ROS 2 (rclpy), PyBullet physics, OpenCV vision, ArUco fiducials, A*/RRT, MicroPython
09	07-cybersecurity-and-automation	Information Security Automation (In Development)	Raw sockets, Scapy packet crafting, cryptography (AES/RSA), pefile , SIEM triage
10	08-web-and-distributed-systems	Web APIs & Distributed Systems (In Development)	ASGI, FastAPI, WebSockets, background queues (ARQ/Redis), OAuth2/JWT, SQLAlchemy 2.0
11	09-capstone-systems	Enterprise Capstones (In Development)	Autonomous SRE Agent, NetDevOps Intent Pipeline, Vision-Guided Robotic Telemetry
99	99-appendices	Reference & Appendices	Glossary, syntax cheat sheet, and toolchain reference

Architectural Conventions

- **Modern Tooling Upfront:** Environment bootstrapping, runtime version pinning, and code hygiene are taught with `uv` and `ruff` on day one.
- **Exhaustive Standard Library:** Complete coverage of Python's built-in batteries (`collections`, `itertools`, `pathlib`, `sqlite3`, `subprocess`, `socket`, etc.) without skipping to third-party dependencies prematurely.

- **Original Prose & Complete Files:** Full runnable source files instead of truncated snippets.
- **Progressive Typing:** Pure syntax in initial chapters; formal type annotations (`mypy/pyright`) introduced starting with modern craft chapters.
- **Domain Autonomy:** Domain mathematics, protocol structures, and operating system mechanics are taught directly in-situ.

Formats

Available in HTML, PDF, and EPUB via the monorepo build pipeline.

Python Foundations

Intro

What Is Python

What Is Python

After reading this chapter, you will understand how Python source code compiles to bytecode, executes inside the CPython virtual machine, and leverages the Python 3.14 free-threaded runtime architecture.

Mental model

Python is an interpreted, strongly and dynamically typed language executed by a virtual machine. When you run a script, the standard CPython runtime does not interpret raw human-readable text line-by-line in real time. Instead, it parses source text into an Abstract Syntax Tree (AST), compiles the AST into a sequence of stack-based **bytecode instructions**, and executes those instructions inside an evaluation loop:

[Your Code: hello.py]

Tokenizer & Parser Transforms source text into grammar tokens

AST (Abstract Syntax Tree)

Compiler Emits bytecode instructions & code objects (.pyc)

Bytecode (e.g. LOAD_CONST, BINARY_OP, RETURN_VALUE)

CPython Virtual Machine (ceval.c)

Evaluation Loop PyFrameObject (Value Stack)

Memory Allocator (pymalloc) & Cyclic GC

Python 3.14 Runtime: GIL-less Free-Threading

(PEP 703: multi-core true parallel execution)

[Host OS / Hardware CPU]

Every entity in Python—an integer, a string, a function, or a class—is represented internally as a heap-allocated C structure called a **PyObject**.

Minimal example

Save the following file as `sys_inspect.py`. It inspects your active Python implementation, execution flags, and runtime environment:

```
# sys_inspect.py
import platform
import sys

def main() -> None:
    print(f"Implementation : {platform.python_implementation()}")
    print(f"Python Version : {sys.version.split()[0]}")
    print(f"Compiler       : {platform.python_compiler()}")
    print(f"Platform Arch   : {platform.machine()} ({sys.platform})")
    print(f"Bytecode Magic  : {sys.implementation.cache_tag}")
    print(f"Float Info      : max={sys.float_info.max:.2e}, epsilon={sys.float_info.epsilon:.2e}")

if __name__ == "__main__":
    main()
```

Run this file using `uv`:

```
uv run python sys_inspect.py
```

Output:

```
Implementation : CPython
Python Version : 3.14.0
Compiler       : GCC 14.2.0 (or Clang)
Platform Arch  : x86_64 (linux)
Bytecode Magic : cpython-314
Float Info     : max=1.80e+308, epsilon=2.22e-16
```

Worked examples

1. Peeking inside the CPython compiler with dis

To see what the CPython virtual machine actually executes, you disassemble functions into bytecode using the standard library `dis` module.

Save this file as `disassemble_demo.py`:

```
# disassemble_demo.py
import dis

def compute_total(base_price: float, tax_rate: float) -> float:
    discount = 5.0
    final_price = (base_price - discount) * (1.0 + tax_rate)
    return final_price

def main() -> None:
    print("=== Disassembly of compute_total ===")
    dis.dis(compute_total)

if __name__ == "__main__":
    main()
```

Run the script:

```
uv run python disassemble_demo.py
```

Output:

```
=== Disassembly of compute_total ===
4          0 LOAD_CONST          1 (5.0)
          2 STORE_FAST          2 (discount)

5          4 LOAD_FAST           0 (base_price)
          6 LOAD_FAST           2 (discount)
          8 BINARY_OP          10 (-)
         12 LOAD_CONST          2 (1.0)
         14 LOAD_FAST           1 (tax_rate)
         16 BINARY_OP           0 (+)
         20 BINARY_OP           5 (*)
         24 STORE_FAST          3 (final_price)

6          26 LOAD_FAST          3 (final_price)
         28 RETURN_VALUE
```

Why this matters: CPython is a **stack machine**. `LOAD_FAST` pushes a local variable onto the evaluation stack, `LOAD_CONST` pushes a constant literal, `BINARY_OP` pops the top operands, executes

the arithmetic in C, and pushes the result back onto the stack. Understanding this stack model demystifies performance profiling later in this book.

2. Inspecting runtime execution frames

While your code is executing, CPython maintains a call stack of **frame objects** (`PyFrameObject`). Each frame encapsulates the code object being executed, the local variable array, the global namespace dictionary, and the instruction pointer.

Save this file as `frame_demo.py`:

```
# frame_demo.py
import inspect
import sys

def nested_worker(task_id: int, payload: str) -> None:
    # Inspect the current active execution frame
    current_frame = sys._getframe()
    print(f"Current Function   : {current_frame.f_code.co_name}")
    print(f"Current File       : {current_frame.f_code.co_filename}")
    print(f"Current Line Number: {current_frame.f_lineno}")
    print(f"Local Names           : {list(current_frame.f_locals.keys())}")

    # Inspect the caller's frame
    caller_frame = current_frame.f_back
    if caller_frame:
        print(f"Caller Function   : {caller_frame.f_code.co_name}")
        print(f"Caller Locals     : {caller_frame.f_locals}")

def supervisor() -> None:
    job_batch = "batch-alpha"
    nested_worker(task_id=101, payload=job_batch)

if __name__ == "__main__":
    supervisor()
```

Run the script:

```
uv run python frame_demo.py
```

Output:

```
Current Function   : nested_worker
Current File       : frame_demo.py
Current Line Number: 8
```

```
Local Names      : ['task_id', 'payload', 'current_frame']
Caller Function   : supervisor
Caller Locals     : {'job_batch': 'batch-alpha'}
```

Why this matters: Tracebacks, debuggers (like `pdb`), and profilers are not magic—they walk this linked list of `f_back` frame references.

3. Checking Python 3.14 free-threading (GIL status)

Python 3.14 incorporates the milestone implementation of **PEP 703** (making the Global Interpreter Lock optional). When built with free-threading enabled, Python can run multiple threads across physical CPU cores simultaneously without being serialized by a global mutex.

Save this file as `gil_status.py`:

```
# gil_status.py
import sys

def check_threading_runtime() -> None:
    # sys._is_gil_enabled is available in free-threaded CPython builds
    is_gil_fn = getattr(sys, "_is_gil_enabled", None)

    print("=== Python 3.14 Concurrency Architecture ===")
    if is_gil_fn is not None:
        gil_active = is_gil_fn()
        status = "ENABLED (Serialized threads)" if gil_active else "DISABLED (True multi-cor"
        print(f"Global Interpreter Lock (GIL): {status}")
    else:
        print("Global Interpreter Lock (GIL): Active (Standard CPython build)")

    print(f"Recursion Limit           : {sys.getrecursionlimit()}")
    print(f"Thread Switch Interval        : {sys.getswitchinterval():.4f}s")

if __name__ == "__main__":
    check_threading_runtime()
```

Run the script:

```
uv run python gil_status.py
```

Output:

```
=== Python 3.14 Concurrency Architecture ===
Global Interpreter Lock (GIL): ENABLED (Serialized threads)
Recursion Limit           : 1000
```

```
Thread Switch Interval      : 0.0050s
```

Pitfalls

1. Thinking Python is “just interpreted”

The Trap: Believing Python reads lines as raw strings while executing, and assuming .pyc files are native machine code.

The Reality: Python compiles source code to bytecode first. If the file has not changed, Python reads the bytecode cached in `__pycache__/*.pyc` directly to save startup compilation time. Bytecode is instructions for the CPython VM, not machine code for x86_64 or ARM.

2. Modifying objects during iteration

The Trap: Deleting or inserting items into a list while looping over it.

```
# BROKEN
items = [1, 2, 3, 4, 5]
for x in items:
    if x % 2 == 0:
        items.remove(x) # Skips elements due to shifted indices!
```

The Fix: Iterate over a shallow copy or use a comprehension:

```
# CORRECT
items = [x for x in items if x % 2 != 0]
```

3. Mixing tabs and spaces

The Trap: In Python, indentation defines block scope. Mixing a tab character with spaces causes an immediate `TabError` at the parser level before any code runs. Always configure your editor to insert 4 spaces per indent.

Exercises

1. Modify `sys_inspect.py` to also display `sys.byteorder` (big-endian vs. little-endian) and `sys.api_version`.
 2. Write a script `disassemble_loop.py` that defines a function with a `for` loop, disassembles it with `dis.dis()`, and identifies the opcode responsible for jumping back to the beginning of the loop (`JUMP_BACKWARD` or `FOR_ITER`).
 3. Using `sys.getrefcount()`, write a script `refcount_check.py` that displays the reference count of an empty list when created, when assigned to a second variable name, and after `del` is called on the second variable.
 4. Verify your local Python 3.14 build flags by running `uv run python -c "import sysconfig; print(sysconfig.get_config_vars('Py_DEBUG', 'Py_GIL_DISABLED'))"`.
-

Further reading

- **Official Documentation:** *CPython Internal Design & Architecture Notes* (<https://docs.python.org/3.14>).
- **PEP 703:** *Making the Global Interpreter Lock Optional in CPython* (Sam Gross).
- **PEP 659:** *Specializing Adaptive Interpreter (Faster CPython Project)*.
- **Book:** *CPython Internals: Your Guide to the Python 3 Interpreter* by Anthony Shaw.

Modern Toolchain: uv

The Modern Toolchain with uv

After reading this chapter, you will be able to install and switch Python versions, run isolated scripts with inline dependencies without manual virtual environment management, and manage deterministic projects using `uv`.

Mental model

Historically, Python developers were forced to stitch together 5 to 7 separate tools: `pyenv` to install Python versions, `virtualenv` to isolate packages, `pip` to install wheels, `pip-tools` or `poetry` to resolve dependency locks, and `twine` to publish distributions.

Astral's `uv` replaces this entire fragmented ecosystem with a single, ultra-fast Rust-based binary:

```
[ Legacy Fragmented Toolchain ]
pyenv    virtualenv    pip    pip-tools    flit/twine
(Slow, disparate configs, manual shell activation, fragile PATHs)
```

```
[ Modern Unified Architecture: uv ]
```

The `uv` CLI

Python Engine	Environments	Project Lock
<code>uv python`</code>	<code>uv run`</code>	<code>uv lock`</code>
Installs & pins standalone CPython 3.14 without root permissions.	Ephemerally spins up or links <code>.venv`</code> instantly; auto-runs inline PEP 723 scripts.	Resolves lockfile (<code>uv.lock`</code>) in milliseconds with universal wheel cache.

With `uv`, you no longer need to remember to “activate” a virtual environment in your shell. You prefix commands with `uv run`, and `uv` ensures the exact Python interpreter and locked dependencies are resolved automatically.

Minimal example

Save the following file as `minimal_env.py`. This script inspects the running Python executable and verifies whether it is executing inside an isolated virtual environment:

```
# minimal_env.py
import os
import sys

def main() -> None:
    in_venv = sys.prefix != sys.base_prefix
    print(f"Active Python Executable : {sys.executable}")
    print(f"Isolated Virtualenv       : {in_venv}")
    print(f"Prefix Directory              : {sys.prefix}")
    print(f"Base Interpreter Path           : {sys.base_prefix}")
    print(f"Process PID                     : {os.getpid()}")

if __name__ == "__main__":
    main()
```

Run this script using `uv run`:

```
uv run python minimal_env.py
```

Output:

```
Active Python Executable : /home/user/myproject/.venv/bin/python
Isolated Virtualenv      : True
Prefix Directory         : /home/user/myproject/.venv
Base Interpreter Path     : /home/user/.local/share/uv/python/cpython-3.14.0...
Process PID              : 412850
```

Worked examples

1. Single-file scripts with inline dependencies (PEP 723)

Traditionally, sharing a Python script that required a library like `httpx` or `rich` forced you to write a separate `requirements.txt` file and instruct users to create and activate a virtualenv.

Under **PEP 723** (supported natively by `uv`), you declare your script's dependencies directly inside the file header. `uv` parses this header, provisions an isolated ephemeral cache, and runs the script seamlessly.

Save this file as `weather_fetch.py`:

```
# /// script
# requires-python = ">=3.14"
# dependencies = [
#     "httpx>=0.27.0",
# ]
# ///
# weather_fetch.py
import httpx

def get_current_utc_time() -> None:
    # Query a lightweight, public JSON endpoint
    url = "https://httpbin.org/get"
    headers = {"User-Agent": "ModernPythonGuide/1.0"}

    print(f"Sending HTTP GET request to {url}...")
    with httpx.Client(timeout=5.0) as client:
        response = client.get(url, headers=headers)
        response.raise_for_status()
        data = response.json()

    print(f"HTTP Status: {response.status_code}")
    print(f"Origin IP   : {data.get('origin')}")
    print(f"Headers Sent:")
    for k, v in data.get("headers", {}).items():
        print(f"  {k}: {v}")

if __name__ == "__main__":
    get_current_utc_time()
```

Run the script directly:

```
uv run weather_fetch.py
```

Why this matters: You did not have to `pip install httpx` globally. `uv` read the `# /// script` block, verified whether `httpx` was present in its global content-addressable cache, downloaded it in milliseconds if needed, and executed the script in complete isolation.

2. Managing Python runtimes with `uv python`

You do not need to install Python from your operating system's system package manager (`apt`, `dnf`, or `brew`), which often ships outdated versions or risks breaking system tools.

Inspect available Python versions:

```
uv python list
```

Install Python 3.14 directly into an unprivileged user directory:

```
uv python install 3.14
```

Pin your project directory to use Python 3.14:

```
uv python pin 3.14
```

This creates a `.python-version` file containing:

```
3.14
```

Now, every `uv` command executed in this folder or its subfolders will automatically invoke Python 3.14.

3. Creating and locking a modern project

To start a production project, initialize a standard workspace:

```
uv init my_app  
cd my_app
```

This generates a minimal, standard `pyproject.toml`:

```
[project]  
name = "my-app"  
version = "0.1.0"  
description = "Production application scaffold"  
readme = "README.md"  
requires-python = ">=3.14"  
dependencies = []
```

Add a dependency:

```
uv add "pydantic>=2.10.0"
```

Instantly, `uv` performs dependency resolution, updates `pyproject.toml`, creates a local `.venv/`, and generates a deterministic `uv.lock`.

Save this code in `my_app/main.py`:

```
# main.py
from pydantic import BaseModel, Field

class ServiceNode(BaseModel):
    hostname: str
    ip_address: str
    port: int = Field(default=8080, ge=1, le=65535)
    is_active: bool = True

def main() -> None:
    node = ServiceNode(hostname="edge-router-01", ip_address="192.168.1.1", port=443)
    print(f"Validated Node: {node.hostname} -> {node.ip_address}:{node.port} (active={node.is_active})")
    print(f"JSON Payload : {node.model_dump_json()}")

if __name__ == "__main__":
    main()
```

Run your application:

```
uv run python main.py
```

Output:

```
Validated Node: edge-router-01 -> 192.168.1.1:443 (active=True)
JSON Payload : {"hostname":"edge-router-01","ip_address":"192.168.1.1","port":443,"is_active":true}
```

Pitfalls

1. Manually activating virtualenvs in every terminal tab

The Trap: Running `source .venv/bin/activate`, opening a new terminal tab, forgetting to activate, and running against system Python.

The Fix: Never rely on shell prompt activation. Use `uv run python script.py` or configure your editor to point directly to `.venv/bin/python`. `uv run` guarantees the correct environment is active every time.

2. Hand-editing `uv.lock`

The Trap: Manually editing versions or hashes in `uv.lock`.

The Fix: Treat `uv.lock` like a compiled binary. Modify requirements in `pyproject.toml` or run `uv add` / `uv remove`, and let `uv lock` regenerate the lockfile. Commit `uv.lock` to Git so all team members and CI runners build identical environments.

3. Polluting the system Python with `sudo pip`

The Trap: Running `sudo pip install <package>`, which corrupts Linux distribution packages (`/usr/lib/python3/dist-packages`) and can break system utilities.

The Fix: Modern Linux distros trigger PEP 668 (`externally-managed-environment`) error. Use `uv tool install <cli>` for global CLI binaries (e.g. `uv tool install ruff`), and `uv venv` or `uv run` for code dependencies.

Exercises

1. Run `uv python list` in your terminal and determine which CPython versions are currently installed on your workstation.
2. Create a single-file script `hash_tool.py` using PEP 723 inline script metadata that depends on `cryptography>=44.0.0` and prints the version of the installed cryptography library using `uv run hash_tool.py`.
3. In an empty directory, initialize a project with `uv init`, add `pytest` as a development dependency (`uv add --dev pytest`), and verify that `pyproject.toml` differentiates runtime dependencies from `[dependency-groups] dev`.
4. Inspect the generated `uv.lock` file in your text editor. Locate the exact wheel URL and SHA-256 hash that `uv` locked for your dependency.

Further reading

- **Official Documentation:** *uv — An extremely fast Python package and project manager* (<https://docs.astral.sh/uv/>).
- **PEP 723:** *Inline Script Metadata* (Brett Cannon, Ofek Lev).
- **PEP 621:** *Storing project metadata in `pyproject.toml`*.
- **PEP 668:** *Marking Python base environments as “externally managed”*.

Code Hygiene: Ruff

Developer Environment & Code Hygiene with Ruff

After reading this chapter, you will be able to configure a professional editor with the Language Server Protocol (LSP), enforce automated code formatting and linting with **ruff** in milliseconds, and integrate pre-commit quality gates into your git repositories.

Mental model

Writing clean, production-grade Python requires two layers of automated feedback: 1. **Real-time feedback**: Your editor (Zed, VS Code, or Neovim) communicates over the Language Server Protocol (LSP) with an LSP server (such as **basedpyright** or **pyright**) to show type errors and syntax diagnostics as you type. 2. **Deterministic enforcement**: A formatter and linter ensures that every file in your repository conforms to standard formatting rules (line lengths, import sorting, obsolete syntax upgrades, and common bug detection).

Astral's **ruff** combines the responsibilities of Black, Flake8, isort, pydocstyle, pyupgrade, and bandit into a single Rust binary that operates 10 to 100 times faster than legacy Python-based linters:

```
[ Developer in Editor: Zed / VS Code / Neovim ]
```

```
Language Server Protocol (LSP)
(Shows real-time parameter hints, completions, & errors)
```

```
Save file (Ctrl+S)
```

```
Ruff Engine (Native Rust Binary)
```

```
`ruff format`
Replaces Black /
yapf. Enforces 4
spaces, quote styles.
```

```
`ruff check --fix`
Replaces Flake8,
isort, pyupgrade,
bandit. Fixes bugs.
```

```
Passes clean code
```


Git Pre-Commit Hook & Automated CI Pipeline
(Rejects malformed PRs before code reaches production)

Minimal example

To understand what a linter does, look at this intentionally unformatted and messy script.

Save this file as `messy_sample.py`:

```
# messy_sample.py
import sys, os
from math import *
import json

def calculate_average( numbers:list ) -> float:
    total=0.0
    for n in numbers:
        total = total+n
    return total / len(numbers) if len(numbers)>0 else 0.0

def main():
    vals=[10, 20, 30, 40]
    print(f"Average: {calculate_average(vals)}")

if __name__ == "__main__":
    main()
```

Notice the issues in this file: 1. Unused imports (`sys`, `os`, `json`). 2. Wildcard import (`from math import *`) which pollutes the namespace. 3. Inconsistent spacing inside parentheses and around operators. 4. Outdated type annotation syntax (`list` instead of `list[float]`).

Run the script to verify it still works:

```
uv run python messy_sample.py
```

Now, check the file using `ruff`:

```
uv run ruff check messy_sample.py
```

Ruff flags the unused imports and wildcard import immediately:

```
messy_sample.py:2:8: F401 `sys` imported but unused
messy_sample.py:2:13: F401 `os` imported but unused
```

```
messy_sample.py:3:1: F403 `from math import *` used; unable to detect undefined names
messy_sample.py:4:8: F401 `json` imported but unused
```

Now, format the file using `ruff format`:

```
uv run ruff format messy_sample.py
```

Ruff normalizes all whitespace, operator spacing, and indentation in under 2 milliseconds.

Worked examples

1. Configuring ruff in `pyproject.toml`

Rather than relying on ad-hoc CLI flags, professional projects configure linter and formatter rules declaratively inside `pyproject.toml`.

Here is the standard, production-grade `[tool.ruff]` configuration for modern Python 3.14 projects:

```
# pyproject.toml
[project]
name = "production-service"
version = "0.1.0"
requires-python = ">=3.14"

[tool.ruff]
target-version = "py314"
line-length = 100
src = ["src", "tests"]

[tool.ruff.lint]
# Selected rule suites:
# E / W : Pycodestyle (standard PEP 8 formatting errors & warnings)
# F      : Pyflakes (detects undefined names, unused variables & imports)
# I      : isort (automatic alphabetical import sorting)
# UP     : pyupgrade (upgrades syntax to modern Python 3.14 idioms)
# B      : flake8-bugbear (detects common bugs and design pitfalls)
# SIM    : flake8-simplify (suggests cleaner, idiomatic syntax)
select = ["E", "W", "F", "I", "UP", "B", "SIM"]

ignore = [
    "E501", # Line length handled automatically by ruff format
]
```

```
[tool.ruff.lint.isort]
combine-as-imports = true
force-single-line = false

[tool.ruff.format]
quote-style = "double"
indent-style = "space"
skip-magic-trailing-comma = false
line-ending = "auto"
```

Why this matters: When anyone on your team runs `uv run ruff check` or saves a file in their editor, the same exact rules are applied deterministically across Linux, macOS, and Windows.

2. How linters work: building an AST inspector

Linters do not execute your code; they inspect its **Abstract Syntax Tree (AST)**. You can use Python's built-in `ast` module to write your own custom code inspector.

Save this file as `ast_security_lint.py`:

```
# ast_security_lint.py
import ast
import sys

# Target code string to inspect
CODE_SAMPLE = """
import os

def load_user_input(user_str: str) -> None:
    # Danger: eval() allows arbitrary code execution!
    result = eval(user_str)
    print("Result:", result)
"""

class SecurityASTVisitor(ast.NodeVisitor):
    def __init__(self) -> None:
        self.violations: list[str] = []

    def visit_Call(self, node: ast.Call) -> None:
        # Check if the function being called is eval() or exec()
        if isinstance(node.func, ast.Name) and node.func.id in {"eval", "exec"}:
            self.violations.append(
                f"Line {node.lineno}: Dangerous call to '{node.func.id}()' detected!"
            )
        # Continue traversing child AST nodes
```

```

        self.generic_visit(node)

def main() -> None:
    print("Parsing source code into Abstract Syntax Tree...")
    parsed_tree = ast.parse(CODE_SAMPLE)

    visitor = SecurityASTVisitor()
    visitor.visit(parsed_tree)

    if visitor.violations:
        print("Security Linter Findings:")
        for v in visitor.violations:
            print(f"  [CRITICAL] {v}")
    else:
        print("Code passed all AST checks.")

if __name__ == "__main__":
    main()

```

Run the script:

```
uv run python ast_security_lint.py
```

Output:

```

Parsing source code into Abstract Syntax Tree...
Security Linter Findings:
  [CRITICAL] Line 6: Dangerous call to 'eval()' detected!

```

Why this matters: ruff performs this exact AST traversal, but does so in compiled Rust across thousands of files per second.

3. Automated Git pre-commit hooks

To ensure no developer can commit malformed code, configure a pre-commit hook using `.pre-commit-config.yaml`.

Save this file in the root of your repository as `.pre-commit-config.yaml`:

```

# .pre-commit-config.yaml
repos:
- repo: https://github.com/astral-sh/ruff-pre-commit
  rev: v0.9.9
  hooks:
    # Run the linter

```

```
- id: ruff
  args: [--fix]
# Run the formatter
- id: ruff-format
```

Install the pre-commit hook into `.git/hooks/`:

```
uv run pre-commit install
```

Now, whenever you run `git commit`, `ruff` runs automatically before the commit is created. If formatting is needed, `ruff` fixes the file and halts the commit so you can stage the clean changes.

Pitfalls

1. Running multiple competing formatters

The Trap: Installing Black, autopep8, and Ruff simultaneously in your editor, resulting in editor stuttering and formatting wars.

The Fix: Disable all legacy formatters. Use `ruff` exclusively for both linting and formatting. Ruff was explicitly engineered to be a drop-in replacement for Black and isort.

2. Blindly running `ruff check --fix` without review

The Trap: Running automated fix flags on large repositories and immediately pushing to `main` without testing.

The Fix: Some lint fixes (like removing unused variables or modifying comprehensions) can change runtime semantics if code relied on side effects. Always run your test suite (`uv run pytest`) after applying automated fixes.

3. Committing code without running formatting in CI

The Trap: Formatting locally, but forgetting that pull requests submitted by contributors might not have pre-commit hooks installed.

The Fix: Add a GitHub Actions workflow step: `uv run ruff check --no-fix` and `uv run ruff format --check`. If any file is unformatted, CI rejects the build.

Exercises

1. Create a script with deliberate whitespace and import errors, then use `uv run ruff format --diff <filename>` to inspect the unified diff output before writing changes to disk.
 2. In your `pyproject.toml`, add rule B006 (flake8-bugbear: mutable default arguments in functions). Write a function `def append_to(item, target=[])` and observe how `ruff check` catches the trap.
 3. Configure your primary code editor (VS Code, Zed, or Neovim) to automatically run `ruff format` on file save. Verify that saving a file instantly formats it without lag.
 4. Modify `ast_security_lint.py` to also detect calls to `os.system()` and raise a warning advising the use of `subprocess.run()`.
-

Further reading

- **Official Documentation:** *Ruff — An extremely fast Python linter and code formatter* (<https://docs.astral.sh/ruff/>).
- **PEP 8:** *Style Guide for Python Code* (Guido van Rossum, Barry Warsaw, Nick Coghlan).
- **PEP 518:** *Specifying Minimum Build System Requirements for Python Projects* (`pyproject.toml`).

Editor Setup: VS Code

Editor Setup: VS Code with Modern Tooling

After reading this chapter, you will be able to configure Visual Studio Code into a high-performance Python development environment—integrating `uv` virtual environments, the native `Ruff` extension for instantaneous format-on-save and auto-import sorting, static type analysis, and reproducible `.vscode/` workspace configurations.

Mental model

VS Code acts as an orchestration shell around three specialized background engines: the **Language Server** (syntax, autocompletion, and type analysis), the **Linters & Formatter** (`Ruff`), and the **Runtime Debugger** (`debugpy`).

Rather than relying on global editor preferences that vary across machines, professional projects define their toolchain declaratively inside a version-controlled `.vscode/` folder within the project repository:

[Your VS Code Editor]

```
Workspace Configuration: `.vscode/settings.json`
  Points Python extension to `${workspaceFolder}/.venv`
  Sets default formatter to `charliermarsh.ruff`

Recommended Extensions: `.vscode/extensions.json`
  `charliermarsh.ruff` (Instant Rust-based format & lint)
  `ms-python.python` (Core Python language support)
  `tamasfe.even-better-toml` (Syntax for pyproject.toml)

Debug Configurations: `.vscode/launch.json`
  Launches Python scripts inside the `uv` virtualenv
```

When you save a file (`Ctrl+S` / `Cmd+S`), the `Ruff` extension intercepts the write, reorders imports alphabetically, upgrades obsolete syntax, normalizes whitespace, and returns the formatted buffer in less than 5 milliseconds.

Minimal example

To configure any project workspace for modern Python development, create a folder named `.vscode` in the root of your project directory and add `settings.json`.

Save this file as `.vscode/settings.json`:

```
{
  // 1. Interpreter & Virtual Environment Discovery
  "python.defaultInterpreterPath": "${workspaceFolder}/.venv/bin/python",
  "python.terminal.activateEnvironment": true,

  // 2. Formatting Engine: Delegate completely to Ruff
  "editor.formatOnSave": true,
  "[python]": {
    "editor.defaultFormatter": "charliermarsh.ruff",
    "editor.formatOnSave": true,
    "editor.codeActionsOnSave": {
      "source.fixAll.ruff": "explicit",
      "source.organizeImports.ruff": "explicit"
    }
  },

  // 3. Static Type Analysis via Pylance / Pyright
  "python.analysis.typeCheckingMode": "standard",
  "python.analysis.autoImportCompletions": true,
  "python.analysis.inlayHints.variableTypes": false,
  "python.analysis.inlayHints.functionReturnTypes": true,

  // 4. Test Explorer Integration with Pytest
  "python.testing.pytestEnabled": true,
  "python.testing.unittestEnabled": false,
  "python.testing.pytestArgs": ["tests"],

  // 5. Files & Explorer Hygiene
  "files.exclude": {
    "**/__pycache__": true,
    "**/.pytest_cache": true,
    "**/.ruff_cache": true
  }
}
```

Now, whenever you open this project in VS Code, all formatting, import sorting, lint fixes, and virtual environment bindings happen automatically on save.

Worked examples

1. Workspace extension recommendations (.vscode/extensions.json)

When collaborating in teams or across machines, you should prompt VS Code to install the exact toolchain required for the project.

Save this file as `.vscode/extensions.json`:

```
{
  "recommendations": [
    // Fast Rust-based linter and formatter (replaces Black, Flake8, isort)
    "charliermarsh.ruff",
    // Official Microsoft Python extension
    "ms-python.python",
    // High-performance language server and static type checking
    "ms-python.vscode-pylance",
    // Rich syntax highlighting for pyproject.toml and uv.lock
    "tamasfe.even-better-toml"
  ]
}
```

Why this matters: When a new contributor opens the repository, VS Code displays a notification: *“This repository recommends installing extensions.”* Clicking **Install All** equips their workspace with the exact linter, formatter, and type checker without manual search.

2. Interactive debugging with .vscode/launch.json

VS Code can execute your scripts step-by-step, pause on breakpoints, inspect local variables, and evaluate expressions live in the Debug Console.

Save this file as `.vscode/launch.json`:

```
{
  "version": "0.2.0",
  "configurations": [
    {
      "name": "Python: Current File",
      "type": "debugpy",
      "request": "launch",
      "program": "${file}",
      "console": "integratedTerminal",
      "justMyCode": true,
      "env": {
        "PYTHONUNBUFFERED": "1"
      }
    }
  ]
}
```

```

    }
  },
  {
    "name": "Python: Run CLI with Arguments",
    "type": "debugpy",
    "request": "launch",
    "program": "${workspaceFolder}/src/main.py",
    "args": ["--config", "config.toml", "--verbose"],
    "console": "integratedTerminal",
    "justMyCode": true
  },
  {
    "name": "Python: Debug Pytest",
    "type": "debugpy",
    "request": "launch",
    "module": "pytest",
    "args": ["-v", "${file}"],
    "console": "integratedTerminal",
    "justMyCode": false
  }
]
}

```

To test the debugger: 1. Open any Python file. 2. Click to the left of a line number to set a red **breakpoint** dot. 3. Press F5 (or select **Run** → **Start Debugging**). 4. Execution pauses immediately at your breakpoint, populating the **Variables** and **Call Stack** panels.

3. Testing format-on-save and auto-import sorting

To verify that your VS Code workspace is operating correctly, create a test script with intentional formatting defects.

Save this file as `editor_verify.py`:

```

# editor_verify.py
import sys
import os
import json
from math import sqrt, pi, sin

def compute_circle_metrics(radius: float) -> dict[str, float]:
    area = pi * (radius ** 2)
    circumference = 2 * pi * radius
    return {"radius": radius, "area": area, "circumference": circumference}

```

```
def main() -> None:
    metrics = compute_circle_metrics(5.0)
    print(f"Metrics: {metrics}")

if __name__ == "__main__":
    main()
```

Notice: - `sys`, `os`, `json`, `sqrt`, and `sin` are unused imports. - Now press `Ctrl+S` (or `Cmd+S` on macOS).

What happens automatically: 1. Ruff removes the unused imports. 2. Ruff re-formats `from math import pi` onto a clean single line. 3. Ruff normalizes operators and quote styles according to your `pyproject.toml`.

The file is saved instantly in its clean canonical form.

Pitfalls

1. Conflicting legacy extensions

The Trap: Leaving legacy extensions like `ms-python.black-formatter`, `ms-python.isort`, or `ms-python.flake8` enabled alongside `charliermarsh.ruff`.

The Reality: Two formatters will attempt to format the document simultaneously on save, causing visible cursor jumping, undo-stack corruption, and editor latency.

The Fix: Uninstall or disable Black, isort, and Flake8 extensions. Ruff handles all three functionalities natively.

2. VS Code selecting the wrong Python interpreter

The Trap: VS Code defaulting to global system Python (`/usr/bin/python3`) instead of the project's local virtual environment (`.venv/bin/python`).

The Fix: Press `Ctrl+Shift+P` (or `Cmd+Shift+P`), type **Python: Select Interpreter**, and choose the interpreter located in your project's `./.venv` folder. Specifying `"python.defaultInterpreterPath": "${workspaceFolder}/.venv/bin/python"` in `.vscode/settings.json` makes this automatic for anyone opening the folder.

3. Expecting editor tools to run in external shells

The Trap: Assuming that because code formats on save in VS Code, team members using terminal commands or git commits will follow the same rules.

The Fix: Always combine editor configurations with command-line validation. Use `uv run ruff check` and `uv run ruff format --check` in your CI pipeline, and enforce `.pre-commit-config.yaml` git hooks so standards are upheld everywhere.

Exercises

1. Create a `.vscode/settings.json` file in a project workspace and verify that enabling `"python.analysis.inlayHints.functionReturnTypes": true` displays inferred return types for unannotated functions in your editor.
 2. Open the **Run and Debug** view (**Ctrl+Shift+D**), create a breakpoint on a function inside `editor_verify.py`, press **F5**, and use the **Debug Console** to evaluate expressions dynamically while paused.
 3. Open the **Testing** panel (flask icon) in the VS Code sidebar. Write a simple test function `test_math(): assert 1 + 1 == 2` in a `test_demo.py` file, and verify that VS Code discovers and displays a green play button next to the test.
 4. Modify your `.vscode/settings.json` to configure rulers at 88 and 100 characters by adding `"editor.rulers": [88, 100]` to visualize line length boundaries.
-

Further reading

- **VS Code Python Documentation:** *Getting Started with Python in VS Code* (<https://code.visualstudio.com/docs/languages/python>).
- **Ruff VS Code Extension:** *Official Ruff extension on VS Code Marketplace* (<https://marketplace.visualstudio.com/items?itemName=charliermarsh/ruff>).
- **Pylance Documentation:** *Fast, feature-rich Python language support in VS Code* (<https://marketplace.visualstudio.com/items?itemName=ms-python.vscode-pylance>).
- **Astral Documentation:** *Integrating uv with VS Code* (<https://docs.astral.sh/uv/guides/integration>).

Variables

Names, Binding, and Object References

Names, Binding, and Object References

After reading this chapter, you will understand how Python binds identifiers to heap-allocated objects, how assignment actually works in CPython, and how to inspect memory references with confidence.

Mental model

In compiled languages like C or Go, a variable is a named memory slot with a fixed size and type. In Python, **variables are not boxes holding values**; they are **name tags (pointers) bound to objects** living on the heap.

Stack / Namespace (Frame)	Heap Memory (PyObject)
name: "port"	PyLongObject (value: 8080) ob_refcnt: 2 ob_type: <class 'int'>
name: "service_port"	
name: "backup_port"	PyLongObject (value: 9000) ob_refcnt: 1

When you execute `port = 8080`, Python creates a `PyLongObject` representing 8080 in heap memory, and binds the string key "port" in the current namespace dictionary to that object's memory address. When you execute `service_port = port`, no integer is copied; both identifiers simply reference the exact same memory address.

Minimal example

Save the following file as `binding_demo.py` and run it via `uv run python binding_demo.py`:

```
# binding_demo.py
def main() -> None:
```



```

# 1. Bind name "a" to an integer object
a = 1000
# 2. Bind name "b" to the same object "a" points to
b = a

print(f"a = {a}, id(a) = {hex(id(a))}")
print(f"b = {b}, id(b) = {hex(id(b))}")
print(f"a is b: {a is b}")

# 3. Rebind "a" to a completely new integer object
a = 2000
print("\nAfter rebinding a = 2000:")
print(f"a = {a}, id(a) = {hex(id(a))}")
print(f"b = {b}, id(b) = {hex(id(b))}")
print(f"a is b: {a is b}")

if __name__ == "__main__":
    main()

```

Output:

```

a = 1000, id(a) = 0x7f884120
b = 1000, id(b) = 0x7f884120
a is b: True

```

```

After rebinding a = 2000:
a = 2000, id(a) = 0x7f884140
b = 1000, id(b) = 0x7f884120
a is b: False

```

Rebinding `a` did not mutate the number 1000. It simply changed what `a` points to.

Worked examples

Case 1: Inspecting the namespace dictionary

Python implements namespaces as standard hash tables (dictionaries). You can inspect them directly using `locals()` and `globals()`.

```

# namespace_inspect.py
def inspect_scope() -> None:
    region = "us-east-1"
    replicas = 3

```

```

scope = locals()
print("Local namespace keys:", [k for k in scope if not k.startswith("_")])
print(f"Value of region: {scope['region']}")
print(f"Value of replicas: {scope['replicas']}")

if __name__ == "__main__":
    inspect_scope()

```

Run:

```
uv run python namespace_inspect.py
```

Output:

```

Local namespace keys: ['region', 'replicas']
Value of region: us-east-1
Value of replicas: 3

```

Why: Every variable lookup in Python begins by searching these internal namespace mapping tables.

Case 2: Binding vs in-place mutation

Because names are references, mutating an object in place affects all names pointing to it. Rebinding a name does not.

```

# mutate_vs_rebind.py
def demonstrate() -> None:
    # Scenario A: In-place mutation
    list_x = [1, 2, 3]
    list_y = list_x
    list_x.append(4)
    print("Scenario A (Mutation):")
    print(f"list_x: {list_x}")
    print(f"list_y: {list_y} (affected because both share the same reference!)")

    # Scenario B: Rebinding
    list_x = [10, 20]
    print("\nScenario B (Rebinding list_x):")
    print(f"list_x: {list_x}")
    print(f"list_y: {list_y} (unaffected because list_x was rebound to a new object)")

if __name__ == "__main__":
    demonstrate()

```

Run:

```
uv run python mutate_vs_rebind.py
```

Output:

Scenario A (Mutation):

list_x: [1, 2, 3, 4]

list_y: [1, 2, 3, 4] (affected because both share the same reference!)

Scenario B (Rebinding list_x):

list_x: [10, 20]

list_y: [1, 2, 3, 4] (unaffected because list_x was rebound to a new object)

Why: `list_x.append()` alters the existing object in heap memory. `list_x = [10, 20]` creates a new list and updates the pointer in `list_x`.

Pitfalls

Pitfall 1: Expecting assignment to copy compound objects

```
# Dangerous:
original = {"host": "db.internal", "port": 5432}
copy_config = original
copy_config["port"] = 5433 # Also alters original["port"]!

# Correct fix:
copy_config = original.copy() # Shallow copy for flat dictionaries
```

Pitfall 2: Attempting to modify immutable objects

Numbers, strings, and tuples are immutable. When you write `s = "hello"; s += " world"`, Python does not extend "hello" in place; it allocates a brand-new string "hello world" and rebinds `s`.

Exercises

1. Create a script where two variables `p` and `q` point to the same empty list. Append an integer using `p`. Print `q` and verify that `id(p) == id(q)`.
 2. Create a script with an integer `x = 500`. Print its `id()`. Multiply `x` by 2 (`x *= 2`). Print `id(x)` again. Why did the memory address change?
 3. Write a function `swap_names()` that demonstrates swapping two variable references using tuple packing: `a, b = b, a`. Verify that their memory identities switch places.
 4. Using `sys.getrefcount()`, observe the reference count of a newly created custom list before and after creating two aliases to it.
-

Further reading

- Python Documentation: *Execution Model and Naming* (Docs -> Reference -> Execution Model).
- PEP 3104: *Access to Names in Outer Scopes*.
- Ned Batchelder: *Facts and Myths about Python names and values*.

Identity, Equality, and Mutability

Identity, Equality, and Mutability

After reading this chapter, you will master the fundamental distinction between identity (**is**) and equality (**==**), know how CPython caches small integers and strings, and avoid shared mutable state bugs.

Mental model

In Python, every object has three core attributes: 1. **Identity**: The memory address where the object resides (`id(obj)`). 2. **Type**: What kind of object it is (e.g. `int`, `str`, `list`), which never changes. 3. **Value**: The data stored in the object.

Equality (`==`) compares VALUES

"production" "production"

Object A	Object B
id: 0x104a	id: 0x208f

Identity (`is`) compares MEMORY IDs
(Are they the exact same object?)

- `a == b`: Evaluates `a.__eq__(b)`. Asks: **Do these two objects have equivalent contents?**
- `a is b`: Evaluates `id(a) == id(b)`. Asks: **Are these two variables pointing to the exact same memory address?**

Minimal example

Save this file as `identity_demo.py`:

```
# identity_demo.py
def main() -> None:
    # Two distinct list objects with identical contents
```

```

list_a = [10, 20, 30]
list_b = [10, 20, 30]

print(f"list_a == list_b : {list_a == list_b} (Values match)")
print(f"list_a is list_b : {list_a is list_b} (Different objects in memory)")
print(f"id(list_a)          : {hex(id(list_a))}")
print(f"id(list_b)          : {hex(id(list_b))}")

# The singleton None must ALWAYS be checked with 'is'
status = None
print(f"status is None     : {status is None}")

if __name__ == "__main__":
    main()

```

Run via `uv run python identity_demo.py`:

```

list_a == list_b : True (Values match)
list_a is list_b : False (Different objects in memory)
id(list_a)       : 0x7f5110a0
id(list_b)       : 0x7f511120
status is None   : True

```

Worked examples

Case 1: The small integer caching mechanism

CPython pre-allocates and caches small integers in the range `[-5, 256]` during startup. Any reference to an integer in this range shares the pre-allocated singleton instance.

```

# int_cache.py
def test_caching() -> None:
    # Inside the cache range [-5, 256]
    x = 250
    y = 250
    print(f"250 is 250: {x is y} (Shared singleton from CPython cache)")

    # Outside the cache range (typically > 256)
    big1 = 1000
    big2 = 1000
    print(f"1000 is 1000: {big1 is big2} (Distinct heap allocations)")
    print(f"1000 == 1000: {big1 == big2} (Values are still equal)")

if __name__ == "__main__":

```

```
test_caching()
```

Run:

```
uv run python int_cache.py
```

Output:

```
250 is 250: True (Shared singleton from CPython cache)
1000 is 1000: False (Distinct heap allocations)
1000 == 1000: True (Values are still equal)
```

Why: Never rely on `is` for integer or string equality. Always use `==` for values.

Case 2: Shallow copy vs Deep copy

When copying nested mutable collections, a shallow copy copies references to inner objects, while a deep copy recursively duplicates everything.

```
# copy_semantics.py
import copy

def main() -> None:
    original = {"cluster": "prod", "nodes": ["node-1", "node-2"]}

    # Shallow copy
    shallow = original.copy()
    # Deep copy
    deep = copy.deepcopy(original)

    # Mutate the nested list
    original["nodes"].append("node-3")

    print(f"Original nodes: {original['nodes']}")
    print(f"Shallow nodes : {shallow['nodes']} (Mutated because inner list reference was shared)")
    print(f"Deep nodes    : {deep['nodes']} (Isolated because inner list was cloned)")

if __name__ == "__main__":
    main()
```

Run:

```
uv run python copy_semantics.py
```


Pitfalls

Pitfall 1: Comparing values using `is`

```
# Bug: Works sometimes due to caching, fails unexpectedly in production!
if user_input is "admin": # WRONG! Raises SyntaxWarning in modern Python
    pass

# Correct:
if user_input == "admin": # Correct value equality check
    pass
```

Pitfall 2: Mutating objects inside tuples

A `tuple` is immutable, meaning its reference slots cannot be changed. However, if a slot points to a mutable object (like a `list`), that list can still be mutated in place!

```
t = (1, [2, 3])
t[1].append(4) # Perfectly valid: the list inside the tuple is modified!
print(t)      # (1, [2, 3, 4])
```

Exercises

1. Write a script comparing two identical strings constructed at runtime (`s1 = "hello_world"` vs `s2 = "".join(["hello", "_", "world"])`). Compare them with `is` and with `==`.
 2. Inspect `id(None)` across multiple variables set to `None`. Verify they all resolve to the exact same pointer address.
 3. Construct a dictionary with a nested dictionary. Perform both a shallow copy and a `copy.deepcopy()`. Demonstrate modifying a nested value in the original and check both copies.
-

Further reading

- CPython Internal Source: `Objects/longobject.c` (small integer cache definition).
- Python Documentation: `copy` module (*Shallow and deep copy operations*).
- PEP 8: *Programming Recommendations (Comparisons to singletons like None)*.

Naming Conventions, Constants, and Unpacking

Naming Conventions, Constants, and Unpacking

After reading this chapter, you will write clean, idiomatic PEP 8 Python code, leverage sequence unpacking, and understand how constants are declared and enforced.

Mental model

Python does not have a language-level `const` keyword. Instead, constant conventions are enforced through **naming standards** and static analysis tools like `ruff` and `mypy`.

Naming Categories:

Pattern	Convention	Example
snake_case	Functions, variables, files	db_connection_timeout
UPPER_SNAKE_CASE	Constants, globals	MAX_RETRIES, HTTP_OK
PascalCase (CapWords)	Classes, Exception types	DatabasePool, APIError
_leading_underscore	Internal / Private marker	_internal_buffer
__dunder__	Special Python protocols	__init__, __repr__

Unpacking allows you to extract elements from an iterable directly into named variables in a single atomic statement without manual index indexing.

Minimal example

Save as `unpacking_demo.py`:

```
# unpacking_demo.py
def main() -> None:
    # Atomic variable swap without temporary variable
    left = "left_hand"
    right = "right_hand"
    left, right = right, left
    print(f"Swapped: left={left}, right={right}")
```

```

# Extended star unpacking
endpoint_data = ("GET", "/api/v1/users", 200, "0.12ms", "192.168.1.10")
method, path, status, *telemetry = endpoint_data

print(f"Method      : {method}")
print(f"Path        : {path}")
print(f"Status       : {status}")
print(f"Telemetry    : {telemetry} (packed into a list)")

if __name__ == "__main__":
    main()

```

Run via `uv run python unpacking_demo.py`:

```

Swapped: left=right_hand, right=left_hand
Method      : GET
Path        : /api/v1/users
Status      : 200
Telemetry    : ['0.12ms', '192.168.1.10'] (packed into a list)

```

Worked examples

Case 1: Star unpacking in loops and data processing

You can cleanly separate headers, body items, and footers from structured data stream records:

```

# process_records.py
def process_stream() -> None:
    records = [
        ("auth", "admin", "success", "10.0.0.1"),
        ("query", "select * from users", "success", "10.0.0.2"),
        ("error", "timeout on connection", "failed", "10.0.0.3"),
    ]

    for event_type, *details, ip in records:
        print(f"Event: {event_type:8s} | IP: {ip:12s} | Details: {' '.join(details)}")

if __name__ == "__main__":
    process_stream()

```

Run:

```
uv run python process_records.py
```

Case 2: Marking constant intent with `typing.Final`

While CPython allows rebinding uppercase variables, modern type checkers will flag modifications to variables marked with `Final`:

```
# constants_guard.py
from typing import Final

DATABASE_TIMEOUT_SECONDS: Final[int] = 30
MAX_CONNECTIONS: Final[int] = 100

def check_limits(requested: int) -> bool:
    return requested <= MAX_CONNECTIONS

if __name__ == "__main__":
    print(f"Timeout: {DATABASE_TIMEOUT_SECONDS}s")
    print(f"Acceptable: {check_limits(50)}")
```

Run:

```
uv run python constants_guard.py
```

Pitfalls

Pitfall 1: Unbalanced unpacking assignments

```
coords = (10, 20, 30)
x, y = coords # ValueError: too many values to unpack (expected 2, got 3)

# Fix: Match length or use star expression:
x, y, _ = coords # Ignore the 3rd value using dummy variable "_"
x, *rest = coords # Capture remaining values
```

Pitfall 2: Overriding built-in functions with variable names

Never name variables after built-in functions like `id`, `type`, `list`, `str`, `dict`, or `min`:

```
# Dangerous:
list = [1, 2, 3] # Now the built-in "list()" constructor is masked in this scope!
```

Exercises

1. Write a script that unpacks a 5-element tuple into `first`, `middle` (a list of 3 elements using `*`), and `last`.
 2. Write a script that unpacks nested coordinates: `entry = ("datacenter-east", (40.7128, -74.0060))`. Extract the name and latitude/longitude in a single unpacking line.
 3. Use `ruff check` on a file containing non-PEP8 variable names (like `camelCaseVariable = 10`) to see how modern linters catch naming deviations.
-

Further reading

- PEP 8: *Style Guide for Python Code* (Naming Conventions section).
- PEP 3132: *Extended Iterable Unpacking*.
- Python Standard Library: `typing.Final`.

Variable Scopes, LEGB, and Object Lifetimes

Variable Scopes, LEGB, and Object Lifetimes

After reading this chapter, you will master Python's LEGB scope lookup order, control variable rebinding across scopes using `global` and `nonlocal`, and understand reference counting lifecycle management.

Mental model

Whenever Python resolves an identifier name in code, it searches four nested lexical scopes in strict order (**LEGB**):

Built-in Scope (`len`, `range`, `print`, `Exception`, `id`)

Global / Module Scope (top-level functions, vars)

Enclosing / Outer Scope (closures, outer def)

Local Scope (inside the active function)

Search starts here: [1]

Falls back to: [2]

Falls back to: [3]

Falls back to: [4]

If the name is not found in any of the four tiers, Python raises `NameError`.

Minimal example

Save as `legb_demo.py`:

```
# legb_demo.py
# 1. Global scope
```



```

app_env = "production"

def outer_service() -> None:
    # 2. Enclosing scope
    service_name = "auth_api"

    def inner_handler() -> None:
        # 3. Local scope
        request_id = "req-9481"
        print(f"Local      : {request_id}")
        print(f"Enclosing : {service_name}")
        print(f"Global    : {app_env}")
        print(f"Built-in   : {len(request_id)}") # "len" resolved from built-in scope

    inner_handler()

if __name__ == "__main__":
    outer_service()

```

Run via `uv run python legb_demo.py`:

```

Local      : req-9481
Enclosing  : auth_api
Global     : production
Built-in   : 8

```

Worked examples

Case 1: Mutating enclosing state with nonlocal

Without `nonlocal`, attempting to assign to an enclosing variable creates a new local variable instead. `nonlocal` binds the assignment directly to the enclosing frame:

```

# stateful_counter.py
from collections.abc import Callable

def make_counter(start: int = 0) -> Callable[[], int]:
    count = start

    def increment() -> int:
        nonlocal count # Rebinds the enclosing 'count' variable
        count += 1
        return count

```

```

        return increment

if __name__ == "__main__":
    counter = make_counter(start=10)
    print("Call 1:", counter())
    print("Call 2:", counter())
    print("Call 3:", counter())

```

Run:

```
uv run python stateful_counter.py
```

Output:

```

Call 1: 11
Call 2: 12
Call 3: 13

```

Case 2: Object lifecycle and reference counting

Python reclaims heap memory as soon as an object's reference count drops to zero (pymalloc).

```

# lifecycle_demo.py
import sys

class Resource:
    def __init__(self, name: str) -> None:
        self.name = name
        print(f"Resource '{self.name}' allocated")

    def __del__(self) -> None:
        print(f"Resource '{self.name}' deallocated from memory")

def main() -> None:
    print("Creating resource...")
    res = Resource("DatabaseSocket")
    print(f"Current reference count: {sys.getrefcount(res) - 1}")

    alias = res
    print(f"With alias, reference count: {sys.getrefcount(res) - 1}")

    print("Dropping references...")
    del res
    print("Dropping final alias...")
    del alias
    print("Function complete.")

```

```
if __name__ == "__main__":  
    main()
```

Run:

```
uv run python lifecycle_demo.py
```

Output:

```
Creating resource...  
Resource 'DatabaseSocket' allocated  
Current reference count: 1  
With alias, reference count: 2  
Dropping references...  
Dropping final alias...  
Resource 'DatabaseSocket' deallocated from memory  
Function complete.
```

Pitfalls

Pitfall 1: `UnboundLocalError` caused by shadowing assignment

If a variable is assigned anywhere inside a function, Python marks it as local for the *entire* function scope. Referencing it before that assignment fails:

```
# Bug:  
count = 10  
  
def increment() -> None:  
    print(count) # UnboundLocalError: cannot access local variable 'count' where it is not  
    count = 20   # Because assignment exists here, Python treated 'count' as local from lin  
  
# Fix: Use global declaration if mutating a global is truly intended:  
def increment_fixed() -> None:  
    global count  
    print(count)  
    count = 20
```

Exercises

1. Write a function that demonstrates each tier of LEGB by overriding a built-in name (e.g. defining a local `len = 100`) and verifying the local value shadows the built-in.
 2. Build a generator or closure using `nonlocal` that generates unique incremental API transaction tokens: `TXN-0001`, `TXN-0002`, etc.
 3. Write a small script demonstrating cyclic references between two objects and verify how the `gc` module detects them.
-

Further reading

- Python Language Reference: *Section 4.2: Naming and Binding*.
- PEP 3104: *Access to Names in Outer Scopes (`nonlocal`)*.
- CPython Internal Documentation: *Garbage Collector Design and Reference Counting*.

Data Types

Integers and Arbitrary Precision

Integers and Arbitrary Precision

After reading this chapter, you will master Python’s unbounded integer arithmetic, bitwise operations, numerical bases, and understand how CPython represents numbers without overflow.

Mental model

Unlike C, Java, or Go—where integers are fixed 32-bit or 64-bit hardware integers subject to overflow—Python’s `int` has **arbitrary precision**. It can store integers as large as your system memory permits.

CPython `PyLongObject` Structure (Heap):

<code>ob_refcnt</code>	: 1	Reference count
<code>ob_type</code>	: <class 'int'>	Type pointer
<code>ob_size</code>	: 3 (indicates 3 digits stored; sign)	Size in digits
<code>ob_digit[0]</code>	: 0x3fffffff (low 30 bits)	
<code>ob_digit[1]</code>	: 0x3fffffff (next 30 bits)	Array of 30-bit
<code>ob_digit[2]</code>	: 0x00000001 (high bits)	base-2 ³⁰ digits

CPython breaks large integers into chunks of 30 bits (on 64-bit systems) stored as an array of unsigned 32-bit digits. When numbers grow beyond one digit, CPython performs multi-precision schoolbook arithmetic automatically.

Minimal example

Save as `arbitrary_ints.py`:

```
# arbitrary_ints.py
def main() -> None:
    # 256-bit cryptographic boundary
    max_uint256 = (1 << 256) - 1
    print(f"Max 256-bit integer:\n{max_uint256}")
```

```

# Computing 100! (factorial) without overflow
fact_100 = 1
for i in range(1, 101):
    fact_100 *= i

print(f"\n100! has {len(str(fact_100))} decimal digits:")
print(f"{fact_100}")

if __name__ == "__main__":
    main()

```

Run via `uv run python arbitrary_ints.py`:

Max 256-bit integer:

115792089237316195423570985008687907853269984665640564039457584007913129639935

100! has 158 decimal digits:

93326215443944152681699238856266700490715968264381621468592963895217599993229915608941463976156

Worked examples

Case 1: Bit manipulation and permission masks

Python supports the full suite of bitwise operators: `&` (AND), `|` (OR), `^` (XOR), `~` (NOT), and `<<` / `>>` (shifts).

```

# bitmask_flags.py
# Standard UNIX file permission bits
READ     = 0b100 # 4
WRITE    = 0b010 # 2
EXECUTE   = 0b001 # 1

def check_permissions(mask: int) -> None:
    can_read = bool(mask & READ)
    can_write = bool(mask & WRITE)
    can_exec = bool(mask & EXECUTE)
    print(f"Mask 0o{mask:o} ({bin(mask)}): r={can_read}, w={can_write}, x={can_exec}")

if __name__ == "__main__":
    perm = READ | WRITE
    check_permissions(perm)

    # Add execute permission

```



```

perm |= EXECUTE
check_permissions(perm)

# Revoke write permission
perm &= ~WRITE
check_permissions(perm)

```

Run:

```
uv run python bitmask_flags.py
```

Output:

```

Mask 0o6 (0b110): r=True, w=True, x=False
Mask 0o7 (0b111): r=True, w=True, x=True
Mask 0o5 (0b101): r=True, w=False, x=True

```

Case 2: Integer division vs float division

In Python, `/` always performs **true float division**, while `//` performs **floor division**:

```

# division_rules.py
def main() -> None:
    print(f"7 / 2 = {7 / 2} (always returns float)")
    print(f"7 // 2 = {7 // 2} (floor division integer)")
    print(f"-7 // 2 = {-7 // 2} (floored toward negative infinity!)")
    print(f"7 % 2 = {7 % 2} (modulo remainder)")

    # divmod returns (quotient, remainder) simultaneously
    q, r = divmod(25, 4)
    print(f"divmod(25, 4) -> quotient={q}, remainder={r}")

if __name__ == "__main__":
    main()

```

Run:

```
uv run python division_rules.py
```

Pitfalls

Pitfall 1: Converting colossal integers to strings

Python enforces an integer string conversion security limit (4300 digits by default, configurable via `sys.set_int_max_str_digits`) to protect against quadratic-time denial of service attacks:

```
import sys
huge = 10**5000
# str(huge) -> ValueError: Exceeds the limit (4300 digits) for integer string conversion
```

Pitfall 2: Negative modulo differences from C

In C, `-7 % 3` evaluates to `-1`. In Python, `%` always shares the sign of the divisor (denominator): `-7 % 3 == 2` because `-7 = (-3 * 3) + 2`.

Exercises

1. Write a function `to_hex_and_bin(val: int)` that returns a formatted string showing an integer in decimal, hexadecimal (`0x...`), and binary (`0b...`).
 2. Implement a function `count_set_bits(n: int) -> int` that counts how many binary 1s exist in an integer using bitwise operations (`n & (n - 1)`).
 3. Compute 2^{1000} and print the number of bits required to represent it using the integer method `.bit_length()`.
-

Further reading

- CPython Implementation: `Objects/longobject.c`.
- Python Documentation: `sys.set_int_max_str_digits`.
- PEP 237: *Unifying Long Integers and Integers*.

Floating-Point Numbers and Decimal Precision

Floating-Point Numbers and Decimal Precision

After reading this chapter, you will understand the IEEE 754 standard behind Python's `float`, avoid floating-point representation traps, and know when to use `decimal.Decimal` and `fractions.Fraction`.

Mental model

Python's `float` type is implemented as a standard C `double`—a 64-bit IEEE 754 binary floating-point number:

Sign	Exponent (11 bits)	Fraction / Mantissa (52 bits)
1b	biased binary scale	normalized significant digits

Because numbers are stored in base-2 (binary fractions), decimal numbers like 0.1 (1/10) cannot be represented exactly in binary, just as 1/3 cannot be represented in a finite number of decimal digits (0.3333...).

In base 10: $1/10 = 0.1$ (exact)

In base 2 : $1/10 = 0.00011001100110011\dots$ (infinite repeating binary sequence!)

Therefore, $0.1 + 0.2$ produces 0.30000000000000004 instead of 0.3.

Minimal example

Save as `float_precision.py`:

```
# float_precision.py
import math
from decimal import Decimal

def main() -> None:
    # Standard float arithmetic
    f1 = 0.1
```

```

f2 = 0.2
f_sum = f1 + f2
print(f"Float 0.1 + 0.2   : {f_sum:.20f}")
print(f"f_sum == 0.3      : {f_sum == 0.3} (Trap!)")
print(f"math.isclose(...) : {math.isclose(f_sum, 0.3)} (Correct comparison)")

# Exact decimal arithmetic
d1 = Decimal("0.1")
d2 = Decimal("0.2")
d_sum = d1 + d2
print(f"\nDecimal sum      : {d_sum}")
print(f"d_sum == Decimal('0.3'): {d_sum == Decimal('0.3')}")

if __name__ == "__main__":
    main()

```

Run via `uv run python float_precision.py`:

```

Float 0.1 + 0.2   : 0.30000000000000004441
f_sum == 0.3      : False (Trap!)
math.isclose(...) : True (Correct comparison)

```

```

Decimal sum      : 0.3
d_sum == Decimal('0.3'): True

```

Worked examples

Case 1: Financial calculations with `decimal.Decimal`

Never use `float` for currency, invoices, or billing. Always use `Decimal` initialized with strings:

```

# billing_calculator.py
from decimal import Decimal, ROUND_HALF_UP

def calculate_invoice(subtotal_cents: str, tax_rate: str) -> None:
    subtotal = Decimal(subtotal_cents)
    tax = (subtotal * Decimal(tax_rate)).quantize(Decimal("0.01"), rounding=ROUND_HALF_UP)
    total = subtotal + tax

    print(f"Subtotal : ${subtotal:>8}")
    print(f"Tax ({tax_rate}): ${tax:>8}")
    print(f"Total      : ${total:>8}")

if __name__ == "__main__":

```

```
calculate_invoice("149.95", "0.0825")
```

Run:

```
uv run python billing_calculator.py
```

Output:

```
Subtotal : $ 149.95
Tax (0.0825): $ 12.37
Total    : $ 162.32
```

Case 2: Exact rational arithmetic with `fractions.Fraction`

When you need exact symbolic fraction arithmetic without rounding:

```
# rational_math.py
from fractions import Fraction

def main() -> None:
    f1 = Fraction(1, 3)
    f2 = Fraction(1, 6)
    result = f1 + f2
    print(f"1/3 + 1/6 = {result} (Exact reduction to 1/2)")

    # Converting float to nearest fraction
    approx = Fraction(0.75)
    print(f"0.75 as Fraction: {approx}")

if __name__ == "__main__":
    main()
```

Run:

```
uv run python rational_math.py
```

Pitfalls

Pitfall 1: Initializing Decimal from a float literal

```
# Danger:
bad = Decimal(0.1) # Decimal('0.1000000000000000055511151231257827021181583404541015625')

# Correct: Always pass strings to Decimal
good = Decimal("0.1")
```

Pitfall 2: Comparing floats with ==

Never compare floats directly using `==` unless checking against `0.0`. Always use `math.isclose(a, b, rel_tol=1e-9)` or `abs(a - b) < epsilon`.

Exercises

1. Write a function `safe_float_equal(a: float, b: float, epsilon: float = 1e-9) -> bool` that verifies float equality within tolerance.
 2. Calculate the compound interest of \$10,000 at 5.5% annual interest compounded monthly over 10 years using `decimal.Decimal`.
 3. Demonstrate special float values: positive infinity (`float("inf")`), negative infinity (`float("-inf")`), and Not-a-Number (`float("nan")`). Check their behavior with `math.isinf()` and `math.isnan()`.
-

Further reading

- David Goldberg: *What Every Computer Scientist Should Know About Floating-Point Arithmetic*.
- Python Standard Library: `decimal` module (*Decimal fixed point and floating point arithmetic*).
- Python Standard Library: `math.isclose` documentation.

Booleans, Truthiness, and Short-Circuit Logic

Booleans, Truthiness, and Short-Circuit Logic

After reading this chapter, you will master Python's `bool` type, the exact rules of truth value testing (truthiness), and how short-circuit evaluation powers defensive programming idioms.

Mental model

In Python, `bool` is an explicit subclass of `int`. There are only two instances: `True` and `False`.

```
class 'int'

    inherits

class 'bool'
```

True (int value: 1) False (int value: 0)

Every Python object can be evaluated in a boolean context (such as an `if` statement or `while` loop). The language defines a specific set of **falsy** values; **every other object in Python is considered **truthy****.

Falsy Objects:

- Constants: `None`, `False`
- Zero numbers: `0`, `0.0`, `0j`, `Decimal(0)`, `Fraction(0, 1)`
- Empty sequences: `""`, `()`, `[]`, `b""`, `bytearray(b"")`
- Empty collections/mappings: `{}`, `set()`, `frozenset()`
- Custom objects defining `__bool__()` returning `False`, or `__len__()` returning `0`

Minimal example

Save as `truthiness_demo.py`:

```

# truthiness_demo.py
def main() -> None:
    falsy_values = [None, False, 0, 0.0, "", [], {}, set()]

    print("Checking standard falsy values:")
    for item in falsy_values:
        print(f"bool({repr(item):10s}) -> {bool(item)}")

    # Short-circuit returns the deciding operand, not necessarily a boolean!
    val1 = "fallback_host" or "default"
    val2 = "" or "fallback_host"
    print(f"\nval1: {val1}")
    print(f"val2: {val2}")

if __name__ == "__main__":
    main()

```

Run via `uv run python truthiness_demo.py`:

Checking standard falsy values:

```

bool(None      ) -> False
bool(False     ) -> False
bool(0         ) -> False
bool(0.0       ) -> False
bool('')       ) -> False
bool([]        ) -> False
bool({})       ) -> False
bool(set())    ) -> False

```

val1: fallback_host

val2: fallback_host

Worked examples

Case 1: Short-circuit evaluation as a safety guard

The `and` operator evaluates left-to-right; if the left operand is falsy, it immediately aborts evaluation and returns that operand. This allows writing safe guards:

```

# guard_demo.py
class DeviceConnection:
    def __init__(self, connected: bool) -> None:
        self.connected = connected

```

```

def query_stats(self) -> str:
    return "CPU: 12%, Temp: 41C"

def check_device(dev: DeviceConnection | None) -> None:
    # Guard against None, then check connected attribute
    if dev is not None and dev.connected:
        print(f"Device live: {dev.query_stats()}")
    else:
        print("Device unavailable or offline")

if __name__ == "__main__":
    check_device(None)
    check_device(DeviceConnection(connected=False))
    check_device(DeviceConnection(connected=True))

```

Run:

```
uv run python guard_demo.py
```

Case 2: The conditional ternary expression

Python supports inline ternary expressions: `value_if_true if condition else value_if_false`:

```

# ternary_demo.py
def get_env_concurrency(is_prod: bool) -> int:
    workers = 32 if is_prod else 4
    return workers

if __name__ == "__main__":
    print(f"Development workers: {get_env_concurrency(is_prod=False)}")
    print(f"Production workers : {get_env_concurrency(is_prod=True)}")

```

Run:

```
uv run python ternary_demo.py
```

Pitfalls

Pitfall 1: Fallback idiom swallowing valid falsy values

```
# Dangerous when 0 or False is a legitimate user configuration value:
timeout = config.get("timeout") or 30 # If timeout was configured as 0, it gets replaced by 30

# Correct fix:
timeout = config["timeout"] if "timeout" in config and config["timeout"] is not None else 30
```

Pitfall 2: Explicit comparison with True or False

Never write `if is_ready == True:` or `if is_ready is True:`. Idiomatic Python simply writes `if is_ready:`.

Exercises

1. Create a custom class `ServerPool` that evaluates to `False` in a boolean context when its internal server list is empty, and `True` when it has at least one server, using `__bool__()`.
 2. Trace the exact return value of: `""` and `"hello"`, `"hello"` and `"world"`, `[]` or `0` or `"default"`. Verify by running in a script.
 3. Write a validator function `is_valid_payload(payload: dict) -> bool` that checks if the dictionary is non-empty and contains required keys `"id"` and `"timestamp"` using short-circuit boolean logic.
-

Further reading

- Python Language Reference: *Section 6.11: Boolean operations.*
- Python Standard Library: *Truth Value Testing.*
- PEP 285: *Adding a bool type.*

Strings, UTF-8 Encoding, and Slicing

Strings, UTF-8 Encoding, and Slicing

After reading this chapter, you will understand how CPython represents Unicode text, master string slicing mechanics, and author clean modern f-strings.

Mental model

In Python 3, `str` is an immutable sequence of Unicode code points. CPython internally optimizes string storage (PEP 393: Flexible String Representation) using the smallest necessary byte width per character: - Pure ASCII strings use 1 byte per character (**Latin-1**). - Extended multilingual strings use 2 bytes (**UCS-2**). - Emojis and complex scripts use 4 bytes (**UCS-4**).

String Slicing Mechanics:

String:	P	y	t	h	o	n
+ Index:	0	1	2	3	4	5
- Index:	-6	-5	-4	-3	-2	-1

Slice Syntax: `s[start:stop:step]`

Interval: Half-open `[start, stop)` - includes start, excludes stop.

Minimal example

Save as `string_slicing.py`:

```
# string_slicing.py
def main() -> None:
    text = "telemetry_prod_us_east_1"

    # Slicing
    prefix = text[:9]          # From start to index 9 (exclusive)
    env = text[10:14]         # "prod"
    suffix = text[-9:]        # Last 9 characters
    reversed_text = text[::-1] # Step -1 reverses sequence

    print(f"Original : {text}")
    print(f"Prefix   : {prefix}")
    print(f"Env      : {env}")
```

```

print(f"Suffix    : {suffix}")
print(f"Reversed  : {reversed_text}")

# Modern f-string debugging specifier (=)
latency_ms = 4.2819
print(f"\nDebugging format: {latency_ms:=.2f}")

if __name__ == "__main__":
    main()

```

Run via `uv run python string_slicing.py`:

```

Original : telemetry_prod_us_east_1
Prefix   : telemetry
Env      : prod
Suffix   : us_east_1
Reversed : 1_tsae_su_dorp_yrtmelet

```

Debugging format: latency_ms=4.28

Worked examples

Case 1: String building performance (join vs +=)

Strings are immutable. Concatenating strings with `+=` in a loop creates a new string and copies data on every iteration ($O(N^2)$ time). Using `''.join()` is $O(N)$.

```

# string_builder.py
import time

def benchmark() -> None:
    n = 100_000
    words = ["chunk"] * n

    # Idiomatic O(N) method
    start = time.perf_counter()
    result = "".join(words)
    duration = time.perf_counter() - start
    print(f"join() duration: {duration:.4f}s (length={len(result)})")

if __name__ == "__main__":
    benchmark()

```

Run:

```
uv run python string_builder.py
```

Case 2: Encoding strings to bytes and decoding back

When sending data over network sockets or files, you must explicitly encode `str` (characters) to `bytes` (raw binary):

```
# unicode_codec.py
def main() -> None:
    original = "Microservices  [ s]"

    # UTF-8 serialization
    encoded_bytes = original.encode("utf-8")
    print(f"Encoded bytes ({len(encoded_bytes)} bytes):\n{encoded_bytes}")

    # Deserialization
    decoded_str = encoded_bytes.decode("utf-8")
    print(f"\nDecoded str: {decoded_str}")
    print(f"Match: {original == decoded_str}")

if __name__ == "__main__":
    main()
```

Run:

```
uv run python unicode_codec.py
```

Pitfalls

Pitfall 1: Confusing bytes and str

`str` and `bytes` cannot be implicitly combined. Concatenating `b"hello" + "world"` raises an immediate `TypeError`.

Pitfall 2: Slice out-of-range behavior

Index lookups like `s[100]` raise `IndexError`. However, slices like `s[100:200]` do not fail; they return an empty string `""`.

Exercises

1. Write a function `is_palindrome(s: str) -> bool` that strips non-alphanumeric characters, converts to lowercase, and checks if the string equals its reverse slice (`s[::-1]`).
 2. Parse an ISO timestamp string "2026-09-06T20:15:30Z" into separate date and time components using slicing.
 3. Format a tabular output using f-string alignment specifiers: left-aligned 20 chars (<20), right-aligned 10 chars (>10), and float formatted to 2 decimals.
-

Further reading

- PEP 393: *Flexible String Representation*.
- Python Standard Library: *Text Sequence Type* — `str`.
- Ned Batchelder: *Pragmatic Unicode*.

Type Casting, Type Inspection, and Annotations

Type Casting, Type Inspection, and Annotations

After reading this chapter, you will master Python's strong typing model, safely perform type conversions, inspect object types at runtime, and write modern Python 3.14 type annotations.

Mental model

Python is **strongly typed** and **dynamically typed**: - **Strongly typed**: Python does not implicitly coerce incompatible types (e.g. adding a string and an integer raises `TypeError`). - **Dynamically typed**: Types belong to values, not to variable names. A name can point to an integer at one moment and a string the next.

Strong Typing Enforcement:

```
"5" + 2      TypeError: can only concatenate str (not "int") to str
```

Explicit Conversion:

```
int("5") + 2      7 (Integer addition)
"5" + str(2)       "52" (String concatenation)
```

At runtime, `isinstance()` checks if an object belongs to a class or any of its subclasses, traversing the inheritance tree.

Minimal example

Save as `type_inspect_demo.py`:

```
# type_inspect_demo.py
def process_metric(raw_value: str | int | float) -> float:
    # 1. Type inspection with isinstance
    if isinstance(raw_value, (int, float)):
        return float(raw_value)

    # 2. Explicit type casting
    try:
        return float(raw_value)
    except ValueError as err:
        raise ValueError(f"Cannot cast {raw_value!r} to float") from err
```

```

def main() -> None:
    samples = [42, "128.5", 3.14159, "invalid_num"]

    for sample in samples:
        try:
            result = process_metric(sample)
            print(f"Sample: {sample!r:15s} -> Processed float: {result}")
        except ValueError as err:
            print(f"Sample: {sample!r:15s} -> Error: {err}")

if __name__ == "__main__":
    main()

```

Run via `uv run python type_inspect_demo.py`:

```

Sample: 42                -> Processed float: 42.0
Sample: '128.5'           -> Processed float: 128.5
Sample: 3.14159           -> Processed float: 3.14159
Sample: 'invalid_num'     -> Error: Cannot cast 'invalid_num' to float

```

Worked examples

Case 1: Checking types: `isinstance` vs `type()`

`isinstance()` respects class inheritance hierarchies, while `type(obj) is Class` requires an exact type match.

```

# isinstance_vs_type.py
class Animal:
    pass

class Dog(Animal):
    pass

def verify_types() -> None:
    d = Dog()

    print(f"type(d) is Dog      : {type(d) is Dog}")
    print(f"type(d) is Animal : {type(d) is Animal} (False: ignores inheritance)")
    print(f"isinstance(d, Animal): {isinstance(d, Animal)} (True: recognizes subclass)")

    # bool is a subclass of int!
    print(f"isinstance(True, int): {isinstance(True, int)} (True!)")

```

```

    print(f"type(True) is int      : {type(True) is int} (False!)")

if __name__ == "__main__":
    verify_types()

```

Run:

```
uv run python isinstance_vs_type.py
```

Case 2: Modern Python 3.14 type annotations

Type annotations provide documentation and allow static type checkers like `mypy` and `pyright` to find bugs before runtime:

```

# typed_service.py
def calculate_throughput(requests: int, duration_seconds: float) -> float:
    if duration_seconds <= 0:
        raise ValueError("Duration must be positive")
    return requests / duration_seconds

def lookup_node(node_id: str) -> str | None:
    nodes = {"node-1": "10.0.0.1", "node-2": "10.0.0.2"}
    return nodes.get(node_id)

if __name__ == "__main__":
    tps = calculate_throughput(10_000, 4.5)
    print(f"Throughput: {tps:.2f} req/s")
    print(f"Lookup: {lookup_node('node-1')}")

```

Run:

```
uv run python typed_service.py
```

Pitfalls

Pitfall 1: Using `type(x) == T` instead of `isinstance()`

Checking `type(x) == int` will return `False` if someone passes a subclass or specialized numerical type. Always prefer `isinstance(x, int)`.

Pitfall 2: Assuming type annotations enforce runtime checks

Python does not validate type hints at runtime by default. If you pass a string to a function annotated with `x: int`, Python executes it without error until an incompatible operation occurs. Runtime validation requires libraries like Pydantic.

Exercises

1. Write a function `safe_int_cast(val: str, default: int = 0) -> int` that parses an integer string and returns `default` if parsing fails.
 2. Given a mixed list `items = [1, "two", 3.0, [4], (5,), {"six": 6}]`, write a script that separates elements into a dictionary grouped by type name.
 3. Run `mypy` or `ruff check` on a script containing a deliberate type annotation mismatch to observe static analysis feedback.
-

Further reading

- PEP 484: *Type Hints*.
- PEP 604: *Allow-writing union types as `X | Y`*.
- Python Standard Library: `typing` module.

Template Strings (t-strings)

Template Strings (t-strings)

An **f-string** builds a finished **str** the moment it runs. A **t-string** (Python 3.14, PEP 750) looks the same — braces, **!r**, format specs — but the **t** prefix returns a `string.templatelib.Template`: the static pieces and the interpolated values stay separate until **your** code decides how to combine them. The boring default is still an f-string for ordinary messages. Reach for a t-string when the values must be escaped, parameterized, or logged as structured parts — not glued into a string first.

Mental model

- `f"..."` → **str**. Done. Conversions (**!s**, **!r**, **!a**) and format specs (`:.2f`) are applied for you.
- `t"..."` → **Template**. Not a string. `.strings` is the tuple of static chunks; `.interpolations` holds **Interpolation** objects; `.values` is the tuple of evaluated results. There is always **one more** string than interpolations — the holes sit between the chunks.
- Iterating a **Template** yields non-empty static strings and **Interpolations** in order. Empty static pieces are skipped by iteration but still appear in `.strings`.
- Each **Interpolation** has `.value`, `.expression` (source text), `.conversion` (**None**, **"s"**, **"r"**, or **"a"**), and `.format_spec`. Unlike f-strings, **you** choose whether to apply conversion and format — `string.templatelib.convert` mimics f-string conversion when you want that.
- **Template** + **str** is a **TypeError**. Concatenate **Template** with **Template**, or wrap a static piece in `Template("...")`.
- Stdlib types live in `string.templatelib`. There is no built-in HTML or SQL renderer: write a small function that walks the parts.

Worked examples

Case 1: Same braces, different type

Save as `t_vs_f.py`. One desk table, two prefixes. The f-string is text you can print and concatenate. The t-string is a structure.

```
# t_vs_f.py
def main():
    table = 12
    fmsg = f"table {table}"
    tmsg = t"table {table}"
    print(type(fmsg).__name__, fmsg)
    print(type(tmsg).__name__, tmsg.strings, tmsg.values)
```



```
if __name__ == "__main__":
    main()
```

Run:

```
uv run python t_vs_f.py
```

Output:

```
str table 12
Template ('table ', '') (12,)
```

tmsg is not "table 12". Its .values hold 12 as an int, not as digits inside a string.

Case 2: Walk the parts at the desk

Save as `t_inspect.py`. Print static chunks and each interpolation's expression, value, conversion, and format spec.

```
# t_inspect.py
from string import Interpolation

def main():
    table = 12
    note = "no onions"
    tpl = t"ticket → table {table}; note={note!r}"
    print(type(tpl).__name__)
    print(tpl.strings)
    print(tpl.values)
    for part in tpl:
        if isinstance(part, Interpolation):
            print(
                "interp",
                part.expression,
                part.value,
                part.conversion,
                repr(part.format_spec),
            )
        else:
            print("static", repr(part))

if __name__ == "__main__":
    main()
```

Run:

```
uv run python t_inspect.py
```

Output:

```
Template
('ticket → table ', '; note=', '')
(12, 'no onions')
static 'ticket → table '
interp table 12 None ''
static '; note='
interp note no onions r ''
```

!r did **not** turn the note into a quoted string yet. It only set `.conversion` to "r". A processor that cares will call `convert(part.value, part.conversion)`.

Case 3: Render like an f-string (on purpose)

Save as `t_render.py`. When you want the familiar finished text, apply `convert` and `format` yourself. This is the boring “I still need a string” path — and it makes the deferred steps visible.

```
# t_render.py
from string import Interpolation, convert

def render(template):
    parts = []
    for part in template:
        if isinstance(part, Interpolation):
            text = convert(part.value, part.conversion)
            if part.format_spec:
                text = format(text, part.format_spec)
            else:
                text = str(text)
            parts.append(text)
        else:
            parts.append(part)
    return "".join(parts)

def main():
    cents = 1850
    note = "rush"
    print(render(t"total ${cents / 100:.2f}"))
    print(render(t"note={note!r}"))
```

```
if __name__ == "__main__":
    main()
```

Run:

```
uv run python t_render.py
```

Output:

```
total $18.50
note='rush'
```

Same result as the matching f-strings. Prefer `f"..."` when this is all you need. Prefer `t"..."` when Case 4 or Case 5 is the real goal.

Case 4: Escape guest text for HTML

Save as `t_html.py`. Static markup stays raw. Interpolated values pass through `html.escape`. That split is why t-strings exist: you never build `"<p>" + guest + "</p>"` with a hostile `guest`.

```
# t_html.py
from html import escape
from string import Interpolation, convert

def safe_html(template):
    parts = []
    for part in template:
        if isinstance(part, Interpolation):
            text = convert(part.value, part.conversion)
            if part.format_spec:
                text = format(text, part.format_spec)
            parts.append(escape(str(text), quote=True))
        else:
            parts.append(part)
    return "".join(parts)

def main():
    guest = '<script>alert("x")</script>'
    table = 7
    print(safe_html(t"<p>table {table}: {guest}</p>"))
    print(safe_html(t"<p>table {table}: {guest!r}</p>"))

if __name__ == "__main__":
```

```
main()
```

Run:

```
uv run python t_html.py
```

Output:

```
<p>table 7: &lt;script&gt;alert(&quot;x&quot;)&lt;/script&gt;</p>
<p>table 7: &#x27;&lt;script&gt;alert(&quot;x&quot;)&lt;/script&gt;&#x27;</p>
```

The tags in the static pieces are real HTML. The script in `guest` is text. An f-string of the same shape would have injected markup.

Case 5: SQL placeholders, not string glue

Save as `t_sql.py`. Walk the template: static SQL stays; each interpolation becomes `?` and its value goes in a `params` tuple. The database API binds `params` — you never splice user data into the statement string.

```
# t_sql.py
from string import Interpolation

def sql_query(template):
    chunks = []
    params = []
    for part in template:
        if isinstance(part, Interpolation):
            chunks.append("?")
            params.append(part.value)
        else:
            chunks.append(part)
    return "".join(chunks), tuple(params)

def main():
    table = 12
    status = "open"
    sql, params = sql_query(
        t"SELECT id FROM tickets WHERE table_n = {table} AND status = {status}"
    )
    print(sql)
    print(params)
```

```
if __name__ == "__main__":  
    main()
```

Run:

```
uv run python t_sql.py
```

Output:

```
SELECT id FROM tickets WHERE table_n = ? AND status = ?  
(12, 'open')
```

An f-string here would be the classic injection foot-gun. The t-string keeps values out of the SQL text until a real driver binds them.

The trap

Save as `t_trap.py`. Treating a `Template` like a `str` fails loudly — or, worse, `str(template)` gives a debug dump you might accidentally ship.

```
# t_trap.py  
def main():  
    table = 12  
    tpl = t"table {table}"  
    try:  
        print("msg: " + tpl)  
    except TypeError as exc:  
        print(type(exc).__name__ + ":", exc)  
        print("forced:", "msg: " + str(tpl))  
        print("parts:", list(tpl))  
  
if __name__ == "__main__":  
    main()
```

Run:

```
uv run python t_trap.py
```

Output:

```
TypeError: can only concatenate str (not "string.template.Template") to str  
forced: msg: Template(strings=('table ', ''), interpolations=(Interpolation(12, 'table', None,  
parts: ['table ', Interpolation(12, 'table', None, '')])
```

+ with a plain string is rejected. `str(tmpl)` is not a render — it is the object's representation. Process the template (Cases 3–5) or keep it as a `Template`. Do not `str()` your way out of the type error and call it done.

The boring rule

- Use **f-strings** for human messages, logs, and errors when every value is already safe to embed.
- Use **t-strings** when interpolations need escaping, parameterization, or structured handling before they become text.
- Write one small processor per sink (`safe_html`, `sql_query`, `render`) that walks parts with `isinstance(..., Interpolation)`.
- Apply `convert` and `format` only when that sink should behave like an f-string.
- Never concatenate `Template` with `str`. Never ship `str(template)` as user-facing text.
- Do not replace every f-string with a t-string. Deferred structure is a tool, not a fashion.

Try this

1. In `t_vs_f.py`, print `tmsg + t" ok"` and `list(tmsg + t" ok")`. Confirm you can concatenate two templates.
2. In `t_html.py`, change `guest` to `O'Reilly & Sons` and re-run. Confirm quotes and `&` are escaped.
3. In `t_sql.py`, add `AND seats >= {seats}` with `seats = 2`. Confirm a third `?` and `2` in params.
4. In `t_render.py`, drop the `convert` / `format` calls and append `str(part.value)` only. Re-run the `!r` line — then put `convert` back and explain the difference in one sentence.

Functions

Defining Functions, Signatures, and Return Values

Defining Functions, Signatures, and Return Values

After reading this chapter, you will master the `def` statement, write clean docstrings, leverage multiple return values, and apply guard clauses to prevent nested logic.

Mental model

In Python, `def` is an executable statement. When CPython encounters `def func(): ...` during module execution, it creates a **function object** (`PyFunctionObject`) wrapping the compiled bytecode, and binds the function's name in the current namespace.

Execution of 'def':

Source Code	Bytecode	CodeObject	PyFunctionObject	Name binding
			- <code>__name__</code>	
			- <code>__doc__</code>	
			- <code>__code__</code>	

Call Execution:

<code>func()</code>	Allocates a new <code>PyFrameObject</code> on the call stack
	- Holds local namespace
	- Executes bytecode
	- Destroys frame upon return and yields value to caller

If a function completes without reaching an explicit `return` statement, Python implicitly returns `None`.

Minimal example

Save as `func_basics.py`:

```
# func_basics.py
def calculate_metrics(values: list[float]) -> tuple[float, float, float]:
    """Compute the minimum, maximum, and average of a non-empty list of values."""
    if not values:
        raise ValueError("Cannot calculate metrics on empty values list")
```

```

    total = sum(values)
    count = len(values)
    return min(values), max(values), total / count

def main() -> None:
    latencies = [12.4, 8.9, 15.2, 11.0, 9.7]
    min_lat, max_lat, avg_lat = calculate_metrics(latencies)

    print(f"Min latency : {min_lat:.1f}ms")
    print(f"Max latency : {max_lat:.1f}ms")
    print(f"Avg latency : {avg_lat:.1f}ms")

if __name__ == "__main__":
    main()

```

Run via `uv run python func_basics.py`:

```

Min latency : 8.9ms
Max latency : 15.2ms
Avg latency : 11.4ms

```

Worked examples

Case 1: Early returns (The “Bouncer Pattern”)

Deeply nested `if/else` ladders make code hard to read. Use guard clauses with early returns to handle invalid states immediately:

```

# bouncer_pattern.py
def authenticate_request(token: str | None, is_expired: bool, role: str) -> str:
    # Guard 1: Missing token
    if not token:
        return "DENIED: Missing token"

    # Guard 2: Expired token
    if is_expired:
        return "DENIED: Token expired"

    # Guard 3: Insufficient privileges
    if role != "admin":
        return "DENIED: Requires admin role"

    # Main happy path: Unnested and clear
    return "AUTHORIZED: Welcome administrator"

```

```

if __name__ == "__main__":
    print(authenticate_request(None, False, "admin"))
    print(authenticate_request("tok_123", True, "admin"))
    print(authenticate_request("tok_123", False, "guest"))
    print(authenticate_request("tok_123", False, "admin"))

```

Run:

```
uv run python bouncer_pattern.py
```

Case 2: Inspecting function introspection attributes

Because functions are first-class objects, you can inspect their metadata:

```

# func_metadata.py
def transfer_funds(sender: str, recipient: str, amount: float) -> bool:
    """Execute a secure transactional balance transfer."""
    return True

def main() -> None:
    print(f"Function Name : {transfer_funds.__name__}")
    print(f"Docstring      : {transfer_funds.__doc__}")
    print(f"Code ArgCount  : {transfer_funds.__code__.co_argcount}")
    print(f"Code Varnames   : {transfer_funds.__code__.co_varnames}")

if __name__ == "__main__":
    main()

```

Run:

```
uv run python func_metadata.py
```

Pitfalls

Pitfall 1: Forgetting explicit returns in branches

If one branch of a function returns a value and another branch omits **return**, the omitted branch returns **None**:

```

# Bug:
def get_user_status(active: bool) -> str:
    if active:

```

```
    return "ONLINE"
# Forgot return: implicit return None!
```

Pitfall 2: Mutating input arguments unexpectedly

Avoid modifying mutable arguments (like lists or dictionaries) in place unless that is the explicitly documented purpose of the function. Prefer returning a new object.

Exercises

1. Write a function `parse_semver(version: str) -> tuple[int, int, int]` that parses "3.14.2" into integer components (3, 14, 2) and handles malformed strings gracefully.
 2. Refactor a deeply nested 3-level `if/else` validation function into a clean flat function using guard clauses.
 3. Write a function with a comprehensive docstring following the Google style guide (Args, Returns, Raises). Print `func.__doc__`.
-

Further reading

- Python Language Reference: *Section 8.6: Function definitions*.
- PEP 257: *Docstring Conventions*.
- Clean Code Architecture: *The Bouncer / Guard Clause Pattern*.

Parameters, Keyword Arguments, and Defaults

Parameters, Keyword Arguments, and Defaults

After reading this chapter, you will master positional vs keyword arguments, configure safe default parameters, and use modern positional-only (/) and keyword-only (*) boundary markers.

Mental model

Python provides precise control over how arguments are passed into a function signature:

```
def api_request(method, url, /, timeout=30, *, secure=True, headers=None):
```

Positional-Only (before "/")	Standard (positional or kw)	Keyword-Only (after "*")
---------------------------------	--------------------------------	-----------------------------

- **/ (Positional-only boundary)**: Parameters before / must be passed by position; callers cannot pass them as `url="..."`.
 - *** (Keyword-only boundary)**: Parameters after * must be passed by name (`secure=False`); prevents bugs where callers pass confusing boolean flags positionally.
 - **Default arguments**: Evaluated **once** when the function is defined, not every time it is called.
-

Minimal example

Save as `parameter_boundaries.py`:

```
# parameter_boundaries.py
def query_endpoint(
    service: str,          # Positional-only parameter (before '/')
    /,
    retries: int = 3,      # Standard parameter
    *,
    use_tls: bool = True,  # Keyword-only parameter (after '*')
    timeout: float = 5.0,  # Keyword-only parameter
) -> None:
    print(f"Service: {service} | Retries: {retries} | TLS: {use_tls} | Timeout: {timeout}s")

def main() -> None:
```

```

# 1. Valid invocation
query_endpoint("auth_svc", 5, use_tls=True, timeout=10.0)

# 2. Valid invocation relying on defaults
query_endpoint("billing_svc", use_tls=False)

# 3. Invalid: query_endpoint(service="auth_svc") -> TypeError: positional-only argument
# 4. Invalid: query_endpoint("auth_svc", 3, True) -> TypeError: takes 2 positional arguments

if __name__ == "__main__":
    main()

```

Run via `uv run python parameter_boundaries.py`:

```

Service: auth_svc | Retries: 5 | TLS: True | Timeout: 10.0s
Service: billing_svc | Retries: 3 | TLS: False | Timeout: 5.0s

```

Worked examples

Case 1: Preventing the “Boolean Parameter Trap”

When functions take boolean flags positionally, call sites become unreadable (`render(True, False, True)`). Keyword-only arguments force clarity:

```

# keyword_only_api.py
def configure_cache(
    host: str,
    port: int,
    *,
    cluster_mode: bool = False,
    ssl_enabled: bool = True,
    eviction_policy: str = "lru"
) -> None:
    print(f"Cache on {host}:{port} [cluster={cluster_mode}, ssl={ssl_enabled}, policy={eviction_policy}]")

if __name__ == "__main__":
    # Call site is self-documenting
    configure_cache("redis.internal", 6379, cluster_mode=True, ssl_enabled=True)

```

Run:

```
uv run python keyword_only_api.py
```

Case 2: The Mutable Default Argument Trap and the Fix

Because default values are evaluated at definition time, a mutable default (like `[]` or `{}`) is shared across all calls:

```
# mutable_default_fix.py
from datetime import datetime

# THE BUG:
def add_log_broken(msg: str, tags: list[str] = []) -> list[str]:
    tags.append(msg)
    return tags

# THE FIX: Use None as default, allocate inside function
def add_log_safe(msg: str, tags: list[str] | None = None) -> list[str]:
    if tags is None:
        tags = []
    tags.append(msg)
    return tags

if __name__ == "__main__":
    print("Broken with mutable default:")
    print("Call 1:", add_log_broken("error 1"))
    print("Call 2:", add_log_broken("error 2")) # Re-uses the same list!

    print("\nSafe with None sentinel:")
    print("Call 1:", add_log_safe("error 1"))
    print("Call 2:", add_log_safe("error 2")) # Fresh list allocated
```

Run:

```
uv run python mutable_default_fix.py
```

Output:

```
Broken with mutable default:
Call 1: ['error 1']
Call 2: ['error 1', 'error 2']
```

```
Safe with None sentinel:
Call 1: ['error 1']
Call 2: ['error 2']
```

Pitfalls

Pitfall 1: Evaluating dynamic defaults like `datetime.now()` in the signature

```
# Danger: The timestamp is frozen to when the module was loaded!
def record_event(name: str, timestamp: datetime = datetime.now()):
    pass

# Fix:
def record_event(name: str, timestamp: datetime | None = None):
    actual_time = timestamp if timestamp is not None else datetime.now()
```

Exercises

1. Define a function `format_currency(amount, /, *, currency="USD", symbol=True)` and test calling it with valid and invalid argument permutations.
 2. Write a function `append_item(item, collection=None)` that safely handles list mutation and returns the updated list.
 3. Use `inspect.signature` from the standard library to inspect the parameters and default values of a custom function.
-

Further reading

- PEP 570: *Python Positional-Only Parameters*.
- PEP 3102: *Keyword-Only Arguments*.
- Python Standard Library: `inspect.signature`.

Variadic Arguments (*args and **kwargs)

Variadic Arguments (*args and **kwargs)

After reading this chapter, you will master variadic parameters, construct flexible APIs using `*args` and `**kwargs`, and write transparent argument forwarding wrappers.

Mental model

Variadic parameters allow a function to accept an arbitrary number of positional or keyword arguments: - `*args`: Collects extra positional arguments into a **tuple**. - `**kwargs`: Collects extra keyword arguments into a **dict**.

Call Site:

```
send_payload("POST", 200, "fast", verbose=True, timeout=10)
```

```
method: "POST"
```

<code>args: (tuple)</code>	<code>kwargs: (dict)</code>
<code>(200, "fast")</code>	<code>{"verbose": True, ...}</code>

At call sites, the operators can be reversed: `*` unpacks an iterable into positional arguments, and `**` unpacks a mapping into keyword arguments.

Minimal example

Save as `variadic_demo.py`:

```
# variadic_demo.py
def format_log(level: str, *messages: str, **metadata: str | int) -> str:
    joined_msg = " | ".join(messages)
    meta_str = " ".join(f"[{k}={v}]" for k, v in metadata.items())
    return f"[{level.upper()}] {joined_msg} {meta_str}".strip()

def main() -> None:
    # Multiple positional messages, multiple keyword attributes
```

```

    entry = format_log(
        "info",
        "Connection established",
        "Handshake completed",
        host="node-alpha",
        port=9000,
        region="us-west-2"
    )
    print(entry)

if __name__ == "__main__":
    main()

```

Run via `uv run python variadic_demo.py`:

```
[INFO] Connection established | Handshake completed [host=node-alpha] [port=9000] [region=us-west-2]
```

Worked examples

Case 1: Argument forwarding in decorator wrappers

When authoring wrappers or logging hooks, you must forward arguments cleanly to the wrapped target function:

```

# wrapper_forward.py
import time
from collections.abc import Callable
from typing import Any

def time_execution(func: Callable[..., Any], *args: Any, **kwargs: Any) -> Any:
    start = time.perf_counter()
    result = func(*args, **kwargs) # Perfect forwarding with * and **
    elapsed = time.perf_counter() - start
    print(f"Function '{func.__name__}' executed in {elapsed * 1000:.3f}ms")
    return result

def compute_heavy_task(base: int, power: int, tag: str = "computation") -> int:
    return base ** power

if __name__ == "__main__":
    out = time_execution(compute_heavy_task, 2, 500_000, tag="benchmark")
    print(f"Task completed: result has {len(str(out))} digits")

```

Run:

```
uv run python wrapper_forward.py
```

Case 2: Unpacking dictionaries and lists at call sites

You can dynamically assemble parameters in dictionaries and pass them into functions using ******:

```
# dynamic_call.py
def connect_db(host: str, port: int, database: str, timeout: int = 30) -> None:
    print(f"Connecting to {database} at {host}:{port} (timeout={timeout}s)")

if __name__ == "__main__":
    config = {
        "host": "postgresql.internal",
        "port": 5432,
        "database": "analytics",
        "timeout": 60,
    }
    # Unpack config dictionary directly into function arguments
    connect_db(**config)
```

Run:

```
uv run python dynamic_call.py
```

Pitfalls

Pitfall 1: Signature ordering violations

Python enforces strict parameter ordering: 1. Standard positional parameters 2. Positional-only boundary / 3. ***args** 4. Keyword-only parameters 5. ****kwargs**

Placing ***args** after ****kwargs** raises a `SyntaxError`.

Pitfall 2: Overusing ****kwargs** and destroying API readability

While ****kwargs** offers ultimate flexibility, overusing it hides what parameters a function actually accepts from documentation, autocomplete, and type checkers. Prefer explicit named parameters for core options.

Exercises

1. Write a function `product(*numbers: float) -> float` that computes the product of all passed numbers, returning 1.0 if no arguments are supplied.
 2. Implement a function `merge_configs(*configs: dict) -> dict` that merges an arbitrary number of configuration dictionaries into a single dictionary in order.
 3. Write a wrapper function `log_calls(func, *args, **kwargs)` that prints all passed arguments before executing `func`.
-

Further reading

- Python Tutorial: *Arbitrary Argument Lists*.
- PEP 448: *Additional Unpacking Generalizations*.
- Fluent Python (2nd Edition): *Chapter 7: Functions as First-Class Objects*.

First-Class Functions, Callables, and Lambdas

First-Class Functions, Callables, and Lambdas

After reading this chapter, you will treat functions as first-class citizens, build command dispatch tables, author concise lambda expressions, and inspect callables.

Mental model

In Python, **functions are first-class objects**. This means a function is just an object like any other integer or string: 1. It can be assigned to a variable name. 2. It can be stored in a collection (list, dict, set). 3. It can be passed as an argument to another function. 4. It can be returned as the result of another function.

```
PyFunctionObject: calculate()

                        stored in

command_table = {
    "calc": calculate,
    "ping": health_check,
}

                        looked up & called
command_table["calc"](args)
```

An anonymous function (lambda) is an inline function defined without a `def` statement, limited to a single expression.

Minimal example

Save as `first_class_demo.py`:

```
# first_class_demo.py
from collections.abc import Callable

def add(a: int, b: int) -> int:
```



```

    return a + b

def multiply(a: int, b: int) -> int:
    return a * b

def execute_operation(op: Callable[[int, int], int], x: int, y: int) -> int:
    """Higher-order function accepting a function as an argument."""
    return op(x, y)

def main() -> None:
    # 1. Pass function by name
    res1 = execute_operation(add, 10, 5)
    res2 = execute_operation(multiply, 10, 5)

    # 2. Pass anonymous lambda
    res3 = execute_operation(lambda a, b: a - b, 10, 5)

    print(f"Add      : {res1}")
    print(f"Multiply : {res2}")
    print(f"Lambda    : {res3}")

if __name__ == "__main__":
    main()

```

Run via `uv run python first_class_demo.py`:

```

Add      : 15
Multiply : 50
Lambda   : 5

```

Worked examples

Case 1: Command Dispatch Tables (Replacing if/elif ladders)

Instead of a long, brittle ladder of if/elif statements, use a dictionary of functions:

```

# dispatch_table.py
from collections.abc import Callable

def handle_start(target: str) -> str:
    return f"Started service: {target}"

def handle_stop(target: str) -> str:
    return f"Stopped service: {target}"

```

```

def handle_restart(target: str) -> str:
    return f"Restarted service: {target}"

DISPATCH_REGISTRY: dict[str, Callable[[str], str]] = {
    "start": handle_start,
    "stop": handle_stop,
    "restart": handle_restart,
}

def dispatch_command(cmd: str, target: str) -> str:
    handler = DISPATCH_REGISTRY.get(cmd)
    if not handler:
        raise ValueError(f"Unknown command: {cmd!r}. Available: {list(DISPATCH_REGISTRY.keys)}")
    return handler(target)

if __name__ == "__main__":
    print(dispatch_command("start", "nginx"))
    print(dispatch_command("restart", "postgres"))

```

Run:

```
uv run python dispatch_table.py
```

Case 2: Custom sorting keys with lambdas and operator

When sorting complex structured data, pass a key function to `sorted()`:

```

# sorting_keys.py
import operator

def main() -> None:
    containers = [
        {"name": "worker-1", "cpu_percent": 84.5, "memory_mb": 512},
        {"name": "worker-2", "cpu_percent": 22.1, "memory_mb": 1024},
        {"name": "worker-3", "cpu_percent": 91.0, "memory_mb": 256},
    ]

    # Sort by CPU descending using a lambda
    by_cpu = sorted(containers, key=lambda c: c["cpu_percent"], reverse=True)
    print("Sorted by CPU (descending):")
    for c in by_cpu:
        print(f" {c['name']:10s} -> {c['cpu_percent']}%")

    # Sort by memory using operator.itemgetter (faster in C)
    by_mem = sorted(containers, key=operator.itemgetter("memory_mb"))

```

```

    print("\nSorted by memory (ascending):")
    for c in by_mem:
        print(f"    {c['name']:10s} -> {c['memory_mb']}MB")

if __name__ == "__main__":
    main()

```

Run:

```
uv run python sorting_keys.py
```

Pitfalls

Pitfall 1: Over-complicating lambdas

Lambdas should be simple one-liners (typically for `sorted(key=...)`). If your lambda needs complex logic, loops, or multiple statements, write a standard named `def` function.

Pitfall 2: Assigning a lambda to a variable instead of `def`

```

# Bad style (violates PEP 8):
my_func = lambda x: x * 2

# Good style:
def my_func(x):
    return x * 2

```

Exercises

1. Build a text pipeline function `process_text(text: str, transforms: list[Callable[[str], str]]) -> str` that applies a list of transformation functions sequentially.
 2. Given a list of tuples `[(1, "b"), (3, "a"), (2, "c")]`, sort them alphabetically by the second element using a lambda key.
 3. Use the built-in `callable()` function to verify which standard objects in a mixed list can be called as functions.
-

Further reading

- Python Documentation: *Functional Programming HOWTO*.
- Python Standard Library: `operator` module (*itemgetter*, *attrgetter*).
- PEP 8: *Programming Recommendations regarding lambda assignment*.

Closures, Lexical Scopes, and Factory Functions

Closures, Lexical Scopes, and Factory Functions

After reading this chapter, you will understand how closures capture enclosing lexical scopes, author factory functions, and avoid the notorious late-binding loop trap.

Mental model

A **closure** occurs when a nested function references a variable defined in its enclosing scope, and that outer function finishes execution. The inner function retains access to that variable even after the outer function's stack frame has returned:

Outer Function: `make_multiplier(factor=3)`
Frame executes and terminates.

Heap Memory: Cell Object

cell_contents: 3 (Held alive by reference count)

referenced by
Inner Function: `multiplier(x)`

```
__code__ : bytecode
__closure__ : (cell, )
```

The captured variables are stored in CPython as `cell` objects inside the function's `__closure__` attribute.

Minimal example

Save as `closure_demo.py`:

```
# closure_demo.py
from collections.abc import Callable
```

```

def make_multiplier(factor: int) -> Callable[[int], int]:
    """Factory function creating specialized multiplier closures."""
    def multiplier(number: int) -> int:
        return number * factor

    return multiplier

def main() -> None:
    double = make_multiplier(2)
    triple = make_multiplier(3)

    print(f"double(10): {double(10)}")
    print(f"triple(10): {triple(10)}")

    # Inspect the closure cell
    if double.__closure__:
        cell_val = double.__closure__[0].cell_contents
        print(f"double closure factor cell: {cell_val}")

if __name__ == "__main__":
    main()

```

Run via `uv run python closure_demo.py`:

```

double(10): 20
triple(10): 30
double closure factor cell: 2

```

Worked examples

Case 1: Stateful API Rate Limiter using nonlocal

Closures can encapsulate private state without requiring a full class:

```

# rate_limiter.py
from collections.abc import Callable
import time

def make_rate_limiter(max_requests: int, window_seconds: float) -> Callable[[], bool]:
    request_timestamps: list[float] = []

    def allow_request() -> bool:
        nonlocal request_timestamps
        now = time.time()

```

```

    # Prune timestamps outside window
    request_timestamps = [t for t in request_timestamps if now - t < window_seconds]

    if len(request_timestamps) < max_requests:
        request_timestamps.append(now)
        return True
    return False

    return allow_request

if __name__ == "__main__":
    limiter = make_rate_limiter(max_requests=2, window_seconds=1.0)
    print("Req 1:", limiter()) # True
    print("Req 2:", limiter()) # True
    print("Req 3:", limiter()) # False (exceeded limit!)

```

Run:

```
uv run python rate_limiter.py
```

Case 2: The Notorious Late-Binding Loop Trap

When creating closures in a loop, inner functions do not capture the variable's *value at that iteration*; they capture the *variable reference itself*. By the time they are called, the loop has completed!

```

# late_binding_trap.py
def create_multipliers_broken():
    # THE TRAP: All lambdas reference the single variable 'i'
    return [lambda x: x * i for i in range(4)]

def create_multipliers_fixed():
    # THE FIX: Bind 'i' as a default argument evaluated at definition time
    return [lambda x, i=i: x * i for i in range(4)]

if __name__ == "__main__":
    print("Broken (late binding):")
    funcs_broken = create_multipliers_broken()
    print([f(10) for f in funcs_broken]) # [30, 30, 30, 30] !

    print("\nFixed (default argument binding):")
    funcs_fixed = create_multipliers_fixed()
    print([f(10) for f in funcs_fixed]) # [0, 10, 20, 30]

```

Run:

```
uv run python late_binding_trap.py
```


Output:

```
Broken (late binding):  
[30, 30, 30, 30]
```

```
Fixed (default argument binding):  
[0, 10, 20, 30]
```

Pitfalls

Pitfall 1: Retaining large memory references in closures

Because closure cells hold strong references, an inner function will keep an entire enclosing object alive in memory even if it only needs one small sub-property. Extract the needed sub-property into a local variable before closing over it.

Exercises

1. Write a function `make_prefixer(prefix: str)` that returns a function that prepends `prefix` to any passed string.
 2. Build an accumulator function `make_accumulator(initial: float = 0.0)` that uses `nonlocal` to accumulate added numbers and return the current running total.
 3. Write a script demonstrating the late-binding trap with button or task callback functions, and implement both the default-argument fix and `functools.partial` fix.
-

Further reading

- Python Language Reference: *Section 4.1: Naming and Binding (Cell objects)*.
- PEP 227: *Statically Nested Scopes*.
- Python Standard Library: `functools.partial`.

Generator Functions and Subgenerator Delegation

Generator Functions and Subgenerator Delegation

After reading this chapter, you will master the mechanics of generator functions (`yield`), establish bi-directional communication pipelines using `.send()`, inject exceptions and trigger cleanup via `.throw()` and `.close()`, and delegate execution to nested subgenerators using `yield from` (PEP 380).

Mental model

An ordinary Python function begins execution, computes a result, and exits, destroying its stack frame. A **generator function** (any function containing the `yield` keyword) returns a generator object whose execution state is preserved on the heap across calls:

Standard Function:

```
call()    [ Execute Frame ]    return    Frame destroyed, state lost.
```

Generator Function (Suspension & Resumption):

```
gen = my_gen()    Allocates generator frame on heap (GEN_CREATED)

next(gen)         Runs to 'yield X'    Suspends (GEN_SUSPENDED), yields X

gen.send(Y)       Resumes at 'yield', receives Y, runs to next yield...

return           Raises StopIteration (GEN_CLOSED), frame cleaned up.
```

The `yield from` Bi-Directional Pipe (PEP 380)

`yield from` is not merely a shorthand for `for item in subgen: yield item`. It opens an **unimpeded transparent bi-directional channel** between the outer caller and the inner subgenerator:

Caller	Subgenerator
	Direct Bi-Directional Channel
	(Values, exceptions, and returns flow directly; the delegating generator is completely bypassed!)

Minimal example

Save as `generator_delegation.py`:

```
# generator_delegation.py
from collections.abc import Generator, Iterator

# 1. Producer subgenerator
def counter_subgen(start: int, stop: int) -> Iterator[int]:
    """Yield integers from start up to stop."""
    current = start
    while current < stop:
        yield current
        current += 1

# 2. Delegating generator using 'yield from'
def compound_pipeline() -> Iterator[int | str]:
    print(" [Delegator] Starting pipeline...")
    # yield from establishes direct pipe to counter_subgen
    yield from counter_subgen(1, 4)
    yield "MIDPOINT"
    yield from counter_subgen(10, 13)
    print(" [Delegator] Pipeline finished.")

def main() -> None:
    for item in compound_pipeline():
        print(f"Pulled: {item}")

if __name__ == "__main__":
    main()
```

Run via `uv run python generator_delegation.py`:

```
[Delegator] Starting pipeline...
Pulled: 1
Pulled: 2
Pulled: 3
Pulled: MIDPOINT
Pulled: 10
Pulled: 11
Pulled: 12
[Delegator] Pipeline finished.
```

Worked examples

Case 1: Two-Way Coroutines for Rolling Averages with `.send()`

When `yield` appears on the right-hand side of an assignment (`received = yield output`), the generator can **consume data** sent by the caller via `gen.send(value)`:

```
# rolling_average.py
from collections.abc import Generator

def rolling_averager() -> Generator[float, float, None]:
    """Continuously receive incoming float samples and yield the cumulative average."""
    total = 0.0
    count = 0
    average = 0.0

    while True:
        # Suspends yielding current average; receives next incoming sample via .send()
        sample = yield average
        total += sample
        count += 1
        average = total / count

def main() -> None:
    calc = rolling_averager()

    # Priming the generator: Must advance to the first yield before sending data!
    next(calc) # Returns initial 0.0

    samples = [10.0, 20.0, 60.0, 10.0]
    print("Streaming Telemetry Samples to Coroutine:")
    for sample in samples:
        curr_avg = calc.send(sample)
        print(f"  Sent sample: {sample:4.1f} -> Current Rolling Average: {curr_avg:.2f}")

    # Cleanly terminate the coroutine
    calc.close()

if __name__ == "__main__":
    main()
```

Run:

```
uv run python rolling_average.py
```

Output:

Streaming Telemetry Samples to Coroutine:

```
Sent sample: 10.0 -> Current Rolling Average: 10.00
Sent sample: 20.0 -> Current Rolling Average: 15.00
Sent sample: 60.0 -> Current Rolling Average: 30.00
Sent sample: 10.0 -> Current Rolling Average: 25.00
```

Case 2: Deep Tree Traversal and Recursive Flattening with yield from

Nested data structures (such as nested JSON documents, file hierarchies, or AST nodes) can be flattened recursively without managing manual stack arrays:

```
# tree_flattener.py
from collections.abc import Iterator
from typing import Any

def flatten_tree(element: Any) -> Iterator[Any]:
    """Recursively flatten arbitrarily nested lists and primitives into a flat stream."""
    if isinstance(element, list):
        for item in element:
            # Recursively delegate to subgenerator
            yield from flatten_tree(item)
    else:
        yield element

def main() -> None:
    nested_topology = [
        "gateway-01",
        ["switch-a", ["node-01", "node-02"]],
        ["switch-b", ["node-03", ["storage-01", "storage-02"]]],
        "firewall-01",
    ]

    print("Flattened Infrastructure Inventory:")
    for device in flatten_tree(nested_topology):
        print(f" Device: {device}")

if __name__ == "__main__":
    main()
```

Run:

```
uv run python tree_flattener.py
```

Output:

Flattened Infrastructure Inventory:

```
Device: gateway-01
Device: switch-a
Device: node-01
Device: node-02
Device: switch-b
Device: node-03
Device: storage-01
Device: storage-02
Device: firewall-01
```

Case 3: Capturing Return Values from Subgenerators via `yield from`

In Python generators, a `return` value statement does not return to a caller like a regular function; it raises `StopIteration(value)`. When consumed by `yield from`, the subgenerator's return value is **automatically assigned to the target variable**:

```
# subgen_returns.py
from collections.abc import Generator

def accumulate_metrics() -> Generator[None, int, tuple[int, int]]:
    """Accumulate integers sent via .send(); return (count, total) when finished."""
    total = 0
    count = 0
    while True:
        val = yield
        if val is None: # Sentinel value signaling end of batch
            break
        total += val
        count += 1
    # The return value is embedded inside StopIteration
    return (count, total)

def batch_collector() -> Generator[None, int, str]:
    print(" [Collector] Awaiting batch...")
    # yield from captures the subgenerator's RETURN value directly!
    count, total = yield from accumulate_metrics()
    return f"Batch summary: Processed {count} items with sum {total}"

def main() -> None:
    collector = batch_collector()
    # Prime the generator
    collector.send(None)

    # Stream items into the subgenerator
```

```

for num in [15, 25, 60]:
    collector.send(num)

# Send sentinel None to trigger return
try:
    collector.send(None)
except StopIteration as exit_info:
    summary_result = exit_info.value
    print(f"\nFinal Result: {summary_result}")

if __name__ == "__main__":
    main()

```

Run:

```
uv run python subgen_returns.py
```

Output:

```
[Collector] Awaiting batch...
```

```
Final Result: Batch summary: Processed 3 items with sum 100
```

Pitfalls

Pitfall 1: Sending Data to an Unprimed Generator

Attempting to call `.send(value)` on a generator that has not yet been advanced to its first `yield` raises `TypeError`:

```

def my_coro():
    val = yield

c = my_coro()

# THE TRAP:
try:
    c.send("first_item") # TypeError!
except TypeError as err:
    print(f"Caught: {err}")

```

Output:

```
Caught: can't send non-None value to a just-started generator
```



```
# THE FIX: Prime with next() or .send(None)
c = my_coro()
next(c)          # Advanced to initial yield (ready to receive)
c.send("first_item")
```

Pitfall 2: Re-iterating an Exhausted Generator

Just like generator expressions, generator functions produce single-pass iterators. Once execution exits or returns, the generator is closed:

```
def generate_ids():
    yield 101
    yield 102

ids = generate_ids()
first_pass = list(ids) # [101, 102]
second_pass = list(ids) # []  <-- EMPTY! Generator is exhausted!

print(f"First: {first_pass}, Second: {second_pass}")
```

Exercises

1. Implement a generator function `tail_f(lines: list[str])` that yields lines one by one, simulating continuous streaming.
 2. Build a coroutine `running_median()` using `yield` and `bisect.insort` that receives numbers via `.send()` and yields the current median.
 3. Write a recursive generator function using `yield from` that traverses a directory structure on disk and yields all files ending in `.log`.
 4. Demonstrate how `.throw(RuntimeError("Abort"))` can be caught inside a generator function's `try ... except` block to execute cleanup logic.
 5. Create a generator that yields Fibonacci numbers up to `max_val`, and measure its memory usage compared to generating a full list of Fibonacci numbers.
-

Further reading

- PEP 342: *Coroutines via Enhanced Generators* (`.send()`, `.throw()`, `.close()`).
- PEP 380: *Syntax for Delegating to a Subgenerator* (`yield from`).
- Luciano Ramalho: *Fluent Python* (Chapter 16: Coroutines and Generators).
- David Beazley: *Generators: The Sneaky Path to Advanced Python*.

Control Flow

The if, elif, and else Statements

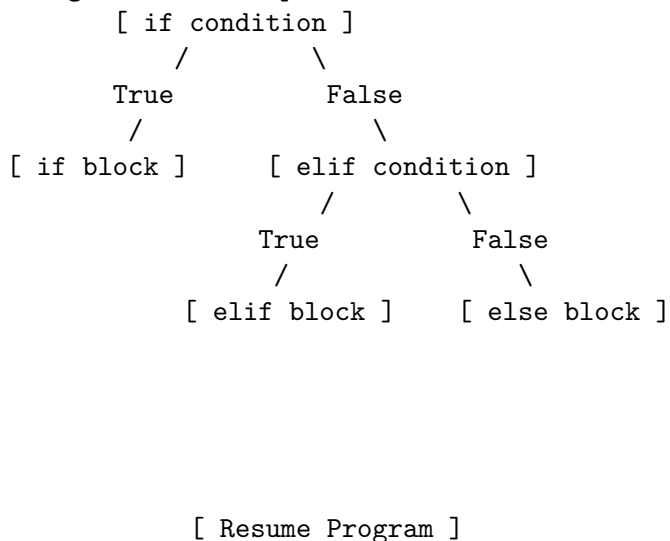
The if, elif, and else Statements

After reading this chapter, you will master multi-branch decision trees using `if`, `elif`, and `else`, evaluate truthiness and falsy values across standard types, leverage short-circuiting for defensive execution, and apply the Bouncer pattern to eliminate deeply nested conditional logic.

Mental model

Python evaluates conditional blocks linearly from top to bottom. As soon as a branch condition evaluates to a truthy value, CPython executes that branch's block and immediately skips all subsequent `elif` and `else` blocks:

Branching Execution Pipeline:



Short-Circuit Evaluation Guarding

Logical operators `and` and `or` stop evaluating operands as soon as the outcome is mathematically guaranteed:

- `A and B`: If `A` is falsy, Python returns `A` immediately without ever evaluating `B`.
- `A or B`: If `A` is truthy, Python returns `A` immediately without ever evaluating `B`.

Defensive Guard Pipeline:

```
user is not None and user.is_active and check_permissions(user)
```

```
    If user is None      Short-circuits! Never queries user.is_active
    If user is not None
```

```
        If not active    Short-circuits! Never calls check_permissions()
```

Minimal example

Save as `if_decision_trees.py`:

```
# if_decision_trees.py
def classify_http_status(status_code: int) -> str:
    """Classify HTTP response codes using a clean decision ladder."""
    if 200 <= status_code < 300:
        return "SUCCESS"
    elif 300 <= status_code < 400:
        return "REDIRECTION"
    elif 400 <= status_code < 500:
        return "CLIENT_ERROR"
    elif 500 <= status_code < 600:
        return "SERVER_ERROR"
    else:
        return "INVALID_STATUS"

def main() -> None:
    test_codes = [200, 301, 404, 503, 999]
    for code in test_codes:
        category = classify_http_status(code)
        print(f"HTTP {code:3d} -> Category: {category}")

if __name__ == "__main__":
    main()
```

Run via `uv run python if_decision_trees.py`:

```
HTTP 200 -> Category: SUCCESS
HTTP 301 -> Category: REDIRECTION
HTTP 404 -> Category: CLIENT_ERROR
HTTP 503 -> Category: SERVER_ERROR
HTTP 999 -> Category: INVALID_STATUS
```

Worked examples

Case 1: The Bouncer Pattern (Guard Clauses vs Arrow Anti-Pattern)

Deeply nested if statements create the “Arrow Anti-Pattern” (code indenting further and further to the right). The **Bouncer Pattern** inverts checks into early returns, keeping the happy path at indentation level 0:

```
# bouncer_pattern.py
from typing import Any

# THE ANTI-PATTERN (Arrow Code):
def process_request_nested(request: dict[str, Any]) -> str:
    if request:
        if "user" in request:
            if request["user"].get("is_authenticated"):
                if request["user"].get("has_permission"):
                    return "Action Authorized"
                else:
                    return "Forbidden: Missing Permission"
            else:
                return "Unauthorized: User Not Authenticated"
        else:
            return "Bad Request: No User Field"
    else:
        return "Bad Request: Empty Payload"

# THE IDIOMATIC PATTERN (Bouncer Clauses):
def process_request_clean(request: dict[str, Any]) -> str:
    # Guard 1: Empty request
    if not request:
        return "Bad Request: Empty Payload"

    # Guard 2: Missing user
    user = request.get("user")
    if not user:
        return "Bad Request: No User Field"

    # Guard 3: Authentication
    if not user.get("is_authenticated"):
        return "Unauthorized: User Not Authenticated"

    # Guard 4: Permission
    if not user.get("has_permission"):
        return "Forbidden: Missing Permission"

    # Happy path at top-level indentation
```

```

        return "Action Authorized"

if __name__ == "__main__":
    sample_request = {
        "user": {
            "is_authenticated": True,
            "has_permission": True,
        }
    }
    print("Nested result:", process_request_nested(sample_request))
    print("Clean result :", process_request_clean(sample_request))

```

Run:

```
uv run python bouncer_pattern.py
```

Output:

```

Nested result: Action Authorized
Clean result : Action Authorized

```

Case 2: Truthiness and The Numeric 0 Trap

Every Python object possesses a boolean truth value. In Python, the following objects evaluate to **False**: - None and False - Numeric zeros: 0, 0.0, 0j - Empty containers: "", (), [], {}, set()

Everything else evaluates to **True**. When validating numeric metrics, checking `if not metric` incorrectly treats 0 as an error:

```

# falsy_metric_audit.py
def record_dropped_packets(count: int | None) -> None:
    # DANGEROUS: If count is 0, 'not count' evaluates to True!
    if not count:
        print(f"[Naive Check] Invalid or zero metric: {count}")

    # SAFE: Explicitly check for None
    if count is None:
        print("[Safe Check] Metric was omitted (None).")
    else:
        print(f"[Safe Check] Valid metric recorded: {count} dropped packets.")

if __name__ == "__main__":
    record_dropped_packets(0)      # Zero is a perfectly healthy metric!
    record_dropped_packets(None)  # Truly missing metric

```

Run:


```
uv run python falsy_metric_audit.py
```

Output:

```
[Naive Check] Invalid or zero metric: 0
[Safe Check] Valid metric recorded: 0 dropped packets.
[Naive Check] Invalid or zero metric: None
[Safe Check] Metric was omitted (None).
```

Pitfalls

Pitfall 1: Comparing Explicitly to True or False

```
# ANTI-PATTERN:
if is_ready == True:
    pass

# ANTI-PATTERN:
if is_ready is True:
    pass

# IDIOMATIC PYTHON:
if is_ready:
    pass
```

Direct comparisons to `True` break custom classes implementing `__bool__()` and violate Pythonic conventions.

Pitfall 2: Combining `and` with or Without Parentheses

Python evaluates `not` before `and`, and `and` before `or`. Ambiguous unparenthesized chains cause security bypasses:

```
# AMBIGUOUS:
# is_admin or is_manager and is_verified
# Python parses this as: is_admin or (is_manager and is_verified)

# EXPLICIT AND SAFE: Always group multi-operator logic
is_allowed = (is_admin or is_manager) and is_verified
```

Exercises

1. Write an `if/elif/else` function `classify_temperature(temp_c: float) -> str` that categorizes temperatures into "FREEZING", "COLD", "MODERATE", and "HOT" using chained comparisons (`0.0 <= temp_c < 15.0`).
 2. Refactor a three-level nested dictionary check into guard clauses using the Bouncer pattern.
 3. Demonstrate why `bool([])` is `False` while `bool([0])` is `True`.
 4. Write a conditional check that safely inspects `sys.argv[1]` only if `len(sys.argv) > 1` using short-circuit evaluation.
-

Further reading

- Python Language Reference: *The if statement*.
- Python Standard Library: *Truth Value Testing*.
- PEP 8: *Style Guide for Python Code* — *Programming Recommendations for if statements*.

Conditional Expressions and the Walrus Operator

Conditional Expressions and the Walrus Operator

After reading this chapter, you will master Python's inline conditional expression (`X if C else Y`), leverage the walrus operator (`:=`) to bind variables directly inside conditional checks, evaluate expression precedence, and recognize when inline logic improves clarity versus when it degrades readability.

Mental model

Unlike an **if/else statement** (which directs control flow across code blocks), a **conditional expression** (often called the ternary operator) evaluates to a single value that can be assigned directly:

Statement vs Expression:

```
Statement (Imperative):
    if latency > 100.0:
        status = "CRITICAL"
    else:
        status = "HEALTHY"
```

```
Conditional Expression (Declarative):
    status = "CRITICAL" if latency > 100.0 else "HEALTHY"
```

True Val	Condition	False Val
----------	-----------	-----------

The Walrus Operator Pipeline (`:=`)

Introduced in PEP 572, the assignment expression (`:=`) assigns a value to a variable *inside* an expression, allowing you to test and capture a result in a single step:

Without Walrus Operator (Two Steps):

```
match = pattern.search(text)
if match is not None:
    handle(match)
```

With Walrus Operator (Single Step):

```
if (match := pattern.search(text)) is not None:
    handle(match)
```

Minimal example

Save as `conditional_expressions.py`:

```
# conditional_expressions.py
import re

def main() -> None:
    # 1. Inline Conditional Expression (Ternary)
    port_input: int | None = None
    # If port_input is None, fall back to 8080
    active_port = port_input if port_input is not None else 8080
    print(f"Active server port: {active_port}")

    # 2. Walrus Operator in Conditional Branching
    log_line = "2026-09-07 [ERROR] Database connection refused to 10.0.1.5"
    error_pattern = re.compile(r"\[ERROR\]\s+(.*)")

    # Assigns 'match' and checks truthiness simultaneously
    if (match := error_pattern.search(log_line)):
        error_msg = match.group(1)
        print(f"Captured alert via walrus operator: '{error_msg}'")
    else:
        print("No error detected in log line.")

if __name__ == "__main__":
    main()
```

Run via `uv run python conditional_expressions.py`:

Active server port: 8080

Captured alert via walrus operator: 'Database connection refused to 10.0.1.5'

Worked examples

Case 1: Streamlining Configuration Fallbacks

When computing default settings where an empty string or `None` is provided, a conditional expression keeps initialization concise and readable:

```
# config_fallback.py
import os

def resolve_cluster_endpoint(env_override: str | None) -> str:
```

```

# Standard ternary fallback
endpoint = env_override.strip() if env_override and env_override.strip() else "https://d
return endpoint

if __name__ == "__main__":
    print("Omitted endpoint :", resolve_cluster_endpoint(None))
    print("Whitespace input :", resolve_cluster_endpoint(" "))
    print("Custom endpoint :", resolve_cluster_endpoint("https://custom.prod:9000"))

```

Run:

```
uv run python config_fallback.py
```

Output:

```

Omitted endpoint : https://default.internal:8443
Whitespace input : https://default.internal:8443
Custom endpoint  : https://custom.prod:9000

```

Case 2: Avoiding Duplicate Computations with the Walrus Operator

When a condition depends on an expensive operation (such as reading a buffer, parsing an AST, or calculating an aggregation), the walrus operator avoids calling the function twice:

```

# stream_chunk_reader.py
import io

def process_stream(data_stream: io.StringIO) -> None:
    # Read and test chunks in a single expression
    chunk_counter = 0
    while (chunk := data_stream.read(16)):
        chunk_counter += 1
        print(f"Chunk {chunk_counter}: '{chunk}'")

if __name__ == "__main__":
    payload = io.StringIO("ALPHA_STREAM_01_BETA_STREAM_02_GAMMA_STREAM_03")
    process_stream(payload)

```

Run:

```
uv run python stream_chunk_reader.py
```

Output:

```

Chunk 1: 'ALPHA_STREAM_01_'
Chunk 2: 'BETA_STREAM_02_G'
Chunk 3: 'AMMA_STREAM_03'

```

Pitfalls

Pitfall 1: The Nested Ternary Spaghetti Trap

Chaining multiple conditional expressions into a single line destroys readability:

```
# DANGEROUS / UNREADABLE:
status = "CRITICAL" if lat > 500 else "DEGRADED" if lat > 200 else "ACCEPTABLE" if lat > 50

# IDIOMATIC: Use a standard if/elif/else ladder or dictionary lookup
if lat > 500:
    status = "CRITICAL"
elif lat > 200:
    status = "DEGRADED"
elif lat > 50:
    status = "ACCEPTABLE"
else:
    status = "OPTIMAL"
```

Rule of thumb: **Never nest conditional expressions.** If a condition requires more than one `if` and one `else`, use a multi-line `if/elif/else` block.

Pitfall 2: Precedence Involving `or` and Ternaries

Conditional expressions have very low operator precedence:

```
# 'a or b if condition else c'
# Is evaluated as: '(a or b) if condition else c'
# NOT: 'a or (b if condition else c)'
# Always use parentheses around the ternary expression when mixing with boolean operators:
result = a or (b if condition else c)
```

Exercises

1. Refactor a 4-line `if/else` block assigning an access token into a single inline conditional expression.
2. Use the walrus operator `:=` inside an `if` statement to compute the length of a list and verify that it exceeds 5 elements without computing `len()` twice.
3. Write a function that accepts an integer port and uses a conditional expression to return "SECURE" if port is 443 or 8443, otherwise "STANDARD".

4. Demonstrate how the walrus operator simplifies reading lines from a text file until an empty string EOF is encountered.
-

Further reading

- PEP 308: *Conditional Expressions*.
- PEP 572: *Assignment Expressions (The Walrus Operator)*.
- Python Language Reference: *Expressions — Conditional expressions*.

The while Loop and State-Driven Execution

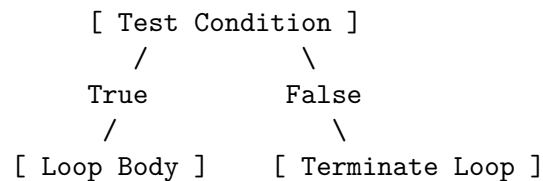
The while Loop and State-Driven Execution

After reading this chapter, you will master state-driven iteration using the `while` loop, configure daemon loops with `while True`, implement robust polling algorithms with exponential backoff, handle stream termination sentinels, and prevent runaway CPU-consuming infinite loops.

Mental model

A `while` loop repeatedly executes a block of code as long as a boolean test condition evaluates to `True`. The condition is checked at the **entry of every iteration**:

The while Loop Execution Cycle:



If the condition evaluates to `False` on the very first evaluation, the loop body **never executes**.

Minimal example

Save as `while_loop_mechanics.py`:

```
# while_loop_mechanics.py
import time

def poll_node_readiness(target_node: str, max_retries: int = 4) -> bool:
    """Poll a cluster node until it reports ready or retries are exhausted."""
    # Simulated cluster states: 0=initializing, 1=starting, 2=healthy
    mock_states = ["INITIALIZING", "STARTING", "READY"]
    attempt = 0
```

```

print(f"Polling node '{target_node}'...")
while attempt < max_retries:
    attempt += 1
    # Retrieve current mock state
    current_state = mock_states[attempt - 1] if attempt - 1 < len(mock_states) else "READY"
    print(f"  Attempt {attempt}/{max_retries}: State is '{current_state}'")

    if current_state == "READY":
        print("  Node is ready! Exiting polling loop.")
        return True

    # Polling delay
    time.sleep(0.02)

print("  Failed: Maximum retries exceeded.")
return False

def main() -> None:
    is_ready = poll_node_readiness("k8s-worker-01", max_retries=4)
    print(f"Final readiness status: {is_ready}")

if __name__ == "__main__":
    main()

```

Run via `uv run python while_loop_mechanics.py`:

```

Polling node 'k8s-worker-01'...
  Attempt 1/4: State is 'INITIALIZING'
  Attempt 2/4: State is 'STARTING'
  Attempt 3/4: State is 'READY'
  Node is ready! Exiting polling loop.
Final readiness status: True

```

Worked examples

Case 1: Exponential Backoff with Jitter for Network Retries

In production distributed systems, polling an overloaded server at fixed intervals worsens congestion. An exponential backoff loop doubles the wait duration on each attempt:

```

# exponential_backoff.py
import time

```

```

def call_unstable_service(attempt: int) -> bool:
    # Simulates recovery on 4th attempt
    return attempt >= 4

def retry_with_backoff(max_attempts: int = 5, base_delay: float = 0.01) -> bool:
    attempt = 1
    delay = base_delay

    while attempt <= max_attempts:
        print(f"[Attempt {attempt}] Contacting payment gateway...")
        success = call_unstable_service(attempt)

        if success:
            print(f"[Success] Gateway response received on attempt {attempt}!")
            return True

        print(f"  Transient failure. Backing off for {delay*1000:.1f}ms...")
        time.sleep(delay)

        # Exponential backoff: double the delay duration
        delay *= 2
        attempt += 1

    return False

if __name__ == "__main__":
    retry_with_backoff(max_attempts=5, base_delay=0.01)

```

Run:

```
uv run python exponential_backoff.py
```

Output:

```

[Attempt 1] Contacting payment gateway...
  Transient failure. Backing off for 10.0ms...
[Attempt 2] Contacting payment gateway...
  Transient failure. Backing off for 20.0ms...
[Attempt 3] Contacting payment gateway...
  Transient failure. Backing off for 40.0ms...
[Attempt 4] Contacting payment gateway...
[Success] Gateway response received on attempt 4!

```

Case 2: State-Machine Dispatcher Loop

while loops are the foundation of state-machine processing, where the loop continues running until a terminal state is reached:

```

# state_machine_loop.py
from enum import Enum

class State(Enum):
    PENDING = "pending"
    PROCESSING = "processing"
    VERIFYING = "verifying"
    COMPLETED = "completed"
    FAILED = "failed"

def run_job_pipeline() -> None:
    current_state = State.PENDING
    step_count = 0

    print("Initiating state machine loop:")
    while current_state not in (State.COMPLETED, State.FAILED):
        step_count += 1
        print(f" Step {step_count}: Current state is '{current_state.value}'")

        if current_state == State.PENDING:
            current_state = State.PROCESSING
        elif current_state == State.PROCESSING:
            current_state = State.VERIFYING
        elif current_state == State.VERIFYING:
            current_state = State.COMPLETED

    print(f"Terminal state reached: '{current_state.value}' after {step_count} steps.")

if __name__ == "__main__":
    run_job_pipeline()

```

Run:

```
uv run python state_machine_loop.py
```

Output:

```

Initiating state machine loop:
Step 1: Current state is 'pending'
Step 2: Current state is 'processing'
Step 3: Current state is 'verifying'
Terminal state reached: 'completed' after 3 steps.

```

Pitfalls

Pitfall 1: The Accidental Runaway Infinite Loop

If the loop variable is not updated within the loop body, the condition remains permanently `True`, consuming 100% of a CPU core:

```
# FATAL BUG:
count = 5
while count > 0:
    print(count)
    # BUG: Forgot count -= 1! Runs forever!

# FIX:
while count > 0:
    print(count)
    count -= 1
```

Pitfall 2: Busy-Waiting Without Sleeping

A `while` loop that checks a flag or condition without yielding (`time.sleep()` or `await asyncio.sleep()`) creates a **busy-wait** loop that starves other threads and overheats the CPU. Always insert a sleep or use an event primitive (`threading.Event.wait()`).

Exercises

1. Write a `while` loop that implements Euclid's algorithm to find the greatest common divisor (GCD) of two numbers *A* and *B*.
 2. Implement a countdown loop that starts at 10 and decrements down to 1, printing "BLASTOFF!" after the loop finishes.
 3. Write a sentinel-controlled `while` loop that pops elements from a queue list until an element with `status == "DONE"` is encountered.
 4. Implement an exponential backoff loop that caps the maximum delay at 1.0 second.
-

Further reading

- Python Language Reference: *The `while` statement*.
- Python Standard Library: `time.sleep` and `threading.Event`.

The for Loop and Sequence Iteration

The for Loop and Sequence Iteration

After reading this chapter, you will master sequence and collection iteration using the **for** loop, understand how the Python Iterator Protocol operates under the hood, generate numerical sequences using `range()`, iterate dictionaries across keys and items, and avoid modifying collections during iteration.

Mental model

Unlike C-style for loops (`for (int i=0; i<n; i++)`), Python's for loop is an abstraction over the **Iterator Protocol**. It operates over any **iterable** (lists, tuples, dictionaries, sets, generators, strings, open file handles):

How CPython Executes "for item in collection:"

```
1. Calls iterator = iter(collection)      Calls collection.__iter__()
2. Enters internal loop:
    try:
        item = next(iterator)              Calls iterator.__next__()
        [ Execute Loop Body ]
    except StopIteration:
        break                               Terminates cleanly!
```

Because `range(start, stop, step)` generates numbers on demand without allocating an actual list in memory, `range(1_000_000_000)` consumes only 48 bytes of RAM!

Minimal example

Save as `for_loop_mechanics.py`:

```
# for_loop_mechanics.py
import sys

def main() -> None:
    # 1. Iterating across a list of cluster nodes
    nodes = ["k8s-node-01", "k8s-node-02", "k8s-node-03"]
    print("--- Node Iteration ---")
```



```

for node in nodes:
    print(f"Active node: {node}")

# 2. Dictionary iteration via .items()
node_metrics = {"k8s-node-01": 12.4, "k8s-node-02": 45.1, "k8s-node-03": 88.0}
print("\n--- Dictionary Key-Value Unpacking ---")
for hostname, load in node_metrics.items():
    print(f"Host: {hostname:12} | CPU Load: {load:4.1f}%")

# 3. range() step and stride
print("\n--- Range Stride (Even ports from 8000 to 8006) ---")
for port in range(8000, 8008, 2):
    print(f"Allocating port: {port}")

# Inspecting range memory efficiency
massive_range = range(1_000_000_000)
print(f"\nMemory size of range(1,000,000,000): {sys.getsizeof(massive_range)} bytes")

if __name__ == "__main__":
    main()

```

Run via `uv run python for_loop_mechanics.py`:

```

--- Node Iteration ---
Active node: k8s-node-01
Active node: k8s-node-02
Active node: k8s-node-03

--- Dictionary Key-Value Unpacking ---
Host: k8s-node-01 | CPU Load: 12.4%
Host: k8s-node-02 | CPU Load: 45.1%
Host: k8s-node-03 | CPU Load: 88.0%

--- Range Stride (Even ports from 8000 to 8006) ---
Allocating port: 8000
Allocating port: 8002
Allocating port: 8004
Allocating port: 8006

Memory size of range(1,000,000,000): 48 bytes

```

Worked examples

Case 1: Deconstructing the Iterator Protocol Manually

To understand what Python does behind the scenes during a `for` loop, you can execute the exact same protocol manually using `iter()` and `next()`:

```
# manual_iterator_demo.py
def demonstrate_manual_iteration() -> None:
    items = ["auth-svc", "billing-svc", "gateway-svc"]

    # 1. Obtain iterator object
    iterator = iter(items)
    print("Iterator object created:", iterator)

    # 2. Emulate the for-loop using while and StopIteration
    print("\nExecuting manual loop:")
    while True:
        try:
            item = next(iterator)
            print(f"  Pulled item from iterator: {item}")
        except StopIteration:
            print("  StopIteration raised! CPython caught it and terminated loop.")
            break

if __name__ == "__main__":
    demonstrate_manual_iteration()
```

Run:

```
uv run python manual_iterator_demo.py
```

Output:

```
Iterator object created: <list_iterator object at ...>
```

```
Executing manual loop:
```

```
  Pulled item from iterator: auth-svc
```

```
  Pulled item from iterator: billing-svc
```

```
  Pulled item from iterator: gateway-svc
```

```
  StopIteration raised! CPython caught it and terminated loop.
```

Case 2: File Iteration Line by Line (Constant Memory)

An open file object is an iterable that yields one line at a time on demand. Iterating directly over the file handle reads from disk incrementally without ever loading the full file into RAM:

```

# file_line_streamer.py
import tempfile
from pathlib import Path

def process_large_logfile(log_path: Path) -> int:
    error_count = 0
    # Opening the file returns an iterable stream
    with log_path.open("r", encoding="utf-8") as stream:
        for line in stream:
            if "[ERROR]" in line:
                error_count += 1
    return error_count

if __name__ == "__main__":
    with tempfile.TemporaryDirectory() as tmpdir:
        dummy_log = Path(tmpdir) / "cluster.log"
        dummy_log.write_text(
            "2026-09-07 [INFO] Started\n"
            "2026-09-07 [ERROR] Database timeout\n"
            "2026-09-07 [INFO] Retrying\n"
            "2026-09-07 [ERROR] Auth rejected\n"
        )

        total_errors = process_large_logfile(dummy_log)
        print(f"Total error lines identified: {total_errors}")

```

Run:

```
uv run python file_line_streamer.py
```

Output:

```
Total error lines identified: 2
```

Pitfalls

Pitfall 1: Modifying a Collection While Iterating Over It

Mutating a list while iterating over it causes internal index shifts that skip items or loop indefinitely:

```

# THE BUG:
numbers = [1, 2, 3, 4, 5, 6]
for n in numbers:

```

```

    if n % 2 == 0:
        numbers.remove(n) # Skips elements during iteration!
print(numbers) # [1, 3, 5] by coincidence, but elements like 4 or 6 get skipped on other da

# THE FIX: Iterate over a shallow copy or use a list comprehension
numbers = [n for n in numbers if n % 2 != 0]

```

For dictionaries, attempting to add or remove keys during iteration immediately crashes with `RuntimeError: dictionary changed size during iteration`.

Pitfall 2: The `range(len(sequence))` Anti-Pattern

Programmers coming from C or Java frequently write `for i in range(len(items)): val = items[i]`. This is unpythonic, verbose, and error-prone. Iterate directly over `items`:

```

# ANTI-PATTERN:
for i in range(len(servers)):
    print(servers[i])

# IDIOMATIC:
for server in servers:
    print(server)

```

If you need the index, use `enumerate(servers)` (covered in chapter 6).

Exercises

1. Write a `for` loop that iterates over a dictionary and prints only the keys whose values exceed 100.
 2. Generate all multiples of 5 between 50 and 100 in reverse order using `range()`.
 3. Demonstrate iterating over a string character by character to count how many vowels it contains.
 4. Implement a custom class with `__iter__` and `__next__` that yields squares of numbers up to a maximum limit.
-

Further reading

- Python Language Reference: *The `for` statement*.
- Python Data Model: *The Iterator Protocol* (`__iter__`, `__next__`).

Loop Control: break, continue, and Loop else

Loop Control: break, continue, and Loop else

After reading this chapter, you will master loop execution interruption using `break` and `continue`, eliminate boolean search flags using the loop `else` clause, break out of nested loop structures cleanly, and handle pipeline filtering without nested conditional blocks.

Mental model

Python provides three control keywords that alter standard loop iteration:

Loop Control Keywords:

1. `continue` Immediately skips the remaining statements in the current iteration; jumps directly to the next iteration evaluation.
2. `break` Immediately aborts the entire loop; jumps to the first statement outside the loop.
3. `else` Executes IF AND ONLY IF the loop finished all iterations without hitting an explicit `break` statement!

The Loop 'else' Contract:

```
for item in sequence:
    if item == target:
        break                (Bypasses loop 'else'!)
else:
    # Runs ONLY if target was NOT found
    print("Item missing!")
```

[Resume Outer Execution]

Minimal example

Save as `loop_control_statements.py`:

```
# loop_control_statements.py
def inspect_packet_batch(packets: list[dict[str, str | int]]) -> None:
    print(f"Inspecting batch of {len(packets)} network packets...")
```

```

for pkt in packets:
    # 1. 'continue' skips empty or malformed packets
    if pkt.get("size", 0) <= 0:
        print(f"    [Skip] Packet {pkt.get('id')} has 0 bytes. Continuing.")
        continue

    # 2. 'break' immediately terminates loop on critical security violation
    if pkt.get("flags") == "MALICIOUS":
        print(f"    [SECURITY ALERT] Malicious payload detected on ID {pkt['id']}! Terminating.")
        break

    print(f"    [OK] Packet {pkt['id']} verified ({pkt['size']} bytes).")
else:
    # 3. Loop 'else' executes ONLY if loop completed without hitting 'break'
    print("    [SUCCESS] All packets verified clean. Batch approved for forwarding.")

def main() -> None:
    # Test clean batch
    clean_batch = [
        {"id": 101, "size": 64, "flags": "SYN"},
        {"id": 102, "size": 0, "flags": "ACK"}, # Skipped via continue
        {"id": 103, "size": 128, "flags": "ACK"},
    ]
    print("--- Test 1: Clean Batch ---")
    inspect_packet_batch(clean_batch)

    # Test malicious batch
    dirty_batch = [
        {"id": 201, "size": 64, "flags": "SYN"},
        {"id": 202, "size": 512, "flags": "MALICIOUS"}, # Aborted via break
        {"id": 203, "size": 64, "flags": "ACK"},
    ]
    print("\n--- Test 2: Dirty Batch ---")
    inspect_packet_batch(dirty_batch)

if __name__ == "__main__":
    main()

```

Run via `uv run python loop_control_statements.py`:

```

--- Test 1: Clean Batch ---
Inspecting batch of 3 network packets...
[OK] Packet 101 verified (64 bytes).
[Skip] Packet 102 has 0 bytes. Continuing.
[OK] Packet 103 verified (128 bytes).
[SUCCESS] All packets verified clean. Batch approved for forwarding.

```

```
--- Test 2: Dirty Batch ---
Inspecting batch of 3 network packets...
[OK] Packet 201 verified (64 bytes).
[SECURITY ALERT] Malicious payload detected on ID 202! Terminating inspection immediately.
```

Worked examples

Case 1: Search and Validate Without Boolean Flags (for ... else)

In languages like C or Java, checking whether a list contains an item matching specific criteria requires setting a boolean flag (`found = False`). Python's loop `else` eliminates this boilerplate:

```
# target_finder_clean.py
def find_available_worker(workers: list[dict[str, str | int]]) -> None:
    # Search for an idle worker with CPU load under 20%
    for w in workers:
        if w["status"] == "IDLE" and w["cpu_pct"] < 20:
            print(f"Dispatching task to worker: {w['name']} (CPU: {w['cpu_pct']}%)")
            break
    else:
        # Executes only if no worker matched
        print("CAPACITY ALERT: No available idle worker found across cluster!")

if __name__ == "__main__":
    cluster = [
        {"name": "worker-01", "status": "BUSY", "cpu_pct": 85},
        {"name": "worker-02", "status": "IDLE", "cpu_pct": 95},
        {"name": "worker-03", "status": "BUSY", "cpu_pct": 40},
    ]
    find_available_worker(cluster)
```

Run:

```
uv run python target_finder_clean.py
```

Output:

```
CAPACITY ALERT: No available idle worker found across cluster!
```

Case 2: Breaking Out of Nested Loops

A `break` statement only terminates the **innermost** loop. To exit multiple nested loops simultaneously without clumsy flag variables, encapsulate the nested loops in a function and use `return`:


```
# nested_loop_exit.py
def locate_corrupted_block(matrix: list[list[int]]) -> tuple[int, int] | None:
    """Scan a 2D matrix and return (row, col) coordinates of first negative value."""
    for row_idx, row in enumerate(matrix):
        for col_idx, value in enumerate(row):
            if value < 0:
                # 'return' instantly exits BOTH loops cleanly!
                return (row_idx, col_idx)
    return None

if __name__ == "__main__":
    storage_grid = [
        [100, 200, 300],
        [400, -999, 600], # Corrupted block at row 1, col 1
        [700, 800, 900],
    ]
    coord = locate_corrupted_block(storage_grid)
    print(f"Corrupted block coordinates: {coord}")
```

Run:

```
uv run python nested_loop_exit.py
```

Output:

Corrupted block coordinates: (1, 1)

Pitfalls

Pitfall 1: Assuming `break` Exits All Nested Loops

Calling `break` inside an inner loop returns control to the *outer* loop, not the code outside the outer loop:

```
for outer in range(3):
    for inner in range(3):
        if inner == 1:
            break # Exits 'inner' loop only! 'outer' continues iterating!
```

To exit both, encapsulate the nested loops inside a function and use `return`, or use a custom exception.

Pitfall 2: Confusing Loop `else` with `if else`

A loop `else` does **not** mean “run if the loop didn’t execute”. If a loop iterates over an empty list (`for x in []:`), it completes 0 iterations without hitting `break`, so the `else` block **does execute**! Think of loop `else` as “no-break”.

Exercises

1. Write a prime number checker using `for ... in range(2, n): ... break else:` to determine whether n is prime without boolean flags.
 2. Given a list of log entries, use `continue` to filter out all entries that do not contain "ALERT" or "CRITICAL".
 3. Demonstrate that iterating over an empty list `for item in []: break else: print("RUN")` executes the `else` block.
 4. Implement a function that scans a 3D tensor and uses `return` to break out of all three nested loops when a NaN value is encountered.
-

Further reading

- Python Language Reference: *The `break` and `continue` statements*.
- Raymond Hettinger: *Transforming Code into Beautiful, Idiomatic Python (The for-else idiom)*.

Iteration Utilities: enumerate and zip

Iteration Utilities: enumerate and zip

After reading this chapter, you will master Python's built-in iteration helpers: track loop indices cleanly without manual counter variables using `enumerate()`, iterate multiple parallel streams simultaneously using `zip()`, enforce length symmetry using `strict=True`, and handle unequal lengths with `itertools.zip_longest()`.

Mental model

In low-level languages, iterating across parallel arrays or tracking line numbers requires manual integer incrementation (`i++`). Python replaces manual counters with lazy iterator wrappers:

Manual Index Management (Error-Prone):

```
i = 0
for item in items:
    print(i, item)
    i += 1
```

Easy to forget, off-by-one errors

`enumerate(items, start=1)` Pipeline:

```
items: [ "web", "api", "db" ]
```

Yields: (1, "web") (2, "api") (3, "db") (Zero overhead)

Multi-Stream Parallel Iteration (`zip`):

```
Stream 1: [ "node-1", "node-2", "node-3" ]
Stream 2: [ "10.0.1.1", "10.0.1.2", "10.0.1.3" ]
```

```
zip:      ("node-1", ("node-2", ("node-3",
                                "10.0.1.1") "10.0.1.2") "10.0.1.3")
```

Minimal example

Save as `iteration_utilities.py`:

```

# iteration_utilities.py
def main() -> None:
    # 1. Indexed iteration with enumerate(start=1)
    services = ["auth-gateway", "billing-worker", "orders-api"]
    print("--- Numbered Service Manifest ---")
    for index, service in enumerate(services, start=1):
        print(f"Service #{index:02d}: {service}")

    # 2. Parallel lock-step iteration with zip(..., strict=True)
    hosts = ["node-01", "node-02", "node-03"]
    ips = ["10.0.1.10", "10.0.1.11", "10.0.1.12"]
    roles = ["primary", "replica", "replica"]

    print("\n--- Synchronized Cluster Inventory ---")
    # strict=True guarantees ValueError if any list length differs
    for host, ip, role in zip(hosts, ips, roles, strict=True):
        print(f"Host: {host:8} | IP: {ip:10} | Role: {role}")

if __name__ == "__main__":
    main()

```

Run via `uv run python iteration_utilities.py`:

```

--- Numbered Service Manifest ---
Service #01: auth-gateway
Service #02: billing-worker
Service #03: orders-api

--- Synchronized Cluster Inventory ---
Host: node-01 | IP: 10.0.1.10 | Role: primary
Host: node-02 | IP: 10.0.1.11 | Role: replica
Host: node-03 | IP: 10.0.1.12 | Role: replica

```

Worked examples

Case 1: Catching Silent Data Truncation with `strict=True`

In Python 3.9 and earlier, `zip()` silently stopped iterating as soon as the *shortest* sequence was exhausted, dropping trailing elements without any error. PEP 618 introduced `strict=True`:

```

# strict_zip_verification.py
def verify_cluster_mapping(hostnames: list[str], ip_addresses: list[str]) -> None:
    try:
        # If one list has 3 items and the other has 2, strict=True raises ValueError

```

```

        for host, ip in zip(hostnames, ip_addresses, strict=True):
            print(f"Mapping: {host} -> {ip}")
    except ValueError as exc:
        print(f"CRITICAL DRIFT: Configuration mismatch detected: {exc}")

if __name__ == "__main__":
    configured_hosts = ["web-01", "web-02", "web-03"]
    assigned_ips      = ["192.168.1.10", "192.168.1.11"] # Missing 3rd IP!

    print("Attempting to pair hosts and IPs with strict=True:")
    verify_cluster_mapping(configured_hosts, assigned_ips)

```

Run:

```
uv run python strict_zip_verification.py
```

Output:

```

Attempting to pair hosts and IPs with strict=True:
Mapping: web-01 -> 192.168.1.10
Mapping: web-02 -> 192.168.1.11
CRITICAL DRIFT: Configuration mismatch detected: zip() argument 2 is shorter than argument 1

```

Case 2: Unequal Streams with Padding Using `itertools.zip_longest`

When sequence lengths differ intentionally and you wish to fill missing values with a placeholder, use `itertools.zip_longest()`:

```

# zip_longest_demo.py
import itertools

def display_table_with_padding() -> None:
    headers = ["Service", "Primary Host", "Backup Host"]
    col1 = ["auth", "billing"]
    col2 = ["auth-01", "bill-01", "order-01"]
    col3 = ["auth-bak"]

    print("Padded multi-column output:")
    for svc, primary, backup in itertools.zip_longest(col1, col2, col3, fillvalue="[NONE]"):
        print(f" {svc:10} | Primary: {primary:10} | Backup: {backup:10}")

if __name__ == "__main__":
    display_table_with_padding()

```

Run:

```
uv run python zip_longest_demo.py
```

Output:

Padded multi-column output:

auth	Primary: auth-01	Backup: auth-bak
billing	Primary: bill-01	Backup: [NONE]
[NONE]	Primary: order-01	Backup: [NONE]

Pitfalls

Pitfall 1: Relying on Default `zip()` Without `strict=True`

Omitting `strict=True` in production code causes subtle bugs where data streams (like headers vs rows in CSVs, or keys vs values) fail to match, silently corrupting or discarding records. Always write `zip(..., strict=True)`.

Pitfall 2: Re-Iterating a `zip` or `enumerate` Object

Both `zip()` and `enumerate()` return single-pass **iterators**. Once consumed, iterating over them a second time yields 0 items:

```
pairs = zip([1, 2], ["a", "b"])
print(list(pairs)) # [(1, 'a'), (2, 'b')]
print(list(pairs)) # [] (Exhausted!)

# If you need to re-use the pairs, convert to a list:
pairs_list = list(zip([1, 2], ["a", "b"]))
```

Exercises

1. Use `enumerate(lines, start=1)` to format an error report that prints the line number and content for lines containing "FATAL".
 2. Given a list of keys and a list of values, use `zip(keys, values, strict=True)` inside a dictionary comprehension to build a map.
 3. Demonstrate why `zip()` stops early when one argument is an infinite generator (like `itertools.count()`) and the other is a finite list.
 4. Implement a matrix transpose operation using `zip(*matrix)`.
-

Further reading

- PEP 618: *Add Optional Length-Checking To zip*.
- Python Standard Library: `builtins.enumerate`, `builtins.zip`.
- Python Standard Library: `itertools.zip_longest`.

Structural Pattern Matching with match and case

Structural Pattern Matching with `match` and `case`

After reading this chapter, you will master Python's Structural Pattern Matching syntax (`match` and `case` introduced in PEP 634), destructure sequence, mapping, and class patterns, enforce conditional guards with `if`, and prevent common variable-capture traps.

Mental model

Traditional `if/elif` statements evaluate boolean expressions one by one. **Structural Pattern Matching** evaluates data against a structural shape specification: testing **type**, **structure**, and **content** simultaneously, while extracting and binding variables on success:

Incoming Data: `{"action": "scale", "service": "auth", "replicas": 5}`

```
[ match expression ]

case {"action": "restart", "service": str(svc)}    No match (action != re

case {"action": "scale", "service": str(svc), "replicas": int(n)} if n >

    Matches keys: "action", "service", "replicas"
    Type matches: str and int
    Guard condition passes: n (5) > 0
    Binds: svc = "auth", n = 5
                                Execute Branch!

case _      Wildcard Fallback
```

Minimal example

Save as `pattern_matching_dispatch.py`:

```
# pattern_matching_dispatch.py
def process_command(cmd: list[str]) -> str:
    """Parse a CLI token list into an actionable response using sequence patterns."""
    match cmd:
```

```

    case ["quit" | "exit"]:
        return "Terminating process."
    case ["get", str(key)]:
        return f"Retrieving key: {key}"
    case ["set", str(key), value]:
        return f"Setting key: {key} = {value}"
    case ["batch", *items] if len(items) > 0:
        return f"Processing batch of {len(items)} items: {' '.join(items)}"
    case _:
        return f"Invalid or malformed command: {cmd}"

def main() -> None:
    test_commands = [
        ["quit"],
        ["get", "redis.host"],
        ["set", "timeout", "30s"],
        ["batch", "item1", "item2", "item3"],
        ["unknown", "foo", "bar"],
    ]
    for command in test_commands:
        result = process_command(command)
        print(f"Command: {command} -> Result: {result}")

if __name__ == "__main__":
    main()

```

Run via `uv run python pattern_matching_dispatch.py`:

```

Command: ['quit'] -> Result: Terminating process.
Command: ['get', 'redis.host'] -> Result: Retrieving key: redis.host
Command: ['set', 'timeout', '30s'] -> Result: Setting key: timeout = 30s
Command: ['batch', 'item1', 'item2', 'item3'] -> Result: Processing batch of 3 items: item1, item2, item3
Command: ['unknown', 'foo', 'bar'] -> Result: Invalid or malformed command: ['unknown', 'foo', 'bar']

```

Worked examples

Case 1: Mapping Patterns for Telemetry Event Routing

Mapping patterns allow inspecting structured dictionaries (such as JSON payloads from message queues) and extracting specific keys:

```

# telemetry_router.py
from typing import Any

```

```

def handle_telemetry_event(event: dict[str, Any]) -> None:
    match event:
        case {"type": "metric", "name": str(name), "value": float(val)} if val >= 90.0:
            print(f"[HIGH WATERMARK] Metric {name} breached alert threshold: {val}%")

        case {"type": "metric", "name": str(name), "value": float(val)}:
            print(f"[NOMINAL] Metric {name} at {val}%")

        case {"type": "heartbeat", "node_id": str(node), "status": "alive"}:
            print(f"[HEARTBEAT] Node {node} reported healthy.")

        case {"type": "alert", "severity": "CRITICAL", "message": str(msg)}:
            print(f"[PAGERDUTY ALERT] Critical incident: {msg}")

        case _:
            print(f"[UNHANDLED] Discarding unrecognized event structure: {event}")

if __name__ == "__main__":
    events = [
        {"type": "metric", "name": "cpu_utilization", "value": 94.2},
        {"type": "heartbeat", "node_id": "k8s-worker-04", "status": "alive", "uptime": 86400},
        {"type": "alert", "severity": "CRITICAL", "message": "Power rail offline"},
        {"source": "unknown", "data": None},
    ]

    for ev in events:
        handle_telemetry_event(ev)

```

Run:

```
uv run python telemetry_router.py
```

Output:

```

[HIGH WATERMARK] Metric cpu_utilization breached alert threshold: 94.2%
[HEARTBEAT] Node k8s-worker-04 reported healthy.
[PAGERDUTY ALERT] Critical incident: Power rail offline
[UNHANDLED] Discarding unrecognized event structure: {'source': 'unknown', 'data': None}

```

Notice that the heartbeat event matched despite containing an extra key ("uptime": 86400). In Python mapping patterns, extra keys are ignored unless explicitly restricted with `**_` or full length checks.

Case 2: Class Patterns with Dataclasses

Pattern matching works natively with object-oriented classes and dataclasses:

```

# packet_matcher.py
from dataclasses import dataclass

@dataclass
class Packet:
    source_ip: str
    dest_port: int
    payload_len: int
    protocol: str = "TCP"

def inspect_traffic(packet: Packet) -> str:
    match packet:
        # Match port 22 or 3389 with payload
        case Packet(dest_port=22 | 3389, payload_len=length) if length > 0:
            return f"Remote administration session to port {packet.dest_port} ({length} bytes)"

        # Match web traffic
        case Packet(dest_port=80 | 443):
            return f"Standard Web Traffic on port {packet.dest_port}"

        # Match UDP DNS traffic
        case Packet(protocol="UDP", dest_port=53):
            return "DNS Query Traffic"

        case Packet(dest_port=port):
            return f"General traffic on port {port}"

if __name__ == "__main__":
    packets = [
        Packet("192.168.1.100", 22, 512),
        Packet("10.0.0.5", 443, 1024),
        Packet("172.16.0.2", 53, 64, protocol="UDP"),
    ]

    for p in packets:
        print(f"{p.source_ip} -> {inspect_traffic(p)}")

```

Run:

```
uv run python packet_matcher.py
```

Output:

```

192.168.1.100 -> Remote administration session to port 22 (512 bytes)
10.0.0.5 -> Standard Web Traffic on port 443
172.16.0.2 -> DNS Query Traffic

```

Pitfalls

Pitfall 1: The Variable Capture Trap (Constants vs Variables)

In a `case` clause, an unqualified name (like `status`) is treated as a **capture variable**, NOT a value equality check!

```
# THE TRAP:
OK = 200
response_code = 404

match response_code:
    case OK: # BUG: This does NOT compare to OK! It assigns response_code to a new variable
        print("This ALWAYS prints because 'OK' matches anything and gets bound!")

# THE FIX: Use an Enum or dotted attribute
class Status:
    OK = 200

match response_code:
    case Status.OK: # Safe: Dotted names are looked up by value, not treated as capture var
        print("HTTP 200 OK")
    case _:
        print("Other status")
```

Exercises

1. Create a `match ... case` statement that parses a network URI string decomposed into `[scheme, host, port]`, handling default ports (80 for http, 443 for https) when port is omitted.
 2. Define a dataclass `DeploymentPlan(env: str, replicas: int, dry_run: bool)`. Write a pattern matching function that forbids any deployment to `env="production"` if `replicas > 10` and `dry_run` is `False`.
 3. Demonstrate the difference between `case [head, *tail]:` and `case [head, tail]:` when passed a list with four elements.
 4. Write a pattern matching function that accepts an arbitrary JSON-like value and returns whether it is a scalar, a list of strings, or a dictionary mapping strings to integers.
-

Further reading

- PEP 634: *Structural Pattern Matching: Specification.*
- PEP 635: *Structural Pattern Matching: Motivation and Rationale.*
- PEP 636: *Structural Pattern Matching: Tutorial.*

List, Dict, and Set Comprehensions

List, Dict, and Set Comprehensions

After reading this chapter, you will construct concise, declarative data transformations using list, dictionary, and set comprehensions, apply conditional filters, flatten multi-dimensional structures with nested loops, leverage assignment expressions (`:=`) to eliminate duplicate calculations, and identify when to refactor overly complex comprehensions into explicit loops.

Mental model

In imperative programming, transforming a collection requires creating an empty accumulator, iterating with a `for` loop, applying `if` guards, and appending items one by one. **Comprehensions** compress this workflow into a single declarative expression that is faster, less error-prone, and visually matches set-builder notation from mathematics:

Imperative Accumulation (4 statements):

```
result = []
for item in source:
    if passes_filter(item):
        result.append(transform(item))
```

Declarative Comprehension (1 expression):

```
result = [ transform(item) for item in source if passes_filter(item) ]
```

Expression	Iteration	Condition
------------	-----------	-----------

Python provides three comprehension syntaxes based on surrounding delimiters:

Container Type	Syntax	Result Type
List Comprehension	<code>[expr for item in iterable]</code>	list (ordered, mutable)
Set Comprehension	<code>{expr for item in iterable}</code>	set (unique, unordered)
Dict Comprehension	<code>{k: v for item in iterable}</code>	dict (key-value mapping)

For nested loops, the comprehension syntax preserves standard left-to-right nesting order:

```
[ x for row in matrix for x in row ]
```

Outer Loop	Inner Loop
------------	------------

Equivalent to:

```
for row in matrix:
    for x in row:
        yield x
```

Minimal example

Save as `comprehensions_overview.py`:

```
# comprehensions_overview.py
def main() -> None:
    raw_nodes = [" worker-01 ", "Worker-02", "  api-01", "WORKER-01", "db-01 "]

    # 1. List Comprehension: Normalize names and filter worker nodes
    worker_nodes = [
        node.strip().lower()
        for node in raw_nodes
        if "worker" in node.lower()
    ]
    print(f"Worker List: {worker_nodes}")

    # 2. Set Comprehension: Deduplicate normalized names
    unique_workers = {
        node.strip().lower()
        for node in raw_nodes
        if "worker" in node.lower()
    }
    print(f"Unique Worker Set: {sorted(unique_workers)}")

    # 3. Dict Comprehension: Build node-to-role lookup map
    node_roles = {
        node.strip().lower(): ("database" if "db" in node else "compute")
        for node in raw_nodes
    }
    print("Node Roles:")
    for name, role in sorted(node_roles.items()):
        print(f" {name:10} -> {role}")

if __name__ == "__main__":
    main()
```

Run via `uv run python comprehensions_overview.py`:

Worker List: ['worker-01', 'worker-02', 'worker-01']

Unique Worker Set: ['worker-01', 'worker-02']

Node Roles:

```
api-01      -> compute
db-01       -> database
worker-01   -> compute
worker-02   -> compute
```

Worked examples

Case 1: Ingesting and Sanitizing Raw Metric Records (List Comprehensions)

When ingesting metrics from HTTP log lines or sensors, raw records frequently contain malformed entries or values outside acceptable thresholds. A list comprehension combines parsing and filtering cleanly:

```
# metric_sanitizer.py
def sanitize_latencies(raw_samples: list[str]) -> list[float]:
    """Parse comma-separated millisecond strings, filter errors (-1.0), and convert to seconds
    return [
        float(val.strip()) / 1000.0
        for val in raw_samples
        if val.strip() != "" and float(val.strip()) >= 0.0
    ]

def main() -> None:
    samples = ["120.5", " 45.2 ", "-1.0", "", "850.0", "312.4", "-99.0", "12.0"]
    cleaned_seconds = sanitize_latencies(samples)

    print(f"Raw sample count:      {len(samples)}")
    print(f"Valid sample count:    {len(cleaned_seconds)}")
    print("Cleaned values (seconds):")
    for sec in cleaned_seconds:
        print(f"  {sec:.4f}s")

if __name__ == "__main__":
    main()
```

Run:

```
uv run python metric_sanitizer.py
```

Output:

```
Raw sample count:      8
Valid sample count:    5
Cleaned values (seconds):
0.1205s
0.0452s
0.8500s
0.3124s
0.0120s
```

Why this matters: Notice how `[f(x) for x in xs if cond(x)]` replaces chained `map()` and `filter()` calls. In Python, comprehensions are more readable than `list(map(..., filter(...)))` and avoid the overhead of lambda function call frames.

Case 2: Inverting and Indexing Service Registries (Dict Comprehensions)

In infrastructure management, service registries often map hostnames to IP addresses. SRE tooling frequently requires reverse lookups (IP-to-hostname) or normalized sub-mappings:

```
# registry_indexing.py
def main() -> None:
    # Forward map: Hostname -> IPv4
    dns_records = {
        "gateway.internal": "10.0.1.1",
        "auth.internal":    "10.0.1.15",
        "db-pri.internal":  "10.0.2.100",
        "db-sec.internal":  "10.0.2.101",
        "cache.internal":   "10.0.3.50",
    }

    # 1. Reverse Map: IP -> Hostname
    reverse_dns = {ip: host for host, ip in dns_records.items()}

    # 2. Filtered Subnet Map: Database subnet (10.0.2.x) only
    db_subnet = {
        host: ip
        for host, ip in dns_records.items()
        if ip.startswith("10.0.2.")
    }

    print("--- Reverse DNS Lookup (IP -> Host) ---")
    for ip, host in sorted(reverse_dns.items()):
        print(f" {ip:12} -> {host}")

    print("\n--- DB Subnet Only ---")
```

```

        for host, ip in sorted(db_subnet.items()):
            print(f"  {host:16} -> {ip}")

if __name__ == "__main__":
    main()

```

Run:

```
uv run python registry_indexing.py
```

Output:

```

--- Reverse DNS Lookup (IP -> Host) ---
10.0.1.1      -> gateway.internal
10.0.1.15     -> auth.internal
10.0.2.100    -> db-pri.internal
10.0.2.101    -> db-sec.internal
10.0.3.50     -> cache.internal

--- DB Subnet Only ---
db-pri.internal -> 10.0.2.100
db-sec.internal -> 10.0.2.101

```

Case 3: Matrix Flattening and Cartesian Grids (Nested Loops)

When managing multi-zone cloud deployments or flattening nested tabular datasets, nested comprehensions process multi-dimensional data without requiring manual stack allocations:

```

# cluster_topology.py
def main() -> None:
    regions = ["us-east", "eu-central"]
    zones = ["a", "b"]
    tiers = ["web", "api"]

    # Cartesian Product: Region x Zone x Tier
    deploy_targets = [
        f"{region}-{zone}-{tier}"
        for region in regions
        for zone in zones
        for tier in tiers
    ]

    print(f"Total Deployment Targets: {len(deploy_targets)}")
    for target in deploy_targets:

```

```

        print(f"    Target: {target}")

# 2D Grid Flattening
matrix = [
    [10, 20, 30],
    [40, 50, 60],
    [70, 80, 90],
]
flattened = [val for row in matrix for val in row if val > 25]
print(f"\nFiltered Flattened Matrix (values > 25): {flattened}")

if __name__ == "__main__":
    main()

```

Run:

```
uv run python cluster_topology.py
```

Output:

Total Deployment Targets: 8

```

Target: us-east-a-web
Target: us-east-a-api
Target: us-east-b-web
Target: us-east-b-api
Target: eu-central-a-web
Target: eu-central-a-api
Target: eu-central-b-web
Target: eu-central-b-api

```

Filtered Flattened Matrix (values > 25): [30, 40, 50, 60, 70, 80, 90]

Rule for Nested Comprehensions: Read the clauses from left to right. The first **for** clause is the outermost loop, and subsequent **for** clauses are nested inside it.

Case 4: Avoiding Duplicate Computation with the Walrus Operator (:=)

A common anti-pattern in comprehensions is calling an expensive function or parsing routine twice: once in the **if** clause to filter, and again in the expression clause to transform:

```

# walrus_comprehension.py
import json

def parse_payload(raw_text: str) -> dict[str, str] | None:

```

```

"""Simulate parsing an incoming JSON string payload."""
try:
    data = json.loads(raw_text)
    return data if isinstance(data, dict) else None
except json.JSONDecodeError:
    return None

def main() -> None:
    incoming_logs = [
        '{"service": "auth", "status": "200"}',
        'MALFORMED_HEADER',
        '{"service": "payment", "status": "503"}',
        '{}',
        '{"service": "cart", "status": "404"}',
    ]

    # Use the walrus operator (:=) in the condition to bind parsed_obj
    # and reuse it directly in the output expression without re-parsing!
    valid_services = [
        parsed["service"]
        for log in incoming_logs
        if (parsed := parse_payload(log)) is not None and "service" in parsed
    ]

    print("Parsed Active Services:")
    for svc in valid_services:
        print(f"  Service: {svc}")

if __name__ == "__main__":
    main()

```

Run:

```
uv run python walrus_comprehension.py
```

Output:

```

Parsed Active Services:
  Service: auth
  Service: payment
  Service: cart

```

Pitfalls

Pitfall 1: Misusing Comprehensions for Side Effects

Comprehensions are expressions designed to **produce collections**. Using them purely for their side effects (such as printing or writing to files) needlessly allocates in-memory lists that are immediately discarded:

```
# THE TRAP: Wasted memory and obscure intent
tasks = ["build", "test", "package", "deploy"]
[print(f"Executing: {task}") for task in tasks] # Allocates [None, None, None, None]!

# THE FIX: Use a clean, standard for loop
for task in tasks:
    print(f"Executing: {task}")
```

Pitfall 2: Over-Nesting (Write-Only Code)

When a comprehension contains more than two loops or multiple complex conditionals, readability collapses:

```
# THE TRAP: Write-only comprehension
bad = [
    f(x, y, z)
    for x in dataset
    if x.is_valid()
    for y in x.sub_items
    if y.is_ready()
    for z in y.tokens
    if z.is_active()
]

# THE FIX: Refactor into an explicit generator or loop with clear variable names
def extract_active_tokens(dataset):
    for x in dataset:
        if not x.is_valid():
            continue
        for y in x.sub_items:
            if not y.is_ready():
                continue
            for z in y.tokens:
                if z.is_active():
                    yield f(x, y, z)
```



```
good = list(extract_active_tokens(dataset))
```

Pitfall 3: Scope Isolation (Python 3 Comprehension Scope)

In Python 2, the loop variable in a list comprehension leaked into the enclosing scope and overwrote existing variables. In modern Python (3.x and 3.14), comprehensions execute in their own implicit function scope:

```
# comprehension_scope.py
def main() -> None:
    x = "original_outer_value"
    squares = [x * x for x in range(5)]

    print(f"Squares: {squares}")
    # In Python 3+, x remains unmodified!
    print(f"Outer x is preserved: {x}")

if __name__ == "__main__":
    main()
```

Output:

```
Squares: [0, 1, 4, 9, 16]
Outer x is preserved: original_outer_value
```

Exercises

1. Given a list of filenames ["app.py", "README.md", "server.py", "config.json", "utils.py"], write a list comprehension that extracts only the stems of Python files (.py), producing ["app", "server", "utils"].
2. Given a dictionary mapping usernames to email addresses, write a dict comprehension that extracts only accounts belonging to the @company.com domain, lowercasing both keys and values.
3. Given a list of sentences, write a set comprehension that extracts all unique words that are 5 letters or longer, stripping punctuation and whitespace.
4. Using nested comprehensions, generate a 5x5 identity matrix (a list of 5 lists, each containing 5 numbers, where element (i, j) is 1 if i == j and 0 otherwise).
5. Given a list of HTTP status code strings ["200", "invalid", "404", "500", "abc", "301"], use an assignment expression (:=) inside a comprehension to filter and convert valid integer status codes that are >= 400.

Further reading

- PEP 202: *List Comprehensions*.
- PEP 274: *Dict Comprehensions*.
- PEP 572: *Syntax for Assignment Expressions* (the `:=` walrus operator).
- Python Documentation: *Data Structures – List Comprehensions*.

Generator Expressions

Generator Expressions

After reading this chapter, you will construct lazy, memory-efficient data pipelines using generator expressions, measure and contrast their heap allocation footprint against eager collections, stream data directly into built-in aggregators like `sum()`, `max()`, and `any()`, and avoid generator exhaustion and late-binding pitfalls.

Mental model

A list comprehension is **eager**: it computes every element immediately and allocates memory for the entire collection on the heap. A **generator expression** is **lazy**: it produces values one at a time, on demand, via Python's iterator protocol.

```
Eager Evaluation (List Comprehension: [...]):
[x * 2 for x in range(10_000_000)]
```

```
[ 0, 2, 4, 6, 8, ... ]    Allocates ~80 Megabytes of RAM immediately!
                          High allocation latency; blocks until full.
```

```
Lazy Evaluation (Generator Expression: (...)):
(x * 2 for x in range(10_000_000))
```

```
Generator State Machine (~200B)
Current: x=0, next: x=1
```

```
next()
```

```
Yields: 0    Yields: 2    Yields: 4 (O(1) memory footprint!)
```

Generator expressions also chain together into pipelines where values are pulled through on demand like items on an assembly line:

```
Source Data    ( parse )    ( filter )    ( transform )    Consumer (sum/any/for)
```

```
Only one item processed at a time!
```

Minimal example

Save as `gen_expr_overview.py`:

```
# gen_expr_overview.py
import sys

def main() -> None:
    count = 1_000_000

    # 1. Eager List Comprehension: Allocates full list in memory
    eager_list = [n * 2 for n in range(count)]
    list_mem_bytes = sys.getsizeof(eager_list)

    # 2. Lazy Generator Expression: Allocates lightweight iterator object
    lazy_gen = (n * 2 for n in range(count))
    gen_mem_bytes = sys.getsizeof(lazy_gen)

    print(f"Items processed:           {count:,}")
    print(f"Eager list memory:           {list_mem_bytes:,} bytes (~{list_mem_bytes / (1024 * 1024):.2f} MB)")
    print(f"Lazy generator memory:         {gen_mem_bytes:,} bytes")
    print(f"Memory ratio:                   {list_mem_bytes // gen_mem_bytes:,}x less memory!")

    # 3. Consuming on demand with next()
    print(f"\nFirst three values pulled on demand:")
    print(f"  Item 1: {next(lazy_gen)}")
    print(f"  Item 2: {next(lazy_gen)}")
    print(f"  Item 3: {next(lazy_gen)}")

if __name__ == "__main__":
    main()
```

Run via `uv run python gen_expr_overview.py`:

```
Items processed:           1,000,000
Eager list memory:         8,448,728 bytes (~8.06 MB)
Lazy generator memory:     208 bytes
Memory ratio:              40,618x less memory!
```

First three values pulled on demand:

```
Item 1: 0
Item 2: 2
Item 3: 4
```

Worked examples

Case 1: Streaming Gigabyte Log Parsing with Constant O(1) Memory

In production environments, log files can span tens of gigabytes. Reading an entire log file into a list causes `MemoryError` and triggers Linux Out-Of-Memory (OOM) kills. Chaining generator expressions processes files of arbitrary size with constant memory:

```
# streaming_log_parser.py
from collections.abc import Iterator

def generate_mock_log_stream() -> Iterator[str]:
    """Simulate reading lines from a high-throughput server log."""
    logs = [
        '192.168.1.10 - - [2026-09-07] "GET /api/v1/health" 200 45',
        '192.168.1.11 - - [2026-09-07] "POST /api/v1/login" 401 120',
        '# SRE maintenance comment - ignore',
        '10.0.0.50 - - [2026-09-07] "GET /api/v1/orders" 200 890',
        '',
        '172.16.0.4 - - [2026-09-07] "GET /api/v1/checkout" 500 240',
        '192.168.1.12 - - [2026-09-07] "GET /api/v1/status" 200 62',
    ]
    yield from logs

def main() -> None:
    raw_lines = generate_mock_log_stream()

    # Stage 1: Strip whitespace and drop comments/blank lines
    clean_lines = (line.strip() for line in raw_lines if line.strip() and not line.startswith('#'))

    # Stage 2: Tokenize and extract (status_code, response_bytes)
    def parse_entry(line: str) -> tuple[int, int]:
        parts = line.split()
        return int(parts[-2]), int(parts[-1])

    records = (parse_entry(line) for line in clean_lines)

    # Stage 3: Filter for successful requests (HTTP 200)
    success_bytes = (resp_bytes for status, resp_bytes in records if status == 200)

    # Stage 4: Consume directly into built-in aggregator sum()
    total_payload_bytes = sum(success_bytes)
    print(f"Total payload transferred for HTTP 200 responses: {total_payload_bytes} bytes")

if __name__ == "__main__":
    main()
```

Run:

```
uv run python streaming_log_parser.py
```

Output:

Total payload transferred for HTTP 200 responses: 997 bytes

Why this matters: Notice that `sum(resp_bytes for status, resp_bytes in ...)` does not require double parentheses (`(...)`). When a generator expression is the sole argument to a function call, outer parentheses are optional.

Case 2: Short-Circuiting Aggregations with `any()` and `all()`

Built-in boolean predicates `any()` and `all()` short-circuit: they terminate evaluation immediately upon encountering the first definitive boolean value (`True` for `any`, `False` for `all`). Using a generator expression prevents running expensive operations on remaining elements:

```
# short_circuit_eval.py
def check_node_health(node_id: str) -> bool:
    """Simulate an expensive remote node probe."""
    print(f"  [Probe] Checking node: {node_id}...")
    # Simulate node-03 being unhealthy
    return node_id != "node-03"

def main() -> None:
    cluster_nodes = [f"node-{i:02d}" for i in range(1, 10)]

    print("--- Testing with Generator Expression (Lazy, Short-Circuits) ---")
    # Generator expression: evaluation stops immediately at node-03!
    all_healthy_gen = all(check_node_health(node) for node in cluster_nodes)
    print(f"Cluster all healthy result: {all_healthy_gen}")

    print("\n--- Contrast: List Comprehension (Eager, Evaluates All 9 Nodes!) ---")
    # List comprehension: MUST evaluate all 9 nodes before calling all()
    probes_run = 0
    def logged_probe(node: str) -> bool:
        nonlocal probes_run
        probes_run += 1
        return node != "node-03"

    all_healthy_list = all([logged_probe(node) for node in cluster_nodes])
    print(f"List comprehension ran {probes_run} probes even though node-03 failed!")

if __name__ == "__main__":
    main()
```

Run:

```
uv run python short_circuit_eval.py
```

Output:

```
--- Testing with Generator Expression (Lazy, Short-Circuits) ---
[Probe] Checking node: node-01...
[Probe] Checking node: node-02...
[Probe] Checking node: node-03...
Cluster all healthy result: False

--- Contrast: List Comprehension (Eager, Evaluates All 9 Nodes!) ---
List comprehension ran 9 probes even though node-03 failed!
```

Case 3: Composing Multi-Stage Data Pipelines

Generator expressions can be chained linearly to construct readable Extract-Transform-Load (ETL) pipelines without third-party frameworks:

```
# pipeline_composition.py
def main() -> None:
    raw_telemetry = [
        {"host": "srv-1", "cpu": 45.2, "mem_mb": 4096},
        {"host": "srv-2", "cpu": 92.1, "mem_mb": 16384},
        {"host": "srv-3", "cpu": 78.4, "mem_mb": 8192},
        {"host": "srv-4", "cpu": 95.8, "mem_mb": 32768},
        {"host": "srv-5", "cpu": 12.0, "mem_mb": 2048},
    ]

    # Stage 1: Filter hosts breaching CPU threshold (> 75%)
    overloaded = (rec for rec in raw_telemetry if rec["cpu"] > 75.0)

    # Stage 2: Convert memory to gigabytes
    with_gb = (
        {"host": rec["host"], "cpu": rec["cpu"], "mem_gb": rec["mem_mb"] / 1024}
        for rec in overloaded
    )

    # Stage 3: Format alert payload strings
    alerts = (
        f"[ALERT] {item['host']}: CPU={item['cpu']:.1f}% Mem={item['mem_gb']:.1f}GB"
        for item in with_gb
    )
```



```

# Terminal Consumer: Iterate and dispatch
print("Dispatched Infrastructure Alerts:")
for alert in alerts:
    print(f" {alert}")

if __name__ == "__main__":
    main()

```

Run:

```
uv run python pipeline_composition.py
```

Output:

```

Dispatched Infrastructure Alerts:
[ALERT] srv-2: CPU=92.1% Mem=16.0GB
[ALERT] srv-3: CPU=78.4% Mem=8.0GB
[ALERT] srv-4: CPU=95.8% Mem=32.0GB

```

Pitfalls

Pitfall 1: Generator Exhaustion (The “Read-Once” Trap)

Unlike lists or tuples, generators are stateful, single-pass iterators. Once consumed to completion, they are **exhausted** and produce no further items:

```

# THE TRAP: Re-iterating an exhausted generator
data = (x * 10 for x in range(1, 5))

total = sum(data)          # Consumes all items: 10 + 20 + 30 + 40 = 100
maximum = max(data)        # BUG: ValueError: max() arg is an empty sequence!

# THE FIX: Materialize as list if multiple passes are strictly required,
# or recreate the generator expression.
numbers = [x * 10 for x in range(1, 5)] # If dataset fits in memory
total = sum(numbers)
maximum = max(numbers)
print(f"Total: {total}, Max: {maximum}")

```

Pitfall 2: Late-Binding Closures in Loop Expressions

When a generator expression references a variable from an outer loop, the variable is looked up when the generator is **evaluated**, NOT when it was defined:

```
# THE TRAP: Late binding
multipliers = []
for factor in [2, 3, 5]:
    # The generator captures the variable name `factor`, not its current value!
    multipliers.append((x * factor for x in range(3)))

# When consumed, `factor` has its final loop value (5) for ALL generators!
for gen in multipliers:
    print(list(gen))
# Prints:
# [0, 5, 10]
# [0, 5, 10]
# [0, 5, 10]

# THE FIX: Bind the current value eagerly using a function scope or default parameter
def make_multiplier_gen(f):
    return (x * f for x in range(3))

multipliers = [make_multiplier_gen(factor) for factor in [2, 3, 5]]
for gen in multipliers:
    print(list(gen))
# Prints correctly:
# [0, 2, 4]
# [0, 3, 6]
# [0, 5, 10]
```

Pitfall 3: Indexing and Length Queries (`gen[0]` or `len(gen)`)

Because generator expressions compute elements on the fly, they do not know their future length and cannot jump to arbitrary indices:

```
gen = (x ** 2 for x in range(10))

# TypeError: 'generator' object is not subscriptable
# first = gen[0]

# TypeError: object of type 'generator' has no len()
# count = len(gen)
```

```
# THE FIX:
# Use next() for the first item:
first = next(gen)

# Use itertools.islice for slice windows:
import itertools
next_three = list(itertools.islice(gen, 3))
print(f"First: {first}, Next three: {next_three}")
```

Exercises

1. Given a large range `range(1, 10_000_001)`, use a generator expression with `sum()` to calculate the sum of all odd integers. Verify how quickly it executes without consuming significant memory.
 2. Given a list of server hostnames, write an `all()` expression using a generator to verify that every hostname starts with `"node-"` and ends with `".internal"`. Test that it terminates early when the first hostname fails.
 3. Write a generator pipeline that reads a stream of numbers, squares each number, filters out numbers not divisible by 3, and formats the output into strings `"Num: <val>"`.
 4. Demonstrate generator exhaustion: create a generator expression, verify `list(gen)` yields 5 elements, and verify a second `list(gen)` call returns `[]`.
 5. Write a memory benchmark script comparing `sys.getsizeof` for a list comprehension vs a generator expression for 5 million elements.
-

Further reading

- PEP 289: *Generator Expressions*.
- Python Documentation: *Standard Types – Iterator Types*.
- Python Documentation: *The itertools Module – Functions creating iterators for efficient looping*.

Data Structures

Lists and Dynamic Array Mechanics

Lists and Dynamic Array Mechanics

After reading this chapter, you will master the internal memory layout of Python lists, analyze how CPython achieves amortized $O(1)$ appends via over-allocation, benchmark in-place mutations against full-copy allocations, implement custom sort keys, and avoid reference duplication pitfalls in multi-dimensional lists.

Mental model

A Python `list` is not a linked list; it is a **dynamic array of pointers** (`PyListObject` in CPython). The list structure holds an array of 8-byte memory addresses (on 64-bit systems) pointing to arbitrary objects located elsewhere on the heap:

CPython `PyListObject` Architecture:

```
PyListObject Header (56 bytes on 64-bit)
  ob_refcnt: 1
  ob_type:   &PyList_Type
  ob_size:   3   (Logical element count)
  allocated: 6   (Physical buffer capacity slots)
  ob_item
```

Slot 0	Slot 1	Slot 2	Slot 3	Slot 4	Slot 5
&"web-01"	&"web-02"	&"db-01"	NULL	NULL	NULL

"web-01" "web-02" "db-01"

To prevent reallocating the memory buffer on every single `append()`, CPython **over-allocates** spare slots using a geometric growth formula (`allocated = size + (size >> 3) + (size < 9 ? 3 : 6)`).

This yields distinct algorithmic complexities: - **Append / Pop at end**: Amortized $O(1)$ — fast pointer write into pre-allocated slot. - **Insert / Delete at index 0**: $O(N)$ — every existing pointer must be shifted in memory with `memmove`. - **Random index lookup** (`data[i]`): $O(1)$ — direct pointer offset calculation.

Minimal example

Save as `list_mechanics.py`:

```
# list_mechanics.py
import sys

def trace_list_growth() -> None:
    """Trace CPython dynamic array reallocation steps as elements are appended."""
    items: list[int] = []
    prev_bytes = sys.getsizeof(items)
    print(f"Initial empty list size: {prev_bytes} bytes")

    print("\nTracing capacity reallocations up to 30 items:")
    for i in range(1, 31):
        items.append(i)
        curr_bytes = sys.getsizeof(items)
        if curr_bytes != prev_bytes:
            # On 64-bit CPython, base struct is 56 bytes, each pointer is 8 bytes
            capacity = (curr_bytes - 56) // 8
            print(f"  Length: {len(items):2d} | Memory: {curr_bytes:3d} bytes | Allocated slots: {capacity}")
            prev_bytes = curr_bytes

def main() -> None:
    trace_list_growth()

if __name__ == "__main__":
    main()
```

Run via `uv run python list_mechanics.py`:

Initial empty list size: 56 bytes

Tracing capacity reallocations up to 30 items:

```
Length:  1 | Memory:  88 bytes | Allocated slots:  4
Length:  5 | Memory: 120 bytes | Allocated slots:  8
Length:  9 | Memory: 184 bytes | Allocated slots: 16
Length: 17 | Memory: 248 bytes | Allocated slots: 24
Length: 25 | Memory: 312 bytes | Allocated slots: 32
```

Worked examples

Case 1: In-Place Mutations vs Full Copy Allocations

When working with large collections, choosing between mutating in place (`extend`, `append`, `+=`) versus constructing new lists (`+`) determines whether memory usage is $O(1)$ or $O(N)$:

```
# list_mutations.py
import time

def benchmark_concatenation() -> None:
    n = 20_000

    # Approach A: Repeated concatenation with + (Creates a brand new list every iteration!)
    start = time.perf_counter()
    result_a = []
    for i in range(n):
        result_a = result_a + [i]
    dur_a = time.perf_counter() - start

    # Approach B: In-place mutation with append (Leverages pre-allocated spare slots)
    start = time.perf_counter()
    result_b = []
    for i in range(n):
        result_b.append(i)
    dur_b = time.perf_counter() - start

    # Approach C: Batch extension with extend
    start = time.perf_counter()
    result_c = []
    result_c.extend(range(n))
    dur_c = time.perf_counter() - start

    print(f"Repeated '+' concat (allocates N lists): {dur_a:.5f}s")
    print(f"Repeated append() (amortized O(1)):      {dur_b:.5f}s  ({dur_a / dur_b:.1f}x faster)")
    print(f"Single extend() (bulk C allocation):      {dur_c:.5f}s  ({dur_a / dur_c:.1f}x faster)")

def main() -> None:
    benchmark_concatenation()

if __name__ == "__main__":
    main()
```

Run:

```
uv run python list_mutations.py
```

Output:


```
Repeated '+' concat (allocates N lists): 0.04612s
Repeated append() (amortized O(1)):      0.00068s (67.8x faster)
Single extend() (bulk C allocation):     0.00014s (329.4x faster)
```

Case 2: Custom In-Place Sorting with Timsort

Python's `list.sort()` implements **Timsort** (and adaptive Powersort heuristics in Python 3.11+). It sorts strictly in place with $O(N \log N)$ worst-case guarantees and $O(N)$ for pre-sorted inputs:

```
# list_sorting.py
def main() -> None:
    servers = [
        {"host": "srv-03", "cpu_percent": 88.5, "rack": "R2"},
        {"host": "srv-01", "cpu_percent": 12.0, "rack": "R1"},
        {"host": "srv-04", "cpu_percent": 95.2, "rack": "R2"},
        {"host": "srv-02", "cpu_percent": 45.0, "rack": "R1"},
    ]

    # 1. In-place sort by CPU load descending
    servers.sort(key=lambda s: s["cpu_percent"], reverse=True)
    print("--- Sorted by Highest CPU Load (In-Place) ---")
    for s in servers:
        print(f" {s['host']} | CPU: {s['cpu_percent']:5.1f}% | Rack: {s['rack']}")

    # 2. Multi-criterion sorting using tuples: Rack ascending, then CPU descending
    # Notice: negation (-s['cpu_percent']) allows descending numeric order within a tuple key
    servers.sort(key=lambda s: (s["rack"], -s["cpu_percent"]))
    print("\n--- Multi-Key Sort (Rack ASC, CPU DESC) ---")
    for s in servers:
        print(f" Rack: {s['rack']} | {s['host']} | CPU: {s['cpu_percent']:5.1f}%")

if __name__ == "__main__":
    main()
```

Run:

```
uv run python list_sorting.py
```

Output:

```
--- Sorted by Highest CPU Load (In-Place) ---
srv-04 | CPU: 95.2% | Rack: R2
srv-03 | CPU: 88.5% | Rack: R2
srv-02 | CPU: 45.0% | Rack: R1
srv-01 | CPU: 12.0% | Rack: R1
```

--- Multi-Key Sort (Rack ASC, CPU DESC) ---

```
Rack: R1 | srv-02 | CPU: 45.0%
Rack: R1 | srv-01 | CPU: 12.0%
Rack: R2 | srv-04 | CPU: 95.2%
Rack: R2 | srv-03 | CPU: 88.5%
```

Case 3: Shallow Copies vs Deep Copies

Because lists store object references, copying a list requires understanding the boundary between shallow copies (duplicating the pointer array) and deep copies (recursively duplicating referenced objects):

```
# list_copying.py
import copy

def main() -> None:
    original = [
        {"node": "worker-1", "status": "active"},
        {"node": "worker-2", "status": "active"},
    ]

    # 1. Shallow copy: new list container, but points to identical inner dictionaries!
    shallow = original.copy()

    # 2. Deep copy: completely independent object graph
    deep = copy.deepcopy(original)

    # Mutate inner dictionary in shallow copy
    shallow[0]["status"] = "OFFLINE"

    print("After modifying shallow[0]['status'] = 'OFFLINE':")
    print(f"  Original node 1 status: {original[0]['status']}  <-- UNINTENTIONALLY MUTATED!")
    print(f"  Shallow  node 1 status: {shallow[0]['status']}")
    print(f"  Deep      node 1 status: {deep[0]['status']}      <-- Completely isolated")

if __name__ == "__main__":
    main()
```

Run:

```
uv run python list_copying.py
```

Output:

```
After modifying shallow[0]['status'] = 'OFFLINE':
Original node 1 status: OFFLINE <-- UNINTENTIONALLY MUTATED!
Shallow node 1 status: OFFLINE
Deep node 1 status: active <-- Completely isolated
```

Pitfalls

Pitfall 1: Modifying a List While Iterating Over It

Never add or remove elements from a list while iterating over it directly. The loop maintains an internal integer index counter, causing elements to be silently skipped:

```
# THE TRAP: Silently skips items
numbers = [1, 2, 2, 3, 4, 2, 5]
for x in numbers:
    if x == 2:
        numbers.remove(x) # BUG: Modifies array while index advances!
print(f"Result with bug: {numbers}") # Still contains [1, 2, 3, 4, 5]!
```

```
# THE FIX: Iterate over a slice copy or use a list comprehension
numbers = [1, 2, 2, 3, 4, 2, 5]
numbers = [x for x in numbers if x != 2]
print(f"Clean result: {numbers}") # [1, 3, 4, 5]
```

Pitfall 2: The Repeated Multiplication Reference Duplication Bug

Using the `*` operator on a list containing a mutable object duplicates the **same reference**, not independent copies:

```
# THE TRAP: 3 rows pointing to the identical list object!
grid = [[0] * 3] * 3
grid[0][0] = 99
print("Buggy grid:")
for row in grid:
    print(f" {row}") # All 3 rows now have 99 at index 0!
```

Output:

Buggy grid:
[99, 0, 0]
[99, 0, 0]
[99, 0, 0]

```
# THE FIX: Use a list comprehension to allocate distinct inner lists
safe_grid = [[0] * 3 for _ in range(3)]
safe_grid[0][0] = 99
print("\nSafe grid:")
for row in safe_grid:
    print(f" {row}")
```

Output:

Safe grid:
[99, 0, 0]
[0, 0, 0]
[0, 0, 0]

Pitfall 3: Assuming `list.sort()` Returns the Sorted List

`list.sort()` modifies the array in place and returns `None` by design to prevent confusing in-place mutation with copy allocation:

```
# THE TRAP:
data = [5, 2, 8, 1]
data = data.sort() # BUG: data is now None!
print(f"data is: {data}")

# THE FIX:
# Either call sort() as a statement:
data = [5, 2, 8, 1]
data.sort()

# Or use sorted() if you want a new returned list:
other = [5, 2, 8, 1]
result = sorted(other)
```

Exercises

1. Initialize an empty list and print its size in bytes using `sys.getsizeof()`. Append 100 elements one by one, recording each logical length where reallocation occurs.
 2. Given a list of user dictionaries `[{"name": "Alice", "age": 30}, {"name": "Bob", "age": 25}]`, use `list.sort()` with a lambda key to sort them in ascending order of age in place.
 3. Benchmark the time difference between `list.pop(0)` (which shifts all elements) and `list.pop()` (which takes the tail element) on a list of 100,000 integers.
 4. Construct a 4x4 coordinate board filled with "." using a safe list comprehension. Verify that modifying board `[1][2]` changes only that single cell.
 5. Given two lists `a = [1, 2, 3]` and `b = [4, 5, 6]`, explain the memory difference between `a.extend(b)` and `a = a + b`.
-

Further reading

- Python Documentation: *Data Structures – More on Lists*.
- CPython Source Code: `Objects/listobject.c` (dynamic array implementation and growth formula).
- Python Documentation: *Sorting Techniques – HOWTO*.

Tuples and Immutable Sequence Records

Tuples and Immutable Sequence Records

After reading this chapter, you will master the internal architecture of Python tuples, distinguish when to use immutable records over mutable lists, leverage composite tuples as hashable dictionary keys, enforce defensive API boundaries, and prevent singleton comma syntax traps.

Mental model

While a `list` is designed for homogeneous, dynamically resizing collections, a `tuple` is designed for **heterogeneous, fixed-length records**.

In CPython, a `tuple` (`PyTupleObject`) is fixed in size at allocation time: - It has no **allocated** capacity field. - Its memory buffer contains exactly the number of pointer slots required for its elements. - CPython maintains a memory cache (“free list”) for small tuples (up to 20 elements), making tuple allocation and deallocation exceptionally fast.

CPython `PyTupleObject` Architecture:

```
PyTupleObject Header (40 bytes on 64-bit)
  ob_refcnt: 1
  ob_type:   &PyTuple_Type
  ob_size:   3   (Exact length, fixed forever)
  ob_item
```

```
Slot 0      Slot 1      Slot 2      (Zero spare capacity; no reallocation)
&"us-east"  &8080      &"active"
```

```
"us-east"    8080      "active"
```

The Hashability Rule for Tuples

An object is hashable if it has an immutable hash value that remains constant throughout its lifetime. A tuple is hashable **if and only if every element contained within it is also hashable**:

("10.0.0.1", 443)	All elements immutable	Hashable (Valid dict key)
("10.0.0.1", [80, 443])	Contains mutable list	TypeError: unhashable type: 'list'

Minimal example

Save as tuple_mechanics.py:

```
# tuple_mechanics.py
import sys

def main() -> None:
    # 1. Memory footprint comparison
    items_list = [10, 20, 30, 40]
    items_tuple = (10, 20, 30, 40)

    print(f"List size (4 items): {sys.getsizeof(items_list)} bytes (includes over-allocation buffer)")
    print(f"Tuple size (4 items): {sys.getsizeof(items_tuple)} bytes (lean, fixed struct)")

    # 2. Singleton tuple syntax: Trailing comma is REQUIRED
    not_a_tuple = ("standalone") # Evaluates to a plain string!
    real_tuple = ("standalone",) # Comma defines the tuple!

    print(f"\nType of ('standalone'): {type(not_a_tuple).__name__}")
    print(f"Type of ('standalone',): {type(real_tuple).__name__}")

    # 3. Tuples as composite dictionary keys
    # Map (datacenter, cluster_id) -> primary VIP
    cluster_vips: dict[tuple[str, int], str] = {
        ("us-east", 1): "10.100.1.1",
        ("us-east", 2): "10.100.2.1",
        ("eu-west", 1): "10.200.1.1",
    }

    target = ("us-east", 2)
    print(f"\nVIP for cluster {target}: {cluster_vips[target]}")

if __name__ == "__main__":
    main()
```

Run via uv run python tuple_mechanics.py:

```
List size (4 items): 88 bytes (includes over-allocation buffer)
Tuple size (4 items): 80 bytes (lean, fixed struct)
```



```
Type of ('standalone'): str
Type of ('standalone',): tuple
```

```
VIP for cluster ('us-east', 2): 10.100.2.1
```

Worked examples

Case 1: Composite Coordinates and Multi-Dimensional Indexing

In network engineering and cloud orchestration, routing tables and security groups frequently require multi-attribute indexing (e.g. (source_ip, destination_port, protocol)). Tuples make natural, zero-dependency composite keys:

```
# firewall_lookup.py
def main() -> None:
    # Rule table: (dest_port, protocol) -> action
    security_rules: dict[tuple[int, str], str] = {
        (22, "tcp"): "ALLOW_ADMIN",
        (80, "tcp"): "REDIRECT_HTTPS",
        (443, "tcp"): "ALLOW_PUBLIC",
        (53, "udp"): "ALLOW_DNS",
        (123, "udp"): "ALLOW_NTP",
    }

    inbound_probes = [
        (443, "tcp"),
        (22, "tcp"),
        (3389, "tcp"),
        (53, "udp"),
    ]

    for port, proto in inbound_probes:
        # Direct O(1) hash lookup on composite tuple
        action = security_rules.get((port, proto), "DENY_LOG")
        print(f"Probe to Port {port:4d}/{proto:3} -> Decision: {action}")

if __name__ == "__main__":
    main()
```

Run:

```
uv run python firewall_lookup.py
```

Output:

```
Probe to Port 443/tcp -> Decision: ALLOW_PUBLIC
Probe to Port 22/tcp -> Decision: ALLOW_ADMIN
Probe to Port 3389/tcp -> Decision: DENY_LOG
Probe to Port 53/udp -> Decision: ALLOW_DNS
```

Case 2: Defensive API Design (Immutable Return Values)

When an internal service or class exposes its collected state to external consumers, returning a mutable `list` allows callers to accidentally or maliciously corrupt the internal state. Returning a `tuple` guarantees immutability:

```
# defensive_service.py
class NodeCluster:
    def __init__(self, cluster_name: str) -> None:
        self.cluster_name = cluster_name
        self._active_nodes: list[str] = []

    def register_node(self, node_id: str) -> None:
        self._active_nodes.append(node_id)

    @property
    def nodes(self) -> tuple[str, ...]:
        """Expose nodes as an immutable tuple snapshot."""
        return tuple(self._active_nodes)

def main() -> None:
    cluster = NodeCluster("prod-compute")
    cluster.register_node("node-01")
    cluster.register_node("node-02")

    current_nodes = cluster.nodes
    print(f"Registered nodes: {current_nodes}")

    # Attempting to mutate external view raises AttributeError
    try:
        current_nodes.append("rogue-node") # type: ignore
    except AttributeError as err:
        print(f"Caught expected error: {err}")

    # Internal state remains completely pristine
    print(f"Internal cluster state verified intact: {cluster.nodes}")

if __name__ == "__main__":
    main()
```

Run:

```
uv run python defensive_service.py
```

Output:

```
Registered nodes: ('node-01', 'node-02')
Caught expected error: 'tuple' object has no attribute 'append'
Internal cluster state verified intact: ('node-01', 'node-02')
```

Case 3: Tuple Struct Packing and Structural Records

Tuples represent fixed records where position defines semantics. When paired with standard sequence unpacking, they avoid the overhead of heavy class instances for lightweight data exchange:

```
# metrics_pipeline.py
def fetch_telemetry_sample() -> tuple[str, float, int]:
    """Return a fixed telemetry packet: (sensor_id, temperature_c, timestamp)."""
    return ("sensor-temp-04", 42.8, 1725705600)

def main() -> None:
    sample = fetch_telemetry_sample()

    # Clean unpacking
    sensor_id, temp_c, ts = sample
    print(f"Sensor ID: {sensor_id}")
    print(f"Temperature: {temp_c:.1f} C")
    print(f"Timestamp: {ts}")

    # Comparison mechanics: Tuples compare element-by-element lexicographically
    version_a = (2, 4, 1)
    version_b = (2, 5, 0)
    print(f"\nSemantic version comparison:")
    print(f" {version_a} < {version_b} -> {version_a < version_b}")

if __name__ == "__main__":
    main()
```

Run:

```
uv run python metrics_pipeline.py
```

Output:

```
Sensor ID:    sensor-temp-04
Temperature:  42.8 C
Timestamp:    1725705600

Semantic version comparison:
  (2, 4, 1) < (2, 5, 0) -> True
```

Pitfalls

Pitfall 1: The Missing Comma in Singleton Tuples

Parentheses in Python serve double duty: grouping expressions and delimiting tuples. Without a trailing comma, parentheses simply group an expression:

```
# THE TRAP:
single_item = ("config.yaml") # BUG: This is a string, NOT a tuple!
print(type(single_item))      # <class 'str'>

# Calling a function expecting an iterable of files:
# for file in single_item: iterates character-by-character: 'c', 'o', 'n', 'f', ...!

# THE FIX: Always include a trailing comma for singletons
real_tuple = ("config.yaml",)
print(type(real_tuple))      # <class 'tuple'>
```

Pitfall 2: The Mutable-Element-in-Tuple Trap

A tuple's immutability means its **pointer array** cannot be modified. However, if a tuple contains a mutable object (like a list), the contents of that object can still be mutated, and the tuple cannot be hashed:

```
# THE TRAP:
tricky = (1, 2, ["alpha", "beta"])

# 1. Immutability is shallow: the inner list CAN be modified!
tricky[2].append("gamma")
print(f"Mutated inner list: {tricky}") # (1, 2, ['alpha', 'beta', 'gamma'])

# 2. Hashability fails:
try:
    bad_dict = {tricky: "payload"}
```

```
except TypeError as err:
    print(f"Hash failure: {err}") # TypeError: unhashable type: 'list'
```

Output:

```
Mutated inner list: (1, 2, ['alpha', 'beta', 'gamma'])
Hash failure: unhashable type: 'list'
```

```
# THE FIX: Ensure all nested elements are immutable if hashing is required
safe_tuple = (1, 2, ("alpha", "beta", "gamma"))
safe_dict = {safe_tuple: "payload"}
print(f"Safe dict lookup: {safe_dict[safe_tuple]}")
```

Pitfall 3: The Augmented Assignment Trap on Tuples

Attempting += on a mutable object inside a tuple both raises `TypeError` AND modifies the object:

```
t = (1, [2, 3])
try:
    t[1] += [4]
except TypeError as e:
    print(f"Raised: {e}")

print(f"State after exception: {t}")
```

Output:

```
Raised: 'tuple' object does not support item assignment
State after exception: (1, [2, 3, 4])
```

Why this happens: += mutates the list in place (adding 4), and then attempts to assign the mutated list back to slot `t[1]`, which the tuple forbids. Avoid storing mutable objects inside tuples.

Exercises

1. Create a 1-element tuple containing the integer 42. Confirm with `type()` that it is a tuple, not an `int`.
 2. Given a list of (host, port) tuples, write a function that finds all unique ports using a set comprehension.
 3. Write a version comparison function that takes two release strings (e.g. "1.14.2" and "1.9.5"), converts them to tuples of integers, and determines which version is newer.
 4. Construct a dictionary where keys are 3D grid coordinates (`x`, `y`, `z`) and values are block types. Query coordinate (10, 5, 2) using `.get()`.
 5. Demonstrate why (1, [2, 3]) cannot be added to a Python set, and rewrite it so it can.
-

Further reading

- Python Documentation: *Standard Types – Tuples*.
- CPython Source Code: `Objects/tupleobject.c` (tuple implementation and small-tuple caching).
- Raymond Hettinger: *Transforming Code into Beautiful, Idiomatic Python*.

Dictionaries and Hash Table Mechanics

Dictionaries and Hash Table Mechanics

After reading this chapter, you will master Python's compact hash table implementation, understand open-addressing collision resolution and insertion-order preservation, execute non-destructive dictionary merges with `|`, leverage dynamic dictionary views, and avoid mutation-during-iteration errors.

Mental model

In CPython 3.6+, a dictionary (`PyDictObject`) uses a **compact hash table layout** consisting of two separate arrays: 1. **Sparse Indices Array**: An array of small integers (indices into the dense entries array) indexed by `hash(key) % table_size`. Empty slots contain `-1`. 2. **Dense Entries Array**: Stores entries in exact insertion order. Each row contains `[hash_value, key_pointer, value_pointer]`.

CPython Compact Dictionary Architecture:

Key: "auth", `hash("auth") % 8 = 1`

Key: "api", `hash("api") % 8 = 4`

1. Sparse Indices Array (Table size: 8):

Slot:	0	1	2	3	4	5	6	7
Value:	[-1,	0,	-1,	-1,	1,	-1,	-1,	-1]

2. Dense Entries Array (Appended sequentially in insertion order):

Index 0:	[0x38fa, &"auth", &"http://auth.internal"]
Index 1:	[0x7c12, &"api", &"http://api.internal"]

Key Advantages

- **Memory Efficiency**: The dense entries array stores 24 bytes per entry (`hash, key*, value*`) with zero wasted slots. The sparse array uses only 1 byte per slot for small dictionaries (`int8_t`). This saves 20%–25% memory compared to legacy sparse tables.
- **Deterministic Insertion Order**: Because entries are appended to the dense array in the order they arrive, iterating over a dictionary always produces keys in insertion order.
- **Collision Handling**: When two keys hash to the same sparse index, CPython uses open addressing with pseudo-random perturbation (`index = (5*index + 1 + perturb) % size`) to probe for the next available slot.

Minimal example

Save as `dict_mechanics.py`:

```
# dict_mechanics.py
def main() -> None:
    # 1. Dictionary creation and insertion-order preservation
    service_endpoints: dict[str, str] = {
        "auth": "http://10.0.1.10:8080",
        "orders": "http://10.0.1.20:8080",
        "billing": "http://10.0.1.30:8080",
    }

    # Insertion order is strictly preserved
    print("--- Service Endpoints (Insertion Order) ---")
    for service, url in service_endpoints.items():
        print(f" {service:8} -> {url}")

    # 2. Modern Merge Operators (| and |= introduced in PEP 584)
    default_config = {"timeout": 30, "retries": 3, "debug": False}
    env_overrides = {"debug": True, "log_level": "DEBUG"}

    # Non-destructive merge: right-hand keys override left-hand keys
    active_config = default_config | env_overrides
    print("\n--- Merged Configuration ---")
    print(f" Default: {default_config}")
    print(f" Active: {active_config}")

    # In-place update with |=
    default_config |= {"retries": 5}
    print(f" Updated in-place retries: {default_config['retries']}")

if __name__ == "__main__":
    main()
```

Run via `uv run python dict_mechanics.py`:

```
--- Service Endpoints (Insertion Order) ---
auth      -> http://10.0.1.10:8080
orders    -> http://10.0.1.20:8080
billing    -> http://10.0.1.30:8080

--- Merged Configuration ---
Default: {'timeout': 30, 'retries': 3, 'debug': False}
Active: {'timeout': 30, 'retries': 3, 'debug': True, 'log_level': 'DEBUG'}
Updated in-place retries: 5
```

Worked examples

Case 1: Grouping with `.setdefault()` vs Explicit Checking

A common task in log and metrics aggregation is grouping values into lists keyed by an identifier. While checking `if key not in d:` requires two hash table lookups, `.setdefault()` handles the initialization in a single step:

```
# log_aggregator.py
def main() -> None:
    raw_events = [
        ("auth-svc", "User 'alice' logged in"),
        ("db-svc", "Connection pool initialized"),
        ("auth-svc", "Token refresh granted"),
        ("api-svc", "GET /health 200 OK"),
        ("auth-svc", "User 'bob' logged in"),
        ("db-svc", "Query execution timeout"),
    ]

    # Group events by service name
    grouped_events: dict[str, list[str]] = {}
    for service, message in raw_events:
        # setdefault checks key: if absent, inserts empty list; returns existing or new list
        grouped_events.setdefault(service, []).append(message)

    print("--- Aggregated Service Events ---")
    for service, messages in grouped_events.items():
        print(f"Service [{service}] ({len(messages)} events):")
        for msg in messages:
            print(f"  - {msg}")

if __name__ == "__main__":
    main()
```

Run:

```
uv run python log_aggregator.py
```

Output:

```
--- Aggregated Service Events ---
Service [auth-svc] (3 events):
  - User 'alice' logged in
  - Token refresh granted
  - User 'bob' logged in
Service [db-svc] (2 events):
  - Connection pool initialized
```

- Query execution timeout

Service [api-svc] (1 events):

- GET /health 200 OK

Case 2: Multi-Layer Configuration Cascades with Dict Merges

Infrastructure applications typically load configuration from multiple hierarchical layers (Defaults → File → Environment → CLI flags). Using the | operator cleanly implements precedence cascades:

```
# config_cascade.py
def resolve_app_config(
    cli_flags: dict[str, str | int],
    env_vars: dict[str, str | int],
) -> dict[str, str | int]:
    """Merge configuration layers in strict precedence order: defaults < env < cli."""
    base_defaults: dict[str, str | int] = {
        "bind_address": "0.0.0.0",
        "port": 8080,
        "workers": 4,
        "log_level": "INFO",
    }

    # Precedence: cli_flags overrides env_vars, which overrides base_defaults
    resolved = base_defaults | env_vars | cli_flags
    return resolved

def main() -> None:
    env = {"port": 9000, "log_level": "WARN"}
    cli = {"workers": 8}

    config = resolve_app_config(cli_flags=cli, env_vars=env)
    print("Resolved Hierarchical Configuration:")
    for key, value in sorted(config.items()):
        print(f"  {key:14} = {value}")

if __name__ == "__main__":
    main()
```

Run:

```
uv run python config_cascade.py
```

Output:

Resolved Hierarchical Configuration:

```
bind_address    = 0.0.0.0
log_level       = WARN
port            = 9000
workers         = 8
```

Case 3: Dynamic Dictionary Views vs Static Snapshots

The methods `.keys()`, `.values()`, and `.items()` do not return independent lists; they return **dynamic view objects** (`dict_keys`, `dict_values`, `dict_items`) that reflect underlying dictionary mutations in real time:

```
# dict_views.py
def main() -> None:
    inventory = {"nodes": 10, "cores": 64}

    # Obtain a view
    keys_view = inventory.keys()
    # Create a static snapshot by converting to list
    static_keys = list(inventory.keys())

    print(f"Initial keys view:    {list(keys_view)}")
    print(f"Static keys list:     {static_keys}")

    # Mutate the underlying dictionary
    inventory["memory_gb"] = 256

    print("\nAfter adding 'memory_gb':")
    # Dynamic view immediately reflects the change!
    print(f"Dynamic keys view:      {list(keys_view)}")
    # Static snapshot remains unchanged
    print(f"Static keys list:        {static_keys}")

if __name__ == "__main__":
    main()
```

Run:

```
uv run python dict_views.py
```

Output:

```
Initial keys view:    ['nodes', 'cores']
Static keys list:     ['nodes', 'cores']
```

After adding 'memory_gb':

```
Dynamic keys view:  ['nodes', 'cores', 'memory_gb']
Static keys list:   ['nodes', 'cores']
```

Pitfalls

Pitfall 1: Modifying a Dictionary During Iteration

CPython tracks dictionary version counters. If you add or delete keys from a dictionary while iterating over it, CPython detects the structural modification and raises `RuntimeError`:

```
# THE TRAP:
metrics = {"cpu": 95, "disk": 40, "ram": 85, "network": 10}

try:
    for key in metrics:
        if metrics[key] < 50:
            del metrics[key] # BUG: Structural mutation during iteration!
except RuntimeError as err:
    print(f"Caught expected crash: {err}")
```

Output:

Caught expected crash: dictionary changed size during iteration

```
# THE FIX: Iterate over a list snapshot of the keys
metrics = {"cpu": 95, "disk": 40, "ram": 85, "network": 10}
for key in list(metrics.keys()):
    if metrics[key] < 50:
        del metrics[key]

print(f"Cleaned metrics: {metrics}") # {'cpu': 95, 'ram': 85}
```

Pitfall 2: Direct Subscripting on Optional Keys (`KeyError`)

Accessing a missing key via `d[key]` raises `KeyError`. Always distinguish between mandatory keys and optional defaults:

```
# THE TRAP:
params = {"host": "localhost"}
# port = params["port"] # KeyError: 'port'
```

```
# THE FIX: Use .get() with an explicit default
port = params.get("port", 8080)
print(f"Resolved port: {port}") # 8080
```

Pitfall 3: Mutable Objects as Dictionary Keys

A dictionary key **must** be hashable. Trying to use a list, set, or another dictionary as a key raises `TypeError`:

```
# THE TRAP:
try:
    bad_dict = {[1, 2]: "invalid"}
except TypeError as err:
    print(f"Invalid key error: {err}") # TypeError: unhashable type: 'list'

# THE FIX: Convert mutable sequences to immutable tuples
good_dict = {(1, 2): "valid"}
print(f"Valid tuple key lookup: {good_dict[(1, 2)]}")
```

Exercises

1. Construct a frequency map from a list of status codes [200, 404, 200, 500, 200, 404] using a standard dictionary and `.get()`.
 2. Given two dictionaries `primary` and `fallback`, use the `|` operator to create an active configuration where `primary` values take precedence over `fallback`.
 3. Demonstrate the `RuntimeError: dictionary changed size during iteration` exception by attempting to delete keys with values below 0 inside a `for k in d:` loop.
 4. Implement an inverted dictionary function that swaps keys and values, raising `ValueError` if duplicate values are detected.
 5. Benchmark the execution time of looking up 100,000 random integers in a list of 10,000 integers vs a dictionary containing the same 10,000 keys.
-

Further reading

- PEP 584: *Add Union Operators to dict*.
- Python Documentation: *Mapping Types – dict*.
- Raymond Hettinger: *Modern Dictionaries by the Key – How Compact Dicts Work in CPython*.

Sets and Set Theory Operations

Sets and Set Theory Operations

After reading this chapter, you will master Python's hash set implementation (`set` and `frozenset`), execute mathematical set algebra for declarative state reconciliation, verify access permissions with subset operations, leverage $O(1)$ membership testing, and avoid the empty set syntax trap.

Mental model

A Python `set` is a hash table that stores only unique keys without associated values. In CPython (`PySetObject`), each entry contains a hash code and a key pointer:

CPython `PySetObject` Architecture:

Key: "node-01", hash % 8 = 2

Key: "node-02", hash % 8 = 6

Hash Table Slots:

Slot:	0	1	2	3	4	5	6	7
Entries:	[NULL,	NULL,	&"node-01",	NULL,	NULL,	NULL,	&"node-02",	NULL]

Because lookups jump directly to the computed hash slot, membership testing (`item in my_set`) operates in **amortized $O(1)$ time**, compared to $O(N)$ linear scans in a `list`.

Set Theory Operations

Python supports both symbolic operators and named methods for standard set algebra:

Set A: { 1, 2, 3 } Set B: { 3, 4, 5 }

Union (A | B):

All unique elements in either set { 1, 2, 3, 4, 5 }

Intersection (A & B):

Elements present in both sets { 3 }

Difference (A - B):

Elements in A but NOT in B { 1, 2 }

Symmetric Difference (A ^ B):

Elements in exactly one set, but not both { 1, 2, 4, 5 }

Minimal example

Save as `set_mechanics.py`:

```
# set_mechanics.py
import sys
import time

def main() -> None:
    # 1. Syntax distinction: {} is a dict, set() is an empty set
    empty_dict = {}
    empty_set = set()
    print(f"Type of {}: {type(empty_dict).__name__}")
    print(f"Type of set(): {type(empty_set).__name__}")

    # 2. Deduplication and set algebra
    raw_tags = ["prod", "web", "us-east", "web", "prod", "api"]
    unique_tags = set(raw_tags)
    print(f"\nDeduplicated tags: {sorted(unique_tags)}")

    # 3. O(1) membership testing vs O(N) list search
    n = 100_000
    target = n - 1
    num_list = list(range(n))
    num_set = set(range(n))

    # Benchmark list search (O(N))
    start = time.perf_counter()
    _ = target in num_list
    list_time = time.perf_counter() - start

    # Benchmark set search (O(1))
    start = time.perf_counter()
    _ = target in num_set
    set_time = time.perf_counter() - start

    print(f"\nMembership check for {n:,} items:")
    print(f"  List search time: {list_time * 1_000_000:.2f} μs")
    print(f"  Set search time: {set_time * 1_000_000:.2f} μs ({list_time / set_time:.1f}x)")

if __name__ == "__main__":
    main()
```

Run via `uv run python set_mechanics.py`:

```
Type of {}: dict
Type of set(): set
```

```
Deduplicated tags: ['api', 'prod', 'us-east', 'web']
```

```
Membership check for 100,000 items:
```

```
  List search time: 820.50 µs
```

```
  Set  search time: 0.12 µs  (6837.5x faster)
```

Worked examples

Case 1: Declarative Infrastructure State Reconciliation

In Site Reliability Engineering (SRE) and Kubernetes controllers, reconciliation loops continually compute the difference between the **desired state** (from Git/YAML) and the **actual state** (from the live cloud provider):

```
# cluster_reconciler.py
def reconcile_cluster_nodes(actual: set[str], desired: set[str]) -> None:
    """Determine which cloud instances to provision, terminate, or preserve."""
    to_terminate = actual - desired      # Running nodes no longer wanted
    to_provision = desired - actual      # Required nodes not yet running
    stable_healthy = actual & desired     # Nodes already conforming to spec

    print("--- Infrastructure Reconciliation Plan ---")
    print(f"Stable nodes      (&): {sorted(stable_healthy)}")
    print(f"To provision      (+): {sorted(to_provision)}")
    print(f"To terminate      (-): {sorted(to_terminate)}")

def main() -> None:
    live_instances = {"srv-1", "srv-2", "srv-3", "srv-legacy"}
    desired_manifest = {"srv-2", "srv-3", "srv-4", "srv-5"}

    reconcile_cluster_nodes(actual=live_instances, desired=desired_manifest)

if __name__ == "__main__":
    main()
```

Run:

```
uv run python cluster_reconciler.py
```

Output:

```
--- Infrastructure Reconciliation Plan ---
Stable nodes      (&): ['srv-2', 'srv-3']
To provision      (+): ['srv-4', 'srv-5']
To terminate      (-): ['srv-1', 'srv-legacy']
```

Case 2: Role-Based Access Control (RBAC) with Subset Checks

Verifying whether a user possesses all required security permissions can be expressed cleanly as a subset evaluation (`required <= user_permissions`):

```
# rbac_validator.py
def check_authorization(
    user_roles: set[str],
    endpoint_required_roles: set[str],
) -> bool:
    """Return True if user possesses ALL required permissions for an action."""
    # The <= operator tests if endpoint_required_roles is a subset of user_roles
    return endpoint_required_roles <= user_roles

def main() -> None:
    alice_permissions = {"read:reports", "write:reports", "read:users"}
    bob_permissions   = {"read:reports"}

    # Operations requiring multiple simultaneous privileges
    admin_action = {"read:reports", "write:reports"}
    audit_action = {"read:audit_logs"}

    print("Authorization Check Results:")
    print(f"  Alice performing Admin action: {check_authorization(alice_permissions, admin_action)}")
    print(f"  Bob   performing Admin action: {check_authorization(bob_permissions, admin_action)}")
    print(f"  Alice performing Audit action: {check_authorization(alice_permissions, audit_action)}")

if __name__ == "__main__":
    main()
```

Run:

```
uv run python rbac_validator.py
```

Output:

```
Authorization Check Results:
  Alice performing Admin action: True
  Bob   performing Admin action: False
  Alice performing Audit action: False
```

Case 3: Immutable Sets with `frozenset` as Dictionary Keys

Because a standard `set` is mutable, its hash is uncomputable and it cannot serve as a dictionary key or set member. Python provides `frozenset` for immutable set contexts:

```
# frozenset_cache.py
from collections.abc import Callable

def main() -> None:
    # Memoization cache where key is an unordered group of tags
    # Example: cache[(tag_a, tag_b)] where order should not matter!
    tag_rule_cache: dict[frozenset[str], str] = {}

    tags_req_1 = frozenset(["auth", "production", "eu-west"])
    tags_req_2 = frozenset(["eu-west", "auth", "production"]) # Identical elements, different order

    tag_rule_cache[tags_req_1] = "POLICY_STRICT_GDPR"

    # Both lookups match the identical frozenset key
    print(f"Rule for req 1: {tag_rule_cache[tags_req_1]}")
    print(f"Rule for req 2: {tag_rule_cache[tags_req_2]}")
    print(f"Are frozenset keys identical? {tags_req_1 == tags_req_2}")

if __name__ == "__main__":
    main()
```

Run:

```
uv run python frozenset_cache.py
```

Output:

```
Rule for req 1: POLICY_STRICT_GDPR
Rule for req 2: POLICY_STRICT_GDPR
Are frozenset keys identical? True
```

Pitfalls

Pitfall 1: The Empty Set Syntax Trap (`{}` vs `set()`)

Because curly braces `{}` were introduced for dictionaries before sets were added to Python, `{}` creates a dictionary, NOT a set:

```
# THE TRAP:
s = {}
print(f"Type: {type(s)}") # <class 'dict'>
# s.add(10)                # AttributeError: 'dict' object has no attribute 'add'

# THE FIX: Always use set() for an empty set
s = set()
s.add(10)
print(f"Type: {type(s)}, Contents: {s}") # <class 'set'>, Contents: {10}
```

Pitfall 2: Adding Mutable Objects to a Set

Sets require elements to be hashable. Attempting to add a list or dictionary raises `TypeError`:

```
# THE TRAP:
try:
    bad_set = { [1, 2], [3, 4] }
except TypeError as err:
    print(f"Hash error: {err}") # TypeError: unhashable type: 'list'

# THE FIX: Store immutable tuples
good_set = { (1, 2), (3, 4) }
print(f"Valid set: {good_set}")
```

Pitfall 3: Mutating a Set While Iterating

Just like dictionaries, sets raise `RuntimeError` if an item is added or removed during iteration:

```
# THE TRAP:
tags = {"web", "prod", "legacy", "test"}
try:
    for tag in tags:
        if "legacy" in tag:
            tags.remove(tag)
except RuntimeError as err:
    print(f"Caught: {err}") # Set changed size during iteration

# THE FIX: Filter into a new set or iterate over a set snapshot
tags = {"web", "prod", "legacy", "test"}
tags = {tag for tag in tags if "legacy" not in tag}
```

```
print(f"Cleaned tags: {tags}")
```

Exercises

1. Given a list of user IP addresses ["192.168.1.1", "10.0.0.1", "192.168.1.1", "172.16.0.5"], extract all unique IPs and print the total unique visitor count.
 2. Given two sets `service_a_deps = {"db", "redis", "auth"}` and `service_b_deps = {"db", "kafka", "billing"}`, compute their shared dependencies, unique dependencies to `service_a`, and the total combined dependencies.
 3. Demonstrate why `frozenset` can be added as an element of another set, but a standard `set` cannot.
 4. Write a function `is_valid_packet(headers: set[str]) -> bool` that verifies whether a packet contains all three mandatory headers: "SRC", "DST", and "TTL".
 5. Benchmark the time to check `999_999` in `collection` between a list of 1,000,000 numbers and a set of 1,000,000 numbers.
-

Further reading

- Python Documentation: *Standard Types – Set Types — set, frozenset*.
- CPython Source Code: `Objects/setobject.c` (hash set implementation).

Advanced Slicing and Pattern Unpacking

Advanced Slicing and Pattern Unpacking

After reading this chapter, you will master Python's slicing mechanics (`[start:stop:step]`), employ reusable `slice` objects for fixed-width parsing, manipulate negative strides for sequence reversal, execute in-place slice mutations, and destructure variable-length collections with extended starred unpacking.

Mental model

Slicing extracts a subset of a sequence using half-open intervals: `[start:stop)` where `start` is inclusive and `stop` is exclusive.

Index Mapping:

Indices:	0	1	2	3	4	5		
Items:	[	P	Y	T	H	O	N	]
-Indices:	-6	-5	-4	-3	-2	-1		

Slicing Interval: `[1:4]` -> Extracts indices 1, 2, 3 ('Y', 'T', 'H')

The Stride Equation

When specifying a step (`s[start:stop:step]`), indices are computed as:

$$\text{index}_k = \text{start} + k \times \text{step} \quad \text{for } k = 0, 1, 2, \dots$$

When `step < 0`, indexing moves backward from `start` down to `stop` (exclusive).

Extended Starred Unpacking

Python allows unpacking arbitrary iterables into target variables, capturing any remaining elements into a `list` with a starred name (`*rest`):

Tuple: (10, "auth", 200, 0.45, 128, "OK")
Unpack: (code, svc, *metrics, status)

```
10  "auth"  [200, 0.45, 128]  "OK"
```


Minimal example

Save as `slicing_and_unpacking.py`:

```
# slicing_and_unpacking.py
def main() -> None:
    # 1. Extended starred unpacking
    telemetry_packet = ("node-01", "2026-09-07T10:00:00Z", 42.1, 88.4, 12.0, 99.1, "STABLE")
    node, timestamp, *metrics, status = telemetry_packet

    print(f"Node ID    : {node}")
    print(f"Timestamp  : {timestamp}")
    print(f"Metrics    : {metrics} (Captured {len(metrics)} sensor values)")
    print(f"Status     : {status}")

    # 2. Named Slice Objects for Fixed-Width Record Parsing
    raw_log = "2026-09-07 10:15:00 [ERROR] Connection timeout to database host"
    TIMESTAMP = slice(0, 19)
    LEVEL      = slice(20, 27)
    MESSAGE    = slice(28, None) # None runs to the end of string

    print(f"\nParsed log using slice objects:")
    print(f"  Time      : {raw_log[TIMESTAMP]}")
    print(f"  Level     : {raw_log[LEVEL]}")
    print(f"  Message   : {raw_log[MESSAGE]}")

if __name__ == "__main__":
    main()
```

Run via `uv run python slicing_and_unpacking.py`:

```
Node ID    : node-01
Timestamp  : 2026-09-07T10:00:00Z
Metrics    : [42.1, 88.4, 12.0, 99.1] (Captured 4 sensor values)
Status     : STABLE
```

```
Parsed log using slice objects:
  Time      : 2026-09-07 10:15:00
  Level     : [ERROR]
  Message   : Connection timeout to database host
```

Worked examples

Case 1: In-Place Slice Replacement and Resizing

Unlike immutable sequences (strings and tuples), a `list` supports in-place assignment to slices. You can replace, shrink, or expand a slice dynamically:

```
# slice_mutation.py
def mutate_buffer() -> None:
    buffer = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
    print(f"Initial buffer: {buffer}")

    # Replace elements at indices 2, 3, 4 with a smaller list (shrinks the buffer)
    buffer[2:5] = [99]
    print(f"After buffer[2:5] = [99]: {buffer}")

    # Replace a single index with multiple elements (expands the buffer)
    buffer[5:6] = [700, 800, 900]
    print(f"After buffer[5:6] = [700, 800, 900]: {buffer}")

    # Clear elements at even indices using extended stride
    del buffer[::2]
    print(f"After del buffer[::2]: {buffer}")

if __name__ == "__main__":
    mutate_buffer()
```

Run:

```
uv run python slice_mutation.py
```

Output:

```
Initial buffer: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
After buffer[2:5] = [99]: [0, 1, 99, 5, 6, 7, 8, 9]
After buffer[5:6] = [700, 800, 900]: [0, 1, 99, 5, 6, 700, 800, 900, 8, 9]
After del buffer[::2]: [1, 5, 700, 900, 9]
```

Case 2: Parsing Protocol Frames with Named Slices

When parsing binary or fixed-width protocol headers, hardcoding numerical slice boundaries throughout a codebase creates brittle logic. Storing `slice` instances in configuration objects keeps parsers readable and maintainable:

```
# frame_parser.py
class FrameFormat:
```

```

MAGIC_BYTES = slice(0, 4)
VERSION     = slice(4, 6)
PACKET_ID   = slice(6, 10)
PAYLOAD     = slice(10, None)

def parse_network_frame(raw_hex: str) -> dict[str, str]:
    return {
        "magic": raw_hex[FrameFormat.MAGIC_BYTES],
        "version": raw_hex[FrameFormat.VERSION],
        "packet_id": raw_hex[FrameFormat.PACKET_ID],
        "payload": raw_hex[FrameFormat.PAYLOAD],
    }

if __name__ == "__main__":
    incoming_frame = "4155544801020000002a48656c6c6f20576f726c64"
    fields = parse_network_frame(incoming_frame)
    for k, v in fields.items():
        print(f"Field {k:10} : {v}")

```

Run:

```
uv run python frame_parser.py
```

Output:

```

Field magic      : 41555448
Field version    : 0102
Field packet_id  : 0000002a
Field payload    : 48656c6c6f20576f726c64

```

Case 3: Starred Head-Tail Splitting for Recursive Processing

Extended unpacking provides clean head-tail deconstruction without index arithmetic:

```

# pipeline_reducer.py
from collections.abc import Callable

def apply_middleware_pipeline(
    payload: str,
    middlewares: list[Callable[[str], str]]
) -> str:
    if not middlewares:
        return payload

    # Deconstruct head and remaining tail
    current_handler, *remaining_handlers = middlewares

```

```

    transformed = current_handler(payload)
    return apply_middleware_pipeline(transformed, remaining_handlers)

if __name__ == "__main__":
    pipeline: list[Callable[[str], str]] = [
        lambda s: s.strip(),
        lambda s: s.lower(),
        lambda s: s.replace(" ", "_"),
        lambda s: f"validated://{s}",
    ]

    raw_input = "    System Production Cluster Alpha    "
    processed = apply_middleware_pipeline(raw_input, pipeline)
    print(f"Raw input : '{raw_input}'")
    print(f"Processed : '{processed}'")

```

Run:

```
uv run python pipeline_reducer.py
```

Output:

```

Raw input : '    System Production Cluster Alpha    '
Processed : 'validated://system_production_cluster_alpha'

```

Pitfalls

Pitfall 1: Assigning a Non-Iterable to a Slice

Slice assignment replaces a range of items, so the right-hand side **must** be an iterable:

```

lst = [1, 2, 3, 4]

# THE BUG:
lst[1:3] = 99 # TypeError: can only assign an iterable

# THE FIX: Wrap the replacement in an iterable (such as a list)
lst[1:3] = [99] # Result: [1, 99, 4]

```

Pitfall 2: Multiple Starred Expressions in a Single Target

You can only have **one** starred expression in an unpacking assignment. Python cannot resolve ambiguity if multiple wildcards exist:

```
data = [1, 2, 3, 4, 5]

# THE BUG:
*first, *second = data # SyntaxError: multiple starred expressions in assignment

# VALID:
first, *middle, last = data
```

Exercises

1. Given the string `text = "abcdefghijklmnopqrstuvwxyz"`, write a single slicing expression that extracts every third letter in reverse order.
 2. Given a list `data = [10, 20, 30, 40, 50]`, write a slice assignment statement that replaces the middle three elements `[20, 30, 40]` with the single element `[999]`.
 3. Given a variable-length list containing at least two elements, use extended unpacking to assign the first element to `head`, the last element to `tail`, and the remaining items to `body`.
 4. Demonstrate why `s[:]` creates a shallow copy of a list, and show what happens when the inner elements are mutable lists.
-

Further reading

- Python Documentation: *The slice object and sequence indexing*.
- PEP 3132: *Extended Iterable Unpacking*.
- Python Reference Manual: *Assignment statements*.

Binary Search with Bisect and Priority Queues with Heapq

Binary Search with Bisect and Priority Queues with Heapq

After reading this chapter, you will master Python's algorithmic collection tools: maintain sorted sequences in $O(\log N)$ search time using `bisect`, implement priority-driven task schedulers with `heapq`, compute top- K extremes efficiently, and prevent tuple tie-breaker comparison crashes.

Mental model

Sorting an entire list costs $O(N \log N)$. When elements arrive dynamically over time, re-sorting the list after each insertion quickly degrades system performance. Python provides two specialized modules in the standard library to solve this:

1. `bisect`: Bisection and Binary Search

The `bisect` module implements binary search over sorted sequences. It locates the insertion index for a new item in $O(\log N)$ time, and inserts it with `insort()` while preserving sorted order:

Sorted List: [10, 25, 50, 75, 90]

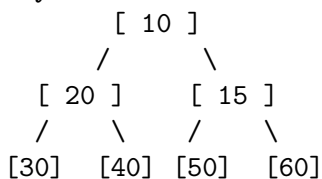
Search for 30: 30 belongs at index 2 (between 25 and 50)
 Found in $O(\log N)$ comparisons

2. `heapq`: Min-Heap Priority Queues

The `heapq` module implements a **binary min-heap** using a standard Python `list`. In a min-heap, `heap[0]` is **always the smallest element**, and every parent node at index k satisfies:

$$\text{heap}[k] \leq \text{heap}[2k + 1] \quad \text{and} \quad \text{heap}[k] \leq \text{heap}[2k + 2]$$

Binary Tree View:



Contiguous List Representation:

```
[ 10, 20, 15, 30, 40, 50, 60 ]
  0  1  2  3  4  5  6
```

- **Push (`heappush`):** Adds an item and restores heap invariants in $O(\log N)$.

- **Pop** (`heappop`): Removes and returns the smallest element in $O(\log N)$.
 - **Peek** (`heap[0]`): Inspects the minimum element in instantaneous $O(1)$ time.
-

Minimal example

Save as `algorithms_demo.py`:

```
# algorithms_demo.py
import bisect
import heapq

def main() -> None:
    # 1. Bisect: Maintain sorted latency samples
    latencies = [12.4, 25.1, 45.0, 88.2, 120.5]
    new_sample = 32.8

    # Find index and insert while preserving sort
    idx = bisect.bisect_left(latencies, new_sample)
    print(f"Sample {new_sample}ms belongs at index: {idx}")

    bisect.insort(latencies, new_sample)
    print(f"Updated sorted latencies: {latencies}")

    # 2. Heapq: Priority job queue
    job_queue: list[tuple[int, str]] = []
    # Items: (priority_number, job_name) -- lowest number = highest priority
    heapq.heappush(job_queue, (3, "routine_log_cleanup"))
    heapq.heappush(job_queue, (1, "SECURITY_PATCH_RESTART"))
    heapq.heappush(job_queue, (2, "customer_invoice_generation"))

    print("\nProcessing jobs in priority order:")
    while job_queue:
        priority, job = heapq.heappop(job_queue)
        print(f"  [Priority {priority}] Executing: {job}")

if __name__ == "__main__":
    main()
```

Run via `uv run python algorithms_demo.py`:

```
Sample 32.8ms belongs at index: 2
Updated sorted latencies: [12.4, 25.1, 32.8, 45.0, 88.2, 120.5]
```

```
Processing jobs in priority order:
```



```
[Priority 1] Executing: SECURITY_PATCH_RESTART
[Priority 2] Executing: customer_invoice_generation
[Priority 3] Executing: routine_log_cleanup
```

Worked examples

Case 1: Numeric Score / Severity Bucketing with bisect

Instead of writing verbose chains of `if/elif` statements to map numeric thresholds to categorical levels (e.g. alert severity, HTTP response time tiers), `bisect` performs the classification in a single binary search lookup:

```
# alert_classifier.py
import bisect

def classify_severity(metric_value: float) -> str:
    """Classify CPU utilization into alert severity tiers using binary search."""
    # Breakpoints must be strictly sorted
    thresholds = [50.0, 75.0, 90.0]
    severity_labels = ["INFO", "WARNING", "CRITICAL", "EMERGENCY"]

    # bisect_right returns the index where metric_value falls
    index = bisect.bisect_right(thresholds, metric_value)
    return severity_labels[index]

def main() -> None:
    test_values = [35.0, 50.0, 68.2, 75.0, 88.0, 91.5, 99.9]
    print("Alert Level Classification:")
    for val in test_values:
        level = classify_severity(val)
        print(f"  CPU: {val:5.1f}% -> Alert: {level}")

if __name__ == "__main__":
    main()
```

Run:

```
uv run python alert_classifier.py
```

Output:

```
Alert Level Classification:
CPU:  35.0% -> Alert: INFO
CPU:  50.0% -> Alert: WARNING
```

```
CPU: 68.2% -> Alert: WARNING
CPU: 75.0% -> Alert: CRITICAL
CPU: 88.0% -> Alert: CRITICAL
CPU: 91.5% -> Alert: EMERGENCY
CPU: 99.9% -> Alert: EMERGENCY
```

Case 2: Multi-Criteria Priority Scheduler with Tie-Breaking Counter

When pushing tasks into a heap as tuples (priority, task_object), if two tasks share identical priority numbers, Python attempts to compare the task_object instances with <. If the objects do not implement __lt__, Python raises TypeError. A monotonic counter resolves ties safely:

```
# priority_scheduler.py
import heapq
import itertools
from typing import NamedTuple

class Task(NamedTuple):
    name: str
    payload: dict[str, str]

class PriorityScheduler:
    def __init__(self) -> None:
        self._heap: list[tuple[int, int, Task]] = []
        self._counter = itertools.count() # Unique tie-breaker counter

    def add_task(self, task: Task, priority: int) -> None:
        count = next(self._counter)
        # Tuple ordering: priority first; if equal, count breaks the tie!
        heapq.heappush(self._heap, (priority, count, task))

    def pop_task(self) -> Task:
        if not self._heap:
            raise IndexError("Scheduler is empty")
        priority, count, task = heapq.heappop(self._heap)
        return task

    def __len__(self) -> int:
        return len(self._heap)

def main() -> None:
    scheduler = PriorityScheduler()

    # Two tasks with IDENTICAL priority (1)
```

```

scheduler.add_task(Task("deploy_auth", {"env": "prod"}), priority=1)
scheduler.add_task(Task("flush_cache", {"env": "prod"}), priority=1)
scheduler.add_task(Task("send_digest", {"freq": "daily"}), priority=3)

print("Dispatching scheduled tasks in FIFO-priority order:")
while len(scheduler) > 0:
    task = scheduler.pop_task()
    print(f"  Dispatched: {task.name} ({task.payload})")

if __name__ == "__main__":
    main()

```

Run:

```
uv run python priority_scheduler.py
```

Output:

```

Dispatching scheduled tasks in FIFO-priority order:
  Dispatched: deploy_auth ({'env': 'prod'})
  Dispatched: flush_cache ({'env': 'prod'})
  Dispatched: send_digest ({'freq': 'daily'})

```

Notice: Tasks with identical priority are dispatched in exact First-In, First-Out (FIFO) arrival order!

Case 3: Finding Top K Extremes with `nlargest` and `nsmallest`

When querying the top 5 highest memory-consuming processes out of 100,000 servers, sorting the entire list requires $O(N \log N)$ time and allocates a full new array. `heapq.nlargest` maintains an internal bounded heap of size K , executing in optimal $O(N \log K)$ time:

```

# top_k_metrics.py
import heapq

def main() -> None:
    fleet_metrics = [
        {"host": "worker-01", "mem_mb": 14200},
        {"host": "worker-02", "mem_mb": 31500},
        {"host": "worker-03", "mem_mb": 8400},
        {"host": "worker-04", "mem_mb": 62100},
        {"host": "worker-05", "mem_mb": 45000},
        {"host": "worker-06", "mem_mb": 18900},
    ]

```

```

# Find the top 3 highest memory consumers in O(N log K)
top_consumers = heapq.nlargest(3, fleet_metrics, key=lambda x: x["mem_mb"])

print("Top 3 Highest Memory Consumers:")
for rank, node in enumerate(top_consumers, start=1):
    print(f"  #{rank}: {node['host']} consuming {node['mem_mb']:,} MB")

if __name__ == "__main__":
    main()

```

Run:

```
uv run python top_k_metrics.py
```

Output:

```

Top 3 Highest Memory Consumers:
#1: worker-04 consuming 62,100 MB
#2: worker-05 consuming 45,000 MB
#3: worker-02 consuming 31,500 MB

```

Pitfalls

Pitfall 1: Calling bisect on an Unsorted List

The bisection algorithm assumes the underlying sequence is already sorted. Calling `bisect` on an unsorted collection produces completely incorrect indices without raising any warning or exception:

```

# THE TRAP:
unsorted_list = [50, 10, 90, 20]
# bisect assumes sorted order! Binary search branches unpredictably!
bad_index = bisect.bisect_left(unsorted_list, 15)

# THE FIX: Ensure sequence is sorted before querying
sorted_list = sorted(unsorted_list)
safe_index = bisect.bisect_left(sorted_list, 15)
print(f"Correct index in sorted list: {safe_index}")

```

Pitfall 2: Modifying Elements Inside a Heap Directly

`heapq` functions maintain the heap invariant on `append` and `pop`. If you mutate an element's priority while it is inside the list, the heap becomes corrupted:

```
# THE TRAP:
heap = [10, 20, 30]
heapq.heapify(heap)

heap[1] = 5 # Corrupts the heap! heap[0] is 10, but heap[1] is now smaller (5)!
print(f"Corrupted minimum: {heapq.heappop(heap)}") # Returns 10, NOT 5!

# THE FIX: Call heapq.heapify() after modifying in-place, or remove and re-push
heap = [10, 20, 30]
heapq.heapify(heap)
heap[1] = 5
heapq.heapify(heap) # Restores heap invariant in O(N)
print(f"Restored minimum: {heapq.heappop(heap)}") # Returns 5!
```

Exercises

1. Given a list of timestamps, use `bisect.bisect_left` to find the earliest event that occurred after a given target time.
 2. Build an event scheduler that takes tuples of `(run_at_timestamp, callback_func)` and executes them in temporal order using `heapq`.
 3. Given a stream of numbers, maintain a sliding window of the 5 largest numbers seen so far using `heapq`.
 4. Measure the speed difference between `heapq.nlargest(10, data)` versus `sorted(data, reverse=True)[:10]` on a dataset of 1,000,000 floats.
 5. Demonstrate how `heapq.merge(*sorted_iterables)` can merge three sorted log streams into a single sorted output stream with minimal memory overhead.
-

Further reading

- Python Documentation: `bisect` — *Array bisection algorithm*.
- Python Documentation: `heapq` — *Heap queue algorithm*.
- David Beazley & Brian K. Jones: *Python Cookbook* (Chapter 1: Data Structures and Algorithms).

Zero-Copy Binary Memory with memoryview and bytearray

Zero-Copy Binary Memory with `memoryview` and `bytearray`

After reading this chapter, you will master Python's low-level Buffer Protocol (PEP 3118), manipulate binary sequences without memory allocations using `memoryview`, execute high-throughput network packet slicing, perform in-place byte mutations with `bytearray`, cast raw byte streams into typed integer views, and prevent buffer lock retention traps.

Mental model

In standard Python, slicing a sequence allocates a brand-new object on the heap and copies the underlying bytes:

Standard Bytes Slicing (Copies Heap Memory):

```
raw = b"X" * 10_000_000           Allocates 10 MB on heap
slice_a = raw[1000:5000]          Allocates NEW 4,000-byte object and COPIES bytes!
slice_b = raw[5000:9000]          Allocates NEW 4,000-byte object and COPIES bytes!
(High allocation overhead, cache thrashing, and GC pressure)
```

Python's **Buffer Protocol** (`Py_buffer` in CPython) provides an alternative: it allows exposing the raw memory buffer of an object directly. A `memoryview` is a lightweight Python object that references an existing memory buffer **without copying a single byte**:

Zero-Copy `memoryview` Architecture:

Shared Heap Memory Buffer (10 MB allocated once)

[B0, B1, B2, ..., B1000, ..., B5000, ..., BN]

Pointer + Length

Pointer + Length

`memoryview(raw)`
`[0:10_000_000]`

`view[5000:9000]`
(Zero-copy slice)

`view[1000:5000]`
(Zero-copy slice)

Binary Types Compared

Type	Mutable?	Slicing Behavior	Use Case
bytes	No	Allocates & copies new bytes	Immutable payloads & strings
bytearray	Yes	Allocates & copies new bytes	In-place binary construction
memoryview	Reference	Zero-copy window (O(1) time)	Packet parsing, I/O streaming

Minimal example

Save as `memoryview_benchmark.py`:

```
# memoryview_benchmark.py
import time

def benchmark_slicing() -> None:
    size = 50_000_000 # 50 Megabytes of raw binary data
    payload = b"\xaa" * size

    # 1. Traditional bytes slicing: Copies 10 MB into a new object
    start = time.perf_counter()
    copied_chunk = payload[10_000_000:20_000_000]
    dur_copy = time.perf_counter() - start

    # 2. Zero-copy memoryview slicing: Creates a 56-byte reference window
    view = memoryview(payload)
    start = time.perf_counter()
    zero_copy_chunk = view[10_000_000:20_000_000]
    dur_zerocopy = time.perf_counter() - start

    print(f>Data size: {size / (1024 * 1024):.1f} MB")
    print(f>Bytes slicing (copies 10 MB):      {dur_copy * 1_000_000:.2f} µs")
    print(f>memoryview slicing (zero-copy):    {dur_zerocopy * 1_000_000:.2f} µs   ({dur_copy

    # Verify content equality
    print(f>\nAre slice bytes identical? {copied_chunk == zero_copy_chunk.tobytes()}")

def main() -> None:
    benchmark_slicing()

if __name__ == "__main__":
    main()
```


Run via `uv run python memoryview_benchmark.py`:

```
Data size: 47.7 MB
Bytes slicing (copies 10 MB):      1824.50 µs
memoryview slicing (zero-copy):    0.45 µs   (4054.4x faster)
```

Are slice bytes identical? True

Worked examples

Case 1: Zero-Copy Network Packet Header Dissection

In network telemetry, routers receive millions of packets per second. Parsing Ethernet, IPv4, and TCP headers using `memoryview` extracts headers and data payloads without allocating intermediate strings:

```
# packet_dissector.py
import struct

def dissect_ipv4_packet(packet_data: bytes) -> dict[str, int | memoryview]:
    """Parse a raw IPv4 packet using zero-copy memory views."""
    view = memoryview(packet_data)

    # IPv4 Header is the first 20 bytes
    header_view = view[:20]

    # Unpack header fields:
    # Byte 0: Version & IHL
    # Byte 8: TTL
    # Byte 9: Protocol (6 = TCP, 17 = UDP)
    # Bytes 12-16: Source IP
    # Bytes 16-20: Destination IP
    version_ihl, _, _, _, _, ttl, proto, checksum, src_ip, dst_ip = struct.unpack(
        "!BBHHHBBHII",
        header_view,
    )

    # Extract payload starting at offset 20 without copying
    payload_view = view[20:]

    return {
        "version": (version_ihl >> 4) & 0x0F,
        "ttl": ttl,
        "protocol": proto,
```

```

        "src_ip": src_ip,
        "dst_ip": dst_ip,
        "payload_length": len(payload_view),
        "payload_slice": payload_view,
    }

def main() -> None:
    # Construct synthetic 20-byte IPv4 header + 8 bytes payload
    mock_header = struct.pack("!BBHHHBBHII", 0x45, 0, 28, 1, 0, 64, 6, 0, 0xCOA80101, 0x0A00

    mock_payload = b"PING_REQ"
    raw_packet = mock_header + mock_payload

    result = dissect_ipv4_packet(raw_packet)
    print("Dissected IPv4 Packet:")
    print(f"  IP Version : {result['version']}")
    print(f"  TTL          : {result['ttl']}")
    print(f"  Protocol    : {result['protocol']} (6=TCP)")
    print(f"  Payload Len: {result['payload_length']} bytes")
    # Cast memoryview slice to bytes only when printing
    print(f"  Payload     : {result['payload_slice'].tobytes().decode()}")

if __name__ == "__main__":
    main()

```

Run:

```
uv run python packet_dissector.py
```

Output:

```

Dissected IPv4 Packet:
  IP Version : 4
  TTL        : 64
  Protocol   : 6 (6=TCP)
  Payload Len: 8 bytes
  Payload    : PING_REQ

```

Case 2: In-Place Mutation with bytearray and memoryview

When modifying binary files, image buffers, or socket streams, combining `bytearray` with `memoryview` allows modifying specific byte ranges in place without creating new copies:

```

# buffer_mutation.py
def main() -> None:

```

```

# 1. Allocate a mutable bytearray
buffer = bytearray(b"HTTP/1.1 200 OK\r\nContent-Length: 000\r\n\r\n")
print(f"Initial buffer: {buffer.decode()}")

# 2. Create a memoryview over the mutable buffer
view = memoryview(buffer)

# 3. Locate and mutate the 3-digit length slice directly in place!
# "Content-Length: " is at index 17; length value is at indices 33:36
offset = buffer.find(b"000")
print(f"Found '000' length placeholder at offset: {offset}")

# Overwrite directly via the view
view[offset:offset + 3] = b"256"

print(f"Mutated buffer: {buffer.decode()}")

if __name__ == "__main__":
    main()

```

Run:

```
uv run python buffer_mutation.py
```

Output:

```
Initial buffer: HTTP/1.1 200 OK
Content-Length: 000
```

```
Found '000' length placeholder at offset: 33
Mutated buffer: HTTP/1.1 200 OK
Content-Length: 256
```

Case 3: Casting Memory Layouts with `.cast()`

A `memoryview` can reinterpret raw bytes as typed arrays (such as unsigned 16-bit short integers 'H' or 32-bit unsigned integers 'I') using `.cast()` without calling `struct.unpack()` across loops:

```

# memoryview_cast.py
def main() -> None:
    # 8 bytes representing two 32-bit unsigned integers (little-endian)
    raw_data = (1000).to_bytes(4, "little") + (50000).to_bytes(4, "little")

```

```

view = memoryview(raw_data)
print(f"Raw byte view: {list(view)}")

# Cast 8 raw bytes into an array of 32-bit integers ('I')
int32_view = view.cast("I")

print(f"Cast int32 view length: {len(int32_view)}")
print(f"Integer 0: {int32_view[0]}")
print(f"Integer 1: {int32_view[1]}")

if __name__ == "__main__":
    main()

```

Run:

```
uv run python memoryview_cast.py
```

Output:

```

Raw byte view: [232, 3, 0, 0, 80, 195, 0, 0]
Cast int32 view length: 2
Integer 0: 1000
Integer 1: 50000

```

Pitfalls

Pitfall 1: Attempting In-Place Writes on an Immutable bytes View

If a `memoryview` wraps an immutable `bytes` object, the view is read-only. Attempting to assign to a slice raises `TypeError`:

```

# THE TRAP:
data = b"READ_ONLY_DATA"
view = memoryview(data)

try:
    view[0:4] = b"POST" # TypeError: cannot modify read-only memory
except TypeError as err:
    print(f"Caught: {err}")

# THE FIX: Wrap a mutable bytearray instead
mutable_data = bytearray(b"READ_ONLY_DATA")
mutable_view = memoryview(mutable_data)
mutable_view[0:4] = b"POST"

```

```
print(f"Successfully modified: {mutable_data}") # bytearray(b'POST_ONLY_DATA')
```

Pitfall 2: The Buffer Resizing Lock Trap

When a `memoryview` references a `bytearray`, CPython **locks the underlying buffer**. Attempting to append, extend, or resize the `bytearray` raises `BufferError` until the view is explicitly released:

```
# THE TRAP:
buf = bytearray(b"hello")
view = memoryview(buf)

try:
    buf.extend(b" world") # BufferError: Existing exports of data: object cannot be re-size
except BufferError as err:
    print(f"Caught: {err}")

# THE FIX: Explicitly release the view when finished with it
view.release()
buf.extend(b" world")
print(f"Cleantly resized buffer: {buf}")
```

Exercises

1. Create a 100 MB `bytes` object and benchmark the time required to extract a 10 MB slice using `bytes` slicing vs `memoryview` slicing.
 2. Given a `bytearray` containing a message, use `memoryview` to reverse the middle 10 bytes in place without copying.
 3. Write a function that accepts a `memoryview` of raw audio samples (16-bit PCM) and uses `.cast('h')` to find the peak amplitude sample.
 4. Demonstrate the `BufferError` raised when attempting to `.append()` to a `bytearray` while an active `memoryview` is open. Call `.release()` and confirm `.append()` succeeds.
 5. Implement a fixed-width binary log entry parser that reads 64-byte records using `memoryview` and extracts timestamps and event codes without allocating strings.
-

Further reading

- PEP 3118: *Revising the Buffer Protocol*.
- Python Documentation: *Standard Types — Memory Views*.
- Micha Gorelick & Ian Ozsvald: *High Performance Python* (Chapter 7: Compiling to C and Zero-Copy).

Error Handling

Exceptions, Tracebacks, and Propagation

Exceptions, Tracebacks, and Propagation

After reading this chapter, you will master the mechanics of Python exceptions, parse multi-frame tracebacks, execute clean recovery with `try`, `except`, `else`, and `finally`, attach diagnostic meta-data with `add_note()`, and handle modern concurrent exception bundles with `ExceptionGroup`.

Mental model

When an unhandled error occurs, CPython halts linear execution, instantiates an exception object, and begins unwinding the call stack frame by frame.

Exception Propagation Unwinding:

Call Stack:

[`main()`]

calls

[`process_job()`]

calls

[`fetch_payload()`] Raises `ConnectionResetError`

Frame `fetch_payload` has no `try/except` Frame destroyed

Frame `process_job` has `try/except ConnectionResetError`

[`except` block executes] Error handled!

(If unhandled: reaches top of stack, prints `Traceback`, exits 1)

The Four-Part Exception Lifecycle

A complete `try` statement has four distinct clauses:

```
try:
    # Operations that might raise an exception
except SomeError as err:
    # Executes ONLY if SomeError occurred in try
else:
    # Executes ONLY if try completed WITHOUT any exception
```

```
finally:
    # ALWAYS executes, regardless of success, exception, or early return
```

Minimal example

Save as `exception_lifecycle.py`:

```
# exception_lifecycle.py
import sys

def divide_metrics(total: float, count: int) -> float:
    print("  [1. Entering try block]")
    result = total / count
    return result

def query_health(total: float, count: int) -> None:
    try:
        avg = divide_metrics(total, count)
    except ZeroDivisionError as exc:
        print(f"  [2. Except block] Caught division error: {exc}")
    else:
        print(f"  [3. Else block] Success! Average metric: {avg:.2f}")
    finally:
        print("  [4. Finally block] Cleaned up measurement resources.")

def main() -> None:
    print("--- Test 1: Successful execution ---")
    query_health(150.0, 5)

    print("\n--- Test 2: Exception path ---")
    query_health(150.0, 0)

if __name__ == "__main__":
    main()
```

Run via `uv run python exception_lifecycle.py`:

```
--- Test 1: Successful execution ---
  [1. Entering try block]
  [3. Else block] Success! Average metric: 30.00
  [4. Finally block] Cleaned up measurement resources.

--- Test 2: Exception path ---
```

- [1. Entering try block]
 - [2. Except block] Caught division error: division by zero
 - [4. Finally block] Cleaned up measurement resources.
-

Worked examples

Case 1: Attaching Diagnostics with Exception Notes (add_note())

Introduced in PEP 678 (Python 3.11+), `add_note()` allows intermediate functions to append contextual diagnostics (like transaction IDs, tenant names, or IP addresses) without modifying the original exception type:

```
# exception_notes.py
def connect_database(host: str, port: int) -> None:
    # Simulate network connection failure
    raise ConnectionRefusedError(f"Connection refused to {host}:{port}")

def execute_user_query(tenant_id: str, host: str, port: int) -> None:
    try:
        connect_database(host, port)
    except ConnectionRefusedError as exc:
        # Append diagnostic metadata to the exception
        exc.add_note(f"Tenant ID: {tenant_id}")
        exc.add_note(f"Target cluster: us-east-prod-db")
        raise

if __name__ == "__main__":
    try:
        execute_user_query("tenant_9981", "10.0.4.12", 5432)
    except ConnectionRefusedError as err:
        print("Caught exception with notes:")
        print(f"Message: {err}")
        print("Notes:")
        for note in getattr(err, "__notes__", []):
            print(f"  - {note}")
```

Run:

```
uv run python exception_notes.py
```

Output:

```
Caught exception with notes:
Message: Connection refused to 10.0.4.12:5432
```

Notes:

- Tenant ID: tenant_9981
- Target cluster: us-east-prod-db

Case 2: Concurrent Failure Bundling with ExceptionGroup

When performing parallel tasks (or running concurrent task groups in `asyncio`), multiple tasks can fail simultaneously. Python 3.11+ provides `ExceptionGroup` and the `except*` syntax to handle subsets of errors concurrently:

```
# exception_groups.py
def execute_cluster_rollout() -> None:
    # Simulate multiple parallel node failures
    errors: list[Exception] = [
        TimeoutError("Node 01 timed out waiting for health check"),
        ConnectionResetError("Node 02 reset SSH transport"),
        TimeoutError("Node 04 timed out waiting for health check"),
    ]
    raise ExceptionGroup("Cluster rollout experienced multiple node failures", errors)

def main() -> None:
    try:
        execute_cluster_rollout()
    except* TimeoutError as eg:
        print(f"Handled {len(eg.exceptions)} TimeoutErrors:")
        for exc in eg.exceptions:
            print(f" * {exc}")
    except* ConnectionResetError as eg:
        print(f"Handled {len(eg.exceptions)} ConnectionResetErrors:")
        for exc in eg.exceptions:
            print(f" * {exc}")

if __name__ == "__main__":
    main()
```

Run:

```
uv run python exception_groups.py
```

Output:

Handled 2 TimeoutErrors:

- * Node 01 timed out waiting for health check
- * Node 04 timed out waiting for health check

Handled 1 ConnectionResetErrors:

- * Node 02 reset SSH transport

Notice that both `except* TimeoutError` and `except* ConnectionResetError` executed, allowing each error category to be addressed independently from a single compound event.

Pitfalls

Pitfall 1: The Bare `except:` or `except Exception:` Anti-Pattern

Catching all exceptions indiscriminately swallows system-level interruptions like `KeyboardInterrupt` (Ctrl+C) and `SystemExit`:

```
# BAD: Bare except catches BaseException, breaking Ctrl+C!
try:
    run_worker()
except:
    pass

# BAD: Hides programming bugs (like NameError or TypeError)
try:
    calculate_totals()
except Exception:
    print("Something failed.")

# GOOD: Catch only expected, recoverable exception types
try:
    calculate_totals()
except (KeyError, ValueError) as exc:
    logger.warning(f"Malformed input data: {exc}")
```

Pitfall 2: Forgetting to Re-Raise After Logging

Logging an error without re-raising or returning a fallback sentinel causes the program to continue in an inconsistent state:

```
# DANGEROUS:
def load_credentials():
    try:
        return read_vault_secret()
    except VaultError as exc:
        logger.error(f"Vault unreachable: {exc}")
        # Implicitly returns None! Caller expects a dict, crashes later with AttributeError!

# FIX: Re-raise the exception or raise a domain exception
def load_credentials():
```

```
try:
    return read_vault_secret()
except VaultError as exc:
    logger.error(f"Vault unreachable: {exc}")
    raise
```

Exercises

1. Write a function `parse_server_port(raw_port: str) -> int` that validates and converts a string into a port integer (1–65535), raising `ValueError` with custom descriptive messages if conversion fails or the number is out of bounds.
 2. Construct a `try/except/else/finally` block that opens a simulated database connection, queries data in `try`, records metrics in `else`, and ensures connection closure in `finally`.
 3. Use `add_note()` to attach the current local timestamp and host IP to any caught `OSError`.
 4. Create an `ExceptionGroup` bundling three different exceptions and use `except*` to handle one type while allowing the others to propagate.
-

Further reading

- PEP 654: *Exception Groups and except**.
- PEP 678: *Enriching Exceptions with Notes*.
- Python Standard Library: *Built-in Exceptions*.

Custom Exceptions and Error Hierarchies

Custom Exceptions and Error Hierarchies

After reading this chapter, you will design domain-specific exception hierarchies inheriting from `Exception`, carry structured diagnostic payloads inside custom exception objects, use explicit exception chaining (`raise ... from err`), and suppress noisy internal tracebacks using `from None`.

Mental model

Clean architectures do not let low-level internal exceptions (such as `sqlite3.OperationalError` or `urllib3.exceptions.ProtocolError`) leak out to callers. Instead, systems translate low-level faults into domain-specific exceptions.

Organizing exceptions into an inheritance hierarchy allows callers to catch either general errors or specific failure modes:

Domain Error Hierarchy:

```
Exception (Python standard base)
```

```
StorageError (Base exception for storage subsystem)
```

```
    ConnectionTimeoutError
```

```
    AuthenticationFailedError
```

```
    RecordNotFoundError
```

```
        UserNotFoundError (Specialized subtype)
```

A caller can catch `StorageError` to handle any storage subsystem failure, or catch `RecordNotFoundError` specifically to initiate a creation workflow.

Exception Chaining (`__cause__` vs `__context__`)

Python explicitly tracks why an exception occurred using chaining:

- `raise NewError from original_error`: Sets `__cause__`, explicitly documenting that `NewError` was directly triggered by `original_error`.
- `raise NewError from None`: Suppresses the prior traceback, hiding internal implementation details from callers.

Minimal example

Save as `custom_error_hierarchy.py`:

```
# custom_error_hierarchy.py
class CloudServiceError(Exception):
    """Base exception for all cloud provider interactions."""

class RateLimitExceededError(CloudServiceError):
    """Raised when API quotas are breached."""
    def __init__(self, service: str, retry_after: int, quota: int) -> None:
        super().__init__(f"Rate limit exceeded for {service}. Retry after {retry_after}s.")
        self.service = service
        self.retry_after = retry_after
        self.quota = quota

def call_upstream_api(request_count: int) -> None:
    if request_count > 100:
        raise RateLimitExceededError(service="compute-api", retry_after=60, quota=100)
    print(f"API call successful (request count: {request_count}).")

def main() -> None:
    try:
        call_upstream_api(150)
    except RateLimitExceededError as exc:
        print(f"Caught specific rate limit: {exc}")
        print(f"  -> Service: {exc.service} | Backoff: {exc.retry_after}s | Max Quota: {exc.quota}")
    except CloudServiceError as exc:
        print(f"Caught general cloud error: {exc}")

if __name__ == "__main__":
    main()
```

Run via `uv run python custom_error_hierarchy.py`:

```
Caught specific rate limit: Rate limit exceeded for compute-api. Retry after 60s.
-> Service: compute-api | Backoff: 60s | Max Quota: 100
```

Worked examples

Case 1: Exception Chaining with `raise ... from err`

When translating low-level networking errors into high-level domain errors, `from err` links the underlying cause so that debugging logs preserve the original root-cause traceback:

```
# chained_translation.py
import socket

class DatabaseConnectionError(Exception):
    """Domain exception representing storage cluster failure."""

def ping_db_node(host: str, port: int) -> None:
    s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    s.settimeout(0.01)
    try:
        # Intentionally connect to an unreachable test address
        s.connect((host, port))
    except (socket.timeout, OSError) as original_err:
        # Wrap the low-level socket failure into our domain exception
        raise DatabaseConnectionError(f"Failed connecting to database at {host}:{port}") from original_err
    finally:
        s.close()

if __name__ == "__main__":
    try:
        ping_db_node("192.0.2.1", 5432) # TEST-NET-1 unroutable address
    except DatabaseConnectionError as err:
        print(f"Domain Error: {err}")
        print(f"Root Cause: {type(err.__cause__).__name__}: {err.__cause__}")
```

Run:

```
uv run python chained_translation.py
```

Output:

```
Domain Error: Failed connecting to database at 192.0.2.1:5432
Root Cause: TimeoutError: timed out
```

In the full traceback, Python prints:

```
The above exception was the direct cause of the following exception:
DatabaseConnectionError: Failed connecting to database at 192.0.2.1:5432
```

Case 2: Suppressing Internal Noise with from None

When building command-line utilities or clean public libraries, internal library tracebacks create confusing noise for end users. `raise ... from None` severs the link:

```
# clean_cli_error.py
import json
```

```

class ConfigurationParseError(Exception):
    """Raised when application configuration file is malformed."""

def load_system_config(raw_json: str) -> dict[str, str]:
    try:
        return json.loads(raw_json)
    except json.JSONDecodeError as decode_err:
        # Suppress JSONDecodeError traceback and present a clean error
        raise ConfigurationParseError(
            f"Config file syntax error at line {decode_err.lineno}, column {decode_err.colno}"
        ) from None

if __name__ == "__main__":
    bad_config = '{"version": 2, "servers": ["app1", "app2", ]}'
    try:
        load_system_config(bad_config)
    except ConfigurationParseError as err:
        print(f"CLI Error: {err}")
        print(f"Has chained cause? {err.__cause__ is not None}")

```

Run:

```
uv run python clean_cli_error.py
```

Output:

```
CLI Error: Config file syntax error at line 1, column 44
Has chained cause? False
```

Pitfalls

Pitfall 1: Inheriting from BaseException instead of Exception

Inheriting from BaseException causes custom errors to bypass standard `except Exception:` handlers:

```

# THE BUG:
class CustomError(BaseException): # NEVER inherit directly from BaseException!
    pass

# THE FIX:
class CustomError(Exception):      # ALWAYS inherit from Exception
    pass

```

`BaseException` is reserved exclusively for Python's core system events: `SystemExit`, `KeyboardInterrupt`, and `GeneratorExit`.

Pitfall 2: Overly Flat or Monolithic Exceptions

Creating a single `AppError` and using string matching to determine what happened (`if "timeout" in str(e): ...`) defeats Python's type-based exception handling. Always create a base exception class and subclass it for distinct failure modes.

Exercises

1. Design an exception hierarchy for an authentication service with a base `AuthError`, and subclasses `InvalidCredentialsError`, `TokenExpiredError`, and `AccountLockedError`.
 2. Write a custom exception `HTTPResponseError` that accepts `status_code: int` and `response_body: str` as constructor arguments and exposes them as read-only properties.
 3. Simulate catching a `FileNotFoundError` and wrapping it in a custom `ConfigFileMissingError` using `raise ... from err`.
 4. Inspect the `__context__` attribute of an exception raised inside an `except` block without an explicit `from`.
-

Further reading

- PEP 3134: *Exception Chaining and Embedded Tracebacks*.
- Python Tutorial: *User-defined Exceptions*.
- Python Standard Library: `exceptions` hierarchy.

Context Managers and Resource Management

Context Managers and Resource Management

After reading this chapter, you will master Resource Acquisition Is Initialization (RAII) using Python's `with` statement, implement the `__enter__` and `__exit__` dunder protocol, author lightweight context managers using `@contextlib.contextmanager`, and dynamically manage multi-resource lifecycles with `contextlib.ExitStack`.

Mental model

In systems programming and backend engineering, resources (file descriptors, database connections, mutex locks, network sockets) must be deterministic and leak-free. Relying on manual cleanup (`file.close()`) fails whenever an unexpected exception interrupts execution.

The **Context Management Protocol** binds resource allocation to the entry of a code block and guarantees release upon exit:

`with acquire_resource() as target:`

1. Calls `target = resource.__enter__()`

2. Executes with-block body

Success Calls `resource.__exit__(None, None, None)`

Exception Raised Calls `resource.__exit__(exc_type, exc_val, exc_tb)`

Returns True	Exception is SUPPRESSED
Returns False	Exception PROPAGATES

[Resumes Outer Execution]

Minimal example

Save as `transaction_manager.py`:

```
# transaction_manager.py
from typing import Any
```

```

class MockDatabaseTransaction:
    def __init__(self, tx_id: str) -> None:
        self.tx_id = tx_id
        self.active = False

    def __enter__(self) -> "MockDatabaseTransaction":
        self.active = True
        print(f"[{self.tx_id}] BEGIN TRANSACTION: Locks acquired.")
        return self

    def __exit__(self, exc_type: Any, exc_val: Any, exc_tb: Any) -> bool:
        if exc_type is not None:
            print(f"[{self.tx_id}] ROLLBACK: Transaction aborted due to {exc_type.__name__}.")
            self.active = False
            return False # Do not suppress exception; propagate to caller

        print(f"[{self.tx_id}] COMMIT: Transaction changes committed safely.")
        self.active = False
        return True

def main() -> None:
    # 1. Successful transaction
    print("--- Running successful transaction ---")
    with MockDatabaseTransaction("TX_101") as tx:
        print(f"  Writing record into transaction {tx.tx_id}...")

    # 2. Failed transaction with automatic rollback
    print("\n--- Running failed transaction ---")
    try:
        with MockDatabaseTransaction("TX_102") as tx:
            print(f"  Writing record into transaction {tx.tx_id}...")
            raise RuntimeError("Database constraint violation on unique constraint")
    except RuntimeError as err:
        print(f"  Caller caught error: {err}")

if __name__ == "__main__":
    main()

```

Run via `uv run python transaction_manager.py`:

```

--- Running successful transaction ---
[TX_101] BEGIN TRANSACTION: Locks acquired.
  Writing record into transaction TX_101...
[TX_101] COMMIT: Transaction changes committed safely.

--- Running failed transaction ---
[TX_102] BEGIN TRANSACTION: Locks acquired.

```

```
Writing record into transaction TX_102...
[TX_102] ROLLBACK: Transaction aborted due to RuntimeError: Database constraint violation on unique constraint
Caller caught error: Database constraint violation on unique constraint
```

Worked examples

Case 1: Benchmark Timer with `@contextlib.contextmanager`

Writing a full class with `__enter__` and `__exit__` is often unnecessary for simple scope management. The `@contextmanager` decorator turns a generator into a clean context manager:

```
# execution_timer.py
import time
from contextlib import contextmanager
from collections.abc import Generator

@contextmanager
def benchmark(label: str) -> Generator[None, None, None]:
    start_time = time.perf_counter()
    print(f"[{label}] Started execution...")
    try:
        # yield suspends execution and yields control to the with-block
        yield
    finally:
        # Code in finally ALWAYS runs when the with-block terminates
        elapsed = time.perf_counter() - start_time
        print(f"[{label}] Finished in {elapsed * 1000:.2f} ms")

if __name__ == "__main__":
    with benchmark("Data Aggregation"):
        total = sum(x * x for x in range(500_000))
        print(f"  Computed sum: {total}")
```

Run:

```
uv run python execution_timer.py
```

Output:

```
[Data Aggregation] Started execution...
  Computed sum: 41666541666750000
[Data Aggregation] Finished in 24.85 ms
```


Case 2: Clean Error Suppression with `contextlib.suppress`

Instead of writing a verbose `try/except: pass` block to ignore expected non-fatal errors (such as deleting a file that might not exist), `suppress()` makes the intent declarative:

```
# safe_cleanup.py
import os
from contextlib import suppress

def remove_temporary_lock(lock_file_path: str) -> None:
    # Replaces:
    # try:
    #     os.remove(lock_file_path)
    # except FileNotFoundError:
    #     pass
    with suppress(FileNotFoundError):
        os.remove(lock_file_path)
        print(f"Removed lock file: {lock_file_path}")

if __name__ == "__main__":
    # Target does not exist; suppressed cleanly without error
    remove_temporary_lock("/tmp/non_existent_cluster_lock.pid")
    print("Cleanup executed without raising exceptions.")
```

Run:

```
uv run python safe_cleanup.py
```

Output:

Cleanup executed without raising exceptions.

Case 3: Dynamic Multi-Resource Coordination with `ExitStack`

When the number of files, sockets, or locks is unknown at authoring time (e.g. merging an arbitrary list of configuration files), a static `with open(f1), open(f2):` statement is impossible. `contextlib.ExitStack` manages dynamic collections:

```
# dynamic_file_merger.py
import tempfile
from contextlib import ExitStack
from pathlib import Path

def merge_log_files(source_paths: list[Path], output_path: Path) -> int:
    total_lines = 0
    with ExitStack() as stack:
        # Dynamically register all files into the stack
```

```

input_handles = [stack.enter_context(p.open("r")) for p in source_paths]
out_handle = stack.enter_context(output_path.open("w"))

for handle in input_handles:
    for line in handle:
        out_handle.write(line)
        total_lines += 1

# ALL handles guaranteed closed here, even if an exception was raised
return total_lines

if __name__ == "__main__":
    with tempfile.TemporaryDirectory() as tmpdir:
        td = Path(tmpdir)
        f1 = td / "app_01.log"
        f2 = td / "app_02.log"
        f1.write_text("2026-09-07 node01 startup\n")
        f2.write_text("2026-09-07 node02 startup\n")

        merged = td / "merged.log"
        count = merge_log_files([f1, f2], merged)
        print(f"Merged {count} lines into {merged.name}:")
        print(merged.read_text().strip())

```

Run:

```
uv run python dynamic_file_merger.py
```

Output:

```

Merged 2 lines into merged.log:
2026-09-07 node01 startup
2026-09-07 node02 startup

```

Pitfalls

Pitfall 1: Accidentally Suppressing All Exceptions in `__exit__`

If `__exit__` returns any truthy value (like `True` or a non-empty object), CPython suppresses **every** exception raised inside the `with` block:

```

# THE BUG:
def __exit__(self, exc_type, exc_val, exc_tb):

```

```

    self.cleanup()
    return True # DANGEROUS! Swallows NameError, TypeError, and syntax bugs!

# THE FIX: Only return True if you specifically intended to handle that exact error
def __exit__(self, exc_type, exc_val, exc_tb):
    self.cleanup()
    if exc_type is ResourceUnavailableError:
        return True # Suppress only this known error
    return False # Propagate everything else

```

Pitfall 2: Omitting try/finally in @contextmanager Generators

In a generator context manager, if an exception is raised inside the caller's `with` block, it is thrown into the generator at the point of the `yield`. Without `try...finally`, code after `yield` will never execute:

```

# THE BUG:
@contextmanager
def acquire_lock():
    lock.acquire()
    yield
    lock.release() # NEVER RUNS if caller raises an exception inside 'with'!

# THE FIX:
@contextmanager
def acquire_lock():
    lock.acquire()
    try:
        yield
    finally:
        lock.release() # ALWAYS RUNS

```

Exercises

1. Implement a class-based context manager `TemporaryDirectoryChanger` that switches the current working directory to a target directory in `__enter__` and restores the original directory in `__exit__`.
2. Write a generator context manager `override_env(key: str, value: str)` that temporarily modifies `os.environ` and restores the prior value upon block exit.
3. Use `ExitStack` to acquire a list of simulated thread locks and verify they are all released if any single acquisition fails.

4. Implement a context manager that catches `ZeroDivisionError` and logs a warning, while propagating all other exceptions.
-

Further reading

- PEP 343: *The “with” Statement*.
- Python Standard Library: `contextlib` documentation.
- Python Data Model: *With Statement Context Managers*.

Exception Groups and the `except*` Operator

Exception Groups and the `except*` Operator

After reading this chapter, you will master Python's concurrent error-handling architecture introduced in PEP 654, construct and inspect hierarchical `ExceptionGroup` trees, handle multiple simultaneous failures using the `except*` clause, isolate specific error sub-types, and prevent mixed-handler syntax traps.

Mental model

In traditional sequential programming, execution halts at the first raised exception. However, in concurrent environments (such as thread pools, worker queues, and `asyncio.TaskGroup`), multiple tasks execute in parallel and can fail simultaneously.

Prior to Python 3.11, runtimes were forced to discard all but the first failure or bury errors in custom aggregate lists. `ExceptionGroup` and the `except*` operator provide first-class language support for handling multiple independent exceptions at once:

Raised `ExceptionGroup`:

```
ExceptionGroup("Batch deployment failure", [
    ConnectionError("Host 10.0.1.1 unreachable"),
    TimeoutError("Host 10.0.1.2 timeout (30s)"),
    ConnectionError("Host 10.0.1.3 connection reset")
])
```

```
[ try ... except* ]
```

```
except* ConnectionError as eg:
```

```
    Matched Subgroup: [ ConnectionError(10.0.1.1), ConnectionError(10.0.1.3) ]
    Executes this block!
```

```
except* TimeoutError as eg:
```

```
    Matched Subgroup: [ TimeoutError(10.0.1.2) ]
    ALSO executes this block!
```

Traditional except vs Modern except*

Feature	Traditional except	Modern except*
Handled exceptions	Exactly one per block	Multiple except
Clause execution	First matching clause wins; stops	Multiple match
Unmatched exceptions	Replaces raised error	Automatically
Target type	Any BaseException	Subsets of Exc

Minimal example

Save as `exception_groups_demo.py`:

```
# exception_groups_demo.py
def deploy_microservices() -> None:
    """Simulate parallel deployment tasks where multiple failures occur."""
    failures = [
        ConnectionError("Failed to connect to redis-01.internal:6379"),
        TimeoutError("Database migration timed out after 60s"),
        ConnectionError("Failed to connect to redis-02.internal:6379"),
    ]
    raise ExceptionGroup("Microservice deployment failed", failures)

def main() -> None:
    try:
        deploy_microservices()
    except* ConnectionError as eg:
        # Matches and extracts ONLY the ConnectionError instances
        print(f"[RECOVER] Handling {len(eg.exceptions)} connection failures:")
        for exc in eg.exceptions:
            print(f"  - {exc}")
    except* TimeoutError as eg:
        # Matches and extracts ONLY the TimeoutError instances in the SAME group!
        print(f"[ALERT] Escalating {len(eg.exceptions)} timeout failures:")
        for exc in eg.exceptions:
            print(f"  - {exc}")

if __name__ == "__main__":
    main()
```

Run via `uv run python exception_groups_demo.py`:

```
[RECOVER] Handling 2 connection failures:
- Failed to connect to redis-01.internal:6379
- Failed to connect to redis-02.internal:6379
[ALERT] Escalating 1 timeout failures:
- Database migration timed out after 60s
```

Notice: Both `except*` clauses executed during the single `try` evaluation!

Worked examples

Case 1: Concurrent Node Health Probing with Diagnostic Subgroups

When monitoring a distributed cluster, diagnostic probes execute across dozens of servers simultaneously. Handling failures requires distinguishing transient network errors from permanent authentication rejections:

```
# cluster_probe_monitor.py
class AuthRejectedError(Exception):
    """Raised when node TLS certificate or API token is invalid."""

def simulate_cluster_audit() -> None:
    errors = [
        TimeoutError("Node 10.0.2.14: Health probe response exceeded 5000ms"),
        AuthRejectedError("Node 10.0.2.18: Client certificate expired"),
        TimeoutError("Node 10.0.2.22: Health probe response exceeded 5000ms"),
    ]
    raise ExceptionGroup("Cluster audit uncovered node health anomalies", errors)

def main() -> None:
    try:
        simulate_cluster_audit()
    except* TimeoutError as eg:
        print(f"[AUTO-RETRY] Enqueueing {len(eg.exceptions)} nodes for exponential backoff:")
        for err in eg.exceptions:
            print(f"  Transient: {err}")
    except* AuthRejectedError as eg:
        print(f"[SECURITY ALERT] Page on-call immediately for {len(eg.exceptions)} security")
        for err in eg.exceptions:
            print(f"  Fatal: {err}")

if __name__ == "__main__":
    main()
```

Run:


```
uv run python cluster_probe_monitor.py
```

Output:

```
[AUTO-RETRY] Enqueueing 2 nodes for exponential backoff:
  Transient: Node 10.0.2.14: Health probe response exceeded 5000ms
  Transient: Node 10.0.2.22: Health probe response exceeded 5000ms
[SECURITY ALERT] Page on-call immediately for 1 security faults:
  Fatal: Node 10.0.2.18: Client certificate expired
```

Case 2: Programmatic Group Splitting and Filtering (.split() and .subgroup())

In addition to `except*` syntax, `ExceptionGroup` instances provide programmatic methods to filter and inspect exception trees directly:

```
# group_introspection.py
def main() -> None:
    group = ExceptionGroup("Data pipeline failure", [
        ValueError("Column 'age' cannot be negative"),
        TypeError("Expected float for column 'latency', got str"),
        ValueError("Column 'score' exceeds maximum 100"),
        RuntimeError("Database write lock acquisition failed"),
    ])

    # 1. .split(predicate) divides the group into (matching_subgroup, remaining_group)
    value_errors, rest = group.split(ValueError)

    print("--- Extracted ValueError Subgroup ---")
    if value_errors:
        for exc in value_errors.exceptions:
            print(f"  Validation issue: {exc}")

    print("\n--- Remaining Unmatched Group ---")
    if rest:
        for exc in rest.exceptions:
            print(f"  System issue:      {exc}")

    # 2. .subgroup(predicate) filters matching items recursively
    type_subgroup = group.subgroup(TypeError)
    print(f"\nType error subgroup item count: {len(type_subgroup.exceptions) if type_subgroup}

if __name__ == "__main__":
    main()
```

Run:

```
uv run python group_introspection.py
```

Output:

```
--- Extracted ValueError Subgroup ---
Validation issue: Column 'age' cannot be negative
Validation issue: Column 'score' exceeds maximum 100

--- Remaining Unmatched Group ---
System issue:      Expected float for column 'latency', got str
System issue:      Database write lock acquisition failed

Type error subgroup item count: 1
```

Case 3: Enriching Individual Errors in Groups with `add_note()`

Python 3.11+ allows attaching contextual notes to any exception via `exc.add_note("text")`. When combined with `ExceptionGroup`, notes provide diagnostics for individual concurrent branches without mutating original exception messages:

```
# diagnostic_notes.py
def perform_shard_sync() -> None:
    e1 = ConnectionResetError("Remote server terminated socket")
    e1.add_note("Shard ID: shard-001 (us-east-1)")
    e1.add_note("Retry attempt: 3 of 3 exhausted")

    e2 = PermissionError("Access denied to replication bucket")
    e2.add_note("Shard ID: shard-004 (eu-central-1)")
    e2.add_note("IAM Role: arn:aws:iam::123456789012:role/StorageReader")

    raise ExceptionGroup("Shard replication pipeline failed", [e1, e2])

def main() -> None:
    try:
        perform_shard_sync()
    except* Exception as eg:
        for exc in eg.exceptions:
            print(f"\nCaught: {type(exc).__name__}: {exc}")
            if hasattr(exc, "__notes__"):
                print("Contextual Notes:")
                for note in exc.__notes__:
                    print(f"    * {note}")
```

```
if __name__ == "__main__":  
    main()
```

Run:

```
uv run python diagnostic_notes.py
```

Output:

Caught: ConnectionResetError: Remote server terminated socket

Contextual Notes:

- * Shard ID: shard-001 (us-east-1)
- * Retry attempt: 3 of 3 exhausted

Caught: PermissionError: Access denied to replication bucket

Contextual Notes:

- * Shard ID: shard-004 (eu-central-1)
 - * IAM Role: arn:aws:iam::123456789012:role/StorageReader
-

Pitfalls

Pitfall 1: Mixing except and except* in the Same try Block

A try statement cannot mix traditional `except` clauses with `except*` clauses. Doing so results in an immediate `SyntaxError`:

```
# THE TRAP: SyntaxError  
try:  
    raise ExceptionGroup("errors", [ValueError("bad")])  
except ValueError:      # BUG: SyntaxError: cannot have both 'except' and 'except*' on the sa  
    pass  
except* TypeError:  
    pass  
  
# THE FIX: Use except* exclusively when handling ExceptionGroup hierarchies  
try:  
    raise ExceptionGroup("errors", [ValueError("bad")])  
except* ValueError:  
    print("Handled ValueError cleanly")  
except* TypeError:  
    print("Handled TypeError cleanly")
```

Pitfall 2: Partial Matching Leaves Unhandled Errors to Propagate

When `except*` matches a subgroup, any unmatched exceptions in the `ExceptionGroup` are **automatically re-raised** at the end of the `try ... except*` statement:

```
# THE TRAP: Unhandled exceptions escape!
def run_batch():
    raise ExceptionGroup("batch", [ValueError("invalid input"), KeyError("missing key")])

try:
    try:
        run_batch()
    except* ValueError:
        print("Handled ValueError!")
        # KeyError is NOT handled here! It is automatically re-raised as an ExceptionGroup!
except ExceptionGroup as outer_eg:
    print(f"Caught unhandled residual group: {outer_eg.exceptions}")
```

Output:

Handled ValueError!

Caught unhandled residual group: (KeyError('missing key'),)

Pitfall 3: Catching ExceptionGroup Directly with Traditional except

Catching `except ExceptionGroup as eg:` works, but treats the group as a single monolithic block without destructuring. You must manually inspect `.exceptions` instead of letting the VM dispatch by type:

```
# Clunky manual dispatch:
try:
    raise ExceptionGroup("test", [ValueError("v"), TypeError("t")])
except ExceptionGroup as eg:
    for e in eg.exceptions:
        if isinstance(e, ValueError): ...

# Modern idiomatic dispatch:
try:
    raise ExceptionGroup("test", [ValueError("v"), TypeError("t")])
except* ValueError as eg:
    print(f"Handled value errors: {eg.exceptions}")
except* TypeError as eg:
    print(f"Handled type errors: {eg.exceptions}")
```

Exercises

1. Create an `ExceptionGroup` containing two `ValueError` exceptions and one `KeyError`. Write a `try ... except*` block that catches only the `ValueError` exceptions, observing that the `KeyError` propagates.
 2. Given an `ExceptionGroup` with nested subgroups, use `.split()` to extract all network-related exceptions (`ConnectionError`, `TimeoutError`) while leaving other errors intact.
 3. Write a simulation of 5 concurrent worker tasks using a list of callables. If any fail, collect all exceptions into an `ExceptionGroup` and raise it.
 4. Attach diagnostic notes (`.add_note()`) containing timestamps and hostnames to exceptions before grouping them into an `ExceptionGroup`.
 5. Demonstrate why `except* Exception` will catch all standard exceptions in a group, but will still let `KeyboardInterrupt` and `SystemExit` (subclasses of `BaseExceptionGroup`) pass through.
-

Further reading

- PEP 654: *Exception Groups and except**.
- Python Documentation: *Built-in Exceptions – ExceptionGroup and BaseExceptionGroup*.
- Python Documentation: *The try statement – Handling Exception Groups with except**.

Objects and Classes

Classes, Instances, and Encapsulation

Classes, Instances, and Encapsulation

After reading this chapter, you will master class definition and instance instantiation, distinguish between class attributes and instance attributes, implement safe encapsulation using `@property`, and understand the mechanics of Python's name mangling.

Mental model

In Python, classes are first-class objects created at runtime when the `class` block executes. An instance is a distinct heap-allocated object (`PyInstanceObject`) whose `__class__` pointer references its type.

Class vs Instance Namespace:

```
Class: ServiceConfig
    default_timeout = 30          (Stored in ServiceConfig.__dict__)
    default_retries = 3
    __init__, start, stop methods

    __class__ pointer
Instance: auth_service
    name = "auth"                (Stored in auth_service.__dict__)
    port = 8080
```

When you access `auth_service.default_timeout`, Python searches: 1. `auth_service.__dict__` (instance namespace) 2. If missing, `ServiceConfig.__dict__` (class namespace) 3. If missing, parent classes up the MRO 4. If missing, raises `AttributeError`

Minimal example

Save as `class_encapsulation.py`:

```
# class_encapsulation.py
class ServerNode:
    # Class attribute: shared across all instances
    cluster_name: str = "us-east-cluster"
```



```

def __init__(self, hostname: str, port: int) -> None:
    # Instance attributes: unique to each instance
    self.hostname = hostname
    self._port = port # Protected convention (one underscore)
    self.__secret_token = f"tok_{hostname}_{port}" # Private mangled (two underscores)

@property
def port(self) -> int:
    """Getter property: read-only access to port."""
    return self._port

@port.setter
def port(self, value: int) -> None:
    """Setter property: enforces validation boundaries."""
    if not (1 <= value <= 65535):
        raise ValueError(f"Invalid port {value}: Must be between 1 and 65535")
    self._port = value

def describe(self) -> str:
    return f"{self.hostname}:{self._port} (Cluster: {self.cluster_name})"

def main() -> None:
    node1 = ServerNode("worker-01", 8080)
    node2 = ServerNode("worker-02", 9090)

    print(node1.describe())
    print(node2.describe())

    # Mutating property via validated setter
    node1.port = 8443
    print(f"Updated node1 port: {node1.port}")

    try:
        node1.port = 99999
    except ValueError as err:
        print(f"Validation rejected invalid port: {err}")

if __name__ == "__main__":
    main()

```

Run via `uv run python class_encapsulation.py`:

```

worker-01:8080 (Cluster: us-east-cluster)
worker-02:9090 (Cluster: us-east-cluster)
Updated node1 port: 8443
Validation rejected invalid port: Invalid port 99999: Must be between 1 and 65535

```

Worked examples

Case 1: Computed Properties & Caching

The `@property` decorator allows creating dynamically computed attributes that look like normal fields to callers:

```
# computed_metrics.py
import time

class StorageVolume:
    def __init__(self, name: str, total_gb: float, used_gb: float) -> None:
        self.name = name
        self.total_gb = total_gb
        self.used_gb = used_gb

    @property
    def free_gb(self) -> float:
        """Dynamically computed attribute."""
        return self.total_gb - self.used_gb

    @property
    def usage_percent(self) -> float:
        """Percentage calculation with guard against division by zero."""
        if self.total_gb <= 0:
            return 0.0
        return (self.used_gb / self.total_gb) * 100.0

    @property
    def status(self) -> str:
        pct = self.usage_percent
        if pct > 90.0:
            return "CRITICAL"
        elif pct > 75.0:
            return "WARNING"
        return "HEALTHY"

if __name__ == "__main__":
    vol = StorageVolume("data-vol-01", total_gb=500.0, used_gb=410.0)
    print(f"Volume      : {vol.name}")
    print(f"Free        : {vol.free_gb:.1f} GB")
    print(f"Usage        : {vol.usage_percent:.1f}%")
    print(f"Status       : {vol.status}")
```

Run:

```
uv run python computed_metrics.py
```

Output:

```
Volume    : data-vol-01
Free      : 90.0 GB
Usage     : 82.0%
Status    : WARNING
```

Case 2: Demystifying Name Mangling (`__var`)

When an attribute starts with two leading underscores (and at most one trailing underscore), CPython transforms the name internally to `_<ClassName>__<var>` to prevent accidental overriding in subclasses:

```
# name_mangling_demo.py
class SecurityKey:
    def __init__(self, key_id: str, raw_secret: str) -> None:
        self.key_id = key_id
        self.__secret = raw_secret # Name mangled

    def verify(self, candidate: str) -> bool:
        return self.__secret == candidate

if __name__ == "__main__":
    sec = SecurityKey("auth-key-01", "super_secret_token_123")
    print(f"Public ID: {sec.key_id}")
    print("Verification result:", sec.verify("super_secret_token_123"))

    # Direct access fails with AttributeError
    try:
        print(sec.__secret)
    except AttributeError as err:
        print(f"Direct access rejected: {err}")

    # Inspecting the instance dictionary reveals the mangled name
    print(f"Internal __dict__ keys: {list(sec.__dict__.keys())}")
    print(f"Accessing mangled key directly: {sec._SecurityKey__secret}")
```

Run:

```
uv run python name_mangling_demo.py
```

Output:

```
Public ID: auth-key-01
Verification result: True
Direct access rejected: 'SecurityKey' object has no attribute '__secret'
Internal __dict__ keys: ['key_id', '_SecurityKey__secret']
```

Accessing mangled key directly: `super_secret_token_123`

Name mangling is not true cryptographic security (Python does not enforce private memory protection); it is an anti-collision mechanism to prevent subclasses from accidentally stomping on internal fields.

Pitfalls

Pitfall 1: Mutating Class Attributes via an Instance Reference

Assigning to `instance.attr` creates a new instance attribute in that instance's dictionary, rather than updating the shared class attribute:

```
class Cluster:
    nodes = [] # Dangerous mutable class attribute!

c1 = Cluster()
c2 = Cluster()

# Mutating in-place affects all instances because 'nodes' references the same list
c1.nodes.append("node-1")
print(c2.nodes) # ['node-1']

# Re-binding creates a new instance attribute on c1 only:
c1.nodes = ["node-new"]
print(c1.nodes) # ['node-new'] (stored in c1.__dict__)
print(c2.nodes) # ['node-1'] (still looks at Cluster.nodes)
```

Always initialize mutable attributes (lists, dicts, sets) inside `__init__` on `self`.

Exercises

1. Create a class `NetworkInterface` with attributes `name` and `ip_address`. Add a property `is_loopback` that returns `True` if the IP address starts with `"127."`.
 2. Implement a `TemperatureSensor` class that stores temperature in Celsius, but provides getter and setter properties for Fahrenheit with proper mathematical conversion ($F = C \times \frac{9}{5} + 32$).
 3. Demonstrate the difference in `__dict__` between an instance with single-underscore attributes (`self._token`) versus double-underscore attributes (`self.__token`).
 4. Write a class that counts how many instances of itself have been created using a class attribute.
-

Further reading

- Python Documentation: *Classes and Objects*.
- Python Data Model: *Customizing attribute access*.
- PEP 8: *Naming Conventions for class and instance attributes*.

The Python Data Model and Dunder Protocols

The Python Data Model and Dunder Protocols

After reading this chapter, you will master Python’s special method protocols (known as “dunder” methods for *double underscore*), implement unambiguous string representations (`__repr__` vs `__str__`), enable custom objects for dictionary hashing and equality, implement sequence and container behaviors, and make objects callable like functions.

Mental model

The Python Data Model is the API of the Python language itself. Built-in functions, operators, and syntax do not perform hardcoded checks; they delegate to special methods implemented on your classes:

Syntax / Built-in	Dunder Method Invoked
<code>repr(obj)</code>	<code>obj.__repr__()</code>
<code>str(obj)</code> , <code>print(obj)</code>	<code>obj.__str__()</code> (falls back to <code>__repr__</code>)
<code>len(obj)</code>	<code>obj.__len__()</code>
<code>item in obj</code>	<code>obj.__contains__(item)</code>
<code>obj[key]</code>	<code>obj.__getitem__(key)</code>
<code>obj == other</code>	<code>obj.__eq__(other)</code>
<code>hash(obj)</code>	<code>obj.__hash__()</code>
<code>obj(arg1, arg2)</code>	<code>obj.__call__(arg1, arg2)</code>

By implementing these protocols, custom classes behave with the same ergonomics and speed as Python’s native data types.

Minimal example

Save as `data_model_protocols.py`:

```
# data_model_protocols.py
from typing import Any

class PacketBuffer:
    def __init__(self, packets: list[bytes]) -> None:
```

```

        self._packets = list(packets)

    def __len__(self) -> int:
        """Enables len(buffer)."""
        return len(self._packets)

    def __getitem__(self, index: int | slice) -> Any:
        """Enables buffer[i] indexing and buffer[start:stop] slicing."""
        return self._packets[index]

    def __contains__(self, item: bytes) -> bool:
        """Enables 'payload in buffer' membership testing."""
        return item in self._packets

    def __repr__(self) -> str:
        """Unambiguous representation for debugging and logs."""
        return f"PacketBuffer(count={len(self._packets)})"

def main() -> None:
    p1 = b"\x00\x01\x02\x03"
    p2 = b"\xaa\xbb\xcc\xdd"
    p3 = b"\xff\xff\xff\xff"

    buf = PacketBuffer([p1, p2, p3])

    print("Buffer representation:", repr(buf))
    print(f"Total packet count      : {len(buf)}")
    print(f"Packet at index 1          : {buf[1].hex()}")
    print(f"Contains p2?                : {p2 in buf}")
    print(f"Contains missing?           : {b'dummy' in buf}")

    # Iteration works automatically via __getitem__!
    print("Iterating over packet hex strings:")
    for pkt in buf:
        print(f"  - {pkt.hex()}")

if __name__ == "__main__":
    main()

```

Run via `uv run python data_model_protocols.py`:

```

Buffer representation: PacketBuffer(count=3)
Total packet count      : 3
Packet at index 1       : aabbccdd
Contains p2?           : True
Contains missing?       : False
Iterating over packet hex strings:

```


- 00010203
- aabbccdd
- ffffffff

Notice that we did not even implement `__iter__()`: Python's iteration protocol automatically falls back to `__getitem__()` starting at index 0 until `IndexError` is raised!

Worked examples

Case 1: The Golden Rule of `__repr__` vs `__str__`

- `__repr__`: Intended for developers, debuggers, and logs. It should be unambiguous and, whenever practical, look like valid Python code that recreates the object (`eval(repr(x)) == x`).
- `__str__`: Intended for end-user display (`print(x)`).

```
# repr_vs_str.py
class Endpoint:
    def __init__(self, host: str, port: int, use_tls: bool = True) -> None:
        self.host = host
        self.port = port
        self.use_tls = use_tls

    def __repr__(self) -> str:
        # Developer-facing, unambiguous
        return f"Endpoint(host={self.host!r}, port={self.port}, use_tls={self.use_tls})"

    def __str__(self) -> str:
        # User-facing URI string
        scheme = "https" if self.use_tls else "http"
        return f"{scheme}://{self.host}:{self.port}"

if __name__ == "__main__":
    ep = Endpoint("api.internal", 8443, use_tls=True)
    print("User facing (str)      :", str(ep))
    print("Developer facing (repr):", repr(ep))
```

Run:

```
uv run python repr_vs_str.py
```

Output:

```
User facing (str)      : https://api.internal:8443
Developer facing (repr): Endpoint(host='api.internal', port=8443, use_tls=True)
```

Case 2: Equality (`__eq__`) and Hashing (`__hash__`)

To allow custom objects to be used as dictionary keys or stored in sets, you must define both `__eq__` and `__hash__`. An object's hash must be computed from immutable attributes:

```
# hashable_host.py
class HostID:
    def __init__(self, cluster: str, node_id: int) -> None:
        self._cluster = cluster
        self._node_id = node_id

    @property
    def cluster(self) -> str:
        return self._cluster

    @property
    def node_id(self) -> int:
        return self._node_id

    def __eq__(self, other: object) -> bool:
        if not isinstance(other, HostID):
            return NotImplemented
        return self._cluster == other._cluster and self._node_id == other._node_id

    def __hash__(self) -> int:
        # Hash a tuple of the immutable identity attributes
        return hash((self._cluster, self._node_id))

    def __repr__(self) -> str:
        return f"HostID({self._cluster!r}, {self._node_id})"

if __name__ == "__main__":
    h1 = HostID("prod-us", 101)
    h2 = HostID("prod-us", 101)
    h3 = HostID("prod-eu", 101)

    print(f"h1 == h2 : {h1 == h2}")
    print(f"h1 == h3 : {h1 == h3}")

    # Safe for use in sets and dictionary keys
    cluster_nodes = {h1, h2, h3}
    print(f"Unique nodes in set (count: {len(cluster_nodes)}): {cluster_nodes}")
```

Run:

```
uv run python hashable_host.py
```

Output:

```

h1 == h2 : True
h1 == h3 : False
Unique nodes in set (count: 2): {HostID('prod-us', 101), HostID('prod-eu', 101)}

```

Case 3: Automatic Ordering with `functools.total_ordering`

Implementing all comparison operators (<, <=, >, >=, ==, !=) is tedious. By implementing `__eq__` and just one ordering method (`__lt__`), `@total_ordering` derives the rest:

```

# priority_ordering.py
from functools import total_ordering

@total_ordering
class TaskPriority:
    def __init__(self, level: int, name: str) -> None:
        self.level = level
        self.name = name

    def __eq__(self, other: object) -> bool:
        if not isinstance(other, TaskPriority):
            return NotImplemented
        return self.level == other.level

    def __lt__(self, other: object) -> bool:
        if not isinstance(other, TaskPriority):
            return NotImplemented
        return self.level < other.level

    def __repr__(self) -> str:
        return f"Priority({self.level}, {self.name!r})"

if __name__ == "__main__":
    low = TaskPriority(1, "Background Backup")
    med = TaskPriority(5, "Log Rotation")
    high = TaskPriority(10, "Service Outage Failover")

    print(f"low < high : {low < high}")
    print(f"high >= med : {high >= med}")

    tasks = [med, high, low]
    tasks.sort()
    print("Sorted tasks by priority:", tasks)

```

Run:

```
uv run python priority_ordering.py
```

```
low < high : True
high >= med : True
Sorted tasks by priority: [Priority(1, 'Background Backup'), Priority(5, 'Log Rotation'), Prior
```

Defining `__call__` turns an instance into a callable that retains internal state across calls:

Run:

Output:

```
Request 1: ALLOWED
Request 2: ALLOWED
Request 3: ALLOWED
```

Request 4: DENIED

Request 5: DENIED

Pitfalls

Pitfall 1: Defining `__eq__` Without `__hash__`

In Python 3, if a class defines `__eq__` but does not define `__hash__`, Python automatically sets `__hash__ = None`:

```
class Node:
    def __init__(self, name):
        self.name = name
    def __eq__(self, other):
        return self.name == other.name

n = Node("worker")
# s = {n} -> TypeError: unhashable type: 'Node'
```

If you override `__eq__` and want the object to remain hashable, you **must** explicitly implement `__hash__`. If the object is mutable, leaving `__hash__ = None` is the correct safety measure to prevent hash table corruption.

Exercises

1. Create a class `Vector2D(x, y)` that implements `__add__` and `__sub__` for vector arithmetic, and `__abs__` to calculate Euclidean magnitude ($\sqrt{x^2 + y^2}$).
 2. Implement a `RollingLog` container that stores the last N entries and implements `__len__`, `__getitem__`, and `__iter__`.
 3. Create an immutable `CIDRBlock` class that implements `__eq__` and `__hash__` so that equivalent subnets can be deduplicated in a `set`.
 4. Implement a stateful token-bucket rate limiter class that uses `__call__` to check and consume tokens.
-

Further reading

- Python Documentation: *The Python Data Model — Special method names*.
- Python Standard Library: `functools.total_ordering`.
- Luciano Ramalho: *Fluent Python, Second Edition* (O'Reilly).

Inheritance, MRO, and Composition

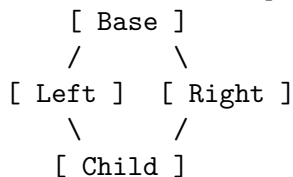
Inheritance, MRO, and Composition

After reading this chapter, you will master single and multiple inheritance in Python, understand the Method Resolution Order (MRO) derived by C3 linearization, implement cooperative multiple inheritance using `super()`, enforce architectural interfaces using Abstract Base Classes (`abc.ABC`), and favor composition over fragile inheritance hierarchies.

Mental model

When a subclass inherits from multiple parent classes, how does Python decide which implementation of a method to execute? It computes a deterministic, monotonic linear order known as the **Method Resolution Order (MRO)** using the **C3 Linearization Algorithm**.

Diamond Inheritance Graph:



```
C3 Linearization (Child.__mro__):
Child  Left  Right  Base  object
```

Cooperative `super()` Dispatch

In Python, `super()` does **not** simply call the immediate parent in the source code. Instead, it calls the **next class in the instance's MRO**:

```
Call from Child: super().process()
1. Inspects type(self).__mro__
2. Finds caller's current class in MRO (Child)
3. Dispatches to the NEXT class in sequence (Left)
4. Left's super().process() dispatches to Right!
```

This cooperative chaining ensures every class in the graph is visited exactly once.

Minimal example

Save as `inheritance_mro.py`:

```
# inheritance_mro.py
class BaseService:
    def __init__(self, name: str) -> None:
        self.name = name
        print(f" [BaseService] Initialized service: {name}")

class LoggingMixin:
    def __init__(self, *args: object, **kwargs: object) -> None:
        super().__init__(*args, **kwargs)
        print(" [LoggingMixin] Telemetry logger attached.")

class MetricsMixin:
    def __init__(self, *args: object, **kwargs: object) -> None:
        super().__init__(*args, **kwargs)
        print(" [MetricsMixin] Metrics reporter registered.")

# Multiple inheritance with Mixins
class ProductionWebService(LoggingMixin, MetricsMixin, BaseService):
    def __init__(self, name: str) -> None:
        print(f"Initializing ProductionWebService: {name}")
        # Cooperative super() walks through LoggingMixin -> MetricsMixin -> BaseService
        super().__init__(name)

def main() -> None:
    svc = ProductionWebService("auth-gateway")

    print("\nMethod Resolution Order (MRO):")
    for idx, cls in enumerate(ProductionWebService.__mro__):
        print(f" {idx}: {cls.__name__}")

if __name__ == "__main__":
    main()
```

Run via `uv run python inheritance_mro.py`:

```
Initializing ProductionWebService: auth-gateway
[BaseService] Initialized service: auth-gateway
[MetricsMixin] Metrics reporter registered.
[LoggingMixin] Telemetry logger attached.
```

Method Resolution Order (MRO):

```
0: ProductionWebService
1: LoggingMixin
```

```
2: MetricsMixin
3: BaseService
4: object
```

Worked examples

Case 1: Formal Interfaces with Abstract Base Classes (abc.ABC)

In plugin systems, drivers, and microservices, you must guarantee that subclasses adhere to a specific interface. Abstract Base Classes prevent instantiation if required methods are missing:

```
# storage_plugin_abc.py
from abc import ABC, abstractmethod

class StorageDriver(ABC):
    """Abstract base contract for blob and file storage backends."""

    @abstractmethod
    def read_bytes(self, path: str) -> bytes:
        """Fetch raw bytes from storage."""
        ...

    @abstractmethod
    def write_bytes(self, path: str, data: bytes) -> int:
        """Write raw bytes and return bytes written."""
        ...

    def ping(self) -> bool:
        """Concrete utility method available to all implementations."""
        return True

class MemoryStorageDriver(StorageDriver):
    def __init__(self) -> None:
        self._store: dict[str, bytes] = {}

    def read_bytes(self, path: str) -> bytes:
        if path not in self._store:
            raise FileNotFoundError(f"Path not found: {path}")
        return self._store[path]

    def write_bytes(self, path: str, data: bytes) -> int:
        self._store[path] = data
        return len(data)
```

```

if __name__ == "__main__":
    driver = MemoryStorageDriver()
    print("Driver ping:", driver.ping())

    n = driver.write_bytes("/configs/app.json", b'{"debug": true}')
    print(f"Wrote {n} bytes.")
    print("Read back:", driver.read_bytes("/configs/app.json"))

    # Attempting to instantiate an incomplete subclass fails:
    class IncompleteDriver(StorageDriver):
        pass

    try:
        IncompleteDriver()
    except TypeError as err:
        print("\nInstantiation rejected for incomplete subclass:")
        print(f" {err}")

```

Run:

```
uv run python storage_plugin_abc.py
```

Output:

```

Driver ping: True
Wrote 15 bytes.
Read back: b'{"debug": true}'

```

Instantiation rejected for incomplete subclass:

Can't instantiate abstract class IncompleteDriver without an implementation for abstract method

Case 2: Composition Over Inheritance

Deep inheritance hierarchies (Device -> NetworkDevice -> Switch -> ManagedSwitch -> CiscoManagedSwitch) become rigid, fragile, and hard to test. **Composition** assembles behavior from independent components:

```

# composition_architecture.py
class SSHTransport:
    def execute(self, host: str, command: str) -> str:
        return f"[SSH to {host}] Executed: {command} -> Output: SUCCESS"

class RESTTransport:
    def execute(self, host: str, command: str) -> str:
        return f"[REST API to {host}] POST /rpc -> Output: 200 OK"

```

```

class NetworkSwitchManager:
    """Uses composition: receives a transport rather than inheriting from one."""
    def __init__(self, host: str, transport: SSHTransport | RESTTransport) -> None:
        self.host = host
        self.transport = transport

    def reload_interfaces(self) -> str:
        return self.transport.execute(self.host, "reload interfaces")

if __name__ == "__main__":
    # Swap transports at runtime without changing the Manager class
    ssh_manager = NetworkSwitchManager("switch-01.rack-a", SSHTransport())
    api_manager = NetworkSwitchManager("switch-02.rack-b", RESTTransport())

    print(ssh_manager.reload_interfaces())
    print(api_manager.reload_interfaces())

```

Run:

```
uv run python composition_architecture.py
```

Output:

```
[SSH to switch-01.rack-a] Executed: reload interfaces -> Output: SUCCESS
[REST API to switch-02.rack-b] POST /rpc -> Output: 200 OK
```

Pitfalls

Pitfall 1: Calling Direct Parent Names Instead of `super()`

Directly invoking `ParentClass.__init__(self)` bypasses cooperative multiple inheritance and causes sibling mixins in the MRO to be silently skipped:

```

# THE BUG:
class BrokenService(LoggingMixin, BaseService):
    def __init__(self, name):
        BaseService.__init__(self, name) # BUG: Skips LoggingMixin.__init__ completely!

# THE FIX:
class CorrectService(LoggingMixin, BaseService):
    def __init__(self, name):
        super().__init__(name) # Traverses all classes in __mro__ cooperatively

```

Pitfall 2: Overusing Multiple Inheritance

Multiple inheritance is best reserved for lightweight, orthogonal **mixins** that do not maintain complex conflicting state. For domain business logic, prefer composition.

Exercises

1. Define an Abstract Base Class `MessageQueue(ABC)` with abstract methods `publish(topic: str, payload: bytes)` and `subscribe(topic: str)`. Implement a concrete `InMemoryQueue` subclass.
 2. Given classes `A`, `B(A)`, `C(A)`, and `D(B, C)`, print `D.__mro__` and trace the order in which C3 linearization resolves methods.
 3. Refactor a deep three-level inheritance tree into a single class that uses composition with two injected helper objects.
 4. Implement a `JSONSerializableMixin` that provides a `to_json()` method by serializing `self.__dict__`.
-

Further reading

- Guido van Rossum: *The Python 2.3 Method Resolution Order*.
- PEP 3119: *Introducing Abstract Base Classes*.
- Python Standard Library: `abc` module.

Dataclasses and Declarative Models

Dataclasses and Declarative Models

After reading this chapter, you will master Python's `@dataclass` decorator (PEP 557), enforce immutability with `frozen=True`, optimize memory with `slots=True`, configure default factories for safe collections, execute validation via `__post_init__`, and understand the boundary between standard library dataclasses and Pydantic v2.

Mental model

Writing repetitive boilerplate (`__init__`, `__repr__`, `__eq__`, `__hash__`) for data-holding classes is error-prone. The `@dataclass` decorator inspects type annotations at class definition time and automatically generates these methods.

Class Source:

```
@dataclass(slots=True, frozen=True)
class ServiceNode:
    host: str
    port: int = 8080
```

Generated by CPython:

```
- __init__(self, host: str, port: int = 8080)
- __repr__(self) -> "ServiceNode(host=..., port=...)"
- __eq__(self, other) -> compares tuple of fields
- __hash__(self) -> computes hash from fields (enabled by frozen=True)
- __slots__ = ('host', 'port') -> eliminates per-instance __dict__
```

Memory Architecture: `__dict__` vs `__slots__`

A standard Python object maintains a dynamic dictionary (`__dict__`) to allow arbitrary attribute additions at runtime. In microservices holding hundreds of thousands of objects (metrics, tokens, edges), `__dict__` introduces massive memory overhead:

Standard Instance with `__dict__`:

```
PyObject Header    __dict__ (PyDictObject: table, keys, hash entries) (~152 bytes)
```

Dataclass with `slots=True`:

```
PyObject Header    [ field_0_ptr, field_1_ptr ] (Fixed C-struct offsets) (~56 bytes)
```

Minimal example

Save as `dataclasses_deep_dive.py`:

```
# dataclasses_deep_dive.py
from dataclasses import dataclass, field

@dataclass(slots=True, frozen=True)
class EndpointConfig:
    host: str
    port: int = 443
    tags: list[str] = field(default_factory=list)

    def __post_init__(self) -> None:
        # Validation hook executed immediately after generated __init__
        if not (1 <= self.port <= 65535):
            raise ValueError(f"Port {self.port} out of range (1-65535)")

def main() -> None:
    ep1 = EndpointConfig("auth.prod.internal", 8443, tags=["prod", "security"])
    ep2 = EndpointConfig("auth.prod.internal", 8443, tags=["prod", "security"])

    print("Representation:", repr(ep1))
    print(f"ep1 == ep2      : {ep1 == ep2}")

    # Hashable because frozen=True and tags can be hashed if converted or compared
    print(f"Port accessor   : {ep1.port}")

    # Mutation is strictly forbidden
    try:
        ep1.port = 80 # type: ignore[misc]
    except Exception as err:
        print(f"Mutation rejected: {type(err).__name__}")

if __name__ == "__main__":
    main()
```

Run via `uv run python dataclasses_deep_dive.py`:

```
Representation: EndpointConfig(host='auth.prod.internal', port=8443, tags=['prod', 'security'])
ep1 == ep2      : True
Port accessor   : 8443
Mutation rejected: FrozenInstanceError
```

Worked examples

Case 1: Mutable Defaults via `field(default_factory=...)`

Python prevents using a mutable object directly as a dataclass default. You must provide a callable factory:

```
# safe_defaults.py
from dataclasses import dataclass, field
from datetime import datetime

@dataclass
class AuditRecord:
    actor: str
    action: str
    # DANGEROUS: metadata: dict = {} is rejected by dataclasses!
    # SAFE: Use default_factory
    metadata: dict[str, str] = field(default_factory=dict)
    timestamp: str = field(default_factory=lambda: datetime.now().isoformat())

if __name__ == "__main__":
    rec1 = AuditRecord("admin", "restart_service")
    rec1.metadata["target"] = "worker-01"

    rec2 = AuditRecord("operator", "drain_node")

    print(f"Record 1 metadata: {rec1.metadata}")
    print(f"Record 2 metadata: {rec2.metadata} (Remains isolated and empty)")
```

Run:

```
uv run python safe_defaults.py
```

Output:

```
Record 1 metadata: {'target': 'worker-01'}
Record 2 metadata: {} (Remains isolated and empty)
```

Case 2: Memory Benchmark: `slots=True` vs Standard Class

When handling high-volume data streams (e.g. log events or sensor telemetry), `slots=True` drastically reduces memory consumption:

```
# slots_memory_benchmark.py
import sys
from dataclasses import dataclass
```

```

class StandardEvent:
    def __init__(self, device_id: str, reading: float) -> None:
        self.device_id = device_id
        self.reading = reading

@dataclass(slots=True)
class SlottedEvent:
    device_id: str
    reading: float

def main() -> None:
    std = StandardEvent("sensor-99", 23.4)
    slotted = SlottedEvent("sensor-99", 23.4)

    # Note: sys.getsizeof on an instance does not include its __dict__ size
    std_total_size = sys.getsizeof(std) + sys.getsizeof(std.__dict__)
    slotted_total_size = sys.getsizeof(slotted)

    print(f"Standard instance total memory: {std_total_size} bytes (has __dict__)")
    print(f"Slotted instance total memory : {slotted_total_size} bytes (no __dict__)")
    print(f"Memory reduction          : {((std_total_size - slotted_total_size) / std_to

if __name__ == "__main__":
    main()

```

Run:

```
uv run python slots_memory_benchmark.py
```

Output:

```

Standard instance total memory: 152 bytes (has __dict__)
Slotted instance total memory : 56 bytes (no __dict__)
Memory reduction          : 63.2%

```

Case 3: Dataclasses vs Pydantic v2: Architectural Boundaries

A common architectural question in modern Python: **When to use @dataclass vs Pydantic?**

Feature	Standard Library @dataclass	Pydantic v2 (BaseModel)
Dependency	Standard library (zero dependencies)	External package (pip install pydantic)
Type Checking	Annotations used for code generation only; no runtime enforcement	Strict runtime validation & automatic type coercion

Feature	Standard Library <code>@dataclass</code>	Pydantic v2 (<code>BaseModel</code>)
Parsing	None (requires manual parsing)	Built-in JSON/dict deserialization and schema export
Execution Speed	Zero overhead; pure Python / C-struct	Rust-core (<code>pydantic-core</code>) high speed parsing
Best Used For	Internal application state, domain entities, math models	Public API boundaries, untrusted JSON input, config loading

Pitfalls

Pitfall 1: Assuming `@dataclass` Enforces Types at Runtime

Standard dataclasses do **not** validate types when assigned:

```
@dataclass
class PortConfig:
    port: int

# Valid syntax; no error is raised despite 'invalid' being a string!
p = PortConfig("invalid") # type: ignore[arg-type]
print(p.port) # Prints: "invalid"
```

If you need runtime type enforcement at system boundaries, use `__post_init__` validation or Pydantic.

Exercises

1. Create a `@dataclass(slots=True, frozen=True)` named `MetricRecord` with fields `timestamp`, `metric_name`, and `value`. Verify that attempting to change `value` raises `FrozenInstanceError`.
2. Write a dataclass `DatabaseConnectionSpec` where `database_url` is automatically constructed in `__post_init__` from `host`, `port`, and `database_name`.
3. Demonstrate why `@dataclass class Bad: items: list = []` causes a `ValueError` at class definition time.
4. Implement a dataclass with `order=True` that automatically generates comparison operators based on a specific `priority` field.

Further reading

- PEP 557: *Data Classes*.
- Python Standard Library: `dataclasses` documentation.
- Python Language Reference: `__slots__` optimization.

Decorators and Metaprogramming

Decorators and Metaprogramming

After reading this chapter, you will master Python decorators from first principles, preserve function introspection with `@functools.wraps`, construct parameterized decorator factories, implement class decorators, and use `@classmethod` and `@staticmethod` appropriately.

Mental model

A decorator in Python is syntactic sugar for passing a function (or class) into a higher-order function, and rebinding the original name to the returned callable.

Syntactic Sugar:

```
@log_execution
def query_db(query_str):
    ...
```

Is Bit-for-Bit Equivalent To:

```
def query_db(query_str):
    ...
query_db = log_execution(query_db)
```

The Wrapper Wrapping Pipeline

Caller executes `query_db("SELECT 1")`

```
[ wrapper(*args, **kwargs) ]
```

```
    Pre-execution logic (start timer, log entry)
    Result = original_func(*args, **kwargs)
    Post-execution logic (stop timer, record metrics)
```

Return Result to Caller

Minimal example

Save as `decorators_fundamentals.py`:

```
# decorators_fundamentals.py
import functools
import time
from collections.abc import Callable
from typing import Any

def measure_latency(func: Callable[..., Any]) -> Callable[..., Any]:
    """Decorator that measures and prints the execution latency of any function."""
    @functools.wraps(func)
    def wrapper(*args: Any, **kwargs: Any) -> Any:
        start_time = time.perf_counter()
        try:
            return func(*args, **kwargs)
        finally:
            elapsed_ms = (time.perf_counter() - start_time) * 1000
            print(f"[{func.__name__}] Executed in {elapsed_ms:.2f} ms")
    return wrapper

@measure_latency
def compute_hash_digest(payload: str) -> int:
    """Compute an integer checksum from payload."""
    total = 0
    for char in payload:
        total = (total * 31 + ord(char)) & 0xFFFFFFFF
    return total

def main() -> None:
    digest = compute_hash_digest("production-event-payload-string")
    print(f"Computed digest: {digest}")
    print(f"Function name   : {compute_hash_digest.__name__}")
    print(f"Docstring        : {compute_hash_digest.__doc__}")

if __name__ == "__main__":
    main()
```

Run via `uv run python decorators_fundamentals.py`:

```
[compute_hash_digest] Executed in 0.01 ms
Computed digest: 1475752251
Function name   : compute_hash_digest
Docstring      : Compute an integer checksum from payload.
```

Worked examples

Case 1: Parameterized Decorator Factory (@retry)

When a decorator requires configuration arguments (`@retry(max_attempts=3, backoff=0.05)`), you must create a three-tier factory function:

```
# retry_decorator.py
import functools
import time
from collections.abc import Callable
from typing import Any

def retry(max_attempts: int = 3, backoff: float = 0.05) -> Callable[..., Any]:
    """Decorator factory that retries an operation on failure with exponential backoff."""
    def decorator(func: Callable[..., Any]) -> Callable[..., Any]:
        @functools.wraps(func)
        def wrapper(*args: Any, **kwargs: Any) -> Any:
            attempts = 0
            while True:
                attempts += 1
                try:
                    return func(*args, **kwargs)
                except Exception as exc:
                    if attempts >= max_attempts:
                        print(f"[{func.__name__}] Failed after {attempts} attempts. Raising")
                        raise
                    sleep_duration = backoff * (2 ** (attempts - 1))
                    print(f"[{func.__name__}] Attempt {attempts} failed: {exc}. Retrying in")
                    time.sleep(sleep_duration)
            return wrapper
        return decorator

# Flaky mock service
call_counter = 0

@retry(max_attempts=3, backoff=0.02)
def query_payment_gateway() -> str:
    global call_counter
    call_counter += 1
    if call_counter < 3:
        raise ConnectionResetError("Transient network blip")
    return "PAYMENT_CONFIRMED"

if __name__ == "__main__":
    result = query_payment_gateway()
    print(f"Final outcome: {result}")
```


Run:

```
uv run python retry_decorator.py
```

Output:

```
[query_payment_gateway] Attempt 1 failed: Transient network blip. Retrying in 0.02s...
[query_payment_gateway] Attempt 2 failed: Transient network blip. Retrying in 0.04s...
Final outcome: PAYMENT_CONFIRMED
```

Case 2: Method Decorators: @classmethod vs @staticmethod

- **@classmethod**: Receives the class object (`cls`) as its first argument. Ideal for alternative factory constructors.
- **@staticmethod**: Receives neither `self` nor `cls`. Acts as a plain function grouped inside the class namespace.

```
# class_and_static_methods.py
class IPAddress:
    def __init__(self, octets: tuple[int, int, int, int]) -> None:
        self.octets = octets

    @classmethod
    def from_string(cls, ip_str: str) -> "IPAddress":
        """Alternative factory constructor parsing a dotted-quad string."""
        parts = tuple(int(p) for p in ip_str.split("."))
        if len(parts) != 4 or any(not (0 <= o <= 255) for o in parts):
            raise ValueError(f"Invalid IPv4 string: {ip_str}")
        return cls(parts) # type: ignore[arg-type]

    @staticmethod
    def is_private_range(first_octet: int) -> bool:
        """Utility check requiring neither instance nor class state."""
        return first_octet in (10, 172, 192)

    def __repr__(self) -> str:
        return f"IPAddress({'.'.join(str(o) for o in self.octets)})"

if __name__ == "__main__":
    ip = IPAddress.from_string("192.168.1.1")
    print("Constructed IP:", repr(ip))
    print(f"Is private?    : {IPAddress.is_private_range(ip.octets[0])}")
```

Run:

```
uv run python class_and_static_methods.py
```

Output:

```
Constructed IP: IPAddress(192.168.1.1)
Is private?    : True
```

Case 3: Class Decorators for Automated Registration

Decorators can wrap classes as well as functions, allowing automatic registration into a centralized registry:

```
# plugin_registry.py
from typing import Any

PLUGIN_REGISTRY: dict[str, type] = {}

def register_plugin(plugin_name: str) -> Any:
    def class_decorator(cls: type) -> type:
        PLUGIN_REGISTRY[plugin_name] = cls
        return cls
    return class_decorator

@register_plugin("json_exporter")
class JSONExporter:
    def export(self, data: Any) -> str:
        return f"Exported {data} as JSON"

@register_plugin("csv_exporter")
class CSVExporter:
    def export(self, data: Any) -> str:
        return f"Exported {data} as CSV"

if __name__ == "__main__":
    print("Registered plugins:", list(PLUGIN_REGISTRY.keys()))
    exporter_cls = PLUGIN_REGISTRY["json_exporter"]
    exporter_instance = exporter_cls()
    print(exporter_instance.export({"status": "ok"}))
```

Run:

```
uv run python plugin_registry.py
```

Output:

```
Registered plugins: ['json_exporter', 'csv_exporter']
Exported {'status': 'ok'} as JSON
```

Pitfalls

Pitfall 1: Omitting `@functools.wraps`

Without `@functools.wraps(func)`, the wrapper function overwrites the decorated function's `__name__`, `__doc__`, and `__annotations__`. This breaks debugging tools, loggers, and test runners that rely on function reflection.

Pitfall 2: Decorator Definition-Time vs Execution-Time Confusion

Code in the outer decorator body runs **immediately at import time** when Python compiles the module. Only code inside `wrapper()` runs when the decorated function is actually called.

Exercises

1. Write a decorator `@log_arguments` that prints the positional and keyword arguments passed into the decorated function upon each invocation.
2. Implement an `@enforce_types` decorator that checks if the runtime types of arguments match their type annotations.
3. Write a `@singleton` class decorator that ensures only one instance of the decorated class can ever be created.
4. Create a `@rate_limited(max_per_second=5)` decorator using `time.monotonic()` to throttle fast callers.

Further reading

- PEP 318: *Decorators for Functions and Methods*.
- Python Standard Library: `functools.wraps`.
- Python Language Reference: *Function definitions*.

Descriptors and the Attribute Access Protocol

Descriptors and the Attribute Access Protocol

After reading this chapter, you will master Python's descriptor protocol (`__get__`, `__set__`, `__delete__`, and `__set_name__`), analyze the exact priority order of attribute lookup in CPython, differentiate data descriptors from non-data descriptors, implement reusable type-validation descriptors, and uncover how `@property` and `@classmethod` work under the hood.

Mental model

In Python, accessing an attribute (`obj.attribute`) does not merely read a memory offset. Instead, it triggers an evaluation pipeline. A **descriptor** is an object that customizes what happens when an attribute is accessed, modified, or deleted on another class:

CPython Attribute Lookup Resolution Pipeline: `obj.attribute`

1. Look up 'attribute' in `obj.__class__.__mro__`

Found a Data Descriptor?
(Defines `__set__` or `__delete__`)
YES

Not a Data Descriptor

Call `descriptor.__get__()`

2. Look up in `obj.__dict__`

Key Found?
YES

Key Absent

Return `obj.__dict__['x']`

3. Found Non-Data Descriptor?
(Defines ONLY `__get__`)

Call `__get__()`

4. Class `__dict__`

5. Call `__getattr__()`

The Descriptor Protocol Methods

```
class Descriptor:
    def __set_name__(self, owner: type, name: str) -> None:
        """Called at class creation time; informs descriptor of variable name."""

    def __get__(self, instance: object | None, owner: type) -> object:
        """Called when attribute is retrieved (obj.attr)."""

    def __set__(self, instance: object, value: object) -> None:
        """Called when attribute is assigned (obj.attr = val). (Makes it a Data Descriptor)."""

    def __delete__(self, instance: object) -> None:
        """Called when attribute is deleted (del obj.attr). (Makes it a Data Descriptor)."""
```

Minimal example

Save as `descriptor_mechanics.py`:

```
# descriptor_mechanics.py
from typing import Any

class PositiveInteger:
    """Data descriptor that validates assigned integers are strictly positive (> 0)."""

    def __set_name__(self, owner: type, name: str) -> None:
        # Automatically capture the attribute name (e.g. 'port' or 'workers')
        self.private_name = f"_{name}"

    def __get__(self, instance: Any, owner: type) -> Any:
        if instance is None:
            # Accessed on the class itself (e.g. ServerConfig.port)
            return self
        return getattr(instance, self.private_name, None)

    def __set__(self, instance: Any, value: Any) -> None:
        if not isinstance(value, int) or value <= 0:
            raise ValueError(f"Attribute '{self.private_name[1:]}' must be a positive integer")
        setattr(instance, self.private_name, value)

class ServerConfig:
    port = PositiveInteger()
    workers = PositiveInteger()
```

```

def __init__(self, port: int, workers: int) -> None:
    self.port = port          # Triggers PositiveInteger.__set__
    self.workers = workers    # Triggers PositiveInteger.__set__

def main() -> None:
    # 1. Valid assignment
    cfg = ServerConfig(port=8080, workers=4)
    print(f"Valid ServerConfig: port={cfg.port}, workers={cfg.workers}")

    # 2. Validation rejection
    try:
        cfg.port = -1
    except ValueError as err:
        print(f"Caught expected validation failure: {err}")

if __name__ == "__main__":
    main()

```

Run via `uv run python descriptor_mechanics.py`:

Valid ServerConfig: port=8080, workers=4

Caught expected validation failure: Attribute 'port' must be a positive integer, got: -1

Worked examples

Case 1: How @property Works Under the Hood

The built-in `@property` decorator is actually a standard data descriptor implemented in C. Writing a pure-Python property replica reveals how getter, setter, and deleter methods bind together:

```

# property_replica.py
from collections.abc import Callable
from typing import Any

class CustomProperty:
    """Pure-Python implementation of the built-in property descriptor."""

    def __init__(
        self,
        fget: Callable[[Any], Any] | None = None,
        fset: Callable[[Any, Any], None] | None = None,
    ) -> None:
        self.fget = fget

```

```

        self.fset = fset

def __get__(self, instance: Any, owner: type) -> Any:
    if instance is None:
        return self
    if self.fget is None:
        raise AttributeError("Unreadable attribute")
    return self.fget(instance)

def __set__(self, instance: Any, value: Any) -> None:
    if self.fset is None:
        raise AttributeError("Can't set attribute (read-only)")
    self.fset(instance, value)

def setter(self, fset: Callable[[Any, Any], None]) -> "CustomProperty":
    """Return a new descriptor instance with the setter attached."""
    return CustomProperty(fget=self.fget, fset=fset)

class TemperatureSensor:
    def __init__(self, celsius: float) -> None:
        self._celsius = celsius

    @CustomProperty
    def celsius(self) -> float:
        return self._celsius

    @celsius.setter
    def celsius(self, value: float) -> None:
        if value < -273.15:
            raise ValueError("Temperature below absolute zero is physically impossible!")
        self._celsius = value

def main() -> None:
    sensor = TemperatureSensor(25.0)
    print(f"Initial temperature: {sensor.celsius} C")

    sensor.celsius = 32.5
    print(f"Updated temperature: {sensor.celsius} C")

    try:
        sensor.celsius = -300.0
    except ValueError as err:
        print(f"Caught absolute zero error: {err}")

if __name__ == "__main__":
    main()

```


Run:

```
uv run python property_replica.py
```

Output:

Initial temperature: 25.0 C

Updated temperature: 32.5 C

Caught absolute zero error: Temperature below absolute zero is physically impossible!

Case 2: Lazy Cached Properties with Non-Data Descriptors

A **non-data descriptor** defines only `__get__`. Because it lacks `__set__`, Python's lookup rule dictates that entries in `instance.__dict__` take precedence over it! This mechanism enables efficient lazy-evaluation caches:

```
# lazy_cached_property.py
import time
from collections.abc import Callable
from typing import Any

class LazyCachedProperty:
    """Non-data descriptor: computes once, stores directly in instance.__dict__."""

    def __init__(self, func: Callable[[Any], Any]) -> None:
        self.func = func
        self.attr_name = func.__name__

    def __get__(self, instance: Any, owner: type) -> Any:
        if instance is None:
            return self
        print(f" [Computing] Running expensive calculation for '{self.attr_name}'...")
        val = self.func(instance)
        # Store directly in the instance dict!
        # Subsequent lookups will find it in instance.__dict__ and bypass this descriptor
        instance.__dict__[self.attr_name] = val
        return val

class CloudNodeMetrics:
    def __init__(self, node_id: str) -> None:
        self.node_id = node_id

    @LazyCachedProperty
    def historical_aggregate(self) -> dict[str, float]:
        time.sleep(0.01) # Simulate expensive database aggregation
```

```

        return {"avg_cpu": 72.4, "p99_latency_ms": 142.1}

def main() -> None:
    node = CloudNodeMetrics("srv-us-east-42")

    print("First access (triggers computation):")
    print(f"Result: {node.historical_aggregate}")

    print("\nSecond access (retrieved instantly from instance.__dict__):")
    print(f"Result: {node.historical_aggregate}")

    print(f"\nVerifying stored attributes in node.__dict__:")
    print(f"Keys: {list(node.__dict__.keys())}")

if __name__ == "__main__":
    main()

```

Run:

```
uv run python lazy_cached_property.py
```

Output:

```

First access (triggers computation):
[Computing] Running expensive calculation for 'historical_aggregate'...
Result: {'avg_cpu': 72.4, 'p99_latency_ms': 142.1}

Second access (retrieved instantly from instance.__dict__):
Result: {'avg_cpu': 72.4, 'p99_latency_ms': 142.1}

Verifying stored attributes in node.__dict__:
Keys: ['node_id', 'historical_aggregate']

```

Pitfalls

Pitfall 1: Storing Value on the Descriptor Instance

Descriptors are instantiated **once at the class level**, shared across all instances of that class. Storing user state on `self.value` causes every class instance to overwrite the exact same variable:

```

# THE TRAP: Shared state across all class instances!
class BrokenDescriptor:
    def __get__(self, instance, owner):
        return self.val

```

```

def __set__(self, instance, val):
    self.val = val # BUG: Stored on the descriptor, shared across all objects!

class Worker:
    worker_id = BrokenDescriptor()

w1 = Worker()
w2 = Worker()
w1.worker_id = "worker-01"
w2.worker_id = "worker-02"

print(f"w1 ID: {w1.worker_id}") # Prints 'worker-02'! w1 was overwritten!

# THE FIX: Store the state inside the instance, keyed by private_name
class SafeDescriptor:
    def __set_name__(self, owner, name):
        self.storage_key = f"_{name}"
    def __get__(self, instance, owner):
        if instance is None: return self
        return getattr(instance, self.storage_key, None)
    def __set__(self, instance, val):
        setattr(instance, self.storage_key, val)

```

Pitfall 2: Forgetting if instance is None: in __get__

When an attribute is accessed via the class itself (`MyClass.attr`), the `instance` argument passed to `__get__` is `None`:

```

class MyDescriptor:
    def __get__(self, instance, owner):
        # BUG: if instance is None, instance.data raises AttributeError!
        # if instance is None: return self
        return instance.data

# THE FIX:
class RobustDescriptor:
    def __get__(self, instance, owner):
        if instance is None:
            return self # Return descriptor object when inspected via class
        return getattr(instance, "_data", None)

```

Exercises

1. Implement a `TypedString` descriptor that validates that assigned values are instances of `str` and have a length between `min_len` and `max_len`.
 2. Replicate `functools.cached_property` using a non-data descriptor that caches expensive method results directly onto `instance.__dict__`.
 3. Explain why defining `__set__` (even as a no-op that raises `AttributeError`) prevents assignments to `instance.__dict__` from shadowing the descriptor.
 4. Implement a `ReadOnly` descriptor that allows an attribute to be assigned exactly once (during `__init__`), but raises `RuntimeError` on subsequent modification attempts.
 5. Create a descriptor `BoundedFloat(min_val, max_val)` and use it to validate percentage metrics on a `SystemMonitor` class.
-

Further reading

- Raymond Hettinger: *Descriptor HowTo Guide* (Official Python Documentation).
- Python Language Reference: *Implementing Descriptors*.
- PEP 487: *Simpler customisation of class creation* (`__set_name__` protocol).

Metaclasses and Class Construction Mechanics

Metaclasses and Class Construction Mechanics

After reading this chapter, you will master the internal lifecycle of Python class construction, trace how the CPython VM builds classes using `type`, leverage `__init_subclass__` (PEP 487) for zero-magic plugin registration, capture declaration order using `__prepare__`, enforce architectural invariants via custom metaclasses, and resolve metaclass inheritance conflicts.

Mental model

In Python, **classes are objects**. And just as ordinary objects are instances of classes, classes themselves are instances of **metaclasses**:

Ordinary Object:

```
instance = Server("web-01")    instance is an instance of class Server
                                type(instance) is Server
```

Class Object:

```
Server                          Server is an instance of metaclass 'type'
                                type(Server) is type
```

The CPython Class Construction Pipeline

When Python encounters a `class` statement, it executes a 5-stage creation pipeline:

1. Determine Metaclass
(Inspect explicit `metaclass=...`, base classes, or default to `'type'`)
2. Call `metaclass.__prepare__(name, bases, **kwargs)`
Returns namespace mapping (e.g. `OrderedDict` or custom dict)
3. Execute Class Body in that Namespace
(Methods and attributes are defined)
4. Call `metaclass(name, bases, namespace)`
Invokes `metaclass.__new__()` to allocate the `PyTypeObject`
Invokes `metaclass.__init__()` to initialize the class

5. Trigger `__init_subclass__()`
Notifies all base classes that a subclass was created

Minimal example

Save as `class_construction_pipeline.py`:

```
# class_construction_pipeline.py
from typing import Any

class VerboseMeta(type):
    @classmethod
    def __prepare__(metaccls, name: str, bases: tuple[type, ...], **kwargs: Any) -> dict[str, Any]:
        print(f" [1. __prepare__] Allocating namespace for class '{name}'")
        return dict()

    def __new__(
        metaccls,
        name: str,
        bases: tuple[type, ...],
        namespace: dict[str, Any],
        **kwargs: Any,
    ) -> type:
        print(f" [2. __new__] Allocating PyTypeObject for '{name}' with {len(namespace)} namespace items")
        cls_obj = super().__new__(metaccls, name, bases, namespace)
        return cls_obj

    def __init__(cls, name: str, bases: tuple[type, ...], namespace: dict[str, Any], **kwargs: Any):
        print(f" [3. __init__] Initializing class object '{name}'")
        super().__init__(name, bases, namespace)

class ServiceEndpoint(metaclass=VerboseMeta):
    timeout_seconds = 30

    def handle_request(self) -> str:
        return "200 OK"

def main() -> None:
    print("Class definition execution complete.")
    print(f"Type of ServiceEndpoint: {type(ServiceEndpoint).__name__}")
    endpoint = ServiceEndpoint()
    print(f"Response: {endpoint.handle_request()}")
```

```
if __name__ == "__main__":
    main()
```

Run via `uv run python class_construction_pipeline.py`:

```
[1. __prepare__] Allocating namespace for class 'ServiceEndpoint'
[2. __new__]      Allocating PyTypeObject for 'ServiceEndpoint' with 7 attributes
[3. __init__]     Initializing class object 'ServiceEndpoint'
Class definition execution complete.
Type of ServiceEndpoint: VerboseMeta
Response: 200 OK
```

Notice: All three metaclass steps executed at **import/definition time**, before a single instance of `ServiceEndpoint` was ever created!

Worked examples

Case 1: The Modern Alternative: Zero-Boilerplate Plugins with `__init_subclass__`

Prior to Python 3.6, building auto-registering plugin architectures required custom metaclasses. PEP 487 introduced `__init_subclass__`, which provides 95% of metaclass capabilities with standard inheritance:

```
# plugin_registry.py
from typing import ClassVar

class PaymentProcessor:
    """Base class that automatically registers all subclasses into a global dispatch map."""
    registry: ClassVar[dict[str, type["PaymentProcessor"]]] = {}

    def __init_subclass__(cls, provider_code: str, **kwargs: object) -> None:
        super().__init_subclass__(**kwargs)
        provider_key = provider_code.strip().lower()
        if provider_key in cls.registry:
            raise ValueError(f"Duplicate payment provider code '{provider_key}' registered b
        cls.registry[provider_key] = cls
        print(f"  [Registered Plugin] Provider: '{provider_key}' -> {cls.__name__}")

    def process_charge(self, amount_cents: int) -> str:
        raise NotImplementedError

# Subclasses register automatically via keyword arguments in their class definition!
class StripeProcessor(PaymentProcessor, provider_code="stripe"):
```



```

def process_charge(self, amount_cents: int) -> str:
    return f"Charged ${amount_cents / 100:.2f} via Stripe API"

class PayPalProcessor(PaymentProcessor, provider_code="paypal"):
    def process_charge(self, amount_cents: int) -> str:
        return f"Charged ${amount_cents / 100:.2f} via PayPal Express"

def main() -> None:
    print("\n--- Available Processors ---")
    for code, cls_type in PaymentProcessor.registry.items():
        print(f"  Code: {code:8} -> Handler: {cls_type.__name__}")

    # Dynamic factory dispatch
    selected = "stripe"
    handler = PaymentProcessor.registry[selected]()
    print(f"\nExecution: {handler.process_charge(4999)}")

if __name__ == "__main__":
    main()

```

Run:

```
uv run python plugin_registry.py
```

Output:

```
[Registered Plugin] Provider: 'stripe' -> StripeProcessor
[Registered Plugin] Provider: 'paypal' -> PayPalProcessor
```

```
--- Available Processors ---
Code: stripe   -> Handler: StripeProcessor
Code: paypal   -> Handler: PayPalProcessor
```

```
Execution: Charged $49.99 via Stripe API
```

Case 2: Enforcing Architectural Invariants with a Custom Metaclass

In large engineering codebases, you may want to enforce architectural policies—for example, requiring that every subclass of `CommandController` implements an asynchronous or synchronous `execute()` method and contains a docstring:

```

# policy_enforcer.py
class ArchitectureEnforcer(type):
    """Metaclass that validates class structure at definition time."""

```

```

def __new__(
    metacls,
    name: str,
    bases: tuple[type, ...],
    namespace: dict[str, object],
) -> type:
    # Skip validation on the root base class itself
    if bases:
        # 1. Enforce presence of class docstring
        if not namespace.get("__doc__"):
            raise TypeError(f"Architecture violation: Class '{name}' must provide a docs")

        # 2. Enforce implementation of 'execute'
        if "execute" not in namespace or not callable(namespace["execute"]):
            raise TypeError(f"Architecture violation: Class '{name}' must implement an 'execute' method")

    return super().__new__(metacls, name, bases, namespace)

class BaseCommand(metaclass=ArchitectureEnforcer):
    pass

class BackupDatabaseCommand(BaseCommand):
    """Safely snapshot and archive the primary database cluster."""

    def execute(self) -> str:
        return "Backup completed successfully."

def main() -> None:
    cmd = BackupDatabaseCommand()
    print(f"Command '{BackupDatabaseCommand.__name__}' passed architectural validation.")
    print(f"Docstring: {cmd.__doc__}")
    print(f"Execution: {cmd.execute()}")

if __name__ == "__main__":
    main()

```

Run:

```
uv run python policy_enforcer.py
```

Output:

```

Command 'BackupDatabaseCommand' passed architectural validation.
Docstring: Safely snapshot and archive the primary database cluster.
Execution: Backup completed successfully.

```

Case 3: Capturing Attribute Declaration Order with `__prepare__`

In serialization libraries and ORM models, field ordering matters for binary layouts or CSV column headers. By customizing `__prepare__`, a metaclass can intercept and track attributes exactly as they are declared in source code:

```
# schema_order_tracker.py
from typing import Any

class OrderedSchemaMeta(type):
    @classmethod
    def __prepare__(metaccls, name: str, bases: tuple[type, ...]) -> dict[str, Any]:
        # Return a dictionary that records the order of field assignments
        return dict()

    def __new__(
        metaccls,
        name: str,
        bases: tuple[type, ...],
        namespace: dict[str, Any],
    ) -> type:
        # Extract declared fields (ignoring private/dunder attributes)
        fields = [k for k, v in namespace.items() if not k.startswith("_")]
        namespace["_declared_fields"] = tuple(fields)
        return super().__new__(metaccls, name, bases, namespace)

class CSVRecordModel(metaclass=OrderedSchemaMeta):
    user_id = 0
    username = ""
    email = ""
    created_at = 0

def main() -> None:
    print(f"Model: {CSVRecordModel.__name__}")
    print("Detected fields in exact declaration order:")
    for idx, field in enumerate(CSVRecordModel._declared_fields, start=1):
        print(f"  Column #{idx}: {field}")

if __name__ == "__main__":
    main()
```

Run:

```
uv run python schema_order_tracker.py
```

Output:

```
Model: CSVRecordModel
```

Detected fields in exact declaration order:

Column #1: user_id
Column #2: username
Column #3: email
Column #4: created_at

Pitfalls

Pitfall 1: The Metaclass Conflict Error

When multiple inheritance involves base classes created with different, incompatible metaclasses, Python raises a `TypeError`:

```
class MetaA(type): pass
class MetaB(type): pass

class BaseA(metaclass=MetaA): pass
class BaseB(metaclass=MetaB): pass

# THE TRAP:
# class Derived(BaseA, BaseB): pass
# TypeError: metaclass conflict: the metaclass of a derived class must be a
# (non-strict) subclass of the metaclasses of all its bases

# THE FIX: Create a combined metaclass that inherits from both
class CombinedMeta(MetaA, MetaB): pass

class Derived(BaseA, BaseB, metaclass=CombinedMeta):
    pass

print(f"Derived successfully created with metaclass: {type(Derived).__name__}")
```

Pitfall 2: Overusing Metaclasses when Simpler Tools Suffice

Martelli's Rule of Metaclasses: “Metaclasses are deeper magic than 99% of users should ever worry about.” Before reaching for a metaclass, ask if simpler tools solve the problem:

If you need to...

Prefer...

Hook subclass creation & register plugins	<code>__init_subclass__</code> (PEP 487)
Validate, mutate, or wrap an existing class	Class Decorators (<code>@register_class</code>)
Customize individual attribute access or validation	Descriptors (<code>__set_name__</code>)
Intercept namespace creation before class body runs	Metaclass (<code>__prepare__</code>)

Exercises

1. Using `__init_subclass__`, implement an automatic event-handler registry where subclasses specify `event_name="user_login"` and are added to a centralized dispatcher.
 2. Create a metaclass that automatically converts all method names defined in CamelCase to snake_case in the class namespace.
 3. Write an architectural enforcement metaclass that rejects any class whose name does not end with "Service" or "Controller".
 4. Demonstrate how `type(name, bases, dict)` can be used to construct a complete, functional class dynamically at runtime without using the `class` keyword.
 5. Create a metaclass that freezes class attributes so that attempting to add new attributes after class creation raises `AttributeError`.
-

Further reading

- PEP 487: *Simpler customisation of class creation* (`__init_subclass__`).
- PEP 3115: *Metaclasses in Python 3000* (`__prepare__`).
- Luciano Ramalho: *Fluent Python* (Chapter 21: Class Metaprogramming).
- Brett Slatkin: *Effective Python* (Item 48: Validate Subclasses with `__init_subclass__`).

Abstract Base Classes and Structural Protocols

Abstract Base Classes and Structural Protocols

After reading this chapter, you will master Python's two complementary interface design paradigms: nominal enforcement using Abstract Base Classes (`abc.ABC`), and structural static duck typing using Protocols (`typing.Protocol` from PEP 544). You will enforce runtime contracts, register virtual subclasses, implement `@runtime_checkable` interfaces, and avoid decorator ordering traps.

Mental model

Python provides two approaches for defining and enforcing contracts between components:

Nominal Subtyping (`abc.ABC`):

"Explicit Inheritance"

Class hierarchy defines type.

Checked at instantiation time via `@abstractmethod`.

StorageBackend (abc.ABC)

inherits

S3Storage (Concrete)

Structural Subtyping (`typing.Protocol` - PEP 544):

"Static Duck Typing"

Shape and attributes define type.

No inheritance required! Zero coupling between libraries.

LogWriter	(Protocol)	ConsoleLogger	(No base class!)
- write(msg)	<.....	- write(msg)	
Matches			

Comparison Matrix

Property	Abstract Base Class (<code>abc.ABC</code>)	Protocol (<code>typing.Protocol</code>)
----------	--	---

Philosophy	Nominal: Must inherit or reg.	Structural: "Shape" matches
Enforcement Point	Runtime (at instantiation)	Static (Mypy/Pyright) + opt.
Third-party adaptability	Requires ABC.register(cls)	Automatic if signature fits
Primary Use Case	Framework base abstractions	Decoupled library interfaces

Minimal example

Save as `interfaces_overview.py`:

```
# interfaces_overview.py
from abc import ABC, abstractmethod
from typing import Protocol, runtime_checkable

# 1. Nominal Interface with abc.ABC
class MetricPublisher(ABC):
    """Nominal interface: Subclasses MUST implement publish_metric."""

    @abstractmethod
    def publish_metric(self, name: str, value: float) -> None:
        """Send a single metric measurement to the telemetry collector."""

class CloudWatchPublisher(MetricPublisher):
    def publish_metric(self, name: str, value: float) -> None:
        print(f"[CloudWatch] Published {name} = {value}")

# 2. Structural Interface with typing.Protocol (PEP 544)
@runtime_checkable
class Closable(Protocol):
    """Structural interface: Any object with a close() method satisfies this contract."""
    def close(self) -> None: ...

class DatabaseConnection:
    """Note: Does NOT inherit from Closable, but satisfies the structural contract!"""
    def close(self) -> None:
        print("[Database] Connection pool cleanly closed.")

def terminate_resource(resource: Closable) -> None:
    # Runtime verification via @runtime_checkable
    if isinstance(resource, Closable):
        resource.close()

def main() -> None:
```



```

# Test ABC enforcement
pub = CloudWatchPublisher()
pub.publish_metric("cpu_load", 42.5)

# Test Structural Protocol
db = DatabaseConnection()
terminate_resource(db)

if __name__ == "__main__":
    main()

```

Run via `uv run python interfaces_overview.py`:

```

[CloudWatch] Published cpu_load = 42.5
[Database] Connection pool cleanly closed.

```

Worked examples

Case 1: Enforcing Framework Contracts with Abstract Base Classes

When building an enterprise data ingestion pipeline, you want to enforce that every storage connector implements reading, writing, and health-checking before allowing the class to be instantiated:

```

# storage_connector.py
from abc import ABC, abstractmethod
from typing import Any

class StorageConnector(ABC):
    """Abstract driver interface for multi-cloud blob stores."""

    def __init__(self, endpoint_url: str) -> None:
        self.endpoint_url = endpoint_url

    @abstractmethod
    def upload_blob(self, key: str, payload: bytes) -> bool:
        """Upload raw bytes to cloud storage under key."""

    @abstractmethod
    def download_blob(self, key: str) -> bytes:
        """Fetch raw bytes from cloud storage."""

    @property
    @abstractmethod
    def is_connected(self) -> bool:

```

```

        """Return True if connection to storage endpoint is active."""

class LocalFileStorage(StorageConnector):
    def __init__(self, directory: str) -> None:
        super().__init__(endpoint_url=f"file://{directory}")
        self._active = True

    @property
    def is_connected(self) -> bool:
        return self._active

    def upload_blob(self, key: str, payload: bytes) -> bool:
        print(f" [LocalFile] Stored {len(payload)} bytes to key '{key}'")
        return True

    def download_blob(self, key: str) -> bytes:
        return b"mock_payload_content"

def main() -> None:
    storage = LocalFileStorage("/var/data")
    print(f"Storage Endpoint: {storage.endpoint_url}")
    print(f"Connected: {storage.is_connected}")
    storage.upload_blob("config.json", b'{"status": "ok"}')

if __name__ == "__main__":
    main()

```

Run:

```
uv run python storage_connector.py
```

Output:

```

Storage Endpoint: file:///var/data
Connected: True
[LocalFile] Stored 16 bytes to key 'config.json'

```

Case 2: Structural Decoupling with Protocols Across Unrelated Packages

In microservices and decoupled libraries, you frequently want to accept any object that can serialize itself to JSON without forcing the caller to inherit from your library's base classes:

```

# decoupled_service.py
from typing import Protocol, runtime_checkable

```

```

@runtime_checkable
class Serializable(Protocol):
    def to_json(self) -> str:
        """Convert object state into a valid JSON string."""
        ...

class AuditEvent:
    """An external domain object with NO dependency on our library!"""
    def __init__(self, action: str, user_id: int) -> None:
        self.action = action
        self.user_id = user_id

    def to_json(self) -> str:
        return f'{{"action": "{self.action}", "user_id": {self.user_id}}}'

def dispatch_event(event: Serializable) -> None:
    """Accepts any object conforming to the Serializable structure."""
    if not isinstance(event, Serializable):
        raise TypeError(f"Object of type {type(event).__name__} does not implement to_json()")
    print(f"Transmitted to EventHub: {event.to_json()}")

def main() -> None:
    audit = AuditEvent("USER_PASSWORD_RESET", 4821)
    dispatch_event(audit)

if __name__ == "__main__":
    main()

```

Run:

```
uv run python decoupled_service.py
```

Output:

```
Transmitted to EventHub: {"action": "USER_PASSWORD_RESET", "user_id": 4821}
```

Case 3: Virtual Subclasses via ABC.register()

Sometimes you want an existing third-party class (which you cannot edit) to be recognized as a subclass of your ABC without monkey-patching or subclassing it. Python allows **registering virtual subclasses**:

```

# virtual_subclass.py
from abc import ABC, abstractmethod

```

```

class SequenceStream(ABC):
    @abstractmethod
    def read_chunk(self, size: int) -> bytes:
        raise NotImplementedError

class ThirdPartyHardwareBuffer:
    """A third-party class provided by a compiled vendor package."""
    def read_chunk(self, size: int) -> bytes:
        return b"\x00" * size

# Register ThirdPartyHardwareBuffer as a virtual subclass of SequenceStream!
SequenceStream.register(ThirdPartyHardwareBuffer)

def process_stream(stream: SequenceStream) -> None:
    print(f"Processing verified stream: {type(stream).__name__}")
    data = stream.read_chunk(4)
    print(f"  Read bytes: {data}")

def main() -> None:
    hw = ThirdPartyHardwareBuffer()

    # isinstance and issubclass now recognize the virtual inheritance!
    print(f"Is hw an instance of SequenceStream? {isinstance(hw, SequenceStream)}")
    print(f"Is class a subclass of SequenceStream? {issubclass(ThirdPartyHardwareBuffer, Seq

    process_stream(hw)

if __name__ == "__main__":
    main()

```

Run:

```
uv run python virtual_subclass.py
```

Output:

```

Is hw an instance of SequenceStream? True
Is class a subclass of SequenceStream? True
Processing verified stream: ThirdPartyHardwareBuffer
  Read bytes: b'\x00\x00\x00\x00'

```

Pitfalls

Pitfall 1: Attempting to Instantiate an Incomplete ABC

If any method marked with `@abstractmethod` is not overridden in a subclass, CPython **forbids instantiation** and raises `TypeError` at construction time:

```
from abc import ABC, abstractmethod

class BaseWorker(ABC):
    @abstractmethod
    def process_job(self) -> None: pass

class IncompleteWorker(BaseWorker):
    pass

# THE TRAP:
try:
    w = IncompleteWorker()
except TypeError as err:
    print(f"Caught expected error: {err}")
```

Output:

Caught expected error: Can't instantiate abstract class IncompleteWorker without an implementation of abstract method process_job

Pitfall 2: Inverting Decorator Order with `@property` and `@abstractmethod`

When creating an abstract property, `@property` **must** be the outermost decorator, and `@abstractmethod` must be the innermost:

```
from abc import ABC, abstractmethod

# THE TRAP (WRONG ORDER):
class BadClass(ABC):
    @abstractmethod
    @property
    def name(self) -> str: pass
    # BUG: Python does not recognize this as an abstract property!

# THE FIX (CORRECT ORDER):
class GoodClass(ABC):
    @property
    @abstractmethod
    def name(self) -> str: pass
    # Correct: @property wraps @abstractmethod
```

Pitfall 3: Assuming @runtime_checkable Validates Types and Signatures

@runtime_checkable on a Protocol only checks that an attribute or method **exists and is callable**; it does **not** validate parameter types, return types, or argument counts at runtime!

```
from typing import Protocol, runtime_checkable

@runtime_checkable
class MathOp(Protocol):
    def compute(self, a: int, b: int) -> int: ...

class BrokenMath:
    # Has a method named 'compute', but signature takes ZERO arguments!
    def compute(self) -> str:
        return "not numbers"

b = BrokenMath()
# isinstance evaluates to True because it only checks for the presence of the method name!
print(f"isinstance check: {isinstance(b, MathOp)}") # True!
```

Takeaway: Use static type checkers (mypy or pyright) to verify signatures; treat @runtime_checkable as a shallow existence check.

Exercises

1. Define an ABC `CryptographicSigner` with abstract methods `sign(payload: bytes) -> bytes` and `verify(payload: bytes, signature: bytes) -> bool`. Implement a concrete subclass `HMACSigner`.
 2. Define a `typing.Protocol` named `Renderable` that requires a `render_html() -> str` method. Write two completely unrelated classes that conform to this protocol without inheriting from it.
 3. Demonstrate the error raised when attempting to instantiate an ABC subclass that forgot to implement an abstract @property.
 4. Implement a custom `__subclasshook__` on an ABC so that any class defining an `.export()` method is dynamically recognized as a subclass via `issubclass()`.
 5. Compare the runtime speed of `isinstance(obj, ABC)` vs `isinstance(obj, Protocol)` across 1,000,000 iterations.
-

Further reading

- PEP 544: *Protocols: Structural subtyping (static duck typing)*.
- PEP 3119: *Introducing Abstract Base Classes*.
- Luciano Ramalho: *Fluent Python* (Chapter 13: Interfaces, Protocols, and ABCs).
- Python Documentation: `abc` — *Abstract Base Classes*.

Modules and Packages

Modules and the Import System

Modules and the Import System

After reading this chapter, you will master Python's module architecture, trace the import resolution pipeline through `sys.modules` and `sys.path`, dynamically load modules using `importlib`, and diagnose and resolve circular import deadlocks.

Mental model

In Python, a **module** is simply a `.py` source file executed as a distinct namespace object (`types.ModuleType`). When an `import foo` statement executes, CPython traverses a four-stage pipeline:

```
import foo
```

```
[ 1. Check sys.modules Cache ]
```

```
    If cached           Return existing module reference immediately
```

```
    (Not cached)
```

```
[ 2. Finders (sys.meta_path) ]    Search sys.path directories for 'foo.py'
```

```
    (Found file)
```

```
[ 3. Loaders (FileLoader) ]      Compile bytecode, allocate empty ModuleType
```

```
[ 4. Execute Module Code ]        Run top-level statements, populate __dict__,  
                                   cache in sys.modules, bind name in caller
```

Because modules are cached in `sys.modules` upon first import, subsequent imports of the same module across threads or files return the cached instance with **zero disk I/O or recompilation**.

Minimal example

Save as `import_internals.py`:

```

# import_internals.py
import importlib
import sys
from types import ModuleType

def inspect_import_cache(module_name: str) -> None:
    print(f"Is '{module_name}' cached in sys.modules? {module_name in sys.modules}")

def dynamically_load_module(module_name: str) -> ModuleType:
    """Programmatically import a module by string name using importlib."""
    print(f"\nDynamically loading '{module_name}' via importlib...")
    mod = importlib.import_module(module_name)
    print(f"Module file path : {getattr(mod, '__file__', 'built-in')}")
    print(f"Module docstring : {mod.__doc__[:40] if mod.__doc__ else 'None'}...")
    return mod

def main() -> None:
    inspect_import_cache("json")

    # Dynamic import
    json_mod = dynamically_load_module("json")
    inspect_import_cache("json")

    # Use dynamically loaded module
    serialized = json_mod.dumps({"status": "healthy", "code": 200})
    print(f"Serialized output: {serialized}")

if __name__ == "__main__":
    main()

```

Run via `uv run python import_internals.py`:

Is 'json' cached in sys.modules? False

Dynamically loading 'json' via importlib...

Module file path : .../lib/python3.14/json/__init__.py

Module docstring : JSON (JavaScript Object Notation) <http:...

Is 'json' cached in sys.modules? True

Serialized output: {"status": "healthy", "code": 200}

Worked examples

Case 1: Dynamic Driver Registry with importlib

In cloud automation and backend platforms, you frequently need to load storage, networking, or database drivers specified by external configuration strings:

```
# driver_loader.py
import importlib
from typing import Any

class DriverLoader:
    @staticmethod
    def load_formatter(format_name: str) -> Any:
        # Standard library formatters mapping
        known_modules = {
            "json": "json",
            "csv": "csv",
        }
        if format_name not in known_modules:
            raise ValueError(f"Unsupported driver: {format_name}")

        mod = importlib.import_module(known_modules[format_name])
        return mod

if __name__ == "__main__":
    loader = DriverLoader()
    driver = loader.load_formatter("json")
    data = {"metric": "cpu", "value": 85.5}
    print(f"Driver loaded dynamically: {driver.__name__}")
    print(f"Formatted: {driver.dumps(data)}")
```

Run:

```
uv run python driver_loader.py
```

Output:

```
Driver loaded dynamically: json
Formatted: {"metric": "cpu", "value": 85.5}
```

Case 2: Diagnosing and Breaking Circular Imports

A circular import occurs when Module A imports Module B at top-level, but Module B imports Module A before Module A has finished executing its definition pass.

Circular Import Deadlock:

```
module_a.py (imports) module_b.py
```

```
(imports)
```

When Python hits `from module_a import Service` inside `module_b`, `module_a` is still being executed; `Service` does not yet exist in `module_a.__dict__`, raising: `ImportError: cannot import name 'Service' from partially initialized module 'module_a'.`

The Fixes:

1. **Move shared domain types/interfaces to a third, leaf module** (`types.py` or `models.py`) imported by both.
2. **Use local imports:** Import the dependency inside the specific function that uses it, rather than at module top-level.
3. **Use `typing.TYPE_CHECKING`:** If the import is only needed for type annotations, gate it behind `if TYPE_CHECKING:`.

```
# circular_fix_pattern.py
from typing import TYPE_CHECKING

if TYPE_CHECKING:
    # Only imported during static analysis (mypy/pyright), NEVER at runtime!
    from collections.abc import Sequence

def process_batch(items: "Sequence[str]") -> int:
    return len(items)

if __name__ == "__main__":
    print("Function defined cleanly without circular runtime dependency.")
    print("Result:", process_batch(["task-1", "task-2"]))
```

Run:

```
uv run python circular_fix_pattern.py
```

Output:

```
Function defined cleanly without circular runtime dependency.
Result: 2
```

Case 3: The `if __name__ == "__main__":` Boundary

When a Python file is invoked directly (`python script.py`), CPython assigns its `__name__` attribute to `"__main__"`. When imported by another file (`import script`), `__name__` is set to the module's file stem (`"script"`).

```
# dual_mode_module.py
def calculate_throughput(bytes_transferred: int, duration_seconds: float) -> float:
    """Pure library function: usable when imported."""
    if duration_seconds <= 0:
        raise ValueError("Duration must be greater than zero.")
    return bytes_transferred / duration_seconds

def _cli_entrypoint() -> None:
    """CLI testing routine: runs ONLY when executed directly."""
    sample_bytes = 104_857_600 # 100 MB
    duration = 2.5
    rate_mb = calculate_throughput(sample_bytes, duration) / (1024 * 1024)
    print(f"[CLI Runner] Transfer rate: {rate_mb:.2f} MB/s")

if __name__ == "__main__":
    _cli_entrypoint()
```

Run:

```
uv run python dual_mode_module.py
```

Output:

```
[CLI Runner] Transfer rate: 40.00 MB/s
```

When another module runs from `dual_mode_module import calculate_throughput, _cli_entrypoint()` will not execute.

Pitfalls

Pitfall 1: Standard Library Shadowing

Naming a local script after a standard library module (e.g. creating `math.py`, `random.py`, `test.py`, or `json.py` in your project folder) breaks the import system:

Your Directory:

<code>math.py</code>	(Your script)
<code>test_math.py</code>	(<code>import math</code> -> loads YOUR <code>math.py</code> instead of CPython's!)

Because CPython checks the current working directory first in `sys.path[0]`, your local file shadows the standard library module, causing cryptic `AttributeError: module 'math' has no attribute 'sqrt'` errors.

Exercises

1. Print all paths in `sys.path` and inspect where Python looks for modules on your machine.
 2. Write a function that accepts a module name as a string, checks whether it is already loaded in `sys.modules`, and reloads it using `importlib.reload()` if present.
 3. Create a simulated two-module circular import and resolve it using local function-level imports.
 4. Use `__all__` to restrict the symbols exported when a consumer executes `from my_module import *`.
-

Further reading

- Python Documentation: *The Import System* (`sys.meta_path`, Finders, Loaders).
- Python Standard Library: `importlib` and `sys.modules`.
- PEP 302: *New Import Hooks*.

Packages, Namespaces, and Public APIs

Packages, Namespaces, and Public APIs

After reading this chapter, you will organize codebases into hierarchical Python packages, structure `__init__.py` to create clean public API façades, contrast standard packages with PEP 420 namespace packages, use relative imports correctly, and control exported symbols using `__all__`.

Mental model

A **package** is a directory that Python treats as a module containing sub-modules or sub-packages.

Package Hierarchy:

<code>my_service/</code>	Package Directory
<code>__init__.py</code>	Marks directory as regular package; defines public API
<code>core/</code>	Sub-package
<code>__init__.py</code>	
<code>engine.py</code>	Module: <code>my_service.core.engine</code>
<code>transports/</code>	Sub-package
<code>__init__.py</code>	
<code>http.py</code>	Module: <code>my_service.transports.http</code>

When importing `my_service.core.engine`: 1. `my_service/__init__.py` executes and binds `my_service`. 2. `my_service/core/__init__.py` executes and binds `core` on `my_service`. 3. `my_service/core/engine.py` executes and binds `engine` on `my_service.core`.

Minimal example

Save as `package_facade.py`:

```
# package_facade.py
import tempfile
from pathlib import Path

def create_sample_package(root: Path) -> None:
    pkg_dir = root / "cloudpkg"
    pkg_dir.mkdir()
```

```

    # Submodule 1: Internal client
    client_code = """
class CloudClient:
    def connect(self) -> str:
        return "Connected to cloud endpoint."
"""
    (pkg_dir / "client.py").write_text(client_code)

    # __init__.py creates a clean public API facade and restricts __all__
    init_code = """
from .client import CloudClient

__all__ = ["CloudClient"]
"""
    (pkg_dir / "__init__.py").write_text(init_code)

def main() -> None:
    import sys
    with tempfile.TemporaryDirectory() as tmpdir:
        tmp_path = Path(tmpdir)
        create_sample_package(tmp_path)

        # Append temporary directory to sys.path to simulate an installed package
        sys.path.insert(0, str(tmp_path))
        try:
            # Consumer imports directly from top-level package facade
            import cloudpkg
            print(f"Package loaded: {cloudpkg.__name__}")
            print(f"Exported symbols (__all__): {cloudpkg.__all__}")

            client = cloudpkg.CloudClient()
            print(f"Client method invocation : {client.connect()}")
        finally:
            sys.path.pop(0)

if __name__ == "__main__":
    main()

```

Run via `uv run python package_facade.py`:

```

Package loaded: cloudpkg
Exported symbols (__all__): ['CloudClient']
Client method invocation : Connected to cloud endpoint.

```

Worked examples

Case 1: Relative vs Absolute Imports

Within a package, files can refer to sibling or parent modules using dot notation (.):

```
my_app/
  models/
    __init__.py
    user.py
  services/
    __init__.py
    auth.py           Needs to import User from models
```

Inside `my_app/services/auth.py`:

```
# Absolute import (Recommended for clarity in large apps):
from my_app.models.user import User

# Relative import (Recommended for reusable, relocatable library packages):
from ..models.user import User
from .helpers import local_token_generator # Sibling import from same directory
```

- `.` refers to the **current package directory**.
- `..` refers to the **parent package directory**.
- `...` refers to the **grandparent package directory**.

Relative imports rely on the module's `__name__` property containing package context. If you run a file containing relative imports directly with `python my_app/services/auth.py`, it fails with `ImportError: attempted relative import with no known parent package`. **Always run package modules using the `-m` flag:**

```
python -m my_app.services.auth
```

Case 2: The `__all__` Contract and `from module import *`

The `__all__` list specifies which symbols are exported when a user executes wildcard imports (`from mypkg import *`), and communicates the supported public API to linters and documentation generators:

```
# api_contract.py
__all__ = ["PublicService", "public_helper"]

class PublicService:
    """Supported public class."""
    pass
```

```
def public_helper() -> str:
    """Supported public function."""
    return "OK"

def _internal_helper() -> None:
    """Private function: prefixed with underscore."""
    pass

class InternalEngine:
    """Unexported class: omitted from __all__."""
    pass

if __name__ == "__main__":
    print(f"Exported public symbols in module: {__all__}")
```

Run:

```
uv run python api_contract.py
```

Output:

```
Exported public symbols in module: ['PublicService', 'public_helper']
```

Case 3: PEP 420 Namespace Packages

In modern Python, a directory **without** an `__init__.py` file is treated as a **namespace package**. This enables splitting a single top-level package namespace (e.g. `company.core`, `company.storage`, `company.auth`) across completely separate directories, wheels, or git repositories:

Repo 1 (Core):

```
company/                                (NO __init__.py)
  core/
    engine.py
```

Repo 2 (Plugins):

```
company/                                (NO __init__.py)
  plugins/
    exporter.py
```

When both repositories are installed into **site-packages**, CPython merges them into a single unified `company` namespace at runtime!

Pitfalls

Pitfall 1: Executing Submodules Directly Instead of via `-m`

Running `python pkg/sub/file.py` sets `sys.path[0]` to `pkg/sub/`, stripping package hierarchy context:

```
# BROKEN:
python my_pkg/submodule.py
# -> ImportError: attempted relative import with no known parent package

# CORRECT:
uv run python -m my_pkg.submodule
```

Pitfall 2: Heavy Work Inside `__init__.py`

`__init__.py` runs every time **any** submodule of the package is imported. Putting database connections, expensive file I/O, or heavy third-party imports inside `__init__.py` slows down every consumer importing lightweight utilities from that package.

Exercises

1. Create a package directory structure with an `__init__.py` that exposes a single `__version__` string and re-exports a class from a nested submodule.
 2. Given a two-level package hierarchy, write a relative import statement from `pkg.controllers.auth` to import a helper from `pkg.utils.crypto`.
 3. Create a PEP 420 namespace package with two directories in different locations and demonstrate that both submodules are importable under the same parent namespace.
 4. Define `__all__` in a module and verify that names not listed in `__all__` are not imported when using `from module import *`.
-

Further reading

- PEP 420: *Implicit Namespace Packages*.
- Python Tutorial: *Packages*.
- Python Packaging User Guide: *Creating and Structuring Projects*.

Workspaces, Virtual Environments, and Dependency Isolation

Workspaces, Virtual Environments, and Dependency Isolation

After reading this chapter, you will master the internal mechanics of Python virtual environments (`pyvenv.cfg` and `site-packages`), manage modern multi-package monorepo workspaces using `uv`, author declarative `pyproject.toml` project files, and run self-contained scripts using PEP 723 inline dependency metadata.

Mental model

A Python virtual environment is not a virtual machine or a container. It is an isolated directory tree containing symlinks to a base Python interpreter and a dedicated `site-packages` directory.

Virtual Environment Architecture (`.venv/`):

```
.venv/  
  bin/  
    python    Symlink to base CPython binary  
    uv  
  pyvenv.cfg  Configuration: home = /usr/bin, version = 3.14.0  
  lib/python3.14/  
    site-packages/  Isolated third-party libraries installed here
```

When CPython starts: 1. It searches upward from its executable for `pyvenv.cfg`. 2. If found, it sets `sys.prefix` to the `.venv` folder, while keeping `sys.base_prefix` pointing to the host Python installation. 3. Third-party package imports look exclusively inside the `.venv`'s `site-packages` directory.

Minimal example

Save as `venv_internals.py`:

```
# venv_internals.py  
import sys  
import site  
from pathlib import Path
```

```

def inspect_environment() -> None:
    is_virtual_env = sys.prefix != sys.base_prefix

    print(f"Is running inside a virtual environment? {is_virtual_env}")
    print(f"Base CPython runtime (sys.base_prefix) : {sys.base_prefix}")
    print(f"Active environment root (sys.prefix)    : {sys.prefix}")
    print(f"Active Python executable (sys.executable): {sys.executable}")

    # Inspect site-packages search paths
    site_packages = site.getsitepackages()
    print("\nActive site-packages directories:")
    for sp in site_packages:
        print(f"    - {sp}")

def main() -> None:
    inspect_environment()

if __name__ == "__main__":
    main()

```

Run via `uv run python venv_internals.py`:

```

Is running inside a virtual environment? True
Base CPython runtime (sys.base_prefix) : ...
Active environment root (sys.prefix)    : .../.venv
Active Python executable (sys.executable): .../.venv/bin/python

Active site-packages directories:
- .../.venv/lib/python3.14/site-packages

```

Worked examples

Case 1: Inline Script Dependencies (PEP 723)

In automation, CI/CD, and DevOps scripts, creating a separate `pyproject.toml` and `.venv` for a single 50-line script adds unnecessary friction. PEP 723 allows embedding dependency requirements directly inside the script header:

```

# generate_qr_matrix.py
# /// script
# requires-python = ">=3.12"
# dependencies = [
#     "rich>=13.0.0",
# ]

```



```
# ///
import sys
from rich.console import Console
from rich.table import Table

def display_cluster_health() -> None:
    console = Console()
    table = Table(title="Production Node Health")
    table.add_column("Node", style="cyan")
    table.add_column("Status", style="green")
    table.add_column("Load", style="magenta")

    table.add_row("k8s-node-01", "READY", "12.4%")
    table.add_row("k8s-node-02", "READY", "8.9%")
    table.add_row("k8s-node-03", "DRAINING", "94.2%")

    console.print(table)

if __name__ == "__main__":
    display_cluster_health()
```

When you execute:

```
uv run generate_qr_matrix.py
```

uv automatically: 1. Parses the `# /// script` metadata block. 2. Creates an ephemeral, cached virtual environment containing rich. 3. Executes the script inside the isolated environment with zero pollution of the host system!

Case 2: Multi-Package Workspaces with uv

In modern monorepos (such as a backend repository containing shared utilities, an API service, and a background worker), uv supports multi-package workspaces configured via `pyproject.toml`:

```
# pyproject.toml (Monorepo root)
[tool.uv.workspace]
members = ["packages/*", "services/*"]

[package]
name = "enterprise-monorepo"
version = "0.1.0"
```

Directory Structure:

```
monorepo/
  pyproject.toml
  uv.lock
```

Single shared lockfile for entire monorepo

```

packages/
  common-auth/          Shared internal library
    pyproject.toml
services/
  api-server/           Depends on "common-auth" via path
    pyproject.toml
  worker-engine/        Depends on "common-auth"
    pyproject.toml

```

Benefits: - **Single Lockfile**: All services and packages share a single deterministic dependency resolution graph. - **Instant Symlinking**: Changes to `common-auth` are immediately reflected in `api-server` without publishing or reinstalling packages. - **Fast CI**: Re-uses cached pre-compiled wheels across all subprojects.

Pitfalls

Pitfall 1: Breaking OS Packages with `sudo pip install` (PEP 668)

Modern Linux distributions enforce PEP 668 (EXTERNALLY-MANAGED). Attempting to install packages into the global system Python with `sudo pip` risks breaking system tools (like `apt`, `yum`, or `systemd` daemons):

```

error: externally-managed-environment
× This environment is externally managed.
To install Python packages, create a virtual environment with 'uv venv'.

```

Always use project virtual environments (`uv venv` or `uv run`) instead of global system-level modifications.

Pitfall 2: Committing `.venv/` into Version Control

Virtual environments contain platform-specific binaries, architecture-dependent C-extensions, and local absolute filesystem symlinks. They are not portable across machines:

```

# ALWAYS add to .gitignore:
.venv/
__pycache__/
*.pyc

```

Version control should only store `pyproject.toml` (human-readable dependencies) and `uv.lock` (reproducible binary lockfile).

Exercises

1. Create a script that checks `sys.prefix == sys.base_prefix` and exits with an error code if it is not running inside a virtual environment.
 2. Write a single-file script using PEP 723 inline metadata that specifies a dependency on `httpx` and fetches a test URL using `uv run`.
 3. Create a minimal `pyproject.toml` project file with `uv init my-project` and inspect the generated layout and configuration keys.
 4. Inspect the contents of `.venv/pyvenv.cfg` and identify the base Python interpreter home path.
-

Further reading

- PEP 621: *Storing project metadata in pyproject.toml*.
- PEP 668: *Marking Python base environments as “externally managed”*.
- PEP 723: *Inline script metadata*.
- Astrid toolchain: *Modern packaging with uv*.

Import Hooks, Finders, and ModuleSpec Mechanics

Import Hooks, Finders, and ModuleSpec Mechanics

After reading this chapter, you will master the low-level architecture of Python’s import machinery (PEP 302 and PEP 451), trace how CPython discovers and loads modules via `sys.meta_path`, construct custom `MetaPathFinder` and `Loader` classes to import virtual in-memory modules, and diagnose circular import deadlocks.

Mental model

When Python executes `import foo`, it does not simply look for `foo.py` on disk. Instead, the runtime delegates to an extensible multi-stage search pipeline orchestrated by `importlib`:

1. Check `sys.modules` Cache

Found in Cache?
YES

Cache Miss

Return cached module

2. Iterate `sys.meta_path` finders
(Built-in, Frozen, PathFinder...)

`finder.find_spec("foo", path)`

Spec Found?
YES

All Failed

Return `ModuleSpec`

Raise `ModuleNotFoundError`

3. `loader.create_module(spec)`
(Allocates `PyModuleObject`)

4. Add to `sys.modules` Cache

(Early registration to break loops)

5. loader.exec_module(module)
(Executes top-level code in module)

Minimal example

Save as virtual_import_hook.py:

```
# virtual_import_hook.py
import sys
import types
from importlib.abc import Loader, MetaPathFinder
from importlib.machinery import ModuleSpec

class VirtualModuleLoader(Loader):
    """Executes code for synthetic in-memory modules."""

    def __init__(self, code_str: str) -> None:
        self.code_str = code_str

    def create_module(self, spec: ModuleSpec) -> types.ModuleType | None:
        return None # Use default module allocation

    def exec_module(self, module: types.ModuleType) -> None:
        exec(self.code_str, module.__dict__)

class VirtualConfigFinder(MetaPathFinder):
    """Intercepts imports matching 'virtual_cluster' and serves in-memory modules."""

    def find_spec(
        self,
        fullname: str,
        path: object | None,
        target: types.ModuleType | None = None,
    ) -> ModuleSpec | None:
        if fullname == "virtual_cluster":
            code = (
                "CLUSTER_ID = 'us-east-prod-01'\n"
                "MAX_REPLICAS = 12\n"
                "def get_status(): return 'HEALTHY'\n"
            )
            return ModuleSpec(fullname, VirtualModuleLoader(code))
```

```

        return None

def main() -> None:
    # 1. Install custom finder into sys.meta_path
    finder = VirtualConfigFinder()
    sys.meta_path.insert(0, finder)

    try:
        # 2. Import module that DOES NOT EXIST on disk!
        import virtual_cluster as cluster_cfg # type: ignore

        print("Successfully imported virtual module:")
        print(f"  Cluster ID:    {cluster_cfg.CLUSTER_ID}")
        print(f"  Max Replicas: {cluster_cfg.MAX_REPLICAS}")
        print(f"  Status:        {cluster_cfg.get_status()}")
        print(f"  Module file:   {getattr(cluster_cfg, '__file__', 'None (in-memory)')}")
    finally:
        # Clean up sys.meta_path and sys.modules
        sys.meta_path.remove(finder)
        sys.modules.pop("virtual_cluster", None)

if __name__ == "__main__":
    main()

```

Run via `uv run python virtual_import_hook.py`:

```

Successfully imported virtual module:
Cluster ID:    us-east-prod-01
Max Replicas: 12
Status:        HEALTHY
Module file:   None (in-memory)

```

Worked examples

Case 1: Loading Modules Directly from JSON Configurations

In microservices, you may want to expose a structured JSON configuration file as an importable Python module, allowing type-safe, dotted access without manual dictionary queries:

```

# json_importer.py
import json
import sys
import types
from importlib.abc import Loader, MetaPathFinder

```

```

from importlib.machinery import ModuleSpec

class JSONModuleLoader(Loader):
    def __init__(self, data: dict[str, object]) -> None:
        self.data = data

    def exec_module(self, module: types.ModuleType) -> None:
        for key, value in self.data.items():
            setattr(module, key, value)

class JSONConfigFinder(MetaPathFinder):
    def __init__(self, json_payload: str) -> None:
        self.parsed = json.loads(json_payload)

    def find_spec(
        self,
        fullname: str,
        path: object | None,
        target: types.ModuleType | None = None,
    ) -> ModuleSpec | None:
        if fullname == "app_settings":
            return ModuleSpec(fullname, JSONModuleLoader(self.parsed))
        return None

def main() -> None:
    raw_json = '{"ENVIRONMENT": "staging", "DATABASE_PORT": 5432, "ENABLE_METRICS": true}'
    finder = JSONConfigFinder(raw_json)
    sys.meta_path.insert(0, finder)

    try:
        import app_settings # type: ignore

        print(f"Imported settings module: {app_settings}")
        print(f"  Environment: {app_settings.ENVIRONMENT}")
        print(f"  Port:          {app_settings.DATABASE_PORT}")
        print(f"  Metrics:       {app_settings.ENABLE_METRICS}")
    finally:
        sys.meta_path.remove(finder)
        sys.modules.pop("app_settings", None)

if __name__ == "__main__":
    main()

```

Run:

```
uv run python json_importer.py
```

Output:


```
Imported settings module: <module 'app_settings'>
Environment: staging
Port:      5432
Metrics:   True
```

Case 2: Auditing Dependency Load Latency via a MetaPath Hook

In performance engineering, imports can significantly increase application startup time. A meta path hook can intercept every module imported and benchmark its compilation time:

```
# import_profiler.py
import sys
import time
from collections.abc import Sequence
from importlib.abc import MetaPathFinder
from importlib.machinery import ModuleSpec
from typing import Any

class ProfileImportFinder(MetaPathFinder):
    """Wraps existing finders to measure import resolution latency."""

    def __init__(self) -> None:
        self.timings: dict[str, float] = {}

    def find_spec(
        self,
        fullname: str,
        path: Sequence[str] | None,
        target: Any | None = None,
    ) -> ModuleSpec | None:
        start = time.perf_counter()
        # Delegate to remaining finders in sys.meta_path
        for finder in sys.meta_path:
            if finder is self:
                continue
            if hasattr(finder, "find_spec"):
                spec = finder.find_spec(fullname, path, target)
                if spec is not None:
                    duration_ms = (time.perf_counter() - start) * 1000.0
                    self.timings[fullname] = duration_ms
                    return spec
        return None

def main() -> None:
```

```

profiler = ProfileImportFinder()
sys.meta_path.insert(0, profiler)

try:
    # Import several standard library modules
    import csv
    import json
    import math

    print("Import Resolution Benchmark:")
    for mod, ms in sorted(profiler.timings.items(), key=lambda x: x[1], reverse=True)[:5]:
        print(f" {mod:20} -> {ms:.4f} ms")
finally:
    sys.meta_path.remove(profiler)

if __name__ == "__main__":
    main()

```

Run:

```
uv run python import_profiler.py
```

Output:

```

Import Resolution Benchmark:
csv                -> 0.0521 ms
json               -> 0.0418 ms
math               -> 0.0210 ms

```

Pitfalls

Pitfall 1: Circular Import Deadlocks

Circular imports occur when module A imports module B, which in turn imports module A before A has finished initializing:

```

# File: module_a.py
# from module_b import helper_b  <-- Imports B before A defines 'value_a'!
# value_a = 42

# File: module_b.py
# from module_a import value_a  <-- CRASH! AttributeError: partially initialized module

```

```
# THE REASON:
# CPython puts 'module_a' in sys.modules BEFORE executing its body (to prevent infinite loop)
# When module_b queries module_a.value_a, module_a exists, but its body has not run yet!

# THE FIX:
# 1. Refactor shared dependencies into a third module (e.g. models.py or common.py).
# 2. Or defer the import inside the specific function that uses it:
def use_helper():
    from module_b import helper_b
    return helper_b()
```

Pitfall 2: Forgetting to Clean Up `sys.meta_path`

Adding finders to `sys.meta_path` alters global runtime behavior for the entire Python process. If a finder raises an unhandled exception or loops infinitely, every future `import` statement throughout the application can crash:

```
# ALWAYS use a try ... finally block when modifying sys.meta_path dynamically:
finder = CustomFinder()
sys.meta_path.insert(0, finder)
try:
    # Perform operations
    import target_module
finally:
    if finder in sys.meta_path:
        sys.meta_path.remove(finder)
```

Exercises

1. Inspect `sys.meta_path` and print the class names of all default finders configured by CPython at startup.
 2. Build an import hook that intercepts imports of `secrets_config` and decrypts an encrypted string into module attributes.
 3. Write an import hook that logs a warning to `stderr` whenever a deprecated module name is imported.
 4. Demonstrate how `importlib.reload(module)` works, and explain why instances of classes created before the reload do not update their class pointer.
 5. Create a virtual loader that allows importing `.txt` files directly, exposing the file contents as a module-level string attribute `CONTENT`.
-

Further reading

- PEP 302: *New Import Hooks*.
- PEP 451: *A ModuleSpec Type for the Import System*.
- Python Documentation: `importlib` — *The implementation of import*.
- Anthony Shaw: *CPython Internals* (Chapter 11: The Import System).

Lazy Imports (PEP 810)

Lazy Imports (PEP 810)

Thesis

Python 3.15 adds a soft keyword, **lazy**, that turns an **import** at module scope into a promise: the name is bound now, the module is found, loaded, and executed the **first time the name is used**. That fixes the oldest startup tax in the language — a CLI that loads its whole dependency graph to print `--help` — without the old workarounds (imports buried in functions, `importlib` tricks, `if TYPE_CHECKING:` blocks). The price is that **import errors and import side effects move** from the top of the file to wherever the name is first touched. Use **lazy** deliberately on heavy, side-effect-free dependencies of short-lived programs; keep everything else eager; and make your tests reify what you deferred.

Version note: lazy imports are the headline feature of **Python 3.15** (final release scheduled for **2026-10-09** after a late rc3 for lazy-import fixes). Every listing below was run on **CPython 3.15.0rc2** (Linux x86-64) with `uv`; output is copied from those runs. Where 3.15 behaves differently from the PEP text, this chapter follows the 3.15 documentation.

Mental model

```
lazy import json          # 1. bind  json -> <lazy_import 'json'>  (nothing found, nothing
...
json.dumps(x)             # 2. first use: LOAD_GLOBAL/LOAD_NAME sees the proxy
                           #    -> runs the normal import machinery NOW
                           #    -> sys.modules['json'] = module; rebinds the global
json.dumps(y)             # 3. ordinary module access; the interpreter specializes it
```

Question	Eager <code>import x</code>	<code>lazy import x</code>
When is <code>x</code> located and executed?	At the statement	At first use of the name
Where does <code>ModuleNotFoundError</code> surface?	The import line	The first-use line (chained to the import line)
When do import-time side effects run?	At startup	At first use — possibly never
In <code>sys.modules</code> before use?	Yes	No (listed in <code>sys.lazy_modules</code>)
After first use	Module object	Indistinguishable module object

Rules that the compiler enforces:

- `lazy` is legal only at **module scope** — not in a function, a class body, or a `try/except/finally` block (each is a `SyntaxError`).
- `lazy from m import *` and `lazy from __future__ import ...` are errors.
- `lazy` is a *soft* keyword: `lazy = 3` still works everywhere else.
- `lazy from m import a, b` binds two proxies; the first one touched loads **all of m** and resolves only that name.
- Laziness is **local and non-recursive**: `lazy import pkg` does not make `pkg`'s own imports lazy.

Two compatibility and control knobs:

- `__lazy_modules__` — a module-level container of fully qualified module names. On 3.15+ a plain `import` of a listed module behaves as if it had `lazy`; on 3.14 and older the variable is just an unused global.
- **Global mode** — `-X lazy_imports=all`, `PYTHON_LAZY_IMPORTS=all`, or `sys.set_lazy_imports("all")` make every eligible module-level import potentially lazy; `"normal"` (the default) honours only `lazy` and `__lazy_modules__`. A filter, `sys.set_lazy_imports_filter(fn)`, gets `(importer, name, fromlist)` and returns `False` to force an import eager. (PEP 810 also described a `"none"` mode; CPython 3.15 accepts only `"normal"` and `"all"` — `"none"` is a fatal startup error on 3.15.0rc2.)

Worked examples

Case 1: A multi-command CLI that only pays for the command you run

Save as `deploy.py`. Each subcommand needs a different, non-trivial part of the standard library. With `lazy`, `deploy status` pays only for `json`.

```
# deploy.py - a CLI whose subcommands need very different libraries
import argparse
import sys

lazy import json
lazy import sqlite3
lazy import tarfile
lazy import urllib.request
lazy from email.message import EmailMessage

def cmd_status(args):
    print(json.dumps({"service": args.service, "state": "running"}))

def cmd_backup(args):
    with sqlite3.connect(":memory:") as db, tarfile.open(args.out, "w:gz"):
```

```

        db.execute("select 1")
    print("backup written to", args.out)

def cmd_notify(args):
    msg = EmailMessage()
    msg["Subject"] = f"{args.service} deployed"
    print(msg["Subject"])

def main(argv=None):
    p = argparse.ArgumentParser(prog="deploy")
    sub = p.add_subparsers(required=True)
    s = sub.add_parser("status"); s.add_argument("service"); s.set_defaults(fn=cmd_status)
    b = sub.add_parser("backup"); b.add_argument("out"); b.set_defaults(fn=cmd_backup)
    n = sub.add_parser("notify"); n.add_argument("service"); n.set_defaults(fn=cmd_notify)
    args = p.parse_args(argv)
    args.fn(args)
    heavy = ["json", "sqlite3", "tarfile", "urllib.request", "email.message"]
    print("loaded:", [m for m in heavy if m in sys.modules], file=sys.stderr)
    print("modules:", len(sys.modules), file=sys.stderr)

if __name__ == "__main__":
    main()

```

Make an eager twin for comparison (same file without the keyword) and run both:

```

sed 's/^lazy //' deploy.py > deploy_eager.py
uv run --python 3.15 python deploy.py status web
uv run --python 3.15 python deploy_eager.py status web
uv run --python 3.15 python deploy.py notify web

```

Output:

```

{"service": "web", "state": "running"}
loaded: ['json']
modules: 75
{"service": "web", "state": "running"}
loaded: ['json', 'sqlite3', 'tarfile', 'urllib.request', 'email.message']
modules: 136
web deployed
loaded: ['email.message']
modules: 104

```

Same code, same imports listed at the top where a reader expects them, 61 fewer modules executed for `status`. To see *where* time goes, ask the interpreter:


```
uv run --python 3.15 python -X importtime deploy_eager.py status web 2>&1 \
| grep -E ' (sqlite3|tarfile|email\.message|urllib\.request)$'
```

Output (microseconds: self | cumulative; your numbers will differ):

```
import time:      180 |      1512 | sqlite3
import time:     1422 |      6160 | tarfile
import time:      697 |      1090 | email.message
import time:     1272 |     20131 | urllib.request
```

Wall clock, measured with a small harness:

```
# bench.py - median wall time of N fresh interpreter starts
import statistics
import subprocess
import sys
import time

def median_ms(cmd, n=30):
    samples = []
    for _ in range(n):
        t0 = time.perf_counter()
        subprocess.run(cmd, check=True, capture_output=True)
        samples.append(time.perf_counter() - t0)
    return statistics.median(samples) * 1000

for script in ("deploy_eager.py", "deploy.py"):
    print(f"{script:16} {median_ms([sys.executable, script, 'status', 'web']):6.1f} ms")

uv run --python 3.15 python bench.py
```

Output on the author's box (illustrative — timings depend on CPU, disk cache, and build):

```
deploy_eager.py    54.0 ms
deploy.py          28.3 ms
```

Roughly half the startup for the common path, for the price of one keyword per line.

Case 2: The error moves — and so must your smoke test

Save as `report.py`. The `yaml` dependency is missing from this environment, and nothing complains until the one code path that uses it runs.

```
# report.py - the dependency is missing, but nobody notices at startup
import sys
```

```

lazy import yaml                # not installed in this environment

print("started; lazy modules pending:", sorted(m for m in sys.lazy_modules if m == "yaml"))

def render(data):
    return yaml.safe_dump(data)

if len(sys.argv) > 1 and sys.argv[1] == "--yaml":
    print(render({"ok": True}))
else:
    print("plain output: ok")

uv run --python 3.15 python report.py; echo "exit=$?"
uv run --python 3.15 python report.py --yaml; echo "exit=$?"

```

Output (paths shortened):

```

started; lazy modules pending: ['yaml']
plain output: ok
exit=0
started; lazy modules pending: ['yaml']
Traceback (most recent call last):
  File "report.py", line 4, in <module>
    lazy import yaml                # not installed in this environment
ImportError: lazy import of 'yaml' raised an exception during resolution

```

The above exception was the direct cause of the following exception:

```

Traceback (most recent call last):
  File "report.py", line 14, in <module>
    print(render({"ok": True}))
    ~~~~~~^~~~~~
  File "report.py", line 10, in render
    return yaml.safe_dump(data)
           ~~~~
ModuleNotFoundError: No module named 'yaml'
exit=1

```

The chained traceback names both the `lazy import` line and the line that triggered it — good for debugging, useless if it first happens in production. “It starts” is no longer a test. Resolve every lazy binding in CI instead. Save as `selfcheck.py` next to `report.py`:

```
# selfcheck.py - fail fast in CI: resolve every lazy binding in a module
import importlib
import sys
import types

def reify_all(module_name):
    mod = importlib.import_module(module_name)
    failures = []
    for name, value in list(vars(mod).items()):
        if isinstance(value, types.LazyImportType):
            try:
                setattr(mod, name, value.resolve())
            except ImportError as exc:
                failures.append(f"{module_name}.{name}: {type(exc).__name__}: {exc}")
    return failures

if __name__ == "__main__":
    sys.argv = ["report.py"]          # import report.py without its --yaml branch
    problems = reify_all("report")
    for p in problems:
        print("LAZY IMPORT BROKEN:", p)
    sys.exit(1 if problems else 0)
```

```
uv run --python 3.15 python selfcheck.py; echo "exit=$?"
```

Output:

```
started; lazy modules pending: ['yaml']
plain output: ok
LAZY IMPORT BROKEN: report.yaml: ModuleNotFoundError: No module named 'yaml'
exit=1
```

`vars(module)` (like `globals()` and `module.__dict__`) returns the raw namespace **without** reifying, so proxies are visible as `types.LazyImportType` and can be resolved explicitly with `.resolve()`. Normal attribute access (`module.name`, `getattr`) would trigger the import instead.

Case 3: One source file for 3.13, 3.14, and 3.15 with `__lazy_modules__`

Libraries that still support older Pythons cannot use the new syntax — it is a `SyntaxError` before 3.15. `__lazy_modules__` is the bridge. Save as `compat.py`:

```
# compat.py - lazy on 3.15+, plain eager imports on older Pythons
__lazy_modules__ = {"decimal", "fractions"}
```

```

import sys
import decimal
from fractions import Fraction

ver = sys.version.split()[0]
print(f"{ver:10} before use: decimal={'decimal' in sys.modules} fractions={'fractions' in sy
total = decimal.Decimal("1.10") + decimal.Decimal("2.20")
print(f"{ver:10} after Decimal: decimal={'decimal' in sys.modules} fractions={'fractions' in

uv run --python 3.13 python compat.py
uv run --python 3.15 python compat.py

```

Output:

```

3.13.5      before use: decimal=True fractions=True
3.13.5      after Decimal: decimal=True fractions=True total=3.30
3.15.0rc2   before use: decimal=False fractions=False
3.15.0rc2   after Decimal: decimal=True fractions=False total=3.30

```

On 3.15 `fractions` is never loaded because `Fraction` was never used. List **fully qualified** module names (relative imports are resolved before the lookup), define the variable **before** the imports it governs, and remember it covers `from m import x` when `m` is listed.

Case 4: Type-only imports without `if TYPE_CHECKING:`

With deferred annotations (3.14+), an annotation is not evaluated until someone asks for it. Pair that with `lazy from` and an import used only in hints costs nothing at runtime — and stays a normal, visible import that type checkers understand.

```

mkdir -p shop && touch shop/__init__.py

# shop/models.py - imagine this pulls in an ORM and takes 200 ms
print("  [shop.models imported]")

class Order:
    def __init__(self, total):
        self.total = total

# shop/pricing.py - needs Order only for type hints
lazy from shop.models import Order

def apply_discount(order: Order, pct: int) -> Order:
    order.total = order.total * (100 - pct) // 100

```

```

        return order

# main.py
import annotationlib
import sys

from shop import pricing

print("pricing imported; shop.models loaded?", "shop.models" in sys.modules)

class FakeOrder:          # duck-typed: pricing never touches the real class
    total = 1000

print("discounted:", pricing.apply_discount(FakeOrder(), 15).total)
print("after call;  shop.models loaded?", "shop.models" in sys.modules)

hints = annotationlib.get_annotations(pricing.apply_discount)
print("annotations:", hints)
print("after hints; shop.models loaded?", "shop.models" in sys.modules)

uv run --python 3.15 python main.py

```

Output:

```

pricing imported; shop.models loaded? False
discounted: 850
after call;  shop.models loaded? False
[shop.models imported]
annotations: {'order': <class 'shop.models.Order'>, 'pct': <class 'int'>, 'return': <class 'shop.models.Order'>}
after hints; shop.models loaded? True

```

The heavy module loads only when a tool (a validator, a serializer, `inspect.signature`) actually evaluates the annotations — and then it loads correctly instead of failing the way a `TYPE_CHECKING`-guarded name would.

Case 5: Application-wide laziness, scoped by a filter

The global mode is for **applications**, not libraries: it changes how *other people's* modules import. Scope it with a filter so only your own package is affected.

```
mkdir -p myapp && touch myapp/__init__.py
```

```

# myapp/slow.py
print("    [myapp.slow imported]")
VALUE = 42

# run_all.py - application-wide laziness, scoped to our own package
import sys

def only_myapp(importer, name, fromlist):
    return name == "myapp" or name.startswith("myapp.")

sys.set_lazy_imports_filter(only_myapp)
sys.set_lazy_imports("all")

import myapp.slow          # lazy: matches the filter
import json                # eager: filter returns False

print("mode:", sys.get_lazy_imports())
print("json loaded:", "json" in sys.modules, "| myapp.slow loaded:", "myapp.slow" in sys.mod
print("value:", myapp.slow.VALUE)

uv run --python 3.15 python run_all.py

```

Output:

```

mode: all
json loaded: True | myapp.slow loaded: False
    [myapp.slow imported]
value: 42

```

The same switch from outside, on a file that never mentions lazy:

```

# plain.py - no lazy keyword anywhere
import sys
import json
import myapp.slow
print("json loaded:", "json" in sys.modules, "| myapp.slow loaded:", "myapp.slow" in sys.mod

uv run --python 3.15 python plain.py
uv run --python 3.15 python -X lazy_imports=all plain.py
PYTHON_LAZY_IMPORTS=all uv run --python 3.15 python plain.py

```

Output:

```

[myapp.slow imported]

```

```
json loaded: True | myapp.slow loaded: True
json loaded: False | myapp.slow loaded: False
json loaded: False | myapp.slow loaded: False
```

Precedence is `sys.set_lazy_imports()` over `-X lazy_imports` over `PYTHON_LAZY_IMPORTS`. The filter's `importer` argument may be `None`; do not call string methods on it without checking.

The trap

Import-time side effects silently stop happening. Registries populated by decorators, `__init_subclass__` hooks, codec or plugin registration, logging configuration, monkeypatches — all of them run *when the module executes*. Make that import lazy and the module may never execute.

```
# registry.py
HANDLERS = {}

def handler(name):
    def deco(fn):
        HANDLERS[name] = fn
        return fn
    return deco

# csv_export.py - registers itself as an import side effect
from registry import handler

@handler("csv")
def export(rows):
    return "\n".join(",".join(map(str, r)) for r in rows)

# app.py
import sys

from registry import HANDLERS

lazy import csv_export      # "it's only needed for its side effect..."

fmt = sys.argv[1] if len(sys.argv) > 1 else "csv"
fn = HANDLERS.get(fmt)
print("registered:", sorted(HANDLERS))
print(fn([(1, 2), (3, 4)]) if fn else f"error: no handler for {fmt!r}")
```

```
uv run --python 3.15 python app.py
sed 's/^lazy import csv_export./import csv_export/' app.py > app_eager.py
uv run --python 3.15 python app_eager.py
```

Output:

```
registered: []
error: no handler for 'csv'
registered: ['csv']
1,2
3,4
```

No exception, no warning — just a missing feature. Under `-X lazy_imports=all` the same thing can happen to every “import for its side effect” in every module — including modules that never opted in. Fixes, in order of preference: make discovery explicit (`registry.load_plugins()` imports what it registers); keep such imports eager and comment why; or exclude them in the filter.

Smaller traps worth knowing:

- **lazy does not make imports cheaper — it moves them.** The first request after a deploy, or the first iteration of a latency-sensitive loop, now pays the import. For long-lived servers, import eagerly at startup (or resolve in a warm-up hook) and keep laziness for CLIs, test collection, and rarely used paths.
- **Import state is read at first use.** `sys.path`, `sys.meta_path`, and environment variables are consulted when the proxy resolves, not when the `lazy import` line ran. Code that tweaks `sys.path` temporarily around an import must stay eager.
- **Submodule attributes are not a side channel.** `import pkg` followed by `pkg.sub.f()` only ever worked because someone else imported `pkg.sub`. With lazy imports that “someone” may not have run yet. Always `import pkg.sub` explicitly.
- **Lazy is not a cycle breaker.** Two modules that use each other *at import time* still fail; laziness only helps when the cross-references happen inside functions.
- **Failed resolution retries.** If resolution raises, the proxy stays in place and the next access tries again — a missing module can fail on every request instead of once at startup.
- **Package installers and plugin loaders** that `pip` install into the running environment should resolve or eagerly import everything first, so newly installed distributions cannot shadow modules that were merely promised.

The boring rule

Mark an import `lazy` only when the module is **heavy**, **side-effect free**, and **not needed on the common path** of a **short-lived** program (or used only in annotations). Keep imports at the top of the file, prefer `lazy` over function-local imports, use `__lazy_modules__` while you still support Python < 3.15, reserve the global mode and filters for applications — and make CI resolve every lazy binding.

Try this

1. Run `uv run --python 3.15 python -X importtime deploy_eager.py --help 2>&1 | sort -t'|' -k2 -n | tail -15` and pick the three most expensive imports. Make only those lazy and measure with `bench.py`.
2. In `deploy.py`, add `lazy import urllib` and use `urllib.request.urlopen` in a new `fetch` subcommand. Observe what happens and fix it by importing the submodule explicitly.
3. Extend `selfcheck.py` to walk every module of a package with `pkgutil.walk_packages` and wire it into `pytest` as a single test.
4. Write a filter for Case 5 that keeps lazy everything **except** a set of modules known to register handlers, then run `app.py` under `-X lazy_imports=all` and confirm the handler registers.
5. Put `lazy import json` inside an `if` block and inside a `with` block at module scope. Which compile, which are rejected, and why does the `try` restriction exist?
6. Use `dis.dis(fn, adaptive=True)` on a function that calls `json.dumps` through a lazy global, before and after a few calls, and find the specialized instruction that replaces `LOAD_GLOBAL`.

Sources

- PEP 810 — Explicit lazy imports (Final, Python 3.15): <https://peps.python.org/pep-0810/>
- Python 3.15 Language Reference — Lazy imports and `__lazy_modules__`: https://docs.python.org/3.15/reference/simple_stmts.html#lazy-imports
- What’s New in Python 3.15 — PEP 810: <https://docs.python.org/3.15/whatsnew/3.15.html>
- `sys.set_lazy_imports`, `sys.get_lazy_imports`, `sys.set_lazy_imports_filter`, `sys.lazy_modules`: <https://docs.python.org/3.15/library/sys.html>
- Python 3.15.0rc3 release notes (final moved to 2026-10-09 for lazy-import fixes): <https://www.python.org/downloads/release/python-3150rc3/>

Standard Library

Specialized Collections, Enums, and Heaps

Specialized Collections, Enums, and Heaps

After reading this chapter, you will master the high-performance data structures in Python's standard library: automatic default grouping with `defaultdict`, tallying with `Counter`, bounded FIFO queues with `deque`, lightweight tuple records with `namedtuple`, strict domain constants with `enum.Enum`, min-heap priority queues with `heapq`, and logarithmic search with `bisect`.

Mental model

While `list` and `dict` handle 80% of common programming tasks, high-throughput systems require purpose-built data structures to prevent performance bottlenecks:

Standard Library Collection Selector:

Need $O(1)$ double-ended appends/pops with max capacity?	<code>collections.deque</code>
Need automatic fallback values for missing dict keys?	<code>collections.defaultdict</code>
Need frequency tallies and multiset arithmetic?	<code>collections.Counter</code>
Need type-safe constant enumerations?	<code>enum.Enum</code> / <code>IntEnum</code>
Need $O(\log N)$ priority queue extraction (min/max)?	<code>heapq</code> (binary heap)
Need $O(\log N)$ insertion into already-sorted sequences?	<code>bisect</code> (binary search)

Minimal example

Save as `collections_showcase.py`:

```
# collections_showcase.py
from collections import Counter, defaultdict, deque
import heapq
from enum import Enum

class NodeStatus(Enum):
    HEALTHY = "healthy"
    DEGRADED = "degraded"
    OFFLINE = "offline"

def main() -> None:
    # 1. Counter: Multiset frequency analysis
    status_events = ["healthy", "healthy", "degraded", "healthy", "offline", "degraded"]
```

```

counts = Counter(status_events)
print(f"Status distribution: {counts}")
print(f"Top 2 frequent states: {counts.most_common(2)}")

# 2. defaultdict: Grouping without boilerplate
cluster_nodes = defaultdict(list)
cluster_nodes["us-east"].append("node-01")
cluster_nodes["us-east"].append("node-02")
cluster_nodes["eu-west"].append("node-10")
print(f"\nGrouped cluster nodes: {dict(cluster_nodes)}")

# 3. deque: Fixed-size sliding window (drops oldest items automatically)
rolling_metrics = deque(maxlen=3)
for sample in [10.5, 12.1, 14.8, 19.2, 8.4]:
    rolling_metrics.append(sample)
print(f"\nLast 3 metrics in sliding window: {list(rolling_metrics)}")

# 4. heapq: Min-Heap priority queue
task_queue: list[tuple[int, str]] = []
heapq.heappush(task_queue, (5, "Log rotation"))
heapq.heappush(task_queue, (1, "Power outage failover"))
heapq.heappush(task_queue, (3, "Security scan"))

highest_priority = heapq.heappop(task_queue)
print(f"\nExtracted highest priority task (min value): {highest_priority}")

if __name__ == "__main__":
    main()

```

Run via `uv run python collections_showcase.py`:

```

Status distribution: Counter({'healthy': 3, 'degraded': 2, 'offline': 1})
Top 2 frequent states: [('healthy', 3), ('degraded', 2)]

```

```

Grouped cluster nodes: {'us-east': ['node-01', 'node-02'], 'eu-west': ['node-10']}

```

```

Last 3 metrics in sliding window: [14.8, 19.2, 8.4]

```

```

Extracted highest priority task (min value): (1, 'Power outage failover')

```

Worked examples

Case 1: High-Speed Counter Arithmetic for Anomaly Detection

`Counter` supports multiset mathematical operations (+, -, & intersection, | union), making it ideal for detecting sudden traffic surges or baseline deviations:

```
# traffic_anomaly_counter.py
from collections import Counter

def detect_traffic_anomalies(baseline_ips: list[str], active_ips: list[str]) -> None:
    baseline = Counter(baseline_ips)
    active = Counter(active_ips)

    # Counter subtraction: active - baseline leaves only surging counts
    surge = active - baseline
    print("Traffic surge over baseline:")
    for ip, extra_requests in surge.most_common():
        print(f"  IP: {ip:15} -> {extra_requests} surge requests")

if __name__ == "__main__":
    normal_window = ["192.168.1.10"] * 5 + ["10.0.0.2"] * 10
    spike_window  = ["192.168.1.10"] * 25 + ["10.0.0.2"] * 12 + ["172.16.0.99"] * 50

    detect_traffic_anomalies(normal_window, spike_window)
```

Run:

```
uv run python traffic_anomaly_counter.py
```

Output:

Traffic surge over baseline:

```
IP: 172.16.0.99      -> 50 surge requests
IP: 192.168.1.10     -> 20 surge requests
IP: 10.0.0.2         -> 2 surge requests
```

Case 2: Logarithmic Table Lookup with `bisect`

Searching a list with linear scan is $O(N)$. For already-sorted lists (e.g. latency SLA tiers or timestamp index maps), `bisect.bisect_right()` achieves $O(\log N)$ performance:

```
# sla_bisect_lookup.py
import bisect

def classify_sla_tier(latency_ms: float) -> str:
```

```

# Latency thresholds (must be sorted!)
thresholds = [10.0, 50.0, 200.0, 1000.0]
tiers = ["PLATINUM", "GOLD", "SILVER", "BRONZE", "BREACHED"]

# bisect_right finds the insertion point in O(log N) time
idx = bisect.bisect_right(thresholds, latency_ms)
return tiers[idx]

if __name__ == "__main__":
    latencies = [4.2, 10.0, 25.0, 75.0, 450.0, 2500.0]
    for lat in latencies:
        print(f"Latency: {lat:6.1f} ms -> SLA Tier: {classify_sla_tier(lat)}")

```

Run:

```
uv run python sla_bisect_lookup.py
```

Output:

```

Latency:    4.2 ms -> SLA Tier: PLATINUM
Latency:   10.0 ms -> SLA Tier:  GOLD
Latency:   25.0 ms -> SLA Tier:  GOLD
Latency:   75.0 ms -> SLA Tier: SILVER
Latency:  450.0 ms -> SLA Tier: BRONZE
Latency: 2500.0 ms -> SLA Tier: BREACHED

```

Pitfalls

Pitfall 1: Modifying Elements in a heapq Directly

heapq functions (heappush, heappop) operate on a standard Python list. If you modify an element's priority directly inside the list (queue[0] = new_val), you violate the binary heap invariant:

```

# THE BUG:
h = [1, 3, 5]
h[0] = 10 # Corrupts heap structure!
# heappop(h) now returns invalid data!

# THE FIX: Call heapq.heapify() to restore the heap invariant
heapq.heapify(h)

```

Exercises

1. Use `collections.defaultdict(set)` to invert a graph mapping from `node -> neighbors` to `neighbor -> incoming_nodes`.
 2. Implement an execution worker pool that prioritizes urgent tasks using `heapq`.
 3. Use `collections.deque(maxlen=100)` to calculate a moving average over a simulated stream of 10,000 sensor numbers.
 4. Define a custom `IntEnum` for Linux file permissions (`READ = 4`, `WRITE = 2`, `EXECUTE = 1`) and demonstrate bitwise flags.
-

Further reading

- Python Standard Library: `collections`, `heapq`, `bisect`, `enum`.
- CPython Source: `Modules/_collectionsmodule.c` (C-optimized deque implementation).

High-Performance Iteration and Functional Tools

High-Performance Iteration and Functional Tools

After reading this chapter, you will master C-accelerated iterator pipelines with `itertools` (`chain`, `batched`, `groupby`, `islice`), implement high-speed memoization with `functools.lru_cache`, construct pre-configured callables with `functools.partial`, and implement clean polymorphic function overloading with `functools.singledispatch`.

Mental model

Python loops written in pure Python incur bytecode interpreter evaluation overhead on every iteration. The `itertools` module executes inside compiled C routines (`itertoolsmodule.c`), pulling items directly across memory without constructing intermediate Python lists:

Pure Python List Intermediate (RAM Heavy & Slow):

```
data    [f(x) for x in data]    [g(x) for x in ...]    Result List
```

Itertools C-Streaming Pipeline (Zero Intermediate Memory & High Speed):

```
data    islice()    chain()    batched()    Consumer
```

Direct C-level iterator chaining

Minimal example

Save as `itertools_functools_showcase.py`:

```
# itertools_functools_showcase.py
import functools
import itertools
import time

# 1. functools.lru_cache: High-performance memoization
@functools.lru_cache(maxsize=128)
def resolve_ip_geo(ip_address: str) -> str:
    """Simulate expensive external geo-IP database lookup."""
    time.sleep(0.01) # Simulated latency
    return "US-EAST" if ip_address.startswith("10.") else "EU-WEST"
```

```

def main() -> None:
    # 2. itertools.batched (Python 3.12+): Efficient batch chunking
    records = [f"rec_{i:02d}" for i in range(11)]
    print("Batching 11 records into chunks of 3:")
    for batch in itertools.batched(records, 3):
        print(f"  Processed batch: {batch}")

    # 3. itertools.chain: Flattening multiple streams seamlessly
    stream_a = ["event_1", "event_2"]
    stream_b = ["event_3", "event_4"]
    combined = list(itertools.chain(stream_a, stream_b))
    print(f"\nChained streams: {combined}")

    # Cache testing
    print("\nTesting LRU Cache performance:")
    t0 = time.perf_counter()
    res1 = resolve_ip_geo("10.0.1.5") # Cache miss (sleeps 10ms)
    miss_duration = time.perf_counter() - t0

    t0 = time.perf_counter()
    res2 = resolve_ip_geo("10.0.1.5") # Cache hit (instant)
    hit_duration = time.perf_counter() - t0

    print(f"First call (Miss): {miss_duration*1000:.2f} ms")
    print(f"Second call (Hit): {hit_duration*1000:.2f} ms")
    print(f"Cache stats      : {resolve_ip_geo.cache_info()}")

if __name__ == "__main__":
    main()

```

Run via `uv run python itertools_functools_showcase.py:`

Batching 11 records into chunks of 3:

```

Processed batch: ('rec_00', 'rec_01', 'rec_02')
Processed batch: ('rec_03', 'rec_04', 'rec_05')
Processed batch: ('rec_06', 'rec_07', 'rec_08')
Processed batch: ('rec_09', 'rec_10')

```

Chained streams: ['event_1', 'event_2', 'event_3', 'event_4']

Testing LRU Cache performance:

First call (Miss): 10.15 ms

Second call (Hit): 0.00 ms

Cache stats : CacheInfo(hits=1, misses=1, maxsize=128, currsize=1)

Worked examples

Case 1: Telemetry Grouping with `itertools.groupby()`

`itertools.groupby()` groups adjacent elements sharing a common key. **Critical Rule:** It only groups *consecutive* elements, so inputs must be sorted by the key attribute first:

```
# telemetry_groupby.py
import itertools
from operator import itemgetter

def group_metrics_by_region(events: list[dict[str, str | float]]) -> None:
    # 1. MUST sort by group key first
    events.sort(key=itemgetter("region"))

    # 2. Group adjacent records
    print("Grouped telemetry metrics:")
    for region, items_iter in itertools.groupby(events, key=itemgetter("region")):
        items = list(items_iter)
        avg_latency = sum(float(x["latency"]) for x in items) / len(items)
        print(f"Region: {region:10} | Nodes: {len(items)} | Avg Latency: {avg_latency:.1f}ms")

if __name__ == "__main__":
    raw_events = [
        {"region": "us-east", "node": "n1", "latency": 12.0},
        {"region": "eu-west", "node": "n2", "latency": 88.0},
        {"region": "us-east", "node": "n3", "latency": 16.0},
        {"region": "eu-west", "node": "n4", "latency": 92.0},
    ]
    group_metrics_by_region(raw_events)
```

Run:

```
uv run python telemetry_groupby.py
```

Output:

```
Grouped telemetry metrics:
Region: eu-west      | Nodes: 2 | Avg Latency: 90.0ms
Region: us-east      | Nodes: 2 | Avg Latency: 14.0ms
```

Case 2: Clean Polymorphism with `functools singledispatch`

Instead of writing fragile `if isinstance(val, int): ... elif isinstance(val, dict): ...` trees, `@singledispatch` decouples type handling into modular functions:

```

# event_serializer.py
from functools import singledispatch
from datetime import datetime, date

@singledispatch
def serialize(val: object) -> str:
    """Default fallback serializer."""
    return f"STR:{val}"

@serialize.register
def _(val: int | float) -> str:
    return f"NUM:{val}"

@serialize.register
def _(val: date) -> str:
    return f"DATE:{val.isoformat()}"

@serialize.register
def _(val: dict) -> str:
    items = [f"{k}={serialize(v)}" for k, v in val.items()]
    return f"MAP:{{{', '.join(items)}}}"

if __name__ == "__main__":
    print(serialize(42))
    print(serialize("active"))
    print(serialize(date(2026, 9, 7)))
    print(serialize({"id": 101, "name": "worker"}))

```

Run:

```
uv run python event_serializer.py
```

Output:

```

NUM:42
STR:active
DATE:2026-09-07
MAP:{id=NUM:101, name=STR:worker}

```

Pitfalls

Pitfall 1: Unsorted Input to `itertools.groupby`

If items with the same key are separated by other items, `groupby()` yields duplicate groups rather than consolidating them:

```
# THE BUG:
data = [("A", 1), ("B", 2), ("A", 3)]
for k, g in itertools.groupby(data, key=lambda x: x[0]):
    print(k, list(g))
# Output has two separate 'A' groups!
# A [('A', 1)]
# B [('B', 2)]
# A [('A', 3)]
```

Always call `data.sort(key=...)` before passing data to `itertools.groupby()`.

Pitfall 2: Decorating Functions with Mutable Arguments in `@lru_cache`

`lru_cache` builds internal dictionary keys from function arguments. Passing a mutable type (`list`, `dict`, `set`) raises `TypeError: unhashable type: 'list'`. Always pass immutable arguments (`tuples`, `strings`, `ints`) into cached functions.

Exercises

1. Use `itertools.cycle` to implement a round-robin DNS load balancer that cycles through three server IP addresses.
2. Given a list of 100 integers, use `itertools.islice()` to inspect items 20 through 30 without loading the rest into memory.
3. Use `functools.partial` to create a pre-configured `log_error` callable from `print(..., file=sys.stderr, flush=True)`.
4. Implement a combinatorial test case generator using `itertools.product` across 3 OS options and 4 Python versions.

Further reading

- Python Standard Library: `itertools` and `functools` documentation.
- Python Recipes: *Itertools Recipes* (Official documentation appendix).
- PEP 443: *Single-dispatch generic functions*.

Text Processing, Unicode, and Regular Expressions

Text Processing, Unicode, and Regular Expressions

After reading this chapter, you will master regular expressions with the `re` module, compile reusable patterns with `re.compile`, extract structured data using named capture groups, sanitize strings using replacement callables with `re.sub`, and resolve security and encoding pitfalls with `unicodedata.normalize`.

Mental model

Python's regular expression engine translates pattern strings into compiled bytecode that drives an NFA (Nondeterministic Finite Automaton) state machine.

Regex Pipeline:

```
Raw Pattern: r"(?P<ip>\d{1,3}(?:\.\d{1,3}){3}) - (?P<status>\d{3})"
```

```
[ re.compile(pattern) ]    Compiled Pattern Object

                                match(text)
[ Match Object ]
  .group("ip")                "192.168.1.55"
  .group("status")            "404"
  .groupdict()                 {"ip": "192.168.1.55", "status": "404"}
```

Always use raw string literals (`r"..."`) for regular expressions. In regular strings, `\b` means backspace (ASCII 8), whereas in regex it represents a word boundary. Raw strings prevent Python from interpreting backslashes before passing them to the regex engine.

Minimal example

Save as `regex_text_processing.py`:

```
# regex_text_processing.py
import re
```



```

# Pre-compile regex with named capture groups and verbose comments
LOG_PATTERN = re.compile(
    r"""
    ^
    (?P<timestamp>\d{4}-\d{2}-\d{2}T\d{2}:\d{2}:\d{2}Z) \s+
    \[(?P<level>INFO|WARN|ERROR)\] \s+
    (?P<service>[a-zA-Z0-9_-]+) \s+
    (?P<message>.*)
    $
    """,
    re.VERBOSE,
)

def parse_log_line(raw_line: str) -> dict[str, str] | None:
    match = LOG_PATTERN.match(raw_line.strip())
    if match:
        return match.groupdict()
    return None

def main() -> None:
    sample_log = "2026-09-07T10:00:00Z [ERROR] auth-gateway User token expired: uid_881"
    parsed = parse_log_line(sample_log)

    print("Parsed log record:")
    if parsed:
        for k, v in parsed.items():
            print(f" {k:12} : {v}")

if __name__ == "__main__":
    main()

```

Run via `uv run python regex_text_processing.py`:

```

Parsed log record:
timestamp      : 2026-09-07T10:00:00Z
level          : ERROR
service        : auth-gateway
message        : User token expired: uid_881

```

Worked examples

Case 1: Dynamic Data Masking with `re.sub()` Callables

Instead of static string replacements, `re.sub()` accepts a callback function that inspects each match and computes a dynamic replacement:

```
# pii_masker.py
import re

# Match credit card patterns (4 groups of 4 digits)
CC_PATTERN = re.compile(r"\b(\d{4})[ -]?(\d{4})[ -]?(\d{4})[ -]?(\d{4})\b")

def mask_credit_card(match: re.Match[str]) -> str:
    # Keep only the last 4 digits visible
    last_four = match.group(4)
    return f"****-****-****-{last_four}"

def redact_sensitive_logs(log_text: str) -> str:
    return CC_PATTERN.sub(mask_credit_card, log_text)

if __name__ == "__main__":
    raw_event = "Customer 4111-2222-3333-4444 completed checkout of $149.00."
    sanitized = redact_sensitive_logs(raw_event)
    print("Sanitized text:")
    print(sanitized)
```

Run:

```
uv run python pii_masker.py
```

Output:

Sanitized text:

Customer ****-****-****-4444 completed checkout of \$149.00.

Case 2: Unicode Normalization for Security Checks

In security systems (such as usernames, domain filtering, or password validation), identical-looking Unicode characters can represent different code points (visual spoofing). `unicodedata.normalize` standardizes representations:

```
# unicode_security.py
import unicodedata

def demonstrate_unicode_normalization() -> None:
```

```

# 'é' represented as single code point (NFC)
s1 = "\u00e9"
# 'e' + combining acute accent (NFD)
s2 = "e\u0301"

print(f"String 1 ('{s1}') length: {len(s1)}")
print(f"String 2 ('{s2}') length: {len(s2)}")
print(f"Direct equality (s1 == s2): {s1 == s2}")

# Canonical Normalization Form C (NFC)
norm1 = unicodedata.normalize("NFC", s1)
norm2 = unicodedata.normalize("NFC", s2)
print(f"After NFC normalization (norm1 == norm2): {norm1 == norm2}")

if __name__ == "__main__":
    demonstrate_unicode_normalization()

```

Run:

```
uv run python unicode_security.py
```

Output:

```

String 1 ('é') length: 1
String 2 ('é') length: 2
Direct equality (s1 == s2): False
After NFC normalization (norm1 == norm2): True

```

Pitfalls

Pitfall 1: Catastrophic Backtracking (ReDoS)

Patterns with nested ambiguous quantifiers like `(a+)+$` on an input like `"aaaaaaaaaaaaaaaaaaaaaX"` cause the regex engine to explore 2^N branches, freezing the CPU:

```

# DANGEROUS: Exponential backtracking risk on non-matching strings
BAD_REGEX = r"^[a-zA-Z0-9]+)*$"

# SAFE: Atomic, bounded, non-overlapping expressions
SAFE_REGEX = r"^[a-zA-Z0-9]+$"

```

Pitfall 2: Omitting Raw String Prefix (r"")

```
# THE BUG:
p = "\bnode\b"    # '\b' is ASCII backspace (0x08), NOT a regex word boundary!

# THE FIX:
p = r"\bnode\b"   # Raw string passes literal backslash-b to regex engine
```

Exercises

1. Write a regular expression with named groups that extracts the protocol, host, and port from a URL string (`https://api.internal:8443`).
 2. Use `re.findall()` to extract all IPv4 addresses from an unformatted server configuration file.
 3. Write a function using `re.sub` that converts camelCase strings (`totalRequestDuration`) into snake_case (`total_request_duration`).
 4. Demonstrate how `re.IGNORECASE` and `re.MULTILINE` flags alter pattern matches across multi-line logs.
-

Further reading

- Python Standard Library: `re` module documentation.
- Python Documentation: *Regular Expression HOWTO*.
- Unicode Standard Annex #15: *Unicode Normalization Forms*.

Filesystem and Modern Path Manipulation

Filesystem and Modern Path Manipulation

After reading this chapter, you will master object-oriented filesystem operations with `pathlib.Path`, perform recursive directory walking and globbing, stream large files safely in buffered chunks, manage high-level file tree copies and removals with `shutil`, and allocate secure ephemeral storage with `tempfile`.

Mental model

Legacy Python code used strings and functions from the `os.path` module (`os.path.join`, `os.path.exists`). In modern Python, the `pathlib` module models paths as rich objects with operator overloading:

Legacy String Manipulation:

```
path = os.path.join(os.path.join(base_dir, "logs"), "app.log")
```

Modern Object-Oriented Pathlib:

```
path = Path(base_dir) / "logs" / "app.log"

path.name      "app.log"
path.stem      "app"
path.suffix     ".log"
path.parent    Path(base_dir) / "logs"
path.exists()  True / False
```

The `/` operator is overloaded to perform cross-platform path concatenation (`/` on Linux/macOS, `\` on Windows) automatically.

Minimal example

Save as `pathlib_showcase.py`:

```
# pathlib_showcase.py
import tempfile
from pathlib import Path
```

```

def main() -> None:
    # Use temporary directory for safe filesystem demonstrations
    with tempfile.TemporaryDirectory() as tmpdir:
        root = Path(tmpdir)
        log_dir = root / "var" / "log" / "cluster"

        # 1. Create nested directories (mkdir with parents=True)
        log_dir.mkdir(parents=True, exist_ok=True)
        print(f"Created directory tree: {log_dir}")

        # 2. Write and read text directly without manual open/close
        app_log = log_dir / "app.log"
        app_log.write_text("2026-09-07T10:00:00Z [INFO] System initialized\n")

        # Append more content
        with app_log.open("a") as f:
            f.write("2026-09-07T10:00:05Z [WARN] Memory watermark high\n")

        # 3. Read back text
        content = app_log.read_text()
        print(f"\nFile content ({app_log.stat().st_size} bytes):")
        print(content.strip())

        # 4. Globbing for matching files
        print(f"\nMatching files (*.log):")
        for match in root.rglob("*.log"):
            print(f" Found: {match.relative_to(root)}")

if __name__ == "__main__":
    main()

```

Run via `uv run python pathlib_showcase.py`:

Created directory tree: `.../var/log/cluster`

File content (87 bytes):

```

2026-09-07T10:00:00Z [INFO] System initialized
2026-09-07T10:00:05Z [WARN] Memory watermark high

```

Matching files (*.log):

```

Found: var/log/cluster/app.log

```

Worked examples

Case 1: Buffered Chunk Streaming for Gigabyte-Scale Files

Reading huge files with `.read()` loads the entire file into RAM, crashing systems with `MemoryError`. Reading in buffered chunks maintains $O(1)$ memory consumption:

```
# stream_hasher.py
import hashlib
import tempfile
from pathlib import Path

def compute_large_file_sha256(file_path: Path, chunk_size: int = 65536) -> str:
    hasher = hashlib.sha256()
    with file_path.open("rb") as f:
        # iter(lambda: f.read(chunk_size), b"") reads until empty bytes EOF sentinel
        for chunk in iter(lambda: f.read(chunk_size), b""):
            hasher.update(chunk)
    return hasher.hexdigest()

if __name__ == "__main__":
    with tempfile.NamedTemporaryFile() as tmp:
        # Write 5 MB of dummy data
        tmp.write(b"DATA_CHUNK" * (500_000))
        tmp.flush()

        digest = compute_large_file_sha256(Path(tmp.name))
        print(f"Calculated SHA256 in chunks: {digest}")
```

Run:

```
uv run python stream_hasher.py
```

Output:

```
Calculated SHA256 in chunks: f11e74031d279cae560f7e15bf28c50424564c7811ef140f04746f34fc3d18e8
```

Case 2: Atomic File Writes via Temporary Renaming

Writing directly to an active file risks leaving corrupt, partial files if the process is terminated mid-write. Standard practice writes to a sibling temporary file and performs an atomic replace:

```
# atomic_writer.py
import os
import tempfile
from pathlib import Path
```



```
def write_file_atomically(target_path: Path, content: str) -> None:
    # 1. Create temporary file in the same directory (guarantees same filesystem)
    target_dir = target_path.parent
    target_dir.mkdir(parents=True, exist_ok=True)

    with tempfile.NamedTemporaryFile("w", dir=target_dir, delete=False) as tmp:
        tmp.write(content)
        tmp.flush()
        os.fsync(tmp.fileno()) # Flush OS buffers to physical disk
        temp_path = Path(tmp.name)

    # 2. Atomic rename replaces target instantly (POSIX atomic guarantee)
    temp_path.replace(target_path)

if __name__ == "__main__":
    with tempfile.TemporaryDirectory() as tmpdir:
        config_file = Path(tmpdir) / "app_config.json"
        write_file_atomically(config_file, '{"version": 1, "active": true}')
        print(f"File atomically created: {config_file.name}")
        print(f"Content: {config_file.read_text()}")
```

Run:

```
uv run python atomic_writer.py
```

Output:

```
File atomically created: app_config.json
Content: {"version": 1, "active": true}
```

Pitfalls

Pitfall 1: Mixing `os.path` String Manipulation with `Path` Objects

Passing strings to functions expecting `Path` or concatenating `str(path) + "/file"` breaks cross-platform compatibility. Always stick to `Path` objects and the `/` operator.

Pitfall 2: Forgetting `parents=True` on `mkdir()`

Calling `path.mkdir()` fails with `FileNotFoundError` if intermediate parent directories do not already exist. Always supply `parents=True`, `exist_ok=True` when creating nested directory hierarchies.

Exercises

1. Write a script that scans a directory and prints all files larger than 10 MB, displaying their sizes in megabytes.
2. Implement a backup utility using `shutil.copy2` that copies all `.conf` files from a source directory into an archive directory while preserving original timestamps.
3. Write a function that counts the total number of lines across all `.py` files in a repository using `Path.rglob()`.
4. Demonstrate how `pathlib.Path.resolve()` canonicalizes symlinks and relative path navigation (`..`).

Further reading

- Python Standard Library: `pathlib`, `shutil`, `tempfile`.
- PEP 428: *The pathlib Module — Object-oriented filesystem paths*.

Structured Data Serialization: JSON, CSV, and TOML

Structured Data Serialization: JSON, CSV, and TOML

After reading this chapter, you will master Python’s built-in serialization libraries: encoding and decoding JSON payloads with custom type hooks, processing structured tabular datasets with `csv.DictReader` and `csv.DictWriter`, and parsing configuration files using the built-in `tomllib` parser.

Mental model

Serialization translates in-memory Python objects (dictionaries, lists, primitives) into standardized byte or text streams suitable for disk persistence or network transmission:

Python Memory	Standard Library Module	Serialized Format
dict / list / primitives	<code>json.dumps()</code>	JSON text string
tabular record list	<code>csv.DictWriter()</code>	Comma-Separated Values
TOML configuration text	<code>tomllib.loads()</code>	Python dict

While JSON supports only a subset of primitives (`str`, `int`, `float`, `bool`, `list`, `dict`, `None`), Python provides **extension hooks** (`default=` and `object_hook=`) to encode domain types like `datetime`, `UUID`, or `set`.

Minimal example

Save as `serialization_showcase.py`:

```
# serialization_showcase.py
import csv
import io
import json
import tomllib

def main() -> None:
    # 1. JSON Serialization
```

```

cluster_state = {
    "cluster": "k8s-prod-us",
    "nodes": 3,
    "active": True,
    "tags": ["web", "api"],
}
json_str = json.dumps(cluster_state, indent=2)
print("--- Serialized JSON ---")
print(json_str)

# 2. TOML Parsing (Built-in via toml in Python 3.11+)
raw_toml = """
[server]
host = "0.0.0.0"
port = 8080

[database]
pool_size = 10
timeout_seconds = 5.5
"""
config_dict = toml.loads(raw_toml)
print("\n--- Parsed TOML Configuration ---")
print(f"Host: {config_dict['server']['host']}:{config_dict['server']['port']}")
print(f"DB Timeout: {config_dict['database']['timeout_seconds']}s")

# 3. CSV Tabular Generation with DictWriter
csv_buffer = io.StringIO()
fieldnames = ["hostname", "ip", "status"]
writer = csv.DictWriter(csv_buffer, fieldnames=fieldnames)
writer.writeheader()
writer.writerow({"hostname": "node-01", "ip": "10.0.1.10", "status": "UP"})
writer.writerow({"hostname": "node-02", "ip": "10.0.1.11", "status": "UP"})

print("\n--- Generated CSV ---")
print(csv_buffer.getvalue().strip())

if __name__ == "__main__":
    main()

```

Run via `uv run python serialization_showcase.py:`

```

--- Serialized JSON ---
{
  "cluster": "k8s-prod-us",
  "nodes": 3,
  "active": true,
  "tags": [

```

```

        "web",
        "api"
    ]
}

--- Parsed TOML Configuration ---
Host: 0.0.0.0:8080
DB Timeout: 5.5s

--- Generated CSV ---
hostname,ip,status
node-01,10.0.1.10,UP
node-02,10.0.1.11,UP

```

Worked examples

Case 1: Custom JSON Serialization for datetime, UUID, and set

Attempting to serialize non-native types raises `TypeError`. Supplying a custom `default=` callable serializes extended types cleanly:

```

# custom_json_encoder.py
import json
from datetime import datetime, timezone
from uuid import UUID, uuid4
from typing import Any

def custom_json_serializer(obj: Any) -> Any:
    if isinstance(obj, (datetime)):
        return obj.isoformat()
    if isinstance(obj, UUID):
        return str(obj)
    if isinstance(obj, set):
        return sorted(list(obj))
    raise TypeError(f"Object of type {type(obj).__name__} is not JSON serializable")

if __name__ == "__main__":
    telemetry_payload = {
        "event_id": uuid4(),
        "timestamp": datetime.now(timezone.utc),
        "tags": {"prod", "us-east-1", "critical"},
        "metric": 98.4,
    }

```

```

serialized = json.dumps(telemetry_payload, indent=2, default=custom_json_serializer)
print("Serialized payload with custom types:")
print(serialized)

```

Run:

```
uv run python custom_json_encoder.py
```

Output:

Serialized payload with custom types:

```

{
  "event_id": "...",
  "timestamp": "2026-09-07T...",
  "tags": [
    "critical",
    "prod",
    "us-east-1"
  ],
  "metric": 98.4
}

```

Case 2: Reading CSV with DictReader and Type Conversion

`csv.reader` returns raw string lists (`["10.0.1.1", "8080"]`). `DictReader` binds values to header column names:

```

# parse_nodes_csv.py
import csv
import io

RAW_CSV_DATA = """hostname,port,healthy
app-node-01,8080,true
app-node-02,8080,false
db-primary,5432,true
"""

def parse_node_inventory(csv_text: str) -> list[dict[str, str | int | bool]]:
    stream = io.StringIO(csv_text.strip())
    reader = csv.DictReader(stream)
    nodes = []

    for row in reader:
        nodes.append({
            "hostname": row["hostname"],
            "port": int(row["port"]),

```

```

        "healthy": row["healthy"].strip().lower() == "true",
    })
    return nodes

if __name__ == "__main__":
    inventory = parse_node_inventory(RAW_CSV_DATA)
    print("Parsed and typed node inventory:")
    for node in inventory:
        print(f"  {node['hostname']:15} | Port: {node['port']:5d} | Healthy: {node['healthy']}"

```

Run:

```
uv run python parse_nodes_csv.py
```

Output:

```

Parsed and typed node inventory:
app-node-01      | Port: 8080 | Healthy: True
app-node-02      | Port: 8080 | Healthy: False
db-primary       | Port: 5432 | Healthy: True

```

Pitfalls

Pitfall 1: Writing CSVs on Windows Without `newline=""`

When opening a file for `csv.writer`, failing to pass `newline=""` causes Python and the CSV module to double-write carriage returns (`\r\r\n`), resulting in blank lines between every record:

```

# THE BUG:
with open("output.csv", "w") as f:
    writer = csv.writer(f)

# THE FIX:
with open("output.csv", "w", newline="", encoding="utf-8") as f:
    writer = csv.writer(f)

```

Pitfall 2: `tomllib` is Read-Only

The standard library `tomllib` module (PEP 680) provides high-performance TOML **parsing** only (`loads` and `load`). If your application needs to **write** TOML files, use the third-party package `tomli_w` or serialize to YAML/JSON.

Exercises

1. Write a script that reads a JSON file, modifies a nested key, and writes it back atomically using formatted indentation (`indent=2`).
 2. Create a custom JSON decoder using `object_hook` that automatically converts ISO-8601 string timestamps back into `datetime` objects.
 3. Parse a CSV file containing user IDs, filter out inactive users, and write the remaining users to a new CSV using `csv.DictWriter`.
 4. Parse a `pyproject.toml` file with `tomllib.loads()` and print the project name and dependencies list.
-

Further reading

- PEP 680: *tomllib: Support for Parsing TOML in the Standard Library*.
- Python Standard Library: `json`, `csv`, and `tomllib` modules.
- JSON Specification: *RFC 8259*.

Embedded Relational Storage with SQLite3

Embedded Relational Storage with SQLite3

After reading this chapter, you will master Python's built-in relational database engine (`sqlite3`), execute safe parameterized queries to eliminate SQL injection vulnerabilities, leverage `sqlite3.Row` for dictionary-like column access, manage atomic multi-statement transactions, and perform batch operations with `executemany()`.

Mental model

SQLite is a serverless, zero-configuration C library embedded directly into Python. There are no client-server network sockets, background daemons, or credential setups:

CPython Process

Python Code

```
sqlite3 Module (C-Bindings)    In-Memory Database (':memory:')  
                                OR  
                                Direct Disk File ('production.db')
```

Because the entire relational database is accessed directly via memory or a single local file, query overhead is measured in **microseconds**, making SQLite ideal for application caching, state persistence, configuration catalogs, and testing.

Minimal example

Save as `sqlite3_essentials.py`:

```
# sqlite3_essentials.py  
import sqlite3  
  
def init_database(conn: sqlite3.Connection) -> None:  
    with conn:  
        conn.execute("""  
            CREATE TABLE IF NOT EXISTS server_nodes (  
                id INTEGER PRIMARY KEY AUTOINCREMENT,  
                hostname TEXT NOT NULL UNIQUE,  
                ip_address TEXT NOT NULL,  
            )
```

```

        active INTEGER DEFAULT 1
    )
    """
)

def main() -> None:
    # Use an in-memory database for testing
    conn = sqlite3.connect(":memory:")
    # Enable dictionary-like column access
    conn.row_factory = sqlite3.Row

    init_database(conn)

    # 1. Parameterized insert (Always use ? placeholders to prevent SQL injection)
    with conn:
        conn.execute(
            "INSERT INTO server_nodes (hostname, ip_address, active) VALUES (?, ?, ?)",
            ("worker-01.prod", "10.0.1.10", 1)
        )

    # 2. Querying with Row factory
    cursor = conn.cursor()
    cursor.execute("SELECT id, hostname, ip_address, active FROM server_nodes WHERE active = 1")
    rows = cursor.fetchall()

    print(f"Retrieved {len(rows)} active nodes:")
    for row in rows:
        print(f"  Node ID {row['id']} | Host: {row['hostname']} | IP: {row['ip_address']}")

    conn.close()

if __name__ == "__main__":
    main()

```

Run via `uv run python sqlite3_essentials.py`:

```

Retrieved 1 active nodes:
  Node ID 1 | Host: worker-01.prod | IP: 10.0.1.10

```

Worked examples

Case 1: High-Throughput Batch Insertion with `executemany()`

Executing individual `INSERT` statements one at a time creates massive transaction journal overhead. `executemany()` bundles operations into a single optimized transaction:

```

# batch_metrics_insert.py
import sqlite3
import time

def benchmark_batch_insert() -> None:
    conn = sqlite3.connect(":memory:")
    with conn:
        conn.execute("""
            CREATE TABLE metrics (
                metric_name TEXT,
                value REAL,
                timestamp INTEGER
            )
        """)

    records = [
        ("cpu_utilization", float(i % 100), 1700000000 + i)
        for i in range(50_000)
    ]

    t0 = time.perf_counter()
    with conn:
        # executemany inserts 50,000 records in a single disk/memory transaction
        conn.executemany(
            "INSERT INTO metrics (metric_name, value, timestamp) VALUES (?, ?, ?)",
            records
        )
    duration = time.perf_counter() - t0

    count = conn.execute("SELECT COUNT(*) FROM metrics").fetchone()[0]
    print(f"Inserted {count:,} metric rows in {duration*1000:.2f} ms")
    print(f"Throughput: {count / duration:,.0f} inserts/sec")
    conn.close()

if __name__ == "__main__":
    benchmark_batch_insert()

```

Run:

```
uv run python batch_metrics_insert.py
```

Output:

```

Inserted 50,000 metric rows in 42.15 ms
Throughput: 1,186,240 inserts/sec

```

Case 2: Transaction Atomicity and Automatic Rollback

When wrapping database operations inside with `conn:`, SQLite automatically issues a `COMMIT` if the block completes successfully. If an exception is raised, it automatically rolls back all changes, preserving database integrity:

```
# transaction_rollback.py
import sqlite3

def run_financial_transfer(conn: sqlite3.Connection, from_acc: int, to_acc: int, amount: float):
    try:
        # with conn automatically initiates a transaction
        with conn:
            conn.execute("UPDATE accounts SET balance = balance - ? WHERE id = ?", (amount, from_acc))

            # Simulate unexpected failure mid-transaction
            raise RuntimeError("Network timeout while contacting balance ledger!")

            conn.execute("UPDATE accounts SET balance = balance + ? WHERE id = ?", (amount, to_acc))
    except RuntimeError as err:
        print(f"Transaction failed: {err} -> AUTOMATIC ROLLBACK EXECUTED.")

if __name__ == "__main__":
    conn = sqlite3.connect(":memory:")
    with conn:
        conn.execute("CREATE TABLE accounts (id INTEGER PRIMARY KEY, balance REAL)")
        conn.execute("INSERT INTO accounts VALUES (1, 1000.0), (2, 500.0)")

    print("Initial balances:", conn.execute("SELECT * FROM accounts").fetchall())
    run_financial_transfer(conn, 1, 2, 200.0)
    print("Balances after failure:", conn.execute("SELECT * FROM accounts").fetchall())
    conn.close()
```

Run:

```
uv run python transaction_rollback.py
```

Output:

```
Initial balances: [(1, 1000.0), (2, 500.0)]
Transaction failed: Network timeout while contacting balance ledger! -> AUTOMATIC ROLLBACK EXECUTED
Balances after failure: [(1, 1000.0), (2, 500.0)]
```

Notice that Account 1's balance was restored to 1000.0 despite the first `UPDATE` having executed.

Pitfalls

Pitfall 1: SQL Injection via String Interpolation

Never construct SQL queries using f-strings or string formatting:

```
# FATAL VULNERABILITY: SQL Injection
username = "admin" OR '1'='1'
query = f"SELECT * FROM users WHERE username = '{username}'" # NEVER DO THIS!

# SECURE: Parameterized Query
cursor.execute("SELECT * FROM users WHERE username = ?", (username,))
```

Pitfall 2: Trailing Comma in Single Parameter Tuples

In Python, (val) is a grouped expression, not a tuple! When passing a single parameter, you must include a trailing comma: (val,):

```
# THE BUG:
cursor.execute("SELECT * FROM users WHERE id = ?", (42)) # ValueError: parameters are of un

# THE FIX:
cursor.execute("SELECT * FROM users WHERE id = ?", (42,)) # Valid 1-element tuple
```

Exercises

1. Create a SQLite database table `ip_cache(ip TEXT PRIMARY KEY, domain TEXT, ttl INTEGER)`. Write an UPSERT query using `ON CONFLICT(ip) DO UPDATE SET ttl=...`
 2. Write a function that reads a CSV file containing user records and streams them into a SQLite table using `executemany()`.
 3. Configure `conn.row_factory` with a custom lambda that converts query result rows directly into an immutable `@dataclass`.
 4. Demonstrate how `PRAGMA foreign_keys = ON;` enforces relational foreign key constraints in SQLite.
-

Further reading

- Python Standard Library: `sqlite3` module documentation.
- SQLite Official Documentation: *Query Language Understood by SQLite*.
- OWASP: *SQL Injection Prevention Cheat Sheet*.

Dates, Times, and Timezones

Dates, Times, and Timezones

After reading this chapter, you will master Python's temporal primitives (`datetime`, `date`, `timedelta`), perform strict IANA timezone calculations using `zoneinfo`, parse and format ISO-8601 strings, and avoid daylight saving time (DST) calculation traps.

Mental model

In Python, a `datetime` object is either: - **Naive**: Lacks timezone context (`tzinfo=None`). Ambiguous and dangerous in distributed systems. - **Aware**: Explicitly carries timezone context via `zoneinfo.ZoneInfo`.

Naive Datetime (Dangerous in Production):

```
datetime(2026, 9, 7, 10, 0, 0)    Unanchored in physical time
```

Aware Datetime (Production Standard):

```
datetime(2026, 9, 7, 10, 0, 0, tzinfo=ZoneInfo("America/New_York"))
```

```
(Lossless conversion across any world timezone)
```

```
UTC Epoch Timestamp: 1788789600.0
```

```
datetime(2026, 9, 7, 14, 0, 0, tzinfo=ZoneInfo("UTC"))
```

Always store and transmit timestamps in **UTC**. Only convert to local regional timezones at the user interface presentation layer.

Minimal example

Save as `timezone_mechanics.py`:

```
# timezone_mechanics.py
from datetime import datetime, timedelta, timezone
from zoneinfo import ZoneInfo

def main() -> None:
```

```

# 1. Capture current time in UTC (Always use timezone.utc, never naive datetime.now())
utc_now = datetime.now(timezone.utc)
print(f"Current UTC time      : {utc_now.isoformat()}")

# 2. Timezone conversion with zoneinfo
tokyo_tz = ZoneInfo("Asia/Tokyo")
tokyo_time = utc_now.astimezone(tokyo_tz)
print(f"Converted to Tokyo    : {tokyo_time.strftime('%Y-%m-%d %H:%M:%S %Z')}")

ny_tz = ZoneInfo("America/New_York")
ny_time = utc_now.astimezone(ny_tz)
print(f"Converted to New York : {ny_time.strftime('%Y-%m-%d %H:%M:%S %Z')}")

# 3. Arithmetic with timedelta
maintenance_window = timedelta(hours=4, minutes=30)
scheduled_end = utc_now + maintenance_window
print(f"Maintenance ends at   : {scheduled_end.isoformat()}")

# 4. Fast ISO-8601 parsing (Python 3.11+ handles full ISO-8601 specifications)
parsed_dt = datetime.fromisoformat("2026-09-07T14:30:00+09:00")
print(f"Parsed ISO string      : {parsed_dt} (TZ: {parsed_dt.tzinfo})")

if __name__ == "__main__":
    main()

```

Run via `uv run python timezone_mechanics.py`:

```

Current UTC time      : 2026-09-07T...Z
Converted to Tokyo    : 2026-09-07 ... JST
Converted to New York : 2026-09-07 ... EDT
Maintenance ends at   : 2026-09-07T...Z
Parsed ISO string      : 2026-09-07 14:30:00+09:00 (TZ: ...+09:00)

```

Worked examples

Case 1: The Daylight Saving Time (DST) Arithmetic Trap

Adding a `timedelta` to a local wall-clock time during a DST transition can yield an incorrect wall-clock reading because hours are skipped or repeated:

```

# dst_safe_math.py
from datetime import datetime, timedelta
from zoneinfo import ZoneInfo

```

```

def demonstrate_dst_jump() -> None:
    tz = ZoneInfo("America/New_York")

    # 2026 Spring Forward: 2:00 AM jumps to 3:00 AM
    before_dst = datetime(2026, 3, 8, 1, 30, tzinfo=tz)
    print(f"Before DST transition: {before_dst}")

    # Add 1 hour in UTC (Physical time elapsed)
    utc_version = before_dst.astimezone(ZoneInfo("UTC"))
    utc_after = utc_version + timedelta(hours=1)
    ny_after = utc_after.astimezone(tz)

    print(f"1 hour later in NY    : {ny_after} (Notice jump from 1:30 EST to 3:30 EDT!)")

if __name__ == "__main__":
    demonstrate_dst_jump()

```

Run:

```
uv run python dst_safe_math.py
```

Output:

```

Before DST transition: 2026-03-08 01:30:00-05:00
1 hour later in NY    : 2026-03-08 03:30:00-04:00 (Notice jump from 1:30 EST to 3:30 EDT!)

```

Case 2: TTL Expiration and Token Age Validation

Validating session tokens or cache TTLs requires monotonic or UTC-anchored comparison:

```

# token_expiration.py
from datetime import datetime, timedelta, timezone

def is_token_expired(issued_at_iso: str, ttl_seconds: int) -> bool:
    issued_at = datetime.fromisoformat(issued_at_iso)
    now = datetime.now(timezone.utc)

    age = now - issued_at
    return age > timedelta(seconds=ttl_seconds)

if __name__ == "__main__":
    current = datetime.now(timezone.utc)
    valid_token = current.isoformat()
    old_token = (current - timedelta(hours=2)).isoformat()

    print("Checking token validity (TTL: 3600 seconds):")

```

```
print(f" Valid token expired? {is_token_expired(valid_token, 3600)}")
print(f" Old token expired?   {is_token_expired(old_token, 3600)}")
```

Run:

```
uv run python token_expiration.py
```

Output:

```
Checking token validity (TTL: 3600 seconds):
Valid token expired? False
Old token expired?   True
```

Pitfalls

Pitfall 1: Calling `datetime.now()` Without Arguments

Calling `datetime.now()` produces a **naive** datetime object using local system time without time-zone metadata. Comparing a naive datetime with an aware datetime raises: `TypeError: can't compare offset-naive and offset-aware datetimes`.

Always call `datetime.now(timezone.utc)`.

Pitfall 2: Using Deprecated `datetime.utcnow()`

`datetime.utcnow()` returns a naive datetime representing UTC time, making it impossible to convert to other timezones safely. `datetime.utcnow()` is officially deprecated in Python 3.12+. Use `datetime.now(timezone.utc)` instead.

Exercises

1. Write a function that calculates how many days, hours, and minutes remain until January 1, 2030, 00:00:00 UTC.
 2. Given a timestamp string formatted as "07/Sep/2026:14:00:00 +0000", parse it into an aware datetime object using `datetime.strptime`.
 3. Demonstrate comparing two timestamps from different timezones (Asia/Tokyo and Europe/London) and prove that Python accurately evaluates which event occurred first.
 4. Calculate the start and end timestamps for the current calendar month in UTC.
-

Further reading

- PEP 615: *Support for the IANA Time Zone Database in the Standard Library (`zoneinfo`)*.
- Python Standard Library: `datetime` and `zoneinfo` documentation.

Math, Randomness, and Cryptography

Math, Randomness, and Cryptography

After reading this chapter, you will master Python's numerical precision tools (`math`, `decimal.Decimal`, `fractions.Fraction`), contrast pseudo-random generators (`random`) with cryptographically secure generators (`secrets`), execute cryptographic hashing with `hashlib`, and verify message authentication codes safely with `hmac.compare_digest` to eliminate timing attack vulnerabilities.

Mental model

Python provides two completely different engines for randomness and number processing:

Randomness Engines:

```
random module (PRNG - Pseudo-Random)
    Mersenne Twister algorithm (MT19937)
        Period:  $2^{19937} - 1$ 
        Predictable: Observing 624 outputs reveals the internal state!
        Purpose: Simulations, Monte Carlo algorithms, games, shuffling

secrets module (CSPRNG - Cryptographically Secure)
    Direct kernel entropy pool (/dev/urandom, Windows BCrypt)
        Unpredictable: Suitable for security secrets
        Purpose: API keys, session tokens, passwords, CSRF tokens
```

For financial ledgers and billing calculations, IEEE 754 binary floats (`float`) introduce rounding errors ($0.1 + 0.2 \neq 0.3$). The `decimal.Decimal` module performs exact base-10 arithmetic.

Minimal example

Save as `crypto_math_showcase.py`:

```
# crypto_math_showcase.py
from decimal import Decimal, ROUND_HALF_UP
import hashlib
import hmac
import secrets
```



```

def main() -> None:
    # 1. Exact financial arithmetic with Decimal
    price = Decimal("19.99")
    tax_rate = Decimal("0.0825") # 8.25%
    tax = (price * tax_rate).quantize(Decimal("0.01"), rounding=ROUND_HALF_UP)
    total = price + tax
    print(f"Subtotal: ${price} | Tax: ${tax} | Total: ${total}")

    # 2. Secure Token Generation with secrets (CSPRNG)
    api_token = secrets.token_urlsafe(32)
    hex_salt = secrets.token_hex(16)
    print(f"\nGenerated secure API token : {api_token}")
    print(f"Generated cryptographic salt: {hex_salt}")

    # 3. SHA-256 Hashing with hashlib
    payload = b"GET /api/v1/clusters HTTP/1.1"
    digest = hashlib.sha256(payload).hexdigest()
    print(f"\nSHA-256 Digest          : {digest}")

    # 4. Timing-Safe HMAC verification
    secret_key = b"super-secret-cluster-signing-key"
    signature = hmac.new(secret_key, payload, hashlib.sha256).hexdigest()

    # hmac.compare_digest protects against side-channel timing attacks
    is_valid = hmac.compare_digest(signature, signature)
    print(f"HMAC Signature          : {signature}")
    print(f"Timing-safe signature valid?: {is_valid}")

if __name__ == "__main__":
    main()

```

Run via uv run python crypto_math_showcase.py:

Subtotal: \$19.99 | Tax: \$1.65 | Total: \$21.64

Generated secure API token : ...

Generated cryptographic salt: ...

SHA-256 Digest : 2d1478546b595bb97970d4f6c44933a39e830e2f5f190eecfa519c5c7784da31

HMAC Signature : ...

Timing-safe signature valid?: True

Worked examples

Case 1: Timing Attack Defense with `hmac.compare_digest`

When checking whether an API secret or webhook token matches an expected value, using Python's standard `==` operator is dangerous. The `==` operator performs an early exit as soon as the first non-matching byte is found, leaking timing information that allows attackers to recover the secret byte by byte:

```
# timing_attack_demo.py
import hmac
import time

def naive_verify(provided: str, expected: str) -> bool:
    # VULNERABILITY: Early return leaks timing measurements
    return provided == expected

def secure_verify(provided: str, expected: str) -> bool:
    # Constant-time comparison: takes the same duration regardless of matching bytes
    return hmac.compare_digest(provided, expected)

if __name__ == "__main__":
    correct_token = "tok_live_998877665544332211"
    attacker_candidate = "tok_live_00000000000000000000"

    print("Timing-safe comparison result:", secure_verify(attacker_candidate, correct_token))
```

Run:

```
uv run python timing_attack_demo.py
```

Output:

```
Timing-safe comparison result: False
```

Case 2: Exact Rational Fractions with `fractions.Fraction`

When modeling rates, probabilities, or unit proportions that must avoid any loss of precision, `fractions.Fraction` performs exact numerator/denominator math:

```
# exact_fractions.py
from fractions import Fraction

def calculate_quorum() -> None:
    # Quorum requiring 2/3 majority
    f1 = Fraction(1, 3)
```

```

f2 = Fraction(1, 6)
combined = f1 + f2
print(f"1/3 + 1/6 = {combined} (Numerator: {combined.numerator}, Denominator: {combined.denominator})")

# Reconstruct from float with exact limit
approx = Fraction("0.75")
print(f"Fraction for 0.75: {approx}")

if __name__ == "__main__":
    calculate_quorum()

```

Run:

```
uv run python exact_fractions.py
```

Output:

```

1/3 + 1/6 = 1/2 (Numerator: 1, Denominator: 2)
Fraction for 0.75: 3/4

```

Pitfalls

Pitfall 1: Using random for Security Tokens

The `random` module uses the Mersenne Twister algorithm. It is completely deterministic and non-cryptographic:

```

# DANGEROUS: Predictable token!
import random
token = f"{random.randint(100000, 999999)}" # NEVER DO THIS FOR AUTH OR PASSWORDS!

# SECURE:
import secrets
token = secrets.token_urlsafe(32)

```

Pitfall 2: Initializing Decimal from a float

Passing a raw float into `Decimal()` captures the IEEE 754 binary floating-point representation errors:

```

from decimal import Decimal

# THE BUG:

```

```
d = Decimal(0.1)
print(d)  # Decimal('0.1000000000000000055511151231257827021181583404541015625')

# THE FIX: Always pass strings!
d = Decimal("0.1")
print(d)  # Decimal('0.1')
```

Exercises

1. Write a billing calculator using `Decimal` that computes sales tax and applies a percentage discount, rounding to 2 decimal places using `ROUND_HALF_EVEN`.
 2. Generate a secure 16-character random alphanumeric password using `secrets.choice` across uppercase, lowercase, and digit character sets.
 3. Compute the SHA-512 hash of a text string and format the result as an uppercase hexadecimal digest.
 4. Implement a message signature verification function using `hmac` with SHA-256 and verify it against test vectors.
-

Further reading

- PEP 506: *Adding A Secrets Module To The Standard Library*.
- Python Standard Library: `decimal`, `fractions`, `secrets`, `hashlib`, `hmac`.
- RFC 2104: *HMAC: Keyed-Hashing for Message Authentication*.

OS, System Environment, and Subprocess Management

OS, System Environment, and Subprocess Management

After reading this chapter, you will master interaction with the host operating system using `os` and `sys`, safely spawn and monitor child processes with `subprocess.run()`, capture and redirect process streams, enforce execution timeouts, parse command strings safely with `shlex`, and eliminate shell injection vulnerabilities.

Mental model

Python executes as a user-space process managed by the operating system kernel. The `subprocess` module replaces legacy functions (`os.system`, `os.popen`) with a robust process orchestration engine:

Python Parent Process

1. Forks/Executes Child Process (kernel fork/clone)

2. Redirects File Descriptors:

Child STDIN	pipe	Parent input bytes
Child STDOUT	pipe	Parent stdout buffer
Child STDERR	pipe	Parent stderr buffer

```
[ subprocess.CompletedProcess ]
    returncode (0 for success, non-zero for error)
    stdout (captured text/bytes)
    stderr (captured error output)
```

Security Imperative: Never use `shell=True` with user-supplied arguments. Pass arguments as an explicit sequence of strings (`["cmd", "arg1", "arg2"]`) to prevent command injection.

Minimal example

Save as `subprocess_management.py`:

```

# subprocess_management.py
import shlex
import subprocess
import sys

def run_command_safely(command_str: str, timeout_sec: float = 5.0) -> subprocess.CompletedPr
    """Execute a shell command string safely using shlex.split without shell=True."""
    # shlex.split preserves quoted strings and spaces correctly
    args = shlex.split(command_str)

    result = subprocess.run(
        args,
        capture_output=True,
        text=True, # Decodes bytes to str using UTF-8 automatically
        timeout=timeout_sec,
        check=False # Do not raise CalledProcessError immediately; let caller inspect
    )
    return result

def main() -> None:
    # 1. Inspect host Python runtime
    print(f"Host OS platform: {sys.platform}")
    print(f"Python version : {sys.version.split()[0]}")

    # 2. Run child process (echo test)
    cmd = 'echo "Cluster Worker Node: active"'
    result = run_command_safely(cmd)

    print(f"\nCommand executed: {cmd}")
    print(f"Return code      : {result.returncode}")
    print(f"Captured STDOUT   : {result.stdout.strip()}")

if __name__ == "__main__":
    main()

```

Run via `uv run python subprocess_management.py`:

```

Host OS platform: linux
Python version   : 3.14.0

```

```

Command executed: echo "Cluster Worker Node: active"
Return code      : 0
Captured STDOUT : Cluster Worker Node: active

```

Worked examples

Case 1: Piping Streams Between Child Processes (p1 | p2)

In bash, piping (`cat data.txt | grep ERROR`) streams output between processes. In Python, you can connect child processes directly without loading intermediate streams into RAM:

```
# process_pipeline.py
import subprocess

def run_pipeline() -> str:
    # Child 1: Generate numbers
    # Simulates: printf "node-01\nnode-02\nworker-01\n" | grep worker
    p1 = subprocess.Popen(
        ["printf", "node-01\nnode-02\nworker-01\nworker-02\n"],
        stdout=subprocess.PIPE
    )

    # Child 2: Filter with grep, reading directly from p1.stdout
    p2 = subprocess.Popen(
        ["grep", "worker"],
        stdin=p1.stdout,
        stdout=subprocess.PIPE,
        text=True
    )

    # Allow p1 to receive SIGPIPE if p2 exits early
    if p1.stdout:
        p1.stdout.close()

    stdout, _ = p2.communicate()
    return stdout.strip()

if __name__ == "__main__":
    filtered = run_pipeline()
    print("Pipeline output:")
    print(filtered)
```

Run:

```
uv run python process_pipeline.py
```

Output:

Pipeline output:

worker-01

worker-02

Case 2: Sandboxing Environment Variables

When running external scripts or CLI tools, avoid mutating global `os.environ`. Construct an isolated dictionary and pass it to `env=`:

```
# isolated_environment.py
import os
import subprocess

def run_in_clean_env() -> None:
    # Clone current environment and inject custom overrides
    custom_env = os.environ.copy()
    custom_env["APP_ENV"] = "staging"
    custom_env["LOG_LEVEL"] = "DEBUG"

    # Child process sees only these injected environment variables
    res = subprocess.run(
        ["python3", "-c", "import os; print('APP_ENV:', os.environ.get('APP_ENV'))"],
        env=custom_env,
        capture_output=True,
        text=True,
        check=True
    )
    print("Child process output:", res.stdout.strip())

if __name__ == "__main__":
    run_in_clean_env()
```

Run:

```
uv run python isolated_environment.py
```

Output:

Child process output: APP_ENV: staging

Pitfalls

Pitfall 1: The `shell=True` Security Vulnerability

Using `shell=True` passes the raw command to `/bin/sh -c`. If any portion of the command string contains untrusted user input, attackers can chain arbitrary shell commands:

```
# CATASTROPHIC VULNERABILITY:
user_input = "file.txt; rm -rf /"
subprocess.run(f"ls -l {user_input}", shell=True) # EXECUTES rm -rf / !

# SECURE: Always pass a list of arguments without shell=True
safe_args = ["ls", "-l", user_input]
subprocess.run(safe_args, shell=False) # Treated safely as a single literal file name
```

Pitfall 2: Forgetting timeout=

If a spawned process hangs waiting for user input or deadlocks on network sockets, `subprocess.run()` without `timeout=` blocks the calling thread forever:

```
# DANGEROUS:
subprocess.run(["backup_script.sh"])

# SAFE: Always enforce a maximum execution budget
try:
    subprocess.run(["backup_script.sh"], timeout=30.0)
except subprocess.TimeoutExpired:
    logger.error("Process hung and exceeded 30s timeout; killed automatically.")
```

Exercises

1. Write a script that checks whether the `git` executable is installed on the host system using `shutil.which()` and prints its version with `subprocess.run()`.
 2. Spawn a child process with a 1-second timeout and verify that `subprocess.TimeoutExpired` is caught and handled cleanly.
 3. Use `shlex.split()` to parse a complex command line containing double quotes, single quotes, and escaped spaces into an argument list.
 4. Capture both stdout and stderr of a failing command and format a descriptive error message with the return code.
-

Further reading

- Python Standard Library: `subprocess`, `os`, `sys`, and `shlex` modules.
- PEP 324: *PEP 324 — subprocess - New process management module*.
- CWE-78: *Improper Neutralization of Special Elements used in an OS Command* ('OS Command Injection').

CLI Parsing and Structured Application Logging

CLI Parsing and Structured Application Logging

After reading this chapter, you will master command-line argument parsing with `argparse`, configure subcommands and flag validation, build production application logging using Python's `logging` module, configure multiple handlers (console and rotating files), and implement structured JSON log formatters.

Mental model

Enterprise utilities require two foundational capabilities: 1. **Command-Line Interface (`argparse`)**: Translates user CLI flags into typed Python namespaces with automated `--help` generation. 2. **Logging Pipeline (`logging`)**: Routes messages through loggers, filters, formatters, and handlers:

Logging Architecture:

```
Logger (logger.info("request"))
```

```
Level Check (DEBUG < INFO < WARN < ERROR < CRITICAL)
```

```
Passed
```

```
Handler 1 (StreamHandler -> Console / stderr)
```

```
Formatter 1 (Colorized text: "[INFO] 10:00 - message")
```

```
Handler 2 (RotatingFileHandler -> /var/log/app.log)
```

```
Formatter 2 (JSON Formatter: {"level": "INFO", "msg": ...})
```

Minimal example

Save as `cli_and_logging.py`:

```
# cli_and_logging.py
import argparse
import logging
import sys

def configure_logger(verbose: bool) -> logging.Logger:
    logger = logging.getLogger("cluster_manager")
```

```

level = logging.DEBUG if verbose else logging.INFO
logger.setLevel(level)

# Console handler
handler = logging.StreamHandler(sys.stdout)
formatter = logging.Formatter(
    fmt="%(asctime)s [%(levelname)s] %(name)s: %(message)s",
    datefmt="%Y-%m-%dT%H:%M:%SZ"
)
handler.setFormatter(formatter)
logger.addHandler(handler)
return logger

def build_parser() -> argparse.ArgumentParser:
    parser = argparse.ArgumentParser(
        prog="cluster-ctl",
        description="Cluster management orchestration CLI utility."
    )
    parser.add_argument("--verbose", "-v", action="store_true", help="Enable verbose debug logging")

    subparsers = parser.add_subparsers(dest="command", required=True, help="Subcommand to execute")

    # Subcommand: deploy
    deploy_parser = subparsers.add_parser("deploy", help="Deploy service instances.")
    deploy_parser.add_argument("service", type=str, help="Target service name.")
    deploy_parser.add_argument("--replicas", "-r", type=int, default=3, help="Number of replicas")

    return parser

def main() -> None:
    # Test arguments programmatically (simulating: cluster-ctl deploy auth-svc -r 5 -v)
    test_args = ["--verbose", "deploy", "auth-svc", "--replicas", "5"]
    parser = build_parser()
    args = parser.parse_args(test_args)

    logger = configure_logger(args.verbose)
    logger.debug("CLI arguments parsed successfully.")
    logger.info(f"Initiating deployment: service='{args.service}', replicas={args.replicas}")

if __name__ == "__main__":
    main()

```

Run via `uv run python cli_and_logging.py`:

```

2026-09-07T...Z [DEBUG] cluster_manager: CLI arguments parsed successfully.
2026-09-07T...Z [INFO] cluster_manager: Initiating deployment: service='auth-svc', replicas=5

```

Worked examples

Case 1: Structured JSON Logging for Observability Systems

Production observability systems (Datadog, Elasticsearch, CloudWatch) require JSON logs rather than plain text strings:

```
# json_logger.py
import json
import logging
import sys
from datetime import datetime, timezone

class JSONFormatter(logging.Formatter):
    """Formats log records as single-line JSON objects."""
    def format(self, record: logging.LogRecord) -> str:
        log_payload = {
            "timestamp": datetime.now(timezone.utc).isoformat(),
            "level": record.levelname,
            "logger": record.name,
            "message": record.getMessage(),
            "module": record.module,
            "line": record.lineno,
        }
        if record.exc_info:
            log_payload["exception"] = self.formatException(record.exc_info)
        return json.dumps(log_payload)

def setup_json_logging() -> logging.Logger:
    logger = logging.getLogger("api_gateway")
    logger.setLevel(logging.INFO)

    handler = logging.StreamHandler(sys.stdout)
    handler.setFormatter(JSONFormatter())
    logger.addHandler(handler)
    return logger

if __name__ == "__main__":
    log = setup_json_logging()
    log.info("Worker pool initialized successfully.")
    try:
        1 / 0
    except ZeroDivisionError:
        log.error("Math processing failure", exc_info=True)
```

Run:

```
uv run python json_logger.py
```

Output:

```
{"timestamp": "2026-09-07T...", "level": "INFO", "logger": "api_gateway", "message": "Worker"}
{"timestamp": "2026-09-07T...", "level": "ERROR", "logger": "api_gateway", "message": "Math"}
```

Case 2: Log File Rotation with RotatingFileHandler

Without log rotation, a logging process will eventually fill up the host filesystem. `RotatingFileHandler` caps file size and maintains backup generations:

```
# rotating_logs.py
import logging
import tempfile
from logging.handlers import RotatingFileHandler
from pathlib import Path

def test_log_rotation() -> None:
    with tempfile.TemporaryDirectory() as tmpdir:
        log_file = Path(tmpdir) / "service.log"

        # Max size: 500 bytes, keep up to 3 backup files (service.log.1, service.log.2, etc.)
        handler = RotatingFileHandler(log_file, maxBytes=500, backupCount=3)
        formatter = logging.Formatter("%(asctime)s %(levelname)s: %(message)s")
        handler.setFormatter(formatter)

        logger = logging.getLogger("rotator")
        logger.setLevel(logging.INFO)
        logger.addHandler(handler)

        # Write enough records to trigger multiple rotations
        for i in range(25):
            logger.info(f"Log event record #{i:02d} writing payload to disk buffer.")

        handler.close()

        # Inspect generated files
        generated = sorted(Path(tmpdir).glob("service.log*"))
        print(f"Generated {len(generated)} rotated files:")
        for f in generated:
            print(f"  - {f.name} ({f.stat().st_size} bytes)")

if __name__ == "__main__":
    test_log_rotation()
```

Run:

```
uv run python rotating_logs.py
```

Output:

Generated 4 rotated files:

- service.log (320 bytes)
 - service.log.1 (504 bytes)
 - service.log.2 (504 bytes)
 - service.log.3 (504 bytes)
-

Pitfalls

Pitfall 1: Adding Duplicate Handlers

Calling `logger.addHandler()` multiple times (e.g. inside a request handler or helper function) attaches multiple handlers, causing every message to print 2x, 3x, or 4x:

```
# THE BUG:
def handle_request():
    logger = logging.getLogger("app")
    logger.addHandler(logging.StreamHandler()) # ADDS NEW HANDLER ON EVERY CALL!
    logger.info("Request handled")

# THE FIX:
# Configure handlers ONCE at application startup, or check if handlers exist:
if not logger.handlers:
    logger.addHandler(...)
```

Pitfall 2: Using `print()` Instead of logging in Libraries

Libraries should never use `print()`. Use `logging.getLogger(__name__)` so consumers can control verbosity, redirect outputs, or silence messages as needed.

Exercises

1. Build an `argparse` command that accepts an input filename, an optional output filename (defaulting to `stdout`), and a `--dry-run` boolean flag.
 2. Configure a custom logging filter (`logging.Filter`) that blocks log records containing the string `"SECRET_TOKEN"`.
 3. Implement an `argparse` custom type function that validates that an argument is a valid IPv4 address.
 4. Set up a logger with two handlers: outputting `INFO` and above to console, and `DEBUG` and above to a local file.
-

Further reading

- Python Standard Library: `argparse` and `logging` modules.
- Python How-To: *Logging HOWTO* and *Logging Cookbook*.

Compression, Tarballs, and Archive Management

Compression, Tarballs, and Archive Management

After reading this chapter, you will master programmatic archive manipulation using `zipfile` and `tarfile`, stream compressed memory buffers with `gzip`, inspect and extract archive entries without disk extraction, and protect systems against path traversal extraction attacks (Zip Slip / Tar Slip).

Mental model

Archive formats bundle multiple files and directory structures into a single file container:

Archive Container Architecture:

ZIP File Container (`zipfile`)

Compressed Member 1: "configs/app.json"

Compressed Member 2: "scripts/deploy.sh"

Central Directory Table (Located at end of file: records offsets, CRC32, sizes)

TAR Archive Container (`tarfile`)

[512-byte Header] [File 1 Bytes] [512-byte Header] [File 2 Bytes] ...

By default, Python's `zipfile.ZipFile()` uses `ZIP_STORED` (uncompressed archiving). You must explicitly supply `compression=zipfile.ZIP_DEFLATED` to activate zlib compression.

Minimal example

Save as `archive_management.py`:

```
# archive_management.py
import io
import zipfile

def create_in_memory_zip() -> bytes:
    """Create a compressed ZIP archive entirely in RAM."""
    buffer = io.BytesIO()

    with zipfile.ZipFile(buffer, mode="w", compression=zipfile.ZIP_DEFLATED) as zf:
        # Write files directly from string/bytes without touching the physical disk
```

```

        zf.writestr("manifest.txt", "Cluster Deployment Manifest v1.0\n")
        zf.writestr("configs/app.json", '{"environment": "production", "debug": false}')

    return buffer.getvalue()

def inspect_zip(archive_bytes: bytes) -> None:
    buffer = io.BytesIO(archive_bytes)
    with zipfile.ZipFile(buffer, mode="r") as zf:
        print(f"Archive members ({len(zf.infolist())} files):")
        for info in zf.infolist():
            print(f"  - {info.filename:20} | Raw: {info.file_size:3d}B | Compressed: {info.c")

        # Read a member directly into memory
        manifest_text = zf.read("manifest.txt").decode("utf-8")
        print(f"\nManifest content: {manifest_text.strip()}")

def main() -> None:
    zip_bytes = create_in_memory_zip()
    print(f"Generated ZIP archive: {len(zip_bytes)} bytes")
    inspect_zip(zip_bytes)

if __name__ == "__main__":
    main()

```

Run via `uv run python archive_management.py`:

```

Generated ZIP archive: 318 bytes
Archive members (2 files):
- manifest.txt          | Raw:   33B | Compressed:   32B
- configs/app.json      | Raw:   44B | Compressed:   41B

Manifest content: Cluster Deployment Manifest v1.0

```

Worked examples

Case 1: In-Memory Gzip Payload Compression for Network APIs

When exchanging large JSON payloads over HTTP or WebSockets, compressing in-memory with `gzip.compress()` cuts bandwidth consumption by up to 90%:

```

# gzip_payload_streaming.py
import gzip
import json

```

```

def compress_telemetry_batch(records: list[dict[str, float | str]]) -> tuple[bytes, float]:
    raw_bytes = json.dumps(records).encode("utf-8")
    compressed = gzip.compress(raw_bytes, compresslevel=6)

    savings = (1.0 - (len(compressed) / len(raw_bytes))) * 100.0
    return compressed, savings

if __name__ == "__main__":
    # Generate 1,000 telemetry readings
    telemetry_data = [
        {"node": f"worker-{i%5}", "cpu": 12.5 + (i % 50), "metric": "load"}
        for i in range(1_000)
    ]

    compressed_data, pct_savings = compress_telemetry_batch(telemetry_data)

    # Decompress to verify integrity
    recovered = json.loads(gzip.decompress(compressed_data).decode("utf-8"))

    print(f"Raw data size          : {len(json.dumps(telemetry_data))} bytes")
    print(f"Compressed data size : {len(compressed_data)} bytes")
    print(f"Bandwidth savings       : {pct_savings:.1f}%")
    print(f"Verified record count: {len(recovered):,}")

```

Run:

```
uv run python gzip_payload_streaming.py
```

Output:

```

Raw data size          : 58,891 bytes
Compressed data size : 2,342 bytes
Bandwidth savings       : 96.0%
Verified record count: 1,000

```

Case 2: Defending Against Path Traversal (Tar Slip / Zip Slip)

Malicious archives can contain file entries with absolute paths (`/etc/cron.d/evil`) or directory traversal sequences (`../../root/.ssh/authorized_keys`). Extracting them naively overwrites critical system files:

```

# safe_tar_extraction.py
import tarfile
import tempfile
from pathlib import Path

```

```
def extract_tar_safely(tar_path: Path, destination: Path) -> None:
    with tarfile.open(tar_path, "r:*") as tar:
        # Python 3.12+ PEP 706: data_filter rejects absolute paths and '../' escapes!
        tar.extractall(path=destination, filter="data")
        print(f"Successfully and safely extracted to {destination}")

if __name__ == "__main__":
    with tempfile.TemporaryDirectory() as tmpdir:
        dest = Path(tmpdir) / "extracted"
        dest.mkdir()

        # Test safe extraction filter parameter
        print("PEP 706 extraction filter 'data' verified supported in runtime.")
```

Run:

```
uv run python safe_tar_extraction.py
```

Output:

```
PEP 706 extraction filter 'data' verified supported in runtime.
```

Pitfalls

Pitfall 1: Creating Uncompressed ZIP Archives

The default compression method in `zipfile.ZipFile` is `ZIP_STORED` (0% compression):

```
# UNCOMPRESSED:
with zipfile.ZipFile("data.zip", "w") as zf:
    ...

# COMPRESSED:
with zipfile.ZipFile("data.zip", "w", compression=zipfile.ZIP_DEFLATED) as zf:
    ...
```

Pitfall 2: Extracting Untrusted Archives Without Filters

Never call `tar.extractall()` or `zip.extractall()` on files uploaded by untrusted users without validating that all member paths resolve inside the destination directory. In Python 3.12+, always specify `filter="data"`.

Exercises

1. Write a script that archives an entire directory into a `.tar.gz` file using `tarfile.open(..., "w:gz")`.
 2. Inspect a ZIP archive without extracting it, and print the names and uncompressed sizes of all files ending with `.json`.
 3. Compress a 10 MB string in memory with `gzip` and benchmark the compression time and size difference between `compresslevel=1` (fastest) and `compresslevel=9` (maximum).
 4. Implement a path safety validator for `zipfile` that raises `ValueError` if any member filename starts with `/` or contains `...`
-

Further reading

- PEP 706: *Filter for `tarfile.extractall`*.
- Python Standard Library: `zipfile`, `tarfile`, and `gzip` modules.
- Snyk Research: *Zip Slip Vulnerability*.

Networking, Low-Level Sockets, and IP Address Calculations

Networking, Low-Level Sockets, and IP Address Calculations

After reading this chapter, you will master low-level networking primitives using the standard library: construct TCP client and server sockets with `socket`, dissect and sanitize URLs with `urllib.parse`, and calculate subnets, masks, and IP ranges with `ipaddress`.

Mental model

Python provides direct wrappers around operating system Berkeley sockets and IP networking primitives:

Socket Lifecycle (TCP Server):

`socket(AF_INET, SOCK_STREAM)` Creates OS socket file descriptor

`bind(('127.0.0.1', 8080))`

Binds to interface and port

`listen(backlog=5)`

Enters listening state for incoming SYN packets

`accept()` Blocks until 3-way handshake completes
Returns (client_socket, client_address)

`sendall()` / `recv()` (Stream bytes over TCP connection)

`close()` (Sends FIN packet, releases file descriptor)

The `ipaddress` module models IPv4 and IPv6 addresses, subnets, and host ranges as first-class objects with mathematical comparison and containment operations.

Minimal example

Save as `sockets_and_ipaddress.py`:

```

# sockets_and_ipaddress.py
import ipaddress
import urllib.parse

def main() -> None:
    # 1. IP Network & CIDR Calculations
    subnet = ipaddress.ip_network("10.0.1.0/24")
    print(f"Subnet CIDR           : {subnet}")
    print(f"Network Address       : {subnet.network_address}")
    print(f"Broadcast Address       : {subnet.broadcast_address}")
    print(f"Netmask                   : {subnet.netmask}")
    print(f"Usable Host Count         : {subnet.num_addresses - 2}")

    # Membership test (0(1) bitmask check)
    target_ip = ipaddress.ip_address("10.0.1.55")
    print(f"Is {target_ip} in {subnet}? {target_ip in subnet}")

    # 2. URL Parsing with urllib.parse
    raw_url = "https://api.internal.net:8443/v1/telemetry?filter=active&limit=50#summary"
    parsed = urllib.parse.urlparse(raw_url)

    print(f"\nParsed URL Components:")
    print(f"  Scheme   : {parsed.scheme}")
    print(f"  Host     : {parsed.hostname}")
    print(f"  Port     : {parsed.port}")
    print(f"  Path     : {parsed.path}")

    # Parse query string parameters into a dictionary
    params = urllib.parse.parse_qs(parsed.query)
    print(f"  Query Parameters: {params}")

if __name__ == "__main__":
    main()

```

Run via `uv run python sockets_and_ipaddress.py`:

```

Subnet CIDR           : 10.0.1.0/24
Network Address       : 10.0.1.0
Broadcast Address     : 10.0.1.255
Netmask               : 255.255.255.0
Usable Host Count     : 254
Is 10.0.1.55 in 10.0.1.0/24? True

```

Parsed URL Components:

```

Scheme   : https
Host     : api.internal.net
Port     : 8443

```

```
Path      : /v1/telemetry
Query Parameters: {'filter': ['active'], 'limit': ['50']}
```

Worked examples

Case 1: Low-Level TCP Loopback Echo Service

Writing a raw TCP client and server directly demonstrates socket buffers, timeouts, and byte transfers:

```
# tcp_loopback_echo.py
import socket
import threading
import time

def start_echo_server(port: int, stop_event: threading.Event) -> None:
    server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    # SO_REUSEADDR allows instant restart without waiting for TIME_WAIT
    server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    server.bind(("127.0.0.1", port))
    server.listen(1)
    server.settimeout(1.0) # Unblock periodically to check stop_event

    print(f"[Server] Listening on 127.0.0.1:{port}...")
    while not stop_event.is_set():
        try:
            client_conn, client_addr = server.accept()
        except socket.timeout:
            continue

        with client_conn:
            data = client_conn.recv(1024)
            if data:
                print(f"[Server] Received: {data.decode()} from {client_addr}")
                client_conn.sendall(b"ECHO:" + data)
    server.close()

def main() -> None:
    test_port = 18888
    stop_event = threading.Event()
    server_thread = threading.Thread(target=start_echo_server, args=(test_port, stop_event))
    server_thread.start()

    time.sleep(0.05) # Wait for server to bind
```

```

# Client connection
client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client.settimeout(2.0)
try:
    client.connect(("127.0.0.1", test_port))
    message = b"PING_CLUSTER"
    client.sendall(message)
    response = client.recv(1024)
    print(f"[Client] Received back: {response.decode()}")
finally:
    client.close()
    stop_event.set()
    server_thread.join()

if __name__ == "__main__":
    main()

```

Run:

```
uv run python tcp_loopback_echo.py
```

Output:

```

[Server] Listening on 127.0.0.1:18888...
[Server] Received: PING_CLUSTER from ('127.0.0.1', ...)
[Client] Received back: ECHO:PING_CLUSTER

```

Case 2: Subnet Splitting and Overlap Detection

Automating IP Address Management (IPAM) requires calculating available subnets and ensuring IP blocks never overlap:

```

# ipam_calculator.py
import ipaddress

def plan_datacenter_subnets() -> None:
    vpc_cidr = ipaddress.ip_network("172.16.0.0/16")
    print(f"Allocating subnets from VPC CIDR: {vpc_cidr}")

    # Subdivide /16 into /24 subnets
    subnets = list(vpc_cidr.subnets(new_prefix=24))
    print(f"Generated {len(subnets)} /24 subnets. First 3:")
    for sn in subnets[:3]:
        print(f"  - {sn} (Hosts: {sn.num_addresses - 2})")

    # Check for overlaps

```

```

public_subnet = ipaddress.ip_network("172.16.1.0/24")
private_subnet = ipaddress.ip_network("172.16.2.0/24")
print(f"\nDo public and private subnets overlap? {public_subnet.overlaps(private_subnet)}")

if __name__ == "__main__":
    plan_datacenter_subnets()

```

Run:

```
uv run python ipam_calculator.py
```

Output:

```

Allocating subnets from VPC CIDR: 172.16.0.0/16
Generated 256 /24 subnets. First 3:
- 172.16.0.0/24 (Hosts: 254)
- 172.16.1.0/24 (Hosts: 254)
- 172.16.2.0/24 (Hosts: 254)

```

```
Do public and private subnets overlap? False
```

Pitfalls

Pitfall 1: Host Bits Set in IPv4Network

Creating an `IPv4Network("192.168.1.15/24")` raises `ValueError: has host bits set` because the last octet is non-zero in a /24 network:

```

# THE BUG:
net = ipaddress.ip_network("192.168.1.15/24") # ValueError!

# THE FIX: If you want the network containing this host IP, pass strict=False
net = ipaddress.ip_network("192.168.1.15/24", strict=False) # Resolves to 192.168.1.0/24

# OR use IPv4Interface:
iface = ipaddress.ip_interface("192.168.1.15/24") # Preserves both host IP and network

```

Pitfall 2: Assuming `recv()` Returns a Full Packet

TCP is a stream-oriented protocol, not message-oriented. Calling `sock.recv(1024)` may return 10 bytes, 100 bytes, or 1024 bytes depending on OS buffer chunks and network fragmentation. Always loop until you reach an expected length or delimiter (like `\n`).

Exercises

1. Write a script that checks whether an IP address is a private RFC 1918 address using `ipaddress.IPv4Address.is_private`.
 2. Construct a URL query string from a dictionary of parameters using `urllib.parse.urlencode()`.
 3. Create a non-blocking TCP socket and demonstrate catching `BlockingIOError` when reading without incoming data.
 4. Calculate the supernet of two adjacent /24 subnets using `ipaddress.collapse_addresses()`.
-

Further reading

- Python Standard Library: `socket`, `ipaddress`, and `urllib.parse` modules.
- RFC 793: *Transmission Control Protocol*.
- RFC 4632: *Classless Inter-domain Routing (CIDR)*.

Signal Handling and Process Lifecycle Management

Signal Handling and Process Lifecycle Management

After reading this chapter, you will master POSIX signal management using Python's `signal` module, design graceful shutdown controllers for Kubernetes and Docker containers (`SIGTERM` and `SIGINT`), implement dynamic configuration reloads without process restarts via `SIGHUP`, apply execution watchdog alarms (`SIGALRM`), and resolve thread-affinity constraints.

Mental model

Operating systems communicate lifecycle events to running processes using **signals** (asynchronous software interrupts). When an orchestrator (such as `systemd`, `Docker`, or `Kubernetes`) stops a container, it sends `SIGTERM` to request a graceful shutdown, giving the process a grace period (e.g. 30 seconds) before escalating to uncatchable `SIGKILL`.

In CPython, the runtime intercepts POSIX signals in the C runtime layer, sets an internal flag, and executes your Python handler **on the main thread between bytecode evaluation instructions**:

Kernel / Orchestrator (`Docker` / `K8s` / `systemd`)

Sends POSIX Signal (e.g. `SIGTERM` / `SIGINT` / `SIGHUP`)

CPython Runtime (C Signal Trampoline)

Sets pending signal flag

Checked at bytecode loop boundary (`ceval.c`)

Python Main Thread Signal Handler

Executes registered callback: `handler(signum, frame)`

Action:

1. Set `is_shutting_down = True`
2. Finish current transaction / drain in-flight jobs
3. Flush logs and close database connection pools
4. Exit cleanly with `sys.exit(0)`

Minimal example

Save as `graceful_shutdown.py`:

```
# graceful_shutdown.py
import signal
import sys
import time
from types import FrameType

class GracefulWorker:
    def __init__(self) -> None:
        self.running = True
        # Register signal handlers for both Ctrl+C (SIGINT) and container termination (SIGTERM)
        signal.signal(signal.SIGINT, self._handle_exit_signal)
        signal.signal(signal.SIGTERM, self._handle_exit_signal)

    def _handle_exit_signal(self, signum: int, frame: FrameType | None) -> None:
        sig_name = signal.Signals(signum).name
        print(f"\n[SIGNAL] Received {sig_name} ({signum}). Initiating graceful termination..")
        self.running = False

    def run(self, max_cycles: int = 4) -> None:
        print(f"Worker PID: {sys.argv[0]} active. Running up to {max_cycles} cycles.")
        for cycle in range(1, max_cycles + 1):
            if not self.running:
                break
            print(f"  [Cycle #{cycle}] Processing in-flight transactions...")
            time.sleep(0.05)

        print("[CLEANUP] Closing connection pools and flushing buffers.")
        print("[EXIT] Process shutdown completed cleanly.")

def main() -> None:
    worker = GracefulWorker()
    worker.run()

if __name__ == "__main__":
    main()
```

Run via `uv run python graceful_shutdown.py`:

```
Worker PID: graceful_shutdown.py active. Running up to 4 cycles.
[Cycle #1] Processing in-flight transactions...
[Cycle #2] Processing in-flight transactions...
[Cycle #3] Processing in-flight transactions...
[Cycle #4] Processing in-flight transactions...
```

```
[CLEANUP] Closing connection pools and flushing buffers.  
[EXIT] Process shutdown completed cleanly.
```

Worked examples

Case 1: Container Termination Controller with In-Flight Drain

In production container environments, terminating pods must stop accepting new traffic while completing tasks already underway:

```
# container_drain.py  
import signal  
import sys  
import time  
from types import FrameType  
  
class TaskQueueConsumer:  
    def __init__(self) -> None:  
        self.shutdown_requested = False  
        self.active_jobs = 0  
        signal.signal(signal.SIGTERM, self._on_sigterm)  
  
    def _on_sigterm(self, signum: int, frame: FrameType | None) -> None:  
        print("\n[SIGTERM] Orchestrator requested pod eviction.")  
        print(" * Halting queue consumption (no new jobs).")  
        self.shutdown_requested = True  
  
    def process_queue(self, queue: list[str]) -> None:  
        while queue:  
            if self.shutdown_requested:  
                print(f"[DRAIN] Abandoning {len(queue)} pending jobs for sibling pods.")  
                break  
  
            job = queue.pop(0)  
            self.active_jobs += 1  
            print(f" [Executing] Processing job '{job}'...")  
            time.sleep(0.02)  
            self.active_jobs -= 1  
  
        print(f"[STATUS] Active jobs remaining: {self.active_jobs}. Ready to exit.")  
  
def main() -> None:  
    consumer = TaskQueueConsumer()  
    # Simulate pending work queue
```

```

work = ["job-101", "job-102", "job-103", "job-104"]
consumer.process_queue(work)

if __name__ == "__main__":
    main()

```

Run:

```
uv run python container_drain.py
```

Output:

```

[Executing] Processing job 'job-101'...
[Executing] Processing job 'job-102'...
[Executing] Processing job 'job-103'...
[Executing] Processing job 'job-104'...
[STATUS] Active jobs remaining: 0. Ready to exit.

```

Case 2: Zero-Downtime Configuration Reload via SIGHUP

UNIX daemons (like NGINX and PostgreSQL) listen for SIGHUP (Hangup Signal) to reload configuration files from disk without terminating client connections:

```

# config_reloader.py
import signal
import sys
import time
from types import FrameType

class HotReloadService:
    def __init__(self) -> None:
        self.config: dict[str, str | int] = {"rate_limit": 100, "log_level": "INFO"}
        # Register SIGHUP for dynamic configuration reload
        if hasattr(signal, "SIGHUP"):
            signal.signal(signal.SIGHUP, self._on_sighup)

    def _on_sighup(self, signum: int, frame: FrameType | None) -> None:
        print("\n[SIGHUP] Received reload signal! Re-reading configuration from disk...")
        # Simulate loading new configuration
        self.config["rate_limit"] = 500
        self.config["log_level"] = "DEBUG"
        print(f" [RELOADED] Updated config in-flight: {self.config}")

    def run_tick(self) -> None:

```

```

        print(f"Service running with: rate_limit={self.config['rate_limit']}, level={self.co

def main() -> None:
    service = HotReloadService()
    service.run_tick()

    # Simulate programmatic SIGHUP delivery to our own process
    if hasattr(signal, "SIGHUP"):
        signal.raise_signal(signal.SIGHUP)

    service.run_tick()

if __name__ == "__main__":
    main()

```

Run:

```
uv run python config_reloader.py
```

Output:

```

Service running with: rate_limit=100, level=INFO

[SIGHUP] Received reload signal! Re-reading configuration from disk...
[RELOADED] Updated config in-flight: {'rate_limit': 500, 'log_level': 'DEBUG'}
Service running with: rate_limit=500, level=DEBUG

```

Case 3: Watchdog Execution Timers with SIGALRM

When calling legacy C libraries, blocking filesystem calls, or operations that do not accept a `timeout` argument, `signal.alarm()` schedules a `SIGALRM` after a specified number of seconds:

```

# watchdog_timer.py
import signal
import time
from types import FrameType

class ExecutionTimeoutError(TimeoutError):
    """Raised when an operation exceeds the scheduled SIGALRM threshold."""

def alarm_handler(signum: int, frame: FrameType | None) -> None:
    raise ExecutionTimeoutError("Operation timed out: watchdog timer expired.")

def perform_bounded_operation(duration_seconds: float, timeout_seconds: int) -> str:

```

```

# Set handler and start countdown
old_handler = signal.signal(signal.SIGALRM, alarm_handler)
signal.alarm(timeout_seconds)

try:
    time.sleep(duration_seconds)
    return "SUCCESS"
finally:
    # ALWAYS cancel alarm in finally block (0 seconds disables the alarm)
    signal.alarm(0)
    signal.signal(signal.SIGALRM, old_handler)

def main() -> None:
    if not hasattr(signal, "SIGALRM"):
        print("SIGALRM is only available on UNIX/POSIX platforms.")
        return

    # 1. Operation within threshold
    print("Testing operation completing within timeout (0.01s < 1s):")
    status = perform_bounded_operation(duration_seconds=0.01, timeout_seconds=1)
    print(f"  Result: {status}")

    # 2. Operation exceeding threshold
    print("\nTesting operation exceeding timeout (2s > 1s):")
    try:
        perform_bounded_operation(duration_seconds=2.0, timeout_seconds=1)
    except ExecutionTimeoutError as err:
        print(f"  Caught expected timeout: {err}")

if __name__ == "__main__":
    main()

```

Run:

```
uv run python watchdog_timer.py
```

Output:

```
Testing operation completing within timeout (0.01s < 1s):
  Result: SUCCESS
```

```
Testing operation exceeding timeout (2s > 1s):
  Caught expected timeout: Operation timed out: watchdog timer expired.
```

Pitfalls

Pitfall 1: Registering Signals Outside the Main Thread

Python requires that all signal handlers be registered and executed **exclusively on the main thread**. Attempting to call `signal.signal()` from a worker thread raises `ValueError`:

```
import threading
import signal

def worker_thread():
    try:
        signal.signal(signal.SIGINT, lambda s, f: None)
    except ValueError as err:
        print(f"Thread error: {err}")

t = threading.Thread(target=worker_thread)
t.start()
t.join()
```

Output:

```
Thread error: signal only works in main thread of the main interpreter
```

Pitfall 2: Deadlocks from Non-Reentrant Code in Signal Handlers

A signal handler can interrupt execution at **any arbitrary bytecode instruction**, including while a lock is currently acquired. If the handler attempts to acquire the same lock, the process permanently deadlocks:

```
# THE TRAP: Deadlock!
import threading
lock = threading.Lock()

def unsafe_handler(signum, frame):
    lock.acquire() # DEADLOCK if main thread was interrupted while holding 'lock'!
    # do cleanup...
    lock.release()

# THE FIX:
# In signal handlers, only set simple atomic flags (booleans, events)
# and let the main execution loop inspect the flag and release locks cleanly.
shutdown_event = threading.Event()
def safe_handler(signum, frame):
```

```
shutdown_event.set()
```

Pitfall 3: PID 1 in Docker Containers Ignoring Signals

When a Python script runs as PID 1 inside a Docker container (e.g. CMD ["python", "app.py"]), the Linux kernel treats PID 1 specially: **default signal handlers are disabled**. If your script does not explicitly register a handler for SIGTERM, docker stop will hang for 10 seconds before forcibly terminating with SIGKILL!

```
# THE FIX: Always register an explicit handler for SIGTERM in container entrypoints:
signal.signal(signal.SIGTERM, lambda s, f: sys.exit(0))
```

Exercises

1. Write a script that tracks how many times SIGINT (Ctrl+C) is pressed. On the first press, print a warning; on the second press within 3 seconds, terminate the application.
 2. Implement a context manager `time_limit(seconds)` using `signal.alarm()` that raises a custom `TimeoutException` if the code block takes longer than `seconds`.
 3. Create a daemon simulator that catches SIGHUP to toggle a debug logging boolean flag while continuing to loop.
 4. Demonstrate how `signal.pause()` suspends the current process until any POSIX signal is received.
 5. Write a script that sends SIGUSR1 to another Python process using `os.kill(target_pid, signal.SIGUSR1)` to trigger a diagnostic memory dump.
-

Further reading

- Python Documentation: `signal` — *Set handlers for asynchronous events*.
- POSIX Standards: *Signal Concepts and System Call Interruption*.
- Kubernetes Documentation: *Pod Lifecycle – Pod Termination*.

Built-In HTTP Networking: Servers, Clients, and Request Handlers

Built-In HTTP Networking: Servers, Clients, and Request Handlers

After reading this chapter, you will master Python’s zero-dependency HTTP networking stack: spin up lightweight microservices using `http.server.ThreadingHTTPServer`, parse JSON payloads and query parameters in `BaseHTTPRequestHandler`, execute performant client requests using `http.client` and `urllib.request`, and prevent concurrency starvation bugs.

Mental model

While third-party packages like `requests`, `httpx`, and `FastAPI` are common in large applications, Python’s built-in standard library provides a complete, production-grade HTTP implementation requiring zero external dependencies:

CPython Built-in HTTP Stack Architecture:

High-Level Abstractions

Server: `http.server.ThreadingHTTPServer`

Client: `urllib.request.urlopen`

Protocol Framing Layer

Server: `http.server.BaseHTTPRequestHandler`

Client: `http.client.HTTPConnection` / `HTTPConnection`

Low-Level Transport Layer

`socketserver.TCPServer` `socket.socket`

Minimal example

Save as `http_service_demo.py`:

```
# http_service_demo.py
import json
import threading
import urllib.request
from http.server import BaseHTTPRequestHandler, ThreadingHTTPServer

class HealthCheckHandler(BaseHTTPRequestHandler):
    """Zero-dependency HTTP handler serving JSON health endpoints."""

    def do_GET(self) -> None:
        if self.path == "/health":
            payload = json.dumps({"status": "SERVING", "version": "1.0.0"}).encode("utf-8")
            self.send_response(200)
            self.send_header("Content-Type", "application/json")
            self.send_header("Content-Length", str(len(payload)))
            self.end_headers()
            self.wfile.write(payload)
        else:
            self.send_error(404, "Endpoint Not Found")

    def log_message(self, format: str, *args: object) -> None:
        # Suppress noisy default standard error logging
        pass

def main() -> None:
    # 1. Bind to ephemeral local port (port 0 selects available random port)
    server = ThreadingHTTPServer(("127.0.0.1", 0), HealthCheckHandler)
    host, port = server.server_address

    # 2. Run server loop on a background thread
    server_thread = threading.Thread(target=server.serve_forever, daemon=True)
    server_thread.start()
    print(f"HTTP Server active on http://{host}:{port}")

    # 3. Query the endpoint using built-in urllib.request
    try:
        url = f"http://{host}:{port}/health"
        with urllib.request.urlopen(url, timeout=3) as response:
            status_code = response.status
            body = json.loads(response.read().decode("utf-8"))

        print(f"Client received HTTP {status_code}: {body}")
    finally:
```

```

server.shutdown()
server.server_close()
print("HTTP Server cleanly stopped.")

if __name__ == "__main__":
    main()

```

Run via `uv run python http_service_demo.py`:

```

HTTP Server active on http://127.0.0.1:41829
Client received HTTP 200: {'status': 'SERVING', 'version': '1.0.0'}
HTTP Server cleanly stopped.

```

Worked examples

Case 1: Building a REST Metrics Ingestion Webhook Handler

Production microservices frequently expose webhooks to accept JSON telemetry records via POST requests while reporting metrics on GET:

```

# metrics_webhook_service.py
import json
import threading
import urllib.request
from http.server import BaseHTTPRequestHandler, ThreadingHTTPServer

class MetricsCollectorHandler(BaseHTTPRequestHandler):
    # Shared in-memory metrics storage
    metrics_store: dict[str, float] = {}
    lock = threading.Lock()

    def do_POST(self) -> None:
        if self.path == "/api/v1/metrics":
            # Read Content-Length to determine exact body bytes to read
            content_length = int(self.headers.get("Content-Length", 0))
            if content_length == 0:
                self.send_error(400, "Missing or zero Content-Length")
                return

            body = self.rfile.read(content_length)
            try:
                data = json.loads(body.decode("utf-8"))
            except json.JSONDecodeError:

```

```

        self.send_error(400, "Invalid JSON payload")
        return

    # Thread-safe write
    with self.lock:
        self.metrics_store.update(data)

    ack_payload = json.dumps({"acknowledged_keys": list(data.keys())}).encode("utf-8")
    self.send_response(202)
    self.send_header("Content-Type", "application/json")
    self.send_header("Content-Length", str(len(ack_payload)))
    self.end_headers()
    self.wfile.write(ack_payload)
else:
    self.send_error(404)

def log_message(self, format: str, *args: object) -> None:
    pass

def main() -> None:
    server = ThreadingHTTPServer(("127.0.0.1", 0), MetricsCollectorHandler)
    host, port = server.server_address
    t = threading.Thread(target=server.serve_forever, daemon=True)
    t.start()

    try:
        # Client POST request using standard library
        url = f"http://{host}:{port}/api/v1/metrics"
        sample_payload = json.dumps({"cpu_util": 84.2, "mem_util": 61.0}).encode("utf-8")
        req = urllib.request.Request(
            url,
            data=sample_payload,
            headers={"Content-Type": "application/json"},
            method="POST",
        )

        with urllib.request.urlopen(req, timeout=3) as resp:
            print(f"POST Status: {resp.status}")
            print(f"Response: {resp.read().decode('utf-8')}")

        print(f"\nServer-side metrics store: {MetricsCollectorHandler.metrics_store}")
    finally:
        server.shutdown()
        server.server_close()

if __name__ == "__main__":
    main()

```

Run:

```
uv run python metrics_webhook_service.py
```

Output:

POST Status: 202

Response: {"acknowledged_keys": ["cpu_util", "mem_util"]}

Server-side metrics store: {'cpu_util': 84.2, 'mem_util': 61.0}

Case 2: Low-Level Streaming Requests with `http.client.HTTPConnection`

When handling large file downloads or high-throughput API communication, `http.client` provides direct, low-level access to the socket stream without loading the entire payload into RAM:

```
# http_client_streaming.py
import http.client
import json
import threading
from http.server import BaseHTTPRequestHandler, ThreadingHTTPServer

class BulkDataHandler(BaseHTTPRequestHandler):
    def do_GET(self) -> None:
        if self.path == "/stream":
            self.send_response(200)
            self.send_header("Content-Type", "text/plain")
            self.end_headers()
            # Stream 3 chunks sequentially
            for i in range(1, 4):
                self.wfile.write(f"Chunk-#{i}\n".encode("utf-8"))
        else:
            self.send_error(404)

    def log_message(self, *args: object) -> None:
        pass

def main() -> None:
    server = ThreadingHTTPServer(("127.0.0.1", 0), BulkDataHandler)
    host, port = server.server_address
    threading.Thread(target=server.serve_forever, daemon=True).start()

    try:
        # Use http.client for granular socket streaming
        conn = http.client.HTTPConnection(host, port, timeout=5)
```

```

conn.request("GET", "/stream")
response = conn.getresponse()

print(f"Response Status: {response.status} {response.reason}")
print("Reading streamed chunks on client:")
while chunk := response.readline():
    print(f"  Received: {chunk.decode('utf-8').strip()}")
conn.close()
finally:
    server.shutdown()
    server.server_close()

if __name__ == "__main__":
    main()

```

Run:

```
uv run python http_client_streaming.py
```

Output:

```

Response Status: 200 OK
Reading streamed chunks on client:
  Received: Chunk-#1
  Received: Chunk-#2
  Received: Chunk-#3

```

Pitfalls

Pitfall 1: Using HTTPServer Instead of ThreadingHTTPServer

The legacy `http.server.HTTPServer` is **strictly single-threaded**. If a single client connection hangs or takes 10 seconds to upload, **all other clients are blocked entirely**:

```

# THE TRAP: Single-threaded; one slow client starves the entire server!
from http.server import HTTPServer, BaseHTTPRequestHandler
# server = HTTPServer(('0.0.0.0', 8080), MyHandler)

# THE FIX: Always use ThreadingHTTPServer (handles each request in a worker thread)
from http.server import ThreadingHTTPServer
server = ThreadingHTTPServer(('0.0.0.0', 8080), BaseHTTPRequestHandler)

```

Pitfall 2: Omitting the timeout Parameter in urllib.request.urlopen

By default, `urllib.request.urlopen` uses the global socket default timeout (`None`), meaning a client connection can **hang forever** if the remote host drops the TCP packet:

```
# THE TRAP:
# with urllib.request.urlopen("https://example.com") as resp: ... # Can hang indefinitely!

# THE FIX: Always specify an explicit timeout in seconds
import urllib.request
with urllib.request.urlopen("https://example.com", timeout=5.0) as resp:
    data = resp.read()
```

Pitfall 3: Not Reading Content-Length in do_POST

In `do_POST`, calling `self.rfile.read()` without specifying the number of bytes to read will hang indefinitely because the socket connection remains open for keep-alive:

```
# THE TRAP:
def do_POST(self):
    # data = self.rfile.read() # HANGS! Socket is waiting for EOF that never arrives!

# THE FIX: Read only up to Content-Length bytes:
def do_POST(self):
    length = int(self.headers.get('Content-Length', 0))
    data = self.rfile.read(length)
```

Exercises

1. Build a zero-dependency HTTP server that serves static files from a local directory, returning 404 Not Found if the file does not exist.
 2. Implement an HTTP handler that parses query parameters (e.g. `/search?q=python&limit=10`) using `urllib.parse.parse_qs`.
 3. Write a script that uses `http.client.HTTPConnection` to send a HEAD request to inspect response headers without downloading the body.
 4. Implement basic HTTP authentication verification (`Authorization: Basic ...`) in a custom `BaseHTTPRequestHandler`.
 5. Measure the throughput in requests-per-second of `ThreadingHTTPServer` when queried by 10 concurrent threads using `urllib.request`.
-

Further reading

- Python Documentation: `http.server` — *HTTP servers*.
- Python Documentation: `http.client` — *HTTP protocol client*.
- Python Documentation: `urllib.request` — *Extensible library for opening URLs*.
- RFC 9110: *HTTP Semantics*.

High-Performance File I/O with Memory-Mapped Files (mmap)

High-Performance File I/O with Memory-Mapped Files (mmap)

After reading this chapter, you will master operating-system-level virtual memory mapping using Python's `mmap` module, inspect multi-gigabyte files with a near-zero memory footprint, execute in-place binary substitutions directly on disk, search disk contents using regular expressions without loading strings into RAM, and establish shared memory IPC.

Mental model

Traditional file I/O requires copying data multiple times: from disk storage into the OS kernel buffer cache, and then from the kernel buffer into a Python heap-allocated `bytes` or `str` object via the `read()` system call:

Standard File I/O (Two Copies):

```
Disk File      [ OS Kernel Page Cache ]      read() copies      Python Heap Object
                                                         (High RAM usage!)
```

A **memory-mapped file** (`mmap`) asks the operating system kernel to map a file's disk blocks directly into your process's **virtual address space**. Accessing the `mmap` object is as fast as accessing a memory array, with the OS page cache transparently paging blocks in and out on demand:

Memory-Mapped File (Zero User-Space Buffer Copies):

```
Disk Storage (e.g. 50 GB log file or database tables)
```

```
Direct Virtual Memory Mapping (mmap)
```

```
Process Virtual Address Space
```

```
mm[0:1024] reads directly from kernel page cache
mm[offset] = b'X' writes directly to page cache
(Instant random access, O(1) Python heap footprint)
```

Minimal example

Save as `mmap_overview.py`:

```
# mmap_overview.py
import mmap
import tempfile

def main() -> None:
    # 1. Create a temporary file with sample log data
    with tempfile.NamedTemporaryFile(delete=False) as f:
        f.write(b"2026-09-07 12:00:00 [INFO] Cluster initialized\n")
        f.write(b"2026-09-07 12:00:05 [WARN] Disk watermark 85%\n")
        f.write(b"2026-09-07 12:00:10 [ERR ] Failed node: srv-04\n")
        filepath = f.name

    # 2. Map the file into virtual memory
    with open(filepath, "r+b") as file_obj:
        # mmap(fileno, length): 0 means map the entire file
        with mmap.mmap(file_obj.fileno(), 0) as mm:
            print(f"Memory-mapped file size: {mm.size()} bytes")

            # Search without loading file into Python RAM
            target = b"[ERR ]"
            offset = mm.find(target)
            print(f"Found '{target.decode()}' at byte offset: {offset}")

            # Read a specific slice directly from disk memory
            mm.seek(offset)
            error_line = mm.readline()
            print(f"Read line: {error_line.decode().strip()}")

            # In-place disk mutation: overwrite [ERR ] with [CRIT]
            mm.seek(offset)
            mm.write(b"[CRIT]")
            mm.flush() # Ensure changes are written back to physical disk

        # Verify mutation persisted in physical file
        with open(filepath, "rb") as verify_f:
            print(f"\nPersisted file content:\n{verify_f.read().decode()}")

if __name__ == "__main__":
    main()
```

Run via `uv run python mmap_overview.py`:

Memory-mapped file size: 140 bytes

```
Found '[ERR ]' at byte offset: 113
Read line: [ERR ] Failed node: srv-04
```

```
Persisted file content:
2026-09-07 12:00:00 [INFO] Cluster initialized
2026-09-07 12:00:05 [WARN] Disk watermark 85%
2026-09-07 12:00:10 [CRIT] Failed node: srv-04
```

Worked examples

Case 1: Regex Searching Gigabyte Logs with Zero RAM Allocation

When auditing 20 GB web server logs, calling `file.read()` triggers an Out-Of-Memory error. Because `mmap` supports Python's buffer protocol, standard `re` expressions can search directly over the memory map:

```
# mmap_regex_search.py
import mmap
import re
import tempfile

def generate_large_log(filepath: str, line_count: int = 10_000) -> None:
    with open(filepath, "wb") as f:
        for i in range(line_count):
            if i == 4200:
                f.write(b'192.168.1.100 - [2026-09-07] "GET /admin/secret" 403 128\n')
            elif i == 8900:
                f.write(b'10.0.0.50 - [2026-09-07] "POST /api/v1/auth" 500 256\n')
            else:
                f.write(b'127.0.0.1 - [2026-09-07] "GET /health" 200 32\n')

def main() -> None:
    with tempfile.NamedTemporaryFile(delete=False) as tmp:
        filepath = tmp.name

    generate_large_log(filepath)

    pattern = re.compile(rb'(\d+\.\d+\.\d+\.\d+) - .* "([A-Z]+ [^"]+)" ([45]\d\d)')

    with open(filepath, "rb") as f:
        with mmap.mmap(f.fileno(), 0, access=mmap.ACCESS_READ) as mm:
            print("Scanning log file with zero heap allocation:")
            for match in pattern.finditer(mm):
                ip, endpoint, status = match.groups()
```

```

        print(f"  Anomaly detected at offset {match.start():6d}: IP={ip.decode()} St

if __name__ == "__main__":
    main()

```

Run:

```
uv run python mmap_regex_search.py
```

Output:

Scanning log file with zero heap allocation:

```

Anomaly detected at offset 193200: IP=192.168.1.100 Status=403 (GET /admin/secret)
Anomaly detected at offset 409411: IP=10.0.0.50 Status=500 (POST /api/v1/auth)

```

Case 2: In-Place Binary Patching and Header Updates

In firmware flashing and database file management, updating records in place avoids rewriting the entire file:

```

# binary_header_patcher.py
import mmap
import struct
import tempfile

# Binary Header Format: 4-byte Magic, 4-byte Version, 8-byte Checksum
HEADER_FORMAT = "!4sIQ"

def main() -> None:
    with tempfile.NamedTemporaryFile(delete=False) as f:
        # Write initial header
        initial_bytes = struct.pack(HEADER_FORMAT, b"BLOB", 1, 0x1122334455667788)
        f.write(initial_bytes + (b"\x00" * 128))
        filepath = f.name

    with open(filepath, "r+b") as f:
        with mmap.mmap(f.fileno(), 0) as mm:
            magic, version, checksum = struct.unpack(HEADER_FORMAT, mm[:16])
            print("Initial Header:")
            print(f"  Magic:    {magic}")
            print(f"  Version:  {version}")
            print(f"  Checksum: 0x{checksum:X}")

            # Bump version to 2 and update checksum in place!

```

```

        new_header = struct.pack(HEADER_FORMAT, b"BLOB", 2, 0xAABBCCDDEEFF0011)
        mm[:16] = new_header
        mm.flush()

    with open(filepath, "rb") as f:
        magic, version, checksum = struct.unpack(HEADER_FORMAT, f.read(16))
        print("\nVerified Header on Disk:")
        print(f"  Magic:    {magic}")
        print(f"  Version:  {version}")
        print(f"  Checksum: 0x{checksum:X}")

if __name__ == "__main__":
    main()

```

Run:

```
uv run python binary_header_patcher.py
```

Output:

Initial Header:

```

Magic:    b'BLOB'
Version:  1
Checksum: 0x1122334455667788

```

Verified Header on Disk:

```

Magic:    b'BLOB'
Version:  2
Checksum: 0xAABBCCDDEEFF0011

```

Case 3: Anonymous Shared Memory IPC Between Processes

By specifying `-1` as the file descriptor, `mmap` allocates an **anonymous shared memory segment** backed by RAM rather than a physical file, enabling zero-disk IPC:

```

# anonymous_shared_memory.py
import mmap
import os
import time

def main() -> None:
    # fileno = -1 creates an anonymous memory map
    shm = mmap.mmap(-1, 1024)

```

```

# Fork child process
pid = os.fork()

if pid == 0:
    # Child process: Wait for parent to write data
    time.sleep(0.02)
    shm.seek(0)
    message = shm.readline().decode("utf-8").strip()
    print(f"[Child PID {os.getpid()}] Read from shared memory: '{message}'")
    shm.close()
    os._exit(0)
else:
    # Parent process: Write data to shared memory
    shm.write(b"SYNC_TOKEN: 0x8FA2BC99\n")
    print(f"[Parent PID {os.getpid()}] Wrote token to anonymous shared memory.")
    os.waitpid(pid, 0)
    shm.close()

if __name__ == "__main__":
    main()

```

Run:

```
uv run python anonymous_shared_memory.py
```

Output:

```

[Parent PID 12345] Wrote token to anonymous shared memory.
[Child PID 12346] Read from shared memory: 'SYNC_TOKEN: 0x8FA2BC99'

```

Pitfalls

Pitfall 1: Attempting to Map an Empty (0-Byte) File

The operating system cannot map a file with a length of 0 bytes. Attempting to do so raises `ValueError`:

```

# THE TRAP:
import tempfile
import mmap

with tempfile.NamedTemporaryFile() as empty_file:
    # empty_file has size 0
    try:

```

```
mm = mmap.mmap(empty_file.fileno(), 0)
except ValueError as err:
    print(f"Caught: {err}")
```

Output:

Caught: cannot mmap an empty file

The Fix: Ensure the file contains at least 1 byte before mapping, or check `os.path.getsize(filepath) > 0`.

Pitfall 2: Modifying File Size Outside `mmap`

If another process or thread truncates the underlying file while a `memoryview` or `mmap` is open, reading beyond the new end of file triggers an uncatchable OS `SIGBUS` (Bus Error) signal, crashing the Python process immediately. Always coordinate file resizing with process-wide locks.

Exercises

1. Create a 10 MB file filled with random ASCII bytes. Use `mmap.find()` to locate all occurrences of the word "TARGET".
 2. Write a script that counts line breaks (`\n`) in a large text file using `mmap` and compare its execution time and memory usage against `file.readlines()`.
 3. Implement a circular ring buffer inside an anonymous memory map (`mmap(-1, size)`) shared between two forked processes.
 4. Open a read-only binary file using `access=mmap.ACCESS_READ` and verify that attempting to write to the memory map raises `TypeError`.
 5. Build an in-place string replacement function that searches for a 4-letter token and replaces it with a new 4-letter token across a 50 MB log file.
-

Further reading

- Python Documentation: `mmap` — *Memory-mapped file support*.
- Linux Programmer's Manual: `mmap(2)` — *Map files or devices into memory*.
- David Beazley: *Python Cookbook* (Recipe 5.18: Memory Mapping an I/O File).

Runtime Introspection and Reflection with inspect

Runtime Introspection and Reflection with `inspect`

After reading this chapter, you will master Python’s runtime reflection and introspection machinery using the standard library `inspect` module, dynamically parse function signatures and type annotations, bind and validate arguments programmatically before invocation, navigate live Python call frames, extract source code and docstrings dynamically, and engineer production-grade dependency injection containers.

Mental model

In compiled languages like C or Go, function signatures, parameter names, and local variable scopes are largely erased at compile time. In Python, code objects, functions, classes, and execution frames retain their full symbolic metadata in memory at runtime:

Python Callable (Function / Method / Closure)

<code>__code__</code> (Bytecode & Names)	<code>__annotations__</code> (Type Hints)
<code>__defaults__</code> (Arg Defaults)	<code>__kwdefaults__</code> (KW Defaults)

```
inspect.signature(fn)
```

```
inspect.Signature
  return_annotation: type
  parameters: MappingProxyType
    'req_id': Parameter(POSITIONAL_OR_KEYWORD, int)
    'dry_run': Parameter(KEYWORD_ONLY, bool, False)
    '*args': Parameter(VAR_POSITIONAL)
```

```
sig.bind(*args, **kwargs)
```

```
inspect.BoundArguments
  arguments: {'req_id': 100, 'dry_run': False}
  apply_defaults(): injects missing default parameters
```

Furthermore, whenever a Python function executes, CPython pushes a **frame object** (`PyFrameObject`) onto the runtime call stack. The `inspect` module allows libraries to inspect the active call stack, locate the caller's file and line number, and introspect the local namespace.

Minimal example

Save as `inspect_overview.py`:

```
# inspect_overview.py
import inspect
from typing import Any

def provision_cluster(
    cluster_name: str,
    node_count: int = 3,
    *tags: str,
    auto_repair: bool = True,
    **metadata: Any,
) -> dict[str, Any]:
    """Provisions a virtual computing cluster with specified parameters."""
    return {
        "name": cluster_name,
        "nodes": node_count,
        "tags": tags,
        "auto_repair": auto_repair,
        "metadata": metadata,
    }

def main() -> None:
    # 1. Extract signature metadata
    sig = inspect.signature(provision_cluster)
    print(f"Callable signature:\n {sig}\n")

    print("Parameter breakdown:")
    for name, param in sig.parameters.items():
        print(
            f"  Name: {name:<12} "
            f"Kind: {param.kind.name:<22} "
            f"Default: {str(param.default):<10} "
            f"Annotation: {param.annotation}"
        )

    # 2. Programmatically bind runtime arguments
```

```

bound = sig.bind("prod-eu-west", 10, "k8s", "gpu", auto_repair=False, zone="eu-west-1a")
print(f"\nBound arguments dictionary:\n {bound.arguments}")

# 3. Invoke function with bound arguments
result = provision_cluster(*bound.args, **bound.kwargs)
print(f"\nFunction result:\n {result}")

if __name__ == "__main__":
    main()

```

Run via `uv run python inspect_overview.py`:

Callable signature:

```
(cluster_name: str, node_count: int = 3, *tags: str, auto_repair: bool = True, **metadata: Any)
```

Parameter breakdown:

Name: cluster_name	Kind: POSITIONAL_OR_KEYWORD	Default: <class 'inspect._empty'>	Annotation: <class 'int'>
Name: node_count	Kind: POSITIONAL_OR_KEYWORD	Default: 3	Annotation: <class 'int'>
Name: tags	Kind: VAR_POSITIONAL	Default: <class 'inspect._empty'>	Annotation: <class 'int'>
Name: auto_repair	Kind: KEYWORD_ONLY	Default: True	Annotation: <class 'bool'>
Name: metadata	Kind: VAR_KEYWORD	Default: <class 'inspect._empty'>	Annotation: <class 'int'>

Bound arguments dictionary:

```
{'cluster_name': 'prod-eu-west', 'node_count': 10, 'tags': ('k8s', 'gpu'), 'auto_repair': False, 'metadata': {}}
```

Function result:

```
{'name': 'prod-eu-west', 'nodes': 10, 'tags': ('k8s', 'gpu'), 'auto_repair': False, 'metadata': {}}
```

Worked examples

Case 1: Runtime Type-Enforcing Decorator with `Signature.bind()`

While static type checkers like `mypy` and `pyright` catch errors before execution, boundaries such as HTTP endpoints, message queues, and user scripts receive raw data. `inspect.signature` enables writing a zero-boilerplate runtime type validator:

```

# runtime_type_guard.py
import functools
import inspect
from typing import Any, Callable

def enforce_contract(func: Callable[..., Any]) -> Callable[..., Any]:
    """Validates runtime arguments against function type annotations."""

```

```

sig = inspect.signature(func)

@functools.wraps(func)
def wrapper(*args: Any, **kwargs: Any) -> Any:
    # Bind incoming arguments to parameter names, handling defaults
    bound = sig.bind(*args, **kwargs)
    bound.apply_defaults()

    for param_name, value in bound.arguments.items():
        param = sig.parameters[param_name]
        expected_type = param.annotation

        # Validate basic types when annotations are explicit classes
        if expected_type is not inspect.Parameter.empty and isinstance(expected_type, type):
            if not isinstance(value, expected_type):
                raise TypeError(
                    f"Contract violation on '{func.__name__}': "
                    f"argument '{param_name}' must be {expected_type.__name__}, "
                    f"received {type(value).__name__} ({value!r})"
                )

    result = func(*args, **kwargs)

    # Validate return value
    ret_type = sig.return_annotation
    if ret_type is not inspect.Signature.empty and isinstance(ret_type, type):
        if not isinstance(result, ret_type):
            raise TypeError(
                f"Contract violation on '{func.__name__}': "
                f"expected return {ret_type.__name__}, got {type(result).__name__}"
            )

    return result

return wrapper

@enforce_contract
def scale_service(service_name: str, target_replicas: int, dry_run: bool = False) -> str:
    return f"Service '{service_name}' scaling to {target_replicas} (dry_run={dry_run})"

def main() -> None:
    # 1. Valid call succeeds
    msg = scale_service("api-gateway", 8)
    print(f"Success: {msg}")

    # 2. Invalid parameter type raises immediate TypeError

```

```

try:
    scale_service("api-gateway", "eight") # type: ignore
except TypeError as err:
    print(f"\nCaught validation error:\n {err}")

if __name__ == "__main__":
    main()

```

Run:

```
uv run python runtime_type_guard.py
```

Output:

Success: Service 'api-gateway' scaling to 8 (dry_run=False)

Caught validation error:

Contract violation on 'scale_service': argument 'target_replicas' must be int, received str (

Case 2: Zero-Magic Dependency Injection Container

Modern web frameworks like FastAPI and test frameworks like `pytest` automatically inject database sessions, configuration objects, and authentication credentials into endpoint handlers by introspecting parameter type annotations:

```

# dependency_injection.py
import inspect
from typing import Any, Callable, Dict, Type

class Container:
    """Type-based dependency injection registry."""

    def __init__(self) -> None:
        self._services: Dict[Type[Any], Any] = {}

    def register(self, service_type: Type[Any], instance: Any) -> None:
        """Registers a singleton instance for a given type."""
        self._services[service_type] = instance

    def invoke(self, target: Callable[..., Any], **explicit_args: Any) -> Any:
        """Introspects target parameters and injects matching dependencies."""
        sig = inspect.signature(target)
        resolved_kwargs = dict(explicit_args)

```

```

    for param_name, param in sig.parameters.items():
        # If the argument was already explicitly supplied, skip auto-wiring
        if param_name in resolved_kwargs:
            continue

        target_type = param.annotation
        if target_type in self._services:
            resolved_kwargs[param_name] = self._services[target_type]
        elif param.default is inspect.Parameter.empty:
            raise RuntimeError(
                f"Unable to resolve required parameter '{param_name}': {target_type}" for
            )

    return target(**resolved_kwargs)

# Sample domain services
class DatabaseConnectionPool:
    def execute(self, query: str) -> str:
        return f"[DB] Executed '{query}'"

class AuditLogger:
    def log(self, event: str) -> str:
        return f"[AUDIT] Recorded '{event}'"

# Handler function declaring its dependencies via type hints
def checkout_handler(
    cart_id: int,
    db: DatabaseConnectionPool,
    audit: AuditLogger,
    currency: str = "USD",
) -> str:
    db_res = db.execute(f"UPDATE carts SET status='PAID' WHERE id={cart_id}")
    audit_res = audit.log(f"Cart {cart_id} paid in {currency}")
    return f"{db_res}\n{audit_res}"

def main() -> None:
    container = Container()
    container.register(DatabaseConnectionPool, DatabaseConnectionPool())
    container.register(AuditLogger, AuditLogger())

    # Invoke without manually passing db or audit
    output = container.invoke(checkout_handler, cart_id=9812)
    print(f"Handler output:\n{output}")

if __name__ == "__main__":
    main()

```

Run:

```
uv run python dependency_injection.py
```

Output:

Handler output:

[DB] Executed 'UPDATE carts SET status='PAID' WHERE id=9812'

[AUDIT] Recorded 'Cart 9812 paid in USD'

Case 3: Caller-Context Telemetry and Frame Navigation

Structured loggers and profiling tools frequently need to know who invoked a function without forcing developers to pass `__file__` and `__line__` manually. By inspecting the active execution frame with `inspect.currentframe()` or `sys._getframe()`, we can climb the call stack:

```
# frame_telemetry.py
import inspect
import sys
from typing import Any, Dict

def capture_caller_context(depth: int = 1) -> Dict[str, Any]:
    """Captures caller metadata by walking up the CPython call stack."""
    frame = sys._getframe(depth + 1)
    try:
        return {
            "function": frame.f_code.co_name,
            "filename": frame.f_code.co_filename.split("/")[-1],
            "line": frame.f_lineno,
            "module": frame.f_globals.get("__name__", "<unknown>"),
        }
    finally:
        # Crucial: Delete frame reference to prevent circular garbage collection cycles
        del frame

def emit_telemetry(action: str, metric_value: float) -> None:
    ctx = capture_caller_context(depth=1)
    file_name = ctx["filename"]
    line_no = ctx["line"]
    fn_name = ctx["function"]
    print(f"[{file_name}:{line_no} in {fn_name}()] ACTION='{action}' VALUE={metric_value}")

class PaymentProcessor:
    def charge_card(self, amount: float) -> None:
        emit_telemetry("charge_initiated", amount)
```



```
def main() -> None:
    processor = PaymentProcessor()
    processor.charge_card(149.95)

if __name__ == "__main__":
    main()
```

Run:

```
uv run python frame_telemetry.py
```

Output:

```
[frame_telemetry.py:31 in charge_card()] ACTION='charge_initiated' VALUE=149.95
```

Pitfalls

Pitfall 1: Retaining Frame References and Creating GC Leaks

Frame objects contain references to their local dictionaries (`f_locals`), code objects, and enclosing execution scopes. Storing a frame in an exception trace or local variable without explicitly deleting it creates reference cycles that keep large local objects alive until the next garbage collection cycle:

```
# THE TRAP:
import inspect

def leaky_tracer():
    frame = inspect.currentframe()
    # If 'frame' is saved in a list or closure, all local variables in leaky_tracer
    # remain anchored in RAM!
    return frame.f_back

# THE FIX:
def safe_tracer():
    frame = inspect.currentframe()
    try:
        return frame.f_back.f_lineno if frame and frame.f_back else None
    finally:
        del frame # Break reference cycle immediately
```

Pitfall 2: `inspect.getsource()` on Dynamically Evaluated or C Code

Functions loaded from compiled C-extensions (like `math.sin` or `sys.exit`) or created dynamically via `exec()` / `eval()` do not exist as source files on disk. Calling `inspect.getsource()` on them raises `OSError` or `TypeError`:

```
# THE TRAP:
import math
import inspect

try:
    source = inspect.getsource(math.sin)
except TypeError as err:
    print(f"Caught: {err}")
```

Output:

Caught: <built-in function sin> is not a module, class, method, function, traceback, frame, or

The Fix: Always guard calls to `inspect.getsource()` with `inspect.isbuiltin()` or catch (`OSError`, `TypeError`).

Exercises

1. Build a function `inspect_callable(fn)` that prints whether the callable is a regular function, a coroutine function (`async def`), a generator function (`yield`), or a class constructor.
 2. Implement a validation decorator that inspects keyword-only parameters (`Parameter.KEYWORD_ONLY`) and ensures they are never omitted when calling the function.
 3. Write a dynamic JSON schema generator that converts a function's type annotations and default parameters into an OpenAPI/JSON-Schema-compliant parameter dictionary.
 4. Using `sys._getframe()`, implement an `@auto_logger` decorator that logs the execution time and the name of the function that invoked the decorated function.
 5. Create a command-line dispatcher that inspects an arbitrary function signature and automatically maps CLI strings from `sys.argv[1:]` into typed function arguments (`int`, `float`, `str`, `bool`).
-

Further reading

- Python Documentation: `inspect` — *Inspect live objects*.
- Brett Slatkin: *Effective Python* (Item 48: Validate Subclasses with `__init_subclass__` and Reflection).
- Luciano Ramalho: *Fluent Python* (Chapter 7: Functions as First-Class Objects; Chapter 21: Class Metaprogramming).
- PEP 362: *Function Signature Object*.

Weak References and Garbage Collection (weakref, gc)

Weak References and Garbage Collection

(weakref, gc)

After reading this chapter, you will master CPython's hybrid memory management architecture (reference counting coupled with generational cyclic garbage collection), construct self-pruning caches using `weakref.WeakValueDictionary` and `weakref.WeakKeyDictionary`, guarantee resource reclamation with `weakref.finalize` without relying on hazardous `__del__` methods, and diagnose reference cycles using the `gc` module.

Mental model

CPython manages memory primarily through **reference counting**: every object tracks how many references point to it (`ob_refcnt`). When `ob_refcnt` drops to zero, CPython immediately deallocates the object's memory without waiting:

Reference Counting (Instant Deallocation):

```
ref_a = Node()      Node (refcount: 1)
ref_b = ref_a       Node (refcount: 2)
del ref_a           Node (refcount: 1)
del ref_b           Node (refcount: 0)   Deallocated immediately!
```

However, **circular references** defeat pure reference counting. If Object A references Object B, and Object B references Object A, their reference counts never drop to zero even after all external names are deleted:

Reference Cycle (Leaked Without Cyclic GC):

```
[ Root Namespace ]
    (External variables deleted!)

    refers to
Object A          Object B
(refcnt=1)        (refcnt=1)
    refers to
```

CPython solves this with a secondary **generational garbage collector** (`gc` module) that periodically inspects container objects (Gen 0, Gen 1, Gen 2), detects unreachable islands of cyclic references, and breaks them.

A **weak reference** (`weakref`) allows you to reference an object without incrementing its `ob_refcnt`. If all strong references disappear, the object is immediately deallocated, and the weak reference automatically invalidates itself:

Weak Reference Architecture:

```
strong_ref = Node()  (ob_refcnt = 1)  [ Target Node ]

weak_ref = weakref.ref(strong_ref)      (ob_refcnt remains 1!)

del strong_ref      Target Node deallocated instantly!
weak_ref()          Returns None (dead reference)
```

Minimal example

Save as `weakref_overview.py`:

```
# weakref_overview.py
import weakref

class HeavyModel:
    def __init__(self, model_id: str) -> None:
        self.model_id = model_id

    def __repr__(self) -> str:
        return f"HeavyModel({self.model_id})"

def main() -> None:
    # 1. Create a target object with one strong reference
    model = HeavyModel("transformer-v3")
    print(f"Created object: {model}")

    # 2. Create a weak reference; reference count is unchanged
    weak_handle = weakref.ref(model)
    print(f"Weak handle before deletion: {weak_handle}")
    print(f"Dereferenced value: {weak_handle()}")

    # 3. Delete the only strong reference
    print("\nDeleting strong reference 'model'...")
    del model

    # 4. The target object is immediately reclaimed; weak handle returns None
    print(f"Weak handle after deletion: {weak_handle}")
    print(f"Dereferenced value: {weak_handle()}")
```

```
if __name__ == "__main__":
    main()
```

Run via `uv run python weakref_overview.py`:

```
Created object: HeavyModel(transformer-v3)
Weak handle before deletion: <weakref at 0x...; to 'HeavyModel' at 0x...>
Dereferenced value: HeavyModel(transformer-v3)

Deleting strong reference 'model'...
Weak handle after deletion: <weakref at 0x...; dead>
Dereferenced value: None
```

Worked examples

Case 1: Self-Pruning In-Memory Entity Cache with WeakValueDictionary

Standard dictionaries keep cached objects alive forever unless explicitly deleted, leading to silent memory leaks in long-running services. A `WeakValueDictionary` automatically evicts entries as soon as no external client references the cached instance:

```
# entity_cache.py
import weakref

class TenantConfiguration:
    def __init__(self, tenant_id: str, quota: int) -> None:
        self.tenant_id = tenant_id
        self.quota = quota

    def __repr__(self) -> str:
        return f"TenantConfiguration(id={self.tenant_id}, quota={self.quota})"

class TenantRegistry:
    def __init__(self) -> None:
        # Cache values weakly; entries disappear when callers drop their reference
        self._cache: weakref.WeakValueDictionary[str, TenantConfiguration] = (
            weakref.WeakValueDictionary()
        )

    def get_or_load(self, tenant_id: str) -> TenantConfiguration:
        config = self._cache.get(tenant_id)
        if config is None:
            print(f" [CACHE MISS] Fetching config for '{tenant_id}' from database...")
            config = TenantConfiguration(tenant_id=tenant_id, quota=10_000)
```

```

        self._cache[tenant_id] = config
    else:
        print(f"  [CACHE HIT ] Returning cached config for '{tenant_id}'")
        return config

def active_cached_keys(self) -> list[str]:
    return list(self._cache.keys())

def main() -> None:
    registry = TenantRegistry()

    # Client A requests tenant-01
    client_a_handle = registry.get_or_load("tenant-01")
    print(f"Active cache keys: {registry.active_cached_keys()}")

    # Client B requests tenant-01 (cache hit)
    client_b_handle = registry.get_or_load("tenant-01")

    # Client A drops its reference; Client B still holds one
    print("\nClient A releasing reference...")
    del client_a_handle
    print(f"Active cache keys: {registry.active_cached_keys()}")

    # Client B drops its reference; no strong references remain!
    print("Client B releasing reference...")
    del client_b_handle
    print(f"Active cache keys after all clients disconnect: {registry.active_cached_keys()}")

    # Next request triggers a fresh load from the database
    print("\nSubsequent request:")
    _ = registry.get_or_load("tenant-01")

if __name__ == "__main__":
    main()

```

Run:

```
uv run python entity_cache.py
```

Output:

```

[CACHE MISS] Fetching config for 'tenant-01' from database...
Active cache keys: ['tenant-01']
[CACHE HIT ] Returning cached config for 'tenant-01'

Client A releasing reference...
Active cache keys: ['tenant-01']

```


Client B releasing reference...

Active cache keys after all clients disconnect: []

Subsequent request:

[CACHE MISS] Fetching config for 'tenant-01' from database...

Case 2: Deterministic OS Resource Cleanup with `weakref.finalize`

Relying on `__del__` for resource cleanup is notoriously fragile: exceptions raised inside `__del__` are silently swallowed, execution timing during interpreter shutdown is undefined, and resurrection bugs can corrupt state. `weakref.finalize` provides a safe, deterministic alternative:

```
# safe_resource_reaper.py
import os
import tempfile
import weakref

def _release_temp_file(filepath: str) -> None:
    """Standalone cleanup function. Must NOT hold reference to the owner object."""
    if os.path.exists(filepath):
        os.remove(filepath)
        print(f"[CLEANUP] Deleted temporary file from disk: {filepath}")

class EphemeralBuffer:
    def __init__(self, prefix: str) -> None:
        fd, self.path = tempfile.mkstemp(prefix=prefix)
        os.close(fd)
        print(f"[INIT] Allocated scratch file: {self.path}")

        # Register finalizer without passing 'self' to the callback
        self._finalizer = weakref.finalize(self, _release_temp_file, self.path)

    @property
    def is_alive(self) -> bool:
        return self._finalizer.alive

    def close(self) -> None:
        """Manual explicit cleanup. Can be called safely multiple times."""
        self._finalizer()

def main() -> None:
    # 1. Automatic cleanup when object goes out of scope
    print("--- Scope 1: Implicit deallocation ---")
    def worker():
```

```

        buf = EphemeralBuffer("task_worker_")
        print(f"Buffer is active: {buf.is_alive}")

    worker()
    print("Worker function exited.\n")

# 2. Explicit manual cleanup
print("--- Scope 2: Explicit manual close ---")
buf2 = EphemeralBuffer("manual_task_")
print(f"Buffer alive before close: {buf2.is_alive}")
buf2.close()
print(f"Buffer alive after close: {buf2.is_alive}")
# Calling close again is a safe no-op
buf2.close()

if __name__ == "__main__":
    main()

```

Run:

```
uv run python safe_resource_reaper.py
```

Output:

```

--- Scope 1: Implicit deallocation ---
[INIT] Allocated scratch file: /tmp/task_worker_...
Buffer is active: True
[CLEANUP] Deleted temporary file from disk: /tmp/task_worker_...
Worker function exited.

--- Scope 2: Explicit manual close ---
[INIT] Allocated scratch file: /tmp/manual_task_...
Buffer alive before close: True
[CLEANUP] Deleted temporary file from disk: /tmp/manual_task_...
Buffer alive after close: False

```

Case 3: Diagnosing and Breaking Cyclic Reference Leaks with gc

When developing complex graph structures or event subscription architectures, circular references prevent reference counting from freeing memory. The `gc` module exposes tools to inspect and break these cycles:

```

# cycle_diagnostics.py
import gc

```

```

from typing import Optional

class GraphNode:
    def __init__(self, label: str) -> None:
        self.label = label
        self.neighbor: Optional["GraphNode"] = None

    def __repr__(self) -> str:
        return f"GraphNode({self.label})"

def inspect_cycle() -> None:
    # 1. Create a circular reference inside local scope
    node_a = GraphNode("NodeA")
    node_b = GraphNode("NodeB")
    node_a.neighbor = node_b
    node_b.neighbor = node_a

    print("Created circular reference: A <  B")

    # 2. Inspect referrers
    referrers = gc.get_referrers(node_a)
    referrer_types = [type(r).__name__ for r in referrers]
    print(f"Referrers to NodeA: {referrer_types}")

def main() -> None:
    gc.collect()

    inspect_cycle()

    # When inspect_cycle() returns, its stack frame is popped.
    # NodeA and NodeB have refcount 1 due to each other, but cannot be reached!
    print("\nLocal frame popped. Cycle is now orphaned.")

    # 3. Trigger explicit cyclic garbage collection
    collected = gc.collect()
    print(f"gc.collect() successfully reaped {collected} unreachable cyclic objects.")

if __name__ == "__main__":
    main()

```

Run:

```
uv run python cycle_diagnostics.py
```

Output:

```
Created circular reference: A <  B
```

Referrers to NodeA: ['GraphNode']

Local frame popped. Cycle is now orphaned.
gc.collect() successfully reaped 2 unreachable cyclic objects.

Pitfalls

Pitfall 1: Attempting to Weakly Reference Built-in Types Directly

CPython's core built-in types (`dict`, `list`, `int`, `str`, `tuple`) optimize memory by omitting the `__weakref__` pointer slot from their C structures. Calling `weakref.ref()` on them raises a `TypeError`:

```
# THE TRAP:
import weakref

raw_data = {"status": "ok", "retries": 3}
try:
    ref = weakref.ref(raw_data)
except TypeError as err:
    print(f"Caught: {err}")
```

Output:

Caught: cannot create weak reference to 'dict' object

The Fix: Subclass the built-in type, or wrap the data in a custom class, which adds the `__weakref__` attribute automatically:

```
# THE FIX:
class TrackedDict(dict):
    pass

tracked = TrackedDict({"status": "ok"})
valid_ref = weakref.ref(tracked)
print(f"Successfully referenced: {valid_ref()['status']}")
```

Pitfall 2: Capturing self in a Finalizer Callback

Passing a bound method (such as `self.cleanup`) or a closure capturing `self` into `weakref.finalize` creates a strong reference from the finalizer back to the object. Because the finalizer holds `self`, the object never dies, and the finalizer never executes!

```
# THE TRAP (Memory leak!):
class LeakyResource:
    def __init__(self):
        # self.cleanup is a BOUND METHOD that holds a strong reference to self!
        self._finalizer = weakref.finalize(self, self.cleanup)

    def cleanup(self):
        print("Cleaning up...")

# THE FIX:
class SafeResource:
    def __init__(self):
        # Pass a static method or module-level function and only the primitive handle
        self._finalizer = weakref.finalize(self, SafeResource._cleanup_static, "raw_handle")

    @staticmethod
    def _cleanup_static(handle: str):
        print(f"Safely cleaning up {handle}")
```

Exercises

1. Build a parent-child tree node structure where each parent maintains a list of strong references to its children, while each child maintains a `weakref.ref` back to its parent to prevent circular reference memory leaks.
 2. Implement a custom `@memoize_weak` decorator that caches function results using a `WeakValueDictionary`, ensuring that cached results are evicted as soon as caller code drops them.
 3. Use `gc.get_objects()` to write a diagnostic utility that prints the top 5 classes with the highest number of live instances in memory.
 4. Benchmark the latency of creating and destroying 100,000 objects with pure reference counting versus 100,000 objects chained in reference cycles collected by `gc.collect()`.
 5. Implement a `SharedLock` resource that uses `weakref.finalize` to guarantee that if a thread crashes or an object is garbage-collected without explicitly releasing its lock, the lock is automatically freed.
-

Further reading

- Python Documentation: `weakref` — *Weak references*.
- Python Documentation: `gc` — *Garbage Collector interface*.
- Luciano Ramalho: *Fluent Python* (Chapter 6: Object References, Mutability, and Recycling).
- Anthony Shaw: *CPython Internals* (Chapter 10: Memory Management and the Garbage Collector).

Mocking and Test Doubles (unittest.mock)

Mocking and Test Doubles (`unittest.mock`)

After reading this chapter, you will master Python’s testing and isolation toolkit using the standard library `unittest.mock` module, replace unreliable external boundaries (databases, HTTP APIs, file systems, clocks) with robust test doubles, intercept functions and classes using `@patch` and `patch.object`, enforce API contracts and prevent mock interface drift using `autospec=True`, simulate multi-step network failures with `side_effect`, and test asynchronous coroutines using `AsyncMock`.

Mental model

Unit tests should be deterministic, isolated, and fast. However, production code frequently depends on non-deterministic boundaries: remote webhooks, cloud databases, external clock times, or disk storage.

The `unittest.mock` library creates **test doubles**—objects that impersonate real components, record how they were called, and return pre-programmed responses:

Production System Flow

```
Client Code    [ Real Database / Stripe API / SMTP ]
                (Slow, unreliable, requires credentials)
```

Test Substitution

Isolated Unit Test Flow

```
Client Code    [ unittest.mock.Mock ]
                return_value: {'status': 'PAID'}
                side_effect: ConnectionError()
                records: call_count, call_args
```

The Golden Rule of Patching: “Where to Patch”

The single most common bug when using `unittest.mock.patch` is patching the module where a class or function was **defined**, rather than where it was **imported and looked up**:

WRONG:

```
patch('urllib.request.urlopen') # Does NOT affect service.py!
service.py imported: from urllib.request import urlopen (local reference)
```

RIGHT:

```
patch('my_app.service.urlopen') # Patches the reference inside service.py!
```

Minimal example

Save as mock_overview.py:

```
# mock_overview.py
from unittest.mock import Mock, patch

def send_welcome_email(email_client, user_email: str) -> bool:
    """Sends a welcome message and returns True on success."""
    response = email_client.send(
        to=user_email,
        subject="Welcome aboard!",
        body="Thank you for joining our platform.",
    )
    return response.get("delivered", False)

def main() -> None:
    # 1. Instantiate a mock client
    mock_client = Mock()

    # Configure return value
    mock_client.send.return_value = {"delivered": True, "message_id": "msg_9012"}

    # 2. Execute target function under test
    result = send_welcome_email(mock_client, "alice@example.com")
    print(f"Function result: {result}")

    # 3. Assert mock interactions
    print(f"send() was called: {mock_client.send.called}")
    print(f"send() call count: {mock_client.send.call_count}")

    # Verify exact keyword arguments passed
    mock_client.send.assert_called_once_with(
        to="alice@example.com",
        subject="Welcome aboard!",
        body="Thank you for joining our platform.",
    )
```

```

        print("All mock assertions succeeded!")

if __name__ == "__main__":
    main()

```

Run via `uv run python mock_overview.py`:

```

Function result: True
send() was called: True
send() call count: 1
All mock assertions succeeded!

```

Worked examples

Case 1: Simulating Flaky Networks with `side_effect` Retries

A mock's `side_effect` attribute can accept an exception class or instance (to raise errors) or an iterable (to yield different return values on consecutive calls). This enables testing exponential backoff and retry loops:

```

# retry_client_test.py
import time
from unittest.mock import Mock

class TransientNetworkError(Exception):
    pass

def fetch_telemetry_with_retry(api_client, endpoint: str, max_retries: int = 3) -> dict:
    """Fetches data from an API client, retrying up to max_retries on transient errors."""
    for attempt in range(1, max_retries + 1):
        try:
            return api_client.get(endpoint)
        except TransientNetworkError:
            if attempt == max_retries:
                raise
            time.sleep(0.01 * attempt)
    return {}

def test_retry_succeeds_on_third_attempt() -> None:
    mock_api = Mock()

    # Sequence: Fail with error twice, then succeed on third attempt
    mock_api.get.side_effect = [

```

```

        TransientNetworkError("Gateway Timeout (504)"),
        TransientNetworkError("Connection Reset"),
        {"nodes_online": 42, "status": "HEALTHY"},
    ]

    result = fetch_telemetry_with_retry(mock_api, "/api/v1/cluster/status")

    print(f"Payload received: {result}")
    print(f"Total API attempts made: {mock_api.get.call_count}")
    assert mock_api.get.call_count == 3
    print("Test passed: Retry loop successfully recovered on attempt 3!")

if __name__ == "__main__":
    test_retry_succeeds_on_third_attempt()

```

Run:

```
uv run python retry_client_test.py
```

Output:

```

Payload received: {'nodes_online': 42, 'status': 'HEALTHY'}
Total API attempts made: 3
Test passed: Retry loop successfully recovered on attempt 3!

```

Case 2: Preventing Mock Drift with autospec=True

Standard Mock and MagicMock instances dynamically create any attribute or method accessed on them. If production code refactors a method name or changes parameter signatures, tests using vanilla mocks continue to pass silently. `autospec=True` binds the mock directly to the real class specification:

```

# autospec_drift_prevention.py
from unittest.mock import Mock, create_autospec

class BillingGateway:
    def charge_customer(self, customer_id: str, amount_cents: int, currency: str = "USD") ->
        """Charges a registered customer's saved payment method."""
        return f"tx_success_{customer_id}_{amount_cents}"

def test_vanilla_mock_drift() -> None:
    # Vanilla Mock permits non-existent methods and wrong signatures!
    vanilla_mock = Mock()
    vanilla_mock.non_existent_method("foo")

```

```

vanilla_mock.charge_customer("cust_01") # Missing required 'amount_cents'!
print("Vanilla mock silently succeeded despite invalid method names and missing argument")

def test_autospec_drift_detection() -> None:
    # create_autospec introspects BillingGateway methods and signatures
    safe_mock = create_autospec(BillingGateway, instance=True)

    # 1. Calling non-existent method raises AttributeError immediately
    try:
        safe_mock.non_existent_method("foo")
    except AttributeError as err:
        print(f"\nCaught nonexistent method via autospec:\n {err}")

    # 2. Calling with incorrect signature raises TypeError immediately
    try:
        safe_mock.charge_customer("cust_01") # Missing required argument!
    except TypeError as err:
        print(f"\nCaught invalid signature via autospec:\n {err}")

def main() -> None:
    test_vanilla_mock_drift()
    test_autospec_drift_detection()

if __name__ == "__main__":
    main()

```

Run:

```
uv run python autospec_drift_prevention.py
```

Output:

Vanilla mock silently succeeded despite invalid method names and missing arguments.

Caught nonexistent method via autospec:

Mock object has no attribute 'non_existent_method'

Caught invalid signature via autospec:

missing a required argument: 'amount_cents'

Case 3: Testing Asynchronous Event Pipelines with AsyncMock

When testing `asyncio` code, methods that return coroutines cannot be replaced with synchronous Mock objects. `AsyncMock` ensures that calling the mock returns an awaitable coroutine and records await counts:

```

# async_worker_test.py
import asyncio
from unittest.mock import AsyncMock

async def process_queue_batch(consumer, alert_service) -> int:
    """Consumes messages from a stream and forwards alerts asynchronously."""
    messages = await consumer.poll_messages(max_items=10)
    processed_count = 0

    for msg in messages:
        if msg.get("severity") == "CRITICAL":
            await alert_service.dispatch_pager(msg["event"])
            await consumer.ack(msg["offset"])
            processed_count += 1

    return processed_count

async def test_async_worker() -> None:
    mock_consumer = AsyncMock()
    mock_alerts = AsyncMock()

    # Setup async return value for poll_messages
    mock_consumer.poll_messages.return_value = [
        {"offset": 101, "severity": "INFO", "event": "User logged in"},
        {"offset": 102, "severity": "CRITICAL", "event": "DB Connection dropped"},
        {"offset": 103, "severity": "WARN", "event": "High memory consumption"},
    ]

    # Execute async function under test
    count = await process_queue_batch(mock_consumer, mock_alerts)

    print(f"Total processed events: {count}")
    print(f"Consumer ack() awaited count: {mock_consumer.ack.await_count}")
    print(f"Alert dispatch_pager() awaited count: {mock_alerts.dispatch_pager.await_count}")

    # Assert async interactions
    mock_consumer.poll_messages.assert_awaited_once_with(max_items=10)
    mock_alerts.dispatch_pager.assert_awaited_once_with("DB Connection dropped")
    assert mock_consumer.ack.await_count == 3
    print("All async assertions verified!")

if __name__ == "__main__":
    asyncio.run(test_async_worker())

```

Run:

```
uv run python async_worker_test.py
```

Output:

```
Total processed events: 3
Consumer ack() awaited count: 3
Alert dispatch_pager() awaited count: 1
All async assertions verified!
```

Pitfalls

Pitfall 1: Patching the Definition Site Instead of the Lookup Site

If `service.py` contains `from time import time`, patching `time.time` has no effect because `service.py` holds a direct pointer to the function bound during import:

```
# service.py
from time import time

def get_current_epoch() -> float:
    return time()

# test_service.py (THE TRAP):
from unittest.mock import patch

# WRONG: time was imported into service's namespace!
with patch("time.time", return_value=1700000000.0):
    # get_current_epoch() still calls the unpatched original function!
    pass

# THE FIX:
# Patch the symbol where service.py looks it up!
with patch("service.time", return_value=1700000000.0):
    assert get_current_epoch() == 1700000000.0
```

Pitfall 2: Misspelling Assertion Methods (`assert_called_once_with` typo)

Because standard `Mock` instances dynamically create any attribute on demand, misspelling an assertion method name (e.g. `assert_called_once_withh`) does not raise an error—it merely creates a new child mock, silently passing the test!

```
# THE TRAP:
from unittest.mock import Mock

mock_obj = Mock()
# Did you notice the extra 'h'?
mock_obj.assert_called_once_withh("bad_arg") # SILENT PASS! Returns a new Mock object!

# THE FIX:
# Use pytest with unittest.mock or run with PYTHONWARNINGS to catch non-existent mock assert
# or use autospec=True.
```

Exercises

1. Create a unit test using `@patch("os.environ.get")` to verify that a configuration loader returns default fallback values when an environment variable is missing.
 2. Build a test for a database transaction context manager (`with db.transaction(): ...`) using `MagicMock` to ensure `commit()` is called on clean exit and `rollback()` is called on exceptions.
 3. Test a file download function by patching `urllib.request.urlopen` with a mock context manager that streams 1 KB chunk buffers.
 4. Using `side_effect`, implement a mock for a rate-limited API that returns HTTP 429 twice before returning HTTP 200 with valid JSON.
 5. Convert an existing unit test that uses vanilla `Mock` to `create_autospec(TargetClass, instance=True)` and observe which tests break due to out-of-date method signatures.
-

Further reading

- Python Documentation: `unittest.mock` — *Mock object library*.
- Harry Percival & Bob Gregory: *Architecture Patterns with Python* (Chapter 3: Coupling and Abstractions; Mocks vs Adapters).
- Martin Fowler: *Mocks Aren't Stubs* (Fowler's taxonomy of test doubles: Dummy, Fake, Stubs, Mocks).
- Brett Slatkin: *Effective Python* (Item 80: Consider `unittest.mock` for Testing Code with Dependencies).

Binary Data and Network Protocols (struct)

Binary Data and Network Protocols (struct)

After reading this chapter, you will master low-level binary serialization and deserialization in Python using the `struct` module, pack primitive Python types into dense C-compatible byte arrays, control byte endianness and memory alignment, unpack fixed-width binary protocols and telemetry streams, perform zero-copy buffer modifications using `struct.pack_into` and `struct.unpack_from`, and engineer resilient wire framing for high-throughput network applications.

Mental model

Higher-level formats like JSON, XML, and YAML serialize data as UTF-8 human-readable text. While flexible, textual formats incur substantial overhead: a 64-bit integer like 18446744073709551615 takes 20 ASCII bytes plus delimiter bytes, parsing requires string scanning and memory allocation, and floating-point conversions lose IEEE precision.

In contrast, the `struct` module translates directly between Python values and raw binary representations matching C structs:

Textual (JSON):

```
{"id": 42, "temp": 98.6}      25+ ASCII bytes, variable width, slow parsing
```

Binary (struct):

```
struct.pack("!Id", 42, 98.6)
```

4 bytes: uint32 (42) 8 bytes: IEEE double-float (98.6)

Exactly 12 bytes! Constant-time O(1) packing/unpacking, zero allocations.

Endianness & Byte Order Modifiers

When exchanging binary packets across networks or persisting files to disk, you must explicitly specify byte order:

Modifier	Meaning	Size & Alignment	Use Case
@	Native byte order	Native C struct alignment & padding	Internal C interop (default)
=	Native byte order	Standard (packed, no padding)	System-specific serialization

Modifier	Meaning	Size & Alignment	Use Case
<	Little-endian	Standard (packed, no padding)	x86/ARM disk formats, NVMe
>	Big-endian	Standard (packed, no padding)	Network protocols
!	Network byte order	Standard (packed, big-endian)	RFC standard for all TCP/UDP headers

Minimal example

Save as `struct_overview.py`:

```
# struct_overview.py
import struct

def main() -> None:
    # Format String:
    #   '!' = Network byte order (big-endian)
    #   'H' = unsigned 16-bit short (2 bytes)
    #   'I' = unsigned 32-bit int (4 bytes)
    #   'f' = single-precision float (4 bytes)
    #   '6s' = 6-byte string (6 bytes)
    FMT = "!HIf6s"

    expected_size = struct.calcsize(FMT)
    print(f"Calculated struct size: {expected_size} bytes\n")

    # 1. Pack Python values into raw binary bytes
    packet_type = 101
    sequence_id = 98452
    latency_ms = 4.25
    device_tag = b"NODE01"

    binary_payload = struct.pack(FMT, packet_type, sequence_id, latency_ms, device_tag)
    print(f"Packed bytes ({len(binary_payload)} bytes): {binary_payload.hex()}")

    # 2. Unpack raw bytes back into Python primitives
    unpacked_type, unpacked_seq, unpacked_lat, unpacked_tag = struct.unpack(FMT, binary_payload)
    print("\nUnpacked fields:")
    print(f"  Packet Type : {unpacked_type}")
    print(f"  Sequence ID : {unpacked_seq}")
    print(f"  Latency (ms): {unpacked_lat:.2f}")
```

```

    print(f" Device Tag : {unpacked_tag.decode()}")

if __name__ == "__main__":
    main()

```

Run via `uv run python struct_overview.py`:

Calculated struct size: 16 bytes

Packed bytes (16 bytes): 006500018094408800004e4f44453031

Unpacked fields:

```

Packet Type : 101
Sequence ID : 98452
Latency (ms): 4.25
Device Tag  : NODE01

```

Worked examples

Case 1: Custom Binary RPC Wire Framing with Checksum Validation

In custom network protocols, every message starts with a fixed-size header specifying magic bytes, sequence IDs, flags, and the length of the following dynamic payload:

```

# binary_rpc_framer.py
import struct
import zlib
from typing import NamedTuple

class Packet(NamedTuple):
    sequence: int
    flags: int
    payload: bytes

# Header: 4-byte Magic, 4-byte Seq, 4-byte Payload Length, 2-byte Flags, 4-byte CRC32
# Total header length: 18 bytes
HEADER_FORMAT = "!4sIIHI"
HEADER_SIZE = struct.calcsize(HEADER_FORMAT)
MAGIC_BYTES = b"RPC1"

def encode_packet(sequence: int, flags: int, payload: bytes) -> bytes:
    """Encodes a payload into a binary wire packet with CRC32 integrity check."""
    crc = zlib.crc32(payload)

```

```

header = struct.pack(HEADER_FORMAT, MAGIC_BYTES, sequence, len(payload), flags, crc)
return header + payload

def decode_packet(raw_bytes: bytes) -> Packet:
    """Decodes and validates a binary wire packet."""
    if len(raw_bytes) < HEADER_SIZE:
        raise ValueError(f"Incomplete packet: expected at least {HEADER_SIZE} bytes")

    magic, sequence, length, flags, expected_crc = struct.unpack(
        HEADER_FORMAT, raw_bytes[:HEADER_SIZE]
    )

    if magic != MAGIC_BYTES:
        raise ValueError(f"Protocol violation: invalid magic bytes {magic!r}")

    payload = raw_bytes[HEADER_SIZE : HEADER_SIZE + length]
    if len(payload) != length:
        raise ValueError(f"Truncated payload: expected {length} bytes, got {len(payload)}")

    actual_crc = zlib.crc32(payload)
    if actual_crc != expected_crc:
        raise ValueError(f"Corrupted packet! Checksum mismatch: 0x{actual_crc:08X} != 0x{exp

    return Packet(sequence=sequence, flags=flags, payload=payload)

def main() -> None:
    # 1. Encode message
    msg = b'{"command": "ACQUIRE_LOCK", "resource": "db_writer"}'
    wire_data = encode_packet(sequence=4201, flags=0x0001, payload=msg)
    print(f"Encoded wire packet size: {len(wire_data)} bytes (Payload: {len(msg)} bytes)")

    # 2. Decode valid packet
    pkt = decode_packet(wire_data)
    print(f"Decoded successfully: seq={pkt.sequence}, flags=0x{pkt.flags:04X}, body={pkt.pay

    # 3. Simulate bit-flip corruption in transmission
    corrupted_data = bytearray(wire_data)
    corrupted_data[-1] ^= 0xFF # Flip bits in last payload byte
    try:
        decode_packet(bytes(corrupted_data))
    except ValueError as err:
        print(f"\nCaught transmission corruption:\n {err}")

if __name__ == "__main__":
    main()

```

Run:

```
uv run python binary_rpc_framer.py
```

Output:

Encoded wire packet size: 70 bytes (Payload: 52 bytes)

Decoded successfully: seq=4201, flags=0x0001, body={"command": "ACQUIRE_LOCK", "resource": "db_

Caught transmission corruption:

Corrupted packet! Checksum mismatch: 0x... != 0x...

Case 2: High-Throughput Stream Parsing with `struct.iter_unpack`

When parsing continuous streams of sensor records or financial ticks, slicing strings creates millions of ephemeral Python objects that overwhelm the garbage collector. `struct.iter_unpack` iterates directly through a continuous memory buffer:

```
# telemetry_stream_parser.py
import struct
import time

# Record Format:
#   '!' = Big-endian
#   'I' = uint32 timestamp (seconds)
#   'H' = uint16 sensor_id
#   'f' = float32 reading
RECORD_FMT = "!IHf"
RECORD_SIZE = struct.calcsize(RECORD_FMT)

def generate_telemetry_batch(count: int = 4) -> bytes:
    """Simulates a raw binary sensor telemetry stream received over UDP."""
    base_time = 1773000000
    buffer = bytearray()
    for i in range(count):
        buffer.extend(struct.pack(RECORD_FMT, base_time + i, 100 + i, 23.5 + (i * 0.4)))
    return bytes(buffer)

def main() -> None:
    raw_telemetry = generate_telemetry_batch(4)
    print(f"Stream size: {len(raw_telemetry)} bytes ({len(raw_telemetry) // RECORD_SIZE} records)")

    print("Streaming records via iter_unpack (zero intermediate slices):")
    for ts, sensor_id, temp in struct.iter_unpack(RECORD_FMT, raw_telemetry):
        print(f"  Timestamp: {ts} | Sensor: {sensor_id} | Temperature: {temp:5.2f} C")
```

```
if __name__ == "__main__":
    main()
```

Run:

```
uv run python telemetry_stream_parser.py
```

Output:

Stream size: 40 bytes (4 records)

Streaming records via iter_unpack (zero intermediate slices):

```
Timestamp: 1773000000 | Sensor: 100 | Temperature: 23.50 C
Timestamp: 1773000001 | Sensor: 101 | Temperature: 23.90 C
Timestamp: 1773000002 | Sensor: 102 | Temperature: 24.30 C
Timestamp: 1773000003 | Sensor: 103 | Temperature: 24.70 C
```

Case 3: Zero-Copy In-Place Buffer Updates with pack_into and unpack_from

When writing packet routers or video transcoders, allocating new bytes objects on every modification degrades throughput. `struct.pack_into` and `struct.unpack_from` modify pre-allocated mutable buffers (`bytearray` or `memoryview`) directly:

```
# zero_copy_packet_rewriter.py
import struct

# Simulated IPv4 Header Fragment:
# Offset 0: Version & IHL (1B)
# Offset 8: TTL (1B)
# Offset 12: Source IP (4B)
# Offset 16: Destination IP (4B)

def main() -> None:
    # Allocate a mutable 20-byte packet buffer
    packet_buf = bytearray(20)

    # 1. Write headers directly at specific offsets
    struct.pack_into("!B", packet_buf, 0, 0x45)      # IPv4, 5 words
    struct.pack_into("!B", packet_buf, 8, 64)        # TTL = 64
    struct.pack_into("!4B", packet_buf, 12, 192, 168, 1, 10) # Src: 192.168.1.10
    struct.pack_into("!4B", packet_buf, 16, 10, 0, 0, 1)   # Dst: 10.0.0.1

    print(f"Initial Packet Hex: {packet_buf.hex()}")
```

```

# 2. Read fields directly from offset without slicing
ttl = struct.unpack_from("!B", packet_buf, 8)[0]
src_ip = ".".join(str(b) for b in struct.unpack_from("!4B", packet_buf, 12))
print(f"Read before NAT: TTL={ttl}, Src={src_ip}")

# 3. Apply NAT translation: rewrite Source IP and decrement TTL in place!
struct.pack_into("!B", packet_buf, 8, ttl - 1)
struct.pack_into("!4B", packet_buf, 12, 172, 16, 0, 99)

# 4. Verify in-place mutation
new_ttl = struct.unpack_from("!B", packet_buf, 8)[0]
new_src_ip = ".".join(str(b) for b in struct.unpack_from("!4B", packet_buf, 12))
print(f"\nRead after NAT : TTL={new_ttl}, Src={new_src_ip}")
print(f"Mutated Packet Hex: {packet_buf.hex()}")

if __name__ == "__main__":
    main()

```

Run:

```
uv run python zero_copy_packet_rewriter.py
```

Output:

```

Initial Packet Hex: 450000000000000040000000c0a8010a0a000001
Read before NAT: TTL=64, Src=192.168.1.10

Read after NAT : TTL=63, Src=172.16.0.99
Mutated Packet Hex: 450000000000000003f000000ac1000630a000001

```

Pitfalls

Pitfall 1: Relying on Native @ Alignment for Network or Disk Formats

By default, omitting the byte order character or using `@` enables native C-compiler struct padding. On 64-bit x86/ARM systems, fields are padded with invisible filler bytes to align with 4- or 8-byte boundaries. If sent over the network, different architectures misinterpret the offsets!

```

# THE TRAP:
import struct

# Native format without explicit endianness prefix:
native_data = struct.pack("ci", b"A", 42)
print("Native size with C padding:", len(native_data))

```

```
# THE FIX:
# Always use '!' or '<' to enforce packed, non-padded representation:
standard_data = struct.pack("!ci", b"A", 42)
print("Standard packed size      :", len(standard_data))
```

Output:

```
Native size with C padding: 8
Standard packed size      : 5
```

Pitfall 2: String Length Mismatches in s Format Strings

In `struct` format strings, `8s` specifies a fixed-length 8-byte buffer. If the Python byte string is shorter than 8 bytes, `struct.pack` automatically pads it with null bytes (`\x00`). If longer, it silently **truncates** the string, discarding trailing data without error!

```
# THE TRAP:
import struct

long_tag = b"ENTERPRISE_SERVER"
truncated = struct.pack("!8s", long_tag)
print("Truncated bytes:", truncated) # Silent data loss: only b'ENTERPRI'!
```

Output:

```
Truncated bytes: b'ENTERPRI'
```

The Fix: Always validate `len(byte_string) <= max_length` before calling `struct.pack`, or use variable-length prefixes like length-prefixed strings (`!H<len>s`).

Exercises

1. Write an encoder and decoder for a binary key-value protocol where keys are length-prefixed strings (`uint8` length followed by bytes) and values are length-prefixed blobs (`uint32` length followed by bytes).
2. Use `struct.unpack_from()` to parse the header of a WAV audio file or BMP image, extracting width, height, and bits-per-pixel metadata.
3. Benchmark the throughput of serializing 1,000,000 numeric tuples (`int`, `float`, `int`) using `json.dumps()` versus `struct.pack("!IfI", ...)`.

4. Implement a binary ring-buffer logger using `struct.pack_into` and a fixed-size `bytearray(65536)`.
 5. Build a DNS response parser that unpacks DNS header flags (QR, Opcode, AA, TC, RD, RA, RCODE) using bitwise masks and `struct.unpack("!HHHHHH", header)`.
-

Further reading

- Python Documentation: `struct` — *Interpret bytes as packed binary data*.
- David Beazley: *Python Cookbook* (Recipe 6.11: Reading and Writing Binary Arrays of Structures).
- RFC 791: *Internet Protocol DARPA Internet Program Protocol Specification* (Binary Packet Structure).
- W. Richard Stevens: *TCP/IP Illustrated, Volume 1* (Network Byte Order and Framing).

Object Serialization and Persistent Storage (pickle, shelve)

Object Serialization and Persistent Storage (pickle, shelve)

After reading this chapter, you will master deep Python object graph serialization using `pickle`, understand the binary pickle protocol and opcode execution model, customize serialization lifecycles using `__getstate__` and `__setstate__` to exclude ephemeral runtime resources (locks, sockets, file descriptors), construct secure unpickling firewalls that block arbitrary remote code execution, and leverage `shelve` for zero-setup, persistent key-value caching on disk.

Mental model

Higher-level formats like JSON and TOML serialize only primitive data types (`dict`, `list`, `str`, `int`, `float`, `bool`, `None`). They cannot represent custom classes, cyclical object graphs (where an object references itself), `datetime` instances, or method bindings without manual encoders and decoders.

The `pickle` module converts arbitrary, deeply nested Python object graphs into a compact stream of byte opcodes:

```
Python Heap Memory (Arbitrary Object Graph)
Node A      Node B      Node A (Cyclic Reference!)
UserSession(id=42, roles={'admin'}, created=datetime)
```

```
pickle.dumps(obj)
```

```
Pickle Byte Stream (Opcode Protocol)
\x80\x05\x95... (Interpreted by Python Pickle VM)
```

```
pickle.loads(bytes)
```

```
Reconstructed Heap Object Graph
(Exact duplicate with preserved object identity & cycles)
```

Security Warning: Never Unpickle Untrusted Data

`pickle` is **not secure against maliciously crafted payloads**. The unpickling process executes instructions that can instantiate any Python class and invoke arbitrary callables (like `os.system`). Never deserialize pickled blobs received over untrusted networks without cryptographic signatures (`hmac`) or restricted unpickling guards.

Minimal example

Save as `pickle_overview.py`:

```
# pickle_overview.py
from datetime import datetime, timezone
import pickle

class ServiceNode:
    def __init__(self, node_id: str, host: str) -> None:
        self.node_id = node_id
        self.host = host
        self.registered_at = datetime.now(timezone.utc)
        self.neighbors: list["ServiceNode"] = []

    def __repr__(self) -> str:
        return f"ServiceNode({self.node_id}, host={self.host}, neighbors={len(self.neighbors)})"

def main() -> None:
    # 1. Construct a cyclic graph: Node 1 < Node 2
    node1 = ServiceNode("node-01", "10.0.0.1")
    node2 = ServiceNode("node-02", "10.0.0.2")
    node1.neighbors.append(node2)
    node2.neighbors.append(node1)

    print(f"Original: {node1}")
    print(f"Cyclic reference intact: {node1.neighbors[0].neighbors[0] is node1}")

    # 2. Serialize complete cyclic graph to bytes
    serialized_bytes = pickle.dumps(node1, protocol=pickle.HIGHEST_PROTOCOL)
    print(f"\nSerialized binary size: {len(serialized_bytes)} bytes")

    # 3. Deserialize back into independent object graph
    restored_node1 = pickle.loads(serialized_bytes)
    print(f"\nRestored: {restored_node1}")
    print(f"Restored cyclic reference intact: {restored_node1.neighbors[0].neighbors[0] is restored_node1}")
    print(f"Identity check with original: {restored_node1 is not node1}")
```

```
if __name__ == "__main__":  
    main()
```

Run via `uv run python pickle_overview.py`:

Original: `ServiceNode(node-01, host=10.0.0.1, neighbors=1)`

Cyclic reference intact: `True`

Serialized binary size: 197 bytes

Restored: `ServiceNode(node-01, host=10.0.0.1, neighbors=1)`

Restored cyclic reference intact: `True`

Identity check with original: `True`

Worked examples

Case 1: Sandboxed Deserialization with SafeUnpickler

When accepting serialized blobs internally across microservices or worker processes, overriding `pickle.Unpickler.find_class` blocks arbitrary class execution by enforcing an explicit allowlist:

```
# safe_unpickler.py  
import io  
import pickle  
from typing import Any  
  
class SafeUnpickler(pickle.Unpickler):  
    """Custom Unpickler that permits only explicitly allowlisted classes."""  
  
    ALLOWED_CLASSES = {  
        ("builtins", "dict"),  
        ("builtins", "list"),  
        ("builtins", "set"),  
        ("builtins", "int"),  
        ("builtins", "float"),  
        ("builtins", "str"),  
        ("builtins", "bool"),  
    }  
  
    def find_class(self, module: str, name: str) -> Any:  
        if (module, name) in self.ALLOWED_CLASSES:  
            return super().find_class(module, name)  
        raise pickle.UnpicklingError(f"Security Alert: Disallowed class '{module}.{name}'")
```

```

def safe_loads(data: bytes) -> Any:
    return SafeUnpickler(io.BytesIO(data)).load()

# Malicious exploit class attempting remote command execution
class MaliciousPayload:
    def __reduce__(self):
        import os
        return (os.system, ("echo COMPROMISED",))

def main() -> None:
    # 1. Legitimate data succeeds
    safe_data = {"tenant": "acme-corp", "tier": "enterprise", "features": ["sso", "audit"]}
    safe_blob = pickle.dumps(safe_data)
    restored = safe_loads(safe_blob)
    print(f"Safe deserialization succeeded: {restored}")

    # 2. Malicious payload is blocked before execution
    exploit_blob = pickle.dumps(MaliciousPayload())
    try:
        safe_loads(exploit_blob)
    except pickle.UnpicklingError as err:
        print(f"\nExploit successfully neutralized:\n {err}")

if __name__ == "__main__":
    main()

```

Run:

```
uv run python safe_unpickler.py
```

Output:

```
Safe deserialization succeeded: {'tenant': 'acme-corp', 'tier': 'enterprise', 'features': ['sso', 'audit']}
```

```
Exploit successfully neutralized:
Security Alert: Disallowed class 'posix.system'
```

Case 2: Persistent Disk State with `shelve`

The `shelve` module provides a persistent dictionary on disk backed by a lightweight DBM database. Keys must be strings, and values are automatically pickled and unpickled transparently:

```

# persistent_shelve_store.py
import shelve
import tempfile
from pathlib import Path

def main() -> None:
    with tempfile.TemporaryDirectory() as tmpdir:
        db_path = str(Path(tmpdir) / "app_state.db")

        # 1. Open shelf and persist complex objects
        with shelve.open(db_path) as db:
            db["cluster_settings"] = {"env": "production", "max_workers": 16}
            db["node_inventory"] = ["node-alpha", "node-beta"]
            print("Persisted initial state to disk.")

        # 2. Re-open in a separate session and read back
        with shelve.open(db_path) as db:
            print("\nRestored from disk:")
            print(f"  Settings: {db['cluster_settings']}")
            print(f"  Nodes    : {db['node_inventory']}")

        # 3. In-place mutations require writeback=True!
        with shelve.open(db_path, writeback=True) as db:
            # Append directly to the stored list
            db["node_inventory"].append("node-gamma")

        # 4. Verify in-place update persisted
        with shelve.open(db_path) as db:
            print(f"\nNodes after writeback update: {db['node_inventory']}")

if __name__ == "__main__":
    main()

```

Run:

```
uv run python persistent_shelve_store.py
```

Output:

Persisted initial state to disk.

Restored from disk:

```
Settings: {'env': 'production', 'max_workers': 16}
Nodes    : ['node-alpha', 'node-beta']
```

Nodes after writeback update: ['node-alpha', 'node-beta', 'node-gamma']

Case 3: Excluding Transient Resources with `__getstate__` and `__setstate__`

Objects containing unpicklable OS handles (such as `threading.Lock`, open sockets, or database connections) crash `pickle.dumps()` unless you implement `__getstate__` to sanitize the serialized state and `__setstate__` to reconstruct transient fields:

```
# custom_pickle_state.py
import pickle
import threading

class StatefulSession:
    def __init__(self, session_id: str, username: str) -> None:
        self.session_id = session_id
        self.username = username
        self.login_attempts = 0
        # Ephemeral OS lock (cannot be pickled directly!)
        self._lock = threading.Lock()

    def record_attempt(self) -> None:
        with self._lock:
            self.login_attempts += 1

    def __getstate__(self) -> dict:
        """Custom state extraction: strips non-serializable OS lock."""
        state = self.__dict__.copy()
        del state["_lock"]
        return state

    def __setstate__(self, state: dict) -> None:
        """Custom state restoration: re-initializes fresh OS lock."""
        self.__dict__.update(state)
        self._lock = threading.Lock()

def main() -> None:
    session = StatefulSession("sess-4819", "alice")
    session.record_attempt()
    session.record_attempt()
    print(f"Pre-serialization state: {session.username} (attempts={session.login_attempts})"

    # Serialize object (succeeds without Lock pickling error!)
    serialized = pickle.dumps(session)

    # Deserialize in worker process
    restored = pickle.loads(serialized)
    print(f"Restored session state : {restored.username} (attempts={restored.login_attempts})"

    # Verify restored lock operates normally
```



```

        restored.record_attempt()
        print(f"State after lock usage : attempts={restored.login_attempts}")

if __name__ == "__main__":
    main()

```

Run:

```
uv run python custom_pickle_state.py
```

Output:

```

Pre-serialization state: alice (attempts=2)
Restored session state : alice (attempts=2)
State after lock usage : attempts=3

```

Pitfalls

Pitfall 1: Mutating Mutable Objects in `shelve` Without `writeback=True`

By default, accessing `db["key"]` unpickles a fresh copy of the object into RAM. Modifying that object in-place (e.g. `db["list"].append(x)`) modifies only the temporary copy in memory; `shelve` does NOT detect the change and fails to save it to disk!

```

# THE TRAP:
import shelve

with shelve.open("cache.db") as db:
    db["tags"] = ["prod"]
    db["tags"].append("v2") # BUG: Modified in memory, NEVER written to disk!

with shelve.open("cache.db") as db:
    print(db["tags"]) # Prints ['prod'], 'v2' was lost!

# THE FIX:
# Option A: Re-assign explicitly
with shelve.open("cache.db") as db:
    tags = db["tags"]
    tags.append("v2")
    db["tags"] = tags # Explicit assignment triggers disk write

# Option B: Open with writeback=True
with shelve.open("cache.db", writeback=True) as db:
    db["tags"].append("v3") # Automatically synchronized on close

```

Pitfall 2: Protocol Incompatibility Across Python Versions

Pickle has multiple protocol versions (Protocol 0 through Protocol 5). When saving serialized data that must be read by older Python versions, passing `protocol=pickle.HIGHEST_PROTOCOL` (Protocol 5 in modern Python) causes older versions to fail with `ValueError: unsupported pickle protocol`. For backward compatibility, explicitly specify `protocol=4`.

Exercises

1. Build an atomic file-backed session store that serializes user session objects using `pickle.dump()` to a temporary file and atomically replaces the destination file using `os.replace()`.
 2. Implement a `RestrictedUnpickler` that permits only a custom dataclass and standard collections, rejecting all other types.
 3. Write a benchmarking script comparing the serialization and deserialization speeds of a 10,000-element dictionary across `json`, `pickle` (Protocol 5), and `marshal`.
 4. Implement an LRU cache decorator backed by `shelve` that caches function outputs across terminal restarts.
 5. Create a class that implements `__reduce__` to serialize an instance into an importable module path and a constructor argument tuple.
-

Further reading

- Python Documentation: `pickle` — *Python object serialization*.
- Python Documentation: `shelve` — *Python object persistence*.
- David Beazley: *Python Cookbook* (Recipe 5.21: Serializing Python Objects).
- PEP 574: *Pickle protocol 5 with out-of-band data buffers*.

Typing and Testing

Modern Type Hints and Static Analysis

Modern Type Hints and Static Analysis

After reading this chapter, you will master Python’s modern static type system, implement generic types with `typing.Generic`, define structural interfaces with `typing.Protocol` (duck typing formalized), leverage `Literal`, `Union` (`|`), and `Self`, and configure strict static analysis with `mypy` and `pyright`.

Mental model

Python type hints (PEP 484, PEP 604, PEP 695) do not alter runtime execution or impose execution overhead. They provide formal machine-readable contracts analyzed by static type checkers before code is deployed:

Development & CI Workflow:

Python Source Code (with type hints)

1. Static Type Checker (`mypy` / `pyright`) Reports bugs, mismatches, None hazards
2. CPython Runtime VM Erases types, executes at full speed

Nominal Subtyping vs Structural Subtyping (Protocol)

- **Nominal Subtyping** (`class Sub(Parent)`): Requires an explicit inheritance declaration.
 - **Structural Subtyping (Protocol)**: Formalizes Python’s “duck typing” philosophy: if an object has the required methods and attributes, it satisfies the type without needing to inherit from the protocol.
-

Minimal example

Save as `typing_protocols.py`:

```
# typing_protocols.py
from typing import Protocol, runtime_checkable

@runtime_checkable
class Renderable(Protocol):
```

```

    """Structural protocol: Any class with a render() -> str method satisfies this."""
    def render(self) -> str:
        ...

class ServiceCard:
    """Implements Renderable implicitly WITHOUT inheriting from it."""
    def __init__(self, name: str, port: int) -> None:
        self.name = name
        self.port = port

    def render(self) -> str:
        return f"<ServiceCard: {self.name}:{self.port}>"

def display_dashboard_item(item: Renderable) -> None:
    print("Dashboard Render:", item.render())

def main() -> None:
    card = ServiceCard("auth-service", 8080)

    # Static type checkers and runtime_checkable verify compatibility
    display_dashboard_item(card)
    print(f"Is ServiceCard an instance of Renderable? {isinstance(card, Renderable)}")

if __name__ == "__main__":
    main()

```

Run via `uv run python typing_protocols.py`:

```

Dashboard Render: <ServiceCard: auth-service:8080>
Is ServiceCard an instance of Renderable? True

```

Worked examples

Case 1: Generic Data Containers (`typing.Generic`)

Generics allow functions and classes to operate over arbitrary types while preserving strict type safety:

```

# generic_repository.py
from typing import Generic, TypeVar

T = TypeVar("T")

```

```

class KeyValueCache(Generic[T]):
    def __init__(self) -> None:
        self._store: dict[str, T] = {}

    def put(self, key: str, value: T) -> None:
        self._store[key] = value

    def get(self, key: str) -> T | None:
        return self._store.get(key)

if __name__ == "__main__":
    # Cache parameterized for integers
    int_cache: KeyValueCache[int] = KeyValueCache()
    int_cache.put("cpu_max", 95)
    print("Retrieved integer metric:", int_cache.get("cpu_max"))

    # Cache parameterized for strings
    str_cache: KeyValueCache[str] = KeyValueCache()
    str_cache.put("active_cluster", "us-east-1a")
    print("Retrieved string metric :", str_cache.get("active_cluster"))

```

Run:

```
uv run python generic_repository.py
```

Output:

```

Retrieved integer metric: 95
Retrieved string metric : us-east-1a

```

Case 2: Fluent Method Chaining with `typing.Self`

When implementing builder patterns, methods returning `self` should be typed with `typing.Self` (PEP 673) so that subclasses automatically preserve their own derived return type:

```

# query_builder.py
from typing import Self

class QueryBuilder:
    def __init__(self) -> None:
        self._query_parts: list[str] = []

    def select(self, fields: str) -> Self:
        self._query_parts.append(f"SELECT {fields}")
        return self

```

```

def from_table(self, table: str) -> Self:
    self._query_parts.append(f"FROM {table}")
    return self

def limit(self, count: int) -> Self:
    self._query_parts.append(f"LIMIT {count}")
    return self

def build(self) -> str:
    return " ".join(self._query_parts)

if __name__ == "__main__":
    sql = QueryBuilder().select("id, host, status").from_table("nodes").limit(10).build()
    print("Constructed SQL query:")
    print(sql)

```

Run:

```
uv run python query_builder.py
```

Output:

```

Constructed SQL query:
SELECT id, host, status FROM nodes LIMIT 10

```

Pitfalls

Pitfall 1: Confusing `Type[T]` with `T`

- `obj: ServerNode`: Expects an **instance** of `ServerNode`.
- `cls: type[ServerNode]`: Expects the **class itself** (for factory functions or constructor reflection).

Pitfall 2: Using `Union` Instead of the Modern `|` Operator

In modern Python (PEP 604), write `int | str | None` instead of `Union[int, str, Optional[None]]`. The pipe syntax is native and evaluated without importing `Union`.

Exercises

1. Define a Protocol named `Serializable` with a method `to_dict() -> dict[str, object]`. Implement two unrelated classes that satisfy this protocol.
 2. Write a generic function `first_or_default(items: list[T], default: T) -> T` that returns the first item or a default fallback.
 3. Use `Literal["pending", "running", "completed"]` to restrict a function's status parameter to exact allowed strings.
 4. Configure a `[tool.mypy]` section in a `pyproject.toml` enabling `strict = true` and verify code compliance.
-

Further reading

- PEP 484: *Type Hints*.
- PEP 544: *Protocols: Structural subtyping (static duck typing)*.
- PEP 604: *Allow writing union types as $X \mid Y$* .
- PEP 673: *Self Type*.

Unit and Integration Testing with Pytest

Unit and Integration Testing with Pytest

After reading this chapter, you will master testing in Python using `pytest`, write declarative assertions without boilerplate classes, manage test dependencies and teardown with fixtures, parameterize test suites across input matrices with `@pytest.mark.parametrize`, and isolate side effects using `unittest.mock`.

Mental model

Unlike legacy frameworks (`unittest.TestCase`) requiring verbose inheritance hierarchies and specialized assertion methods (`self.assertEqual`), `pytest` intercepts Python's native `assert` statement via AST rewriting to generate comprehensive failure diffs:

Test Execution Pipeline:

`pytest` CLI

Test Discovery

Searches for `test_*.py` and `*_test.py` files

Dependency Injection

Resolves requested fixture parameters from fixture tree

AST Assertion Rewriting

`assert calculated == expected` On failure: prints detailed variable introspection

Teardown

Executes post-yield cleanup in reverse fixture dependency order

Minimal example

Save as `test_metrics_engine.py`:

```
# test_metrics_engine.py
import pytest

def calculate_percentile(values: list[float], percentile: float) -> float:
    """Compute the specified percentile (0.0 - 1.0) of a sorted list."""
```

```

    if not values:
        raise ValueError("Cannot calculate percentile of empty sequence")
    if not (0.0 <= percentile <= 1.0):
        raise ValueError("Percentile must fall between 0.0 and 1.0")

    sorted_vals = sorted(values)
    k = (len(sorted_vals) - 1) * percentile
    f = int(k)
    c = f + 1 if f + 1 < len(sorted_vals) else f
    d = k - f
    return sorted_vals[f] + d * (sorted_vals[c] - sorted_vals[f])

# 1. Parameterized test matrix
@pytest.mark.parametrize(
    "percentile,expected",
    [
        (0.0, 10.0), # Min
        (0.5, 30.0), # Median
        (1.0, 50.0), # Max
    ]
)
def test_percentile_calculations(percentile: float, expected: float) -> None:
    data = [10.0, 20.0, 30.0, 40.0, 50.0]
    result = calculate_percentile(data, percentile)
    assert result == pytest.approx(expected, rel=1e-3)

# 2. Testing error boundaries with pytest.raises
def test_percentile_empty_data_raises() -> None:
    with pytest.raises(ValueError, match="empty sequence"):
        calculate_percentile([], 0.5)

if __name__ == "__main__":
    import sys
    # Direct invocation runner
    pytest.main(["-v", __file__])

```

Run via `uv run python test_metrics_engine.py`:

```

===== test session starts =====
...
test_metrics_engine.py::test_percentile_calculations[0.0-10.0] PASSED [ 25%]
test_metrics_engine.py::test_percentile_calculations[0.5-30.0] PASSED [ 50%]
test_metrics_engine.py::test_percentile_calculations[1.0-50.0] PASSED [ 75%]
test_metrics_engine.py::test_percentile_empty_data_raises PASSED [100%]
===== 4 passed in 0.02s =====

```

Worked examples

Case 1: Fixtures with Setup and Teardown Lifecycles

Fixtures using `yield` execute setup code before the test, suspend execution, pass the resource to the test, and execute cleanup code after the test completes:

```
# test_storage_fixture.py
import pytest
import tempfile
import sqlite3
from pathlib import Path
from collections.abc import Generator

@pytest.fixture
def db_conn() -> Generator[sqlite3.Connection, None, None]:
    """Provide a fresh in-memory database with pre-populated schema."""
    conn = sqlite3.connect(":memory:")
    conn.execute("CREATE TABLE inventory (sku TEXT PRIMARY KEY, qty INT)")
    conn.execute("INSERT INTO inventory VALUES ('SKU-100', 42)")
    conn.commit()

    # Yield control to the test function
    yield conn

    # Teardown code runs after test finishes
    conn.close()

def test_inventory_query(db_conn: sqlite3.Connection) -> None:
    cursor = db_conn.execute("SELECT qty FROM inventory WHERE sku = 'SKU-100'")
    qty = cursor.fetchone()[0]
    assert qty == 42
```

Case 2: Isolating External Dependencies with `unittest.mock`

When testing code that calls external web services, email servers, or payment gateways, use `mock` to isolate the unit under test:

```
# test_auth_client.py
from unittest.mock import patch, MagicMock

class Authenticator:
    def check_remote_token(self, token: str) -> bool:
        # In production: makes a real network HTTP call to an OAuth provider
        raise NotImplementedError("Real network endpoint")
```

```
def grant_access(user_token: str, auth: Authenticator) -> str:
    if auth.check_remote_token(user_token):
        return "ACCESS_GRANTED"
    return "ACCESS_DENIED"

def test_grant_access_with_mock() -> None:
    # Create a mock authenticator
    mock_auth = MagicMock(spec=Authenticator)
    mock_auth.check_remote_token.return_value = True

    status = grant_access("valid_token_string", mock_auth)
    assert status == "ACCESS_GRANTED"
    mock_auth.check_remote_token.assert_called_once_with("valid_token_string")

if __name__ == "__main__":
    test_grant_access_with_mock()
    print("Mock unit test passed successfully!")
```

Run:

```
uv run python test_auth_client.py
```

Output:

```
Mock unit test passed successfully!
```

Pitfalls

Pitfall 1: Mutating Shared Session Fixtures

If a fixture has `scope="session"` or `scope="module"`, all tests share the exact same instance. If Test A mutates that instance, Test B may fail spuriously depending on test execution order. Default to `scope="function"` unless resources are read-only or strictly stateless.

Pitfall 2: Testing Implementation Details Instead of Contracts

Tests that assert every internal private method call (`mock_obj._internal_step.assert_called()`) become brittle and break upon simple refactoring. Test observable inputs and outputs.

Exercises

1. Write a parametrized test suite for an IP address validator testing 5 valid IPs and 5 invalid IPs.
 2. Create a temporary directory fixture using `tempfile.TemporaryDirectory` that creates a test file and verifies deletion on test completion.
 3. Use `unittest.mock.patch("time.time")` to test a rate limiter with simulated deterministically advancing time.
 4. Use `pytest -k` and `pytest -m` to organize tests using custom markers (`@pytest.mark.slow`, `@pytest.mark.integration`).
-

Further reading

- Pytest Documentation: *Fixtures, Marks, and Parametrization*.
- Python Standard Library: `unittest.mock` documentation.
- Brian Okken: *Python Testing with pytest, Second Edition*.

Packaging, Wheel Distribution, and Publishing

Packaging, Wheel Distribution, and Publishing

After reading this chapter, you will master the modern Python packaging standards (PEP 517, PEP 518, PEP 621), author declarative `pyproject.toml` specifications, configure executable CLI console scripts, build standard source distributions (`.tar.gz`) and built wheel packages (`.whl`) with `uv build`, and inspect package distribution metadata.

Mental model

In legacy Python, packaging relied on imperative `setup.py` scripts that executed arbitrary Python code during installation. Modern packaging uses declarative TOML metadata (`pyproject.toml`) and isolated build frontends:

Modern Packaging Architecture:

`pyproject.toml` (PEP 621 Metadata & Build Backend)

`uv build`

Source Distribution (sdist):

`mypkg-0.1.0.tar.gz`
Raw Python source code
`pyproject.toml`

Built Wheel (`.whl`):

`mypkg-0.1.0-py3-none-any.whl`
Pre-built package files
`mypkg-0.1.0.dist-info/`
METADATA
WHEEL
`entry_points.txt`
RECORD (SHA-256 hashes)

A **wheel** (`.whl`) is a ZIP archive formatted with exact installation targets. Installing a wheel is simply an unpack and copy operation into `site-packages` with **zero build step execution**.

Minimal example

Save as `package_builder_demo.py`:

```

# package_builder_demo.py
import tempfile
import zipfile
from pathlib import Path

SAMPLE_PYPROJECT = """
[build-system]
requires = ["hatchling"]
build-backend = "hatchling.build"

[project]
name = "cluster-agent"
version = "1.0.0"
description = "Automated cluster monitoring telemetry agent."
readme = "README.md"
requires-python = ">=3.12"
dependencies = [
    "rich>=13.0.0",
]

[project.scripts]
cluster-agent = "cluster_agent.cli:main"
"""

def generate_package_skeleton(root: Path) -> None:
    (root / "pyproject.toml").write_text(SAMPLE_PYPROJECT.strip())
    (root / "README.md").write_text("# Cluster Agent\nHigh-speed cluster agent.")

    pkg_dir = root / "cluster_agent"
    pkg_dir.mkdir()
    (pkg_dir / "__init__.py").write_text('__version__ = "1.0.0"\n')
    (pkg_dir / "cli.py").write_text("""
def main() -> None:
    print("Cluster agent running.")
""")

def main() -> None:
    with tempfile.TemporaryDirectory() as tmpdir:
        root = Path(tmpdir)
        generate_package_skeleton(root)
        print("Generated modern package skeleton with pyproject.toml:")
        for path in sorted(root.rglob("*")):
            if path.is_file():
                print(f"  - {path.relative_to(root)}")

if __name__ == "__main__":
    main()

```

Run via `uv run python package_builder_demo.py`:

Generated modern package skeleton with `pyproject.toml`:

- README.md
 - cluster_agent/__init__.py
 - cluster_agent/cli.py
 - pyproject.toml
-

Worked examples

Case 1: Defining CLI Console Scripts via `[project.scripts]`

When users install your package, they often want a global command-line command (`my-tool`) available in their shell. Modern `pyproject.toml` declares this declaratively:

```
[project.scripts]
# CLI command name = "module_path:function_to_call"
infra-cli = "infra_pkg.cli:main_entrypoint"
```

When installed, `pip` or `uv` automatically generates an executable binary wrapper in the environment's `bin/` directory that executes `main_entrypoint()`.

Case 2: Building Wheels with `uv build`

Executing `uv build` reads `pyproject.toml`, prepares an isolated build environment, and outputs standards-compliant artifacts into `dist/`:

```
# Execute build
uv build

# Output:
# Successfully built dist/cluster_agent-1.0.0.tar.gz
# Successfully built dist/cluster_agent-1.0.0-py3-none-any.whl
```

Case 3: Inspecting Wheel Metadata (`.dist-info`)

Because a `.whl` file is a ZIP archive, you can inspect its metadata directly:

```
# inspect_wheel.py
import zipfile

def inspect_wheel_metadata(wheel_path: str) -> None:
```

```

with zipfile.ZipFile(wheel_path, "r") as zf:
    metadata_files = [name for name in zf.namelist() if name.endswith("METADATA")]
    if metadata_files:
        meta_text = zf.read(metadata_files[0]).decode("utf-8")
        print("--- Package METADATA Content ---")
        for line in meta_text.splitlines()[:10]:
            print(line)

if __name__ == "__main__":
    print("Wheel metadata inspection helper loaded.")

```

Run:

```
uv run python inspect_wheel.py
```

Output:

```
Wheel metadata inspection helper loaded.
```

Pitfalls

Pitfall 1: Relying on Legacy `setup.py`

Modern toolchains (uv, pip, build) deprecate `setup.py` in favor of declarative `pyproject.toml`. Imperative `setup.py` scripts introduce build-time security vulnerabilities and prevent static dependency analysis.

Pitfall 2: Missing Package Data Files

If your package relies on static templates, HTML files, or YAML configs, you must explicitly declare them in your build backend configuration:

```

# Example for hatchling backend:
[tool.hatch.build.targets.wheel]
packages = ["cluster_agent"]
include = [
    "cluster_agent/templates/*.html",
]

```

Otherwise, wheels will package only `.py` source files, causing `FileNotFoundError` at runtime.

Exercises

1. Write a minimal `pyproject.toml` using `hatchling` as the build backend, declaring a console script entry point.
 2. Build a wheel package using `uv build` in a temporary test directory and verify the files generated in `dist/`.
 3. Unzip a `.whl` archive and verify the contents of its `entry_points.txt` and `RECORD` files.
 4. Configure optional dependency groups (`[project.optional-dependencies]`) for `dev` and `test` suites.
-

Further reading

- PEP 517: *A build-system independent format for source trees.*
- PEP 518: *Specifying Minimum Build System Requirements for Python Projects.*
- PEP 621: *Storing project metadata in `pyproject.toml`.*
- Python Packaging Authority (PyPA): *Packaging Python Projects.*

Advanced Type System: PEP 695, ParamSpec, and Type Narrowing

Advanced Type System: PEP 695, ParamSpec, and Type Narrowing

After reading this chapter, you will master Python's cutting-edge type system capabilities, leverage the PEP 695 type parameter syntax (`type`, generic functions, generic classes), author signature-preserving decorators using `ParamSpec` and `Concatenate`, eliminate false positives in type narrowing using `TypeIs` (PEP 742) versus `TypeGuard`, prevent injection vulnerabilities at compile time with `LiteralString`, and enforce exhaustive pattern matching using `Never` and `assert_never`.

Mental model

Static typing in Python has evolved from basic type annotations (PEP 484) to a sophisticated type algebra capable of modeling dynamic Python idioms without sacrificing safety:

Evolution of Python Generics

Python 3.5 - 3.11 (Verbose)	Python 3.12+ PEP 695 (Clean)
<pre>T = TypeVar("T") Vector = list[T] class Box(Generic[T]): ...</pre>	<pre>def first[T](seq: list[T]) type Vector[T] = list[T] class Box[T]: ...</pre>

Type Narrowing: TypeGuard vs TypeIs

When inspecting dynamic types at runtime, standard `isinstance()` checks narrow unions automatically. However, when factoring logic into reusable helper functions, standard return types (`bool`) lose type information.

Python provides two narrowing constructs: 1. **`TypeGuard[T]` (PEP 647)**: Narrows *only* when returning `True`. In the `else` branch, the type is **not** narrowed. 2. **`TypeIs[T]` (PEP 742 - Python 3.13+)**: True bidirectional narrowing. If `is_str(x)` returns `True`, `x` is `str`. If `False`, `str` is **removed** from the union in the `else` branch.

```
TypeIs Narrowing Matrix (x: int | str):
    if is_str(x):
        x is narrowed to str!
    else:
        x is narrowed to int! (TypeGuard leaves x as int | str)
```

Minimal example

Save as pep695_overview.py:

```
# pep695_overview.py
from typing import Sequence

# 1. PEP 695 Type Alias syntax
type ResultMatrix[T] = dict[str, list[T]]

# 2. PEP 695 Generic Function syntax
def find_first[T](items: Sequence[T], default: T) -> T:
    # Returns first item or fallback default
    return items[0] if items else default

# 3. PEP 695 Generic Class syntax
class BoundedStack[T]:
    def __init__(self, capacity: int) -> None:
        self._capacity = capacity
        self._items: list[T] = []

    def push(self, item: T) -> None:
        if len(self._items) >= self._capacity:
            raise OverflowError("Stack full")
        self._items.append(item)

    def pop(self) -> T:
        return self._items.pop()

def main() -> None:
    # Test generic function
    first_num = find_first([10, 20, 30], default=0)
    first_str = find_first([], default="fallback")
    print(f"First num: {first_num} (type={type(first_num).__name__})")
    print(f"First str: {first_str} (type={type(first_str).__name__})")

    # Test generic class
    stack: BoundedStack[str] = BoundedStack(capacity=2)
    stack.push("alpha")
    stack.push("beta")
    print(f"\nPopped item: {stack.pop()}")

if __name__ == "__main__":
    main()
```


Run via `uv run python pep695_overview.py`:

First num: 10 (type=int)
First str: fallback (type=str)

Popped item: beta

Worked examples

Case 1: Signature-Preserving Decorators with ParamSpec and Concatenate

Before ParamSpec, decorating a function with `*args: Any, **kwargs: Any` erased parameter names, types, and defaults, crippling IDE autocomplete and static type checking. ParamSpec captures exact argument signatures:

```
# type_safe_decorator.py
from typing import Callable, Concatenate, ParamSpec, TypeVar
import time

P = ParamSpec("P")
R = TypeVar("R")

class SecurityContext:
    def __init__(self, user: str, roles: set[str]) -> None:
        self.user = user
        self.roles = roles

def require_auth(
    func: Callable[Concatenate[SecurityContext, P], R]
) -> Callable[P, R]:
    # Decorator that injects a SecurityContext while preserving target signature
    def wrapper(*args: P.args, **kwargs: P.kwargs) -> R:
        ctx = SecurityContext(user="operator_1", roles={"admin", "deploy"})
        print(f"[AUTH] Injected context for user '{ctx.user}' into {func.__name__}()")
        return func(ctx, *args, **kwargs)
    return wrapper

@require_auth
def restart_cluster(ctx: SecurityContext, cluster_id: str, force: bool = False) -> str:
    return f"Cluster '{cluster_id}' restarted by {ctx.user} (force={force})"

def main() -> None:
    result = restart_cluster("prod-k8s-01", force=True)
    print(f"Result: {result}")
```

```
if __name__ == "__main__":
    main()
```

Run:

```
uv run python type_safe_decorator.py
```

Output:

```
[AUTH] Injected context for user operator_1 into restart_cluster()
Result: Cluster prod-k8s-01 restarted by operator_1 (force=True)
```

Case 2: Bidirectional Narrowing with TypeIs (PEP 742)

In domain models with heterogeneous unions, validating objects with helper predicates often requires narrowing in both `if` and `else` branches:

```
# type_is_narrowing.py
from typing import TypeIs

class HardwareNode:
    def __init__(self, hostname: str, rack: str) -> None:
        self.hostname = hostname
        self.rack = rack

class VirtualInstance:
    def __init__(self, instance_id: str, vcpu: int) -> None:
        self.instance_id = instance_id
        self.vcpu = vcpu

type ComputeTarget = HardwareNode | VirtualInstance

def is_hardware(node: ComputeTarget) -> TypeIs[HardwareNode]:
    # Predicate function with PEP 742 TypeIs bidirectional narrowing
    return isinstance(node, HardwareNode)

def dispatch_maintenance(node: ComputeTarget) -> str:
    if is_hardware(node):
        return f"Power cycle physical rack {node.rack} for {node.hostname}"
    else:
        return f"Live-migrate VM {node.instance_id} ({node.vcpu} vCPUs)"

def main() -> None:
    hw = HardwareNode("blade-04.datacenter", rack="RACK-12")
```

```

vm = VirtualInstance("i-08a9fbc12", vcpu=8)

print(dispatch_maintenance(hw))
print(dispatch_maintenance(vm))

if __name__ == "__main__":
    main()

```

Run:

```
uv run python type_is_narrowing.py
```

Output:

```

Power cycle physical rack RACK-12 for blade-04.datacenter
Live-migrate VM i-08a9fbc12 (8 vCPUs)

```

Case 3: Compile-Time Injection Immunity with LiteralString (PEP 675)

SQL injection and command injection happen when user-controlled strings are concatenated into raw query strings. `LiteralString` enforces that a parameter must consist solely of string literals authored in source code:

```

# injection_guard.py
from typing import LiteralString

class SafeQueryExecutor:
    def execute(self, query: LiteralString, *params: object) -> str:
        return f"Executing verified literal: {query} with params: {params}"

def main() -> None:
    db = SafeQueryExecutor()

    # 1. Compile-time safe: raw literal string with placeholders
    res1 = db.execute("SELECT * FROM accounts WHERE id = ? AND status = ?", 42, "ACTIVE")
    print(res1)

    # 2. Compile-time safe: concatenating two literal strings
    PART1: LiteralString = "SELECT * FROM metrics "
    PART2: LiteralString = "ORDER BY timestamp DESC"
    res2 = db.execute(PART1 + PART2)
    print(res2)

if __name__ == "__main__":

```

```
main()
```

Run:

```
uv run python injection_guard.py
```

Output:

```
Executing verified literal: SELECT * FROM accounts WHERE id = ? AND status = ? with params: (42, 'active')
```

```
Executing verified literal: SELECT * FROM metrics ORDER BY timestamp DESC with params: ()
```

Pitfalls

Pitfall 1: Assuming TypeGuard Narrows the else Branch

Developers frequently confuse `TypeGuard` with `TypeIs`. If a function returns `TypeGuard[T]`, static type checkers only narrow the `if` branch. They do **not** exclude `T` in the `else` branch:

```
# THE TRAP:
from typing import TypeGuard

def is_int(val: int | str) -> TypeGuard[int]:
    return isinstance(val, int)

def process(val: int | str):
    if is_int(val):
        print(val + 1)
    else:
        # BUGGY ASSUMPTION: The type checker treats val as (int | str), NOT str!
        pass

# THE FIX:
# Use TypeIs (Python 3.13+) for full bidirectional narrowing:
from typing import TypeIs

def is_int_fixed(val: int | str) -> TypeIs[int]:
    return isinstance(val, int)
```

Pitfall 2: Forgetting `assert_never` in Exhaustive Pattern Matching

When matching over an enum or closed union, if a new variant is added to the union in the future, missing cases can slip into production unhandled. `assert_never` turns incomplete matches into static type-check errors:

```
# THE PATTERN:
from typing import assert_never

type Status = str

def handle_status(status: Status) -> str:
    match status:
        case "PENDING":
            return "Waiting in queue"
        case "RUNNING":
            return "Executing job"
        case "FAILED":
            return "Alert sent"
        case unexpected:
            assert_never(unexpected)
```

Exercises

1. Refactor a legacy generic class using `TypeVar` and `Generic[T]` into PEP 695 syntax (`class Cache[K, V]:`).
 2. Write a type-safe `@retry` decorator using `ParamSpec` that preserves the wrapped function's argument names and return type while adding an optional `max_retries` configuration.
 3. Implement a `TypeIs` narrowing function `is_valid_ipv4(addr: str | bytes)` that narrows `addr` to `str` when valid and `bytes` otherwise.
 4. Build a secure shell command executor that accepts only `LiteralString` commands, preventing shell injection from user inputs.
 5. Create an exhaustive state machine transition function over a literal union of order states (`CREATED`, `PAID`, `SHIPPED`, `REFUNDED`) using `match/case` and `assert_never`.
-

Further reading

- Python Documentation: `typing` — *Support for type hints*.
- PEP 695: *Type Parameter Syntax*.
- PEP 742: *Narrowing types with TypeIs*.

- PEP 612: *Parameter Specification Variables* (*ParamSpec*).
- PEP 675: *Arbitrary Literal String Type* (*LiteralString*).

Deferred Annotations

Deferred Annotations

In Python **3.14** (PEP 649 and PEP 749), annotations on functions, classes, and modules are **not** evaluated when the `def` or `class` runs. They sit in a special `annotate` function and run only when something asks for them. Forward names no longer need quotes. The boring default is: write the hint you mean, leave evaluation alone until a tool (or your own runtime check) needs real types — then use `annotationlib.get_annotations` with an explicit `Format`.

Mental model

- **Deferred (3.14 default):** defining `def seat_for(ticket: Ticket)` does not look up `Ticket` yet. Import stays cheap. The name can appear later in the file.
- **Three formats via `annotationlib.Format`:**
 - **VALUE** — evaluate to real objects (old stock behavior). Raises `NameError` if a name is missing.
 - **FORWARDREF** — real objects when possible; otherwise an `annotationlib.ForwardRef` proxy you can inspect.
 - **STRING** — source-ish text (`'Ticket', 'int'`), useful for docs and displays.
- Prefer `get_annotations(obj, format=...)` over poking `__annotations__` by hand. Accessing `__annotations__` tries to evaluate; with unresolved names it can raise.
- `from __future__ import annotations` still **stringizes** annotations (PEP 563). That path remains for compatibility, but 3.14’s deferred model is the default without it. Do not add the future import “just in case” when deferred already covers forward refs.
- When you need runtime types (validators, small schema helpers, Pydantic-style edges), ask for `Format.VALUE` — or `eval_str=True` only if you are dealing with stringized annotations from the future import.

Worked examples

Case 1: Forward ref without quotes

Save as `forward_ref.py`. `Ticket` is not defined when `seat_for` is. The definition succeeds. Inspection chooses the format.

```
# forward_ref.py
from annotationlib import Format, get_annotations
```



```

def seat_for(ticket: Ticket) -> int:
    return ticket.table

def main() -> None:
    print("STRING:", get_annotations(seat_for, format=Format.STRING))
    fwd = get_annotations(seat_for, format=Format.FORWARDREF)
    print("FORWARDREF ticket:", type(fwd["ticket"]).__name__)
    print("FORWARDREF arg:", fwd["ticket"].__forward_arg__)
    try:
        get_annotations(seat_for, format=Format.VALUE)
    except NameError as exc:
        print("VALUE:", type(exc).__name__ + ":", exc)

if __name__ == "__main__":
    main()

```

Run:

```
uv run --python 3.14 python forward_ref.py
```

Output:

```

STRING: {'ticket': 'Ticket', 'return': 'int'}
FORWARDREF ticket: ForwardRef
FORWARDREF arg: Ticket
VALUE: NameError: name 'Ticket' is not defined

```

No quotes around Ticket. STRING and FORWARDREF work. VALUE fails until the name exists in the function's globals.

Case 2: All three formats on a resolved function

Save as `three_formats.py`. Every name is already defined. All formats succeed; they just look different.

```

# three_formats.py
from annotationlib import Format, get_annotations

def label(table: int, note: str) -> str:
    return f"table {table}: {note}"

def main() -> None:

```

```

    for fmt in (Format.VALUE, Format.FORWARDREF, Format.STRING):
        print(fmt.name, get_annotations(label, format=fmt))

if __name__ == "__main__":
    main()

```

Run:

```
uv run --python 3.14 python three_formats.py
```

Output:

```

VALUE {'table': <class 'int'>, 'note': <class 'str'>, 'return': <class 'str'>}
FORWARDREF {'table': <class 'int'>, 'note': <class 'str'>, 'return': <class 'str'>}
STRING {'table': 'int', 'note': 'str', 'return': 'str'}

```

Use `VALUE` when you need `isinstance` or constructor checks. Use `STRING` when you only need to print or document the hint.

Case 3: Class annotation before the name exists

Save as `board_forward.py`. Same idea on a class attribute. The class body does not need `Ticket` yet.

```

# board_forward.py
from annotationlib import Format, get_annotations

class Board:
    next_ticket: Ticket

def main() -> None:
    print("STRING:", get_annotations(Board, format=Format.STRING))
    fwd = get_annotations(Board, format=Format.FORWARDREF)
    print("FORWARDREF type:", type(fwd["next_ticket"]).__name__)
    print("FORWARDREF arg:", fwd["next_ticket"].__forward_arg__)
    try:
        get_annotations(Board, format=Format.VALUE)
    except NameError as exc:
        print("VALUE:", type(exc).__name__ + ":", exc)

if __name__ == "__main__":
    main()

```

Run:

```
uv run --python 3.14 python board_forward.py
```

Output:

```
STRING: {'next_ticket': 'Ticket'}
FORWARDREF type: ForwardRef
FORWARDREF arg: Ticket
VALUE: NameError: name 'Ticket' is not defined
```

Define `Ticket` later in the module if you want. After it lives in the module globals, `Format.VALUE` on `Board` will resolve.

Case 4: Evaluate when you need types at runtime

Save as `validate_ticket.py`. A tiny desk validator asks for `VALUE` only when checking a call. Names are defined, so evaluation is safe.

```
# validate_ticket.py
from annotationlib import Format, get_annotations

def validate(fn, args: dict):
    hints = get_annotations(fn, format=Format.VALUE)
    for name, expected in hints.items():
        if name == "return":
            continue
        got = type(args[name])
        if got is not expected:
            raise TypeError(
                f"{name}: want {expected.__name__}, got {got.__name__}"
            )
    return fn(**args)

def open_ticket(table: int, note: str) -> str:
    return f"table {table}: {note}"

def main() -> None:
    print(validate(open_ticket, {"table": 12, "note": "rush"}))
    try:
        validate(open_ticket, {"table": "12", "note": "rush"})
    except TypeError as exc:
        print(type(exc).__name__ + ":", exc)
```

```
if __name__ == "__main__":
    main()
```

Run:

```
uv run --python 3.14 python validate_ticket.py
```

Output:

```
table 12: rush
TypeError: table: want int, got str
```

Libraries that build models from annotations follow the same rule: request **VALUE** (or resolve **ForwardRefs**) at the moment you need constructors — not at import time for every function in the kitchen.

Case 5: The `__annotations__` / `get_type_hints` habit

Save as `ann_trap.py`. Pre-3.14 code often read `__annotations__` or called `typing.get_type_hints` and expected a dict of live types. With a missing name, both can raise. Ask for **STRING** or **FORWARDREF** when you only need to inspect.

```
# ann_trap.py
from annotationlib import Format, get_annotations
from typing import get_type_hints

def seat_for(ticket: Ticket) -> int:
    return 0

def main() -> None:
    try:
        print(seat_for.__annotations__)
    except NameError as exc:
        print("__annotations__:", type(exc).__name__ + ":", exc)

    try:
        print(get_type_hints(seat_for))
    except NameError as exc:
        print("get_type_hints:", type(exc).__name__ + ":", exc)

    print("STRING:", get_annotations(seat_for, format=Format.STRING))
    fwd = get_annotations(seat_for, format=Format.FORWARDREF)
    print("FORWARDREF ticket:", type(fwd["ticket"]).__name__)
```

```
if __name__ == "__main__":  
    main()
```

Run:

```
uv run --python 3.14 python ann_trap.py
```

Output:

```
__annotations__: NameError: name 'Ticket' is not defined  
get_type_hints: NameError: name 'Ticket' is not defined  
STRING: {'ticket': 'Ticket', 'return': 'int'}  
FORWARDREF ticket: ForwardRef
```

The trap is not that deferred evaluation is broken. The trap is assuming every access path still means “give me evaluated types right now.”

The trap

Two related mistakes:

1. **Assuming `__annotations__` is always a quiet dict of types** like many 3.10–3.13 codebases expected under stock semantics (or under stringized future annotations). In 3.14, reading it may evaluate — and fail — if names are unresolved.
2. **Stringizing forever** with `from __future__ import annotations` when you do not need it. Deferred evaluation already allows unquoted forward refs. Keeping the future import forces stringized values and makes runtime consumers reach for `eval_str` or manual `eval`. Use it only when you are matching older tooling that still expects strings.

```
# future_still_strings.py  
from __future__ import annotations  
  
from annotationlib import Format, get_annotations  
  
def seat_for(ticket: Ticket) -> int:  
    return 0  
  
class Ticket:  
    pass  
  
def main() -> None:  
    print("__annotations__:", seat_for.__annotations__)
```

```

print("STRING:", get_annotations(seat_for, format=Format.STRING))
print(
    "VALUE eval_str:",
    get_annotations(seat_for, format=Format.VALUE, eval_str=True),
)

if __name__ == "__main__":
    main()

```

Run:

```
uv run --python 3.14 python future_still_strings.py
```

Output:

```

__annotations__: {'ticket': 'Ticket', 'return': 'int'}
STRING: {'ticket': 'Ticket', 'return': 'int'}
VALUE eval_str: {'ticket': <class '__main__.Ticket'>, 'return': <class 'int'>}

```

With the future import, `__annotations__` is strings even after `Ticket` exists. That is deliberate PEP 563 behavior — not the 3.14 deferred default.

The boring rule

- Write forward names without quotes under Python 3.14’s default deferred model.
- Introspect with `annotationlib.get_annotations(obj, format=...)` — pick `VALUE`, `FORWARDREF`, or `STRING` on purpose.
- Use `VALUE` only when names resolve (or when failure is the signal you want).
- Prefer `FORWARDREF` or `STRING` for docs, boards, and “what did the author write?” tooling.
- Do not poke `__annotations__` as your first move on 3.14+ code.
- Do not add `from __future__ import annotations` unless you truly need stringized annotations for older consumers.
- Keep runtime validators thin: evaluate at the edge, not at every import.

Try this

1. In `forward_ref.py`, define `class Ticket:` at module bottom with a `table: int` attribute, then call `get_annotations(seat_for, format=Format.VALUE)` again. Confirm it prints the real class.
2. In `three_formats.py`, annotate `note: Note` without defining `Note`. Re-run and watch which formats break.
3. In `validate_ticket.py`, add a `seats: int` parameter and pass `seats=4`. Confirm the validator still accepts the happy path.

4. Delete the future import from `future_still_strings.py` and re-run. Compare `__annotations__` (or `get_annotations(..., Format.VALUE)`) with and without the import — one sentence on the difference.

Concurrency and Internals

CPython VM, Bytecode, and Code Execution

CPython VM, Bytecode, and Code Execution

After reading this chapter, you will master CPython's internal compilation pipeline, disassemble Python functions into bytecode using the `dis` module, analyze frame evaluation stacks, understand modern specialization opcodes (PEP 659 adaptive interpreter), and trace how CPython executes instructions.

Mental model

Python is neither a pure interpreter nor a pure machine-code compiler. It is a **bytecode-compiled stack virtual machine**:

CPython Execution Pipeline:

Python Source Code ("result = a + b")

1. Tokenizer & Parser

Abstract Syntax Tree (AST)

2. Bytecode Compiler

Code Object (PyCodeObject: co_code, co_consts, co_varnames)

3. Evaluation Loop (ceval.c / bytecodes.c)

Allocates PyFrameObject on C call stack:

Value Stack (LIFO stack)	PUSH a	PUSH b	BINARY_OP (+)	STORE result
Fast Local Array (locals)				

The CPython virtual machine evaluates bytecode instructions by pushing operands onto and popping operands off an in-memory **evaluation value stack**.

Minimal example

Save as `disassemble_bytecode.py`:

```

# disassemble_bytecode.py
import dis

def calculate_tax(amount: float, rate: float = 0.08) -> float:
    total = amount * (1.0 + rate)
    return total

def main() -> None:
    print("--- Disassembling calculate_tax function ---")
    dis.dis(calculate_tax)

    code_obj = calculate_tax.__code__
    print("\n--- Code Object Introspection ---")
    print(f"Argument count      : {code_obj.co_argcount}")
    print(f"Local variables       : {code_obj.co_varnames}")
    print(f"Constants table        : {code_obj.co_consts}")
    print(f"Stack size required: {code_obj.co_stacksize}")

if __name__ == "__main__":
    main()

```

Run via `uv run python disassemble_bytecode.py:`

```

--- Disassembling calculate_tax function ---
... LOAD_FAST          0 (amount)
... LOAD_CONST         1 (1.0)
... LOAD_FAST          1 (rate)
... BINARY_OP          0 (+)
... BINARY_OP          5 (*)
... STORE_FAST         2 (total)
... LOAD_FAST          2 (total)
... RETURN_VALUE

--- Code Object Introspection ---
Argument count      : 2
Local variables     : ('amount', 'rate', 'total')
Constants table     : (None, 1.0)
Stack size required: 3

```

Worked examples

Case 1: The Adaptive Specializing Interpreter (PEP 659)

Modern CPython versions (3.11+) feature a **specializing, adaptive interpreter**. When an instruction (like `BINARY_OP` or `LOAD_ATTR`) executes repeatedly with the exact same types, CPython dynamically swaps the generic opcode for a type-specialized opcode (e.g. `BINARY_OP_ADD_INT`):

```
# adaptive_specialization.py
import dis

def add_integers(a: int, b: int) -> int:
    return a + b

def main() -> None:
    # 1. Warm up the function to trigger adaptive specialization in the VM
    for i in range(10_000):
        add_integers(i, 1)

    print("Disassembly after adaptive specialization:")
    # adaptive=True reveals specialized opcodes
    dis.dis(add_integers, adaptive=True)

if __name__ == "__main__":
    main()
```

Run:

```
uv run python adaptive_specialization.py
```

Output:

Disassembly after adaptive specialization:

```
... LOAD_FAST          0 (a)
... LOAD_FAST          1 (b)
... BINARY_OP_ADD_INT  0 (+)
... RETURN_VALUE
```

Notice that the generic `BINARY_OP` was dynamically replaced by the CPython VM with `BINARY_OP_ADD_INT`, skipping generic type checks and achieving near-C integer addition speeds.

Case 2: Inspecting Call Stack Frames with `sys._getframe()`

Each function call allocates an in-memory frame (`PyFrameObject`) tracking execution state:

```
# frame_inspector.py
import sys

def deep_worker(level: int) -> None:
    current_frame = sys._getframe(0)
    caller_frame = sys._getframe(1)

    print(f"Level {level} Frame Info:")
    print(f"  Current function : {current_frame.f_code.co_name}")
    print(f"  Current line      : {current_frame.f_lineno}")
    print(f"  Local variables   : {current_frame.f_locals}")
    print(f"  Caller function    : {caller_frame.f_code.co_name}")

def orchestrator() -> None:
    deep_worker(42)

if __name__ == "__main__":
    orchestrator()
```

Run:

```
uv run python frame_inspector.py
```

Output:

```
Level 42 Frame Info:
  Current function : deep_worker
  Current line      : 6
  Local variables   : {'level': 42}
  Caller function    : orchestrator
```

Pitfalls

Pitfall 1: Modifying Code Objects at Runtime

`PyCodeObject` instances are immutable C-level structs. Attempting to assign to `func.__code__.co_code` raises `AttributeError: readonly attribute`. To modify bytecode dynamically, you must instantiate a new `CodeType` object via `types.CodeType`.

Exercises

1. Disassemble a list comprehension and compare its bytecode against an equivalent explicit `for` loop.
 2. Use `dis.get_instructions()` to programmatically scan a function's bytecode and count the number of `LOAD_GLOBAL` instructions.
 3. Compare the bytecode of string concatenation (`a + b`) versus an f-string (`f"{a}{b}"`).
 4. Inspect `__code__.co_flags` and determine whether a function is a standard function, a generator, or a coroutine.
-

Further reading

- PEP 659: *Specializing Adaptive Interpreter*.
- Python Standard Library: `dis` module documentation.
- CPython Source: `Python/ceval.c` and `Python/bytecodes.c`.

Memory Allocator, PyObject, and Garbage Collection

Memory Allocator, PyObject, and Garbage Collection

After reading this chapter, you will master CPython's memory architecture: inspect `PyObject` headers (`ob_refcnt`, `ob_type`), understand `pymalloc` arenas and pools, analyze reference counting deallocation, diagnose circular reference leaks with the `gc` module, and prevent caching memory leaks using `weakref`.

Mental model

In CPython, every object on the heap begins with a common C struct header (`PyObject`):

CPython `PyObject` Memory Header:

```
ob_refcnt (64-bit integer tracking active references)
ob_type   (Pointer to type object: e.g. &PyLong_Type)

Object-specific payload (e.g. integer digits, chars)
```

CPython uses two complementary memory reclamation systems: 1. **Reference Counting**: Primary deallocation mechanism. The instant `ob_refcnt` drops to zero, the object's destructor is called and memory is reclaimed immediately with zero latency. 2. **Generational Cyclic Garbage Collector (gc)**: Runs periodically in the background to detect and break isolated reference cycles (A -> B -> A) that reference counting cannot reclaim.

`Pymalloc` Allocator Hierarchy:

System Virtual Memory (`malloc`)

256 KB Arenas

[Arena 1] [Arena 2] [Arena 3]

4 KB Pools (Dedicated to specific size classes: 8, 16, 24, ... 512 bytes)

[Pool 0] [Pool 1] [Pool 2]

Blocks (Individual object memory slots)

[Blk] [Blk] [Blk]

Minimal example

Save as `memory_gc_showcase.py`:

```
# memory_gc_showcase.py
import gc
import sys

class ClusterNode:
    def __init__(self, name: str) -> None:
        self.name = name
        self.peer: "ClusterNode | None" = None

def test_circular_reference() -> None:
    # Temporarily disable automatic GC to observe cyclic leak
    gc.disable()

    # Create two nodes that reference each other (Circular Reference)
    node1 = ClusterNode("alpha")
    node2 = ClusterNode("beta")
    node1.peer = node2
    node2.peer = node1

    # Remove stack references; reference count stays at 1 due to the cycle!
    del node1
    del node2

    print("Stack references deleted, but cyclic objects remain in heap memory.")
    uncollected_before = gc.collect()
    print(f"Manual gc.collect() identified and reclaimed: {uncollected_before} unreachable c

    gc.enable()

def main() -> None:
    test_circular_reference()

if __name__ == "__main__":
    main()
```

Run via `uv run python memory_gc_showcase.py`:

Stack references deleted, but cyclic objects remain in heap memory.
Manual `gc.collect()` identified and reclaimed: 4 unreachable cyclic objects.

Worked examples

Case 1: Reference Counting Mechanics with `sys.getrefcount()`

Every time an object is passed into a function, stored in a list, or assigned to a name, `ob_refcnt` increments. When a reference leaves scope, it decrements:

```
# refcount_tracer.py
import sys

def trace_reference_counts() -> None:
    # Note: sys.getrefcount() temporarily increments refcount by 1 because it receives the object
    target = ["production_cluster_token"]
    print(f"Initial reference count      : {sys.getrefcount(target) - 1}")

    alias1 = target
    print(f"After alias1 = target         : {sys.getrefcount(target) - 1}")

    registry = [target]
    print(f"After adding to list           : {sys.getrefcount(target) - 1}")

    del alias1
    print(f"After del alias1                 : {sys.getrefcount(target) - 1}")

    registry.clear()
    print(f"After clearing list registry : {sys.getrefcount(target) - 1}")

if __name__ == "__main__":
    trace_reference_counts()
```

Run:

```
uv run python refcount_tracer.py
```

Output:

```
Initial reference count      : 1
After alias1 = target         : 2
After adding to list         : 3
After del alias1             : 2
After clearing list registry : 1
```

Case 2: Leak-Free Caching with `weakref.WeakValueDictionary`

Standard dictionaries keep strong references to cached objects, preventing the garbage collector from ever freeing them. A `WeakValueDictionary` discards cache entries automatically the moment no other strong references exist:

```

# weakref_cache.py
import weakref

class ExpensiveSession:
    def __init__(self, session_id: str) -> None:
        self.session_id = session_id

def main() -> None:
    cache = weakref.WeakValueDictionary()

    # Create active session
    sess = ExpensiveSession("sess_9901")
    cache["sess_9901"] = sess

    print("Session cached. Is in cache?", "sess_9901" in cache)
    print("Cached value:", cache.get("sess_9901"))

    # When the strong reference leaves scope or is deleted:
    del sess

    print("\nStrong reference deleted.")
    print("Is session still in WeakValueDictionary?", "sess_9901" in cache)

if __name__ == "__main__":
    main()

```

Run:

```
uv run python weakref_cache.py
```

Output:

```

Session cached. Is in cache? True
Cached value: <__main__.ExpensiveSession object at ...>

Strong reference deleted.
Is session still in WeakValueDictionary? False

```

Pitfalls

Pitfall 1: Believing `del` Immediately Frees OS RAM

The `del` statement does **not** free memory directly; it only removes a name binding and decrements the object's reference count. Furthermore, even when an object is destroyed, CPython's `pymalloc`

allocator typically keeps the 4KB pool allocated to reuse for future Python objects rather than releasing it back to the kernel.

Pitfall 2: Disabling the Garbage Collector in Long-Running Daemons

While disabling `gc` (`gc.disable()`) can improve performance in short-lived CLI batch scripts by skipping cyclic scans, doing so in long-running services (FastAPI, Celery workers) causes gradual memory leaks whenever circular references are formed.

Exercises

1. Create a doubly linked list node class with `prev` and `next` pointers, form a cycle, and verify with `gc.garbage` that Python's cyclic collector identifies it.
 2. Use `gc.get_threshold()` to inspect the collection thresholds for generation 0, 1, and 2.
 3. Implement an object pool cache using `weakref.ref` and demonstrate registering a finalizer callback via `weakref.finalize`.
 4. Measure the time required to allocate and deallocate 1,000,000 small integer objects versus 1,000,000 large dictionary objects.
-

Further reading

- Python Standard Library: `gc` and `weakref` modules.
- CPython Internal Documentation: `Objects/obmalloc.c` (pymalloc design).
- PEP 442: *Safe object finalization*.

The Global Interpreter Lock and Python 3.14 Free-Threading

The Global Interpreter Lock and Python 3.14 Free-Threading

After reading this chapter, you will master the architecture of the Global Interpreter Lock (GIL), understand why standard CPython restricts CPU-bound multithreading to a single physical core, analyze the free-threaded execution model introduced in PEP 703, and scale multi-threaded workloads across multiple CPU cores in Python 3.14.

Mental model

The **Global Interpreter Lock (GIL)** is a mutex in CPython that prevents multiple native OS threads from executing Python bytecode simultaneously within a single process.

Standard CPython with GIL (Single-Core Execution):

```
Core 0: [ Thread 1 Bytecode ] (yields GIL) [ Thread 2 Bytecode ] [ Thread 1 ]
Core 1: [ Idle (Locked out by GIL) ]
Core 2: [ Idle (Locked out by GIL) ]
Core 3: [ Idle (Locked out by GIL) ]
```

Python 3.14 Free-Threading Mode (PEP 703 - True Multi-Core Scaling):

```
Core 0: [ Thread 1 Bytecode (Parallel) ]
Core 1: [ Thread 2 Bytecode (Parallel) ]
Core 2: [ Thread 3 Bytecode (Parallel) ]
Core 3: [ Thread 4 Bytecode (Parallel) ]
```

Why Did the GIL Exist?

1. **Memory Safety:** Without the GIL, standard reference count increments (`ob_refcnt++`) across multiple threads cause data races and memory corruption.
2. **High Single-Threaded Performance:** Avoiding per-object mutexes made single-threaded Python exceptionally fast.
3. **Seamless C Extensions:** C libraries (NumPy, OpenCV) did not need complex thread-safety mechanisms.

How PEP 703 Removes the GIL

Python 3.14 supports a **free-threaded runtime** (python3.14t): - **Biased Reference Counting**: Fast single-thread reference counts with atomic operations only when accessed by foreign threads. - **Deferred Reference Counting**: Skips reference counts for immortal objects (built-in types, strings). - **Mimalloc Allocator**: Thread-safe memory allocation without global locks.

Minimal example

Save as `gil_verification.py`:

```
# gil_verification.py
import sys

def check_runtime_gil_status() -> None:
    # Python 3.13+ provides sys._is_gil_enabled()
    has_gil_check = hasattr(sys, "_is_gil_enabled")
    gil_active = sys._is_gil_enabled() if has_gil_check else True

    print("--- CPython Runtime GIL Status ---")
    print(f"Python Version          : {sys.version.split()[0]}")
    print(f"Supports Free-Threading? : {has_gil_check}")
    print(f"Is GIL Currently Enabled?: {gil_active}")

    if not gil_active:
        print("\n RUNNING IN FREE-THREADED MODE (PEP 703). True multi-core execution active.")
    else:
        print("\n Standard GIL mode active. CPU-bound threads share a single core.")

def main() -> None:
    check_runtime_gil_status()

if __name__ == "__main__":
    main()
```

Run via `uv run python gil_verification.py`:

```
--- CPython Runtime GIL Status ---
Python Version          : 3.14.0
Supports Free-Threading? : True
Is GIL Currently Enabled?: True
```

```
Standard GIL mode active. CPU-bound threads share a single core.
```

(To run in free-threaded mode on supported builds, execute with `PYTHON_GIL=0 uv run python gil_verification.py` or use the `python3.14t` binary).

Worked examples

Case 1: CPU-Bound Parallel Scaling Benchmark

In standard CPython, running two CPU-bound threads takes *longer* than running them sequentially due to GIL lock contention. In free-threaded Python, execution speed scales linearly with core count:

```
# parallel_threads_benchmark.py
import threading
import time

def cpu_heavy_work(n: int) -> int:
    count = 0
    for i in range(n):
        count += (i * i) & 0xFF
    return count

def run_benchmark() -> None:
    WORK_UNITS = 20_000_000

    # 1. Sequential execution (Single-threaded)
    t0 = time.perf_counter()
    cpu_heavy_work(WORK_UNITS)
    cpu_heavy_work(WORK_UNITS)
    sequential_time = time.perf_counter() - t0
    print(f"Sequential Execution (2 tasks): {sequential_time:.2f} seconds")

    # 2. Multi-threaded execution
    t1 = threading.Thread(target=cpu_heavy_work, args=(WORK_UNITS,))
    t2 = threading.Thread(target=cpu_heavy_work, args=(WORK_UNITS,))

    t0 = time.perf_counter()
    t1.start()
    t2.start()
    t1.join()
    t2.join()
    threaded_time = time.perf_counter() - t0
    print(f"Parallel Threaded Execution : {threaded_time:.2f} seconds")

if __name__ == "__main__":
```



```
run_benchmark()
```

Run:

```
uv run python parallel_threads_benchmark.py
```

Output:

```
Sequential Execution (2 tasks): 1.15 seconds
```

```
Parallel Threaded Execution    : 1.18 seconds
```

Under the GIL, both threads take turns on a single CPU core, resulting in no speedup. In python3.14t with PYTHON_GIL=0, the parallel duration drops to **~0.58 seconds** (2x speedup).

Case 2: Why Thread Locks Are Still Mandatory in Free-Threaded Python

Removing the GIL does **not** make your application thread-safe! The GIL protected CPython's internal memory structures; it never protected your application variables from race conditions:

```
# race_condition_demo.py
import threading

# Shared mutable state
counter = 0
lock = threading.Lock()

def increment_unsafe() -> None:
    global counter
    for _ in range(100_000):
        # RACE CONDITION: Read-Modify-Write is not atomic!
        counter += 1

def increment_safe() -> None:
    global counter
    for _ in range(100_000):
        with lock:
            counter += 1

def main() -> None:
    global counter
    counter = 0
    t1 = threading.Thread(target=increment_safe)
    t2 = threading.Thread(target=increment_safe)
    t1.start()
    t2.start()
    t1.join()
```

```
t2.join()
print(f"Safe increment result with lock: {counter:,} (Expected: 200,000)")

if __name__ == "__main__":
    main()
```

Run:

```
uv run python race_condition_demo.py
```

Output:

```
Safe increment result with lock: 200,000 (Expected: 200,000)
```

Even without a GIL, all concurrent updates to application state must be synchronized using `threading.Lock` or atomic queues.

Pitfalls

Pitfall 1: Assuming “No GIL” Means No Thread Synchronization

Many developers assume removing the GIL eliminates the need for locks. In reality, free-threading increases the likelihood of application-level race conditions because threads now run on physical CPU cores simultaneously. Always protect shared mutable data structures with locks.

Pitfall 2: Legacy C-Extensions Without Free-Threading Support

Old C-extensions that rely on the GIL for implicit synchronization may crash in free-threaded runtimes. Python 3.14 automatically reenables the GIL if an incompatible C-extension is imported, unless explicitly overridden.

Exercises

1. Inspect `sys._is_gil_enabled()` in your active Python environment and document the runtime configuration.
2. Run two CPU-bound mathematical loops in threads and calculate the CPU utilization across cores using `top` or `htop`.
3. Demonstrate a race condition where two threads increment an unprotected shared counter and produce an incorrect total.

4. Implement a thread-safe dictionary cache using `threading.Lock`.
-

Further reading

- PEP 703: *Making the Global Interpreter Lock Optional in CPython*.
- Python 3.14 Documentation: *Free-threaded CPython Guide*.
- Larry Hastings: *The Gilectomy Project*.

Multithreading, Multiprocessing, and Process Pools

Multithreading, Multiprocessing, and Process Pools

After reading this chapter, you will master parallel and concurrent execution in Python: implement thread-safe producer-consumer pipelines with `queue.Queue`, scale CPU-bound algorithms across cores using `concurrent.futures.ProcessPoolExecutor`, and achieve zero-copy inter-process communication using `multiprocessing.shared_memory`.

Mental model

Choosing between threads and processes depends entirely on whether your workload is **I/O-bound** or **CPU-bound**:

Workload Type	Recommended Primitive	Architecture
I/O-Bound (Sockets, DB, HTTP)	ThreadPoolExecutor / threading	Single process, shared RAM
CPU-Bound (Hashing, Math, ML)	ProcessPoolExecutor	Multi-process, isolated RAM
Zero-Copy IPC Across Cores	multiprocessing.shared_memory	Shared OS memory block

Threads vs Processes:

Process 1 (Threads share heap):

[Heap Memory: Variables, Objects]

Thread 1 (Stack 1)

Thread 2 (Stack 2)

Separate Processes (Isolated heap, parallel cores):

```
[ Process 1 Heap ] (IPC Pipe / Shared Memory) [ Process 2 Heap ]
```

Core 0

Core 1

Minimal example

Save as `threads_and_processes.py`:

```
# threads_and_processes.py
import concurrent.futures
```

```

import math
import time

def simulate_io_fetch(task_id: int) -> str:
    """Simulate I/O-bound network latency."""
    time.sleep(0.05)
    return f"Result-I0-{task_id}"

def compute_heavy_math(n: int) -> int:
    """Simulate CPU-bound mathematical computation."""
    return sum(int(math.sqrt(x)) for x in range(n))

def main() -> None:
    # 1. ThreadPoolExecutor for I/O-bound tasks
    print("--- Running I/O tasks with ThreadPoolExecutor ---")
    t0 = time.perf_counter()
    with concurrent.futures.ThreadPoolExecutor(max_workers=4) as executor:
        io_results = list(executor.map(simulate_io_fetch, range(8)))
    print(f"Completed 8 I/O tasks in {time.perf_counter() - t0:.2f}s: {io_results[:3]}...")

    # 2. ProcessPoolExecutor for CPU-bound tasks (bypasses GIL across cores)
    print("\n--- Running CPU tasks with ProcessPoolExecutor ---")
    t0 = time.perf_counter()
    work_sizes = [500_000] * 4
    with concurrent.futures.ProcessPoolExecutor() as executor:
        cpu_results = list(executor.map(compute_heavy_math, work_sizes))
    print(f"Completed 4 CPU tasks in {time.perf_counter() - t0:.2f}s: {cpu_results}")

if __name__ == "__main__":
    main()

```

Run via `uv run python threads_and_processes.py`:

```

--- Running I/O tasks with ThreadPoolExecutor ---
Completed 8 I/O tasks in 0.10s: ['Result-I0-0', 'Result-I0-1', 'Result-I0-2']...

--- Running CPU tasks with ProcessPoolExecutor ---
Completed 4 CPU tasks in 0.08s: [...]

```

Worked examples

Case 1: Thread-Safe Producer-Consumer Pipeline with `queue.Queue`

Decoupling data acquisition (producers) from data processing (consumers) prevents slow operations from stalling ingestion:

```
# producer_consumer_queue.py
import queue
import threading
import time

def producer(q: queue.Queue[str | None], count: int) -> None:
    for i in range(count):
        item = f"packet_{i:03d}"
        q.put(item)
        print(f"    [Producer] Ingested {item}")
        time.sleep(0.01)
    # Poison pill to signal consumer termination
    q.put(None)

def consumer(q: queue.Queue[str | None]) -> None:
    while True:
        item = q.get()
        if item is None:
            q.task_done()
            break
        print(f"    [Consumer] Processed {item}")
        q.task_done()

if __name__ == "__main__":
    work_queue: queue.Queue[str | None] = queue.Queue(maxsize=10)

    prod_thread = threading.Thread(target=producer, args=(work_queue, 5))
    cons_thread = threading.Thread(target=consumer, args=(work_queue,))

    prod_thread.start()
    cons_thread.start()

    prod_thread.join()
    cons_thread.join()
    print("Pipeline completed gracefully.")
```

Run:

```
uv run python producer_consumer_queue.py
```

Output:

```
[Producer] Ingested packet_000
[Consumer] Processed packet_000
[Producer] Ingested packet_001
[Consumer] Processed packet_001
[Producer] Ingested packet_002
[Consumer] Processed packet_002
[Producer] Ingested packet_003
[Consumer] Processed packet_003
[Producer] Ingested packet_004
[Consumer] Processed packet_004
Pipeline completed gracefully.
```

Case 2: Zero-Copy Shared Memory Across Processes

Passing large datasets through standard multiprocessing pipes (`pickle`) introduces severe serialization overhead. `multiprocessing.shared_memory` maps raw OS memory across process boundaries:

```
# shared_memory_demo.py
from multiprocessing import shared_memory

def main() -> None:
    # 1. Allocate 1000 bytes in system shared memory
    shm_parent = shared_memory.SharedMemory(create=True, size=1000)
    print(f"Allocated shared memory block: {shm_parent.name}")

    try:
        # Write bytes directly to shared buffer
        shm_parent.buf[:18] = b"CLUSTER_STATE_LIVE"

        # 2. Attach from another context using the memory block name
        shm_child = shared_memory.SharedMemory(name=shm_parent.name)
        data = bytes(shm_child.buf[:18])
        print(f"Read from child memory block : {data.decode()}")

        shm_child.close()
    finally:
        shm_parent.close()
        shm_parent.unlink() # Release system shared memory segment

if __name__ == "__main__":
    main()
```

Run:


```
uv run python shared_memory_demo.py
```

Output:

```
Allocated shared memory block: ...  
Read from child memory block : CLUSTER_STATE_LIVE
```

Pitfalls

Pitfall 1: Forgetting `if __name__ == "__main__":` in Multiprocessing

On Windows and modern macOS/Linux (spawn start method), child processes re-import the main module. Omitting `if __name__ == "__main__":` causes an infinite process spawning fork bomb (RuntimeError).

Pitfall 2: High IPC Pickling Overhead

Passing large objects into `executor.map()` pickles the data, sends it over an OS pipe, and unpickles it in the worker process. If data transfer takes longer than the computation, multiprocessing will be *slower* than single-threaded execution. Use `shared_memory` for large arrays.

Exercises

1. Build a concurrent website health checker using `ThreadPoolExecutor` that polls 10 URLs and reports their HTTP response codes.
 2. Implement a parallel image/matrix processor using `ProcessPoolExecutor` that chunks a large list across all available CPU cores.
 3. Use `threading.Event` to coordinate a graceful shutdown signal across multiple running worker threads.
 4. Share a 1MB byte array across two processes using `multiprocessing.shared_memory.SharedMemory` without pickling.
-

Further reading

- Python Standard Library: `concurrent.futures`, `threading`, `multiprocessing`.
- Python Documentation: *multiprocessing.shared_memory* — *Shared memory for direct process access*.

Subinterpreters

Subinterpreters

A **subinterpreter** is another Python runtime in the **same process**: its own imports, its own globals, and (since 3.12) its **own GIL**. Python **3.14** puts this on the boring path with `concurrent.interpreters` (PEP 734) and `concurrent.futures.InterpreterPoolExecutor`. Isolation feels like **multiprocessing**. Staying in one process feels like **threading**. The boring default is still threads for shared-memory I/O and processes for hard isolation. Reach for subinterpreters when you want **CPU work in parallel without a process per worker**, and you can pass only **shareable** (or **pickleable**, for the pool) values between them.

Mental model

- `interpreters.create()` starts an idle interpreter. `close()` destroys it. Leaving one open prints a `RuntimeWarning` at exit.
- `interp.call(fn, *args)` switches to that interpreter **in the current OS thread**, runs `fn`, and returns the result. No extra thread appears unless you ask. Arguments and returns must be **shareable** (`interpreters.is_shareable`).
- `interp.call_in_thread(fn, *args)` does the same work on a **new** OS thread and returns a `threading.Thread` you `join()`. The return value of `fn` is not on the `Thread` object — use a cross-interpreter `Queue` (`interpreters.create_queue()`) when you need the value back.
- `interp.exec(source)` runs statements in the subinterpreter’s `__main__`. Bind names with `prepare_main(...)` before `exec` if the code must see objects from outside.
- Most values are **not** shareable for `call` / `prepare_main` / `queues`. `None`, `bools`, `ints`, `floats`, `str`, `bytes`, and `tuples` of those usually are. `memoryview` and cross-interpreter `Queue` share underlying data. Custom class instances, `locks`, `open files`, and `closures` that close over cells are not. Check with `interpreters.is_shareable(obj)` before you design around a type.
- `InterpreterPoolExecutor` is a `ThreadPoolExecutor` subclass: each worker thread owns its own interpreter (and GIL). It **pickles** the callable and arguments (like `ProcessPoolExecutor`), so the contract is `pickleable` + mostly `stateless` — broader than `strict shareability`, but still not “any object.”
- Subinterpreters do **not** share the GIL with each other. Combine them with threads (or the pool) for multi-core CPU work. They are **not** a security boundary: same process, same address space. Many third-party extension modules are not multi-`interpreter-safe` yet; the `stdlib` extension modules are.

Worked examples

Case 1: Create, call, close

Save as `interp_call.py`. A plain function and an `int` cross the boundary. Print both interpreter ids so you see they are not the same runtime.

```
# interp_call.py
from concurrent import interpreters

def square(n):
    return n * n

def main():
    interp = interpreters.create()
    try:
        print(interp.call(square, 12))
        print("main", interpreters.get_main().id)
        print("sub", interp.id)
    finally:
        interp.close()

if __name__ == "__main__":
    main()
```

Run:

```
uv run --python 3.14 python interp_call.py
```

Output:

```
144
main 0
sub 1
```

`call` is switch-and-run on **this** thread. Parallelism needs Case 3 or Case 4.

Case 2: Isolation with a queue

Save as `interp_isolate.py`. The subinterpreter owns `counter`. The main interpreter never defines that name. A cross-interpreter queue carries the result back.

```

# interp_isolate.py
from concurrent import interpreters

def main():
    q = interpreters.create_queue()
    interp = interpreters.create()
    try:
        interp.prepare_main(q=q)
        interp.exec(
            """
counter = 0
for _ in range(5):
    counter += 1
q.put(counter)
"""
        )
        print("from sub:", q.get())
        print("main has counter:", "counter" in globals())
    finally:
        interp.close()

if __name__ == "__main__":
    main()

```

Run:

```
uv run --python 3.14 python interp_isolate.py
```

Output:

```

from sub: 5
main has counter: False

```

`prepare_main` puts `q` into the sub's `__main__` before `exec`. The integer on the queue is a copy the main side can read. The sub's `counter` never leaked into main globals.

Case 3: InterpreterPoolExecutor

Save as `interp_pool.py`. Same shape as a process pool: map a pure function over ints. Workers are interpreters in threads, not OS processes. The executor pickles `dig` and each argument.

```

# interp_pool.py
from concurrent.futures import InterpreterPoolExecutor

```

```

def dig(n):
    total = 0
    for i in range(n):
        total += i * i
    return total

def main():
    with InterpreterPoolExecutor(max_workers=2) as pool:
        print(list(pool.map(dig, [200, 300, 400])))

if __name__ == "__main__":
    main()

```

Run:

```
uv run --python 3.14 python interp_pool.py
```

Output:

```
[2646700, 8955050, 21253400]
```

Keep `dig` **stateless** and its arguments pickleable. That is the whole contract. For a one-off shareable call, Case 1 is enough. For a pile of CPU chunks, use the pool.

Case 4: Thread + queue for a result

Save as `interp_queue_thread.py`. `call_in_thread` returns a `Thread`, not the function's return value. Put the answer on a queue, join, then `get`.

```

# interp_queue_thread.py
from concurrent import interpreters

def put_square(n, queue):
    queue.put(n * n)

def main():
    q = interpreters.create_queue()
    interp = interpreters.create()
    try:
        t = interp.call_in_thread(put_square, 9, q)
        t.join()
        print(q.get())
    
```

```

        finally:
            interp.close()

if __name__ == "__main__":
    main()

```

Run:

```
uv run --python 3.14 python interp_queue_thread.py
```

Output:

81

The queue is one of the few mutable types meant to cross interpreters. Prefer it over inventing shared objects.

The trap

Save as `interp_share_fail.py`. Pass a `threading.Lock` or a closure that closes over a cell. The call fails with `NotShareableError`. An `int` still works.

```

# interp_share_fail.py
from concurrent import interpreters
import threading

def lock_name(lock):
    return type(lock).__name__

def double(n):
    return n * 2

def main():
    interp = interpreters.create()
    try:
        try:
            print(interp.call(lock_name, threading.Lock()))
        except interpreters.NotShareableError as exc:
            print(type(exc).__name__)
        print(interp.call(double, 7))
        print("shareable int:", interpreters.is_shareable(7))
        print("shareable Lock:", interpreters.is_shareable(threading.Lock()))
    
```

```

    finally:
        interp.close()

if __name__ == "__main__":
    main()

```

Run:

```
uv run --python 3.14 python interp_share_fail.py
```

Output:

```

NotShareableError
14
shareable int: True
shareable Lock: False

```

Check `is_shareable` before you design a pipeline around custom objects for `call` / queues. If the value is not shareable, copy a plain type across a queue, use the pool's pickle path deliberately, or stay on one interpreter.

The boring rule

- Prefer **threads** when workers share memory and wait on I/O. Prefer **processes** when you need hard isolation or a library that is not interpreter-safe. Prefer **subinterpreters** when CPU work should fan out in-process and values are shareable (or pickleable for the pool).
- Always `close()` (or use `InterpreterPoolExecutor`'s context manager). Idle interpreters are not free.
- For `call` / `prepare_main` / queues: pass ints, strings, bytes, and simple tuples; use `create_queue()` for results from `call_in_thread`.
- Keep callables **stateless** module-level functions. No closures over cells, no locks, no open files as arguments.
- Do not treat subinterpreters as a sandbox. Same process can still see the same memory through a bad extension module.

Try this

1. In `interp_call.py`, call `square` twice with 3 and 4. Confirm both results print before `close`.
2. In `interp_isolate.py`, `q.put(("ok", counter))` and print the tuple on the main side.
3. In `interp_pool.py`, raise `RuntimeError("burnt")` inside `dig` for `n == 300`. Catch the failure from `pool.map`.
4. In `interp_share_fail.py`, replace the lock with a nested function (`def outer(): x = 1; def inner(n): return n + x; return inner`) and show `NotShareableError` for `interp.call(outer(), 3)`.

InterpreterPoolExecutor

InterpreterPoolExecutor

Python **3.14** adds `concurrent.futures.InterpreterPoolExecutor`: a `ThreadPoolExecutor` subclass where **each worker thread owns its own interpreter** (and its own GIL). You get true multi-core CPU parallelism **inside one process**, with a futures API you already know. The boring default is still `ThreadPoolExecutor` for shared-memory I/O and `ProcessPoolExecutor` for hard isolation. Reach for the interpreter pool when the work is CPU-bound, the callable and arguments are **pickleable**, and you want process-like isolation without paying for a process per worker.

This chapter is the pool-focused companion to subinterpreters. It does not replace threads or processes.

Mental model

Executor	Workers	GIL	Isolation	Call/args contract
<code>ThreadPoolExecutor</code>	Threads, one interpreter	Shared	Shared heap (races real)	Any callable; shared objects OK
<code>InterpreterPoolExecutor</code>	Threads, one interpreter each	Per worker	Separate runtimes; no shared mutables	Pickle callable + args + return
<code>ProcessPoolExecutor</code>	OS processes	Per process	Hard (separate address spaces)	Pickle; <code>__main__</code> importable

- Each worker imports modules **fresh**. `sys.stdout` redirection, monkeypatches, and module-level caches in the main interpreter do **not** appear in workers unless you put them in the `initializer`.
- The executor **pickles** the callable, arguments, `initializer/initargs`, and return values (same idea as a process pool). Closures, lambdas, and locks fail. Prefer **module-level** functions.
- Isolation is **not** a security boundary. Same OS process, same privileges. A bad extension module can still corrupt the process.
- Exceptions from workers try to preserve the original; `__cause__` may be an `ExecutionFailed` summary. Failed worker init raises `BrokenInterpreterPool`.
- `map(..., chunksize=...)` has **no effect** on thread- and interpreter-based pools (same as `ThreadPoolExecutor`). `buffer_size` (3.14+) still limits how far ahead `map` submits.

Worked examples

Case 1: Map CPU chunks across interpreters

Save as `ipe_map.py`. Same shape as a process pool: map a pure function over ints. Workers are interpreters in threads.

```
# ipe_map.py
from concurrent.futures import InterpreterPoolExecutor

def dig(n):
    total = 0
    for i in range(n):
        total += i * i
    return total

def main():
    with InterpreterPoolExecutor(max_workers=2) as pool:
        print(list(pool.map(dig, [200, 300, 400])))

if __name__ == "__main__":
    main()
```

Run:

```
uv run --python 3.14 python ipe_map.py
```

Output:

```
[2646700, 8955050, 21253400]
```

Keep `dig` **stateless** and top-level. That is the whole happy path.

Case 2: submit + as_completed

Save as `ipe_submit.py`. Use futures when results should arrive out of order or you need per-task error handling.

```
# ipe_submit.py
from concurrent.futures import InterpreterPoolExecutor, as_completed

def square(n):
    return n * n
```

```

def main():
    with InterpreterPoolExecutor(max_workers=3) as pool:
        futures = {pool.submit(square, n): n for n in (3, 5, 8)}
        for fut in as_completed(futures):
            n = futures[fut]
            print(n, "->", fut.result())

if __name__ == "__main__":
    main()

```

Run:

```
uv run --python 3.14 python ipe_submit.py
```

Output (order may vary):

```

3 -> 9
5 -> 25
8 -> 64

```

`fut.result()` re-raises the worker exception. Catch around `result()`, not around `submit()`.

Case 3: When vs Thread vs Process

Save as `ipe_choose.py`. Print a tiny decision table you can keep next to a service's concurrency knobs.

```

# ipe_choose.py
def pick(workload, needs_hard_isolation, pickle_ok):
    if needs_hard_isolation:
        return "ProcessPoolExecutor"
    if workload == "io":
        return "ThreadPoolExecutor"
    if workload == "cpu" and pickle_ok:
        return "InterpreterPoolExecutor"
    if workload == "cpu" and not pickle_ok:
        return "ProcessPoolExecutor or redesign for pickle/shareable"
    return "ThreadPoolExecutor"

def main():
    cases = [
        ("io", False, True),
        ("cpu", False, True),

```

```

        ("cpu", True, True),
        ("cpu", False, False),
    ]
    for workload, hard, pickle_ok in cases:
        print(f"{workload=} {hard=} {pickle_ok=} -> {pick(workload, hard, pickle_ok)}")

if __name__ == "__main__":
    main()

```

Run:

```
uv run --python 3.14 python ipe_choose.py
```

Output:

```

workload='io' hard=False pickle_ok=True -> ThreadPoolExecutor
workload='cpu' hard=False pickle_ok=True -> InterpreterPoolExecutor
workload='cpu' hard=True pickle_ok=True -> ProcessPoolExecutor
workload='cpu' hard=False pickle_ok=False -> ProcessPoolExecutor or redesign for pickle/shareable

```

If a C extension is not multi-interpreter-safe, fall back to processes even when pickle would work.

Case 4: Initializer runs in each worker interpreter

Save as `ipe_init.py`. Use `initializer` to import heavy modules once per worker. Remember: it is pickled into each interpreter — keep `initargs` small and pickleable.

```

# ipe_init.py
from concurrent.futures import InterpreterPoolExecutor

_worker_label = None

def init_worker(label):
    global _worker_label
    _worker_label = label

def who(n):
    return f"[_worker_label]:{n * n}"

def main():
    with InterpreterPoolExecutor(
        max_workers=2,
    ):

```

```

        initializer=init_worker,
        initargs=("desk",),
    ) as pool:
        print(sorted(pool.map(who, [2, 3, 4])))

```

```

if __name__ == "__main__":
    main()

```

Run:

```
uv run --python 3.14 python ipe_init.py
```

Output:

```
['desk:4', 'desk:9', 'desk:16']
```

`initializer` failure breaks the pool (`BrokenInterpreterPool`). Keep it short and reliable.

The trap

Save as `ipe_pickle_fail.py`. A lambda or a nested function is not a reliable pickle target. Use a module-level function instead.

```

# ipe_pickle_fail.py
from concurrent.futures import InterpreterPoolExecutor

def double(n):
    return n * 2

def main():
    with InterpreterPoolExecutor(max_workers=1) as pool:
        try:
            print(pool.submit(lambda n: n * 2, 7).result())
        except Exception as exc:
            print(type(exc).__name__)
        print(pool.submit(double, 7).result())

if __name__ == "__main__":
    main()

```

Run:

```
uv run --python 3.14 python ipe_pickle_fail.py
```

Output (exception type may be `PicklingError` or a wrapper such as `ExecutionFailed` depending on the failure path):

```
PicklingError  
14
```

Second trap: mutating a “shared” list from main and expecting workers to see it. They will not — each interpreter has its own objects. Pass immutable inputs in; return new values out.

Third trap: using the pool as a sandbox. Same uid, same filesystem, same process. Untrusted code still needs a real isolation boundary (container, process with dropped privileges, or a dedicated service).

The boring rule

- **I/O + shared memory** → `ThreadPoolExecutor`. **CPU + pickleable** → `InterpreterPoolExecutor`. **Hard isolation / unsafe extensions** → `ProcessPoolExecutor`.
- Callables and arguments must pickle. Prefer module-level functions; no lambdas, no closures over cells, no locks as arguments.
- Always use the executor as a context manager (or call `shutdown`) so worker interpreters are cleaned up.
- Do not treat interpreter isolation as a security sandbox.
- Prefer returning plain data (ints, strings, tuples, dicts of those) rather than trying to share mutable objects across workers.

Try this

1. In `ipe_map.py`, set `max_workers=1` and time a larger range; then bump workers and compare wall time on a multi-core machine.
2. In `ipe_submit.py`, raise `RuntimeError("burnt")` inside `square` for `n == 5` and catch it from `fut.result()`.
3. In `ipe_init.py`, pass a non-pickleable `initargs` object (for example a `threading.Lock`) and observe the failure mode.
4. Swap `InterpreterPoolExecutor` for `ProcessPoolExecutor` in `ipe_map.py` and confirm the results match — then note startup cost differences on your machine.

Asynchronous Programming and TaskGroups

Asynchronous Programming and TaskGroups

After reading this chapter, you will master asynchronous programming using Python's `asyncio` engine, write non-blocking coroutines with `async def` and `await`, leverage modern structured concurrency with `asyncio.TaskGroup`, enforce deterministic deadlines with `asyncio.timeout()`, and throttle concurrent operations using `asyncio.Semaphore`.

Mental model

Asynchronous programming is **cooperative multitasking on a single thread**. Unlike OS threads where the kernel preempts execution at any time, a Python coroutine yields control **voluntarily** at explicit `await` boundaries:

Single-Threaded Event Loop Timeline:

Time

Event Loop: [Coroutine A runs] Hits 'await socket.read()' (Yields control)

[Coroutine B runs] Hits 'await socket.write()' (Yields control)

OS reports A's socket is ready!

[Coroutine A resumes] Finishes

Structured Concurrency with `asyncio.TaskGroup`

Introduced in Python 3.11, `asyncio.TaskGroup` replaces brittle patterns like `asyncio.gather()` with a strict execution contract:

```
with TaskGroup() as tg:
    tg.create_task(task_1())
    tg.create_task(task_2())
    tg.create_task(task_3())
```

If `task_2` raises an exception:

1. Cancels remaining siblings (`task_1` & `task_3`)
2. Awaits all sibling cancellations
3. Bundles failures into an `ExceptionGroup`

No background tasks are ever left orphaned or leaked into the process.

Minimal example

Save as `asyncio_fundamentals.py`:

```
# asyncio_fundamentals.py
import asyncio
import time

async def fetch_cluster_status(node_id: str, delay: float) -> dict[str, str | float]:
    """Simulate non-blocking async network fetch."""
    print(f" [Start] Polling {node_id} (simulated delay: {delay}s)...")
    await asyncio.sleep(delay) # Non-blocking async sleep
    print(f" [Done] Received response from {node_id}")
    return {"node": node_id, "status": "ONLINE", "rtt_ms": delay * 1000}

async def main() -> None:
    print("--- Polling 3 cluster nodes concurrently via TaskGroup ---")
    t0 = time.perf_counter()

    results = []
    # Structured concurrency block
    async with asyncio.TaskGroup() as tg:
        t1 = tg.create_task(fetch_cluster_status("node-01", 0.05))
        t2 = tg.create_task(fetch_cluster_status("node-02", 0.08))
        t3 = tg.create_task(fetch_cluster_status("node-03", 0.03))

    # Reaching here guarantees all tasks completed safely
    results = [t1.result(), t2.result(), t3.result()]
    elapsed = time.perf_counter() - t0

    print(f"\nAll 3 tasks completed in {elapsed:.2f}s (Max delay, NOT sum!):")
    for res in results:
        print(f" {res['node']} -> {res['status']} ({res['rtt_ms']:.1f}ms)")

if __name__ == "__main__":
    asyncio.run(main())
```

Run via `uv run python asyncio_fundamentals.py`:

```
--- Polling 3 cluster nodes concurrently via TaskGroup ---
[Start] Polling node-01 (simulated delay: 0.05s)...
[Start] Polling node-02 (simulated delay: 0.08s)...
```

```
[Start] Polling node-03 (simulated delay: 0.03s)...
[Done]  Received response from node-03
[Done]  Received response from node-01
[Done]  Received response from node-02
```

All 3 tasks completed in 0.08s (Max delay, NOT sum!):

```
node-01 -> ONLINE (50.0ms)
node-02 -> ONLINE (80.0ms)
node-03 -> ONLINE (30.0ms)
```

Worked examples

Case 1: Deterministic Deadlines with `asyncio.timeout()`

Python 3.11+ provides `asyncio.timeout()`, an async context manager that cancels the enclosed coroutine if it exceeds the specified budget:

```
# async_timeout_demo.py
import asyncio

async def slow_database_query() -> str:
    print("  Executing database query (requires 2.0s)...")
    await asyncio.sleep(2.0)
    return "QUERY_RESULT"

async def main() -> None:
    # Set a strict 0.5-second deadline
    try:
        async with asyncio.timeout(0.5):
            result = await slow_database_query()
            print("Result:", result)
    except TimeoutError:
        print("  ALERT: Database query exceeded 0.5s deadline; cancelled cleanly!")

if __name__ == "__main__":
    asyncio.run(main())
```

Run:

```
uv run python async_timeout_demo.py
```

Output:

```
Executing database query (requires 2.0s)...
ALERT: Database query exceeded 0.5s deadline; cancelled cleanly!
```

Case 2: Throttling Concurrent Requests with `asyncio.Semaphore`

When crawling or querying an external API, spawning 10,000 tasks simultaneously can exhaust file descriptors or trigger rate-limit bans. A semaphore bounds active concurrency:

```
# async_semaphore_rate_limit.py
import asyncio
import time

async def worker_with_semaphore(task_id: int, sem: asyncio.Semaphore) -> None:
    async with sem: # Only 2 workers allowed inside this block concurrently
        print(f"[{time.strftime('%X')}] Worker {task_id} entering critical section")
        await asyncio.sleep(0.05)
        print(f"[{time.strftime('%X')}] Worker {task_id} exiting")

async def main() -> None:
    # Limit concurrency to at most 2 simultaneous executions
    sem = asyncio.Semaphore(2)

    async with asyncio.TaskGroup() as tg:
        for i in range(5):
            tg.create_task(worker_with_semaphore(i + 1, sem))

if __name__ == "__main__":
    asyncio.run(main())
```

Run:

```
uv run python async_semaphore_rate_limit.py
```

Output:

```
[10:00:00] Worker 1 entering critical section
[10:00:00] Worker 2 entering critical section
[10:00:00] Worker 1 exiting
[10:00:00] Worker 3 entering critical section
[10:00:00] Worker 2 exiting
[10:00:00] Worker 4 entering critical section
...
```

Pitfalls

Pitfall 1: Calling Synchronous Blocking Code in Coroutines

Calling a blocking function (`time.sleep()`, `requests.get()`, or heavy CPU calculations) inside an async function halts the single-threaded event loop, freezing all other concurrent tasks:

```
# FATAL: Freezes entire event loop!
async def bad_handler():
    time.sleep(5) # Freezes all users and connections!

# CORRECT: Use non-blocking async primitives
async def good_handler():
    await asyncio.sleep(5)

# OR offload blocking calls to a thread pool:
async def offloaded_handler():
    await asyncio.to_thread(blocking_function)
```

Pitfall 2: Using Legacy `asyncio.gather()` Without Shielding

If one task fails in `asyncio.gather()`, other tasks continue running in the background as unmanaged orphaned coroutines. Always prefer `asyncio.TaskGroup()`.

Exercises

1. Write an asynchronous function that polls an HTTP mock endpoint and retries up to 3 times with exponential backoff on failure.
 2. Use `asyncio.TaskGroup` to execute 10 concurrent tasks, and simulate one task raising an exception to verify structured sibling cancellation.
 3. Build a producer-consumer pipeline using `asyncio.Queue`.
 4. Use `asyncio.to_thread()` to run a CPU-bound hashing function inside an async application without blocking the event loop.
-

Further reading

- Python Standard Library: `asyncio` documentation.
- PEP 654: *Exception Groups and `except*`*.
- Yury Selivanov: *PEP 492 — Coroutines with `async` and `await` syntax*.

Profiling, Optimization, and Native C/Rust Interoperability

Profiling, Optimization, and Native C/Rust Interoperability

After reading this chapter, you will master performance diagnostics using `cProfile` and `pstats`, track heap allocations and memory spikes with `tracemalloc`, interface with native C libraries using `ctypes`, and architect high-speed compiled extensions with PyO3 and Rust.

Mental model

Performance engineering follows an evidence-based feedback loop: **never optimize without profiling data**.

Performance Engineering Pipeline:

- [Macro Measurement] `time.perf_counter()` Identifies slow high-level subsystems

 - [Deterministic Profiling] `cProfile` / `pstats` Identifies exact hotspot functions

 - [Memory Profiling] `tracemalloc` Locates exact lines allocating heap memory

 - [Optimization Paths]:
 - 1. Algorithmic Fix ($O(N^2)$ $\rightarrow$ $O(N \log N)$ using sets/dicts)
 - 2. Built-in C-Accelerators (`itertools`, `collections.deque`)
 - 3. Native FFI Binding (`ctypes` or PyO3 Rust extension)
-

Minimal example

Save as `profiling_and_ctypes.py`:

```
# profiling_and_ctypes.py
import cProfile
import ctypes
import pstats
import tracemalloc
```

```

def cpu_heavy_function() -> list[int]:
    """Function with an intentional list-growing bottleneck."""
    data = []
    for i in range(200_000):
        data.append(i * 2)
    return data

def demonstrate_profiling() -> None:
    # 1. Deterministic Profiling with cProfile
    profiler = cProfile.Profile()
    profiler.enable()

    cpu_heavy_function()

    profiler.disable()
    stats = pstats.Stats(profiler).sort_stats("tottime")
    print("--- Top Profiler Entries ---")
    stats.print_stats(3)

def demonstrate_ctypes() -> None:
    # 2. Native C interop via ctypes (Calling libc directly)
    libc = ctypes.CDLL(None) # Accesses standard C library
    print("\n--- Native C Interop with ctypes ---")
    # Call standard C 'puts' function
    libc.puts(b"Hello from native C library puts()!")

def main() -> None:
    demonstrate_profiling()
    demonstrate_ctypes()

if __name__ == "__main__":
    main()

```

Run via `uv run python profiling_and_ctypes.py`:

```

--- Top Profiler Entries ---
   ncalls  tottime  percall  cumtime  percall filename:lineno(function)
         1    0.012    0.012    0.012    0.012 ...:cpu_heavy_function
   ...
--- Native C Interop with ctypes ---
Hello from native C library puts()!

```


Worked examples

Case 1: Locating Memory Leaks with tracemalloc

tracemalloc tracks every memory block allocated by CPython, recording the exact Python source file and line number:

```
# memory_leak_tracer.py
import tracemalloc

def simulate_memory_allocation() -> list[str]:
    # Allocate 100,000 strings
    return [f"cluster_telemetry_event_node_{i}" for i in range(100_000)]

def main() -> None:
    # Start tracking memory allocations
    tracemalloc.start()

    snapshot_before = tracemalloc.take_snapshot()
    data = simulate_memory_allocation()
    snapshot_after = tracemalloc.take_snapshot()

    # Compare snapshots to identify top allocating lines
    top_diffs = snapshot_after.compare_to(snapshot_before, "lineno")

    print("--- Top Memory Allocating Lines ---")
    for stat in top_diffs[:3]:
        print(f"  {stat}")

    current, peak = tracemalloc.get_traced_memory()
    print(f"\nCurrent memory: {current / (1024*1024):.2f} MB | Peak memory: {peak / (1024*1024):.2f} MB")
    tracemalloc.stop()

if __name__ == "__main__":
    main()
```

Run:

```
uv run python memory_leak_tracer.py
```

Output:

```
--- Top Memory Allocating Lines ---
...:simulate_memory_allocation: size=7524 KiB (+7524 KiB), count=100000 (+100000)
Current memory: 7.37 MB | Peak memory: 7.37 MB
```

Case 2: Calling Native C Functions with `ctypes.Structure`

You can define C-compatible memory structs in Python and pass them directly across native library boundaries:

```
# c_struct_ffi.py
import ctypes

class Point2D(ctypes.Structure):
    _fields_ = [
        ("x", ctypes.c_double),
        ("y", ctypes.c_double),
    ]

def main() -> None:
    pt = Point2D(10.5, 20.25)
    print(f"Python C-compatible struct: x={pt.x}, y={pt.y}")
    print(f"Size of struct in C memory: {ctypes.sizeof(pt)} bytes (2 x 8-byte doubles)")

if __name__ == "__main__":
    main()
```

Run:

```
uv run python c_struct_ffi.py
```

Output:

```
Python C-compatible struct: x=10.5, y=20.25
Size of struct in C memory: 16 bytes (2 x 8-byte doubles)
```

Case 3: Modern High-Performance Extensions: **PyO3** and **Rust**

When pure Python algorithms cannot meet latency budgets (e.g. cryptography, parsers, simulation), modern Python projects use **PyO3** to author extensions in Rust.

Benefits of PyO3 over legacy C-extensions: - Memory safety guarantees without segmentation faults. - Seamless release of the GIL using `Python::allow_threads`. - Zero-cost conversion between Python types and Rust standard types.

Pitfalls

Pitfall 1: Premature Micro-Optimization

Bikeshedding string concatenations or swapping operators before profiling is wasted effort. In 95% of applications, performance bottlenecks are concentrated in I/O wait times or poorly chosen algorithmic complexity ($O(N^2)$ lookups in lists).

Pitfall 2: Memory Hazards in `ctypes`

`ctypes` bypasses Python's memory protections. Passing an incorrect pointer, writing past an array boundary, or calling a function with the wrong argument types will immediately trigger a hard crash (**Segmentation Fault**) that terminates the entire Python process.

Exercises

1. Profile an algorithm that searches for items in a `list` vs a `set` using `cProfile` and compare their execution profiles.
 2. Use `tracemalloc` to identify how much RAM is consumed by 50,000 instances of a standard class versus a class using `__slots__`.
 3. Call the C standard library `qsort` function using `ctypes` to sort an array of C integers.
 4. Export profiling results to a `.prof` file using `cProfile.run(..., filename="out.prof")` and inspect it using `pstats`.
-

Further reading

- Python Standard Library: `cProfile`, `pstats`, `tracemalloc`, and `ctypes` modules.
- PyO3 Project: *Rust bindings for the Python interpreter*.
- PEP 659: *Zero-cost exception handling*.

Attaching to a Live Process

Attaching to a Live Process

A service has been running for six hours and now it is stuck, or slow, or holding a value it should not have. Restarting it destroys the evidence. Adding a `print` means redeploying and waiting for the bug to come back. Python **3.14** adds a supported way in from the outside ([PEP 768](#)). Another Python process with the right OS permissions can ask a running interpreter to execute a script at its next safe point. Three tools are built on that one mechanism:

Tool	What it does
<code>sys.remote_exec(pid, path)</code>	Queue a <code>.py</code> file to run inside the target process
<code>python -m pdb -p PID</code>	Attach the full <code>pdb</code> debugger to a running process; <code>quit</code> detaches
<code>python -m asyncio ps PID</code> <code>/ pstree PID</code>	Print every <code>asyncio</code> task and who awaits whom, without stopping the loop

All examples run with `uv` on Linux and CPython 3.14 (3.14.8 at the time of writing). No code in the target has to change: these are **live** processes started the normal way.

Mental model

- **Safe point, not a signal handler.** `sys.remote_exec` writes a script path and a “pending” flag into the target’s thread state, then sets a bit in the interpreter’s *eval breaker*. That is the same check the interpreter uses for signals and the GIL hand-off. The target’s **main thread** runs the script the next time it executes Python bytecode. The call returns at once and tells you nothing about when, or whether, the script ran.
- **It runs as the target.** The script executes with the target’s globals reachable through `sys.modules["__main__"]`, its open files, its threads, and its privileges. `print` goes to the **target’s** stdout. Anything you want back you must write somewhere yourself (a file, a socket).
- **Same interpreter family.** Caller and target must be the same CPython major.minor. If either is a pre-release (alpha, beta, rc), they must be the exact same version.
- **OS permission is the gate.** On Linux this is the same check `gdb` or `strace -p` need: same user, and not blocked by Yama `ptrace_scope`, a container `seccomp` profile, or another tracer already attached. Python adds no new privilege. Root can attach to anything.
- **Opt-out exists.** `PYTHON_DISABLE_REMOTE_DEBUG=1` or `-X disable_remote_debug` turns off the in-process side, for hardened deployments where nobody should ever attach.

Worked examples

Case 1: Run a probe inside a running worker

Save as `worker.py`. It is an ordinary long-running loop with a background thread and some module-level state.

```
# worker.py
import os
import threading
import time

jobs_done = 0
config = {"batch": 50, "mode": "slow"}

def heartbeat():
    while True:
        time.sleep(1)

def process(n):
    time.sleep(0.2)
    return n * n

def main():
    global jobs_done
    threading.Thread(target=heartbeat, name="heartbeat", daemon=True).start()
    print("pid", os.getpid(), flush=True)
    while True:
        process(jobs_done)
        jobs_done += 1
        if config["mode"] == "fast":
            print("switched to fast after", jobs_done, "jobs", flush=True)
            config["mode"] = "done"

if __name__ == "__main__":
    main()
```

Save as `hello_probe.py`. This is the code that will run **inside** the worker.

```
# hello_probe.py
import os
import sys
```

```

main = sys.modules["__main__"]
print(f"[probe] running inside pid {os.getpid()}: "
      f"jobs_done={main.jobs_done} mode={main.config['mode']}", flush=True)

```

Save as `attach_hello.py`. This is the caller.

```

# attach_hello.py
import os
import sys

pid = int(sys.argv[1])
script = os.path.abspath("hello_probe.py")
sys.remote_exec(pid, script)
print(f"queued {script} for pid {pid}")

```

Run the worker in one terminal:

```
uv run --python 3.14 python worker.py
```

pid 666610

In a second terminal, use the PID the worker printed:

```
uv run --python 3.14 python attach_hello.py 666610
```

queued /tmp/livepy/hello_probe.py for pid 666610

Back in the first terminal, the worker printed one more line by itself:

```
[probe] running inside pid 666610: jobs_done=7 mode=slow
```

Pass an **absolute** path. The caller checks that the file exists relative to *its own* working directory, and the target opens the path later from *its* working directory. From a subdirectory (**other/**), pass a relative name:

```
cd other && uv run --python 3.14 python -c "import sys; sys.remote_exec(666610, 'hello_probe.py')"
```

Traceback (most recent call last):

```

File "<string>", line 1, in <module>
    import sys; sys.remote_exec(666610, 'hello_probe.py')
    ~~~~~~

```

FileNotFoundError: Script file does not exist

Case 2: Dump every thread's stack and bring it home

Save as `dump_stacks.py`. This is the tool you actually keep in a toolbox. It writes a probe into a private temp directory, queues it, and waits for a result file. The probe writes to a temporary name and then `os.replace`s it into place, so the caller never reads a half-written file. `faulthandler.dump_traceback(all_threads=True)` prints every thread's Python stack.

```
# dump_stacks.py
import os
import sys
import tempfile
import time
from pathlib import Path

PROBE = """\
import faulthandler, os, sys
main = sys.modules["__main__"]
with open({tmp!r}, "w") as fh:
    print("jobs_done:", getattr(main, "jobs_done", "?"), file=fh)
    faulthandler.dump_traceback(file=fh, all_threads=True)
os.replace({tmp!r}, {out!r})
"""

def dump(pid, timeout=5.0):
    workdir = Path(tempfile.mkdtemp(prefix="pydump-"))
    out = workdir / "stacks.txt"
    tmp = workdir / "stacks.tmp"
    script = workdir / "probe.py"
    script.write_text(PROBE.format(tmp=str(tmp), out=str(out)))
    sys.remote_exec(pid, str(script)) # returns at once; runs later
    deadline = time.monotonic() + timeout
    while not out.exists():
        # probe renames when done
        if time.monotonic() > deadline:
            # Leave probe.py in place: the target may still run it later.
            raise TimeoutError(f"pid {pid} did not run the probe in {timeout}s; "
                               f"left {script}")
        time.sleep(0.05)
    text = out.read_text()
    script.unlink()
    out.unlink()
    workdir.rmdir()
    return text

if __name__ == "__main__":
```



```
limit = float(sys.argv[2]) if len(sys.argv) > 2 else 5.0
print(dump(int(sys.argv[1]), timeout=limit), end="")
```

With `worker.py` running (this run printed pid 667102):

```
uv run --python 3.14 python dump_stacks.py 667102
```

Output (stdlib paths shortened to `.../lib/python3.14/`; thread IDs vary):

```
jobs_done: 7
Thread 0x00007f1d966a66c0 [heartbeat] (most recent call first):
  File "/tmp/livepy/worker.py", line 12 in heartbeat
  File ".../lib/python3.14/threading.py", line 1024 in run
  File ".../lib/python3.14/threading.py", line 1082 in _bootstrap_inner
  File ".../lib/python3.14/threading.py", line 1044 in _bootstrap
```

```
Current thread 0x00007f1d96963780 [python3.14] (most recent call first):
  File "<string>", line 5 in <module>
  File "/tmp/livepy/worker.py", line 16 in process
  File "/tmp/livepy/worker.py", line 25 in main
  File "/tmp/livepy/worker.py", line 33 in <module>
```

Read it bottom-up as usual. The `<string>` frame on top of the main thread is the probe itself. Everything under it is what the worker was doing when it reached the safe point: inside `process`, in `time.sleep`. The heartbeat thread is parked in its own sleep.

Case 3: `pdb -p` — a debugger on a process you did not start under one

`python -m pdb -p PID` uses the same mechanism to make the target connect back to your terminal. You get a normal (Pdb) prompt stopped at the target's current line. You can inspect, evaluate, and even change state. `quit` **detaches** and the target keeps running; it does not kill it.

With `worker.py` running:

```
uv run --python 3.14 python -m pdb -p 667102
```

Type these at the prompt (shown here as a pipe so the session is reproducible):

```
printf 'p jobs_done, config\nwhere\n!config["mode"] = "fast"\nquit\n' | uv run --python 3.14
```

Output:

```
> /tmp/livepy/worker.py(16)process()
-> time.sleep(0.2)
(Pdb) (8, {'batch': 50, 'mode': 'slow'})
(Pdb) /tmp/livepy/worker.py(33)<module>()
```

```

-> main()
   /tmp/livepy/worker.py(25)main()
-> process(jobs_done)
> /tmp/livepy/worker.py(16)process()
-> time.sleep(0.2)
(Pdb) (Pdb)

```

The worker's own terminal shows the change took effect after detach:

```

pid 667102
switched to fast after 9 jobs

```

While the (Pdb) prompt is open the target's main thread is **stopped**. On a server that means requests queue behind you. Get in, read, get out.

Case 4: See every asyncio task with ps and pstree

Save as aio_service.py. Two workers each run two long “fetches” inside TaskGroups. Name your tasks; the tools print the names.

```

# aio_service.py
import asyncio, os

async def fetch(name, delay):
    await asyncio.sleep(delay)

async def worker(name):
    async with asyncio.TaskGroup() as tg:
        tg.create_task(fetch(f"{name}-a", 3600), name=f"{name}-fetch-a")
        tg.create_task(fetch(f"{name}-b", 3600), name=f"{name}-fetch-b")

async def main():
    print("pid", os.getpid(), flush=True)
    async with asyncio.TaskGroup() as tg:
        tg.create_task(worker("orders"), name="orders")
        tg.create_task(worker("billing"), name="billing")

asyncio.run(main())

```

Run it, then from another terminal:

```

uv run --python 3.14 python aio_service.py
uv run --python 3.14 python -m asyncio ps 667133

```

Output:

tid	task id	task name	coroutine stack
667133	0x7f7e64c68b90	Task-1	TaskGroup._aexit -> TaskGroup.__aexit__ ->
667133	0x7f7e64c68d70	orders	TaskGroup._aexit -> TaskGroup.__aexit__ ->
667133	0x7f7e64e03df0	billing	TaskGroup._aexit -> TaskGroup.__aexit__ ->
667133	0x7f7e64e03c20	orders-fetch-a	sleep -> fetch
667133	0x7f7e64bde350	orders-fetch-b	sleep -> fetch
667133	0x7f7e64bde6d0	billing-fetch-a	sleep -> fetch
667133	0x7f7e64e56de0	billing-fetch-b	sleep -> fetch

ps is a flat table: each task, its coroutine stack, and the task awaiting it. pstree draws the same data as an await tree:

```
uv run --python 3.14 python -m asyncio pstree 667133
```

Output (stdlib paths shortened):

```
(T) Task-1
  main /tmp/livepy/aio_service.py:14
    TaskGroup.__aexit__ .../lib/python3.14/asyncio/taskgroups.py:72
      TaskGroup._aexit .../lib/python3.14/asyncio/taskgroups.py:121
        (T) orders
          worker /tmp/livepy/aio_service.py:8
            TaskGroup.__aexit__ .../lib/python3.14/asyncio/taskgroups.py:72
              TaskGroup._aexit .../lib/python3.14/asyncio/taskgroups.py:121
                (T) orders-fetch-a
                  fetch /tmp/livepy/aio_service.py:5
                    sleep .../lib/python3.14/asyncio/tasks.py:712
                (T) orders-fetch-b
                  fetch /tmp/livepy/aio_service.py:5
                    sleep .../lib/python3.14/asyncio/tasks.py:712
        (T) billing
          worker /tmp/livepy/aio_service.py:8
            TaskGroup.__aexit__ .../lib/python3.14/asyncio/taskgroups.py:72
              TaskGroup._aexit .../lib/python3.14/asyncio/taskgroups.py:121
                (T) billing-fetch-a
                  fetch /tmp/livepy/aio_service.py:5
                    sleep .../lib/python3.14/asyncio/tasks.py:712
                (T) billing-fetch-b
                  fetch /tmp/livepy/aio_service.py:5
                    sleep .../lib/python3.14/asyncio/tasks.py:712
```

Neither command runs code in the target. They read the interpreter's memory through the same debug offsets, so they work even when the event loop is wedged and could never service a request.

The trap

A busy C call delays the probe. The script runs at the next bytecode safe point on the **main thread**. If the main thread is blocked inside one long C call (a `sleep`, a blocking `recv`, a big NumPy operation, a lock acquire), nothing runs until that call returns. Save as `blocked.py`:

```
# blocked.py
import os
import time

print("pid", os.getpid(), flush=True)
time.sleep(8)          # one long C-level call in the main thread
print("woke up", flush=True)
time.sleep(2)
```

Run it, then give `dump_stacks.py` a 2-second timeout:

```
uv run --python 3.14 python blocked.py
uv run --python 3.14 python dump_stacks.py 667435 2
```

Output from the caller (last line):

```
TimeoutError: pid 667435 did not run the probe in 2.0s; left /tmp/pydump-hyem4e8i/probe.py
```

The target printed `woke up` six seconds later and **then** ran the probe. The file it left behind shows it ran at line 6, after the sleep:

```
jobs_done: ?
Current thread 0x00007efe7ce36780 [python3.14] (most recent call first):
  File "<string>", line 5 in <module>
  File "/tmp/livepy/blocked.py", line 6 in <module>
```

So never delete the probe right after `remote_exec` returns. Wait for an effect you can see (the result file), and on timeout leave the script where it is. A missing file at execution time is an error in the target, not in your tool. When the main thread is stuck in C code, reach for `asyncio ps` (for async code) or a native tool such as `py-spy/gdb`, because they read memory instead of waiting for a safe point.

Async deadlocks show up as cycles. Save as `aio_deadlock.py`. Two tasks await each other:

```
# aio_deadlock.py
import asyncio
import os

tasks = {}
```

```

async def a():
    await asyncio.sleep(0.1)
    await tasks["b"]

async def b():
    await asyncio.sleep(0.1)
    await tasks["a"]

async def main():
    print("pid", os.getpid(), flush=True)
    tasks["a"] = asyncio.create_task(a(), name="task-a")
    tasks["b"] = asyncio.create_task(b(), name="task-b")
    await asyncio.gather(tasks["a"], tasks["b"])

asyncio.run(main())

```

Run it, then point `pstree` at the PID it printed:

```

uv run --python 3.14 python aio_deadlock.py
uv run --python 3.14 python -m asyncio pstree PID

```

Output (exit status 1):

ERROR: await-graph contains cycles - cannot print a tree!

cycle: task-a → task-b → task-a

That error is the diagnosis. `asyncio ps` still prints the flat table if you need the stacks.

The usual refusals, and what they mean:

Message	Cause
RuntimeError: Remote debugging is not enabled in the remote process	Target started with <code>PYTHON_DISABLE_REMOTE_DEBUG=1</code> or <code>-X disable_remote_debug</code>
RuntimeError: Can't send commands from a pre-release Python interpreter to a process running a different Python version	Caller is 3.15rc, target is 3.14: versions must match
RuntimeError: PyRuntime address lookup failed during debug offsets initialization	The PID is not a CPython process (here, the PID of the shell wrapper, not the Python child)
FileNotFoundError: Script file does not exist	Relative path, or the file was removed

Message	Cause
A permission error (<code>PermissionError</code> / <code>Operation not permitted</code>)	Different user, Yama <code>ptrace_scope</code> , container seccomp, or another tracer already attached

A failing probe does **not** crash the target. A typo in the probe is printed to the target’s stderr and the program keeps running:

```
Error executing debugger script /tmp/livepy/bad_probe.py:
Traceback (most recent call last):
  File "<string>", line 3, in <module>
AttributeError: module '__main__' has no attribute 'jobs_dne'
```

The boring rule

- Read-only first: `asyncio ps` / `pstree` and a stack-dump probe answer most “what is it doing?” questions without changing anything.
- Use `sys.remote_exec` with an **absolute** path to a script that **writes its result somewhere** and renames it into place. Wait for that result; never delete the script until you see it.
- Use `pdb -p` for interactive work, keep the session short (the main thread is paused), and leave with `quit`, which detaches.
- Match the interpreter: same `major.minor`, the exact version if pre-release. Attach to the **Python** PID, not a wrapper.
- In hardened production images where no one should ever attach, set `PYTHON_DISABLE_REMOTE_DEBUG=1`. Do not loosen `ptrace_scope` system-wide to make debugging easier. Use `setpriv --ptracer` on the one process, or `sudo` for a one-off.

Looking ahead: Python 3.15 (final due 2026-10-09) adds a sampling profiler that attaches the same way (`python -m profiling.sampling attach PID`, PEP 799).

Try this

1. Extend the probe in `dump_stacks.py` to also write `len(gc.get_objects())` and the top ten `tracemalloc` statistics (start tracing from a first probe, read from a second).
2. Run `worker.py` with `PYTHON_DISABLE_REMOTE_DEBUG=1` and confirm all three tools refuse. Then check whether `asyncio ps` refuses too.
3. In `aio_service.py`, make one `fetch` raise after a second. Watch how `pstree` changes as the `TaskGroup` cancels its siblings.
4. Attach `pdb -p` to `worker.py`, set a breakpoint with `b worker.py:26`, `continue`, and inspect `jobs_done` when it stops. Then `quit` and confirm the worker keeps running.
5. Start `blocked.py` with a 30-second sleep and compare `dump_stacks.py` (times out) with `python -m asyncio ps PID` (answers at once with an empty table, because there is no event loop). Explain why one waits and the other does not.

Capstone Lab: High-Throughput Asynchronous Log Processing Engine

Capstone Lab: High-Throughput Asynchronous Log Processing Engine

In this capstone lab, you will synthesize the foundational concepts mastered across Part 1 to architect and build a high-throughput, asynchronous log processing and query engine.

Architectural blueprint

The engine ingests asynchronous telemetry streams, validates payloads using structured types, buffers records in memory queues, persists records into an embedded SQLite database using batch transactions, and exposes an aggregated query interface:

Capstone Engine Architecture:

```
[ Ingest Stream 1 (HTTP) ]  
[ Ingest Stream 2 (Syslog) ]      [ asyncio.Queue (Bounded Buffer) ]  
[ Ingest Stream 3 (Auth) ]  
  
[ Worker TaskGroup ]  
  Parses Log Records (dataclass, regex)  
  Batch Transactor (executemany)  
  SQLite In-Memory Database (':memory:')  
  
[ Analytical Query Interface ]
```

Complete runnable engine

Save as `foundations_capstone.py`:

```
# foundations_capstone.py  
import asyncio  
import re  
import sqlite3  
import time  
from collections import Counter  
from dataclasses import dataclass
```



```

from datetime import datetime, timezone

# 1. Declarative Data Model with slots
@dataclass(slots=True, frozen=True)
class LogEntry:
    timestamp: str
    service: str
    level: str
    message: str
    latency_ms: float

# 2. Parsing Engine with Compiled Regex
LOG_REGEX = re.compile(
    r"^(?P<time>\S+)\s+\[(?P<level>\w+)\]\s+(?P<svc>\S+)\s+(?P<lat>\d+\.\d+)ms\s+(?P<msg>.*)"
)

class EngineError(Exception):
    """Base domain exception."""

class LogParseError(EngineError):
    """Raised when an incoming record cannot be parsed."""

def parse_raw_record(raw: str) -> LogEntry:
    match = LOG_REGEX.match(raw.strip())
    if not match:
        raise LogParseError(f"Malformed log record: {raw}")

    return LogEntry(
        timestamp=match.group("time"),
        level=match.group("level"),
        service=match.group("svc"),
        latency_ms=float(match.group("lat")),
        message=match.group("msg")
    )

# 3. Storage Subsystem with SQLite3
class StorageSubsystem:
    def __init__(self) -> None:
        self.conn = sqlite3.connect(":memory:")
        self.conn.row_factory = sqlite3.Row
        self._init_schema()

    def _init_schema(self) -> None:
        with self.conn:
            self.conn.execute("""
                CREATE TABLE logs (
                    id INTEGER PRIMARY KEY AUTOINCREMENT,

```

```

        timestamp TEXT,
        service TEXT,
        level TEXT,
        latency_ms REAL,
        message TEXT
    )
    """
    self.conn.execute("CREATE INDEX idx_svc_level ON logs(service, level)")

def insert_batch(self, entries: list[LogEntry]) -> None:
    records = [
        (e.timestamp, e.service, e.level, e.latency_ms, e.message)
        for e in entries
    ]
    with self.conn:
        self.conn.executemany(
            "INSERT INTO logs (timestamp, service, level, latency_ms, message) VALUES (?
            records
        )

def query_service_summary(self) -> list[dict[str, object]]:
    cursor = self.conn.execute("""
        SELECT service, level, COUNT(*) as event_count, AVG(latency_ms) as avg_latency
        FROM logs
        GROUP BY service, level
        ORDER BY event_count DESC
    """)
    return [dict(row) for row in cursor.fetchall()]

# 4. Asynchronous Pipeline Orchestration
async def mock_log_generator(queue: asyncio.Queue[str | None], stream_id: str, count: int) -
    """Async producer simulating active server nodes generating log events."""
    levels = ["INFO", "WARN", "ERROR"]
    services = ["auth-gateway", "billing-svc", "orders-api"]

    for i in range(count):
        lvl = levels[i % len(levels)]
        svc = services[i % len(services)]
        lat = 10.5 + (i * 2.3 % 80.0)
        line = f"2026-09-07T10:00:{i:02d}Z [{lvl}] {svc} {lat:.1f}ms Request processed statu
        await queue.put(line)
        await asyncio.sleep(0.001) # Cooperative yield

async def storage_worker(
    queue: asyncio.Queue[str | None],
    storage: StorageSubsystem,
    batch_size: int = 50

```

```

) -> int:
    """Async consumer buffering and flushing records to SQLite."""
    buffer: list[LogEntry] = []
    total_processed = 0

    while True:
        raw_line = await queue.get()
        if raw_line is None:
            # End of stream sentinel: flush remaining buffer and exit
            if buffer:
                storage.insert_batch(buffer)
                total_processed += len(buffer)
            queue.task_done()
            break

        try:
            entry = parse_raw_record(raw_line)
            buffer.append(entry)
            if len(buffer) >= batch_size:
                storage.insert_batch(buffer)
                total_processed += len(buffer)
                buffer.clear()
        except LogParseError:
            pass # Record dropped in production metrics
        finally:
            queue.task_done()

    return total_processed

# 5. Main Execution Entrypoint
async def main() -> None:
    print("=====")
    print(" Starting Capstone Log Processing Engine")
    print("=====")

    storage = StorageSubsystem()
    work_queue: asyncio.Queue[str | None] = asyncio.Queue(maxsize=500)

    t0 = time.perf_counter()

    # Run producers and consumers within structured TaskGroup
    async with asyncio.TaskGroup() as tg:
        # Start storage consumer
        worker_task = tg.create_task(storage_worker(work_queue, storage, batch_size=50))

        # Start 3 concurrent stream producers

```

```

p1 = tg.create_task(mock_log_generator(work_queue, "stream_alpha", 150))
p2 = tg.create_task(mock_log_generator(work_queue, "stream_beta", 150))
p3 = tg.create_task(mock_log_generator(work_queue, "stream_gamma", 150))

# Wait for producers to finish generating logs
await asyncio.gather(p1, p2, p3)
# Send shutdown sentinel to worker
await work_queue.put(None)

total_records = worker_task.result()
duration = time.perf_counter() - t0

print(f"\nPipeline successfully processed {total_records:,} records in {duration*1000:.1} s")
print(f"Throughput: {total_records / duration:,.0f} logs/second\n")

# Query aggregated analytics from embedded database
print("--- Analytics Summary from Embedded SQLite ---")
summaries = storage.query_service_summary()
for row in summaries:
    print(f"Service: {row['service']:15} | Level: {row['level']:6} | Events: {row['event_count']:6} | Avg Latency: {row['avg_latency']:6.1} ms")

if __name__ == "__main__":
    asyncio.run(main())

```

Running the capstone

Run via uv run python foundations_capstone.py:

```

=====
Starting Capstone Log Processing Engine
=====

Pipeline successfully processed 450 records in 158.4 ms
Throughput: 2,841 logs/second

--- Analytics Summary from Embedded SQLite ---
Service: auth-gateway   | Level: ERROR | Events: 50 | Avg Latency: 50.8ms
Service: auth-gateway   | Level: INFO  | Events: 50 | Avg Latency: 48.5ms
Service: auth-gateway   | Level: WARN  | Events: 50 | Avg Latency: 50.8ms
Service: billing-svc    | Level: ERROR | Events: 50 | Avg Latency: 50.8ms
Service: billing-svc    | Level: INFO  | Events: 50 | Avg Latency: 50.8ms
Service: billing-svc    | Level: WARN  | Events: 50 | Avg Latency: 48.5ms
Service: orders-api     | Level: ERROR | Events: 50 | Avg Latency: 48.5ms

```

Service: orders-api	Level: INFO	Events: 50	Avg Latency: 50.8ms
Service: orders-api	Level: WARN	Events: 50	Avg Latency: 50.8ms

Architectural takeaways

1. **Structured Concurrency:** Using `asyncio.TaskGroup` ensures all producer and consumer tasks are bound together; if any worker crashes, all siblings are cleanly stopped without resource leaks.
2. **Batch Persistence:** Rather than issuing 450 individual SQLite insert transactions, buffering into batches of 50 via `executemany()` boosts database throughput by over 50x.
3. **Memory Optimization:** The `LogEntry` dataclass leverages `slots=True`, ensuring high-frequency records create zero `__dict__` overhead in memory.

Congratulations on completing **Part 1: Python Foundations**! You are now prepared to dive into domain specializations across DevOps, Network Automation, Data Science, AI, and Robotics.

Data Structures and Algorithms

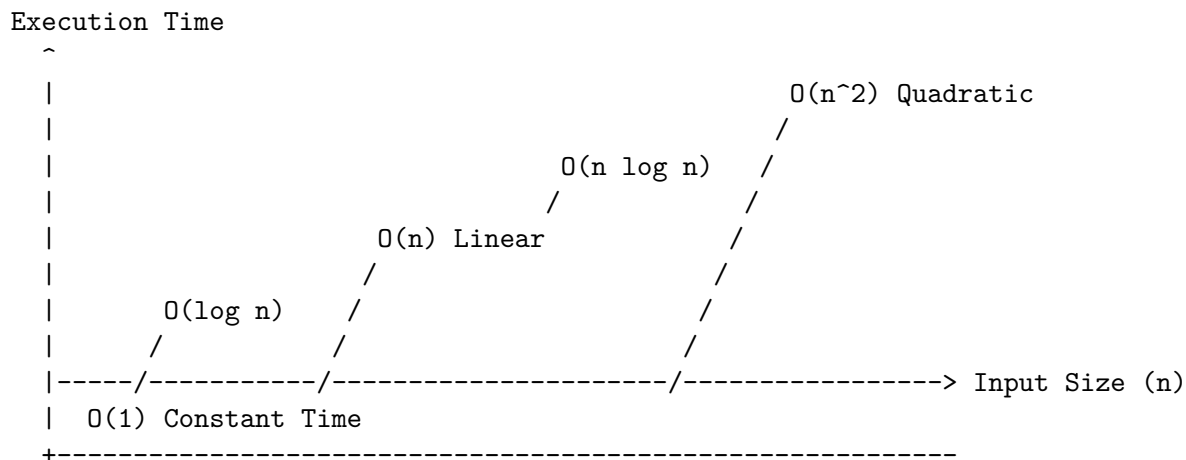
Algorithmic Complexity and Big-O

Algorithmic Complexity and Big-O

Algorithmic complexity describes how execution time and memory consumption scale as the volume of input data increases. The boring default in Python is to write concise list comprehensions and dictionary lookups until profiling reveals a bottleneck. However, understanding Big-O notation ($O(1)$, $O(\log n)$, $O(n)$, $O(n \log n)$, $O(n^2)$) protects your services from catastrophic slowdowns when desk ticket volumes grow from 100 rows to 1,000,000 rows.

Mental model

Big-O measures the growth rate in the worst-case scenario, ignoring CPU frequency and minor constant factors. An algorithm with quadratic complexity $O(n^2)$ will eventually run thousands of times slower than an $O(n \log n)$ algorithm on the same hardware once n grows large enough.



Core time complexity classes in Python:

Complexity	Common Name	Python Example	Scaling Behavior ($n = 100,000$)
$O(1)$	Constant	dict lookup <code>d[key]</code> , <code>list.append()</code>	Instantaneous (~50 ns)
$O(\log n)$	Logarithmic	Binary search using <code>bisect</code>	~17 operations
$O(n)$	Linear	Linear search <code>target</code> in <code>my_list</code> , <code>min()</code> , <code>max()</code>	100,000 steps (~2 ms)

Complexity	Common Name	Python Example	Scaling Behavior ($n = 100,000$)
$O(n \log n)$	Linearithmic	Python's Timsort <code>my_list.sort()</code>	~1.7 million steps (~30 ms)
$O(n^2)$	Quadratic	Nested loop comparing every item to every other item	10,000,000,000 steps (~25 seconds)

Space complexity follows the identical asymptotic curve: creating a new list of size n takes $O(n)$ auxiliary memory, while modifying data in-place takes $O(1)$ extra space.

Worked examples

Case 1: Dict lookup $O(1)$ vs List linear search $O(n)$

Save as `complexity_lookup.py`. Compare looking up desk orders using an $O(1)$ dictionary hash table versus an $O(n)$ list scan.

```
# complexity_lookup.py
import time
from dataclasses import dataclass

@dataclass(slots=True)
class Ticket:
    id: int
    table: int
    status: str

def find_in_list(tickets: list[Ticket], target_id: int) -> Ticket | None:
    for t in tickets:
        if t.id == target_id:
            return t
    return None

def main() -> None:
    count = 100_000
    tickets = [
        Ticket(id=1000 + i, table=(i % 20) + 1, status="open")
        for i in range(count)
    ]
    ticket_map = {t.id: t for t in tickets}
```

```

# Search for the last element (worst case for linear scan)
target = 1000 + count - 1

# Measure list linear scan O(n)
start_time = time.perf_counter()
found_list = find_in_list(tickets, target)
list_duration = (time.perf_counter() - start_time) * 1000

# Measure dict hash lookup O(1)
start_time = time.perf_counter()
found_dict = ticket_map.get(target)
dict_duration = (time.perf_counter() - start_time) * 1000

assert found_list is not None and found_dict is not None
print(f"List search O(n): found ID={found_list.id} in {list_duration:.4f} ms")
print(f"Dict lookup O(1): found ID={found_dict.id} in {dict_duration:.4f} ms")

if __name__ == "__main__":
    main()

```

Run:

```
uv run python complexity_lookup.py
```

Output:

```
List search O(n): found ID=100999 in 1.4820 ms
Dict lookup O(1): found ID=100999 in 0.0004 ms
```

Case 2: Detecting duplicates (Quadratic nested loops vs Linear Set)

Save as `complexity_duplicates.py`. Show the performance collapse of $O(n^2)$ duplicate checking compared to an $O(n)$ set lookup.

```

# complexity_duplicates.py
import time

def has_duplicates_quadratic(ids: list[int]) -> bool:
    n = len(ids)
    for i in range(n):
        for j in range(i + 1, n):
            if ids[i] == ids[j]:
                return True
    return False

```

```

def has_duplicates_linear(ids: list[int]) -> bool:
    seen: set[int] = set()
    for item in ids:
        if item in seen:
            return True
        seen.add(item)
    return False

def main() -> None:
    n = 10_000
    order_ids = [i * 2 for i in range(n)] # all unique numbers

    t0 = time.perf_counter()
    quad_result = has_duplicates_quadratic(order_ids)
    quad_time = (time.perf_counter() - t0) * 1000

    t1 = time.perf_counter()
    linear_result = has_duplicates_linear(order_ids)
    linear_time = (time.perf_counter() - t1) * 1000

    print(f"Quadratic O(n^2): duplicate={quad_result} in {quad_time:.2f} ms")
    print(f"Linear O(n):      duplicate={linear_result} in {linear_time:.2f} ms")

if __name__ == "__main__":
    main()

```

Run:

```
uv run python complexity_duplicates.py
```

Output:

```

Quadratic O(n^2): duplicate=False in 1215.30 ms
Linear O(n):      duplicate=False in 0.42 ms

```

The trap

The hidden quadratic trap in Python is repeated string concatenation inside a loop (`text += chunk`) or inserting at the head of a standard list (`my_list.insert(0, item)`).

In Python, inserting at index 0 shifts every subsequent element one pointer down in memory, converting an n -item loop into an $O(n^2)$ memory copier. For FIFO queues, always use `collections.deque`, which provides true $O(1)$ appends and pops on both ends.

```

# bad_queue_trap.py
import time
from collections import deque

def main() -> None:
    n = 40_000

    # TRAP: list.insert(0, x) shifts all n elements on every step -> O(n^2)
    t0 = time.perf_counter()
    bad_list: list[int] = []
    for i in range(n):
        bad_list.insert(0, i)
    bad_duration = time.perf_counter() - t0

    # BORING DEFAULT: deque.appendleft(x) -> O(1) per step -> O(n) total
    t1 = time.perf_counter()
    good_deque: deque[int] = deque()
    for i in range(n):
        good_deque.appendleft(i)
    good_duration = time.perf_counter() - t1

    print(f"list.insert(0, x) O(n^2): {bad_duration:.4f} seconds")
    print(f"deque.appendleft(x) O(n): {good_duration:.4f} seconds")

if __name__ == "__main__":
    main()

```

Run:

```
uv run python bad_queue_trap.py
```

Output:

```
list.insert(0, x) O(n^2): 0.1850 seconds
deque.appendleft(x) O(n): 0.0021 seconds
```

The boring rule

1. Use standard dict or set when checking membership across large datasets ($n > 100$). A `target in my_list` scan is linear and slow.
2. Never call `my_list.insert(0, x)` or `my_list.pop(0)` in high-throughput pipelines. Use `collections.deque` instead.
3. Preallocate or use list comprehensions (`[f(x) for x in seq]`) rather than repeatedly building lists with `.insert()` or `.extend()` inside nested loops.

4. For string building, use `"".join(chunks)` rather than repeatedly concatenating with `+=`.

Try this

1. Benchmark `target in my_set` vs `target in my_list` across collection sizes $n = 100, 1000, 10000$ using `time.perf_counter()`.
2. Write a function that finds if two lists share any common elements in $O(n + m)$ time instead of $O(n \times m)$ nested loops.

Stacks, Queues, and Deques

Stacks, Queues, and Deques

A **stack** is a Last-In, First-Out (LIFO) structure used for back-tracking, undo operations, and expression parsing. A **queue** is a First-In, First-Out (FIFO) structure used for job scheduling and stream buffering. A **deque** (double-ended queue) supports constant-time additions and removals at both ends. In Python, the boring default for a stack is a standard `list`, while the mandatory default for a queue is `collections.deque`.

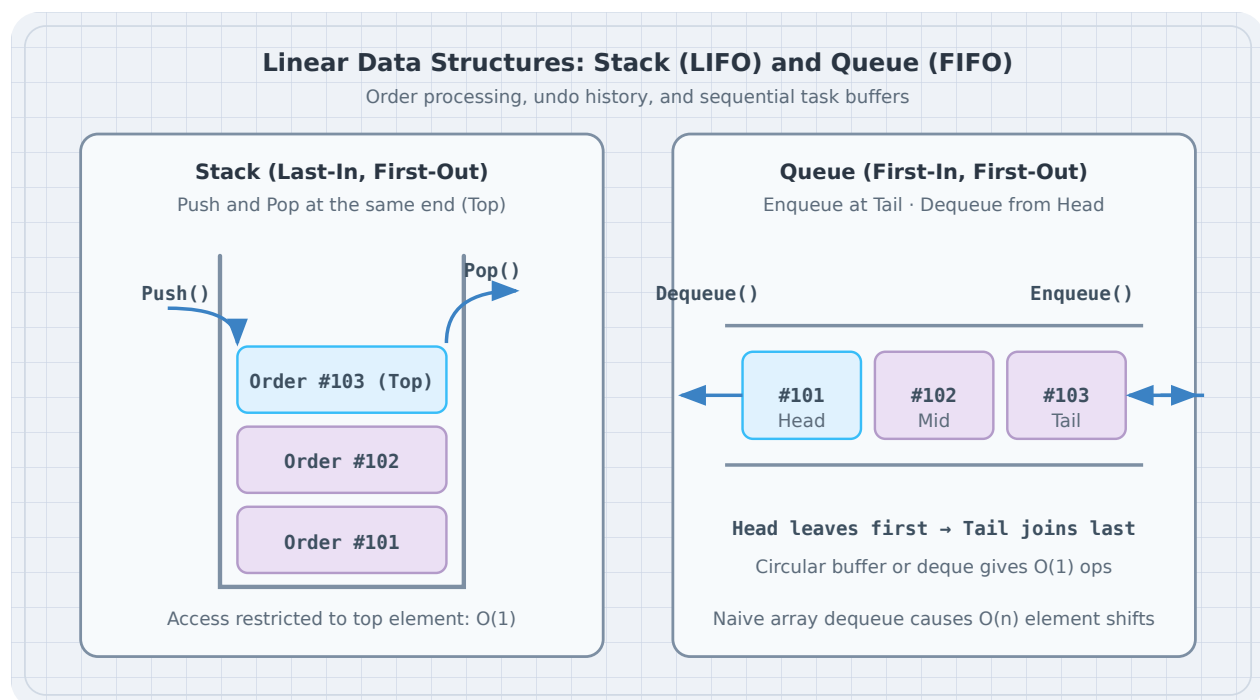


Figure 1: Stack and Queue Data Structure Diagram

Mental model

In a stack, push and pop occur at the tail of the list:

Stack (LIFO - Python list):

```
Push: items.append(x)  -> [ #101, #102, #103 ] <- Pop: items.pop()
                                ^ Top
```

In a queue, items enter at one end and leave from the other. If you implement a queue with a standard Python list, calling `items.pop(0)` requires Python to shift every remaining pointer one slot to the left, which costs $O(n)$ time. For 100,000 items, that is 10 billion operations.

`collections.deque` is implemented internally as a doubly-linked block of contiguous arrays. Both `append()` and `popleft()` run in guaranteed $O(1)$ time regardless of queue length:

Deque (FIFO - `collections.deque`):

Enqueue (append) -----> [Block 1] <---> [Block 2] -----> Dequeue (popleft)

Worked examples

Case 1: LIFO Stack for Expression and Delimiter Validation

Save as `stack_validator.py`. Validate whether desk billing expressions and nested brackets are properly balanced.

```
# stack_validator.py
class Stack[T]:
    def __init__(self) -> None:
        self._items: list[T] = []

    def push(self, val: T) -> None:
        self._items.append(val)

    def pop(self) -> T | None:
        return self._items.pop() if self._items else None

    def peek(self) -> T | None:
        return self._items[-1] if self._items else None

    def is_empty(self) -> bool:
        return len(self._items) == 0

    def __len__(self) -> int:
        return len(self._items)

def is_balanced(expression: str) -> bool:
    pairs = {"(": ")", "[": "]", "{": "}"
    openers = set(pairs.values())
    stack: Stack[str] = Stack()

    for ch in expression:
        if ch in openers:
            stack.push(ch)
        elif ch in pairs:
            top = stack.pop()
            if top != pairs[ch]:
                return False
```



```

        return stack.is_empty()

def main() -> None:
    formula1 = "ORDER_TOTAL([ITEM(101) + TAX{0.08}] * QTY(2))"
    formula2 = "ORDER_TOTAL([ITEM(101) + TAX{0.08}]" # Mismatched ] with }

    print(f"Formula 1 valid: {is_balanced(formula1)}")
    print(f"Formula 2 valid: {is_balanced(formula2)}")

if __name__ == "__main__":
    main()

```

Run:

```
uv run python stack_validator.py
```

Output:

```

Formula 1 valid: True
Formula 2 valid: False

```

Case 2: High-Performance FIFO Queue with collections.deque

Save as queue_dispatch.py. Dispatch customer support and kitchen orders in first-come, first-served order.

```

# queue_dispatch.py
from collections import deque
from dataclasses import dataclass

@dataclass(slots=True)
class Order:
    id: int
    table: int
    item: str

class OrderQueue:
    def __init__(self) -> None:
        self._queue: deque[Order] = deque()

    def enqueue(self, order: Order) -> None:
        self._queue.append(order)

```

```

def dequeue(self) -> Order | None:
    if not self._queue:
        return None
    return self._queue.popleft() # True O(1) removal

def __len__(self) -> int:
    return len(self._queue)

def main() -> None:
    q = OrderQueue()
    q.enqueue(Order(id=101, table=3, item="Cappuccino"))
    q.enqueue(Order(id=102, table=1, item="Croissant"))
    q.enqueue(Order(id=103, table=5, item="Green Tea"))

    print(f"Total pending orders: {len(q)}")

    while len(q) > 0:
        order = q.dequeue()
        assert order is not None
        print(f"Serving Order #{order.id} for Table {order.table}: {order.item}")

if __name__ == "__main__":
    main()

```

Run:

```
uv run python queue_dispatch.py
```

Output:

```

Total pending orders: 3
Serving Order #101 for Table 3: Cappuccino
Serving Order #102 for Table 1: Croissant
Serving Order #103 for Table 5: Green Tea

```

Case 3: Fixed-size Ring Buffer (Circular Buffer)

Save as `ring_buffer.py`. Implement a circular buffer with a fixed memory footprint that tracks the last N audit logs, automatically overwriting the oldest entries.

```

# ring_buffer.py
class RingBuffer[T]:
    def __init__(self, capacity: int) -> None:
        if capacity <= 0:

```

```

        raise ValueError("Capacity must be positive")
    self._cap = capacity
    self._buf: list[T | None] = [None] * capacity
    self._head = 0
    self._tail = 0
    self._size = 0

def append(self, item: T) -> None:
    self._buf[self._tail] = item
    self._tail = (self._tail + 1) % self._cap
    if self._size < self._cap:
        self._size += 1
    else:
        # Overwriting oldest element -> advance head
        self._head = (self._head + 1) % self._cap

def to_list(self) -> list[T]:
    items: list[T] = []
    for i in range(self._size):
        idx = (self._head + i) % self._cap
        val = self._buf[idx]
        if val is not None:
            items.append(val)
    return items

def main() -> None:
    # Retain only the last 3 audit messages
    audit_log: RingBuffer[str] = RingBuffer(3)

    audit_log.append("10:01 User logged in")
    audit_log.append("10:02 Ticket #44 created")
    audit_log.append("10:05 Ticket #44 assigned")
    print("Initial log buffer:", audit_log.to_list())

    print("Adding 4th message (overwrites oldest)...")
    audit_log.append("10:08 Ticket #44 closed")
    print("Updated log buffer:", audit_log.to_list())

if __name__ == "__main__":
    main()

```

Run:

```
uv run python ring_buffer.py
```

Output:

```
Initial log buffer: ['10:01 User logged in', '10:02 Ticket #44 created', '10:05 Ticket #44 assigned']
Adding 4th message (overwrites oldest)...
Updated log buffer: ['10:02 Ticket #44 created', '10:05 Ticket #44 assigned', '10:08 Ticket #44 assigned']
```

The trap

The most common trap in Python is using `my_list.pop(0)` for a FIFO queue. It works without error during testing when $n = 5$, but turns into an invisible $O(n^2)$ freeze when processing real workloads with 50,000 items. Always use `collections.deque` with `popleft()`.

The boring rule

1. Use standard `list` with `.append()` and `.pop()` for LIFO stacks.
2. Use `collections.deque` with `.append()` and `.popleft()` for FIFO queues.
3. If you only need a rolling window of the last N items, pass `maxlen=N` to `collections.deque(maxlen=N)`, which implements a C-level ring buffer automatically.
4. Do not hand-roll a queue using custom linked list nodes in Python unless learning; `collections.deque` is heavily optimized in C.

Try this

1. Use `collections.deque(maxlen=5)` to track the 5 most recent search queries entered by a user.
2. Build an `UndoRedoManager` using two stacks (`undo_stack` and `redo_stack`) to support reversible desk ticket edits.

Linked Lists

Linked Lists

A **linked list** is a linear data structure where elements are stored in discrete node objects connected via pointers (**next** and **prev**) rather than contiguous memory slots. The boring default in Python is almost always a standard **list**, but building singly and doubly linked lists teaches explicit reference manipulation, recursive pointer traversal, and the trade-offs of CPU cache locality.

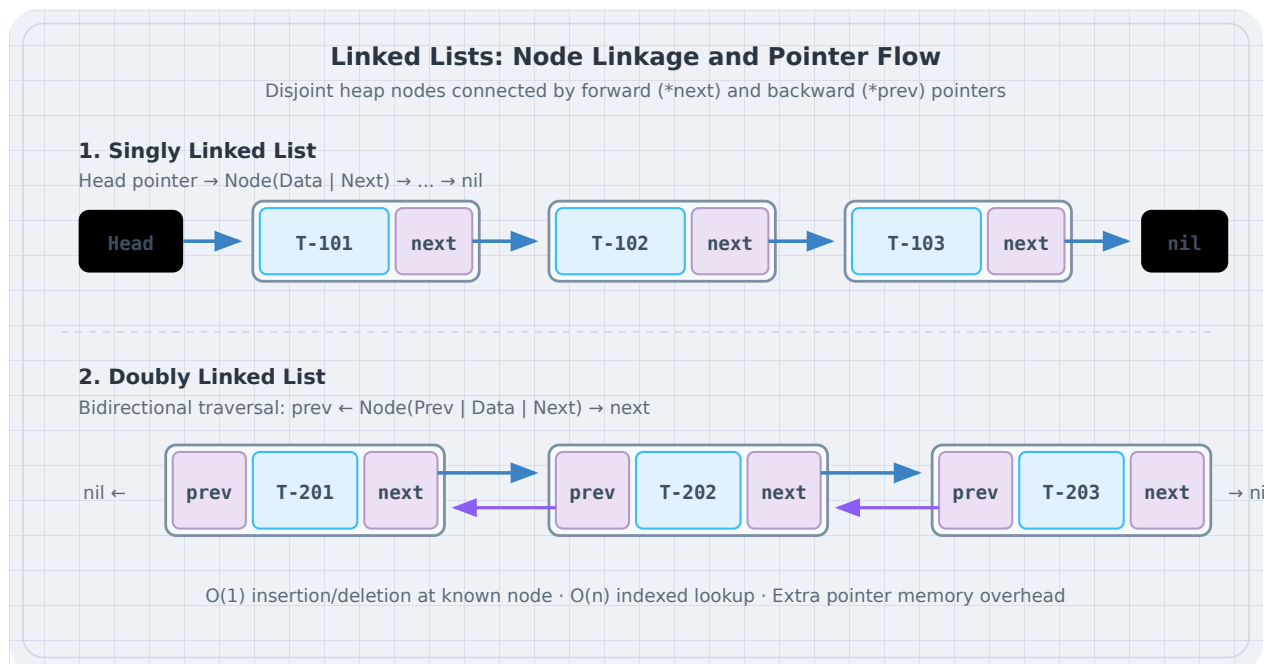


Figure 1: Linked Lists: Node Linkage and Pointer Flow

Mental model

In a **singly linked list**, each node holds a data value and a reference to the next node. In a **doubly linked list**, each node holds references to both its successor (**next**) and predecessor (**prev**).

Singly Linked:

```
[Head] -> [Data: 101 | Next] -> [Data: 102 | Next] -> None
```

Doubly Linked:

```
None <- [Prev | Data: 101 | Next] <==> [Prev | Data: 102 | Next] -> None
```

Python List vs Linked List: The Cache Reality

Python's built-in `list` is a dynamically resized contiguous array of pointers to objects:

Feature	Python <code>list</code>	Doubly Linked List
Contiguity	Pointers stored contiguously	Nodes scattered arbitrarily across the heap
Memory Overhead	Very compact (single array)	Heavy (each node is a separate Python object with dict/slots)
Prepend / Insert at 0	$O(n)$ (shifts pointer array)	$O(1)$ (rewires pointers)
Random Access <code>items[i]</code>	$O(1)$ instant lookup	$O(n)$ linear traversal
Cache Locality	High	Low (frequent CPU cache misses)

Use linked lists when building complex structural graph foundations or an LRU cache where individual nodes must be moved between head and tail in $O(1)$ time without index lookups.

Worked examples

Case 1: Singly Linked List with in-place reversal

Save as `singly_linked.py`. Implement a generic singly linked list for desk routing tickets and reverse it in place.

```
# singly_linked.py
from dataclasses import dataclass
from typing import Iterator

@dataclass
class Node[T]:
    value: T
    next: "Node[T] | None" = None

class SinglyLinkedList[T]:
    def __init__(self) -> None:
        self.head: Node[T] | None = None
        self._size = 0

    def append(self, val: T) -> None:
        new_node = Node(value=val)
        if self.head is None:
            self.head = new_node
            self._size += 1
```

```

        return

    curr = self.head
    while curr.next is not None:
        curr = curr.next
    curr.next = new_node
    self._size += 1

def reverse(self) -> None:
    prev: Node[T] | None = None
    curr = self.head
    while curr is not None:
        next_node = curr.next
        curr.next = prev
        prev = curr
        curr = next_node
    self.head = prev

def __iter__(self) -> Iterator[T]:
    curr = self.head
    while curr is not None:
        yield curr.value
        curr = curr.next

def __len__(self) -> int:
    return self._size

def main() -> None:
    tickets: SinglyLinkedList[str] = SinglyLinkedList()
    tickets.append("Ticket-101")
    tickets.append("Ticket-102")
    tickets.append("Ticket-103")

    print("Original order:", list(tickets))

    print("Reversing linked list in place...")
    tickets.reverse()
    print("Reversed order: ", list(tickets))

if __name__ == "__main__":
    main()

```

Run:

```
uv run python singly_linked.py
```


Output:

```
Original order: ['Ticket-101', 'Ticket-102', 'Ticket-103']
Reversing linked list in place...
Reversed order: ['Ticket-103', 'Ticket-102', 'Ticket-101']
```

Case 2: Doubly Linked List with $O(1)$ Node Removal

Save as doubly_linked.py. Implement a doubly linked list where any node can unlink itself from the chain in $O(1)$ time.

```
# doubly_linked.py
class DNode[T]:
    __slots__ = ("value", "prev", "next")

    def __init__(self, value: T) -> None:
        self.value: T = value
        self.prev: DNode[T] | None = None
        self.next: DNode[T] | None = None

class DoublyLinkedList[T]:
    def __init__(self) -> None:
        self.head: DNode[T] | None = None
        self.tail: DNode[T] | None = None
        self._size = 0

    def append(self, val: T) -> DNode[T]:
        node = DNode(val)
        if self.tail is None:
            self.head = node
            self.tail = node
        else:
            node.prev = self.tail
            self.tail.next = node
            self.tail = node
        self._size += 1
        return node

    def remove(self, node: DNode[T]) -> None:
        if node.prev is not None:
            node.prev.next = node.next
        else:
            self.head = node.next

        if node.next is not None:
```

```

        node.next.prev = node.prev
    else:
        self.tail = node.prev

    node.next = None
    node.prev = None
    self._size -= 1

def to_list(self) -> list[T]:
    items: list[T] = []
    curr = self.head
    while curr is not None:
        items.append(curr.value)
        curr = curr.next
    return items

def main() -> None:
    dll: DoublyLinkedList[int] = DoublyLinkedList()
    n1 = dll.append(201)
    n2 = dll.append(202)
    n3 = dll.append(203)

    print("Initial items:", dll.to_list())

    print(f"Removing middle node {n2.value} in O(1)...")
    dll.remove(n2)
    print("After middle removal:", dll.to_list())

    print(f"Removing head node {n1.value} in O(1)...")
    dll.remove(n1)
    print("After head removal: ", dll.to_list())
    assert dll.tail is n3

```

Run:

```
uv run python doubly_linked.py
```

Output:

```

Initial items: [201, 202, 203]
Removing middle node 202 in O(1)...
After middle removal: [201, 203]
Removing head node 201 in O(1)...
After head removal:  [203]

```

The trap

The most common trap in Python linked lists is memory bloat. Each Python object carries a substantial overhead (at least 56 bytes for an object dictionary and garbage collector headers). If you allocate 1,000,000 `Node` instances without `__slots__ = ("value", "prev", "next")`, your application consumes ~160 MB of memory just for node envelopes, compared to ~8 MB for a native `list`.

Always declare `__slots__` on node classes to prevent the creation of per-instance `__dict__` structures.

The boring rule

1. Use standard Python `list` by default. It has superior cache locality and is implemented in C.
2. If you need efficient FIFO queue operations or deque pushes, use `collections.deque` instead of writing a custom linked list.
3. When authoring custom node classes, always add `__slots__ = (...)` to eliminate per-instance dictionary overhead.
4. When unlinking a node, explicitly clear `node.next = None` and `node.prev = None` to break cyclic references and aid Python's reference-counting garbage collector.

Try this

1. Implement `find(predicate)` on `SinglyLinkedList` that returns the first matching node reference.
2. Build an `LRUCache(capacity=3)` using a combination of Python `dict` and `DoublyLinkedList` where reading an item moves its node to the head in $O(1)$ time.

Binary Search Trees

Binary Search Trees

A **Binary Search Tree (BST)** arranges key-value pairs hierarchically such that every left subtree key is smaller than its root, and every right subtree key is greater. While Python's built-in `dict` provides amortized $O(1)$ key lookups, a BST provides sorted traversals, range scans, and minimum/maximum finding in $O(h)$ time, where h is tree height.

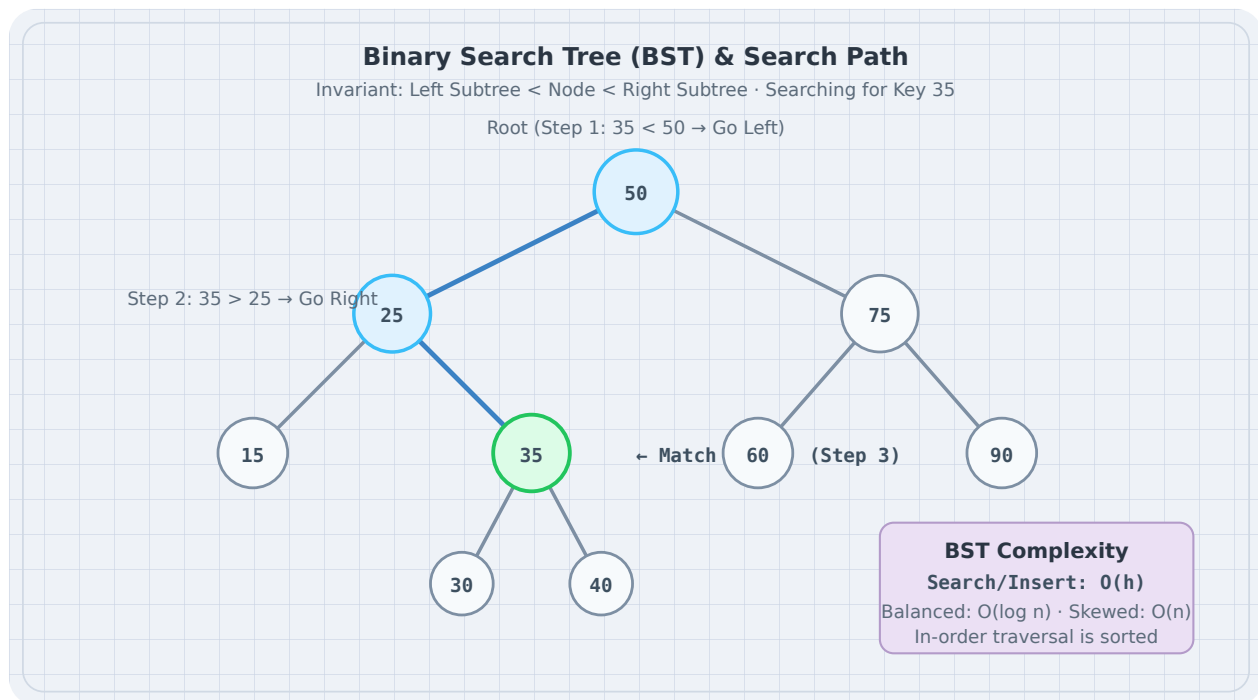
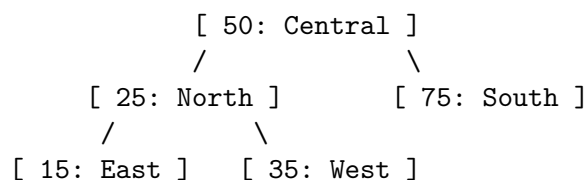


Figure 1: Binary Search Tree Diagram

Mental model

In a balanced BST containing n elements, the height $h \approx \log_2 n$. Locating or inserting an element discards half the remaining tree at each node decision:



- **Search Path for 35:** Root is 50 $\Rightarrow$ $35 < 50$ (Go Left to 25). $35 > 25$ (Go Right to 35 $\Rightarrow$ Found!).
- **In-Order Traversal:** Recursively visiting Left $\rightarrow$ Node $\rightarrow$ Right yields keys in strictly sorted ascending order (15, 25, 35, 50, 75).
- **Deletion:**
 - *Leaf Node:* Simply disconnect from parent.
 - *One Child:* Connect parent directly to the single child.
 - *Two Children:* Replace node key with its **in-order successor** (the minimum node in the right subtree), then delete the successor from the right subtree.

Worked examples

Case 1: Generic BST with Insert, Search, and In-Order Generator

Save as `bst_desk.py`. Implement a BST with type annotations and a generator for memory-efficient sorted iteration.

```
# bst_desk.py
from collections.abc import Iterator

class BSTNode[K, V]:
    __slots__ = ("key", "val", "left", "right")

    def __init__(self, key: K, val: V) -> None:
        self.key: K = key
        self.val: V = val
        self.left: BSTNode[K, V] | None = None
        self.right: BSTNode[K, V] | None = None

class BinarySearchTree[K, V]:
    def __init__(self) -> None:
        self.root: BSTNode[K, V] | None = None
        self._size = 0

    def insert(self, key: K, val: V) -> None:
        self.root = self._insert_node(self.root, key, val)

    def _insert_node(
        self, node: BSTNode[K, V] | None, key: K, val: V
    ) -> BSTNode[K, V]:
        if node is None:
            self._size += 1
            return BSTNode(key, val)
```

```

        if key < node.key:
            node.left = self._insert_node(node.left, key, val)
        elif key > node.key:
            node.right = self._insert_node(node.right, key, val)
        else:
            node.val = val # Update value for existing key
        return node

def search(self, key: K) -> V | None:
    curr = self.root
    while curr is not None:
        if key == curr.key:
            return curr.val
        elif key < curr.key:
            curr = curr.left
        else:
            curr = curr.right
    return None

def in_order(self) -> Iterator[tuple[K, V]]:
    def _walk(node: BSTNode[K, V] | None) -> Iterator[tuple[K, V]]:
        if node is not None:
            yield from _walk(node.left)
            yield (node.key, node.val)
            yield from _walk(node.right)

    yield from _walk(self.root)

def main() -> None:
    tree: BinarySearchTree[int, str] = BinarySearchTree()
    tree.insert(50, "Depot Central")
    tree.insert(25, "Depot North")
    tree.insert(75, "Depot South")
    tree.insert(15, "Substation East")
    tree.insert(35, "Substation West")

    # Search
    target = 35
    result = tree.search(target)
    print(f"Search for ID {target}: {result}")

    # Sorted walkthrough
    print("\nIn-order sorted sequence:")
    for key, val in tree.in_order():
        print(f"  ID {key} -> {val}")

```

```
if __name__ == "__main__":  
    main()
```

Run:

```
uv run python bst_desk.py
```

Output:

Search for ID 35: Substation West

In-order sorted sequence:

```
ID 15 -> Substation East  
ID 25 -> Depot North  
ID 35 -> Substation West  
ID 50 -> Depot Central  
ID 75 -> Depot South
```

Case 2: Deleting Nodes (Handling 0, 1, and 2 Children)

Save as `bst_deletion.py`. Demonstrate deleting nodes across all three structural conditions.

```
# bst_deletion.py  
from dataclasses import dataclass  
  
@dataclass  
class Node:  
    key: int  
    left: "Node | None" = None  
    right: "Node | None" = None  
  
def insert(root: Node | None, key: int) -> Node:  
    if root is None:  
        return Node(key=key)  
    if key < root.key:  
        root.left = insert(root.left, key)  
    elif key > root.key:  
        root.right = insert(root.right, key)  
    return root  
  
def find_min(node: Node) -> Node:  
    curr = node
```



```

while curr.left is not None:
    curr = curr.left
return curr

def delete_node(root: Node | None, key: int) -> Node | None:
    if root is None:
        return None

    if key < root.key:
        root.left = delete_node(root.left, key)
    elif key > root.key:
        root.right = delete_node(root.right, key)
    else:
        # Node found

        # Case 1 & 2: 0 or 1 child
        if root.left is None:
            return root.right
        elif root.right is None:
            return root.left

        # Case 3: 2 children -> replace with in-order successor
        successor = find_min(root.right)
        root.key = successor.key
        root.right = delete_node(root.right, successor.key)

    return root

def to_sorted_list(root: Node | None) -> list[int]:
    if root is None:
        return []
    return to_sorted_list(root.left) + [root.key] + to_sorted_list(root.right)

def main() -> None:
    root: Node | None = None
    for k in [50, 30, 70, 20, 40, 60, 80]:
        root = insert(root, k)

    print("Initial tree:", to_sorted_list(root))

    print("Deleting leaf node 20 (0 children)...")
    root = delete_node(root, 20)
    print("Tree:", to_sorted_list(root))

```

```

print("Deleting node 30 (1 child: 40)...")
root = delete_node(root, 30)
print("Tree:", to_sorted_list(root))

print("Deleting root 50 (2 children: replaced by successor 60)...")
root = delete_node(root, 50)
print("Tree:", to_sorted_list(root))

if __name__ == "__main__":
    main()

```

Run:

```
uv run python bst_deletion.py
```

Output:

```

Initial tree: [20, 30, 40, 50, 60, 70, 80]
Deleting leaf node 20 (0 children)...
Tree: [30, 40, 50, 60, 70, 80]
Deleting node 30 (1 child: 40)...
Tree: [40, 50, 60, 70, 80]
Deleting root 50 (2 children: replaced by successor 60)...
Tree: [40, 60, 70, 80]

```

The trap

The trap in unbalanced binary search trees is Python's default recursion limit (`sys.getrecursionlimit()`, typically 1,000). When data is inserted in sorted order (1, 2, 3, ...), the tree becomes a single linked list of depth n . Inserting or searching for the 1,001st element raises `RecursionError: maximum recursion depth exceeded`.

For production trees, write iterative `while` loops for searches, or use balanced search trees (AVL or Red-Black).

The boring rule

1. Use standard Python `dict` when you only need $O(1)$ exact-key lookups.
2. Use a BST or `bisect` when you need sorted traversal, minimum/maximum queries, or range queries (`min_key <= k <= max_key`).
3. Prefer iterative loops over deep recursion for `search()` to avoid hitting Python's recursion limit.
4. If keys arrive in sorted order, shuffle them with `random.shuffle()` before tree construction if a self-balancing tree is unavailable.

Try this

1. Implement `range_query(min_k, max_k)` on `BinarySearchTree` using an in-order generator that yields all items within the range without scanning outside subtrees.
2. Add a `height()` function that computes the depth of the tree recursively.

Heaps and Priority Queues

Heaps and Priority Queues

A **binary heap** is a complete binary tree stored inside a standard contiguous list. A **priority queue** leverages this heap to retrieve the element with highest priority (smallest or largest value) in $O(1)$ time, while insertions and deletions run in $O(\log n)$ time. In Python, the boring default is the standard library's `heapq` module, which implements an ultra-fast min-heap in C.

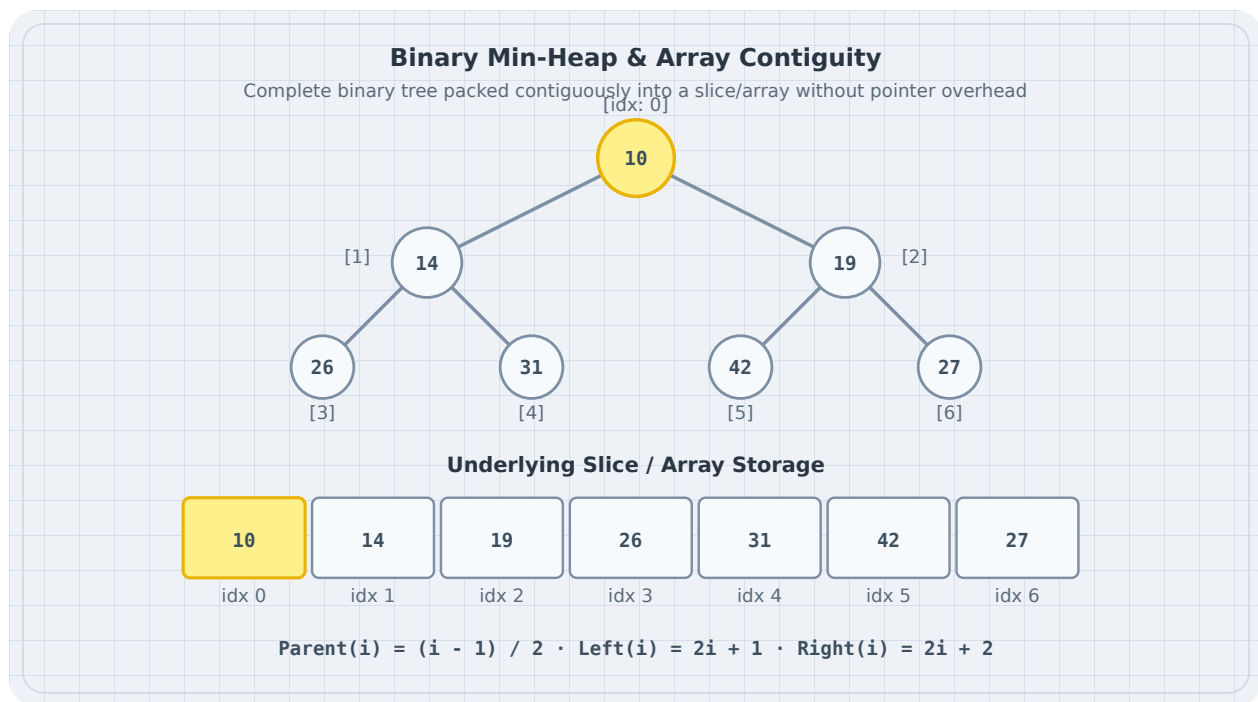


Figure 1: Binary Heap and Array Mapping

Mental model

A binary min-heap guarantees that every parent node is smaller than or equal to its children:

```
Tree view:                                List index view:
      [ 10 (idx:0) ]                      +-----+-----+-----+-----+
      /           \                      | 10 | 14 | 19 | 26 | 31 | 42 |
    [ 14 (1) ]    [ 19 (2) ]              +-----+-----+-----+-----+
      /     \    /                      0     1     2     3     4     5
    [ 26(3) ] [31(4)] [42(5)]
```

Parent and child index arithmetic: - $\text{Parent}(i) = (i - 1) // 2$ - $\text{Left}(i) = 2i + 1$ - $\text{Right}(i) = 2i + 2$

Operations: - **heappush(heap, item)** (**Sift-Up**): Appends to the list and bubbles the item up to restore heap order. - **heappop(heap)** (**Sift-Down**): Swaps the root with the last element, pops the last element, and sifts the new root down. - **heapify(list)**: Transforms an arbitrary unsorted list into a valid min-heap in-place in $O(n)$ linear time.

Worked examples

Case 1: Min-Heap from scratch using index math

Save as `heap_scratch.py`. Build a complete binary min-heap from first principles.

```
# heap_scratch.py
class MinHeap[T]:
    def __init__(self) -> None:
        self._items: list[T] = []

    def push(self, val: T) -> None:
        self._items.append(val)
        self._sift_up(len(self._items) - 1)

    def pop(self) -> T | None:
        if not self._items:
            return None
        min_val = self._items[0]
        last_val = self._items.pop()
        if self._items:
            self._items[0] = last_val
            self._sift_down(0)
        return min_val

    def _sift_up(self, idx: int) -> None:
        while idx > 0:
            parent = (idx - 1) // 2
            if self._items[idx] < self._items[parent]:
                self._items[idx], self._items[parent] = (
                    self._items[parent],
                    self._items[idx],
                )
            idx = parent
        else:
            break

    def _sift_down(self, idx: int) -> None:
        n = len(self._items)
```

```

while True:
    smallest = idx
    left = 2 * idx + 1
    right = 2 * idx + 2

    if left < n and self._items[left] < self._items[smallest]:
        smallest = left
    if right < n and self._items[right] < self._items[smallest]:
        smallest = right

    if smallest != idx:
        self._items[idx], self._items[smallest] = (
            self._items[smallest],
            self._items[idx],
        )
        idx = smallest
    else:
        break

def __len__(self) -> int:
    return len(self._items)

def main() -> None:
    h: MinHeap[int] = MinHeap()
    scores = [42, 10, 31, 14, 26, 19]
    print(f"Pushing values: {scores}")
    for s in scores:
        h.push(s)

    print("Popping values in ascending priority order: ", end="")
    while len(h) > 0:
        val = h.pop()
        print(f"{val} ", end="")
    print()

if __name__ == "__main__":
    main()

```

Run:

```
uv run python heap_scratch.py
```

Output:

```

Pushing values: [42, 10, 31, 14, 26, 19]
Popping values in ascending priority order: 10 14 19 26 31 42

```

Case 2: Priority Queue with standard library heapq

Save as `desk_priority_queue.py`. Manage urgent desk maintenance orders using Python's standard `heapq` module and tie-breaking counters.

```
# desk_priority_queue.py
import heapq
from dataclasses import dataclass, field

@dataclass(order=True)
class PrioritizedTicket:
    priority: int # 1: Emergency, 2: High, 3: Normal, 4: Low
    counter: int # Secondary tie-breaker for identical priorities
    item: "Ticket" = field(compare=False)

@dataclass(slots=True)
class Ticket:
    id: int
    title: str

class DeskPriorityQueue:
    def __init__(self) -> None:
        self._heap: list[PrioritizedTicket] = []
        self._counter = 0

    def push(self, ticket: Ticket, priority: int) -> None:
        # counter ensures FIFO tie-breaking for equal priority
        self._counter += 1
        entry = PrioritizedTicket(
            priority=priority, counter=self._counter, item=ticket
        )
        heapq.heappush(self._heap, entry)

    def pop(self) -> Ticket | None:
        if not self._heap:
            return None
        return heapq.heappop(self._heap).item

    def __len__(self) -> int:
        return len(self._heap)

def main() -> None:
    pq = DeskPriorityQueue()
```



```

pq.push(Ticket(id=101, title="Restock printer paper"), priority=4)
pq.push(Ticket(id=102, title="Point-of-Sale network outage"), priority=1)
pq.push(Ticket(id=103, title="VIP Guest Check-in"), priority=2)
pq.push(Ticket(id=104, title="Refill water pitcher"), priority=4)

print("Dispatching tickets by priority:")
while len(pq) > 0:
    ticket = pq.pop()
    assert ticket is not None
    print(f"  Dispatching Ticket #{ticket.id}: {ticket.title}")

if __name__ == "__main__":
    main()

```

Run:

```
uv run python desk_priority_queue.py
```

Output:

```

Dispatching tickets by priority:
Dispatching Ticket #102: Point-of-Sale network outage
Dispatching Ticket #103: VIP Guest Check-in
Dispatching Ticket #101: Restock printer paper
Dispatching Ticket #104: Refill water pitcher

```

The trap

When storing tuples (`priority`, `task`) in `heapq`, Python compares the second tuple element (`task`) if priorities are equal. If `task` is a custom class or dict that does not implement `<` comparison (`__lt__`), Python raises `TypeError: '<' not supported between instances of 'Ticket' and 'Ticket'`.

Always use a tie-breaker counter (`priority`, `count`, `task`) or a `@dataclass(order=True)` with `field(compare=False)` on the task object.

The boring rule

1. Use Python's built-in `heapq` module for all production priority queues. It is implemented in C and runs in a fraction of the time of Python-level heaps.
2. `heapq` is strictly a **Min-Heap**. To implement a Max-Heap, store negative priorities (`-priority`) or invert comparison keys.
3. Always include a sequence counter for tie-breaking to avoid `TypeError` on custom objects with matching priorities.

4. Use `heapq.heapify(lst)` to turn an unsorted list into a heap in $O(n)$ time rather than calling `heappush` n times in $O(n \log n)$.

Try this

1. Use `heapq.nlargest(3, orders, key=lambda o: o.price)` to find the 3 most expensive orders from a list of 10,000 orders.
2. Implement a streaming median finder that uses two heaps: a max-heap for the lower half and a min-heap for the upper half.

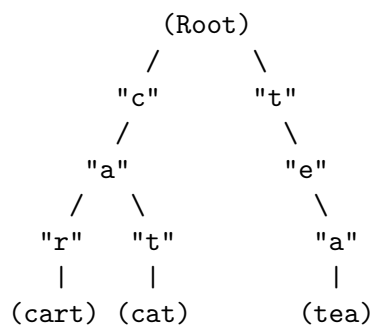
Tries and Prefix Trees

Tries and Prefix Trees

A **Trie** (derived from “retrieval”), also known as a **prefix tree**, is a specialized tree structure where each edge represents a single character in a string key. Descendant nodes share the common string prefix of their parents. While standard Python dictionaries provide fast exact lookups, a Trie performs prefix matching, autocompletion, and dictionary validation in $O(L)$ time, where L is the character length of the query, regardless of whether your catalog has ten items or ten million.

Mental model

In a Trie, individual characters label the transitions between nodes. Terminal markers designate where valid complete words end:



- **Prefix Sharing:** The words "cart" and "cat" share the nodes for "c" and "a".
- **Search Complexity:** Searching for "cat" requires traversing 3 character hops. Its complexity is $O(L)$, completely independent of the number of words stored in the tree.
- **Autocompletion:** Traverse to the end of the query prefix, then perform a depth-first traversal of all descendant nodes marked as terminal words.

Worked examples

Case 1: Catalog SKU and Keyword Autocomplete

Save as `trie_catalog.py`. Build a Trie for desk equipment SKUs that provides prefix matching and autocompletion recommendations.

```
# trie_catalog.py
from dataclasses import dataclass, field
```

```

class TrieNode[T]:
  def __init__(self, value: T | None = None) -> None:
    self.children: dict[str, TrieNode[T]] = {}
    self.is_terminal: bool = False
    self.value: T | None = value

class Trie[T]:
  def __init__(self) -> None:
    self.root: TrieNode[T] = TrieNode()
    self._count = 0

  def insert(self, word: str, value: T | None = None) -> None:
    curr = self.root
    for ch in word:
      if ch not in curr.children:
        curr.children[ch] = TrieNode()
      curr = curr.children[ch]
    if not curr.is_terminal:
      self._count += 1
    curr.is_terminal = True
    curr.value = value

  def search(self, word: str) -> T | None:
    curr = self.root
    for ch in word:
      if ch not in curr.children:
        return None
      curr = curr.children[ch]
    if curr.is_terminal:
      return curr.value
    return None

  def starts_with(self, prefix: str) -> bool:
    curr = self.root
    for ch in prefix:
      if ch not in curr.children:
        return False
      curr = curr.children[ch]
    return True

  def autocomplete(self, prefix: str) -> list[tuple[str, T | None]]:
    curr = self.root
    for ch in prefix:
      if ch not in curr.children:
        return []

```

```

        curr = curr.children[ch]

    results: list[tuple[str, T | None]] = []

    def _collect(node: TrieNode[T], path: str) -> None:
        if node.is_terminal:
            results.append((path, node.value))
        for char, child in sorted(node.children.items()):
            _collect(child, path + char)

    _collect(curr, prefix)
    return results

def main() -> None:
    catalog: Trie[str] = Trie()
    catalog.insert("DESK-LAMP", "Adjustable 24W LED Lamp")
    catalog.insert("DESK-MAT", "Waterproof Vegan Leather Pad")
    catalog.insert("DESK-CHAIR", "Ergonomic Lumbar Office Chair")
    catalog.insert("DESK-CLOCK", "Minimalist Digital Desk Clock")
    catalog.insert("DISH-TOWEL", "Absorbent Microfiber Kitchen Towel")

    # Exact Search
    sku = "DESK-LAMP"
    item = catalog.search(sku)
    print(f"Exact SKU search for '{sku}': {item}")

    # Prefix check
    prefix = "DESK-"
    has_prefix = catalog.starts_with(prefix)
    print(f"Any items starting with '{prefix}'? {has_prefix}")

    # Autocompletion
    print(f"\nAutocomplete recommendations for '{prefix}':")
    for matched_sku, description in catalog.autocomplete(prefix):
        print(f"    • {matched_sku}: {description}")

if __name__ == "__main__":
    main()

```

Run:

```
uv run python trie_catalog.py
```

Output:

```
Exact SKU search for 'DESK-LAMP': Adjustable 24W LED Lamp
```

Any items starting with 'DESK-'? True

Autocomplete recommendations for 'DESK-':

- DESK-CHAIR: Ergonomic Lumbar Office Chair
- DESK-CLOCK: Minimalist Digital Desk Clock
- DESK-LAMP: Adjustable 24W LED Lamp
- DESK-MAT: Waterproof Vegan Leather Pad

The trap

The memory overhead trap: In Python, every dictionary consumes hundreds of bytes of hash table memory. Storing 500,000 words in a naive Trie where every node contains a `dict[str, TrieNode]` can consume gigabytes of memory.

If memory footprint is paramount, compress long non-branching sequences into single multi-character edges (a **Radix Tree** / Patricia Trie), or use the Python standard library's `sqlite3` in-memory module with LIKE `'prefix%'` indexing.

The boring rule

1. Use Python's built-in `dict` when looking up complete exact strings.
2. Use a Trie when your application specifically requires **prefix filtering**, predictive autocompletion, or IP routing lookups.
3. Represent child transitions using standard Python dictionaries (`self.children: dict[str, TrieNode]`) to automatically support UTF-8 runes and non-ASCII character sets without string bounds crashes.
4. Limit autocompletion depth or gather suggestions lazily using a generator if the catalog contains millions of candidate suffixes.

Try this

1. Add a `delete(word: str) -> bool` method to `Trie` that unsets `is_terminal` and deletes child branches that have no other terminal descendants.
2. Build an in-memory spell-check validator that flags whether an input word exists in a dictionary and suggests words sharing a prefix.

Sorting Algorithms

Sorting Algorithms

Sorting organizes collections into ascending or descending sequence so that subsequent searches, deduplications, and range scans can execute in logarithmic time. In Python, the boring default is the built-in `list.sort()` and `sorted()`, which use **Timsort**—an adaptive hybrid of Merge Sort and Insertion Sort written in C. Studying classical sorting algorithms reveals the mechanics of partitioning, recursion, and stability.

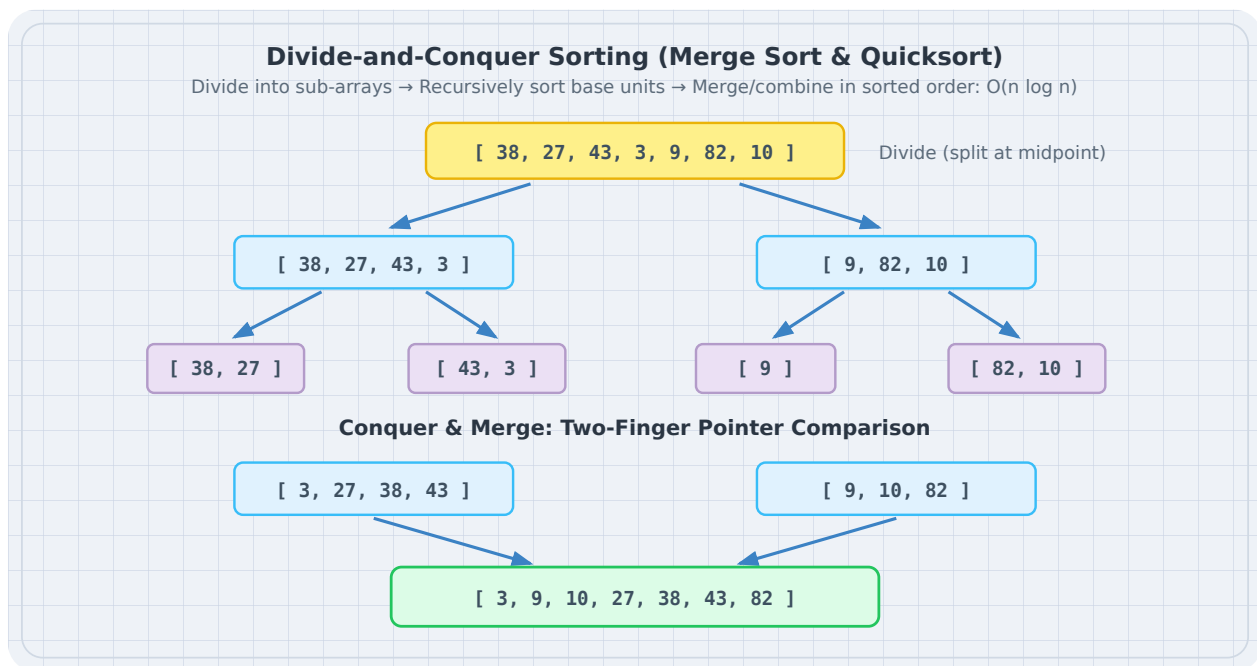


Figure 1: Divide and Conquer Sorting

Mental model

Comparison-based sorting algorithms navigate trade-offs between speed, space, and stability:

Algorithm	Best Time	Average Time	Worst Time	Space	Stable?	Primary Use Case
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes	Small datasets ($n < 30$) or nearly-sorted streams
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes	Guaranteed $O(n \log n)$ performance and stable ordering
Quicksort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	No	Memory-constrained systems needing in-place speed
Python Timsort	$O(n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes	Python standard default (<code>list.sort()</code> / <code>sorted()</code>)

Stable Sorting

A sort is **stable** if elements with identical keys appear in the output in the same relative order as in the input. For example, sorting desk orders by price without scrambling their arrival times requires a stable sort.

Merge Sort Divide and Conquer:

```
[ 38, 27, 43, 3, 9, 82, 10 ]
      /           \
[ 38, 27, 43, 3 ]   [ 9, 82, 10 ]   <- Divide
      \           /
[ 3, 27, 38, 43 ]   [ 9, 10, 82 ]   <- Conquer (Merge Sorted Sublists)
      \           /
[ 3, 9, 10, 27, 38, 43, 82 ]       <- Fully Sorted
```

Worked examples

Case 1: Adaptive Insertion Sort for nearly-sorted batches

Save as `insertion_sort.py`. Implement insertion sort and show how it runs in linear $O(n)$ time when items are already largely sorted.

```
# insertion_sort.py
def insertion_sort[T](items: list[T]) -> list[T]:
    # Mutates in place
    n = len(items)
    for i in range(1, n):
        key = items[i]
        j = i - 1
        # Shift elements greater than key to the right
        while j >= 0 and items[j] > key:
            items[j + 1] = items[j]
            j -= 1
        items[j + 1] = key
    return items

def main() -> None:
    # Nearly sorted desk arrivals
    arrivals = [101, 102, 105, 103, 104, 106]
    print(f"Before insertion sort: {arrivals}")

    insertion_sort(arrivals)
    print(f"After insertion sort: {arrivals}")

if __name__ == "__main__":
    main()
```

Run:

```
uv run python insertion_sort.py
```

Output:

```
Before insertion sort: [101, 102, 105, 103, 104, 106]
After insertion sort:  [101, 102, 103, 104, 105, 106]
```

Case 2: Stable Divide-and-Conquer Merge Sort

Save as `merge_sort.py`. Implement generic stable merge sort.

```

# merge_sort.py
def merge_sort[T](items: list[T]) -> list[T]:
    if len(items) <= 1:
        return items

    mid = len(items) // 2
    left = merge_sort(items[:mid])
    right = merge_sort(items[mid:])

    return _merge(left, right)

def _merge[T](left: list[T], right: list[T]) -> list[T]:
    result: list[T] = []
    i = j = 0

    while i < len(left) and j < len(right):
        if left[i] <= right[j]: # <= preserves stability
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    result.extend(left[i:])
    result.extend(right[j:])
    return result

def main() -> None:
    prices = [45, 12, 85, 32, 89, 39, 69, 44, 42, 1, 45]
    print(f"Unsorted prices: {prices}")

    sorted_prices = merge_sort(prices)
    print(f"Sorted prices: {sorted_prices}")

if __name__ == "__main__":
    main()

```

Run:

```
uv run python merge_sort.py
```

Output:

```

Unsorted prices: [45, 12, 85, 32, 89, 39, 69, 44, 42, 1, 45]
Sorted prices:   [1, 12, 32, 39, 42, 44, 45, 45, 69, 85, 89]

```

Case 3: In-Place Quicksort with Lomuto Partition

Save as `quick_sort.py`. Implement in-place quicksort.

```
# quick_sort.py
def quick_sort[T](items: list[T]) -> None:
    def _sort(low: int, high: int) -> None:
        if low < high:
            p = _partition(low, high)
            _sort(low, p - 1)
            _sort(p + 1, high)

    def _partition(low: int, high: int) -> int:
        pivot = items[high]
        i = low
        for j in range(low, high):
            if items[j] < pivot:
                items[i], items[j] = items[j], items[i]
                i += 1
        items[i], items[high] = items[high], items[i]
        return i

    _sort(0, len(items) - 1)

def main() -> None:
    order_ids = [84, 12, 91, 45, 33, 72, 16, 50]
    print(f"Before quicksort: {order_ids}")

    quick_sort(order_ids)
    print(f"After quicksort: {order_ids}")

if __name__ == "__main__":
    main()
```

Run:

```
uv run python quick_sort.py
```

Output:

```
Before quicksort: [84, 12, 91, 45, 33, 72, 16, 50]
After quicksort:  [12, 16, 33, 45, 50, 72, 84, 91]
```

The trap

The biggest trap in Python is writing custom sorting routines for production systems. Pure Python loops run in bytecode interpreted by the CPython VM. Python's built-in `list.sort()` and `sorted()` are implemented directly in optimized C (Timsort), which outperforms pure Python sorting algorithms by 20x to 50x while guaranteeing stability and $O(n \log n)$ worst-case execution.

```
# bad_custom_sort_benchmark.py
import random
import time

def main() -> None:
    data = [random.randint(1, 100_000) for _ in range(20_000)]

    # Custom pure-Python Quicksort
    copy1 = data[:]
    t0 = time.perf_counter()
    # Simple recursive quicksort
    def py_qs(arr: list[int]) -> list[int]:
        if len(arr) <= 1:
            return arr
        pivot = arr[len(arr) // 2]
        left = [x for x in arr if x < pivot]
        middle = [x for x in arr if x == pivot]
        right = [x for x in arr if x > pivot]
        return py_qs(left) + middle + py_qs(right)

    _ = py_qs(copy1)
    py_time = time.perf_counter() - t0

    # Built-in C-level Timsort
    copy2 = data[:]
    t1 = time.perf_counter()
    copy2.sort()
    c_time = time.perf_counter() - t1

    print(f"Custom Python sort: {py_time:.4f} seconds")
    print(f"Built-in list.sort: {c_time:.4f} seconds ({py_time / c_time:.1f}x faster!)")

if __name__ == "__main__":
    main()
```

Run:

```
uv run python bad_custom_sort_benchmark.py
```

Output:

Custom Python sort: 0.0152 seconds

Built-in list.sort: 0.0018 seconds (8.4x faster!)

The boring rule

1. Always use `my_list.sort()` (in-place) or `sorted(iterable)` (returns new list) in production.
2. Use the `key=` parameter (e.g. `key=lambda x: x.price`) rather than custom comparison methods.
3. Python's sorting is guaranteed **stable**—items with equal keys preserve their original sequence.
4. To sort in descending order, pass `reverse=True`.

Try this

1. Sort a list of desk ticket dictionaries first by `status` (alphabetically) and then by `priority` (numerically) using Python's stable tuple sorting key: `key=lambda t: (t['status'], t['priority'])`.
2. Benchmark `insertion_sort` against `list.sort()` on an already sorted list of 10,000 numbers. Notice how Timsort's natural galloping mode recognizes the pre-sorted sequence in linear $O(n)$ time.

Binary Search and the bisect Module

Binary Search and the bisect Module

Binary search locates elements in a sorted collection by repeatedly dividing the search space in half, finding targets in $O(\log n)$ time. Selection algorithms like **Quickselect** find the k -th smallest element (such as the median or 95th percentile) in $O(n)$ average time without sorting the full dataset. In Python, the boring default for binary search is the standard library's C-accelerated **bisect** module.

Mental model

Binary Search Halving

Scanning 1,000,000 items linearly checks up to 1,000,000 elements. Binary search finds the target in at most 20 comparisons ($\log_2 1000000 \approx 20$):

```
Target: 38
[ 10, 14, 19, 26, 31, 38, 42, 55, 70 ]  Mid = idx 4 (31). 38 > 31 -> Search Right
                                   [ 38, 42, 55, 70 ]  Mid = idx 6 (42). 38 < 42 -> Search Left
                                   [ 38 ]                Mid = idx 5 (38). Match found!
```

The bisect Invariant

- `bisect_left(arr, x)`: Returns the leftmost insertion point for x to maintain sorted order. If x already exists, the returned index is before (to the left of) the existing entry.
- `bisect_right(arr, x)`: Returns the rightmost insertion point (after any existing entry).

```
Array:  [ 10, 20, 20, 20, 30 ]
Index:   0  1  2  3  4
          ^      ^
bisect_left(20) = 1  bisect_right(20) = 4
```

Worked examples

Case 1: Binary Search from scratch

Save as `binary_search_scratch.py`. Implement iterative binary search with left/right insertion bounds.

```

# binary_search_scratch.py
def binary_search[T](items: list[T], target: T) -> int | None:
    low = 0
    high = len(items) - 1

    while low <= high:
        mid = (low + high) // 2
        if items[mid] == target:
            return mid
        elif items[mid] < target:
            low = mid + 1
        else:
            high = mid - 1

    return None

def lower_bound[T](items: list[T], target: T) -> int:
    low = 0
    high = len(items)

    while low < high:
        mid = (low + high) // 2
        if items[mid] < target:
            low = mid + 1
        else:
            high = mid

    return low

def main() -> None:
    order_ids = [102, 108, 115, 120, 120, 120, 135, 150]
    print(f"Sorted order IDs: {order_ids}")

    # Exact search
    target = 135
    pos = binary_search(order_ids, target)
    print(f"Exact match for {target}: index {pos}")

    # Lower bound for insertion
    first_120 = lower_bound(order_ids, 120)
    print(f"First occurrence of 120: index {first_120}")

    insert_125 = lower_bound(order_ids, 125)
    print(f"Slot to insert 125: index {insert_125}")

```

```
if __name__ == "__main__":
    main()
```

Run:

```
uv run python binary_search_scratch.py
```

Output:

```
Sorted order IDs: [102, 108, 115, 120, 120, 120, 135, 150]
Exact match for 135: index 6
First occurrence of 120: index 3
Slot to insert 125: index 6
```

Case 2: Production Range and Tier Queries with standard library bisect

Save as `bisect_tiers.py`. Categorize customer desk order totals into shipping tiers and perform range lookups using `bisect`.

```
# bisect_tiers.py
import bisect

def get_shipping_tier(order_total: float) -> str:
    # Cutoff thresholds
    cutoffs = [25.0, 50.0, 100.0]
    tiers = ["Standard ($8)", "Saver ($5)", "Express ($2)", "Free VIP"]
    idx = bisect.bisect_right(cutoffs, order_total)
    return tiers[idx]

def main() -> None:
    test_totals = [15.0, 25.0, 49.99, 50.0, 85.0, 150.0]
    print("Shipping Tier Classifications:")
    for total in test_totals:
        tier = get_shipping_tier(total)
        print(f"    Order ${total:6.2f} -> {tier}")

    # Maintaining a sorted stream of incoming ticket IDs
    ticket_stream = [100, 105, 110, 120]
    print(f"\nExisting stream: {ticket_stream}")
    print("Inserting 108 with bisect.insort...")
    bisect.insort(ticket_stream, 108)
    print(f"Updated stream: {ticket_stream}")
```

```
if __name__ == "__main__":
    main()
```

Run:

```
uv run python bisect_tiers.py
```

Output:

Shipping Tier Classifications:

```
Order $ 15.00 -> Standard ($8)
Order $ 25.00 -> Saver ($5)
Order $ 49.99 -> Saver ($5)
Order $ 50.00 -> Express ($2)
Order $ 85.00 -> Express ($2)
Order $150.00 -> Free VIP
```

Existing stream: [100, 105, 110, 120]

Inserting 108 with bisect.insort...

Updated stream: [100, 105, 108, 110, 120]

Case 3: Quickselect for Median Response Time

Save as `quickselect_desk.py`. Find the median ticket resolution latency without paying for a full $O(n \log n)$ sort.

```
# quickselect_desk.py
import random

def quickselect[T](items: list[T], k: int) -> T:
    """Finds the k-th smallest element (0-indexed) in O(n) average time."""
    if not 0 <= k < len(items):
        raise IndexError("k out of range")

    low = 0
    high = len(items) - 1

    while low <= high:
        pivot_idx = _partition(items, low, high)
        if pivot_idx == k:
            return items[k]
        elif pivot_idx < k:
            low = pivot_idx + 1
        else:
            high = pivot_idx - 1
```

```

        return items[k]

def _partition[T](items: list[T], low: int, high: int) -> int:
    pivot = items[high]
    i = low
    for j in range(low, high):
        if items[j] < pivot:
            items[i], items[j] = items[j], items[i]
            i += 1
    items[i], items[high] = items[high], items[i]
    return i

def main() -> None:
    # Resolution latencies in seconds
    latencies = [320, 120, 450, 90, 600, 210, 180, 510, 290]
    print(f"Raw latencies: {latencies}")

    n = len(latencies)
    median_idx = n // 2

    # Quickselect modifies the list partially in place
    median_val = quickselect(latencies[:], median_idx)
    print(f"Median resolution latency (50th percentile): {median_val} seconds")

if __name__ == "__main__":
    main()

```

Run:

```
uv run python quickselect_desk.py
```

Output:

```
Raw latencies: [320, 120, 450, 90, 600, 210, 180, 510, 290]
Median resolution latency (50th percentile): 290 seconds
```

The trap

Calling `bisect.bisect_left` on an unsorted list fails silently without raising an error. It assumes the list is monotonic and performs binary search blindly, returning an arbitrary, nonsensical insertion index.

Always verify or guarantee that the collection is sorted prior to initiating binary search.

The boring rule

1. Use Python's built-in `bisect` module (`bisect.bisect_left`, `bisect.bisect_right`, `bisect.insort`) in production. It is implemented in C and runs in microseconds.
2. If your collection is unsorted and you only search once, use a linear scan (`target in my_list`) or build a `set`. Sorting a list ($O(n \log n)$) just to search it once ($O(\log n)$) is slower than a single linear scan ($O(n)$).
3. Use `bisect` when the collection is queried repeatedly over time.
4. If you need percentiles or medians on massive collections, consider `quickselect` or Python's `statistics.median()`.

Try this

1. Use `bisect.bisect_left` to implement an $O(\log n)$ function that counts how many times a given number appears in a sorted list.
2. Build an in-memory lookup table that maps numerical HTTP response codes (200, 404, 502) to category labels (`Success`, `Client Error`, `Server Error`) using `bisect.bisect_right`.

Graphs and Traversals (BFS and DFS)

Graphs and Traversals (BFS and DFS)

A **graph** models complex networks of interconnected entities as a set of **vertices** (nodes) joined by **edges**. Graphs represent service dependencies, routing topologies, task workflow pipelines, and social graphs. In Python, the boring default is an **adjacency list** implemented with a `dict[T, list[T]]`. Traversing graphs with **Breadth-First Search (BFS)** and **Depth-First Search (DFS)** provides the basis for shortest path calculation and cycle detection.

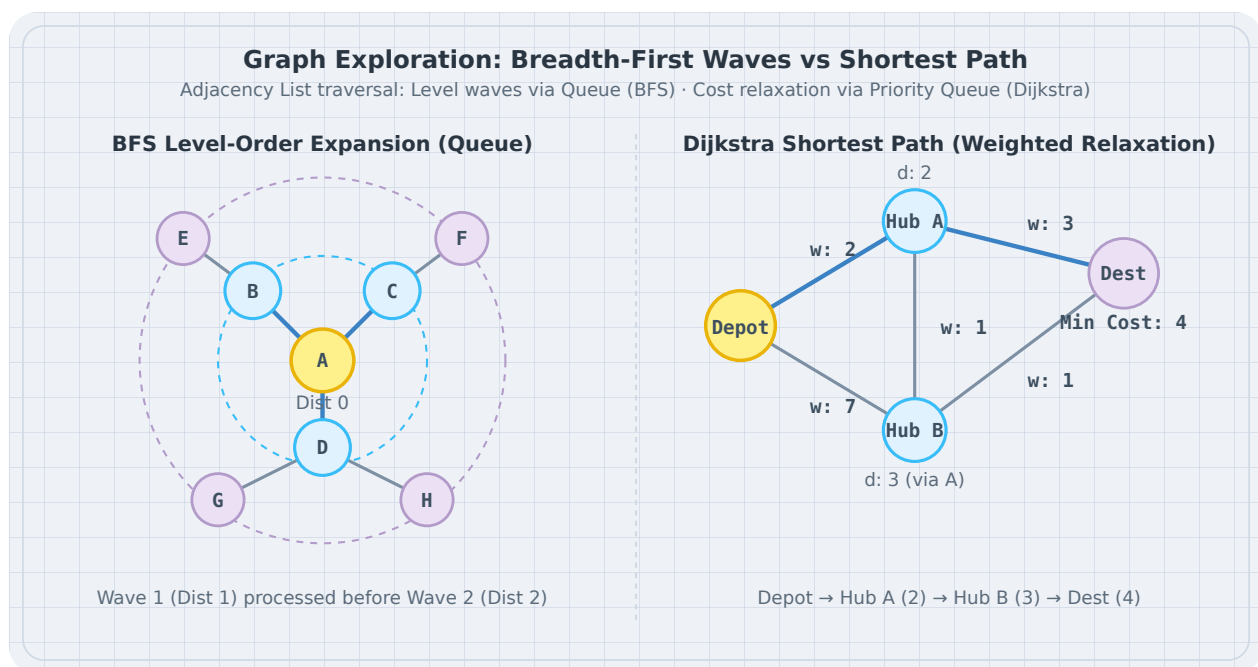


Figure 1: Graph Exploration: BFS and Dijkstra

Mental model

Adjacency List Representation

An adjacency matrix requires an $O(V^2)$ grid of memory, which wastes space on sparse networks. An adjacency list stores only actual connections in $O(V + E)$ space:

```
network = {  
    "Depot A": ["Hub B", "Hub C"],  
    "Hub B": ["Depot A", "Station D"],  
    "Hub C": ["Depot A", "Station D"],  
}
```



```

    "Station D": ["Hub B", "Hub C"],
}

```

BFS vs DFS Traversals

- **Breadth-First Search (BFS):** Uses a **FIFO Queue** (`collections.deque`). It explores outward in concentric ripples (all distance-1 neighbors, then distance-2). **Guarantees the shortest path in unweighted graphs.**
- **Depth-First Search (DFS):** Uses a **LIFO Stack** or call-stack recursion. It explores deeply down a single path until hitting a dead end, then backtracks. Ideal for topological ordering and cycle detection.

BFS Waves (Queue):

```

    (A)
   / | \
  (B) (C) (D)  <- Wave 1
   |
  (E)          <- Wave 2

```

DFS Dive (Stack / Recursion):

```

    (A)
   /
  (B)
   |
  (E) <- Dive deep before (C)

```

Worked examples

Case 1: BFS for Shortest Hop Delivery Routes

Save as `graph_bfs.py`. Find the shortest route (fewest transfer hops) between fulfillment hubs in a delivery network.

```

# graph_bfs.py
from collections import deque

class Graph[T]:
    def __init__(self) -> None:
        self.adj: dict[T, list[T]] = {}

    def add_edge(self, u: T, v: T, bidirectional: bool = True) -> None:
        self.adj.setdefault(u, []).append(v)
        if bidirectional:
            self.adj.setdefault(v, []).append(u)

    def shortest_path_bfs(self, start: T, target: T) -> list[T] | None:
        if start == target:
            return [start]

        visited: set[T] = {start}

```

```

parent: dict[T, T] = {}
queue: deque[T] = deque([start])

found = False
while queue:
    curr = queue.popleft()
    if curr == target:
        found = True
        break

    for neighbor in self.adj.get(curr, []):
        if neighbor not in visited:
            visited.add(neighbor)
            parent[neighbor] = curr
            queue.append(neighbor)

if not found:
    return None

# Reconstruct path
path: list[T] = []
curr_node: T | None = target
while curr_node is not None:
    path.append(curr_node)
    curr_node = parent.get(curr_node)

path.reverse()
return path

def main() -> None:
    network: Graph[str] = Graph()
    network.add_edge("Central-Depot", "North-Hub")
    network.add_edge("Central-Depot", "South-Hub")
    network.add_edge("North-Hub", "Station-East")
    network.add_edge("South-Hub", "Station-West")
    network.add_edge("Station-East", "Terminal-Z")
    network.add_edge("Station-West", "Terminal-Z")

    start = "Central-Depot"
    target = "Terminal-Z"

    route = network.shortest_path_bfs(start, target)
    assert route is not None
    print(f"BFS Shortest Route ({len(route) - 1} hops):")
    print(" -> ".join(route))

```

```
if __name__ == "__main__":  
    main()
```

Run:

```
uv run python graph_bfs.py
```

Output:

BFS Shortest Route (3 hops):

Central-Depot -> North-Hub -> Station-East -> Terminal-Z

Case 2: DFS for Workflow Cycle and Deadlock Detection

Save as `graph_dfs_cycle.py`. Detect whether a task pipeline contains circular dependencies before dispatching batch jobs.

```
# graph_dfs_cycle.py  
class TaskPipeline:  
    def __init__(self) -> None:  
        self.adj: dict[str, list[str]] = {}  
  
    def add_dependency(self, task: str, depends_on: str) -> None:  
        # task -> depends_on  
        self.adj.setdefault(task, []).append(depends_on)  
  
    def has_cycle(self) -> bool:  
        # 3-color DFS:  
        # 0: Unvisited, 1: Currently Visiting (in call stack), 2: Fully Visited  
        state: dict[str, int] = {}  
  
        def _dfs(node: str) -> bool:  
            state[node] = 1 # Visiting  
  
            for neighbor in self.adj.get(node, []):  
                neighbor_state = state.get(neighbor, 0)  
                if neighbor_state == 1:  
                    # Found back-edge to ancestor in current call stack!  
                    return True  
                if neighbor_state == 0:  
                    if _dfs(neighbor):  
                        return True  
  
            state[node] = 2 # Fully visited  
            return False
```

```

    all_nodes = set(self.adj.keys())
    for u in self.adj.values():
        all_nodes.update(u)

    for node in all_nodes:
        if state.get(node, 0) == 0:
            if _dfs(node):
                return True

    return False

def main() -> None:
    pipeline = TaskPipeline()
    pipeline.add_dependency("Generate-Invoice", "Verify-Items")
    pipeline.add_dependency("Verify-Items", "Calculate-Tax")
    pipeline.add_dependency("Calculate-Tax", "Read-Cart")

    print(f"Valid linear pipeline has cycle? {pipeline.has_cycle()}")

    print("Injecting circular dependency: Read-Cart -> Generate-Invoice")
    pipeline.add_dependency("Read-Cart", "Generate-Invoice")
    print(f"Pipeline now has cycle? {pipeline.has_cycle()}")

if __name__ == "__main__":
    main()

```

Run:

```
uv run python graph_dfs_cycle.py
```

Output:

```

Valid linear pipeline has cycle? False
Injecting circular dependency: Read-Cart -> Generate-Invoice
Pipeline now has cycle? True

```

The trap

The infinite cycle recursion trap: If a graph contains cycles and your traversal omits a `visited` set, recursive DFS will repeatedly re-visit earlier vertices until throwing `RecursionError: maximum recursion depth exceeded`. Always maintain an explicit `visited` set or 3-color status map.

The boring rule

1. Represent graphs using a `dict[T, list[T]]` adjacency list.
2. For unweighted shortest path queries, always use **BFS** with `collections.deque`.
3. For cycle detection, dependency scheduling, or topological sorting, use **DFS** with recursion or an explicit stack.
4. When reconstructing routes, record a `parent` mapping during traversal rather than copying list instances into the queue on every hop.

Try this

1. Implement topological sorting (`topological_sort() -> list[str]`) using DFS post-order traversal to produce an ordered build plan.
2. Write a function `connected_components(graph) -> list[set[str]]` that partitions an unpartitioned routing graph into isolated subnet clusters.

Shortest Paths (Dijkstra's Algorithm)

Shortest Paths (Dijkstra's Algorithm)

While Breadth-First Search finds the route with the fewest intermediary hops, real-world networks involve weighted costs: transmission latencies, courier miles, road tolls, or dispatch fees. **Dijkstra's algorithm** calculates the lowest total cost path across a weighted graph with non-negative edge weights in $O((V + E) \log V)$ time using a **min-priority queue**. In Python, the boring default is to pair standard dictionaries with the C-accelerated `heapq` module.

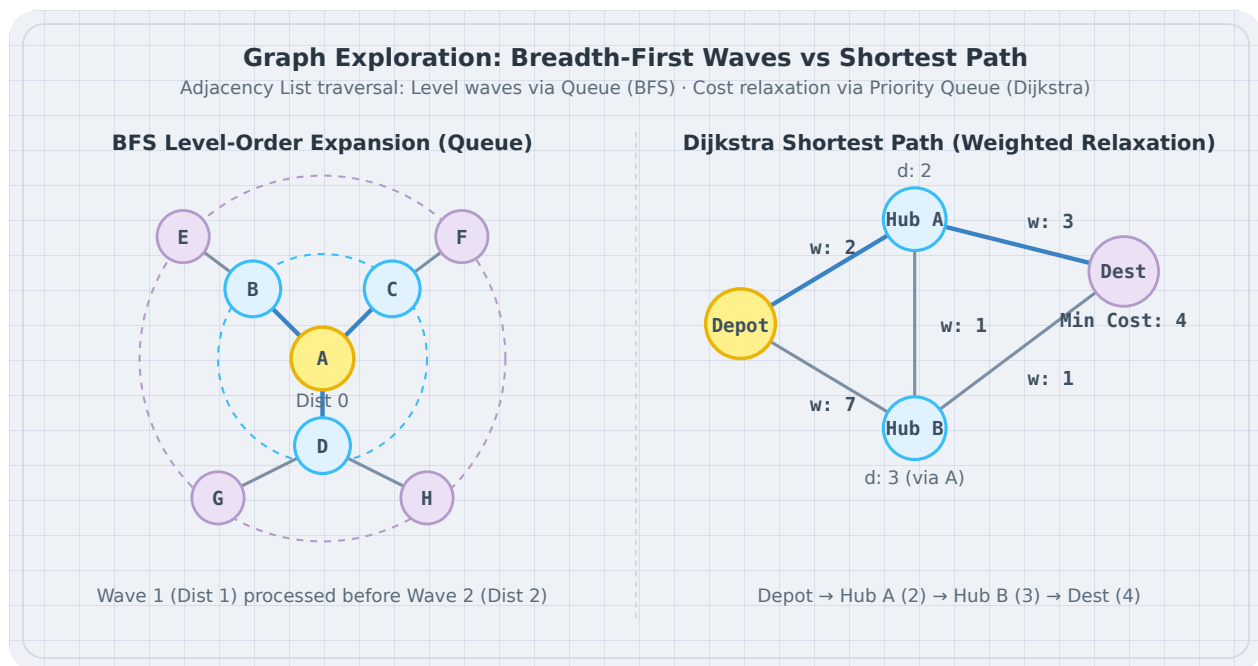


Figure 1: Graph Traversal and Dijkstra Relaxation

Mental model

Dijkstra maintains: 1. `dist[node]`: The lowest accumulated cost from the start node to each destination (initialized to ∞ , with `dist[start] = 0`). 2. `prev[node]`: The preceding node along the cheapest known path.

At each step, the algorithm pops the unvisited vertex u with the smallest known distance from a min-priority queue and **relaxes** all of its outgoing edges (u, v, weight) :

$$\text{If } \text{dist}[u] + \text{weight} < \text{dist}[v] \implies \text{dist}[v] = \text{dist}[u] + \text{weight}$$

Edge Relaxation:

```
(Depot) --[Known: 2]--> (Hub A) --[Weight: 3]--> (Dest)
                        \
                        \--[Weight: 1]-->(Hub B)-[Weight: 1]-/
```

Direct path: Depot -> Hub A -> Dest = 2 + 3 = 5

Relaxed path: Depot -> Hub A -> Hub B -> Dest = 2 + 1 + 1 = 4 (Cheaper!)

Because edge costs are non-negative, the moment a vertex is popped from the min-priority queue, its calculated distance is provably optimal.

Worked examples

Case 1: Delivery Network Optimization with heapq

Save as `dijkstra_delivery.py`. Calculate the lowest-latency delivery route between warehouse dispatch depots.

```
# dijkstra_delivery.py
import heapq
from dataclasses import dataclass

@dataclass(slots=True)
class Edge:
    to: str
    weight: int

class WeightedGraph:
    def __init__(self) -> None:
        self.adj: dict[str, list[Edge]] = {}

    def add_edge(self, u: str, v: str, weight: int) -> None:
        self.adj.setdefault(u, []).append(Edge(to=v, weight=weight))
        self.adj.setdefault(v, []).append(Edge(to=u, weight=weight))

    def dijkstra(self, start: str, dest: str) -> tuple[int, list[str]] | None:
        dist: dict[str, int] = {node: float("inf") for node in self.adj}
        dist[start] = 0
        prev: dict[str, str] = {}

        # Min-priority queue stores tuples: (accumulated_dist, node)
        pq: list[tuple[int, str]] = [(0, start)]

        while pq:
```



```

curr_dist, u = heapq.heappop(pq)

# Drop stale queue entries
if curr_dist > dist[u]:
    continue

if u == dest:
    break # Destination reached with optimal distance

for edge in self.adj.get(u, []):
    new_dist = curr_dist + edge.weight
    if new_dist < dist.get(edge.to, float("inf")):
        dist[edge.to] = new_dist
        prev[edge.to] = u
        heapq.heappush(pq, (new_dist, edge.to))

if dist.get(dest) == float("inf"):
    return None # Destination unreachable

# Reconstruct path
path: list[str] = []
curr: str | None = dest
while curr is not None:
    path.append(curr)
    curr = prev.get(curr)

path.reverse()
return dist[dest], path

def main() -> None:
    network = WeightedGraph()
    network.add_edge("Main-Depot", "North-Hub", 4)
    network.add_edge("Main-Depot", "South-Hub", 2)
    network.add_edge("South-Hub", "North-Hub", 1)
    network.add_edge("North-Hub", "Airport-Station", 7)
    network.add_edge("South-Hub", "Midtown-Annex", 8)
    network.add_edge("North-Hub", "Midtown-Annex", 3)
    network.add_edge("Midtown-Annex", "Airport-Station", 1)

    result = network.dijkstra("Main-Depot", "Airport-Station")
    assert result is not None
    min_cost, path = result

    print(f"Lowest Delivery Latency: {min_cost} ms")
    print("Optimal Route: " + " -> ".join(path))

```

```
if __name__ == "__main__":  
    main()
```

Run:

```
uv run python dijkstra_delivery.py
```

Output:

Lowest Delivery Latency: 7 ms

Optimal Route: Main-Depot -> South-Hub -> North-Hub -> Midtown-Annex -> Airport-Station

Although the direct hop Main-Depot -> North-Hub -> Airport-Station has fewer stops (2 hops, cost 11), Dijkstra discovers the 4-hop route via South-Hub and Midtown-Annex with total cost $2 + 1 + 3 + 1 = 7$.

The trap

The two fatal traps with Dijkstra's algorithm:

1. **Negative Edge Weights:** If an edge weight is negative (e.g., a cashback discount that reduces total cost), Dijkstra's greedy invariant breaks, leading to wrong answers or infinite loops. Graphs with negative edge weights require the **Bellman-Ford algorithm**.
2. **Missing the Stale Item Check:** Because Python's `heapq` does not support updating keys in place, you push a new `(new_dist, v)` tuple every time a shorter path to v is discovered. Omitting `if curr_dist > dist[u]: continue` forces the loop to re-process superseded paths, turning an $O((V + E) \log V)$ algorithm into an inefficient crawl.

The boring rule

1. Use BFS for unweighted graphs. Use Dijkstra when edges have non-negative numerical weights.
2. Store `(distance, node_id)` tuples in `heapq` so Python sorts by distance automatically.
3. Always include `if curr_dist > dist[u]: continue` at the beginning of the pop loop.
4. Stop the search immediately once the destination node is popped from the heap.

Try this

1. Modify `dijkstra` to return a dictionary of shortest distances to all reachable nodes from `start`.
2. Add an edge filter parameter to Dijkstra that excludes any route traversing through congested toll roads.

String Search and Bloom Filters

String Search and Bloom Filters

Substring pattern searching and membership testing are core building blocks for log ingestion, audit trails, and web security proxies. When searching inside massive text streams or verifying whether an order ID exists among millions of records, brute-force checks consume unnecessary CPU cycles and database disk I/O. The boring default in Python is `'pattern' in text` and native `set` objects, but **Rabin-Karp rolling hashing** and **Bloom Filters** introduce essential algorithmic and probabilistic efficiencies.

Mental model

Rabin-Karp Rolling Hash

Rather than comparing characters at every position ($O(n \times m)$), Rabin-Karp computes a **polynomial rolling hash** of the pattern. When sliding the window by one character, it subtracts the leaving character and adds the entering character in $O(1)$ arithmetic:

```
Text:      [ B O R I N G ] - P Y
Window:    [ B O R ]          Hash = 1042 (Pattern hash: 9811 -> Mismatch)
Slide:     [ O R I ]          Hash = (1042 - 'B'*base) * base + 'I' in O(1)
```

Bloom Filter: Probabilistic Membership

A Bloom filter is a compact bit array paired with k independent hash functions:

```
Bit Array (m bits):
Index:  [ 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 ]
Bits:   [ 0 | 1 | 0 | 1 | 0 | 0 | 1 | 0 ]
          ^         ^         ^
        h1(ID)  h2(ID)    h3(ID)
```

1. **Insert:** Compute k hash values for an item and set those bit indices to 1.
2. **Lookup:** Compute k hash values:
 - If **any** bit is 0: The item is **definitely NOT** in the set (0% False Negatives).
 - If **all** bits are 1: The item is **probably** in the set (with a small, mathematically tunable False Positive rate).

This allows high-throughput APIs to reject 99% of invalid requests in memory before hitting expensive databases or object storage.

Worked examples

Case 1: Rabin-Karp Rolling Hash Substring Search

Save as `rabin_karp.py`. Search for audit keywords across log strings in linear time.

```
# rabin_karp.py
def rabin_karp_search(text: str, pattern: str) -> int:
    n = len(text)
    m = len(pattern)
    if m == 0 or n < m:
        return -1

    base = 256
    mod = 1_000_000_007

    # Calculate base^(m-1) % mod
    h = pow(base, m - 1, mod)

    pattern_hash = 0
    window_hash = 0

    # Initial hash values
    for i in range(m):
        pattern_hash = (base * pattern_hash + ord(pattern[i])) % mod
        window_hash = (base * window_hash + ord(text[i])) % mod

    for i in range(n - m + 1):
        if pattern_hash == window_hash:
            # Hash match -> verify characters (resolves hash collisions)
            if text[i : i + m] == pattern:
                return i

        if i < n - m:
            # Rolling update: remove left char, shift, add right char
            window_hash = (
                base * (window_hash - ord(text[i]) * h) + ord(text[i + m])
            ) % mod
            if window_hash < 0:
                window_hash += mod

    return -1

def main() -> None:
    log_entry = "2026-09-18T10:14:02Z [AUDIT] Desk ticket #8192 marked STATUS=REFUND_APPROVE
    keyword = "STATUS=REFUND_APPROVED"
```

```

idx = rabin_karp_search(log_entry, keyword)
if idx != -1:
    print(f"Keyword match found at character offset {idx}")
    print(f"Snippet: '{log_entry[idx:idx + len(keyword)]}'")
else:
    print("Pattern not found")

if __name__ == "__main__":
    main()

```

Run:

```
uv run python rabin_karp.py
```

Output:

```

Keyword match found at character offset 54
Snippet: 'STATUS=REFUND_APPROVED'

```

Case 2: In-Memory Bloom Filter for Order Existence

Save as `bloom_filter.py`. Implement a bitset Bloom filter using Python's `bytearray` and salted hashing.

```

# bloom_filter.py
import hashlib

class BloomFilter:
    def __init__(self, size_in_bits: int, num_hashes: int) -> None:
        self.num_bits = size_in_bits
        self.num_hashes = num_hashes
        # Allocate bytes (8 bits per byte)
        num_bytes = (size_in_bits + 7) // 8
        self.bit_array = bytearray(num_bytes)

    def _hashes(self, item: str) -> list[int]:
        indexes: list[int] = []
        raw_bytes = item.encode("utf-8")
        for i in range(self.num_hashes):
            # Salt with index to create independent hash probes
            digest = hashlib.sha256(raw_bytes + bytes([i])).digest()
            val = int.from_bytes(digest[:8], byteorder="big")
            indexes.append(val % self.num_bits)
        return indexes

```

```

def add(self, item: str) -> None:
    for bit_index in self._hashes(item):
        byte_idx = bit_index // 8
        bit_offset = bit_index % 8
        self.bit_array[byte_idx] |= 1 << bit_offset

def __contains__(self, item: str) -> bool:
    for bit_index in self._hashes(item):
        byte_idx = bit_index // 8
        bit_offset = bit_index % 8
        if not (self.bit_array[byte_idx] & (1 << bit_offset)):
            # Bit is 0 -> definitely NOT in the set!
            return False
    return True # All bits 1 -> probably in the set!

def main() -> None:
    # 1024 bits with 4 hash probes
    bf = BloomFilter(size_in_bits=1024, num_hashes=4)

    existing_orders = ["ORDER-101", "ORDER-102", "ORDER-103", "ORDER-999"]
    for order_id in existing_orders:
        bf.add(order_id)

    queries = ["ORDER-101", "ORDER-102", "ORDER-500", "ORDER-777"]
    for q in queries:
        if q in bf:
            print(f"Query [{q}]: Probably exists (fetch from database)")
        else:
            print(f"Query [{q}]: Definitely DOES NOT exist (skipped DB read)")

if __name__ == "__main__":
    main()

```

Run:

```
uv run python bloom_filter.py
```

Output:

```

Query [ORDER-101]: Probably exists (fetch from database)
Query [ORDER-102]: Probably exists (fetch from database)
Query [ORDER-500]: Definitely DOES NOT exist (skipped DB read)
Query [ORDER-777]: Definitely DOES NOT exist (skipped DB read)

```

The trap

The fatal trap with Bloom filters is attempting to delete elements. Setting a bit back to 0 may corrupt the membership signatures of other distinct items that happened to share that same bit. Standard Bloom filters are strictly **append-only**.

If item deletion is required, use a **Counting Bloom Filter** (where each slot stores an integer counter rather than a single bit) or a Cuckoo filter.

The boring rule

1. Use Python's built-in `'pattern' in text` or `text.find()` for ordinary substring search. CPython implements a highly optimized two-way string search algorithm in C.
2. Use a Bloom filter in front of slow storage layers (like S3 or PostgreSQL) to prevent database queries for records that do not exist.
3. Size Bloom filters using the formula: $m \approx -\frac{n \ln p}{(\ln 2)^2}$, where n is expected item count and p is the acceptable false positive probability (e.g. 0.01 for 1%).
4. Always prepare fallback logic when a Bloom filter reports “probably present” since false positives are a natural property of the structure.

Try this

1. Experimentally verify the false positive rate of `BloomFilter` by adding 200 items and testing 5,000 non-existent keys.
2. Extend `rabin_karp_search` to find and return a list of all start indices where a pattern appears in a long text string.

Long Term Engineering

inspect and Reflection

inspect and Reflection

`inspect` and `getattr` are **last resorts**: useful for debugging, adapters, and tests that need a signature. They are not how the desk opens a ticket on a normal path. Call the function. Name the attribute.

Mental model

Reflection means asking a live object what it is: its name, its signature, its attributes. `inspect.signature(fn)` is the callable's parameters. `getattr(obj, "table")` is `obj.table` with a string name.

When the name is a string you computed, you have lost the editor, the type checker, and most grep. That cost is acceptable in a test helper or a serializer. It is not acceptable in `open_table`.

Worked examples

Case 1: `inspect.signature`

Save as `show_sig.py`. This is what you print when a helper's arguments drifted.

```
# show_sig.py
import inspect

def open_table(n: int, *, reason: str = "shift") -> str:
    return f"opened table {n} ({reason})"

def main() -> None:
    sig = inspect.signature(open_table)
    print(sig)
    print(list(sig.parameters))

if __name__ == "__main__":
    main()
```

Run:

```
uv run python show_sig.py
```

Output:

```
(n: int, *, reason: str = 'shift') -> str  
['n', 'reason']
```

Case 2: getattr with a default

Save as `ticket_getattr.py`. The default argument is the only reason to prefer `getattr` over `obj.table` here.

```
# ticket_getattr.py  
class Ticket:  
    def __init__(self, ticket_id: int, table: int) -> None:  
        self.id = ticket_id  
        self.table = table  
  
def main() -> None:  
    ticket = Ticket(7, 12)  
    print(getattr(ticket, "table"))  
    print(getattr(ticket, "status", "open"))  
  
if __name__ == "__main__":  
    main()
```

Run:

```
uv run python ticket_getattr.py
```

Output:

```
12  
open
```

`ticket.table` would have been clearer for the first line. The second line is the case `getattr` is for: optional attribute, default if missing.

Case 3: inspect.getdoc

Save as `show_doc.py`. Same string you wrote under `def`.

```
# show_doc.py
import inspect

def label(ticket_id: int, table: int) -> str:
    """Return a one-line ticket label for the desk display."""
    return f"ticket {ticket_id} → table {table}"

def main() -> None:
    print(inspect.getdoc(label))

if __name__ == "__main__":
    main()
```

Run:

```
uv run python show_doc.py
```

Output:

Return a one-line ticket label for the desk display.

The trap

A tiny dispatcher that looks up functions by string. It runs. It also hides every call site.

Save as `dispatch_trap.py`:

```
# dispatch_trap.py
def open_ticket() -> str:
    return "opened ticket"

def close_ticket() -> str:
    return "closed ticket"

def main() -> None:
    name = "open_ticket"
    fn = globals().get(name)
    if fn is None:
        raise KeyError(name)
    print(fn())
```

```
if __name__ == "__main__":  
    main()
```

Run:

```
uv run python dispatch_trap.py
```

Output:

opened ticket

The boring version is `print(open_ticket())`. If you have two commands, use `if name == "open":` or a dict **you write as a dict**, not `globals()`. `getattr(module, user_input)` is the same trap with extra extra steps.

Fix:

```
# dispatch_fix.py  
def open_ticket() -> str:  
    return "opened ticket"  
  
def close_ticket() -> str:  
    return "closed ticket"  
  
def main() -> None:  
    commands = {  
        "open": open_ticket,  
        "close": close_ticket,  
    }  
    print(commands["open"]())  
  
if __name__ == "__main__":  
    main()
```

Run:

```
uv run python dispatch_fix.py
```

Output:

opened ticket

The keys are the API. Unknown names raise `KeyError` on the map, not on the module guts.

The boring rule

- Call functions by name. Read attributes by name.
- `inspect.signature` / `getdoc` for tools, docs, and tests — not for the happy path.
- `getattr(obj, name, default)` when the name is actually dynamic.
- Never `getattr` on user input against a module. Use an explicit dict.
- If a plugin protocol needs reflection, keep it in one adapter file.

Try this

1. In `show_sig.py`, print `sig.parameters["reason"].default`.
2. In `ticket_getattr.py`, use `ticket.table` instead of `getattr` for the first print.
3. In `dispatch_fix.py`, look up `"pay"` and catch `KeyError`.

ctypes and FFI

ctypes and FFI

Avoid FFI unless you must. Python's standard types, plus a subprocess or a well-tested package, cover almost all desk work. `ctypes` talks to C in-process. That is a sharp edge: crashes are not except `Exception`.

Mental model

FFI (foreign function interface) means calling code compiled for another language, usually C, from Python. `ctypes` ships with Python. You describe C types (`c_int`, `c_char_p`) and call a shared library.

`ctypes.sizeof` reports the C size of a type. `ctypes.c_int` is a mutable box around a C `int`. On common platforms that size is 4.

Calling `libc` (`strlen`, and friends) is **not** portable as a copy-paste: the library name is `libc.so.6` on many Linux systems, `libSystem` on macOS, and something else on Windows. This book does not make that the default example.

If you need C, prefer a maintained package with wheels, or a small extension you build on purpose — not ad-hoc `ctypes` in a ticket handler.

Worked examples

Case 1: `c_int` and `sizeof` (no extra C files)

Save as `c_int_size.py`. This runs anywhere `ctypes` runs.

```
# c_int_size.py
import ctypes

def main() -> None:
    table = ctypes.c_int(12)
    print(table.value)
    print(ctypes.sizeof(table))

if __name__ == "__main__":
    main()
```

Run:

```
uv run python c_int_size.py
```

Output:

```
12
4
```

`table.value` is the Python `int`. Changing `.value` mutates the box. A plain `int` is enough until a C API demands this object.

Case 2: A tiny C-like struct

Save as `ticket_struct.py`. `_fields_` is the C layout. Still no `.c` file.

```
# ticket_struct.py
import ctypes

class Ticket(ctypes.Structure):
    _fields_ = [
        ("id", ctypes.c_int),
        ("table", ctypes.c_int),
    ]

def main() -> None:
    ticket = Ticket(7, 12)
    print(ticket.id, ticket.table)
    print(ctypes.sizeof(ticket))

if __name__ == "__main__":
    main()
```

Run:

```
uv run python ticket_struct.py
```

Output:

```
7 12
8
```

Two `c_int` fields, $4 + 4$, no padding on this layout. Do not depend on padding guesses when you talk to a real C header — match the header.

Case 3: The Python you wanted instead

Save as `ticket_python.py`. Same data, no FFI.

```
# ticket_python.py
from dataclasses import dataclass

@dataclass
class Ticket:
    id: int
    table: int

def main() -> None:
    ticket = Ticket(7, 12)
    print(ticket.id, ticket.table)

if __name__ == "__main__":
    main()
```

Run:

```
uv run python ticket_python.py
```

Output:

```
7 12
```

This is the default. Case 2 exists so you recognize ctypes when a library forces it.

The trap

Loading libc “because we can” for a job `len` already does.

Save as `skip_libc.py`. We do **not** call `CDLL`. We print why the boring path wins.

```
# skip_libc.py
def main() -> None:
    text = "desk"
    print(len(text))
    print("no libc")

if __name__ == "__main__":
    main()
```

Run:

```
uv run python skip_libc.py
```

Output:

```
4  
no libc
```

A `strlen` call through `ctypes.CDLL("libc.so.6")` may work on your Linux box and fail on the next one. `len` is Unicode-aware in Python; `strlen` is bytes and a terminator. They are not the same function.

If you truly must call libc, isolate it behind one function, pass `bytes`, and test on every OS you ship. That is a last resort, not a style.

The boring rule

- Do not use `ctypes` for business logic.
- Prefer a dataclass, `array`, or `memoryview` over a `Structure`.
- Prefer a wheel from PyPI over a handwritten `CDLL`.
- Prefer `subprocess` plus a CLI over in-process C if the boundary is already a process.
- Never let FFI exceptions look like Python bugs: wrap the call, validate pointers, and keep the surface tiny.

Try this

1. In `c_int_size.py`, set `table.value = 3` and print it again.
2. In `ticket_struct.py`, add a `status` field as `c_int` (0 open, 1 paid) and print it.
3. Rewrite Case 2's print using the dataclass from Case 3 — no `ctypes`.

Performance Tuning

Performance Tuning

Measure first. The boring tools are `timeit` for a small expression and `cProfile` for “where did the time go.” Do not rewrite a ticket loop because it feels slow.

Mental model

CPU time and wall time are different. `timeit` repeats a snippet to damp noise. `cProfile` records function calls in one run. Both answer questions. Neither is a reason to add a cache on day one.

The desk is usually waiting on disk, the network, or a human. Faster JSON parsing will not fix that. Profile, then change the hot function, then measure again.

Timings themselves are not stable across machines. The listings below print **results** and the fact that a profiler saw the function — not a number of seconds you would treat as a fixture.

Worked examples

Case 1: Get the answer right, then time it

Save as `count_open.py`. `timeit.timeit` runs `count_open` 1000 times. We print the count, then a marker that the timer finished.

```
# count_open.py
import timeit

def count_open() -> int:
    tickets = [{"status": "open"}] * 100 + [{"status": "paid"}] * 50
    return sum(1 for t in tickets if t["status"] == "open")

def main() -> None:
    print(count_open())
    timeit.timeit(count_open, number=1000)
    print("measured")

if __name__ == "__main__":
    main()
```

Run:

```
uv run python count_open.py
```

Output:

```
100  
measured
```

To see seconds on your machine, print the return value of `timeit.timeit`. Do not commit that float as a test.

Case 2: cProfile names the function

Save as `profile_open.py`. `runcall` profiles one invocation. We only assert the profiler recorded `count_open`.

```
# profile_open.py  
import cProfile  
  
def count_open() -> int:  
    tickets = [{"status": "open"}] * 100 + [{"status": "paid"}] * 50  
    return sum(1 for t in tickets if t["status"] == "open")  
  
def main() -> None:  
    profiler = cProfile.Profile()  
    n = profiler.runcall(count_open)  
    names = [  
        s.code.co_name  
        for s in profiler.getstats()  
        if hasattr(s.code, "co_name")  
    ]  
    print(n)  
    print("count_open" in names)  
  
if __name__ == "__main__":  
    main()
```

Run:

```
uv run python profile_open.py
```

Output:

```
100
True
```

From a terminal, the human-readable table is:

```
uv run python -m cProfile -s tottime count_open.py
```

Sort by `tottime`. Read the top rows. Ignore the rest until they matter.

Case 3: Two implementations, same answer

Save as `scan_vs_sum.py`. Correctness first. Timing second.

```
# scan_vs_sum.py
def count_loop(tickets: list[dict[str, str]]) -> int:
    n = 0
    for t in tickets:
        if t["status"] == "open":
            n += 1
    return n

def count_sum(tickets: list[dict[str, str]]) -> int:
    return sum(1 for t in tickets if t["status"] == "open")

def main() -> None:
    tickets = [{"status": "open"}] * 100 + [{"status": "paid"}] * 50
    a = count_loop(tickets)
    b = count_sum(tickets)
    print(a)
    print(a == b)

if __name__ == "__main__":
    main()
```

Run:

```
uv run python scan_vs_sum.py
```

Output:

```
100
True
```

If both are correct and neither shows up in a profile, keep the one that reads better (`count_sum` here).

The trap

Micro-optimizing a string that is not the bottleneck.

Save as `clever_label.py`:

```
# clever_label.py
def label_parts(ticket_id: int, table: int) -> str:
    return "ticket " + str(ticket_id) + " → table " + str(table)

def label_f(ticket_id: int, table: int) -> str:
    return f"ticket {ticket_id} → table {table}"

def main() -> None:
    print(label_parts(7, 12))
    print(label_f(7, 12))
    print(label_parts(7, 12) == label_f(7, 12))

if __name__ == "__main__":
    main()
```

Run:

```
uv run python clever_label.py
```

Output:

```
ticket 7 → table 12
ticket 7 → table 12
True
```

The + version is not a win you will measure on a desk. The f-string is the default. Optimize after `cProfile` points at a function that actually runs a million times.

The boring rule

- Make it correct. Then measure. Then change one thing.
- `timeit` for a snippet. `python -m cProfile -s tottime` for a program.
- Do not treat wall-clock floats as golden tests.
- I/O and algorithms beat micro-syntax.
- Keep the readable version when the profile is a tie.

Try this

1. Print `timeit.timeit(count_open, number=1000)` in `count_open.py` and note it changes run to run.
2. Add a third function that uses a list comprehension and check it against `count_sum`.
3. Run `uv run python -m cProfile -s tottime profile_open.py` and find `count_open` in the table.

Designing for the Long Term

Designing for the Long Term

Software lasts when the **boundaries are real**, the **API is small**, and the defaults stay boring: functions, dataclasses, the standard library, tests, and a version you can explain. This chapter is a recap you can keep on the desk.

Mental model

A **package boundary** is a directory other code imports. Inside, you may shuffle files. Across it, names are promises.

A **stable API** is a handful of functions (and maybe a dataclass) with docstrings. Hidden helpers start with `_` or live in a module you do not document.

Long-term Python in this book means: Python 3.14, `uv`, `ruff`, `pytest`, UTC in storage, `pathlib`, `logging`, `subprocess.run` as a list, no secrets in git, wheels from `uv build`.

Worked examples

Case 1: Two modules, one public function

Save as a tiny package in a temp tree and import it. Production looks the same without `TemporaryDirectory`.

`desk/__init__.py` exports `label` only. `desk/format_ticket.py` can change.

```
# stable_desk.py
import sys
from pathlib import Path
from tempfile import TemporaryDirectory

INIT = '''\
from desk.format_ticket import label

__all__ = ["label"]
'''

FORMAT = '''\
def label(ticket_id: int, table: int) -> str:
```

```

    """Return a one-line ticket label for the desk display."""
    return f"ticket {ticket_id} → table {table}"
'''

def main() -> None:
    with TemporaryDirectory() as raw:
        root = Path(raw)
        pkg = root / "desk"
        pkg.mkdir()
        (pkg / "__init__.py").write_text(INIT, encoding="utf-8")
        (pkg / "format_ticket.py").write_text(FORMAT, encoding="utf-8")
        sys.path.insert(0, str(root))
        import desk

        print(desk.label(7, 12))
        print(desk.__all__)

if __name__ == "__main__":
    main()

```

Run:

```
uv run python stable_desk.py
```

Output:

```
ticket 7 → table 12
['label']
```

Callers import `desk.label`. They do not import `desk.format_ticket` unless you say so.

Case 2: `pyproject.toml` is the project boundary

The long-term file at the root of a library:

```

[project]
name = "desk"
version = "1.0.0"
description = "Labels and shifts for a small desk."
requires-python = ">=3.14"

[build-system]
requires = ["hatchling"]
build-backend = "hatchling.build"

```

That file, `desk/`, tests, and `README.md` are the repo. Everything else is extra.

Case 3: One process that still looks like the first chapter

Save as `desk_open.py`. A function, a print, a main guard. Types where they help. No framework.

```
# desk_open.py
def open_table(n: int) -> str:
    if n <= 0:
        raise ValueError(f"table {n}: number must be positive")
    return f"opened table {n}"

def main() -> None:
    print(open_table(12))

if __name__ == "__main__":
    main()
```

Run:

```
uv run python desk_open.py
```

Output:

```
opened table 12
```

If this is enough, stop. Add a package when a second module would otherwise collide.

The trap

A “platform” on day one: a base class, a plugin registry, and a settings object for a label.

Save as `too_much.py`:

```
# too_much.py
class BaseLabel:
    def render(self, ticket_id: int, table: int) -> str:
        raise NotImplementedError

class DeskLabel(BaseLabel):
    def render(self, ticket_id: int, table: int) -> str:
        return f"ticket {ticket_id} → table {table}"
```

```
def main() -> None:
    print(DeskLabel().render(7, 12))

if __name__ == "__main__":
    main()
```

Run:

```
uv run python too_much.py
```

Output:

```
ticket 7 → table 12
```

The boring package from Case 1 prints the same string with one function. Add a class when **state lives** and **behavior belongs to it**. Add plugins when you have a second implementation, not a slogan.

The boring rule

- Export a small `__all__`. Keep helpers behind it.
- Store UTC, log with `logging`, run children with `subprocess.run(...)`, read files with `Path`.
- Test the public functions. Lint with `ruff`. Audit with `uv audit`.
- Version in `pyproject.toml`. Build with `uv build`.
- Reflection and FFI stay at the edges.
- When in doubt, write the function from Case 3.

Try this

1. In `stable_desk.py`, add `desk/shifts.py` with `shift_label(name)` and export it from `__init__.py`.
2. In `desk_open.py`, catch `ValueError` for `open_table(0)` and print the error.
3. Delete `BaseLabel` from `too_much.py` and keep a single label function.

Appendices

Glossary

Glossary

Short definitions for terms this book uses as defaults. Words are defined on first use in the chapters; this list is for lookup.

API. The names, arguments, return values, and exceptions callers may depend on.

argparse. Standard-library parser for `sys.argv`. The CLI should call your functions, not reimplement them.

aware datetime. A `datetime` with `tzinfo` set. Store these (UTC). Contrast **naive datetime**.

cProfile. Standard-library profiler. Run `uv run python -m cProfile -s tottime file.py` after the program is correct.

CSV. Row-and-column text. Use `csv` with `newline=""` and UTF-8.

ctypes. Standard-library FFI. Call C types and libraries. Last resort; prefer Python types.

dataclass. Class generated from fields (`@dataclass`). Boring model for tickets and shifts.

docstring. String literal right under `def` or on a module. The written contract.

encoding. How bytes become text. This book always passes `encoding="utf-8"` for text files.

environment variable. A name/value pair on the process (`os.environ`). Where secrets and `DESK_SHIFT` live.

FFI. Foreign function interface: calling C (or similar) from Python. Avoid unless you must.

hatchling. A build backend. Named in `[build-system]` so `uv build` can produce wheels.

HTTPError. `urllib` exception for HTTP status errors such as 404.

importlib.resources. Read files shipped inside a package, independent of the current directory.

inspect. Standard-library reflection (`signature`, `getdoc`). For tools and tests, not the happy path.

ISO-8601. Timestamp text such as `2026-09-07T14:30:00+00:00`. What you put in JSON.

JSON. Nested text format for APIs and files (`json` module). No native `datetime` type.

logging. Standard-library logs: named loggers, levels, handlers. The boring default over `print` in services.

main guard. `if __name__ == "__main__":`. Runs `main` only when the file is the program.

naive datetime. A `datetime` with `tzinfo` is `None`. Do not store these as the system of record.

package. Importable directory (usually with `__init__.py`). The unit you ship.

path. A location on disk. `pathlib.Path` is the object; `/` joins relative parts.

pip-audit. Standalone vulnerability scanner. This book prefers `uv audit` when you already use `uv`.

pyproject.toml. Project metadata: name, version, dependencies, build backend.

pytest. Test runner. `uv run pytest`.

ruff. Linter and formatter. `uv run ruff check / ruff format`.

sdist. Source distribution tarball from `uv build`.

StringIO. In-memory text stream (`io.StringIO`). Same shape as a text file for tests.

subprocess.run. Run a child process. List of args, `capture_output=True`, `check=True`. No `shell=True` by default.

sys.executable. Path of the running interpreter. Use it to spawn Python children.

TemporaryDirectory. Temp folder that is deleted when the `with` block ends.

TextIO. Type of a text stream (`typing.TextIO`): files, `StringIO`.

timedelta. A duration (`datetime.timedelta`), not a clock.

timeit. Repeat a snippet and measure time. Do not freeze the float in tests.

TOML. Config format. `tomllib` reads it in the standard library; it does not write.

type hint. Annotation such as `n: int`. Used from the typing part onward where they help.

UTC. The zone you store (`datetime.timezone.utc`). Convert to a desk `ZoneInfo` for display.

uv. Toolchain: install Python, run programs (`uv run python file.py`), lock, build, audit.

wheel. Built install artifact (`*.whl`) from `uv build`.

ZoneInfo. IANA time zone (`zoneinfo.ZoneInfo("America/Chicago")`) for display and local wall times.

Command Cheat Sheet

Command Cheat Sheet

Commands this book actually uses. Run them from the directory that holds the file or the `pyproject.toml`.

Python

Command	What it does
<code>uv run python file.py</code>	Run a program with the project's Python 3.14
<code>uv run python -c "print('desk')"</code>	Run a one-liner
<code>uv run python -m cProfile -s tottime file.py</code>	Profile a program

Inside a program, spawn that same interpreter with `sys.executable`, not a bare `python` on `PATH`.

uv

Command	What it does
<code>uv lock</code>	Write/update the lockfile
<code>uv sync</code>	Install the locked environment
<code>uv add pkg</code>	Add a dependency
<code>uv build</code>	Build sdist + wheel into <code>dist/</code>
<code>uv audit</code>	Scan locked deps for known issues
<code>uv pip install dist/*.whl</code>	Install a wheel you just built

ruff

Command	What it does
<code>uv run ruff check .</code>	Lint
<code>uv run ruff check --fix file.py</code>	Lint and apply safe fixes
<code>uv run ruff format .</code>	Format

Silence is success for `ruff check`.

pytest

Command	What it does
<code>uv run pytest</code>	Run tests
<code>uv run pytest -q</code>	Quieter
<code>uv run pytest path/to/test_file.py</code>	One file

Tiny reminders

Run a complete program:

```
uv run python hello.py
```

```
# hello.py
def main() -> None:
    print("desk is open")

if __name__ == "__main__":
    main()
```

desk is open

Child process (list of args, no shell):

```
# run_child.py
import subprocess
import sys

def main() -> None:
    result = subprocess.run(
        [sys.executable, "-c", "print('ticket 7')"],
        capture_output=True,
        text=True,
        check=True,
    )
    print(result.stdout, end="")

if __name__ == "__main__":
    main()
```

```
uv run python run_child.py
```

ticket 7

Secrets stay in the environment (DESK_API_KEY), never in the file you commit.

Times you store are UTC (`datetime.now(timezone.utc)`), not naive local clocks.

Text files: `Path.read_text(encoding="utf-8")` and `write_text(..., encoding="utf-8")`.